30-Day Django Mastery — Nexora CRM

Oct 15, 2026 · @tanvik

30 days, 40 code exercises a day (1,200 total). Every exercise feeds one project: a working Nexora CRM with leads, clients, brokers, call logs and reports by Day 30.

How each day works

Each Day tab = 40 exercises in four blocks, solutions at the bottom. Tick each box once your code runs.

Block	Exercises	Focus
A. Drills	1–10	Syntax and commands, one concept each
B. Core	11–20	The day's main Django feature
C. Nexora CRM	21–30	Same feature applied to the CRM project
D. Challenge	31–40	Edge cases, refactors, tests

Rules:

- One project for all 30 days: folder `nexora-crm`, project package `django_project`. Each day builds on the last.
- Type the code, don't paste. Solve first, then check the solutions.
- End every day with `python manage.py test` and `git commit -m "day N"`.
- Target: Django 5.2 LTS, Python 3.10+.
- Days arrive 3 at a time; say "next" for the following batch.

Roadmap

Five phases: foundations, data, views and users, advanced Django, then APIs and a two-day capstone.

Day	Phase	Topic	CRM piece built
1	Foundations	Project, apps, settings, function views	`nexora_crm` project, `core` + `leads` apps, first pages and JSON
2	Foundations	URL routing and the request object	Converters, namespaces, query params, JSON create/update/delete
3	Foundations	Templates, inheritance, static files	CRM layout with sidebar, lead list/detail pages, dashboard cards
4	Foundations	Class-based views	Days 1–3 pages rebuilt as CBVs, RedirectView, view mixins
5	Foundations	Models and migrations	Lead, Client, Broker models with choices, Meta, constraints
6	Data	ORM CRUD and the shell	Seed 200 leads; create, get, filter, update, delete
7	Data	Lookups, Q, F, ordering, values	Lead search, stale-lead finders, status counts
8	Data	Relationships	Custom user (agents) → Lead FK, call logs, Client↔TradingAccount↔Deposit, M2M tags, self-FK broker tree
9	Data	Aggregation and annotation	Conversion per agent, deposits per broker, Case/When, Subquery
10	Data	Query performance and transactions	select_related, prefetch_related, bulk ops, indexes, atomic
11	Data	Admin customization	Full back office: list_display, filters, search, inlines, actions
12	Views and users	ListView, DetailView, pagination	Paginated lead list, lead detail with call history
13	Views and users	Forms and validation	LeadForm, ModelForms, clean methods, phone/email validation
14	Views and users	Editing views and messages	Lead create/edit/delete, status changes, flash messages

Day	Phase	Topic	CRM piece built
15	Views and users	Authentication	Login/logout, password change/reset, admin-created accounts only (user model built Day 8)
16	Views and users	Permissions and groups	Agents see own leads, managers see their team
17	Views and users	Search, filters, sorting	Lead filter bar: status, source, date range, sort
18	Views and users	Testing	Model, view, form and permission test suite
19	Advanced	Files: uploads, CSV import/export	Bulk lead import, CSV export, KYC document uploads
20	Advanced	Managers, QuerySets, signals	Custom lead QuerySets, audit log via signals
21	Advanced	Middleware, context processors, template tags	Activity logger, global counters, CRM template filters
22	Advanced	Settings, env vars, PostgreSQL, logging	environs, Postgres connection, split settings
23	Advanced	Email and management commands	Lead-assignment emails, `daily_report` command
24	Advanced	Caching and profiling	Debug toolbar, query counts, cached dashboards
25	APIs and capstone	Django REST Framework 1	Serializers, APIView, lead API
26	APIs and capstone	Django REST Framework 2	ViewSets, routers, token auth, filtering, pagination
27	APIs and capstone	JSON + JavaScript front end	fetch() with CSRF, Chart.js dashboard from annotate()
28	APIs and capstone	Security and deployment	check --deploy, WhiteNoise, gunicorn, Docker basics

Day	Phase	Topic	CRM piece built
29	APIs and capstone	Capstone 1	Dialer call-log flow, SOP compliance checks, broker reports
30	APIs and capstone	Capstone 2	Full test pass, polish, deploy, review

Day 1 — Project, apps, settings, function views

Goal: a running `nexora-crm` project with `core` and `leads` apps, 15+ function views, JSON and CSV responses, and first tests.

A. Drills (1–10)

- [ ] **1.** Create folder `nexora-crm`, a virtual env `.venv` inside it, and activate it.
- [ ] **2.** Install Django 5.2 and print the installed version from the command line.
- [ ] **3.** Create project `django_project` in the current folder (no nested duplicate folder).
- [ ] **4.** Apply the default migrations, then run the dev server on port 8080.
- [ ] **5.** Create two apps: `core` and `leads`.
- [ ] **6.** Register both apps in `INSTALLED_APPS` using their AppConfig paths.
- [ ] **7.** Set the project time zone to Dubai and confirm `USE_TZ` is on.
- [ ] **8.** Write a `.gitignore` (venv, database, pycache, `.env`), `git init`, first commit.
- [ ] **9.** Save installed packages to `requirements.txt`.
- [ ] **10.** Run Django's system check and list migration status for every app.

B. Core (11–20) — function views in `core`

- [ ] **11.** View `home` returning the text `Nexora CRM`.
- [ ] **12.** Create `core/urls.py`, map `""` to `home` (name `home`), include it from the project urls.
- [ ] **13.** View `health` at `/health/` returning `OK` with an explicit status 200.
- [ ] **14.** View `dashboard` at `/dashboard/` returning the HTML `<h1>Dashboard</h1>`.
- [ ] **15.** View `about` at `/about/` returning plain text (not HTML) via `content_type`.
- [ ] **16.** View `api_info` at `/api/info/` returning JSON `{"app": "Nexora CRM", "version": "0.1"}`.
- [ ] **17.** View `secret` at `/secret/` that raises a 404 with the message `Not here`.
- [ ] **18.** View at `/home/` that redirects to the `home` URL by name.
- [ ] **19.** View at `/request-info/` returning the request's method and path, e.g. `GET /request-info/`.
- [ ] **20.** View at `/get-only/` that allows GET only; confirm other methods get 405.

C. Nexora CRM (21–30) — `leads` app, in-memory data

- [ ] **21.** In `leads/views.py` define a module-level `LEADS` list of 5 dicts: `id, name, phone, status, source` (UAE numbers like `971501234567`; statuses `new/contacted/qualified/lost`).
- [ ] **22.** View `lead_list` returning all lead names separated by `<br>`.
- [ ] **23.** Create `leads/urls.py` with `app_name = "leads"`; mount it at `/leads/`.
- [ ] **24.** Helper `get_lead(lead_id)` that raises 404 if missing; view `lead_detail` at `/leads/<id>/` returning `name — status`.
- [ ] **25.** View `lead_count` at `/leads/count/` returning `{"total": 5}`.
- [ ] **26.** View `leads_by_status` at `/leads/status/<status>/` returning matching names.
- [ ] **27.** View `lead_list_json` at `/leads/json/` returning the whole list as JSON.
- [ ] **28.** Make `lead_list` filter by an optional `?source=` query parameter.
- [ ] **29.** View `lead_phone` at `/leads/<id>/phone/` returning a masked number: `9715****4567`.
- [ ] **30.** View `lead_menu` at `/leads/menu/` returning an HTML list of links built with `reverse()` (list, JSON, count).

D. Challenge (31–40)

- [ ] **31.** Custom 404 page: a `custom_404` view in `core` wired with `handler404`. See it with `DEBUG = False`.
- [ ] **32.** View `leads_csv` at `/leads/export/` that downloads `leads.csv` using the `csv` module.
- [ ] **33.** Make `leads_by_status` return 400 for any status outside the four valid ones.
- [ ] **34.** Write a `log_request` decorator printing method, path, status code and duration in ms; apply it to `lead_list`.
- [ ] **35.** Add `CRM_COMPANY_NAME = "Nexora"` to settings and read it in a `welcome` view.
- [ ] **36.** View `quick_add` at `/leads/quick-add/`: GET shows instructions, POST adds a lead from form field `name` and returns it as JSON with status 201. Test with curl.
- [ ] **37.** `core/tests.py`: test that `home` returns 200 and contains `Nexora CRM`.
- [ ] **38.** `leads/tests.py`: test that lead 999 returns 404.
- [ ] **39.** Test that `/api/info/` JSON has `app == "Nexora CRM"`.
- [ ] **40.** In `python manage.py shell`, use Django's test `Client` to GET `/leads/` and print the status code and first 50 bytes.

Solutions

A. Drills

1–5.

```
mkdir nexora-crm && cd nexora-crm
python -m venv .venv
.venv\Scripts\Activate.ps1           # macOS/Linux: source .venv/bin/activate

python -m pip install "django~=5.2"   # 2
python -m django --version

django-admin startproject django_project .   # 3

python manage.py migrate              # 4
python manage.py runserver 8080

python manage.py startapp core        # 5
python manage.py startapp leads
```

6–7.

```
# django_project/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "core.apps.CoreConfig",
    "leads.apps.LeadsConfig",
]

TIME_ZONE = "Asia/Dubai"
USE_TZ = True
```

8–10.

```
# .gitignore
.venv/
db.sqlite3
__pycache__/
.env
```

```
git init
git add -A
git commit -m "day 1: project setup"

python -m pip freeze > requirements.txt   # 9

python manage.py check                    # 10
python manage.py showmigrations
```

B. Core

11–20. All views in one file, then the urls.

```
# core/views.py
from django.http import HttpResponse, JsonResponse, Http404
from django.shortcuts import redirect
from django.views.decorators.http import require_GET


def home(request):                                  # 11
    return HttpResponse("Nexora CRM")


def health(request):                                # 13
    return HttpResponse("OK", status=200)


def dashboard(request):                             # 14
    return HttpResponse("<h1>Dashboard</h1>")


def about(request):                                 # 15
    return HttpResponse("Nexora CRM — internal tool",
content_type="text/plain")


def api_info(request):                              # 16
    return JsonResponse({"app": "Nexora CRM", "version": "0.1"})


def secret(request):                                # 17
    raise Http404("Not here")


def old_home(request):                              # 18
```

```
    return redirect("home")


def request_info(request):                              # 19
    return HttpResponse(f"{request.method} {request.path}")


@require_GET                                            # 20
def get_only(request):
    return HttpResponse("GET only")
```

```
# core/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
    path("health/", views.health, name="health"),
    path("dashboard/", views.dashboard, name="dashboard"),
    path("about/", views.about, name="about"),
    path("api/info/", views.api_info, name="api_info"),
    path("secret/", views.secret, name="secret"),
    path("home/", views.old_home, name="old_home"),
    path("request-info/", views.request_info, name="request_info"),
    path("get-only/", views.get_only, name="get_only"),
]
```

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("leads/", include("leads.urls")),   # added in 23
    path("", include("core.urls")),
]
```

```
# 20: OPTIONS skips CSRF, so you see the 405 (a POST would hit CSRF first → 403)
curl -i -X OPTIONS http://127.0.0.1:8000/get-only/
```

C. Nexora CRM

21–30.

```
# leads/views.py
from django.http import HttpResponse, JsonResponse, Http404
from django.urls import reverse

LEADS = [                                                          # 21
    {"id": 1, "name": "Aisha Khan", "phone": "971501234567", "status": "new",
"source": "facebook"},
    {"id": 2, "name": "Omar Haddad", "phone": "971502345678", "status":
"contacted", "source": "google"},
    {"id": 3, "name": "Priya Nair", "phone": "971503456789", "status":
"qualified", "source": "referral"},
    {"id": 4, "name": "Daniel Ortiz", "phone": "971504567890", "status":
"new", "source": "google"},
    {"id": 5, "name": "Mei Lin", "phone": "971505678901", "status": "lost",
"source": "facebook"},
]


def get_lead(lead_id):                                             # 24
    for lead in LEADS:
        if lead["id"] == lead_id:
            return lead
    raise Http404("Lead not found")


def lead_list(request):                                            # 22 + 28
    source = request.GET.get("source")
    leads = [l for l in LEADS if not source or l["source"] == source]
    return HttpResponse("<br>".join(l["name"] for l in leads))


def lead_detail(request, lead_id):                                 # 24
    lead = get_lead(lead_id)
    return HttpResponse(f"{lead['name']} — {lead['status']}")


def lead_count(request):                                           # 25
    return JsonResponse({"total": len(LEADS)})


def leads_by_status(request, status):                              # 26
    names = [l["name"] for l in LEADS if l["status"] == status]
    return HttpResponse("<br>".join(names) or "No leads")


def lead_list_json(request):                                       # 27
```

```
    return JsonResponse(LEADS, safe=False)   # safe=False: top-level list


def lead_phone(request, lead_id):                                    # 29
    phone = get_lead(lead_id)["phone"]
    return HttpResponse(phone[:4] + "*" * (len(phone) - 8) + phone[-4:])


def lead_menu(request):                                              # 30
    links = [
        ("All leads", reverse("leads:list")),
        ("JSON", reverse("leads:json")),
        ("Count", reverse("leads:count")),
    ]
    items = "".join(f'<li><a href="{url}">{label}</a></li>' for label, url in
links)
    return HttpResponse(f"<ul>{items}</ul>")
```

```
# leads/urls.py                                                      # 23
from django.urls import path
from . import views

app_name = "leads"
urlpatterns = [
    path("", views.lead_list, name="list"),
    path("count/", views.lead_count, name="count"),
    path("json/", views.lead_list_json, name="json"),
    path("menu/", views.lead_menu, name="menu"),
    path("status/<str:status>/", views.leads_by_status, name="by_status"),
    path("<int:lead_id>/", views.lead_detail, name="detail"),
    path("<int:lead_id>/phone/", views.lead_phone, name="phone"),
]
```

D. Challenge

31.

```
# core/views.py
def custom_404(request, exception):
    return HttpResponse("<h1>Nexora CRM — page not found</h1>", status=404)

# django_project/urls.py (module level, below urlpatterns)
handler404 = "core.views.custom_404"

# settings.py — to see it
DEBUG = False
ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
```

32.

```
# leads/views.py
import csv

def leads_csv(request):
    response = HttpResponse(content_type="text/csv")
    response["Content-Disposition"] = 'attachment; filename="leads.csv"'
    writer = csv.DictWriter(response, fieldnames=["id", "name", "phone",
"status", "source"])
    writer.writeheader()
    writer.writerows(LEADS)
    return response

# leads/urls.py
path("export/", views.leads_csv, name="export"),
```

33.

```
from django.http import HttpResponseBadRequest

VALID_STATUSES = {"new", "contacted", "qualified", "lost"}

def leads_by_status(request, status):
    if status not in VALID_STATUSES:
        return HttpResponseBadRequest(f"Unknown status: {status}")
    names = [l["name"] for l in LEADS if l["status"] == status]
    return HttpResponse("<br>".join(names) or "No leads")
```

34.

```
# core/decorators.py
import functools
import time
```

```
def log_request(view):
    @functools.wraps(view)
    def wrapper(request, *args, **kwargs):
        start = time.perf_counter()
        response = view(request, *args, **kwargs)
        ms = (time.perf_counter() - start) * 1000
        print(f"{request.method} {request.path} -> {response.status_code}
({ms:.1f} ms)")
        return response
    return wrapper

# leads/views.py
from core.decorators import log_request

@log_request
def lead_list(request):
    ...
```

35.

```
# settings.py
CRM_COMPANY_NAME = "Nexora"

# core/views.py
from django.conf import settings

def welcome(request):
    return HttpResponse(f"Welcome to {settings.CRM_COMPANY_NAME} CRM")

# core/urls.py
path("welcome/", views.welcome, name="welcome"),
```

36.

```
# leads/views.py
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt            # practice only — real forms use {% csrf_token %}
(Day 13)
def quick_add(request):
    if request.method == "POST":
        name = request.POST.get("name", "").strip()
        new_id = max(l["id"] for l in LEADS) + 1
        lead = {"id": new_id, "name": name, "phone": "", "status": "new",
"source": "manual"}
        LEADS.append(lead)
        return JsonResponse(lead, status=201)
    return HttpResponse("POST a 'name' field to add a lead")

# leads/urls.py
path("quick-add/", views.quick_add, name="quick_add"),
```

```
curl -X POST -d "name=Sara Ali" http://127.0.0.1:8000/leads/quick-add/
```

37–39.

```
# core/tests.py
from django.test import SimpleTestCase
from django.urls import reverse


class CoreTests(SimpleTestCase):
    def test_home(self):                                    # 37
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Nexora CRM")

    def test_api_info(self):                                # 39
        response = self.client.get(reverse("api_info"))
        self.assertEqual(response.json()["app"], "Nexora CRM")
```

```
# leads/tests.py
from django.test import SimpleTestCase
from django.urls import reverse


class LeadViewTests(SimpleTestCase):
    def test_missing_lead_404(self):                        # 38
        response = self.client.get(reverse("leads:detail", args=[999]))
        self.assertEqual(response.status_code, 404)
```

40.

```
# python manage.py shell
from django.test.utils import setup_test_environment
setup_test_environment()        # allows the 'testserver' host outside the
test runner

from django.test import Client
r = Client().get("/leads/")
print(r.status_code, r.content[:50])
```

Day 2 — URL routing and the request object

Goal: every URL feature you'll need (converters, namespaces, reverse/resolve) plus full control of the request: query params, headers, body, cookies, session. Builds on Day 1's `LEADS` list.

A. Drills (1–10) — URL patterns

- [ ] **1.** Route `leads/type-check/<int:pk>/` to a view that returns the Python type name of `pk`.
- [ ] **2.** Route `leads/source/<slug:source>/` returning names of leads from that source (name `by_source`).
- [ ] **3.** Route `leads/ref/<uuid:ref>/` echoing the UUID and its type.
- [ ] **4.** Route `files/<path:doc_path>` in `core` that echoes the full path, slashes included.
- [ ] **5.** Use `re_path` to match 7-digit account numbers: `accounts/1234567/`.
- [ ] **6.** In the shell: `reverse()` `leads:detail` once with `args`, once with `kwargs`, and `leads:by_source`.
- [ ] **7.** Mount `leads.urls` a second time at `/archive/` with instance namespace `archive`; reverse `archive:list` and `leads:list`.
- [ ] **8.** View `latest_lead` that redirects to the detail page of the last lead using `redirect()` with kwargs.
- [ ] **9.** Module-level constant `LEADS_HOME = reverse_lazy("leads:list")`, used by a `go_leads` redirect view.
- [ ] **10.** In the shell, `resolve("/leads/3/")` and print `view_name`, `kwargs`, `func.__name__`.

B. Core (11–20) — the request object (put these under `/debug/...` in `core`)

- [ ] **11.** `echo_get`: return all query params as JSON.
- [ ] **12.** `page_param`: read `?page=`, default 1, fall back to 1 on non-numbers.
- [ ] **13.** In `leads`: `multi_status` reading repeated `?status=new&status=lost` and returning matching leads as JSON.
- [ ] **14.** `user_agent`: return the User-Agent header, default `unknown`.
- [ ] **15.** `client_ip`: return the caller's IP from `META`.
- [ ] **16.** `echo_post` (CSRF-exempt): return all POST form fields as JSON; test with curl.

- [ ] **17.** `json_echo` (CSRF-exempt): parse a JSON body, return `{"received": ...}`, 400 on invalid JSON.
- [ ] **18.** `absolute_url`: return the full absolute URL of lead 1's detail page.
- [ ] **19.** In `leads`: `remember_lead` at `<id>/remember/` that shows the previously viewed lead id from a cookie, then sets cookie `last_lead` (1 hour).
- [ ] **20.** `visit_counter`: count visits in the session.

C. Nexora CRM (21–30)

- [ ] **21.** Custom path converter `uae_phone` (pattern `9715` + 8 digits); route `leads/phone/<uae_phone:phone>/` returning the lead as JSON.
- [ ] **22.** `search` at `leads/search/?q=` — case-insensitive name match, return `query, count, results`.
- [ ] **23.** `paged` at `leads/paged/?page=&size=` — return `total, page, size, results` (size capped at 50).
- [ ] **24.** `sorted_leads` at `leads/sorted/?sort=name` or `-name`; 400 for unknown fields.
- [ ] **25.** `create_lead` at `leads/create/`: POST-only, JSON body, append lead with status `new`, return 201.
- [ ] **26.** `update_status` at `leads/<id>/status/`: POST-only form field `status`, validate, return updated lead.
- [ ] **27.** `delete_lead` at `leads/<id>/delete/`: DELETE-only, return 204 with empty body.
- [ ] **28.** New `clients` app: `CLIENTS` list with `ref` (UUID), `name`, `account`; routes `clients:list` and `clients:detail` by UUID.
- [ ] **29.** Versioned API: `leads/api_urls.py` (namespace `leads_api`) mounted at `/api/v1/` with `leads/` and `leads/<id>/`.
- [ ] **30.** "Recently viewed": `lead_detail` stores the last 5 viewed ids in the session; `leads/recent/` returns them as JSON.

D. Challenge (31–40)

- [ ] **31.** Decorator `require_api_key`: header `X-API-Key` must equal `settings.CRM_API_KEY`, else 401 JSON. Apply to the v1 API.
- [ ] **32.** Helper `json_error(message, status=400)` in `core/utils.py`; refactor 24 and 26 to use it.
- [ ] **33.** Decorator `rate_limited(limit=10, window=60)` using the session; return 429 when exceeded.

- [ ] **34.** One view, two routes: `leads/browse/` (page 1 default) and `leads/browse/<int:page>/`.
- [ ] **35.** Move the `/debug/` views into an inline `include((patterns, "debug"))` in the project urls; reverse `debug:get`.
- [ ] **36.** Test `search` with query params passed as a dict to `client.get`.
- [ ] **37.** Test `create_lead` posting JSON (`content_type="application/json"`) → 201, status `new`.
- [ ] **38.** Test that `/leads/phone/12345/` returns 404 (converter rejects it).
- [ ] **39.** One test that reverses 5 named URLs in a loop with `subTest`.
- [ ] **40.** A POST-only `protected_action` view (no csrf_exempt); test with `Client(enforce_csrf_checks=True)` that a token-less POST gets 403.

Solutions

A. Drills

1–3, 8–9.

```
# leads/views.py (add)
from django.shortcuts import redirect
from django.urls import reverse_lazy

LEADS_HOME = reverse_lazy("leads:list")   # 9: plain reverse() here fails —
URLconf not loaded at import time


def type_check(request, pk):                                    # 1
    return HttpResponse(f"{pk!r} is {type(pk).__name__}")       # → 3 is int


def leads_by_source(request, source):                           # 2
    names = [l["name"] for l in LEADS if l["source"] == source]
    return HttpResponse("<br>".join(names) or "No leads")


def lead_by_ref(request, ref):                                  # 3
    return HttpResponse(f"{ref} is {type(ref).__name__}")       # → UUID


def latest_lead(request):                                       # 8
    return redirect("leads:detail", lead_id=LEADS[-1]["id"])
```

```
def go_leads(request):                                          # 9
    return redirect(LEADS_HOME)
```

4–5.

```
# core/views.py (add)
def echo_path(request, doc_path):
    return HttpResponse(doc_path)            # /files/kyc/2026/passport.pdf → kyc/2026/passport.pdf


def account_detail(request, acc):
    return HttpResponse(f"Account {acc} ({type(acc).__name__})")   # re_path captures are str

# core/urls.py
from django.urls import path, re_path

path("files/<path:doc_path>", views.echo_path, name="echo_path"),
re_path(r"^accounts/(?P<acc>\d{7})/$", views.account_detail, name="account_detail"),
```

6, 10.

```
# python manage.py shell
from django.urls import reverse, resolve

reverse("leads:detail", args=[3])                          # '/leads/3/'
reverse("leads:detail", kwargs={"lead_id": 3})             # '/leads/3/'
reverse("leads:by_source", kwargs={"source": "google"})  # '/leads/source/google/'

match = resolve("/leads/3/")                               # 10
print(match.view_name, match.kwargs, match.func.__name__)
# leads:detail {'lead_id': 3} lead_detail
```

7.

```
# django_project/urls.py
path("leads/", include("leads.urls")),
path("archive/", include("leads.urls", namespace="archive")),

# shell
reverse("archive:list")   # '/archive/'
reverse("leads:list")     # '/leads/'
```

B. Core

11–12, 14–18, 20.

```
# core/views.py (add)
import json
from django.urls import reverse
from django.views.decorators.csrf import csrf_exempt


def echo_get(request):                                          # 11
    return JsonResponse(request.GET.dict())


def page_param(request):                                        # 12
    try:
        page = int(request.GET.get("page", 1))
    except ValueError:
        page = 1
    return HttpResponse(f"Page {page}")


def user_agent(request):                                        # 14
    return HttpResponse(request.headers.get("User-Agent", "unknown"))


def client_ip(request):                                         # 15
    return HttpResponse(request.META.get("REMOTE_ADDR"))        # behind a
proxy: X-Forwarded-For


@csrf_exempt
def echo_post(request):                                         # 16
    return JsonResponse(request.POST.dict())


@csrf_exempt
```

```
def json_echo(request):                                          # 17
    try:
        data = json.loads(request.body)
    except json.JSONDecodeError:
        return JsonResponse({"error": "invalid JSON"}, status=400)
    return JsonResponse({"received": data})


def absolute_url(request):                                       # 18
    return HttpResponse(request.build_absolute_uri(reverse("leads:detail",
args=[1])))


def visit_counter(request):                                      # 20
    count = request.session.get("visits", 0) + 1
    request.session["visits"] = count
    return HttpResponse(f"Visits this session: {count}")
```

```
# core/urls.py (add)
path("debug/get/", views.echo_get, name="echo_get"),
path("debug/page/", views.page_param, name="page_param"),
path("debug/ua/", views.user_agent, name="user_agent"),
path("debug/ip/", views.client_ip, name="client_ip"),
path("debug/post/", views.echo_post, name="echo_post"),
path("debug/json/", views.json_echo, name="json_echo"),
path("debug/absolute/", views.absolute_url, name="absolute_url"),
path("debug/visits/", views.visit_counter, name="visit_counter"),
```

```
# PowerShell: type curl.exe instead of curl
curl -X POST -d "name=Ali&source=google" http://127.0.0.1:8000/debug/post/
curl -X POST -H "Content-Type: application/json" -d "{\"name\": \"Ali\"}"
http://127.0.0.1:8000/debug/json/
```

13, 19.

```
# leads/views.py (add)
def multi_status(request):                                          # 13
    statuses = request.GET.getlist("status")
    leads = [l for l in LEADS if l["status"] in statuses] if statuses else
LEADS
    return JsonResponse(leads, safe=False)


def remember_lead(request, lead_id):                                # 19
    lead = get_lead(lead_id)
    previous = request.COOKIES.get("last_lead", "none")
    response = HttpResponse(f"Viewing {lead['name']}. Previously viewed:
{previous}")
    response.set_cookie("last_lead", str(lead_id), max_age=3600)
    return response
```

C. Nexora CRM

21.

```
# leads/converters.py
class UAEPhoneConverter:
    regex = r"9715\d{8}"

    def to_python(self, value):
        return value

    def to_url(self, value):
        return value

# leads/views.py
def lead_by_phone(request, phone):
    for lead in LEADS:
        if lead["phone"] == phone:
            return JsonResponse(lead)
    raise Http404("No lead with that phone")
```

22–27, 30.

```
# leads/views.py (add)
import json
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_POST, require_http_methods
```

```
def search(request):                                                    # 22
    q = request.GET.get("q", "").strip().lower()
    results = [l for l in LEADS if q in l["name"].lower()] if q else []
    return JsonResponse({"query": q, "count": len(results), "results":
results})


def paged(request):                                                     # 23
    page = max(int(request.GET.get("page", 1)), 1)
    size = min(int(request.GET.get("size", 2)), 50)
    start = (page - 1) * size
    return JsonResponse({
        "total": len(LEADS), "page": page, "size": size,
        "results": LEADS[start:start + size],
    })


def sorted_leads(request):                                              # 24
    sort = request.GET.get("sort", "id")
    key = sort.lstrip("-")
    if key not in {"id", "name", "status", "source"}:
        return JsonResponse({"error": f"cannot sort by {key}"}, status=400)
    leads = sorted(LEADS, key=lambda l: l[key], reverse=sort.startswith("-"))
    return JsonResponse(leads, safe=False)


@csrf_exempt
@require_POST
def create_lead(request):                                               # 25
    data = json.loads(request.body)
    lead = {
        "id": max(l["id"] for l in LEADS) + 1,
        "name": data["name"],
        "phone": data.get("phone", ""),
        "status": "new",
        "source": data.get("source", "manual"),
    }
    LEADS.append(lead)
    return JsonResponse(lead, status=201)


@csrf_exempt
@require_POST
def update_status(request, lead_id):                                    # 26
    lead = get_lead(lead_id)
    status = request.POST.get("status")
```

```
    if status not in VALID_STATUSES:
        return JsonResponse({"error": "invalid status"}, status=400)
    lead["status"] = status
    return JsonResponse(lead)


@csrf_exempt
@require_http_methods(["DELETE"])
def delete_lead(request, lead_id):                                  # 27
    LEADS.remove(get_lead(lead_id))
    return HttpResponse(status=204)


def lead_detail(request, lead_id):                                  # 30
(replaces Day 1 version)
    lead = get_lead(lead_id)
    recent = request.session.get("recent_leads", [])
    if lead_id in recent:
        recent.remove(lead_id)
    recent.insert(0, lead_id)
    request.session["recent_leads"] = recent[:5]
    return HttpResponse(f"{lead['name']} — {lead['status']}")


def recent_leads(request):                                          # 30
    ids = request.session.get("recent_leads", [])
    return JsonResponse([get_lead(i) for i in ids], safe=False)
```

```
curl -X DELETE http://127.0.0.1:8000/leads/5/delete/
curl -X POST -d "status=qualified" http://127.0.0.1:8000/leads/1/status/
```

Full `leads/urls.py` after Day 2:

```
from django.urls import path, register_converter
from . import converters, views

register_converter(converters.UAEPhoneConverter, "uae_phone")

app_name = "leads"
urlpatterns = [
    path("", views.lead_list, name="list"),
    path("count/", views.lead_count, name="count"),
    path("json/", views.lead_list_json, name="json"),
    path("menu/", views.lead_menu, name="menu"),
    path("export/", views.leads_csv, name="export"),
    path("quick-add/", views.quick_add, name="quick_add"),
```

```
    path("latest/", views.latest_lead, name="latest"),
    path("go/", views.go_leads, name="go"),
    path("multi/", views.multi_status, name="multi_status"),
    path("search/", views.search, name="search"),
    path("paged/", views.paged, name="paged"),
    path("sorted/", views.sorted_leads, name="sorted"),
    path("create/", views.create_lead, name="create"),
    path("recent/", views.recent_leads, name="recent"),
    path("browse/", views.lead_page, name="browse"),                      #
34
    path("browse/<int:page>/", views.lead_page, name="browse_page"),      #
34
    path("protected/", views.protected_action, name="protected"),          #
40
    path("type-check/<int:pk>/", views.type_check, name="type_check"),
    path("status/<str:status>/", views.leads_by_status, name="by_status"),
    path("source/<slug:source>/", views.leads_by_source, name="by_source"),
    path("ref/<uuid:ref>/", views.lead_by_ref, name="by_ref"),
    path("phone/<uae_phone:phone>/", views.lead_by_phone, name="by_phone"),
    path("<int:lead_id>/", views.lead_detail, name="detail"),
    path("<int:lead_id>/phone/", views.lead_phone, name="phone"),
    path("<int:lead_id>/remember/", views.remember_lead, name="remember"),
    path("<int:lead_id>/status/", views.update_status, name="update_status"),
    path("<int:lead_id>/delete/", views.delete_lead, name="delete"),
]
```

28.

```
python manage.py startapp clients     # add "clients.apps.ClientsConfig" to
INSTALLED_APPS
```

```
# clients/views.py
import uuid
from django.http import JsonResponse, Http404

CLIENTS = [
    {"ref": uuid.UUID("3f1c2b8e-5d4a-4c1e-9a7b-1e2f3a4b5c6d"), "name": "Aisha
Khan", "account": "1000001"},
    {"ref": uuid.UUID("7a9d0e1f-2b3c-4d5e-8f90-a1b2c3d4e5f6"), "name": "Priya
Nair", "account": "1000002"},
]


def client_list(request):
    return JsonResponse(CLIENTS, safe=False)   # JsonResponse's encoder
handles UUID
```

```
def client_detail(request, ref):
    for client in CLIENTS:
        if client["ref"] == ref:
            return JsonResponse(client)
    raise Http404("Client not found")

# clients/urls.py
from django.urls import path
from . import views

app_name = "clients"
urlpatterns = [
    path("", views.client_list, name="list"),
    path("<uuid:ref>/", views.client_detail, name="detail"),
]

# django_project/urls.py
path("clients/", include("clients.urls")),
```

29 + 31.

```
# core/decorators.py (add)                                          # 31
from django.conf import settings
from django.http import JsonResponse


def require_api_key(view):
    @functools.wraps(view)
    def wrapper(request, *args, **kwargs):
        if request.headers.get("X-API-Key") != settings.CRM_API_KEY:
            return JsonResponse({"error": "invalid API key"}, status=401)
        return view(request, *args, **kwargs)
    return wrapper

# settings.py
CRM_API_KEY = "dev-key-123"     # moves to .env on Day 22
```

```
# leads/api_urls.py                                                    # 29
from django.urls import path
from core.decorators import require_api_key
from . import views

app_name = "leads_api"
urlpatterns = [
    path("leads/", require_api_key(views.lead_list_json), name="list"),
    path("leads/<int:lead_id>/", require_api_key(views.lead_detail_json),
name="detail"),
]

# leads/views.py
def lead_detail_json(request, lead_id):
    return JsonResponse(get_lead(lead_id))

# django_project/urls.py
path("api/v1/", include("leads.api_urls")),
```

```
curl -H "X-API-Key: dev-key-123" http://127.0.0.1:8000/api/v1/leads/
```

D. Challenge

32–33.

```
# core/utils.py
from django.http import JsonResponse


def json_error(message, status=400, **extra):
    return JsonResponse({"error": message, "status": status, **extra},
status=status)

# leads/views.py — e.g. in sorted_leads / update_status
from core.utils import json_error
return json_error(f"cannot sort by {key}")
return json_error("invalid status")
```

```
# core/decorators.py (add)                                             # 33
from core.utils import json_error


def rate_limited(limit=10, window=60):
    def decorator(view):
        @functools.wraps(view)
```

```
        def wrapper(request, *args, **kwargs):
            now = time.time()
            hits = [t for t in request.session.get("hits", []) if now - t <
window]
            if len(hits) >= limit:
                return json_error("rate limit exceeded", status=429)
            hits.append(now)
            request.session["hits"] = hits
            return view(request, *args, **kwargs)
        return wrapper
    return decorator

# usage
@rate_limited(limit=5, window=60)
def search(request): ...
```

34.

```
def lead_page(request, page=1):          # default covers the route with no
<page>
    size = 2
    start = (page - 1) * size
    names = [l["name"] for l in LEADS[start:start + size]]
    return HttpResponse(f"Page {page}: " + ", ".join(names))
```

35.

```
# django_project/urls.py
from core import views as core_views

debug_patterns = [
    path("get/", core_views.echo_get, name="get"),
    path("page/", core_views.page_param, name="page"),
    path("ua/", core_views.user_agent, name="ua"),
    path("ip/", core_views.client_ip, name="ip"),
    path("post/", core_views.echo_post, name="post"),
    path("json/", core_views.json_echo, name="json"),
    path("absolute/", core_views.absolute_url, name="absolute"),
    path("visits/", core_views.visit_counter, name="visits"),
]

urlpatterns = [
    path("admin/", admin.site.urls),
    path("debug/", include((debug_patterns, "debug"))),   # (patterns,
app_name)
```

```
    ...
]
# delete the debug/ lines from core/urls.py; reverse("debug:get") →
'/debug/get/'
```

36–40.

```
# leads/views.py (add)                                          # 40
@require_POST
def protected_action(request):
    return HttpResponse("done")
```

```
# leads/tests.py (add to LeadViewTests)
from django.test import Client

    def test_search(self):                                      # 36
        response = self.client.get(reverse("leads:search"), {"q": "aisha"})
        self.assertEqual(response.json()["count"], 1)

    def test_create_lead(self):                                 # 37
        response = self.client.post(
            reverse("leads:create"), {"name": "Test Lead"},
content_type="application/json"
        )
        self.assertEqual(response.status_code, 201)
        self.assertEqual(response.json()["status"], "new")

    def test_invalid_phone_404(self):                           # 38
        self.assertEqual(self.client.get("/leads/phone/12345/").status_code,
404)

    def test_named_urls(self):                                  # 39
        cases = [
            ("leads:list", {}, "/leads/"),
            ("leads:detail", {"lead_id": 1}, "/leads/1/"),
            ("leads:by_source", {"source": "google"},
"/leads/source/google/"),
            ("clients:list", {}, "/clients/"),
            ("leads_api:list", {}, "/api/v1/leads/"),
        ]
        for name, kwargs, expected in cases:
            with self.subTest(name=name):
                self.assertEqual(reverse(name, kwargs=kwargs), expected)
```

```
    def test_csrf_enforced(self):                                    # 40
        client = Client(enforce_csrf_checks=True)
        response = client.post(reverse("leads:protected"))
        self.assertEqual(response.status_code, 403)
```

Day 3 — Templates, inheritance, static files

Goal: replace raw `HttpResponse` HTML with real templates — a CRM layout with sidebar, lead pages, dashboard cards, and your first custom template filters and tags.

A. Drills (1–10) — template language

- [ ] **1.** Create a project-level `templates/` folder and register it in `TEMPLATES["DIRS"]`.
- [ ] **2.** Create `core/templates/core/home.html`; make `home` use `render()` with it.
- [ ] **3.** Pass context `company="Nexora"` and `user_count=12`; show `Nexora CRM — 12 users`.
- [ ] **4.** Apply `upper`, `lower`, `title`, `length`, and `default:"N/A"` (on a missing variable).
- [ ] **5.** Pass `timezone.now()` and display it as `15 Oct 2026, 14:30`.
- [ ] **6.** Loop over `leads` with `{% for %}` and an `{% empty %}` fallback.
- [ ] **7.** Number the loop with `forloop.counter`, tag the first item, comma-separate all but the last.
- [ ] **8.** `{% if %}` / `{% elif %}` with `and`, `or`, `not`, `in`: label leads Hot / Warm / Cold.
- [ ] **9.** Link each lead with `{% url %}`: once positional (`leads:detail`), once keyword (`leads:by_source`).
- [ ] **10.** Use `{# #}`, `{% comment %}`, and `{% with %}` + `pluralize` to print `5 leads`.

B. Core (11–20) — inheritance, includes, static

- [ ] **11.** `templates/base.html` with blocks `title`, `extra_css`, `content`, `extra_js`.
- [ ] **12.** Make `core/home.html` extend base and override `title`.
- [ ] **13.** Use `{{ block.super }}` so the leads page title reads `Leads · Nexora CRM`.
- [ ] **14.** Partial `templates/partials/_navbar.html`, included from base.
- [ ] **15.** Partial `_badge.html` included with `with status=lead.status only`.
- [ ] **16.** Static setup: `STATICFILES_DIRS`, `static/css/crm.css`, linked from base via `{% static %}`.
- [ ] **17.** Show `static/img/logo.png` in the navbar.
- [ ] **18.** `static/js/crm.js` logging `Nexora CRM loaded`, loaded with `defer`.
- [ ] **19.** Enable `django.contrib.humanize`; show `intcomma`, `floatformat:2`, `truncatechars:30`.
- [ ] **20.** Pass `note = "<b>VIP</b> client"`: show it escaped, with `|safe`, and inside `{% autoescape off %}`.

C. Nexora CRM (21–30)

- [ ] **21.** CRM layout in base: dark sidebar (Dashboard, Leads, Clients, Reports), topbar, content area (CSS grid).
- [ ] **22.** Highlight the active sidebar link using `request.resolver_match`.
- [ ] **23.** Convert `lead_list` to `render()` with `leads/lead_list.html`: table of name, phone, status, source.
- [ ] **24.** Status badges with a colour per status (new, contacted, qualified, lost).
- [ ] **25.** Convert `lead_detail` to `leads/lead_detail.html` using a `<dl>`; keep the session "recent" logic.
- [ ] **26.** Convert `dashboard` to `core/dashboard.html`: total leads + one card per status.
- [ ] **27.** On the lead list, add a "By status" section using `{% regroup %}`.
- [ ] **28.** Zebra-stripe table rows with `{% cycle %}`.
- [ ] **29.** Search form (GET `q`) above the table; keep the typed value; empty-state row when nothing matches.
- [ ] **30.** Footer with the current year via `{% now %}`.

D. Challenge (31–40)

- [ ] **31.** Custom filter `mask_phone` in `leads/templatetags/crm_filters.py`; use it in the lead table.
- [ ] **32.** `simple_tag` `status_count leads "new"`, also used with `as new_count`.
- [ ] **33.** `templates/404.html` extending base; make `custom_404` render it.
- [ ] **34.** `inclusion_tag` `lead_card` rendering `leads/partials/_lead_card.html`.
- [ ] **35.** Dashboard "Recent activity" list with `naturaltime` (`5 minutes ago`).
- [ ] **36.** In the shell, `render_to_string` the lead card for `LEADS[0]` and print it.
- [ ] **37.** Test the lead list uses both `leads/lead_list.html` and `base.html`.
- [ ] **38.** Test the list contains the `badge-new` class.
- [ ] **39.** Test that a lead named `<script>alert(1)</script>` is escaped in the HTML.
- [ ] **40.** Prove lookup order: a project-level `templates/leads/lead_list.html` overrides the app one. Test it, then delete it.

Solutions

A. Drills

1–2.

```
# django_project/settings.py
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],   # project-level, searched first
        "APP_DIRS": True,                   # then <app>/templates/
        ...
    },
]
```

```
# core/views.py
from django.shortcuts import render
from django.utils import timezone
from leads.views import LEADS


def home(request):
    context = {
        "company": "Nexora",
        "user_count": 12,
        "now": timezone.now(),
        "leads": LEADS,
        "paid_sources": ["google", "facebook"],
        "note": "<b>VIP</b> client",   # 20
    }
    return render(request, "core/home.html", context)
```

3–10, 12, 19–20. Final `home.html` (extends base from 12 onward).

```
<!-- core/templates/core/home.html -->
{% extends "base.html" %}
{% load humanize %}
{% block title %}Home · {{ block.super }}{% endblock title %}

{% block content %}
<h1>{{ company }} CRM — {{ user_count }} users</h1>
{# 3 #}

<p>{{ company|upper }} · {{ company|lower }} · {{ "omar haddad"|title }}</p>
{# 4 #}
<p>{{ leads|length }} leads · Region: {{ region|default:"N/A" }}</p>

<p>{{ now|date:"d M Y, H:i" }}</p>
{# 5 #}
```

```
<ul>
{# 6 #}
  {% for lead in leads %}
    <li>{{ lead.name }}</li>
  {% empty %}
    <li>No leads yet.</li>
  {% endfor %}
</ul>

<p>
{# 7 #}
  {% for lead in leads %}
    {{ forloop.counter }}. {{ lead.name }}{% if forloop.first %} (first){%
endif %}{% if not forloop.last %}, {% endif %}
  {% endfor %}
</p>

{% for lead in leads %}
{# 8 #}
  <p>{{ lead.name }}:
    {% if lead.status == "new" and lead.source in paid_sources %}Hot
    {% elif lead.status == "lost" or not lead.phone %}Cold
    {% else %}Warm{% endif %}
  </p>
{% endfor %}

{% for lead in leads %}
{# 9 #}
  <a href="{% url 'leads:detail' lead.id %}">{{ lead.name }}</a>
  (<a href="{% url 'leads:by_source' source=lead.source %}">{{ lead.source }}
</a>)
{% endfor %}

{# 10: one-line comment #}
{% comment "hidden on purpose" %}
  <p>Multi-line comment — never rendered.</p>
{% endcomment %}
{% with total=leads|length %}
  <p>{{ total }} lead{{ total|pluralize }}</p>
{% endwith %}

<p>{{ 1250000|intcomma }} AED · {{ 1234.5678|floatformat:2 }} ·
{# 19 #}
   {{ "Client asked for a callback about the gold account"|truncatechars:30
}}</p>
```

```
<p>{{ note }}</p>                                   {# 20: shows the tags as
text #}
<p>{{ note|safe }}</p>                              {# renders bold #}
{% autoescape off %}<p>{{ note }}</p>{% endautoescape %}
{% endblock content %}
```

```
# settings.py — 19
INSTALLED_APPS = [..., "django.contrib.humanize", ...]
```

B. Core

11, 14, 16–18, 21–22, 30. Final `base.html`.

```
<!-- templates/base.html -->
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}Nexora CRM{% endblock title %}</title>
  <link rel="stylesheet" href="{% static 'css/crm.css' %}">
  {% block extra_css %}{% endblock extra_css %}
</head>
<body>
<div class="layout">
  <aside class="sidebar">
{# 21 #}
    <a href="{% url 'home' %}" class="brand">Nexora CRM</a>
    {% with ns=request.resolver_match.namespace
name=request.resolver_match.url_name %}   {# 22 #}
      <a href="{% url 'dashboard' %}" class="{% if name == 'dashboard'
%}active{% endif %}">Dashboard</a>
      <a href="{% url 'leads:list' %}" class="{% if ns == 'leads' %}active{%
endif %}">Leads</a>
      <a href="{% url 'clients:list' %}" class="{% if ns == 'clients'
%}active{% endif %}">Clients</a>
      <a href="#">Reports</a>
    {% endwith %}
  </aside>
  <div class="main">
    {% include "partials/_navbar.html" %}
{# 14 #}
    <main class="content">
      {% block content %}{% endblock content %}
    </main>
```

```
    <footer class="footer">© {% now "Y" %} Nexora · Nexora CRM</footer>
{# 30 #}
  </div>
</div>
<script src="{% static 'js/crm.js' %}" defer></script>
{# 18 #}
{% block extra_js %}{% endblock extra_js %}
</body>
</html>
```

```
<!-- templates/partials/_navbar.html  (14, 17) -->
{% load static %}   {# loads don't carry into included templates #}
<header class="topbar">
  <img src="{% static 'img/logo.png' %}" alt="Nexora" height="28">
  <span>Nexora CRM</span>
</header>
```

```
<!-- templates/partials/_badge.html  (15, 24) -->
<span class="badge badge-{{ status }}">{{ status|title }}</span>
```

```
# settings.py — 16
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
```

```
mkdir -p static/css static/js static/img     # drop any PNG in
static/img/logo.png (17)
```

```
// static/js/crm.js — 18
console.log("Nexora CRM loaded");
```

```
/* static/css/crm.css — 16, 21, 24, 26, 28 */
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; color: #0f172a;
background: #f8fafc; }
.layout { display: grid; grid-template-columns: 220px 1fr; min-height: 100vh;
}
.sidebar { background: #0f172a; padding: 1rem; }
.sidebar .brand { color: #fff; font-weight: 700; display: block; margin-
bottom: 1.5rem; text-decoration: none; }
.sidebar a { display: block; color: #cbd5e1; padding: .5rem .75rem; border-
radius: 6px; text-decoration: none; }
.sidebar a.active, .sidebar a:hover { background: #1e293b; color: #fff; }
.topbar { display: flex; align-items: center; gap: .75rem; padding: .75rem
1.5rem; background: #fff; border-bottom: 1px solid #e2e8f0; }
```

```
.content { padding: 1.5rem; }
.footer { padding: 1rem 1.5rem; color: #64748b; font-size: .875rem; }
.table { width: 100%; border-collapse: collapse; background: #fff; }
.table th, .table td { padding: .6rem .75rem; text-align: left; }
.row-dark { background: #f1f5f9; }
.badge { padding: .15rem .5rem; border-radius: 999px; font-size: .75rem;
font-weight: 600; }
.badge-new { background: #dbeafe; color: #1e40af; }
.badge-contacted { background: #fef9c3; color: #854d0e; }
.badge-qualified { background: #dcfce7; color: #166534; }
.badge-lost { background: #fee2e2; color: #991b1b; }
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(160px,
1fr)); gap: 1rem; }
.card { background: #fff; border: 1px solid #e2e8f0; border-radius: 10px;
padding: 1rem; }
.card h3 { margin: 0; font-size: 1.75rem; }
.lead-card { background: #fff; border: 1px solid #e2e8f0; border-radius: 8px;
padding: .75rem; margin-bottom: .5rem; }
```

C. Nexora CRM

23, 25, 29. Views.

```
# leads/views.py (replace these two)
from django.shortcuts import render


@log_request
def lead_list(request):
    source = request.GET.get("source")
    q = request.GET.get("q", "").strip().lower()                # 29
    leads = [l for l in LEADS if not source or l["source"] == source]
    if q:
        leads = [l for l in leads if q in l["name"].lower()]
    return render(request, "leads/lead_list.html", {"leads": leads})


def lead_detail(request, lead_id):
    lead = get_lead(lead_id)
    recent = request.session.get("recent_leads", [])
    if lead_id in recent:
        recent.remove(lead_id)
    recent.insert(0, lead_id)
    request.session["recent_leads"] = recent[:5]
    return render(request, "leads/lead_detail.html", {"lead": lead})
```

13, 23–24, 27–29, 31. `lead_list.html` (add the `crm_filters` load and `mask_phone` after exercise 31).

```
<!-- leads/templates/leads/lead_list.html -->
{% extends "base.html" %}
{% load crm_filters %}
{% block title %}Leads · {{ block.super }}{% endblock title %}
{# 13 #}

{% block content %}
<h1>Leads</h1>

<form method="get" action="{% url 'leads:list' %}">
{# 29 #}
  <input type="text" name="q" value="{{ request.GET.q }}" placeholder="Search
by name">
  <button type="submit">Search</button>
</form>

<table class="table">
  <thead><tr><th>#</th><th>Name</th><th>Phone</th><th>Status</th>
<th>Source</th></tr></thead>
  <tbody>
  {% for lead in leads %}
    <tr class="{% cycle 'row-light' 'row-dark' %}">
{# 28 #}
      <td>{{ forloop.counter }}</td>
      <td><a href="{% url 'leads:detail' lead.id %}">{{ lead.name }}</a></td>
      <td>{{ lead.phone|mask_phone }}</td>
{# 31 #}
      <td>{% include "partials/_badge.html" with status=lead.status only %}
</td>
      <td>{{ lead.source }}</td>
    </tr>
  {% empty %}
    <tr><td colspan="5">No leads match "{{ request.GET.q }}".</td></tr>
  {% endfor %}
  </tbody>
</table>

<h2>By status</h2>
{# 27 #}
{% regroup leads|dictsort:"status" by status as status_groups %}
{# regroup needs sorted input #}
{% for group in status_groups %}
```

```
  <h3>{{ group.grouper|title }} ({{ group.list|length }})</h3>
  <ul>{% for lead in group.list %}<li>{{ lead.name }}</li>{% endfor %}</ul>
{% endfor %}
{% endblock content %}
```

```
<!-- leads/templates/leads/lead_detail.html  (25) -->
{% extends "base.html" %}
{% load crm_filters %}
{% block title %}{{ lead.name }} · {{ block.super }}{% endblock title %}

{% block content %}
<h1>{{ lead.name }}</h1>
<dl>
  <dt>Phone</dt><dd>{{ lead.phone|mask_phone|default:"—" }}</dd>
  <dt>Status</dt><dd>{% include "partials/_badge.html" with
status=lead.status only %}</dd>
  <dt>Source</dt><dd>{{ lead.source|title }}</dd>
</dl>
<p><a href="{% url 'leads:list' %}">← All leads</a></p>
{% endblock content %}
```

26, 32, 34–35. Dashboard.

```
# core/views.py
from datetime import timedelta


def dashboard(request):
    statuses = ["new", "contacted", "qualified", "lost"]
    counts = {s: sum(1 for l in LEADS if l["status"] == s) for s in statuses}
    now = timezone.now()
    activity = [                                                    # 35
        {"text": "Aisha Khan called back", "at": now - timedelta(minutes=5)},
        {"text": "Priya Nair qualified", "at": now - timedelta(hours=2)},
        {"text": "Mei Lin marked lost", "at": now - timedelta(days=1)},
    ]
    context = {"total": len(LEADS), "counts": counts, "leads": LEADS,
"activity": activity}
    return render(request, "core/dashboard.html", context)
```

```
<!-- core/templates/core/dashboard.html -->
{% extends "base.html" %}
{% load humanize crm_filters %}
{% block title %}Dashboard · {{ block.super }}{% endblock title %}

{% block content %}
```

```
<h1>Dashboard</h1>
<div class="cards">
  <div class="card"><h3>{{ total }}</h3><p>Total leads</p></div>
  {% for status, n in counts.items %}
    <div class="card"><h3>{{ n }}</h3><p>{{ status|title }}</p></div>
  {% endfor %}
</div>

{% status_count leads "new" as new_count %}
{# 32 #}
<p>{{ new_count }} new lead{{ new_count|pluralize }} waiting for a first
call.</p>

<h2>Recent activity</h2>
{# 35 #}
<ul>
  {% for item in activity %}<li>{{ item.text }} — {{ item.at|naturaltime }}
</li>{% endfor %}
</ul>

<h2>Lead cards</h2>
{# 34 #}
{% for lead in leads %}{% lead_card lead %}{% endfor %}
{% endblock content %}
```

D. Challenge

31–32, 34. Custom filters and tags (restart `runserver` after creating the package).

```
# leads/templatetags/__init__.py   ← empty file, required

# leads/templatetags/crm_filters.py
from django import template

register = template.Library()


@register.filter
def mask_phone(value):                                          # 31
    value = str(value)
    if len(value) < 8:
        return value
    return value[:4] + "*" * (len(value) - 8) + value[-4:]


@register.simple_tag
```

```
def status_count(leads, status):                                    # 32
    return sum(1 for lead in leads if lead["status"] == status)


@register.inclusion_tag("leads/partials/_lead_card.html")
def lead_card(lead):                                                # 34
    return {"lead": lead}
```

```
<!-- leads/templates/leads/partials/_lead_card.html -->
{% load crm_filters %}
<div class="lead-card">
  <strong>{{ lead.name }}</strong>
  {% include "partials/_badge.html" with status=lead.status only %}
  <div>{{ lead.phone|mask_phone }}</div>
</div>
```

33.

```
<!-- templates/404.html -->
{% extends "base.html" %}
{% block title %}Not found · {{ block.super }}{% endblock title %}
{% block content %}
<h1>Page not found</h1>
<p><a href="{% url 'dashboard' %}">Back to the dashboard</a></p>
{% endblock content %}
```

```
# core/views.py
def custom_404(request, exception):
    return render(request, "404.html", status=404)
```

36.

```
# python manage.py shell
from django.template.loader import render_to_string
from leads.views import LEADS

print(render_to_string("leads/partials/_lead_card.html", {"lead": LEADS[0]}))
```

37–40.

```
# leads/tests.py (add to LeadViewTests)
from .views import LEADS

    def test_lead_list_templates(self):                             # 37
        response = self.client.get(reverse("leads:list"))
        self.assertEqual(response.status_code, 200)
```

```
        self.assertTemplateUsed(response, "leads/lead_list.html")
        self.assertTemplateUsed(response, "base.html")

    def test_status_badge(self):                                    # 38
        response = self.client.get(reverse("leads:list"))
        self.assertContains(response, "badge-new")

    def test_lead_name_is_escaped(self):                            # 39
        evil = {"id": 9999, "name": "<script>alert(1)</script>",
                "phone": "", "status": "new", "source": "test"}
        LEADS.append(evil)
        try:
            response = self.client.get(reverse("leads:list"))
            self.assertNotContains(response, "<script>alert(1)</script>")
            self.assertContains(response,
"&lt;script&gt;alert(1)&lt;/script&gt;")
        finally:
            LEADS.remove(evil)
```

```
<!-- 40: templates/leads/lead_list.html  (project-level copy) -->
{% extends "base.html" %}
{% block content %}PROJECT OVERRIDE{% endblock content %}
```

```
    def test_project_template_overrides_app(self):              # 40 — DIRS
beats APP_DIRS
        response = self.client.get(reverse("leads:list"))
        self.assertContains(response, "PROJECT OVERRIDE")
# passes → delete the override file and this test
```

Day 4 — Class-based views

Goal: rebuild the Day 1–3 function views as classes and write your own reusable mixins. URL names stay the same, so every template and test keeps working. Data is still the in-memory `LEADS` list (the database arrives on Day 5–6).

A. Drills (1–10) — the base classes

- [ ] **1.** Serve `/about/` with `TemplateView.as_view(template_name=...)` straight in `urls.py` (new `core/about.html`).
- [ ] **2.** `HomeView(TemplateView)` replacing `home`; move the context into `get_context_data`.
- [ ] **3.** Move the static part of the home context (`company`, `user_count`) into `extra_context`.
- [ ] **4.** `/home/` → `home` with `RedirectView` using `pattern_name`, non-permanent.
- [ ] **5.** `/crm/` → `/leads/` with `RedirectView(url=...)`, keeping the query string (`/crm/?source=google`).
- [ ] **6.** `HealthView(View)` with a `get()` returning `OK`.
- [ ] **7.** `GetOnlyView` restricted with `http_method_names`.
- [ ] **8.** `GreetingView` with class attribute `greeting = "Hello"`; override it in `as_view(greeting="Marhaba")`.
- [ ] **9.** `LeadPhoneView(View)` reading the id from `self.kwargs`; returns the masked phone.
- [ ] **10.** `RequestInfoView` overriding `dispatch()` to print the method and class name before handling.

B. Core (11–20) — converting and extending

- [ ] **11.** `QuickAddView` with `get()` and `post()`; CSRF-exempt via `method_decorator` on `dispatch`.
- [ ] **12.** Apply Day 1's `log_request` decorator to a class with `method_decorator(..., name="dispatch")`.
- [ ] **13.** `core/mixins.py`: `JsonResponseMixin.render_to_json(data, status=200)` handling dicts and lists; use it in `LeadCountView`.
- [ ] **14.** `LeadListView(TemplateView)` rendering `leads/lead_list.html` with filtered leads.
- [ ] **15.** `LeadDetailView(TemplateView)` with the 404 and session "recent" logic.

- [ ] **16.** `DashboardView(TemplateView)` replacing `dashboard`.
- [ ] **17.** Override `get_template_names()` on `LeadListView`: `?compact=1` uses `leads/lead_list_compact.html`.
- [ ] **18.** Send an OPTIONS request to any CBV and read the `Allow` header Django builds for you.
- [ ] **19.** `JsonMethodNotAllowedMixin` overriding `http_method_not_allowed` to return JSON 405 with the allowed methods.
- [ ] **20.** Override `setup()` on `LeadCountView` to attach `self.leads` before `dispatch` runs.

C. Nexora CRM (21–30) — your own mixins

- [ ] **21.** `LeadQueryMixin.get_leads()` applying `source`, `status` and `q` from `request.GET`; use it in `LeadListView`.
- [ ] **22.** `LeadListJSONView(LeadQueryMixin, JsonResponseMixin, View)` — same filters, JSON out.
- [ ] **23.** `LeadCSVView(LeadQueryMixin, View)` — filtered CSV download.
- [ ] **24.** `LeadStatusUpdateView` — POST only, CSRF-exempt, JSON 405 for anything else.
- [ ] **25.** `LeadDeleteView` — implement `delete()`, return 204.
- [ ] **26.** `RecentLeadsView` — session "recent" ids as JSON.
- [ ] **27.** `ClientListView` and `ClientDetailView` as CBVs in `clients`.
- [ ] **28.** `ApiKeyRequiredMixin` checking `X-API-Key` in `dispatch`; replace the Day 2 decorator in `api_urls.py`.
- [ ] **29.** `PageTitleMixin` adding `page_title` to the context; show it in `_navbar.html`.
- [ ] **30.** Rewrite `leads/urls.py` to use the classes. Keep every URL name unchanged; run the tests.

D. Challenge (31–40)

- [ ] **31.** Write `simple_as_view(view_class, **initkwargs)` — your own mini `as_view()` — and route a URL through it.
- [ ] **32.** Print the MRO of `LeadListJSONView` as a list of class names.
- [ ] **33.** Turn Day 2's `rate_limited` decorator into `RateLimitMixin` with `rate_limit` and `rate_window` attributes.
- [ ] **34.** `CompanyContextMixin` + `PageTitleMixin` on one `AboutView`; confirm both context keys arrive (cooperative `super()`).
- [ ] **35.** `LatestLeadRedirectView(RedirectView)` overriding `get_redirect_url()`.

- [ ] **36.** `/robots.txt` from a `TemplateView` with `content_type="text/plain"`.
- [ ] **37.** Test that `resolve("/").func.view_class` is `HomeView`.
- [ ] **38.** Test `LeadListJSONView` directly with `RequestFactory` and `?status=lost`.
- [ ] **39.** Test that OPTIONS on `leads:json` lists `GET` in `Allow`.
- [ ] **40.** Test that GET on `leads:update_status` returns JSON 405.

Solutions

Shared mixins (13, 19, 21, 28, 29, 33, 34)

Built across the day; final file shown once.

```
# core/mixins.py
import time

from django.conf import settings
from django.http import JsonResponse

from core.utils import json_error


class JsonResponseMixin:                                            # 13
    def render_to_json(self, data, status=200):
        return JsonResponse(data, status=status, safe=isinstance(data, dict))


class JsonMethodNotAllowedMixin:                                    # 19
    def http_method_not_allowed(self, request, *args, **kwargs):
        return JsonResponse(
            {"error": f"{request.method} not allowed", "allowed":
self._allowed_methods()},
            status=405,
        )


class ApiKeyRequiredMixin:                                          # 28
    def dispatch(self, request, *args, **kwargs):
        if request.headers.get("X-API-Key") != settings.CRM_API_KEY:
            return JsonResponse({"error": "invalid API key"}, status=401)
        return super().dispatch(request, *args, **kwargs)


class PageTitleMixin:                                               # 29
    page_title = ""
```

```
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["page_title"] = self.page_title
        return context


class CompanyContextMixin:                                          # 34
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["company"] = settings.CRM_COMPANY_NAME
        return context


class RateLimitMixin:                                               # 33
    rate_limit = 10
    rate_window = 60

    def dispatch(self, request, *args, **kwargs):
        now = time.time()
        hits = [t for t in request.session.get("hits", []) if now - t <
self.rate_window]
        if len(hits) >= self.rate_limit:
            return json_error("rate limit exceeded", status=429)
        hits.append(now)
        request.session["hits"] = hits
        return super().dispatch(request, *args, **kwargs)
```

Rule: mixins go left, the Django base class goes last. Every overridden method calls `super()` so the chain continues.

A. Drills

1–8, 10, 16, 34–36. `core` views.

```
# core/views.py (class versions)
from datetime import timedelta

from django.http import HttpResponse
from django.utils import timezone
from django.views import View
from django.views.generic import TemplateView

from core.mixins import PageTitleMixin, CompanyContextMixin
from leads.views import LEADS
```

```
class HomeView(TemplateView):                                        # 2–3
    template_name = "core/home.html"
    extra_context = {"company": "Nexora", "user_count": 12}

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)   # includes
extra_context
        context.update(
            now=timezone.now(),
            leads=LEADS,
            paid_sources=["google", "facebook"],
            note="<b>VIP</b> client",
        )
        return context


class HealthView(View):                                              # 6
    def get(self, request):
        return HttpResponse("OK")


class GetOnlyView(View):                                             # 7
    http_method_names = ["get"]

    def get(self, request):
        return HttpResponse("GET only")


class GreetingView(View):                                            # 8
    greeting = "Hello"

    def get(self, request):
        return HttpResponse(f"{self.greeting} from Nexora CRM")


class RequestInfoView(View):                                         # 10
    def dispatch(self, request, *args, **kwargs):
        print(f"dispatching {request.method} to {self.__class__.__name__}")
        return super().dispatch(request, *args, **kwargs)

    def get(self, request):
        return HttpResponse(f"{request.method} {request.path}")


class DashboardView(PageTitleMixin, TemplateView):                   # 16
```

```
    template_name = "core/dashboard.html"
    page_title = "Dashboard"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        statuses = ["new", "contacted", "qualified", "lost"]
        now = timezone.now()
        context.update(
            total=len(LEADS),
            counts={s: sum(1 for l in LEADS if l["status"] == s) for s in
statuses},
            leads=LEADS,
            activity=[
                {"text": "Aisha Khan called back", "at": now -
timedelta(minutes=5)},
                {"text": "Priya Nair qualified", "at": now -
timedelta(hours=2)},
            ],
        )
        return context


class AboutView(CompanyContextMixin, PageTitleMixin, TemplateView):   # 34
    template_name = "core/about.html"
    page_title = "About"
```

```
<!-- core/templates/core/about.html -->
{% extends "base.html" %}
{% block content %}<h1>About {{ company|default:"Nexora" }} CRM</h1>{%
endblock content %}
```

```
# templates/robots.txt  (36)
User-agent: *
Disallow: /
```

```
# core/urls.py
from django.urls import path
from django.views.generic import TemplateView, RedirectView
from . import views

urlpatterns = [
    path("", views.HomeView.as_view(), name="home"),
    path("about/", views.AboutView.as_view(), name="about"),         # 1 → 34
    # 1 first version: path("about/",
TemplateView.as_view(template_name="core/about.html"), name="about"),
    path("home/", RedirectView.as_view(pattern_name="home", permanent=False),
name="old_home"),   # 4
    path("crm/", RedirectView.as_view(url="/leads/", query_string=True),
name="crm"),            # 5
    path("health/", views.HealthView.as_view(), name="health"),
    path("get-only/", views.GetOnlyView.as_view(), name="get_only"),
    path("hi/", views.GreetingView.as_view(greeting="Marhaba"),
name="greeting"),                # 8
    path("request-info/", views.RequestInfoView.as_view(),
name="request_info"),
    path("dashboard/", views.DashboardView.as_view(), name="dashboard"),
    path("robots.txt", TemplateView.as_view(template_name="robots.txt",
content_type="text/plain")),  # 36
    # keep the remaining Day 1–2 function views (api/info, secret, files,
accounts...) as they are
]
```

`as_view()` only accepts keyword arguments that already exist as class attributes (8).

9.

```
# leads/views.py
class LeadPhoneView(View):
    def get(self, request, *args, **kwargs):
        phone = get_lead(self.kwargs["lead_id"])["phone"]
        return HttpResponse(phone[:4] + "*" * (len(phone) - 8) + phone[-4:])
```

B + C. Leads and clients as classes

11–15, 17, 20–26, 35.

```
# leads/views.py (class versions — keep LEADS, VALID_STATUSES, get_lead)
import csv

from django.http import HttpResponse, JsonResponse
from django.urls import reverse
```

```
from django.utils.decorators import method_decorator
from django.views import View
from django.views.decorators.csrf import csrf_exempt
from django.views.generic import TemplateView, RedirectView

from core.decorators import log_request
from core.mixins import (
    ApiKeyRequiredMixin, JsonResponseMixin, JsonMethodNotAllowedMixin,
PageTitleMixin,
)
from core.utils import json_error


class LeadQueryMixin:                                           # 21
    def get_leads(self):
        params = self.request.GET
        leads = LEADS
        if source := params.get("source"):
            leads = [l for l in leads if l["source"] == source]
        if status := params.get("status"):
            leads = [l for l in leads if l["status"] == status]
        if q := params.get("q", "").strip().lower():
            leads = [l for l in leads if q in l["name"].lower()]
        return leads


@method_decorator(log_request, name="dispatch")                  # 12
class LeadListView(PageTitleMixin, LeadQueryMixin, TemplateView):   # 14
    template_name = "leads/lead_list.html"
    page_title = "Leads"

    def get_template_names(self):                               # 17
        if self.request.GET.get("compact") == "1":
            return ["leads/lead_list_compact.html"]
        return [self.template_name]

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["leads"] = self.get_leads()
        return context


class LeadDetailView(PageTitleMixin, TemplateView):             # 15
    template_name = "leads/lead_detail.html"
    page_title = "Lead"
```

```
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        lead_id = self.kwargs["lead_id"]
        lead = get_lead(lead_id)                                  # raises
Http404
        recent = self.request.session.get("recent_leads", [])
        if lead_id in recent:
            recent.remove(lead_id)
        recent.insert(0, lead_id)
        self.request.session["recent_leads"] = recent[:5]
        context["lead"] = lead
        return context


class LeadCountView(JsonResponseMixin, View):                     # 13 + 20
    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, **kwargs)
        self.leads = LEADS

    def get(self, request):
        return self.render_to_json({"total": len(self.leads)})


class LeadListJSONView(LeadQueryMixin, JsonResponseMixin, View):   # 22
    def get(self, request, *args, **kwargs):
        return self.render_to_json(self.get_leads())


class LeadCSVView(LeadQueryMixin, View):                          # 23
    def get(self, request):
        response = HttpResponse(content_type="text/csv")
        response["Content-Disposition"] = 'attachment; filename="leads.csv"'
        writer = csv.DictWriter(response, fieldnames=["id", "name", "phone",
"status", "source"])
        writer.writeheader()
        writer.writerows(self.get_leads())
        return response


@method_decorator(csrf_exempt, name="dispatch")
class QuickAddView(View):                                         # 11
    def get(self, request):
        return HttpResponse("POST a 'name' field to add a lead")

    def post(self, request):
        name = request.POST.get("name", "").strip()
```

```
        lead = {"id": max(l["id"] for l in LEADS) + 1, "name": name,
                "phone": "", "status": "new", "source": "manual"}
        LEADS.append(lead)
        return JsonResponse(lead, status=201)


@method_decorator(csrf_exempt, name="dispatch")
class LeadStatusUpdateView(JsonMethodNotAllowedMixin, View):     # 24 — only
post() exists
    def post(self, request, lead_id):
        lead = get_lead(lead_id)
        status = request.POST.get("status")
        if status not in VALID_STATUSES:
            return json_error("invalid status")
        lead["status"] = status
        return JsonResponse(lead)


@method_decorator(csrf_exempt, name="dispatch")
class LeadDeleteView(View):                                      # 25
    def delete(self, request, lead_id):
        LEADS.remove(get_lead(lead_id))
        return HttpResponse(status=204)


class RecentLeadsView(JsonResponseMixin, View):                  # 26
    def get(self, request):
        ids = request.session.get("recent_leads", [])
        return self.render_to_json([get_lead(i) for i in ids])


class LatestLeadRedirectView(RedirectView):                      # 35
    def get_redirect_url(self, *args, **kwargs):
        return reverse("leads:detail", args=[LEADS[-1]["id"]])
```

```
<!-- leads/templates/leads/lead_list_compact.html  (17) -->
{% extends "base.html" %}
{% block content %}
<ul>{% for lead in leads %}<li>{{ lead.name }} · {{ lead.status }}</li>{%
endfor %}</ul>
{% endblock content %}
```

```
<!-- templates/partials/_navbar.html  (29) -->
{% load static %}
<header class="topbar">
  <img src="{% static 'img/logo.png' %}" alt="Nexora" height="28">
  <span>{{ page_title|default:"Nexora CRM" }}</span>   {# include sees the
parent context #}
</header>
```

27.

```
# clients/views.py
from django.http import Http404
from django.views import View
from core.mixins import JsonResponseMixin


class ClientListView(JsonResponseMixin, View):
    def get(self, request):
        return self.render_to_json(CLIENTS)


class ClientDetailView(JsonResponseMixin, View):
    def get(self, request, ref):
        for client in CLIENTS:
            if client["ref"] == ref:
                return self.render_to_json(client)
        raise Http404("Client not found")

# clients/urls.py
path("", views.ClientListView.as_view(), name="list"),
path("<uuid:ref>/", views.ClientDetailView.as_view(), name="detail"),
```

28.

```
# leads/views.py
class ApiLeadListView(ApiKeyRequiredMixin, LeadListJSONView):
    pass


class ApiLeadDetailView(ApiKeyRequiredMixin, JsonResponseMixin, View):
    def get(self, request, lead_id):
        return self.render_to_json(get_lead(lead_id))

# leads/api_urls.py
urlpatterns = [
    path("leads/", views.ApiLeadListView.as_view(), name="list"),
    path("leads/<int:lead_id>/", views.ApiLeadDetailView.as_view(),
name="detail"),
]
```

30. Swap the converted routes in `leads/urls.py`; names unchanged. Function views not converted stay as they are — both styles mix fine.

```
# leads/urls.py (changed lines)
path("", views.LeadListView.as_view(), name="list"),
path("count/", views.LeadCountView.as_view(), name="count"),
path("json/", views.LeadListJSONView.as_view(), name="json"),
path("export/", views.LeadCSVView.as_view(), name="export"),
path("quick-add/", views.QuickAddView.as_view(), name="quick_add"),
path("latest/", views.LatestLeadRedirectView.as_view(), name="latest"),
path("recent/", views.RecentLeadsView.as_view(), name="recent"),
path("<int:lead_id>/", views.LeadDetailView.as_view(), name="detail"),
path("<int:lead_id>/phone/", views.LeadPhoneView.as_view(), name="phone"),
path("<int:lead_id>/status/", views.LeadStatusUpdateView.as_view(),
name="update_status"),
path("<int:lead_id>/delete/", views.LeadDeleteView.as_view(), name="delete"),
```

```
python manage.py test     # all Day 1–3 tests should still pass
```

18.

```
curl -i -X OPTIONS http://127.0.0.1:8000/leads/json/
# Allow: GET, HEAD, OPTIONS   (HEAD is added automatically when get() exists)
```

D. Challenge

31.

```
# core/views.py
def simple_as_view(view_class, **initkwargs):
    def view(request, *args, **kwargs):
        self = view_class(**initkwargs)      # View.__init__ sets the
attributes
        self.setup(request, *args, **kwargs)
        return self.dispatch(request, *args, **kwargs)
    return view

# core/urls.py
path("simple-health/", views.simple_as_view(views.HealthView),
name="simple_health"),
```

32.

```
# python manage.py shell
from leads.views import LeadListJSONView
print([c.__name__ for c in LeadListJSONView.__mro__])
# ['LeadListJSONView', 'LeadQueryMixin', 'JsonResponseMixin', 'View',
'object']
```

33. Usage of `RateLimitMixin`:

```
class SearchView(RateLimitMixin, JsonResponseMixin, View):
    rate_limit = 5

    def get(self, request):
        q = request.GET.get("q", "").strip().lower()
        results = [l for l in LEADS if q in l["name"].lower()] if q else []
        return self.render_to_json({"query": q, "count": len(results),
"results": results})
```

37–40.

```
# leads/tests.py (new class)
import json

from django.test import RequestFactory, SimpleTestCase
from django.urls import resolve, reverse

from core.views import HomeView
from .views import LeadListJSONView


class ClassBasedViewTests(SimpleTestCase):
    def test_home_uses_homeview(self):                          # 37
```

```
        self.assertEqual(resolve("/").func.view_class, HomeView)

    def test_json_view_filters_status(self):                    # 38
        request = RequestFactory().get("/leads/json/", {"status": "lost"})
        response = LeadListJSONView.as_view()(request)
        data = json.loads(response.content)
        self.assertTrue(data)
        self.assertTrue(all(l["status"] == "lost" for l in data))

    def test_options_allow_header(self):                        # 39
        response = self.client.options(reverse("leads:json"))
        self.assertIn("GET", response["Allow"])

    def test_status_update_get_is_json_405(self):               # 40
        response = self.client.get(reverse("leads:update_status", args=[1]))
        self.assertEqual(response.status_code, 405)
        self.assertIn("error", response.json())
```

`RequestFactory` skips middleware and URL routing: you call the view directly, so sessions and messages aren't available.

Day 5 — Models and migrations

Goal: real database tables for leads, clients and brokers — built field by field, one migration at a time. Views still use `LEADS` today; Day 6 switches them to the database.

Run `makemigrations` + `migrate` after every exercise that changes a model.

A. Drills (1–10) — fields and migrations

- [ ] **1.** `Lead` model in `leads/models.py` with only `name` (max 120). Make and apply the migration.
- [ ] **2.** Print the SQL of that migration with `sqlmigrate`.
- [ ] **3.** Add `phone` (max 15) and `email` (optional).
- [ ] **4.** Add `status` using a nested `TextChoices` class `Status` (new, contacted, qualified, lost), default new.
- [ ] **5.** Add `source` using `TextChoices` `Source` (facebook, google, referral, walk_in, manual), default manual.
- [ ] **6.** Add `created_at` (set once on insert) and `updated_at` (set on every save).
- [ ] **7.** Add `budget`: decimal, 12 digits, 2 decimal places, nullable.
- [ ] **8.** Add `is_active` (default True) and `notes` (optional long text).
- [ ] **9.** `__str__` returning `Aisha Khan (New)` via `get_status_display()`; `get_absolute_url` to `leads:detail`.
- [ ] **10.** `showmigrations leads`; migrate `leads` back to zero, then forward again.

B. Core (11–20) — Meta, constraints, behaviour

- [ ] **11.** `Meta`: newest first, `verbose_name` and `verbose_name_plural`.
- [ ] **12.** Rename the table to `crm_leads` with `db_table`.
- [ ] **13.** Composite index on `status` + `created_at`, named.
- [ ] **14.** `UniqueConstraint` on `phone`.
- [ ] **15.** `CheckConstraint`: budget must be ⩾ 0 (Django 5.1+ uses `condition=`).
- [ ] **16.** `phone`: verbose name `Phone number` and a help text; index `email`.
- [ ] **17.** Register `Lead` in the admin, create a superuser, add two leads by hand.
- [ ] **18.** In the shell: create a lead, read `status` vs `get_status_display()`, list `Lead.Status.choices`.
- [ ] **19.** Property `is_hot` (new + paid source) and method `mark_contacted()` saving only the changed fields.

- [ ] **20.** Data migration (`--empty` , `RunPython` with a reverse function) that loads the 5 `LEADS` dicts into the table.

C. Nexora CRM (21–30)

- [ ] **21.** `Client` model in `clients` : `ref` (UUID, auto, not editable, unique), name, email, account_number, `account_type` (standard/gold/vip), balance, `kyc_status` (pending/approved/rejected), date_joined, is_active.
- [ ] **22.** `Client.Meta` : order by name; unique account number; balance ⩾ 0.
- [ ] **23.** `RegexValidator` on `account_number` : exactly 7 digits.
- [ ] **24.** New `brokers` app + `Broker` : code (unique), name, email, commission_rate (0–100 with min/max validators), `tier` as `IntegerChoices` 1–3, is_active, created_at.
- [ ] **25.** `RegexValidator` on broker code: `BRF` + 4 digits.
- [ ] **26.** UAE mobile validator on `Lead.phone` : `9715` + 8 digits.
- [ ] **27.** Register `Client` and `Broker` in the admin.
- [ ] **28.** In the shell, prove validators run on `full_clean()` but not on `save()` .
- [ ] **29.** Dialer fields on `Lead` : `contact_attempts` (small positive int, default 0) and `last_contacted_at` (nullable); update `mark_contacted()` to use them.
- [ ] **30.** `__str__` + `get_absolute_url` for `Client` (by `ref`); `__str__` for `Broker` (`BRF1001 — Name`); property `Client.is_kyc_approved` .

D. Challenge (31–40)

- [ ] **31.** Override `Lead.save()` to normalise the phone (`+971 50 123 4567` , `00971...` , `050...` → `971501234567`) and title-case the name.
- [ ] **32.** `Lead.clean()` : a lost lead must have notes. Check it with `full_clean()` in the shell.
- [ ] **33.** Rename `budget` → `expected_deposit` ; answer yes to the rename prompt; fix the constraint.
- [ ] **34.** Add required `country` (max 2) with no default; give `AE` as the one-off default in the prompt; then add `default="AE"` to the model.
- [ ] **35.** Roll `leads` back one migration, then forward again.
- [ ] **36.** CI check: fail if model changes have no migration (`makemigrations --check --dry-run`).
- [ ] **37.** `LeadModelTests(TestCase)` : default status, `__str__` (after title-casing), Meta ordering.
- [ ] **38.** Test duplicate phone raises `IntegrityError` .

- [ ] **39.** Test negative `expected_deposit` raises `IntegrityError`.
- [ ] **40.** Test `full_clean()` rejects a bad lead phone and a bad broker code.

Solutions

Final `Lead` model (1–9, 11–16, 19, 26, 29, 31–34)

The exercise number sits beside each line. Build it step by step, migrating as you go.

```
# leads/models.py
from django.core.exceptions import ValidationError
from django.core.validators import RegexValidator
from django.db import models
from django.urls import reverse
from django.utils import timezone

uae_phone_validator = RegexValidator(r"^9715\d{8}$", "Use UAE mobile format:
9715XXXXXXXX")   # 26


def normalize_uae_phone(raw):
# 31
    digits = "".join(ch for ch in raw if ch.isdigit())
    if digits.startswith("00"):
        digits = digits[2:]
    if digits.startswith("05"):
        digits = "971" + digits[1:]
    return digits


class Lead(models.Model):
    class Status(models.TextChoices):
# 4
        NEW = "new", "New"
        CONTACTED = "contacted", "Contacted"
        QUALIFIED = "qualified", "Qualified"
        LOST = "lost", "Lost"

    class Source(models.TextChoices):
# 5
        FACEBOOK = "facebook", "Facebook"
        GOOGLE = "google", "Google"
        REFERRAL = "referral", "Referral"
        WALK_IN = "walk_in", "Walk-in"
        MANUAL = "manual", "Manual"
```

```
    PAID_SOURCES = {Source.FACEBOOK, Source.GOOGLE}

    name = models.CharField(max_length=120)
# 1
    phone = models.CharField(
# 3, 16, 26
        "Phone number", max_length=15,
        validators=[uae_phone_validator], help_text="9715XXXXXXXX",
    )
    email = models.EmailField(blank=True, db_index=True)
# 3, 16
    status = models.CharField(max_length=20, choices=Status.choices,
default=Status.NEW)
    source = models.CharField(max_length=20, choices=Source.choices,
default=Source.MANUAL)
    expected_deposit = models.DecimalField(
# 7 (was budget), 33
        max_digits=12, decimal_places=2, null=True, blank=True,
    )
    country = models.CharField(max_length=2, default="AE")
# 34
    is_active = models.BooleanField(default=True)
# 8
    notes = models.TextField(blank=True)
    contact_attempts = models.PositiveSmallIntegerField(default=0)
# 29
    last_contacted_at = models.DateTimeField(null=True, blank=True)
# 29
    created_at = models.DateTimeField(auto_now_add=True)
# 6
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = "crm_leads"
# 12
        ordering = ["-created_at"]
# 11
        verbose_name = "lead"
        verbose_name_plural = "leads"
        indexes = [
# 13
            models.Index(fields=["status", "created_at"],
name="lead_status_created_idx"),
        ]
        constraints = [
```

```
            models.UniqueConstraint(fields=["phone"],
name="lead_unique_phone"),        # 14
            models.CheckConstraint(
# 15, 33
                condition=models.Q(expected_deposit__gte=0),
                name="lead_deposit_non_negative",
            ),
        ]

    def __str__(self):
# 9
        return f"{self.name} ({self.get_status_display()})"

    def get_absolute_url(self):
        return reverse("leads:detail", kwargs={"lead_id": self.pk})

    @property
    def is_hot(self):
# 19
        return self.status == self.Status.NEW and self.source in
self.PAID_SOURCES

    def mark_contacted(self):
# 19 → 29
        self.status = self.Status.CONTACTED
        self.contact_attempts += 1
        self.last_contacted_at = timezone.now()
        self.save(update_fields=[
            "status", "contact_attempts", "last_contacted_at",
            "updated_at",   # auto_now only updates when listed in
update_fields
        ])

    def clean(self):
# 32
        if self.status == self.Status.LOST and not self.notes.strip():
            raise ValidationError({"notes": "Give a reason when marking a
lead lost."})

    def save(self, *args, **kwargs):
# 31
        self.phone = normalize_uae_phone(self.phone)
        self.name = self.name.strip().title()
        super().save(*args, **kwargs)
```

`UniqueConstraint` on one field = `unique=True` on the field; the constraint form is preferred because it's named and extensible (14).

A. Drills — commands

1–2, 10.

```
python manage.py makemigrations leads
python manage.py migrate
python manage.py sqlmigrate leads 0001      # 2

python manage.py showmigrations leads       # 10
python manage.py migrate leads zero         # drops the leads tables
python manage.py migrate leads
```

B. Core

17.

```
# leads/admin.py
from django.contrib import admin
from .models import Lead

admin.site.register(Lead)
```

```
python manage.py createsuperuser
```

18.

```
# python manage.py shell
from leads.models import Lead

lead = Lead.objects.create(name="Zara Ahmed", phone="971507770001",
source="google")
lead.status                    # 'new'
lead.get_status_display()      # 'New'
Lead.Status.choices            # [('new', 'New'), ('contacted', 'Contacted'),
...]
Lead.Status.values             # ['new', 'contacted', 'qualified', 'lost']
lead.is_hot                    # True
```

20.

```
python manage.py makemigrations leads --empty --name seed_initial_leads
```

```
# leads/migrations/000X_seed_initial_leads.py
from django.db import migrations

INITIAL = [
    {"name": "Aisha Khan", "phone": "971501234567", "status": "new",
"source": "facebook"},
    {"name": "Omar Haddad", "phone": "971502345678", "status": "contacted",
"source": "google"},
    {"name": "Priya Nair", "phone": "971503456789", "status": "qualified",
"source": "referral"},
    {"name": "Daniel Ortiz", "phone": "971504567890", "status": "new",
"source": "google"},
    {"name": "Mei Lin", "phone": "971505678901", "status": "lost", "source":
"facebook"},
]


def seed(apps, schema_editor):
    Lead = apps.get_model("leads", "Lead")     # historical model: no custom
save()/methods
    for row in INITIAL:
        Lead.objects.get_or_create(phone=row["phone"], defaults=row)


def unseed(apps, schema_editor):
    Lead = apps.get_model("leads", "Lead")
    Lead.objects.filter(phone__in=[r["phone"] for r in INITIAL]).delete()


class Migration(migrations.Migration):
    dependencies = [("leads", "000N_previous")]   # filled in for you by --
empty
    operations = [migrations.RunPython(seed, unseed)]
```

Data migrations also run when the test database is built, so these 5 rows exist in every test.

C. Nexora CRM

21–23, 30.

```
# clients/models.py
import uuid

from django.core.validators import RegexValidator
from django.db import models
```

```
from django.urls import reverse


class Client(models.Model):
    class AccountType(models.TextChoices):
        STANDARD = "standard", "Standard"
        GOLD = "gold", "Gold"
        VIP = "vip", "VIP"

    class KYCStatus(models.TextChoices):
        PENDING = "pending", "Pending"
        APPROVED = "approved", "Approved"
        REJECTED = "rejected", "Rejected"

    ref = models.UUIDField(default=uuid.uuid4, editable=False, unique=True)
    name = models.CharField(max_length=120)
    email = models.EmailField()
    account_number = models.CharField(
# 23
        max_length=7, validators=[RegexValidator(r"^\d{7}$", "Account number
must be 7 digits")],
    )
    account_type = models.CharField(max_length=10,
choices=AccountType.choices, default=AccountType.STANDARD)
    balance = models.DecimalField(max_digits=14, decimal_places=2, default=0)
    kyc_status = models.CharField(max_length=10, choices=KYCStatus.choices,
default=KYCStatus.PENDING)
    date_joined = models.DateField(auto_now_add=True)
    is_active = models.BooleanField(default=True)

    class Meta:
# 22
        ordering = ["name"]
        constraints = [
            models.UniqueConstraint(fields=["account_number"],
name="client_unique_account"),
            models.CheckConstraint(condition=models.Q(balance__gte=0),
name="client_balance_non_negative"),
        ]

    def __str__(self):
# 30
        return f"{self.name} ({self.account_number})"

    def get_absolute_url(self):
        return reverse("clients:detail", kwargs={"ref": self.ref})
```

```
    @property
    def is_kyc_approved(self):
        return self.kyc_status == self.KYCStatus.APPROVED
```

24–25, 30.

```
python manage.py startapp brokers     # add "brokers.apps.BrokersConfig" to
INSTALLED_APPS
```

```
# brokers/models.py
from decimal import Decimal

from django.core.validators import MaxValueValidator, MinValueValidator,
RegexValidator
from django.db import models


class Broker(models.Model):
    class Tier(models.IntegerChoices):
        TIER_1 = 1, "Tier 1"
        TIER_2 = 2, "Tier 2"
        TIER_3 = 3, "Tier 3"

    code = models.CharField(
# 25
        max_length=7, unique=True,
        validators=[RegexValidator(r"^BRF\d{4}$", "Format: BRF0000")],
    )
    name = models.CharField(max_length=120)
    email = models.EmailField(blank=True)
    commission_rate = models.DecimalField(
        max_digits=5, decimal_places=2, default=Decimal("0.00"),
        validators=[MinValueValidator(Decimal("0")),
MaxValueValidator(Decimal("100"))],
    )
    tier = models.PositiveSmallIntegerField(choices=Tier.choices,
default=Tier.TIER_3)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["code"]
```

```
    def __str__(self):
# 30
        return f"{self.code} — {self.name}"
```

27.

```
# clients/admin.py
from django.contrib import admin
from .models import Client

admin.site.register(Client)

# brokers/admin.py
from django.contrib import admin
from .models import Broker

admin.site.register(Broker)
```

28.

```
# python manage.py shell
from django.core.exceptions import ValidationError
from leads.models import Lead

bad = Lead(name="Test", phone="12345")
try:
    bad.full_clean()                # runs field validators + clean()
except ValidationError as e:
    print(e.message_dict)           # {'phone': ['Use UAE mobile format:
9715XXXXXXXX']}

bad.save()                          # no validation — the row is saved
bad.delete()
```

Forms (Day 13) call `full_clean()` for you; `save()` and `objects.create()` never do.

D. Challenge

32.

```
lead = Lead(name="Lost One", phone="971507770005", status=Lead.Status.LOST)
lead.full_clean()      # ValidationError: {'notes': ['Give a reason when
marking a lead lost.']}
```

33–36.

```
# 33: rename budget → expected_deposit in the model AND in the
CheckConstraint's Q(...)
python manage.py makemigrations leads
# Was lead.budget renamed to lead.expected_deposit (a DecimalField)? [y/N] y
python manage.py migrate

# 34: add country = models.CharField(max_length=2) with no default
python manage.py makemigrations leads
# 1) Provide a one-off default now ... Select an option: 1
# >>> "AE"
# then add default="AE" to the field and run makemigrations again

# 35
python manage.py showmigrations leads            # note the second-to-last
number
python manage.py migrate leads 000N              # roll back to it
python manage.py migrate leads                   # forward again

# 36: exits non-zero if a migration is missing
python manage.py makemigrations --check --dry-run
```

37–40.

```
# leads/tests.py (new class — TestCase, because it touches the database)
from decimal import Decimal

from django.core.exceptions import ValidationError
from django.db import IntegrityError, transaction
from django.test import TestCase

from brokers.models import Broker
from .models import Lead


class LeadModelTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.lead = Lead.objects.create(name="test lead",
phone="971509999001")

    def test_defaults_and_str(self):                              # 37
        self.assertEqual(self.lead.status, Lead.Status.NEW)
        self.assertEqual(str(self.lead), "Test Lead (New)")       # save()
title-cased it
        self.assertEqual(Lead._meta.ordering, ["-created_at"])
```

```
    def test_unique_phone(self):                                    # 38
        with self.assertRaises(IntegrityError), transaction.atomic():
            Lead.objects.create(name="Dup", phone="971509999001")

    def test_negative_deposit_rejected(self):                       # 39
        with self.assertRaises(IntegrityError), transaction.atomic():
            Lead.objects.create(name="Neg", phone="971509999002",
expected_deposit=Decimal("-1"))

    def test_validators(self):                                      # 40
        with self.assertRaises(ValidationError):
            Lead(name="Bad", phone="12345").full_clean()
        with self.assertRaises(ValidationError):
            Broker(code="XYZ1", name="Bad broker").full_clean()
```

`transaction.atomic()` inside `assertRaises` rolls back the failed insert so the rest of the test can keep using the database.

Day 6 — ORM CRUD and the shell

Goal: fluent create/read/update/delete in the shell, 200 realistic seeded leads, and the lead views running on the database instead of `LEADS`.

Do A and B in `python manage.py shell`.

A. Drills (1–10) — reading and writing

- [ ] **1.** Import `Lead` and print how many rows exist.
- [ ] **2.** Create a lead with `objects.create()`.
- [ ] **3.** Create one by instantiating `Lead(...)` and calling `save()`; print its `pk`.
- [ ] **4.** `get()` by pk; catch `Lead.DoesNotExist` for pk 99999.
- [ ] **5.** `filter()` new leads; print the count.
- [ ] **6.** `exclude()` lost leads.
- [ ] **7.** `first()`, `last()`, `latest("created_at")`, `earliest("created_at")` — note how `first()` follows Meta ordering.
- [ ] **8.** Check whether any lead has source `walk_in` with `exists()`.
- [ ] **9.** Five leads by name A→Z; five newest.
- [ ] **10.** `values()` (name, status) for 3 rows; `values_list("phone", flat=True)` for 3 rows.

B. Core (11–20) — updates, deletes, lookups

- [ ] **11.** Change one lead's status and save only that field (plus `updated_at`).
- [ ] **12.** Bulk-update every new Google lead to contacted with `queryset.update()`; print the row count.
- [ ] **13.** Delete one lead; delete a filtered queryset; read the returned tuples.
- [ ] **14.** `get_or_create` by phone with `defaults`.
- [ ] **15.** `update_or_create` by phone, changing status.
- [ ] **16.** `in_bulk()` by pk list and by phone.
- [ ] **17.** Distinct list of sources — then see why default ordering breaks `distinct()` and fix it.
- [ ] **18.** Lookups: `icontains`, `startswith`, `__date` = today, `gte` on deposit, `endswith` on phone, `__in` on status, last 7 days.
- [ ] **19.** Chain filter → exclude → order_by and print `qs.query` before it runs.
- [ ] **20.** `count()` vs `len()`; loop 200 rows with `.iterator(chunk_size=50)`.

C. Nexora CRM (21–30) — seed + switch views to the DB

- [ ] **21.** `leads/seed.py` with `run(n=200, seed=42)`: random names, unique UAE phones, weighted statuses, sources, deposits, notes for lost leads, `created_at` spread over 60 days. Use `bulk_create`.
- [ ] **22.** Run the seed in the shell; print the count per status by looping `Lead.Status.choices`.
- [ ] **23.** `LeadQueryMixin.get_leads()` returns a queryset (active leads, `source`, `status`, `q` via `icontains`).
- [ ] **24.** `LeadDetailView` uses `get_object_or_404`; detail page shows `get_source_display`.
- [ ] **25.** `DashboardView` counts per status from the DB; `status_count` tag takes a queryset; lead cards show the 6 newest.
- [ ] **26.** `LeadListJSONView` returns `values()` rows from the DB.
- [ ] **27.** `LeadStatusUpdateView` validates against `Lead.Status.values` and saves to the DB.
- [ ] **28.** `LeadDeleteView` deletes the row.
- [ ] **29.** `LeadCSVView` exports from the DB with `values_list`.
- [ ] **30.** `StaleLeadsView` at `leads/stale/`: new leads older than 7 days, oldest first, JSON.

D. Challenge (31–40)

- [ ] **31.** Clone a lead (new pk, new phone) by resetting `pk` and `_state.adding`.
- [ ] **32.** Show an in-memory object going stale after `queryset.update()`; fix with `refresh_from_db()`.
- [ ] **33.** Soft delete: `Lead.archive()` sets `is_active=False`; archived leads disappear from the list.
- [ ] **34.** Trigger and catch `Lead.MultipleObjectsReturned`.
- [ ] **35.** Count queries with `CaptureQueriesContext`: `len(qs)` twice vs `count()` twice on a fresh queryset.
- [ ] **36.** Show `qs[0]` hits the DB on every call until the queryset is evaluated.
- [ ] **37.** Raw SQL two ways: `Lead.objects.raw()` with params, and `connection.cursor()` for a GROUP BY.
- [ ] **38.** Update the Day 1–4 lead tests for the DB: `TestCase` base, escape test creates a real row.
- [ ] **39.** Test `update_status` changes the row (`refresh_from_db`).
- [ ] **40.** Test `delete` removes the row.

Solutions

Tip: run the seed (21–22) first if you want bigger result sets for A and B.

A. Drills

1–10.

```
# python manage.py shell
from leads.models import Lead

Lead.objects.count()                                                    # 1

Lead.objects.create(name="Zara Ahmed", phone="971507770011", source="google")
# 2

lead = Lead(name="Karim Aziz", phone="971507770012")                    # 3
lead.save()
lead.pk

try:                                                                    # 4
    Lead.objects.get(pk=99999)
except Lead.DoesNotExist:
    print("not found")

new = Lead.objects.filter(status=Lead.Status.NEW)                       # 5
new.count()

Lead.objects.exclude(status=Lead.Status.LOST).count()                   # 6

Lead.objects.first()                  # 7: newest — Meta ordering is -
created_at
Lead.objects.last()                   # oldest
Lead.objects.latest("created_at")
Lead.objects.earliest("created_at")

Lead.objects.filter(source=Lead.Source.WALK_IN).exists()                # 8

Lead.objects.order_by("name")[:5]                                       # 9
Lead.objects.order_by("-created_at")[:5]

Lead.objects.values("name", "status")[:3]                               #
10: list of dicts
Lead.objects.values_list("phone", flat=True)[:3]                        #
flat list
```

B. Core

11–20.

```
from datetime import timedelta
from django.utils import timezone

lead = Lead.objects.get(phone="971507770011")                                  # 11
lead.status = Lead.Status.CONTACTED
lead.save(update_fields=["status", "updated_at"])

n = (Lead.objects                                                              # 12
     .filter(source="google", status="new")
     .update(status="contacted", updated_at=timezone.now()))
print(n)          # rows changed — update() skips save(): no auto_now, no
normalising

Lead.objects.get(phone="971507770012").delete()                                # 13
→ (1, {'leads.Lead': 1})
Lead.objects.filter(name__startswith="Zara").delete()

lead, created = Lead.objects.get_or_create(                                    # 14
    phone="971507770013",
    defaults={"name": "Huda Saleh", "source": "referral"},
)

lead, created = Lead.objects.update_or_create(                                 # 15
    phone="971507770013",
    defaults={"status": "qualified"},          # Django 5.0+:
create_defaults={...} for create-only values
)

ids = list(Lead.objects.values_list("pk", flat=True)[:3])                      # 16
Lead.objects.in_bulk(ids)                                                      #
{pk: <Lead>, ...}
Lead.objects.in_bulk(["971501234567"], field_name="phone")                     #
field must be unique

list(Lead.objects.values_list("source", flat=True).distinct())                # 17:
duplicates!
# Meta ordering adds created_at to SELECT DISTINCT. Fix by clearing or
setting ordering:
list(Lead.objects.order_by().values_list("source", flat=True).distinct())
list(Lead.objects.order_by("source").values_list("source",
flat=True).distinct())

Lead.objects.filter(name__icontains="khan")                                    # 18
```

```
Lead.objects.filter(name__startswith="A")
Lead.objects.filter(created_at__date=timezone.localdate())
Lead.objects.filter(expected_deposit__gte=10_000)
Lead.objects.filter(phone__endswith="67")
Lead.objects.filter(status__in=["new", "contacted"])
Lead.objects.filter(created_at__gte=timezone.now() - timedelta(days=7))

qs = (Lead.objects                                                    # 19
      .filter(status="new")
      .exclude(source="manual")
      .order_by("name"))
print(qs.query)      # SQL text — nothing has run yet (querysets are lazy)
list(qs)             # now it runs

Lead.objects.count()           # 20: SELECT COUNT(*)
len(Lead.objects.all())        # loads every row, then counts in Python
for lead in Lead.objects.iterator(chunk_size=50):
    pass                       # streams rows, no result cache
```

C. Nexora CRM

21.

```
# leads/seed.py
import random
from datetime import timedelta
from decimal import Decimal

from django.utils import timezone

from .models import Lead

FIRST = ["Aisha", "Omar", "Priya", "Daniel", "Mei", "Sara", "Ali", "Fatima",
"Ravi",
         "Elena", "Yusuf", "Noor", "Arjun", "Lina", "Hassan", "Chen",
"Maria", "Tariq"]
LAST = ["Khan", "Haddad", "Nair", "Ortiz", "Lin", "Al Mansoori", "Sharma",
"Petrova",
        "Rahman", "Iyer", "Saleh", "Costa", "Wong", "Farouk"]
STATUS_WEIGHTS = [40, 30, 15, 15]          # new, contacted, qualified, lost


def run(n=200, seed=42):
    rng = random.Random(seed)
    used = set(Lead.objects.values_list("phone", flat=True))
    batch = []
```

```
    while len(batch) < n:
        phone = f"9715{rng.randint(0, 99_999_999):08d}"
        if phone in used:
            continue
        used.add(phone)
        status = rng.choices(Lead.Status.values, weights=STATUS_WEIGHTS)[0]
        batch.append(Lead(
            name=f"{rng.choice(FIRST)} {rng.choice(LAST)}",
            phone=phone,
            status=status,
            source=rng.choice(Lead.Source.values),
            expected_deposit=Decimal(rng.randrange(500, 50_000, 500)),
            notes="No response after 3 attempts" if status ==
Lead.Status.LOST else "",
        ))
    Lead.objects.bulk_create(batch)       # few INSERTs; skips save() and
clean()

    # auto_now_add overwrote created_at on insert → backdate afterwards
    now = timezone.now()
    created = list(Lead.objects.filter(phone__in=[l.phone for l in batch]))
    for lead in created:
        lead.created_at = now - timedelta(days=rng.randint(0, 60),
hours=rng.randint(0, 23))
    Lead.objects.bulk_update(created, ["created_at"], batch_size=100)
    return len(created)
```

22.

```
from leads.seed import run
run()            # 200 — each call adds n more

for value, label in Lead.Status.choices:
    print(f"{label:<10} {Lead.objects.filter(status=value).count()}")
```

23–24, 26–30.

```
# leads/views.py (DB versions — replace the LEADS-based classes)
from datetime import timedelta

from django.shortcuts import get_object_or_404
from django.utils import timezone

from .models import Lead

LEAD_FIELDS = ("id", "name", "phone", "status", "source")
```

```
class LeadQueryMixin:                                              # 23
    def get_leads(self):
        qs = Lead.objects.filter(is_active=True)                   # 33:
archived leads hidden
        params = self.request.GET
        if source := params.get("source"):
            qs = qs.filter(source=source)
        if status := params.get("status"):
            qs = qs.filter(status=status)
        if q := params.get("q", "").strip():
            qs = qs.filter(name__icontains=q)
        return qs


class LeadDetailView(PageTitleMixin, TemplateView):                # 24
    template_name = "leads/lead_detail.html"
    page_title = "Lead"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        lead_id = self.kwargs["lead_id"]
        lead = get_object_or_404(Lead, pk=lead_id)
        recent = self.request.session.get("recent_leads", [])
        if lead_id in recent:
            recent.remove(lead_id)
        recent.insert(0, lead_id)
        self.request.session["recent_leads"] = recent[:5]
        context["lead"] = lead
        return context


class LeadListJSONView(LeadQueryMixin, JsonResponseMixin, View):   # 26
    def get(self, request, *args, **kwargs):
        return
self.render_to_json(list(self.get_leads().values(*LEAD_FIELDS)))


class LeadCSVView(LeadQueryMixin, View):                           # 29
    def get(self, request):
        response = HttpResponse(content_type="text/csv")
        response["Content-Disposition"] = 'attachment; filename="leads.csv"'
        writer = csv.writer(response)
        writer.writerow(LEAD_FIELDS)
        writer.writerows(self.get_leads().values_list(*LEAD_FIELDS))
```

```
        return response


@method_decorator(csrf_exempt, name="dispatch")
class LeadStatusUpdateView(JsonMethodNotAllowedMixin, View):     # 27
    def post(self, request, lead_id):
        lead = get_object_or_404(Lead, pk=lead_id)
        status = request.POST.get("status")
        if status not in Lead.Status.values:
            return json_error("invalid status")
        lead.status = status
        lead.save(update_fields=["status", "updated_at"])
        return JsonResponse({"id": lead.pk, "status": lead.status})


@method_decorator(csrf_exempt, name="dispatch")
class LeadDeleteView(View):                                      # 28
    def delete(self, request, lead_id):
        get_object_or_404(Lead, pk=lead_id).delete()
        return HttpResponse(status=204)


class StaleLeadsView(JsonResponseMixin, View):                   # 30
    def get(self, request):
        cutoff = timezone.now() - timedelta(days=7)
        qs = (Lead.objects
              .filter(status=Lead.Status.NEW, created_at__lt=cutoff)
              .order_by("created_at"))
        return self.render_to_json({
            "count": qs.count(),
            "results": list(qs.values("id", "name", "phone", "created_at")
[:50]),
        })


# switch these too so nothing points at stale LEADS ids
class LeadCountView(JsonResponseMixin, View):
    def get(self, request):
        return self.render_to_json({"total":
Lead.objects.filter(is_active=True).count()})


class RecentLeadsView(JsonResponseMixin, View):
    def get(self, request):
        ids = request.session.get("recent_leads", [])
        rows = {r["id"]: r for r in
```

```
Lead.objects.filter(pk__in=ids).values(*LEAD_FIELDS)}
        return self.render_to_json([rows[i] for i in ids if i in rows])   #
keep session order


class LatestLeadRedirectView(RedirectView):
    def get_redirect_url(self, *args, **kwargs):
        return Lead.objects.latest("created_at").get_absolute_url()

# leads/urls.py
path("stale/", views.StaleLeadsView.as_view(), name="stale"),
```

```
<!-- leads/templates/leads/lead_detail.html (24: change one line) -->
<dt>Source</dt><dd>{{ lead.get_source_display }}</dd>
```

25.

```
# core/views.py
from leads.models import Lead


class DashboardView(PageTitleMixin, TemplateView):
    template_name = "core/dashboard.html"
    page_title = "Dashboard"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        active = Lead.objects.filter(is_active=True)
        now = timezone.now()
        context.update(
            total=active.count(),
            counts={label: active.filter(status=value).count()     # 4
queries — Day 9 makes it 1
                    for value, label in Lead.Status.choices},
            all_leads=active,
            leads=active[:6],                                      # newest 6
            activity=[
                {"text": "Aisha Khan called back", "at": now -
timedelta(minutes=5)},
                {"text": "Priya Nair qualified", "at": now -
timedelta(hours=2)},
            ],
        )
        return context
```

```
# leads/templatetags/crm_filters.py — status_count now takes a queryset
@register.simple_tag
def status_count(leads, status):
    return leads.filter(status=status).count()
```

```
<!-- core/dashboard.html (changed line) — a sliced queryset can't be
filtered, so pass all_leads -->
{% status_count all_leads "new" as new_count %}
```

D. Challenge

31–34.

```
# 31: clone
clone = Lead.objects.first()
original_pk = clone.pk
clone.pk = None
clone._state.adding = True
clone.phone = "971507770020"
clone.save()
clone.pk != original_pk          # True

# 32: stale object
lead = Lead.objects.filter(status="new").first()
Lead.objects.filter(pk=lead.pk).update(status="qualified")
lead.status                      # 'new' — Python object didn't change
lead.refresh_from_db()
lead.status                      # 'qualified'

# 34
try:
    Lead.objects.get(status="new")
except Lead.MultipleObjectsReturned:
    print("More than one match — use filter() or a unique field")
```

```
# leads/models.py — 33
def archive(self):
    self.is_active = False
    self.save(update_fields=["is_active", "updated_at"])

# shell
Lead.objects.first().archive()
Lead.objects.filter(is_active=False).count()     # 1 — and it's gone from
/leads/
```

35–36.

```
from django.db import connection
from django.test.utils import CaptureQueriesContext

qs = Lead.objects.filter(status="new")
with CaptureQueriesContext(connection) as ctx:
    len(qs)
    len(qs)
print(len(ctx.captured_queries))     # 1 — second len() used the result cache

fresh = Lead.objects.filter(status="new")
with CaptureQueriesContext(connection) as ctx:
    fresh.count()
    fresh.count()
print(len(ctx.captured_queries))     # 2 — unevaluated queryset: count()
always queries
print(ctx.captured_queries[0]["sql"])

lazy = Lead.objects.all()            # 36
with CaptureQueriesContext(connection) as ctx:
    lazy[0]
    lazy[0]
print(len(ctx.captured_queries))     # 2 — each index is its own LIMIT 1
query
list(lazy)                           # evaluate once
with CaptureQueriesContext(connection) as ctx:
    lazy[0]
    lazy[0]
print(len(ctx.captured_queries))     # 0 — served from cache
```

37.

```
for lead in Lead.objects.raw(
    "SELECT * FROM crm_leads WHERE status = %s ORDER BY created_at DESC LIMIT
5", ["new"]
):
    print(lead.pk, lead.name)

with connection.cursor() as cursor:
    cursor.execute("SELECT status, COUNT(*) FROM crm_leads GROUP BY status
ORDER BY 2 DESC")
    print(cursor.fetchall())
# always pass values as params — never f-string user input into SQL
```

38–40.

```
# leads/tests.py
from django.test import TestCase
from .models import Lead


class LeadViewTests(TestCase):                # 38: was SimpleTestCase —
these views now query the DB
    @classmethod
    def setUpTestData(cls):
        cls.lead = Lead.objects.create(name="Badge Test",
phone="9715099999100")

    # ...keep the existing Day 1–3 tests...

    def test_lead_name_is_escaped(self):       # 38: replaces the
LEADS.append version
        Lead.objects.bulk_create([             # bulk_create skips save() →
name not title-cased
            Lead(name="<script>alert(1)</script>", phone="9715099999101"),
        ])
        response = self.client.get(reverse("leads:list"))
        self.assertNotContains(response, "<script>alert(1)</script>")
        self.assertContains(response,
"&lt;script&gt;alert(1)&lt;/script&gt;")

    def test_update_status_changes_db(self):   # 39
        response = self.client.post(
            reverse("leads:update_status", args=[self.lead.pk]), {"status":
"qualified"}
        )
        self.assertEqual(response.status_code, 200)
        self.lead.refresh_from_db()
        self.assertEqual(self.lead.status, Lead.Status.QUALIFIED)

    def test_delete_removes_row(self):         # 40
        lead = Lead.objects.create(name="Delete Me", phone="9715099999102")
        response = self.client.delete(reverse("leads:delete", args=
[lead.pk]))
        self.assertEqual(response.status_code, 204)
        self.assertFalse(Lead.objects.filter(pk=lead.pk).exists())


class ClassBasedViewTests(TestCase):          # 38: Day 4 class → TestCase
too
    ...
```

```
python manage.py test
git add -A && git commit -m "day 6"
```

Day 7 — Lookups, Q, F, ordering, values

Goal: express any lead question as a queryset — lookups, `Q` for OR/NOT, `F` for field-to-field maths — and package the CRM's everyday questions (search, stale, follow-up, burned, high value) into reusable query functions.

Do A and B in the shell.

A. Drills (1–10) — field lookups

- [ ] **1.** `exact` vs `iexact` on name.
- [ ] **2.** `contains` vs `icontains` — and why they behave the same on SQLite.
- [ ] **3.** `startswith`, `istartswith`, `endswith` on name and phone.
- [ ] **4.** `in` with a list of statuses, and with `Lead.PAID_SOURCES`.
- [ ] **5.** `gt`, `gte`, `lte` on `expected_deposit`; `range` between 5,000 and 20,000.
- [ ] **6.** Leads never contacted (`isnull`).
- [ ] **7.** Date parts: this year + month, Fridays (`week_day`), last 7 days with `__date__range`.
- [ ] **8.** `regex`: phones on the 050 prefix (`97150...`).
- [ ] **9.** Blank vs NULL: count leads with no email both ways; see which one works.
- [ ] **10.** Ordering: two fields, random (`?`), `last_contacted_at` desc with NULLs last.

B. Core (11–20) — Q, F, values

- [ ] **11.** OR: new leads OR referral leads.
- [ ] **12.** AND with `&`, NOT with `~`: new leads not from paid sources.
- [ ] **13.** Mix a `Q` with keyword filters in one `filter()` call (order matters).
- [ ] **14.** Build an OR `Q` dynamically from a list of sources (`reduce` or a loop with `|=`).
- [ ] **15.** Add 1 to `contact_attempts` for every contacted lead in one SQL statement.
- [ ] **16.** Leads updated more than a day after they were created (`F` + `timedelta`).
- [ ] **17.** `F()` on an instance: increment, save, see what the attribute holds, `refresh_from_db()`.
- [ ] **18.** `values()` with renamed keys via `F`.
- [ ] **19.** `values_list(..., named=True)` and read rows as named tuples.
- [ ] **20.** Database functions: names longer than 12 chars (`Length`), case-insensitive sort (`Lower`).

C. Nexora CRM (21–30) — `leads/queries.py`

- [ ] **21.** `search_q(term)` → a `Q` matching name or email (icontains) or phone digits (≥ 4 digits, ignoring `+` and spaces); `search_leads(term)` uses it.
- [ ] **22.** `stale_new_leads(days=7)`: new, older than N days, never contacted, oldest first.
- [ ] **23.** `follow_up_due(hours=48, max_attempts=3)`: contacted, last touch older than N hours, under the attempt cap.
- [ ] **24.** `burned_leads(max_attempts=3)`: still open (new/contacted) with ≥ 3 attempts.
- [ ] **25.** `high_value(threshold=20_000)`: deposit ≥ threshold, not lost, biggest first.
- [ ] **26.** `today_new()`: created today in Dubai time.
- [ ] **27.** Upgrade `LeadQueryMixin`: `q` uses `search_q`; add `finder`, `min_deposit`, `max_deposit`, `created_from`, `created_to`, and a whitelisted `sort`.
- [ ] **28.** `mark_burned_as_lost()`: one `update()` that sets status lost and appends `| auto-closed after 3 attempts` to notes.
- [ ] **29.** `LeadFinderView` at `leads/find/<finder>/` returning any finder as JSON (404 JSON for unknown names). Replaces Day 6's `StaleLeadsView`.
- [ ] **30.** Quick-filter chips above the lead table (All, Stale, Follow-up, Burned, High value, Sort by deposit) that keep the current search text.

D. Challenge (31–40)

- [ ] **31.** `get_object_or_404` with a `Q`: active leads only.
- [ ] **32.** Annotate a 2% commission estimate with `ExpressionWrapper`.
- [ ] **33.** Race-safe status change: `claim_new_lead(pk)` returns True only if *this* call moved it from new to contacted.
- [ ] **34.** Case-insensitive unique email (ignoring blanks) with a functional `UniqueConstraint(Lower("email"), condition=...)`.
- [ ] **35.** `explain()` a status filter and spot the Day 5 index in the plan.
- [ ] **36.** List every week that has leads with `datetimes("created_at", "week")`.
- [ ] **37.** Test `search_leads` matches by name, formatted phone and email, and not otherwise.
- [ ] **38.** Test `stale_new_leads` (backdate with `update()`).
- [ ] **39.** Test two `F()` increments give +2.
- [ ] **40.** Test `?sort=password` falls back to the default ordering.

Solutions

A. Drills

1–10.

```
# python manage.py shell
from datetime import timedelta
from django.db.models import F, Q
from django.db.models.functions import Length, Lower
from django.utils import timezone
from leads.models import Lead

Lead.objects.filter(name__exact="Aisha Khan")        # 1 — same as
name="Aisha Khan"
Lead.objects.filter(name__iexact="aisha khan")

Lead.objects.filter(name__contains="khan")           # 2 — SQLite's LIKE
ignores ASCII case,
Lead.objects.filter(name__icontains="khan")          #     so both match;
PostgreSQL won't

Lead.objects.filter(name__startswith="A")            # 3
Lead.objects.filter(name__istartswith="a")
Lead.objects.filter(phone__endswith="67")

Lead.objects.filter(status__in=["new", "contacted"]) # 4
Lead.objects.filter(source__in=Lead.PAID_SOURCES)

Lead.objects.filter(expected_deposit__gt=10_000)     # 5
Lead.objects.filter(expected_deposit__lte=1_000)
Lead.objects.filter(expected_deposit__range=(5_000, 20_000))   # inclusive
both ends

Lead.objects.filter(last_contacted_at__isnull=True)  # 6

now = timezone.localtime()                           # 7
Lead.objects.filter(created_at__year=now.year, created_at__month=now.month)
Lead.objects.filter(created_at__week_day=6)          # 1 = Sunday … 7 =
Saturday → 6 = Friday
today = timezone.localdate()
Lead.objects.filter(created_at__date__range=(today - timedelta(days=6),
today))

Lead.objects.filter(phone__regex=r"^97150")          # 8
```

```
Lead.objects.filter(email="").count()                # 9 — blank strings: the
real answer
Lead.objects.filter(email__isnull=True).count()      # 0 — EmailField has no
null=True
Lead.objects.exclude(email="")

Lead.objects.order_by("status", "-expected_deposit")                    #
10
Lead.objects.order_by("?")[:5]                       # random — slow on big
tables
Lead.objects.order_by(F("last_contacted_at").desc(nulls_last=True))
```

B. Core

11–20.

```
Lead.objects.filter(Q(status="new") | Q(source="referral"))                #
11

Lead.objects.filter(Q(status="contacted") & Q(contact_attempts__gte=2))    #
12
Lead.objects.filter(Q(status="new") & ~Q(source__in=Lead.PAID_SOURCES))

Lead.objects.filter(Q(status="new") | Q(status="contacted"), is_active=True)
# 13: Q first, kwargs after

import operator
# 14
from functools import reduce
sources = ["google", "facebook"]
cond = reduce(operator.or_, (Q(source=s) for s in sources))
Lead.objects.filter(cond)

cond = Q()                       # same thing with a loop; empty Q() matches
everything
for s in sources:
    cond |= Q(source=s)

Lead.objects.filter(status="contacted").update(
# 15
    contact_attempts=F("contact_attempts") + 1,     # UPDATE ... SET x = x +
1 — no race
)

Lead.objects.filter(updated_at__gt=F("created_at") + timedelta(days=1))
# 16
```

```
lead = Lead.objects.first()
# 17
lead.contact_attempts = F("contact_attempts") + 1
lead.save(update_fields=["contact_attempts", "updated_at"])
lead.contact_attempts            # <CombinedExpression ...> — not a number
yet
lead.refresh_from_db()
lead.contact_attempts            # real value

Lead.objects.values(lead_name=F("name"), lead_status=F("status"))[:3]       #
18

for row in Lead.objects.values_list("name", "status", named=True)[:3]:      #
19
    print(row.name, row.status)

Lead.objects.annotate(name_len=Length("name")).filter(name_len__gt=12)      #
20
Lead.objects.order_by(Lower("name"))
```

C. Nexora CRM

The seed has no call data until Day 8. To try 23–24 now, fake some:

```
# update() can't run on a slice, so collect pks first
ids = list(Lead.objects.filter(status="contacted").values_list("pk",
flat=True)[:20])
Lead.objects.filter(pk__in=ids).update(
    contact_attempts=3, last_contacted_at=timezone.now() - timedelta(days=3),
)
```

21–26, 28.

```
# leads/queries.py
import re
from datetime import timedelta

from django.db.models import F, Q, TextField, Value
from django.db.models.functions import Concat
from django.utils import timezone

from .models import Lead

OPEN_STATUSES = [Lead.Status.NEW, Lead.Status.CONTACTED]
```

```
def search_q(term):                                                    # 21
    term = term.strip()
    if not term:
        return Q()
    q = Q(name__icontains=term) | Q(email__icontains=term)
    digits = re.sub(r"\D", "", term)          # "+971 50 999" → "97150999"
    if len(digits) >= 4:
        q |= Q(phone__contains=digits)
    return q


def search_leads(term, qs=None):
    qs = Lead.objects.all() if qs is None else qs
    return qs.filter(search_q(term))


def stale_new_leads(days=7):                                           # 22
    cutoff = timezone.now() - timedelta(days=days)
    return Lead.objects.filter(
        status=Lead.Status.NEW, created_at__lt=cutoff,
last_contacted_at__isnull=True,
    ).order_by("created_at")


def follow_up_due(hours=48, max_attempts=3):                           # 23
    cutoff = timezone.now() - timedelta(hours=hours)
    return Lead.objects.filter(
        status=Lead.Status.CONTACTED,
        last_contacted_at__lt=cutoff,
        contact_attempts__lt=max_attempts,
    ).order_by("last_contacted_at")


def burned_leads(max_attempts=3):                                      # 24
    return Lead.objects.filter(status__in=OPEN_STATUSES,
contact_attempts__gte=max_attempts)


def high_value(threshold=20_000):                                      # 25
    return (Lead.objects
            .filter(expected_deposit__gte=threshold)
            .exclude(status=Lead.Status.LOST)
            .order_by("-expected_deposit"))
```

```
def today_new():                                                    # 26
    return Lead.objects.filter(created_at__date=timezone.localdate())   #
__date uses TIME_ZONE


def mark_burned_as_lost():                                          # 28
    return burned_leads().update(
        status=Lead.Status.LOST,
        notes=Concat(F("notes"), Value(" | auto-closed after 3 attempts"),
output_field=TextField()),
        updated_at=timezone.now(),
    )


FINDERS = {
    "stale": stale_new_leads,
    "follow_up": follow_up_due,
    "burned": burned_leads,
    "high_value": high_value,
    "today": today_new,
}
```

`Concat` needs `output_field` when mixing a TextField with a plain string value (28).

27.

```
# leads/views.py
from decimal import Decimal, InvalidOperation

from django.utils.dateparse import parse_date

from .queries import FINDERS, search_q

SORTABLE = {"name", "created_at", "expected_deposit", "status",
"contact_attempts"}


class LeadQueryMixin:
    default_sort = "-created_at"

    def get_leads(self):
        p = self.request.GET
        qs = Lead.objects.filter(is_active=True)
        if source := p.get("source"):
            qs = qs.filter(source=source)
        if status := p.get("status"):
            qs = qs.filter(status=status)
```

```
        if q := p.get("q", "").strip():
            qs = qs.filter(search_q(q))
        if finder := FINDERS.get(p.get("finder", "")):
            qs = qs.filter(pk__in=finder().values("pk"))          # subquery
        try:
            if min_dep := p.get("min_deposit"):
                qs = qs.filter(expected_deposit__gte=Decimal(min_dep))
            if max_dep := p.get("max_deposit"):
                qs = qs.filter(expected_deposit__lte=Decimal(max_dep))
        except InvalidOperation:
            pass                                                  # ignore
junk numbers
        if created_from := parse_date(p.get("created_from", "")):  # "2026-
10-01"
            qs = qs.filter(created_at__date__gte=created_from)
        if created_to := parse_date(p.get("created_to", "")):
            qs = qs.filter(created_at__date__lte=created_to)
        sort = p.get("sort", self.default_sort)
        if sort.lstrip("-") not in SORTABLE:                      # never
order by raw user input
            sort = self.default_sort
        return qs.order_by(sort)
```

29.

```
# leads/views.py
class LeadFinderView(JsonResponseMixin, View):
    def get(self, request, finder):
        fn = FINDERS.get(finder)
        if fn is None:
            return json_error(f"unknown finder: {finder}", status=404,
available=sorted(FINDERS))
        qs = fn()
        return self.render_to_json({
            "finder": finder,
            "count": qs.count(),
            "results": list(qs.values(*LEAD_FIELDS)[:100]),
        })

# leads/urls.py (replace the stale/ route)
path("find/<slug:finder>/", views.LeadFinderView.as_view(), name="finder"),
```

30.

```
<!-- leads/lead_list.html — above the table -->
{% with q=request.GET.q|urlencode %}
<nav class="chips">
  <a href="?q={{ q }}">All</a>
  <a href="?q={{ q }}&finder=stale">Stale</a>
  <a href="?q={{ q }}&finder=follow_up">Follow-up due</a>
  <a href="?q={{ q }}&finder=burned">3+ attempts</a>
  <a href="?q={{ q }}&finder=high_value">High value</a>
  <a href="?q={{ q }}&sort=-expected_deposit">Sort by deposit</a>
</nav>
{% endwith %}

<!-- inside the search form, so searching keeps the active chip -->
<input type="hidden" name="finder" value="{{ request.GET.finder }}">
```

```
/* static/css/crm.css */
.chips { display: flex; gap: .5rem; margin: 1rem 0; flex-wrap: wrap; }
.chips a { padding: .25rem .75rem; border: 1px solid #cbd5e1; border-radius:
999px; text-decoration: none; color: inherit; }
```

D. Challenge

31–33.

```
from decimal import Decimal
from django.db.models import DecimalField, ExpressionWrapper
from django.shortcuts import get_object_or_404

lead = get_object_or_404(Lead, Q(pk=1) & Q(is_active=True))                #
31

Lead.objects.annotate(
# 32
    commission_estimate=ExpressionWrapper(
        F("expected_deposit") * Decimal("0.02"),
        output_field=DecimalField(max_digits=12, decimal_places=2),
    )
).values("name", "expected_deposit", "commission_estimate")[:5]


# leads/queries.py
# 33
def claim_new_lead(lead_id):
    changed = (Lead.objects
               .filter(pk=lead_id, status=Lead.Status.NEW)    # condition
```

```
checked in the same UPDATE
                .update(status=Lead.Status.CONTACTED,
updated_at=timezone.now()))
    return changed == 1          # False → someone else already moved it
```

34.

```
# leads/models.py — Lead.Meta.constraints
from django.db.models.functions import Lower

models.UniqueConstraint(
    Lower("email"),
    name="lead_unique_email_ci",
    condition=~models.Q(email=""),      # blanks don't count as duplicates
),
```

```
python manage.py makemigrations leads && python manage.py migrate
```

35–36.

```
print(Lead.objects.filter(status="new").order_by("-created_at").explain())
# ... SEARCH crm_leads USING INDEX lead_status_created_idx (status=?)

Lead.objects.datetimes("created_at", "week")      # aware datetimes, one per
week with leads
```

37–40.

```
# leads/tests.py
from datetime import timedelta

from django.db.models import F
from django.utils import timezone

from .queries import search_leads, stale_new_leads


class LeadQueryTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.a = Lead.objects.create(name="Search Target",
phone="9715099900001", email="target@nexora.test")
        cls.b = Lead.objects.create(name="Other Person",
phone="9715099900002")

    def test_search(self):                                          # 37
```

```
        self.assertIn(self.a, search_leads("search targ"))
        self.assertIn(self.a, search_leads("+971 50 999 0001"))
        self.assertIn(self.a, search_leads("TARGET@"))
        self.assertNotIn(self.b, search_leads("target"))

    def test_stale_new_leads(self):                                     # 38
        old = timezone.now() - timedelta(days=10)
        Lead.objects.filter(pk=self.a.pk).update(created_at=old)       #
update() skips auto_now_add
        Lead.objects.filter(pk=self.b.pk).update(created_at=old,
last_contacted_at=timezone.now())
        stale = stale_new_leads(days=7)
        self.assertIn(self.a, stale)
        self.assertNotIn(self.b, stale)

    def test_f_increments(self):                                        # 39
        qs = Lead.objects.filter(pk=self.a.pk)
        qs.update(contact_attempts=F("contact_attempts") + 1)
        qs.update(contact_attempts=F("contact_attempts") + 1)
        self.a.refresh_from_db()
        self.assertEqual(self.a.contact_attempts, 2)

    def test_unknown_sort_falls_back(self):                            # 40
        response = self.client.get(reverse("leads:list"), {"sort":
"password"})
        self.assertEqual(response.status_code, 200)
        expected = list(Lead.objects.filter(is_active=True).order_by("-
created_at"))
        self.assertEqual(list(response.context["leads"]), expected)
```

Day 8 — Relationships

Goal: the full Nexora data model — agents as a custom user, leads assigned to agents, call logs, brokers in a referral tree, clients converted from leads, trading accounts and deposits — seeded with realistic data.

Why the custom user lands today: Django's user model must be swapped before any table points at users. Today is the first FK to a user, so you reset the database once (exercise 1) and never again.

A. Drills (1–10) — users, FK, M2M

- [ ] **1.** `accounts` app with `CustomUser(AbstractUser)`; set `AUTH_USER_MODEL`; delete `db.sqlite3`, migrate, create a superuser, re-run the Day 6 seed.
- [ ] **2.** On `CustomUser`: `role` (agent, team_lead, manager, admin) and `team_lead` (self-FK, related name `team_members`).
- [ ] **3.** `CustomUserCreationForm` / `CustomUserChangeForm` and a `UserAdmin` subclass showing role and team lead.
- [ ] **4.** `Lead.assigned_to`: FK to the user model, nullable, `SET_NULL`, related name `leads`.
- [ ] **5.** In the shell: create agent `sara`, assign a lead, read it forward (`lead.assigned_to`) and backward (`sara.leads.all()`).
- [ ] **6.** `Lead.broker`: FK to `Broker`, nullable, `SET_NULL`, related name `leads`.
- [ ] **7.** `Tag` model (slug name, colour) and `Lead.tags` M2M; practise `add`, `remove`, `set`, `clear`.
- [ ] **8.** Reverse M2M (`tag.leads.all()`) and filtering leads by tag name(s) without duplicates.
- [ ] **9.** Filter across FKs: by agent username, agent role, broker tier, and unassigned leads.
- [ ] **10.** Reverse filter: agents who own at least one qualified lead (no duplicates).

B. Core (11–20) — the CRM data model

- [ ] **11.** `CallLog`: lead (CASCADE, `calls`), agent (PROTECT, `calls`), started_at (default now), duration_seconds, outcome (answered/no_answer/busy/voicemail), notes; index on lead + started_at.
- [ ] **12.** In the shell: delete a lead and watch its calls go (CASCADE); delete an agent with calls and catch `ProtectedError`.
- [ ] **13.** `Client.lead`: `OneToOneField` to `Lead` (`SET_NULL`, related name `client`); read both directions; check safely with `hasattr`.

- [ ] **14.** `Client.broker`: FK to `Broker`, related name `clients`.
- [ ] **15.** `TradingAccount`: client (CASCADE, `accounts`), unique 7-digit number, platform (MT4/MT5), currency (USD/AED), leverage, balance, opened_at.
- [ ] **16.** `Deposit`: account (PROTECT, `deposits`), amount (> 0 constraint), method (card/bank/crypto), created_at.
- [ ] **17.** `Broker.parent`: self-FK, related name `sub_brokers`. Build a 3-level tree in the shell.
- [ ] **18.** `Broker.ancestors()` (walk up) and `Broker.descendants()` (recursive walk down).
- [ ] **19.** `limit_choices_to` on `Lead.assigned_to` so only agents appear in forms and admin.
- [ ] **20.** Register `Tag`, `CallLog`, `TradingAccount`, `Deposit` in the admin; `CallLog` list shows lead, agent, outcome, duration, time.

C. Nexora CRM (21–30) — seed and pages

- [ ] **21.** `seed_people()`: 1 manager, 2 team leads, 8 agents (each under a team lead), 3 tier-1 brokers with 2 sub-brokers each, 5 tags.
- [ ] **22.** `assign_leads()`: round-robin every lead to an agent with `bulk_update`.
- [ ] **23.** Attach a broker to ~30% of leads; give leads 0–2 random tags by bulk-creating rows in the M2M through table.
- [ ] **24.** `seed_calls()`: 0–5 calls per assigned lead (weighted outcomes, talk time only when answered); update `contact_attempts` and `last_contacted_at`.
- [ ] **25.** `convert_qualified()`: each qualified lead becomes a `Client` (same broker) with 1–2 trading accounts and 1–4 deposits each. One transaction.
- [ ] **26.** Lead detail shows agent, broker, tags and a call-history table.
- [ ] **27.** Agent page `/team/agents/<pk>/`: the agent, their team lead, leads grouped by status.
- [ ] **28.** Broker page `/brokers/<code>/`: breadcrumb of ancestors, sub-broker links, client count.
- [ ] **29.** Client detail page from the DB: KYC, broker, source lead link, each account with its deposits.
- [ ] **30.** Lead list: `?agent=<id>` and `?tag=<name>` filters and an Agent column.

D. Challenge (31–40)

- [ ] **31.** M2M with extra data: `Campaign` ↔ `Lead` through `CampaignEnrollment` (enrolled_at, outcome, unique pair); add with `through_defaults`.
- [ ] **32.** Symmetrical self M2M: `Client.related_clients` (households). Add A→B and see B→A.
- [ ] **33.** Replace `PROTECT` on `CallLog.agent` with `SET(get_sentinel_agent)` so calls survive under a `deleted-agent` user.
- [ ] **34.** Multi-valued filters: one `filter()` with two `calls__` conditions vs two chained `filter()` calls — compare counts and explain.
- [ ] **35.** `Broker.descendants_sql()` with a `WITH RECURSIVE` raw query (one query for the whole tree).
- [ ] **36.** Tag usage counts by querying the M2M through model directly.
- [ ] **37.** Test: deleting a user leaves their leads unassigned.
- [ ] **38.** Test: deleting an agent moves their calls to the sentinel user.
- [ ] **39.** Test: deleting a trading account that has deposits raises `ProtectedError`.
- [ ] **40.** Test: `descendants()` and `ancestors()` on a 3-level tree.

Solutions

A. Drills

1–2.

```
python manage.py startapp accounts
```

```
# accounts/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models


class CustomUser(AbstractUser):
    class Role(models.TextChoices):
        AGENT = "agent", "Agent"
        TEAM_LEAD = "team_lead", "Team lead"
        MANAGER = "manager", "Manager"
        ADMIN = "admin", "Admin"

    role = models.CharField(max_length=20, choices=Role.choices,
default=Role.AGENT)
    team_lead = models.ForeignKey(
```

```
        "self", null=True, blank=True, on_delete=models.SET_NULL,
        related_name="team_members", limit_choices_to={"role": "team_lead"},
    )
    phone_ext = models.CharField(max_length=6, blank=True)

    def __str__(self):
        return self.get_full_name() or self.username
```

```
# django_project/settings.py
INSTALLED_APPS = [..., "accounts.apps.AccountsConfig", ...]
AUTH_USER_MODEL = "accounts.CustomUser"
```

```
# stop runserver first
del db.sqlite3                     # PowerShell: Remove-Item db.sqlite3 ·
macOS: rm db.sqlite3
python manage.py makemigrations accounts
python manage.py migrate           # rebuilds every table; the Day 5 data
migration re-adds 5 leads
python manage.py createsuperuser
python manage.py shell -c "from leads.seed import run; print(run())"
```

The leads/clients/brokers migration files stay: none of them referenced `auth.User`, so only the database had to go.

3.

```
# accounts/forms.py
from django.contrib.auth.forms import UserChangeForm, UserCreationForm
from .models import CustomUser


class CustomUserCreationForm(UserCreationForm):
    class Meta(UserCreationForm.Meta):
        model = CustomUser
        fields = UserCreationForm.Meta.fields + ("email", "role",
"team_lead")


class CustomUserChangeForm(UserChangeForm):
    class Meta(UserChangeForm.Meta):
        model = CustomUser
        fields = "__all__"
```

```
# accounts/admin.py
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin
from .forms import CustomUserChangeForm, CustomUserCreationForm
from .models import CustomUser


@admin.register(CustomUser)
class CustomUserAdmin(UserAdmin):
    add_form = CustomUserCreationForm
    form = CustomUserChangeForm
    model = CustomUser
    list_display = ["username", "email", "role", "team_lead", "is_staff"]
    list_filter = UserAdmin.list_filter + ("role",)
    fieldsets = UserAdmin.fieldsets + (("CRM", {"fields": ("role",
"team_lead", "phone_ext")}),)
    add_fieldsets = UserAdmin.add_fieldsets + (("CRM", {"fields": ("email",
"role", "team_lead")}),)
```

4, 6–7, 19. New `Lead` fields + `Tag`.

```
# leads/models.py
from django.conf import settings


class Tag(models.Model):
# 7 (define above Lead)
    name = models.SlugField(max_length=30, unique=True)
    color = models.CharField(max_length=7, default="#64748b")

    class Meta:
        ordering = ["name"]

    def __str__(self):
        return self.name


class Lead(models.Model):
    ...
    assigned_to = models.ForeignKey(
# 4, 19
        settings.AUTH_USER_MODEL,          # never import CustomUser directly
here
        null=True, blank=True, on_delete=models.SET_NULL,
        related_name="leads", limit_choices_to={"role": "agent"},
    )
```

```
    broker = models.ForeignKey(
# 6
        "brokers.Broker", null=True, blank=True, on_delete=models.SET_NULL,
related_name="leads",
    )
    tags = models.ManyToManyField(Tag, blank=True, related_name="leads")
# 7
```

5, 7–10.

```
# python manage.py shell
from accounts.models import CustomUser
from leads.models import Lead, Tag

sara = CustomUser.objects.create_user("sara", password="nexora123",
role="agent")   # 5
lead = Lead.objects.first()
lead.assigned_to = sara
lead.save(update_fields=["assigned_to", "updated_at"])
lead.assigned_to.username          # forward
sara.leads.all()                   # reverse, via related_name

vip, _ = Tag.objects.get_or_create(name="vip")                                  #
7
callback, _ = Tag.objects.get_or_create(name="callback")
lead.tags.add(vip, callback)       # M2M changes save immediately — no
lead.save()
lead.tags.remove(callback)
lead.tags.set([vip, callback])
lead.tags.clear()
lead.tags.add(vip)

vip.leads.all()
# 8
Lead.objects.filter(tags__name="vip")
Lead.objects.filter(tags__name__in=["vip", "callback"]).distinct()   # a lead
with both tags would repeat

Lead.objects.filter(assigned_to__username="sara")                               #
9
Lead.objects.filter(assigned_to__role=CustomUser.Role.AGENT)
Lead.objects.filter(broker__tier=1)
Lead.objects.filter(assigned_to__isnull=True)

CustomUser.objects.filter(leads__status="qualified").distinct()             #
10
```

B. Core

11.

```
# leads/models.py (below Lead)
from django.utils import timezone


class CallLog(models.Model):
    class Outcome(models.TextChoices):
        ANSWERED = "answered", "Answered"
        NO_ANSWER = "no_answer", "No answer"
        BUSY = "busy", "Busy"
        VOICEMAIL = "voicemail", "Voicemail"

    lead = models.ForeignKey(Lead, on_delete=models.CASCADE,
related_name="calls")
    agent = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.PROTECT, related_name="calls")  # 33 changes this
    started_at = models.DateTimeField(default=timezone.now)     # default,
not auto_now_add → seedable
    duration_seconds = models.PositiveIntegerField(default=0)
    outcome = models.CharField(max_length=20, choices=Outcome.choices)
    notes = models.TextField(blank=True)

    class Meta:
        ordering = ["-started_at"]
        indexes = [models.Index(fields=["lead", "started_at"],
name="call_lead_started_idx")]

    def __str__(self):
        return f"{self.lead.name} · {self.get_outcome_display()} ·
{self.duration_seconds}s"
```

12.

```
from django.db.models import ProtectedError
from leads.models import CallLog

temp = Lead.objects.create(name="Temp", phone="971507770030")
CallLog.objects.create(lead=temp, agent=sara, outcome="answered",
duration_seconds=60)
temp.delete()                       # (2, {'leads.CallLog': 1, 'leads.Lead':
1}) — CASCADE

temp2 = Lead.objects.create(name="Temp Two", phone="971507770031")
CallLog.objects.create(lead=temp2, agent=sara, outcome="busy")
try:
    sara.delete()
except ProtectedError as e:
    print("blocked by:", e.protected_objects)
```

13–16.

```
# clients/models.py
from django.utils import timezone


class Client(models.Model):
    ...
    lead = models.OneToOneField(
# 13
        "leads.Lead", null=True, blank=True, on_delete=models.SET_NULL,
related_name="client",
    )
    broker = models.ForeignKey(
# 14
        "brokers.Broker", null=True, blank=True, on_delete=models.SET_NULL,
related_name="clients",
    )


class TradingAccount(models.Model):
# 15
    class Platform(models.TextChoices):
        MT4 = "mt4", "MetaTrader 4"
        MT5 = "mt5", "MetaTrader 5"

    class Currency(models.TextChoices):
        USD = "USD", "USD"
        AED = "AED", "AED"
```

```
    client = models.ForeignKey(Client, on_delete=models.CASCADE,
related_name="accounts")
    number = models.CharField(max_length=7, unique=True, validators=
[RegexValidator(r"^\d{7}$")])
    platform = models.CharField(max_length=3, choices=Platform.choices,
default=Platform.MT5)
    currency = models.CharField(max_length=3, choices=Currency.choices,
default=Currency.USD)
    leverage = models.PositiveSmallIntegerField(default=100)
    balance = models.DecimalField(max_digits=14, decimal_places=2, default=0)
    opened_at = models.DateTimeField(default=timezone.now)

    def __str__(self):
        return f"{self.number} ({self.get_platform_display()})"


class Deposit(models.Model):
# 16
    class Method(models.TextChoices):
        CARD = "card", "Card"
        BANK = "bank", "Bank transfer"
        CRYPTO = "crypto", "Crypto"

    account = models.ForeignKey(TradingAccount, on_delete=models.PROTECT,
related_name="deposits")
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    method = models.CharField(max_length=10, choices=Method.choices)
    created_at = models.DateTimeField(default=timezone.now)

    class Meta:
        ordering = ["-created_at"]
        constraints = [
            models.CheckConstraint(condition=models.Q(amount__gt=0),
name="deposit_amount_positive"),
        ]
```

Deleting a client cascades to its accounts, then hits `PROTECT` on deposits → clients with money history can't be deleted.

```
# 13 in the shell
client.lead                 # forward
lead.client                 # reverse — raises
Lead.client.RelatedObjectDoesNotExist if none
hasattr(lead, "client")     # safe check (the exception subclasses
AttributeError)
```

17–18.

```
# brokers/models.py — add to Broker
parent = models.ForeignKey(
    "self", null=True, blank=True, on_delete=models.SET_NULL,
related_name="sub_brokers",
)

def ancestors(self):                    # nearest first
    chain, node = [], self.parent
    while node is not None:
        chain.append(node)
        node = node.parent
    return chain

def descendants(self):                  # one query per node — 35 does it in
one
    result = []
    for child in self.sub_brokers.all():
        result.append(child)
        result.extend(child.descendants())
    return result
```

```
# shell
from brokers.models import Broker

top = Broker.objects.create(code="BRF1000", name="Gulf Partners", tier=1)
mid = Broker.objects.create(code="BRF1100", name="Marina Desk", tier=2,
parent=top)
leaf = Broker.objects.create(code="BRF1110", name="JLT Intro", tier=3,
parent=mid)

top.sub_brokers.all()                       # direct children
Broker.objects.filter(parent__parent=top)   # grandchildren
leaf.ancestors()                            # [mid, top]
top.descendants()                           # [mid, leaf]
```

20.

```
# leads/admin.py
from django.contrib import admin
from .models import CallLog, Lead, Tag

admin.site.register(Lead)
admin.site.register(Tag)


@admin.register(CallLog)
class CallLogAdmin(admin.ModelAdmin):
    list_display = ["lead", "agent", "outcome", "duration_seconds",
"started_at"]

# clients/admin.py
from .models import Client, Deposit, TradingAccount

admin.site.register(Client)
admin.site.register(TradingAccount)
admin.site.register(Deposit)
```

C. Nexora CRM

21–25. Add to `leads/seed.py`; run `run_all()` once after the reset.

```
# leads/seed.py (add)
from django.contrib.auth import get_user_model
from django.db import transaction

from brokers.models import Broker
from clients.models import Client, Deposit, TradingAccount
from .models import CallLog, Lead, Tag

User = get_user_model()
TAGS = ["vip", "callback", "arabic-speaker", "high-risk", "whale"]


def _user(username, **fields):
    user, created = User.objects.get_or_create(username=username,
defaults=fields)
    if created:
        user.set_password("nexora123")
        user.save(update_fields=["password"])
    return user
```

```
def seed_people():
# 21
    _user("manager", role="manager", first_name="Rania")
    team_leads = [_user(f"tl{i}", role="team_lead", first_name=name)
                  for i, name in enumerate(["Karim", "Leila"], start=1)]
    agents = [_user(f"agent{i}", role="agent", first_name=f"Agent{i}",
team_lead=team_leads[i % 2])
              for i in range(1, 9)]
    brokers = []
    for i in range(1, 4):
        top, _ = Broker.objects.get_or_create(
            code=f"BRF{i}000", defaults={"name": f"Partner {i}", "tier": 1,
"commission_rate": 30})
        brokers.append(top)
        for j in range(1, 3):
            sub, _ = Broker.objects.get_or_create(
                code=f"BRF{i}{j}00",
                defaults={"name": f"Partner {i}.{j}", "tier": 2, "parent":
top, "commission_rate": 20})
            brokers.append(sub)
    tags = [Tag.objects.get_or_create(name=t)[0] for t in TAGS]
    return agents, brokers, tags


def assign_leads(agents):
# 22
    leads = list(Lead.objects.order_by("pk"))
    for i, lead in enumerate(leads):
        lead.assigned_to = agents[i % len(agents)]
    Lead.objects.bulk_update(leads, ["assigned_to"], batch_size=200)


def attach_brokers_and_tags(brokers, tags, seed=7):                            #
23
    rng = random.Random(seed)
    leads = list(Lead.objects.all())
    for lead in leads:
        lead.broker = rng.choice(brokers) if rng.random() < 0.3 else None
    Lead.objects.bulk_update(leads, ["broker"], batch_size=200)

    Through = Lead.tags.through                          # the auto-created
join table
    rows = [Through(lead_id=lead.pk, tag_id=tag.pk)
            for lead in leads
            for tag in rng.sample(tags, k=rng.randint(0, 2))]
    Through.objects.bulk_create(rows, ignore_conflicts=True)
```

```
def seed_calls(seed=11):
# 24
    rng = random.Random(seed)
    now = timezone.now()
    calls, touched = [], []
    for lead in Lead.objects.exclude(assigned_to=None):
        n = rng.randint(0, 5)
        if n == 0:
            continue
        times = sorted(
            min(lead.created_at + timedelta(hours=rng.randint(1, 24 * 20)),
now) for _ in range(n)
        )
        for t in times:
            outcome = rng.choices(CallLog.Outcome.values, weights=[45, 35,
10, 10])[0]
            calls.append(CallLog(
                lead=lead, agent_id=lead.assigned_to_id, started_at=t,
outcome=outcome,
                duration_seconds=rng.randint(30, 900) if outcome ==
"answered" else 0,
            ))
        lead.contact_attempts = n
        lead.last_contacted_at = times[-1]
        touched.append(lead)
    CallLog.objects.bulk_create(calls, batch_size=500)
    Lead.objects.bulk_update(touched, ["contact_attempts",
"last_contacted_at"], batch_size=200)


@transaction.atomic
# 25
def convert_qualified(seed=13):
    rng = random.Random(seed)
    next_number = 2_000_000 + TradingAccount.objects.count()
    for lead in Lead.objects.filter(status=Lead.Status.QUALIFIED,
client__isnull=True):
        client = Client.objects.create(
            lead=lead, broker=lead.broker, name=lead.name,
            email=lead.email or f"client{lead.pk}@example.com",
            account_number=f"{1_000_000 + lead.pk:07d}",
        )
        for _ in range(rng.randint(1, 2)):
            next_number += 1
```

```
            account = TradingAccount.objects.create(client=client,
number=str(next_number))
            for _ in range(rng.randint(1, 4)):
                Deposit.objects.create(
                    account=account,
                    amount=Decimal(rng.randrange(250, 20_000, 250)),
                    method=rng.choice(Deposit.Method.values),
                    created_at=timezone.now() - timedelta(days=rng.randint(0,
60)),
                )


def run_all():
    if not Lead.objects.exists():
        run(200)
    agents, brokers, tags = seed_people()
    assign_leads(agents)
    attach_brokers_and_tags(brokers, tags)
    seed_calls()          # not idempotent — run once per fresh database
    convert_qualified()
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py shell -c "from leads.seed import run_all; run_all()"
```

26.

```
<!-- leads/lead_detail.html — add inside <dl> -->
<dt>Agent</dt><dd>{{ lead.assigned_to|default:"Unassigned" }}</dd>
<dt>Broker</dt><dd>{% if lead.broker %}<a href="{% url 'brokers:detail'
lead.broker.code %}">{{ lead.broker }}</a>{% else %}—{% endif %}</dd>
<dt>Tags</dt>
<dd>{% for tag in lead.tags.all %}<span class="badge" style="background:{{
tag.color }};color:#fff">{{ tag }}</span> {% empty %}—{% endfor %}</dd>

<!-- below </dl> -->
<h2>Calls ({{ lead.calls.count }})</h2>
<table class="table">
  <tr><th>When</th><th>Agent</th><th>Outcome</th><th>Talk time</th></tr>
  {% for call in lead.calls.all %}
    <tr>
      <td>{{ call.started_at|date:"d M H:i" }}</td>
      <td>{{ call.agent }}</td>
      <td>{{ call.get_outcome_display }}</td>
      <td>{{ call.duration_seconds }}s</td>
    </tr>
```

```
  {% empty %}
    <tr><td colspan="4">No calls yet.</td></tr>
  {% endfor %}
</table>
```

27.

```
# accounts/views.py
from django.contrib.auth import get_user_model
from django.shortcuts import get_object_or_404
from django.views.generic import TemplateView

from core.mixins import PageTitleMixin
from leads.models import Lead


class AgentDetailView(PageTitleMixin, TemplateView):
    template_name = "accounts/agent_detail.html"
    page_title = "Agent"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        agent = get_object_or_404(get_user_model(), pk=self.kwargs["pk"],
role="agent")
        context["agent"] = agent
        context["by_status"] = {label: agent.leads.filter(status=value)
                                for value, label in Lead.Status.choices}
        return context

# accounts/urls.py
from django.urls import path
from . import views

app_name = "accounts"
urlpatterns = [
    path("agents/<int:pk>/", views.AgentDetailView.as_view(),
name="agent_detail"),
]

# django_project/urls.py
path("team/", include("accounts.urls")),
```

```
<!-- accounts/templates/accounts/agent_detail.html -->
{% extends "base.html" %}
{% block content %}
<h1>{{ agent }}</h1>
<p>Team lead: {{ agent.team_lead|default:"—" }} · {{ agent.leads.count }}
leads</p>
{% for label, leads in by_status.items %}
  <h2>{{ label }} ({{ leads|length }})</h2>
  <ul>{% for lead in leads|slice:":10" %}<li><a href="{{
lead.get_absolute_url }}">{{ lead.name }}</a></li>{% endfor %}</ul>
{% endfor %}
{% endblock content %}
```

28.

```
# brokers/views.py
from django.shortcuts import get_object_or_404
from django.views.generic import TemplateView
from core.mixins import PageTitleMixin
from .models import Broker


class BrokerDetailView(PageTitleMixin, TemplateView):
    template_name = "brokers/broker_detail.html"
    page_title = "Broker"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        broker = get_object_or_404(Broker, code=self.kwargs["code"])
        context.update(
            broker=broker,
            ancestors=broker.ancestors()[::-1],       # top → parent
            children=broker.sub_brokers.all(),
            client_count=broker.clients.count(),
        )
        return context

# brokers/urls.py
app_name = "brokers"
urlpatterns = [path("<str:code>/", views.BrokerDetailView.as_view(),
name="detail")]

# django_project/urls.py
path("brokers/", include("brokers.urls")),
```

```
<!-- brokers/templates/brokers/broker_detail.html -->
{% extends "base.html" %}
{% block content %}
<p>{% for b in ancestors %}<a href="{% url 'brokers:detail' b.code %}">{{
b.code }}</a> › {% endfor %}{{ broker.code }}</p>
<h1>{{ broker.name }} <small>Tier {{ broker.tier }} · {{
broker.commission_rate }}%</small></h1>
<p>{{ client_count }} referred client{{ client_count|pluralize }}</p>
<h2>Sub-brokers</h2>
<ul>
  {% for child in children %}<li><a href="{% url 'brokers:detail' child.code
%}">{{ child }}</a></li>
  {% empty %}<li>None</li>{% endfor %}
</ul>
{% endblock content %}
```

29.

```
# clients/views.py — replaces the JSON ClientDetailView (URL name
clients:detail unchanged)
from django.shortcuts import get_object_or_404
from django.views.generic import TemplateView
from core.mixins import PageTitleMixin
from .models import Client


class ClientDetailView(PageTitleMixin, TemplateView):
    template_name = "clients/client_detail.html"
    page_title = "Client"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["client"] = get_object_or_404(Client, ref=self.kwargs["ref"])
        return context
```

```
<!-- clients/templates/clients/client_detail.html -->
{% extends "base.html" %}
{% block content %}
<h1>{{ client.name }} <small>{{ client.account_number }}</small></h1>
<p>KYC: {{ client.get_kyc_status_display }} · Broker: {{
client.broker|default:"Direct" }} ·
   Lead: {% if client.lead %}<a href="{{ client.lead.get_absolute_url }}">{{
client.lead.name }}</a>{% else %}—{% endif %}</p>
{% for account in client.accounts.all %}
  <h2>{{ account }} · {{ account.currency }} · 1:{{ account.leverage }}</h2>
  <table class="table">
    {% for dep in account.deposits.all %}
      <tr><td>{{ dep.created_at|date:"d M Y" }}</td><td>{{
dep.get_method_display }}</td><td>{{ dep.amount|floatformat:2 }}</td></tr>
    {% empty %}
      <tr><td colspan="3">No deposits.</td></tr>
    {% endfor %}
  </table>
{% endfor %}
{% endblock content %}
```

30.

```
# LeadQueryMixin.get_leads — add
if (agent := p.get("agent", "")).isdigit():
    qs = qs.filter(assigned_to_id=agent)
if tag := p.get("tag"):
    qs = qs.filter(tags__name=tag)          # one tag → no duplicate rows
```

```
<!-- lead_list.html: add <th>Agent</th> and -->
<td>{{ lead.assigned_to|default:"—" }}</td>
```

D. Challenge

31–33.

```
# leads/models.py
class Campaign(models.Model):
# 31
    name = models.CharField(max_length=100, unique=True)
    starts_on = models.DateField()
    leads = models.ManyToManyField(Lead, through="CampaignEnrollment",
related_name="campaigns")

    def __str__(self):
```

```
        return self.name


class CampaignEnrollment(models.Model):
    class Outcome(models.TextChoices):
        PENDING = "pending"
        INTERESTED = "interested"
        DECLINED = "declined"

    campaign = models.ForeignKey(Campaign, on_delete=models.CASCADE)
    lead = models.ForeignKey(Lead, on_delete=models.CASCADE)
    enrolled_at = models.DateTimeField(default=timezone.now)
    outcome = models.CharField(max_length=20, choices=Outcome.choices,
default=Outcome.PENDING)

    class Meta:
        constraints = [models.UniqueConstraint(fields=["campaign", "lead"],
name="unique_campaign_lead")]


def get_sentinel_agent():
# 33
    from django.contrib.auth import get_user_model
    return get_user_model().objects.get_or_create(
        username="deleted-agent", defaults={"role": "agent", "is_active":
False},
    )[0]

# CallLog.agent becomes:
agent = models.ForeignKey(
    settings.AUTH_USER_MODEL, on_delete=models.SET(get_sentinel_agent),
related_name="calls",
)
```

```
# shell — 31
from datetime import date
from leads.models import Campaign, CampaignEnrollment

camp = Campaign.objects.create(name="Gold Q4", starts_on=date.today())
camp.leads.add(Lead.objects.first(), through_defaults={"outcome":
"interested"})
CampaignEnrollment.objects.filter(campaign=camp,
outcome="interested").count()
Lead.objects.first().campaigns.all()
```

```
# clients/models.py — 32
related_clients = models.ManyToManyField("self", blank=True,
symmetrical=True)

# shell
a, b = Client.objects.all()[:2]
a.related_clients.add(b)
b.related_clients.all()          # contains a — symmetrical adds both
directions
```

34.

```
from datetime import timedelta
week = timezone.now() - timedelta(days=7)

# ONE call that is both answered AND in the last 7 days
a = Lead.objects.filter(calls__outcome="answered",
calls__started_at__gte=week).distinct().count()

# SOME answered call (any time) AND SOME call (maybe another one) in the last
7 days
b =
Lead.objects.filter(calls__outcome="answered").filter(calls__started_at__gte=
week).distinct().count()

a <= b   # True — chained filters on a multi-valued relation each get their
own join
```

35–36.

```
# brokers/models.py — 35
def descendants_sql(self):
    return list(Broker.objects.raw("""
        WITH RECURSIVE tree(id) AS (
            SELECT id FROM brokers_broker WHERE parent_id = %s
            UNION ALL
            SELECT b.id FROM brokers_broker b JOIN tree t ON b.parent_id =
t.id
        )
        SELECT * FROM brokers_broker WHERE id IN (SELECT id FROM tree)
    """, [self.pk]))
```

```
# shell — 36
from django.db.models import Count
Through = Lead.tags.through
Through.objects.values("tag__name").annotate(n=Count("id")).order_by("-n")
```

37–40.

```
# leads/tests.py
from django.contrib.auth import get_user_model
from django.db.models import ProtectedError

from brokers.models import Broker
from clients.models import Client, Deposit, TradingAccount
from .models import CallLog


class RelationshipTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.agent = get_user_model().objects.create_user("t_agent",
password="x", role="agent")
        cls.lead = Lead.objects.create(name="Rel Test", phone="971509990100",
assigned_to=cls.agent)

    def test_deleting_user_unassigns_leads(self):                    # 37
        self.agent.delete()
        self.lead.refresh_from_db()
        self.assertIsNone(self.lead.assigned_to)

    def test_deleted_agent_calls_move_to_sentinel(self):             # 38
        CallLog.objects.create(lead=self.lead, agent=self.agent,
outcome="answered")
        self.agent.delete()
        call = CallLog.objects.get(lead=self.lead)
        self.assertEqual(call.agent.username, "deleted-agent")

    def test_account_with_deposits_is_protected(self):               # 39
        client = Client.objects.create(name="C", email="c@x.test",
account_number="7000001")
        account = TradingAccount.objects.create(client=client,
number="7000001")
        Deposit.objects.create(account=account, amount=100, method="card")
        with self.assertRaises(ProtectedError):
            account.delete()

    def test_broker_tree(self):                                      # 40
```

```
        top = Broker.objects.create(code="BRF9000", name="Top", tier=1)
        mid = Broker.objects.create(code="BRF9100", name="Mid", tier=2,
parent=top)
        leaf = Broker.objects.create(code="BRF9110", name="Leaf", tier=3,
parent=mid)
        self.assertEqual(set(top.descendants()), {mid, leaf})
        self.assertEqual(leaf.ancestors(), [mid, top])
```

Day 9 — Aggregation and annotation

Goal: push every number the CRM reports into SQL — one query per report — and know the two traps (join fan-out and NULL sums). Ends with a `/reports/` page: agent leaderboard, broker deposits, source performance, SOP breaches.

Needs the Day 8 seed (`run_all()`). Do A and B in the shell.

A. Drills (1–10) — aggregate vs annotate

- [ ] **1.** One `aggregate()`: lead count, sum, average, min and max expected deposit.
- [ ] **2.** `aggregate()` with `filter=`: total, qualified and lost counts in one query.
- [ ] **3.** Leads per status (`values().annotate()` = GROUP BY), biggest first.
- [ ] **4.** Per source: count and average expected deposit.
- [ ] **5.** Annotate each agent with their lead count.
- [ ] **6.** Agents with more than 20 leads (filter on an annotation = HAVING).
- [ ] **7.** "New leads per agent" two ways: `filter()` before `annotate()` vs `Count(..., filter=Q(...))`. Which drops agents with zero?
- [ ] **8.** Leads per day with `TruncDate`.
- [ ] **9.** Calls per hour of day with `ExtractHour`.
- [ ] **10.** `Sum` of lost leads' deposits returning 0 instead of `None` (`default=`).

B. Core (11–20) — expressions, subqueries, windows

- [ ] **11.** Agent conversion rate (qualified ÷ total × 100) with `ExpressionWrapper`, best first.
- [ ] **12.** The fan-out trap: `Count("leads")` + `Count("calls")` on agents gives wrong numbers. Fix with `distinct=True`.
- [ ] **13.** `Case` / `When`: label each lead hot / warm / cold.
- [ ] **14.** Deposits by method in one row: `Sum(..., filter=)` and the `Case` / `When` version.
- [ ] **15.** One `update()` with `Case` / `When`: commission 30 / 20 / 10 by broker tier.
- [ ] **16.** `Subquery` + `OuterRef`: each lead's latest call outcome and time.
- [ ] **17.** `Exists`: open leads with no call in the last 7 days.
- [ ] **18.** Total deposits per client: `Subquery` sum vs direct `Sum("accounts__deposits__amount")`.
- [ ] **19.** `Window` + `Rank`: rank agents by qualified leads; keep the top 3.
- [ ] **20.** Running deposit total per trading account with `Window(Sum(...), partition_by=...)`.

C. Nexora CRM (21–30) — `reports` app

- [ ] **21.** Rewrite the dashboard's status counts as a single `aggregate()` query.
- [ ] **22.** `agent_leaderboard()`: leads, qualified, calls, answered calls, average talk time, conversion % — one query, correct counts.
- [ ] **23.** `broker_report()`: clients, leads, sub-brokers, total deposits (deposits via `Subquery` so the joins can't inflate them).
- [ ] **24.** `source_performance()`: leads, qualified, conversion %, average deposit per source.
- [ ] **25.** `daily_new_leads(days=30)`: date + count rows for a chart.
- [ ] **26.** `calls_by_hour()`: calls, answered, answer rate per hour.
- [ ] **27.** Lead list: annotate `call_count` and `last_call_at`; make both sortable with NULLs last.
- [ ] **28.** `sop_first_call_breaches(hours=24)`: leads not called within 24 h of creation (`Exists` + `OuterRef` to an annotated deadline).
- [ ] **29.** `/reports/` page (`ReportsView`) with tables for 22–24 and the first 20 SOP breaches; wire the sidebar link.
- [ ] **30.** JSON endpoints: `/reports/api/daily-leads/` and `/reports/api/calls-by-hour/`.

D. Challenge (31–40)

- [ ] **31.** Team rollup: leads and qualified per team lead (two-hop `values()`).
- [ ] **32.** Broker tree total: deposits for a broker plus all its descendants.
- [ ] **33.** `Cast` + `Round`: call duration in minutes to 1 decimal.
- [ ] **34.** Cohort table: leads per created-month × status, pivoted into a dict.
- [ ] **35.** `Lag` window: time between consecutive calls for each lead.
- [ ] **36.** Cross-check `agent_leaderboard()` against hand-written SQL with correlated subqueries.
- [ ] **37.** Test: the one-query dashboard counts equal separate `count()` calls.
- [ ] **38.** Test: the fan-out trap (2 leads × 3 calls → 6 and 6) and the `distinct=True` fix (2 and 3).
- [ ] **39.** Test: an agent with 1 qualified out of 2 leads shows 50.0% conversion.
- [ ] **40.** Test: `sop_first_call_breaches` flags a lead first called at 30 h but not one called at 2 h.

Solutions

`aggregate()` returns one dict for the whole queryset. `annotate()` adds a value per row; after `values()` it becomes GROUP BY.

A. Drills

1–10.

```
# python manage.py shell
from datetime import timedelta
from decimal import Decimal

from django.db.models import Avg, Count, F, Max, Min, Q, Sum
from django.db.models.functions import ExtractHour, TruncDate
from django.utils import timezone

from accounts.models import CustomUser
from leads.models import CallLog, Lead

Lead.objects.aggregate(                                                 # 1
    total=Count("id"),
    deposit_sum=Sum("expected_deposit"),
    deposit_avg=Avg("expected_deposit"),
    smallest=Min("expected_deposit"),
    largest=Max("expected_deposit"),
)

Lead.objects.aggregate(                                                 # 2
    total=Count("id"),
    qualified=Count("id", filter=Q(status="qualified")),
    lost=Count("id", filter=Q(status="lost")),
)

Lead.objects.values("status").annotate(n=Count("id")).order_by("-n")    # 3

(Lead.objects.values("source")                                          # 4
    .annotate(n=Count("id"), avg_deposit=Avg("expected_deposit"))
    .order_by("source"))

agents =
CustomUser.objects.filter(role="agent").annotate(lead_count=Count("leads"))
# 5
for a in agents:
    print(a.username, a.lead_count)
```

```
CustomUser.objects.annotate(lead_count=Count("leads")).filter(lead_count__gt=
20)        # 6 → HAVING

# 7: filter() first drops agents with zero new leads
CustomUser.objects.filter(leads__status="new").annotate(n=Count("leads"))
# filter= inside Count keeps every agent, 0 where none
CustomUser.objects.filter(role="agent").annotate(n=Count("leads",
filter=Q(leads__status="new")))

(Lead.objects.annotate(day=TruncDate("created_at"))                        # 8
— Dubai dates
    .values("day").annotate(n=Count("id")).order_by("day"))

(CallLog.objects.annotate(hour=ExtractHour("started_at"))                  # 9
    .values("hour").annotate(n=Count("id")).order_by("hour"))

Lead.objects.filter(status="lost").aggregate(total=Sum("expected_deposit",
default=0))   # 10
```

B. Core

11–15.

```
from django.db.models import Case, CharField, DecimalField,
ExpressionWrapper, FloatField, Value, When
from brokers.models import Broker
from clients.models import Deposit

board = (CustomUser.objects.filter(role="agent")                           #
11
    .annotate(
        total=Count("leads"),
        qualified=Count("leads", filter=Q(leads__status="qualified")),
    )
    .filter(total__gt=0)                                    # avoid divide-by-
zero
    .annotate(rate=ExpressionWrapper(F("qualified") * 100.0 / F("total"),
output_field=FloatField()))
    .order_by("-rate"))

# 12: each agent's leads JOIN each agent's calls → leads × calls rows → both
counts inflated
bad =
CustomUser.objects.filter(role="agent").annotate(leads_n=Count("leads"),
calls_n=Count("calls"))
good = CustomUser.objects.filter(role="agent").annotate(
```

```
    leads_n=Count("leads", distinct=True),
    calls_n=Count("calls", distinct=True),
)

Lead.objects.annotate(priority=Case(                                    #
13
    When(status="new", source__in=Lead.PAID_SOURCES, then=Value("hot")),
    When(status="contacted", then=Value("warm")),
    default=Value("cold"),
    output_field=CharField(),
)).values("name", "priority")[:10]

Deposit.objects.aggregate(                                              #
14
    card=Sum("amount", filter=Q(method="card"), default=0),
    bank=Sum("amount", filter=Q(method="bank"), default=0),
    crypto=Sum("amount", filter=Q(method="crypto"), default=0),
)
Deposit.objects.aggregate(        # Case/When form — what filter= becomes on
DBs without FILTER
    card=Sum(Case(When(method="card", then="amount"), default=Value(0),
output_field=DecimalField())),
)

Broker.objects.update(commission_rate=Case(                             #
15 — one UPDATE
    When(tier=1, then=Value(Decimal("30.00"))),
    When(tier=2, then=Value(Decimal("20.00"))),
    default=Value(Decimal("10.00")),
))
```

16–18.

```
from django.db.models import Exists, OuterRef, Subquery
from django.db.models.functions import Coalesce
from clients.models import Client

latest_call = CallLog.objects.filter(lead=OuterRef("pk")).order_by("-
started_at")   # 16
Lead.objects.annotate(
    last_outcome=Subquery(latest_call.values("outcome")[:1]),
    last_call_at=Subquery(latest_call.values("started_at")[:1]),
).values("name", "last_outcome", "last_call_at")[:10]

cutoff = timezone.now() - timedelta(days=7)
# 17
```

```
recent = CallLog.objects.filter(lead=OuterRef("pk"), started_at__gte=cutoff)
Lead.objects.filter(status__in=["new", "contacted"]).filter(~Exists(recent))

deposits = (Deposit.objects
# 18
    .filter(account__client=OuterRef("pk"))
    .order_by()
    .values("account__client")          # group by client inside the subquery
    .annotate(total=Sum("amount"))
    .values("total"))
Client.objects.annotate(total_deposits=Coalesce(
    Subquery(deposits), Value(Decimal("0")), output_field=DecimalField(),
))
Client.objects.annotate(total_deposits=Sum("accounts__deposits__amount",
default=0))   # same numbers here;
# the subquery stays correct even when you add other multi-valued joins to
the same query
```

19–20.

```
from django.db.models import Window
from django.db.models.functions import Rank

ranked = (CustomUser.objects.filter(role="agent")                          #
19
    .annotate(qualified=Count("leads", filter=Q(leads__status="qualified")))
    .annotate(rank=Window(expression=Rank(), order_by=F("qualified").desc()))
    .order_by("rank"))
ranked.filter(rank__lte=3)          # filtering on a window works from Django
4.2

running = (Deposit.objects                                                 #
20
    .annotate(running_total=Window(
        Sum("amount"), partition_by=F("account"),
order_by=F("created_at").asc(),
    ))
    .order_by("account", "created_at")
    .values("account__number", "created_at", "amount", "running_total")[:15])
```

C. Nexora CRM

21.

```
# core/views.py — DashboardView.get_context_data
stats = active.aggregate(
    total=Count("id"),
    **{value: Count("id", filter=Q(status=value)) for value in
Lead.Status.values},
)
context.update(
    total=stats["total"],
    counts={label: stats[value] for value, label in Lead.Status.choices},   #
1 query instead of 5
    ...
)
```

22–26, 28.

```
python manage.py startapp reports    # add "reports.apps.ReportsConfig" to
INSTALLED_APPS
```

```
# reports/queries.py
from datetime import timedelta
from decimal import Decimal

from django.contrib.auth import get_user_model
from django.db.models import (
    Avg, Case, Count, DateTimeField, DecimalField, Exists, ExpressionWrapper,
    F, FloatField, OuterRef, Q, Subquery, Sum, Value, When,
)
from django.db.models.functions import Coalesce, ExtractHour, TruncDate
from django.utils import timezone

from brokers.models import Broker
from clients.models import Deposit
from leads.models import CallLog, Lead


def pct(part, whole):
    """part / whole × 100 as a float expression (whole must be > 0)."""
    return ExpressionWrapper(F(part) * 100.0 / F(whole),
output_field=FloatField())


def agent_leaderboard():                                                #
22
    return (get_user_model().objects
        .filter(role="agent")
```

```
        .annotate(
            leads_n=Count("leads", distinct=True),
            qualified_n=Count("leads", filter=Q(leads__status="qualified"),
distinct=True),
            calls_n=Count("calls", distinct=True),
            answered_n=Count("calls", filter=Q(calls__outcome="answered"),
distinct=True),
            # Avg survives the fan-out (every call repeats equally); a Sum
here would not
            avg_talk=Avg("calls__duration_seconds",
filter=Q(calls__outcome="answered")),
        )
        .annotate(conversion=Case(
            When(leads_n=0, then=Value(0.0)),
            default=pct("qualified_n", "leads_n"),
            output_field=FloatField(),
        ))
        .order_by("-conversion", "-qualified_n"))


def broker_report():                                                    #
23
    deposits = (Deposit.objects
        .filter(account__client__broker=OuterRef("pk"))
        .order_by()
        .values("account__client__broker")
        .annotate(total=Sum("amount"))
        .values("total"))
    return (Broker.objects
        .annotate(
            clients_n=Count("clients", distinct=True),
            leads_n=Count("leads", distinct=True),
            sub_brokers_n=Count("sub_brokers", distinct=True),
            deposits_total=Coalesce(
                Subquery(deposits), Value(Decimal("0")),
                output_field=DecimalField(max_digits=14, decimal_places=2),
            ),
        )
        .order_by("-deposits_total"))


def source_performance():                                               #
24
    return (Lead.objects
        .values("source")
        .annotate(
```

```
            leads_n=Count("id"),
            qualified_n=Count("id", filter=Q(status="qualified")),
            avg_deposit=Avg("expected_deposit"),
        )
        .annotate(conversion=pct("qualified_n", "leads_n"))
        .order_by("-conversion"))


def daily_new_leads(days=30):                                                  #
25
    since = timezone.now() - timedelta(days=days)
    return list(Lead.objects
        .filter(created_at__gte=since)
        .annotate(day=TruncDate("created_at"))
        .values("day")
        .annotate(n=Count("id"))
        .order_by("day"))


def calls_by_hour():                                                           #
26
    return list(CallLog.objects
        .annotate(hour=ExtractHour("started_at"))
        .values("hour")
        .annotate(calls=Count("id"), answered=Count("id",
filter=Q(outcome="answered")))
        .annotate(answer_rate=pct("answered", "calls"))
        .order_by("hour"))


def sop_first_call_breaches(hours=24):                                         #
28
    window = timedelta(hours=hours)
    called_in_time = CallLog.objects.filter(
        lead=OuterRef("pk"), started_at__lte=OuterRef("deadline"),
    )
    return (Lead.objects
        .filter(created_at__lte=timezone.now() - window)          # deadline
already passed
        .annotate(deadline=ExpressionWrapper(F("created_at") + window,
output_field=DateTimeField()))
        .annotate(called_in_time=Exists(called_in_time))
        .filter(called_in_time=False)
        .select_related("assigned_to")
        .order_by("created_at"))
```

27.

```
# leads/views.py — LeadQueryMixin
from django.db.models import Count, F, Max

SORTABLE = {"name", "created_at", "expected_deposit", "status",
"contact_attempts",
            "call_count", "last_call_at"}

    # in get_leads(), replace the sorting block at the end:
        qs = qs.annotate(call_count=Count("calls", distinct=True),
last_call_at=Max("calls__started_at"))
        sort = p.get("sort", self.default_sort)
        if sort.lstrip("-") not in SORTABLE:
            sort = self.default_sort
        field = sort.lstrip("-")
        order = F(field).desc(nulls_last=True) if sort.startswith("-") else
F(field).asc(nulls_last=True)
        return qs.order_by(order, "-pk")       # pk = stable tie-breaker
```

```
<!-- lead_list.html: two more columns -->
<th><a href="?sort=-call_count">Calls</a></th><th><a href="?sort=-
last_call_at">Last call</a></th>
...
<td>{{ lead.call_count }}</td><td>{{ lead.last_call_at|date:"d M
H:i"|default:"—" }}</td>
```

29–30.

```
# reports/views.py
from django.views import View
from django.views.generic import TemplateView

from core.mixins import JsonResponseMixin, PageTitleMixin
from . import queries


class ReportsView(PageTitleMixin, TemplateView):                              #
29
    template_name = "reports/index.html"
    page_title = "Reports"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context.update(
            leaderboard=queries.agent_leaderboard(),
```

```
            brokers=queries.broker_report()[:10],
            sources=queries.source_performance(),
            breaches=queries.sop_first_call_breaches()[:20],
        )
        return context


class DailyLeadsAPI(JsonResponseMixin, View):                                  #
30
    def get(self, request):
        days = request.GET.get("days", "30")
        return self.render_to_json(queries.daily_new_leads(int(days) if
days.isdigit() else 30))


class CallsByHourAPI(JsonResponseMixin, View):
    def get(self, request):
        return self.render_to_json(queries.calls_by_hour())
```

```
# reports/urls.py
from django.urls import path
from . import views

app_name = "reports"
urlpatterns = [
    path("", views.ReportsView.as_view(), name="index"),
    path("api/daily-leads/", views.DailyLeadsAPI.as_view(),
name="daily_leads"),
    path("api/calls-by-hour/", views.CallsByHourAPI.as_view(),
name="calls_by_hour"),
]

# django_project/urls.py
path("reports/", include("reports.urls")),
```

```
<!-- templates/base.html — sidebar link -->
<a href="{% url 'reports:index' %}" class="{% if ns == 'reports' %}active{%
endif %}">Reports</a>
```

```
<!-- reports/templates/reports/index.html -->
{% extends "base.html" %}
{% block title %}Reports · {{ block.super }}{% endblock title %}
{% block content %}
<h1>Reports</h1>
```

```
<h2>Agent leaderboard</h2>
<table class="table">
  <tr><th>Agent</th><th>Leads</th><th>Qualified</th><th>Conv.</th>
<th>Calls</th><th>Answered</th><th>Avg talk</th></tr>
  {% for a in leaderboard %}
  <tr>
    <td><a href="{% url 'accounts:agent_detail' a.pk %}">{{ a }}</a></td>
    <td>{{ a.leads_n }}</td><td>{{ a.qualified_n }}</td><td>{{
a.conversion|floatformat:1 }}%</td>
    <td>{{ a.calls_n }}</td><td>{{ a.answered_n }}</td>
    <td>{% if a.avg_talk %}{{ a.avg_talk|floatformat:0 }}s{% else %}—{% endif
%}</td>
  </tr>
  {% endfor %}
</table>

<h2>Top brokers by deposits</h2>
<table class="table">
  <tr><th>Broker</th><th>Clients</th><th>Leads</th><th>Sub-brokers</th>
<th>Deposits</th></tr>
  {% for b in brokers %}
  <tr>
    <td><a href="{% url 'brokers:detail' b.code %}">{{ b }}</a></td>
    <td>{{ b.clients_n }}</td><td>{{ b.leads_n }}</td><td>{{ b.sub_brokers_n
}}</td>
    <td>{{ b.deposits_total|floatformat:2 }}</td>
  </tr>
  {% endfor %}
</table>

<h2>Sources</h2>
<table class="table">
  <tr><th>Source</th><th>Leads</th><th>Qualified</th><th>Conv.</th><th>Avg
deposit</th></tr>
  {% for s in sources %}
  <tr><td>{{ s.source }}</td><td>{{ s.leads_n }}</td><td>{{ s.qualified_n }}
</td>
      <td>{{ s.conversion|floatformat:1 }}%</td><td>{{
s.avg_deposit|floatformat:0 }}</td></tr>
  {% endfor %}
</table>

<h2>SOP: no call within 24 h</h2>
<table class="table">
  <tr><th>Lead</th><th>Created</th><th>Agent</th></tr>
  {% for lead in breaches %}
```

```
  <tr><td><a href="{{ lead.get_absolute_url }}">{{ lead.name }}</a></td>
      <td>{{ lead.created_at|date:"d M H:i" }}</td><td>{{
lead.assigned_to|default:"—" }}</td></tr>
  {% empty %}
  <tr><td colspan="3">No breaches.</td></tr>
  {% endfor %}
</table>
{% endblock content %}
```

D. Challenge

31–35.

```
from collections import defaultdict
from django.db.models.functions import Cast, Lag, Round, TruncMonth

(Lead.objects                                                          #
31
    .values(team=F("assigned_to__team_lead__username"))
    .annotate(leads_n=Count("id"), qualified_n=Count("id",
filter=Q(status="qualified")))
    .order_by("team"))


def tree_deposits(broker):                                             #
32
    ids = [broker.pk, *(b.pk for b in broker.descendants())]
    return (Deposit.objects
            .filter(account__client__broker_id__in=ids)
            .aggregate(total=Sum("amount", default=0))["total"])

tree_deposits(Broker.objects.get(code="BRF1000"))

CallLog.objects.annotate(                                              #
33
    minutes=Round(Cast("duration_seconds", FloatField()) / 60, 1),
).values("duration_seconds", "minutes")[:5]

rows = (Lead.objects                                                   #
34
    .annotate(month=TruncMonth("created_at"))
    .values("month", "status")
    .annotate(n=Count("id"))
    .order_by("month", "status"))
pivot = defaultdict(dict)
for r in rows:
```

```
    pivot[r["month"].strftime("%Y-%m")][r["status"]] = r["n"]
dict(pivot)          # {'2026-09': {'contacted': 28, 'lost': 15, ...}, ...}

gaps = (CallLog.objects                                                    #
35
    .annotate(prev_call=Window(Lag("started_at"), partition_by=F("lead"),
order_by=F("started_at").asc()))
    .order_by("lead_id", "started_at")
    .values("lead_id", "started_at", "prev_call")[:20])
for g in gaps:
    if g["prev_call"]:
        print(g["lead_id"], g["started_at"] - g["prev_call"])
```

36.

```
from django.db import connection
from reports.queries import agent_leaderboard

SQL = """
SELECT u.username,
       (SELECT COUNT(*) FROM crm_leads l WHERE l.assigned_to_id = u.id)
AS leads_n,
       (SELECT COUNT(*) FROM crm_leads l WHERE l.assigned_to_id = u.id AND
l.status = 'qualified') AS qualified_n,
       (SELECT COUNT(*) FROM leads_calllog c WHERE c.agent_id = u.id)
AS calls_n
FROM accounts_customuser u
WHERE u.role = 'agent'
"""
with connection.cursor() as cur:
    cur.execute(SQL)
    raw = {row[0]: tuple(row[1:]) for row in cur.fetchall()}

orm = {a.username: (a.leads_n, a.qualified_n, a.calls_n) for a in
agent_leaderboard()}
assert raw == orm, (raw, orm)
```

37–40.

```
# reports/tests.py
from datetime import timedelta

from django.contrib.auth import get_user_model
from django.db.models import Count, Q
from django.test import TestCase
from django.utils import timezone
```

```
from leads.models import CallLog, Lead
from .queries import agent_leaderboard, sop_first_call_breaches


class ReportQueryTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.agent = get_user_model().objects.create_user("r_agent",
password="x", role="agent")
        cls.l1 = Lead.objects.create(name="R One", phone="971509990201",
assigned_to=cls.agent, status="qualified")
        cls.l2 = Lead.objects.create(name="R Two", phone="971509990202",
assigned_to=cls.agent)
        for lead in (cls.l1, cls.l1, cls.l2):
            CallLog.objects.create(lead=lead, agent=cls.agent,
outcome="answered", duration_seconds=60)

    def test_dashboard_aggregate(self):                                    #
37
        stats = Lead.objects.aggregate(total=Count("id"), new=Count("id",
filter=Q(status="new")))
        self.assertEqual(stats["total"], Lead.objects.count())
        self.assertEqual(stats["new"],
Lead.objects.filter(status="new").count())

    def test_fanout_and_distinct(self):                                    #
38
        User = get_user_model()
        bad = User.objects.annotate(l=Count("leads"),
c=Count("calls")).get(pk=self.agent.pk)
        good = User.objects.annotate(
            l=Count("leads", distinct=True), c=Count("calls", distinct=True),
        ).get(pk=self.agent.pk)
        self.assertEqual((bad.l, bad.c), (6, 6))        # 2 leads × 3 calls
        self.assertEqual((good.l, good.c), (2, 3))

    def test_conversion_rate(self):                                        #
39
        row = agent_leaderboard().get(pk=self.agent.pk)
        self.assertEqual(row.qualified_n, 1)
        self.assertAlmostEqual(row.conversion, 50.0)

    def test_sop_breaches(self):                                           #
40
        old = timezone.now() - timedelta(days=3)
```

```
        Lead.objects.filter(pk__in=[self.l1.pk,
self.l2.pk]).update(created_at=old)
        CallLog.objects.filter(lead=self.l1).update(started_at=old +
timedelta(hours=2))
        CallLog.objects.filter(lead=self.l2).update(started_at=old +
timedelta(hours=30))
        breaches = sop_first_call_breaches(hours=24)
        self.assertNotIn(self.l1, breaches)
        self.assertIn(self.l2, breaches)
```

```
python manage.py test
git add -A && git commit -m "day 9"
```

Day 10 — Query performance and transactions

Goal: every CRM page runs a fixed, small number of queries no matter how many rows it shows, and every multi-step write is all-or-nothing.

Measure everything with `CaptureQueriesContext` (Day 6). Do A and B in the shell.

A. Drills (1–10) — loading related data

- [ ] **1.** Reproduce N+1: loop 20 leads printing `lead.assigned_to`; count the queries.
- [ ] **2.** Fix it with `select_related("assigned_to")`; count again.
- [ ] **3.** Two-hop `select_related`: `assigned_to__team_lead` and `broker__parent`.
- [ ] **4.** `prefetch_related("tags")`: print each lead's tag names in 2 queries total.
- [ ] **5.** Prefetch a reverse FK: `calls`.
- [ ] **6.** `Prefetch` object: only answered calls, stored on `answered_calls` via `to_attr`.
- [ ] **7.** Nested prefetch: clients → accounts → deposits.
- [ ] **8.** `only()` and `defer()`; touch a deferred field and watch the extra query.
- [ ] **9.** Same report three ways: model instances vs `values()` vs `values_list()` — compare speed for 200 rows.
- [ ] **10.** Second counting tool: `DEBUG=True` + `reset_queries()` + `connection.queries`.

B. Core (11–20) — bulk writes and transactions

- [ ] **11.** Time 200 `create()` calls vs one `bulk_create()` of 1,000 call logs (mark them `notes="perf-test"`).
- [ ] **12.** `bulk_update()` the perf-test calls; then do the same change with a single `update()`; delete them.
- [ ] **13.** Upsert tags with `bulk_create(update_conflicts=True, unique_fields=..., update_fields=...)`.
- [ ] **14.** `log_call()` service: one atomic function that creates a `CallLog`, bumps `contact_attempts` with `F()`, sets `last_contacted_at`, and moves new → contacted with `Case` / `When`.
- [ ] **15.** `atomic()` as a context manager: create a call, raise on purpose, prove nothing was saved.
- [ ] **16.** Savepoint: inner `atomic()` fails with `IntegrityError`, outer block carries on and commits.

- [ ] **17.** `transaction.on_commit()`: show the callback runs only after the outer block commits.
- [ ] **18.** `claim_lead(lead_id, agent)` with `select_for_update()` inside `atomic()`.
- [ ] **19.** Composite indexes: `CallLog(agent, started_at)` and `Lead(assigned_to, status)`; compare `explain()` before and after.
- [ ] **20.** Partial index: `created_at` only for new leads (`condition=Q(status="new")`).

C. Nexora CRM (21–30) — fix the pages

- [ ] **21.** `LeadQueryMixin`: `select_related("assigned_to", "broker")` + `prefetch_related("tags")`; show tags in the list.
- [ ] **22.** Lead detail: one queryset with `select_related` (agent, broker, client) + `Prefetch` of calls with their agents.
- [ ] **23.** Dashboard lead cards: load only the fields the card uses.
- [ ] **24.** Client detail: `prefetch_related("accounts__deposits")`.
- [ ] **25.** Agent page: replace the 4 per-status queries with one ordered query + `{% regroup %}`.
- [ ] **26.** `brokers/tree.py`: load all brokers in one query, build parent → children in Python; use it for ancestors/descendants on the broker page.
- [ ] **27.** `convert_lead_to_client(lead_id)` service: atomic, locks the lead, creates client + trading account, marks qualified, queues a "welcome email" with `on_commit`.
- [ ] **28.** `reassign_open_leads(from_agent, to_agent)`: one atomic `update()`, logs the count on commit.
- [ ] **29.** `upsert_leads(rows)`: insert new phones, update existing ones, in one `bulk_create` call.
- [ ] **30.** Query audit script: hit every CRM page with the test client and print query counts.

D. Challenge (31–40)

- [ ] **31.** Sliced `Prefetch` (Django 4.2+): each lead's 3 latest calls in one extra query.
- [ ] **32.** `prefetch_related_objects()` on a list you already loaded.
- [ ] **33.** "Leads with an answered call" three ways — `Exists`, `pk__in` subquery, join + `distinct()`; compare counts and plans.
- [ ] **34.** `atomic(durable=True)`: see the error when it's nested.
- [ ] **35.** `next_lead_to_dial(agent)` with `select_for_update(skip_locked=True)` — the work-queue pattern.
- [ ] **36.** `log_queries(label)` context manager printing query count and milliseconds.

- [ ] **37.** Test: lead list query count stays the same after adding 30 tagged, assigned leads.
- [ ] **38.** Test: `convert_lead_to_client` rolls back fully when account creation fails (`mock.patch`).
- [ ] **39.** Test: the welcome-email callback fires once (`captureOnCommitCallbacks`).
- [ ] **40.** Test: `upsert_leads` updates an existing phone and inserts a new one.

Solutions

Rule of thumb: FK / one-to-one you read forward → `select_related` (SQL JOIN). Reverse FK and M2M → `prefetch_related` (one extra query per relation, joined in Python).

A. Drills

1–10.

```
# python manage.py shell
import time
from django.db import connection, reset_queries
from django.db.models import Prefetch
from django.test.utils import CaptureQueriesContext

from clients.models import Client
from leads.models import CallLog, Lead

with CaptureQueriesContext(connection) as ctx:                          # 1
    for lead in Lead.objects.all()[:20]:
        print(lead.assigned_to)
print(len(ctx.captured_queries))        # 21 = 1 for leads + 1 per lead

with CaptureQueriesContext(connection) as ctx:                          # 2
    for lead in Lead.objects.select_related("assigned_to")[:20]:
        print(lead.assigned_to)
print(len(ctx.captured_queries))        # 1

qs = Lead.objects.select_related("assigned_to__team_lead", "broker__parent")
# 3
for lead in qs[:5]:
    print(lead.assigned_to.team_lead, lead.broker and lead.broker.parent)

with CaptureQueriesContext(connection) as ctx:                          # 4
    for lead in Lead.objects.prefetch_related("tags")[:20]:
        print(lead.name, [t.name for t in lead.tags.all()])
print(len(ctx.captured_queries))        # 2
```

```
for lead in Lead.objects.prefetch_related("calls")[:5]:                    # 5
    print(lead.name, len(lead.calls.all()))

answered = Prefetch(                                                       # 6
    "calls",

queryset=CallLog.objects.filter(outcome="answered").select_related("agent"),
    to_attr="answered_calls",            # a plain list, not a manager
)
for lead in Lead.objects.prefetch_related(answered)[:5]:
    print(lead.name, [c.agent.username for c in lead.answered_calls])

for client in Client.objects.prefetch_related("accounts__deposits")[:5]:   #
7 — 3 queries total
    for account in client.accounts.all():
        print(client.name, account.number, sum(d.amount for d in
account.deposits.all()))

slim = Lead.objects.only("id", "name", "status")[:5]                       # 8
with CaptureQueriesContext(connection) as ctx:
    for lead in slim:
        lead.notes                          # deferred → 1 extra query per
row
print(len(ctx.captured_queries))            # 6
Lead.objects.defer("notes", "email")

for label, qs in [                                                         # 9
    ("models", Lead.objects.all()),
    ("values", Lead.objects.values("name", "status")),
    ("values_list", Lead.objects.values_list("name", "status")),
]:
    t = time.perf_counter()
    list(qs)
    print(f"{label:<12} {(time.perf_counter() - t) * 1000:.1f} ms")

reset_queries()                           # 10 — only records when DEBUG=True
list(Lead.objects.all()[:5])
print(len(connection.queries), connection.queries[-1]["sql"])
```

B. Core

11–13.

```
from accounts.models import CustomUser
from leads.models import Tag

agent = CustomUser.objects.filter(role="agent").first()                  # 11
lead = Lead.objects.first()

t = time.perf_counter()
for _ in range(200):
    CallLog.objects.create(lead=lead, agent=agent, outcome="busy",
notes="perf-test")
print("200 x create:", time.perf_counter() - t)

t = time.perf_counter()
CallLog.objects.bulk_create(
    [CallLog(lead=lead, agent=agent, outcome="busy", notes="perf-test") for _
in range(1000)],
    batch_size=500,
)
print("1000 x bulk_create:", time.perf_counter() - t)

calls = list(CallLog.objects.filter(notes="perf-test")[:500])            # 12
for c in calls:
    c.duration_seconds = 5
CallLog.objects.bulk_update(calls, ["duration_seconds"], batch_size=250)
CallLog.objects.filter(notes="perf-test").update(duration_seconds=5)     #
same value → one UPDATE wins
CallLog.objects.filter(notes="perf-test").delete()

Tag.objects.bulk_create(                                                 # 13
    [Tag(name="vip", color="#b91c1c"), Tag(name="kyc-pending",
color="#0ea5e9")],
    update_conflicts=True,          # INSERT ... ON CONFLICT(name) DO UPDATE
    unique_fields=["name"],
    update_fields=["color"],
)
```

14, 18.

```
# leads/services.py
from django.db import transaction
from django.db.models import Case, F, Value, When
from django.utils import timezone

from .models import CallLog, Lead
```

```
@transaction.atomic                                                    #
14
def log_call(lead, agent, outcome, duration_seconds=0, notes=""):
    call = CallLog.objects.create(
        lead=lead, agent=agent, outcome=outcome,
        duration_seconds=duration_seconds, notes=notes,
started_at=timezone.now(),
    )
    Lead.objects.filter(pk=lead.pk).update(
        contact_attempts=F("contact_attempts") + 1,
        last_contacted_at=call.started_at,
        status=Case(
            When(status=Lead.Status.NEW, then=Value(Lead.Status.CONTACTED)),
            default=F("status"),
        ),
        updated_at=timezone.now(),
    )
    return call


@transaction.atomic                                                    #
18
def claim_lead(lead_id, agent):
    # row lock until commit on PostgreSQL; a no-op on SQLite (which locks the
whole file on write)
    lead = Lead.objects.select_for_update().get(pk=lead_id)
    if lead.assigned_to_id and lead.assigned_to_id != agent.pk:
        return False
    lead.assigned_to = agent
    lead.save(update_fields=["assigned_to", "updated_at"])
    return True
```

15–17.

```
from django.db import IntegrityError, transaction

lead = Lead.objects.exclude(assigned_to=None).first()                  # 15
before = CallLog.objects.count()
try:
    with transaction.atomic():
        CallLog.objects.create(lead=lead, agent=lead.assigned_to,
outcome="answered")
        raise RuntimeError("dialer crashed")
except RuntimeError:
    pass
CallLog.objects.count() == before        # True — rolled back
```

```
with transaction.atomic():                                              # 16
    Tag.objects.create(name="outer-ok")
    try:
        with transaction.atomic():           # savepoint
            Tag.objects.create(name="outer-ok")      # duplicate →
IntegrityError
    except IntegrityError:
        print("inner rolled back to the savepoint; outer continues")
    Tag.objects.create(name="outer-ok-2")
Tag.objects.filter(name__startswith="outer-ok").count()      # 2

with transaction.atomic():                                              # 17
    Tag.objects.create(name="commit-demo")
    transaction.on_commit(lambda: print("2. runs after COMMIT"))
    print("1. still inside the transaction")
```

Catch the `IntegrityError` *outside* the inner `atomic()`, never inside it — otherwise the transaction is broken for every following query (16).

19–20.

```
# leads/models.py
class Lead(models.Model):
    class Meta:
        indexes = [
            models.Index(fields=["status", "created_at"],
name="lead_status_created_idx"),
            models.Index(fields=["assigned_to", "status"],
name="lead_agent_status_idx"),        # 19
            models.Index(fields=["created_at"], name="lead_new_created_idx",
# 20
                         condition=models.Q(status="new")),
        ]

class CallLog(models.Model):
    class Meta:
        indexes = [
            models.Index(fields=["lead", "started_at"],
name="call_lead_started_idx"),
            models.Index(fields=["agent", "started_at"],
name="call_agent_started_idx"),       # 19
        ]
```

```
# before and after makemigrations + migrate
print(CallLog.objects.filter(agent_id=3).order_by("-started_at").explain())
# after: SEARCH leads_calllog USING INDEX call_agent_started_idx (agent_id=?)
```

C. Nexora CRM

21.

```
# leads/views.py — first line of LeadQueryMixin.get_leads()
qs = (Lead.objects.filter(is_active=True)
      .select_related("assigned_to", "broker")
      .prefetch_related("tags"))
```

```
<!-- lead_list.html: add <th>Tags</th> and -->
<td>{% for tag in lead.tags.all %}<span class="badge">{{ tag }}</span> {%
endfor %}</td>
```

22–25.

```
# leads/views.py — LeadDetailView.get_context_data                          #
22
from django.db.models import Prefetch

lead = get_object_or_404(
    Lead.objects
        .select_related("assigned_to", "broker", "client")      # reverse
one-to-one works here too
        .prefetch_related(
            "tags",
            Prefetch("calls",
queryset=CallLog.objects.select_related("agent")),
        ),
    pk=lead_id,
)

# core/views.py — DashboardView                                            # 23
leads=active.only("id", "name", "phone", "status")[:6],

# clients/views.py — ClientDetailView                                      # 24
client = get_object_or_404(
    Client.objects.select_related("broker",
"lead").prefetch_related("accounts__deposits"),
    ref=self.kwargs["ref"],
)
```

```
# accounts/views.py — AgentDetailView                                    # 25
(replaces by_status)
context["leads"] = agent.leads.order_by("status", "-created_at")
```

```
<!-- accounts/agent_detail.html — 25 -->
{% regroup leads by get_status_display as groups %}
{% for group in groups %}
  <h2>{{ group.grouper }} ({{ group.list|length }})</h2>
  <ul>{% for lead in group.list|slice:":10" %}<li><a href="{{
lead.get_absolute_url }}">{{ lead.name }}</a></li>{% endfor %}</ul>
{% endfor %}
```

26.

```
# brokers/tree.py
from collections import defaultdict
from .models import Broker


def load_tree():
    """One query → (by_id, children) maps."""
    brokers = list(Broker.objects.order_by("code"))
    by_id = {b.pk: b for b in brokers}
    children = defaultdict(list)
    for b in brokers:
        children[b.parent_id].append(b)
    return by_id, children


def walk(children, parent_id=None, depth=0):
    """Yield (broker, depth) depth-first."""
    for b in children[parent_id]:
        yield b, depth
        yield from walk(children, b.pk, depth + 1)


def ancestors_of(broker, by_id):
    chain, pid = [], broker.parent_id
    while pid is not None:
        node = by_id[pid]
        chain.append(node)
        pid = node.parent_id
    return chain

# brokers/views.py — BrokerDetailView.get_context_data
```

```
by_id, children = load_tree()
broker = get_object_or_404(Broker, code=self.kwargs["code"])
context.update(
    broker=broker,
    ancestors=ancestors_of(broker, by_id)[::-1],
    children=children[broker.pk],
    descendant_count=sum(1 for _ in walk(children, broker.pk)),
    client_count=broker.clients.count(),
)
```

27.

```
# clients/services.py
from django.db import transaction

from leads.models import Lead
from .models import Client, TradingAccount


def next_account_number():
    last = TradingAccount.objects.order_by("-number").values_list("number",
flat=True).first()
    return str(int(last) + 1) if last else "2000001"


@transaction.atomic
def convert_lead_to_client(lead_id, platform="mt5", currency="USD"):
    lead = Lead.objects.select_for_update().get(pk=lead_id)
    if hasattr(lead, "client"):
        raise ValueError("Lead already converted")
    client = Client.objects.create(
        lead=lead, broker=lead.broker, name=lead.name,
        email=lead.email or f"client{lead.pk}@example.com",
        account_number=f"{1_000_000 + lead.pk:07d}",
    )
    TradingAccount.objects.create(
        client=client, number=next_account_number(), platform=platform,
currency=currency,
    )
    lead.status = Lead.Status.QUALIFIED
    lead.save(update_fields=["status", "updated_at"])
    transaction.on_commit(lambda: print(f"Welcome email queued for
{client.email}"))   # Day 23: real email
    return client
```

28–29.

```
# leads/services.py (add)
from .models import normalize_uae_phone
from .queries import OPEN_STATUSES


@transaction.atomic                                                    #
28
def reassign_open_leads(from_agent, to_agent):
    n = (Lead.objects
         .filter(assigned_to=from_agent, status__in=OPEN_STATUSES)
         .update(assigned_to=to_agent, updated_at=timezone.now()))
    transaction.on_commit(lambda: print(f"Reassigned {n} leads: {from_agent}
→ {to_agent}"))
    return n


def upsert_leads(rows):                                                #
29
    objs = [
        Lead(
            name=r["name"].strip().title(),
            phone=normalize_uae_phone(r["phone"]),     # bulk_create skips
save() → normalise here
            source=r.get("source", Lead.Source.MANUAL),
            email=r.get("email", ""),
        )
        for r in rows
    ]
    return Lead.objects.bulk_create(
        objs,
        update_conflicts=True,
        unique_fields=["phone"],                       # needs the unique
index on phone
        update_fields=["name", "source", "email", "updated_at"],
        batch_size=500,
    )
```

30.

```
# python manage.py shell
from django.db import connection
from django.test import Client as HttpClient          # not
clients.models.Client
from django.test.utils import CaptureQueriesContext, setup_test_environment

from clients.models import Client
```

```
from leads.models import Lead

setup_test_environment()
http = HttpClient()
lead = Lead.objects.first()
client = Client.objects.first()
agent = lead.assigned_to

urls = ["/leads/", lead.get_absolute_url(), "/dashboard/",
client.get_absolute_url(),
        f"/team/agents/{agent.pk}/", "/brokers/BRF1000/", "/reports/"]
for url in urls:
    with CaptureQueriesContext(connection) as ctx:
        status = http.get(url).status_code
    print(f"{len(ctx.captured_queries):>4} queries  {status}  {url}")
```

D. Challenge

31–36.

```
from django.db import transaction
from django.db.models import Exists, F, OuterRef, prefetch_related_objects

recent = Prefetch(                                                  #
31
    "calls", queryset=CallLog.objects.order_by("-started_at")[:3],
to_attr="recent_calls",
)
for lead in Lead.objects.prefetch_related(recent)[:10]:
    print(lead.name, [c.outcome for c in lead.recent_calls])

leads = list(Lead.objects.filter(status="qualified")[:20])              # 32
prefetch_related_objects(leads, "tags", "calls")

answered = CallLog.objects.filter(outcome="answered")                   # 33
a = Lead.objects.filter(Exists(answered.filter(lead=OuterRef("pk"))))
b = Lead.objects.filter(pk__in=answered.values("lead_id"))
c = Lead.objects.filter(calls__outcome="answered").distinct()
for qs in (a, b, c):
    print(qs.count())
    print(qs.explain())

with transaction.atomic():                                              # 34
    try:
        with transaction.atomic(durable=True):
            pass
```

```
    except RuntimeError as e:
        print(e)      # A durable atomic block cannot be nested within
another atomic block.


# leads/services.py                                                    # 35
def next_lead_to_dial(agent):
    with transaction.atomic():
        lead = (Lead.objects
                .select_for_update(skip_locked=True)    # other agents'
locked rows are skipped (PostgreSQL)
                .filter(assigned_to=agent, status__in=OPEN_STATUSES)
                .order_by(F("last_contacted_at").asc(nulls_first=True),
"created_at")
                .first())
        if lead:

Lead.objects.filter(pk=lead.pk).update(last_contacted_at=timezone.now())
        return lead
```

```
# core/debug.py                                                        # 36
import time
from contextlib import contextmanager

from django.db import connection
from django.test.utils import CaptureQueriesContext


@contextmanager
def log_queries(label="block"):
    start = time.perf_counter()
    with CaptureQueriesContext(connection) as ctx:
        yield ctx
    ms = (time.perf_counter() - start) * 1000
    print(f"[{label}] {len(ctx.captured_queries)} queries in {ms:.1f} ms")

# usage
with log_queries("leaderboard"):
    list(agent_leaderboard())
```

37–40.

```
# leads/tests.py
from unittest import mock

from django.db import IntegrityError, connection
```

```
from django.test.utils import CaptureQueriesContext

from clients.models import Client
from clients.services import convert_lead_to_client
from .models import Tag
from .services import upsert_leads


class PerformanceAndTransactionTests(TestCase):
    def _count(self, url):
        with CaptureQueriesContext(connection) as ctx:
            self.client.get(url)
        return len(ctx.captured_queries)

    def test_lead_list_query_count_is_constant(self):                   # 37
        agent = get_user_model().objects.create_user("qc_agent",
password="x", role="agent")
        tag = Tag.objects.create(name="qc")
        before = self._count(reverse("leads:list"))
        Lead.objects.bulk_create([
            Lead(name=f"QC {i}", phone=f"9715098{i:05d}", assigned_to=agent)
for i in range(30)
        ])
        for lead in Lead.objects.filter(name__startswith="QC"):
            lead.tags.add(tag)
        self.assertEqual(self._count(reverse("leads:list")), before)

    def test_conversion_rolls_back(self):                               # 38
        lead = Lead.objects.create(name="Rollback", phone="971509990501")
        with mock.patch("clients.services.next_account_number",
side_effect=IntegrityError("boom")):
            with self.assertRaises(IntegrityError):
                convert_lead_to_client(lead.pk)
        self.assertFalse(Client.objects.filter(lead=lead).exists())   #
client insert undone
        lead.refresh_from_db()
        self.assertEqual(lead.status, Lead.Status.NEW)

    def test_welcome_email_on_commit(self):                             # 39
        lead = Lead.objects.create(name="Commit", phone="971509990502")
        with self.captureOnCommitCallbacks(execute=True) as callbacks:
            convert_lead_to_client(lead.pk)
        self.assertEqual(len(callbacks), 1)

    def test_upsert(self):                                              # 40
        Lead.objects.create(name="Old Name", phone="971509990601")
```

```
        upsert_leads([
            {"name": "new name", "phone": "+971 50 999 0601", "source":
"google"},
            {"name": "fresh lead", "phone": "971509990602", "source":
"referral"},
        ])
        self.assertEqual(Lead.objects.get(phone="971509990601").name, "New
Name")
        self.assertTrue(Lead.objects.filter(phone="971509990602").exists())
```

`TestCase` wraps each test in a transaction, so `on_commit` callbacks never fire on their own — `captureOnCommitCallbacks(execute=True)` runs them (39).

Day 11 — Admin customization

Goal: turn `/admin/` into Nexora's back office — searchable, filterable lists for every model, inline history, bulk actions, and guard rails on who can edit what.

Replace each `admin.site.register(Model)` line from Days 5 and 8 with a full `@admin.register` class as you go (registering a model twice raises `AlreadyRegistered`).

A. Drills (1–10) — `LeadAdmin` list page

- [ ] **1.** `list_display`: name, phone, status, source, agent, created.
- [ ] **2.** `list_filter` (status, source, agent, tags) + `date_hierarchy` on `created_at`.
- [ ] **3.** `search_fields`: name, phone, email, agent username.
- [ ] **4.** `ordering`, `list_per_page = 50`, and `status` editable straight from the list (`list_editable`).
- [ ] **5.** `list_display_links` on name; `readonly_fields` for timestamps and dialer counters.
- [ ] **6.** `fieldsets`: Contact, Pipeline, Dialer (collapsed), Notes, Timestamps (collapsed).
- [ ] **7.** `autocomplete_fields` for agent and broker (their admins need `search_fields`).
- [ ] **8.** `filter_horizontal` for tags.
- [ ] **9.** Computed column `masked_phone` with `@admin.display(description=...)`, reusing the Day 3 filter.
- [ ] **10.** Boolean column `hot` from `Lead.is_hot` (`boolean=True`).

B. Core (11–20) — inlines, actions, hooks

- [ ] **11.** `CallLogInline` (tabular, read-only, newest first, link to each call) on the lead page.
- [ ] **12.** `TradingAccountInline` (stacked) on clients; `DepositInline` (tabular) on trading accounts.
- [ ] **13.** Action "Mark selected as contacted" with a success message.
- [ ] **14.** Reusable `export_as_csv` action in `core/admin_actions.py`; attach it to leads.
- [ ] **15.** `SimpleListFilter` "Contact state": never contacted / no touch in 48 h / touched in 48 h.
- [ ] **16.** `get_queryset`: `select_related` + annotate a call count; sortable "Calls" column.
- [ ] **17.** Site header, site title and index title.
- [ ] **18.** `save_model`: a new lead created by an agent with no assignee is assigned to that agent.

- [ ] **19.** `get_readonly_fields`: agents can't edit `expected_deposit`.
- [ ] **20.** `formfield_for_foreignkey`: agent dropdown lists active agents only.

C. Nexora CRM (21–30) — the other models

- [ ] **21.** `BrokerAdmin`: code, name, tier, parent, commission, client count; filter tier; search; autocomplete parent.
- [ ] **22.** Sub-brokers inline on the broker page (`fk_name="parent"`).
- [ ] **23.** `ClientAdmin`: KYC filter, accounts inline, read-only `ref`, total deposits column, "Approve KYC" action.
- [ ] **24.** `TradingAccountAdmin`: deposits inline and a deposits-total column.
- [ ] **25.** `DepositAdmin`: date hierarchy, method filter, search by account number or client name, client column.
- [ ] **26.** `CallLogAdmin`: `list_select_related`, filters for outcome and agent, date hierarchy, search.
- [ ] **27.** `CustomUserAdmin`: lead-count column, role/team-lead filters, read-only team-members inline.
- [ ] **28.** `TagAdmin`: colour swatch with `format_html` and a lead count.
- [ ] **29.** Lead list column linking to the converted client's admin page (`reverse("admin:...")`).
- [ ] **30.** Attach `export_as_csv` to clients and deposits too.

D. Challenge (31–40)

- [ ] **31.** Intermediate-page action "Reassign to agent…": form to pick the agent, then apply.
- [ ] **32.** Custom admin page `/admin/leads/lead/stats/` via `get_urls()`, showing the Day 9 reports.
- [ ] **33.** Make `CallLog` read-only in the admin (no add, change, delete) — it's audit data.
- [ ] **34.** Second `AdminSite` at `/backoffice/` with only leads, calls, users and brokers.
- [ ] **35.** Override the lead change-list template to show "N new leads today" above the table.
- [ ] **36.** `get_search_results`: a phone typed in any format (`+971 50 …`) finds the lead.
- [ ] **37.** Test: lead change list loads for a superuser with filters applied.
- [ ] **38.** Test: the "mark contacted" action updates the row.
- [ ] **39.** Test: the CallLog add page returns 403.
- [ ] **40.** Test: searching `+971 50 999 0301` finds the lead.

Solutions

Shared action (14, 30)

```
# core/admin_actions.py
import csv

from django.contrib import admin
from django.http import HttpResponse


@admin.action(description="Export selected to CSV")
def export_as_csv(modeladmin, request, queryset):
    meta = modeladmin.model._meta
    fields = [f.name for f in meta.concrete_fields]
    response = HttpResponse(content_type="text/csv")
    response["Content-Disposition"] = f'attachment; filename="
{meta.model_name}s.csv"'
    writer = csv.writer(response)
    writer.writerow(fields)
    for obj in queryset:
        writer.writerow([getattr(obj, f) for f in fields])
    return response              # returning a response sends it instead of
reloading the list
```

A + B + D. Final `leads/admin.py` (1–20, 29, 31–33, 35–36)

```
# leads/admin.py
import re
from datetime import timedelta

from django import forms
from django.contrib import admin, messages
from django.contrib.admin import helpers
from django.contrib.auth import get_user_model
from django.db.models import Count
from django.shortcuts import render
from django.template.response import TemplateResponse
from django.urls import path, reverse
from django.utils import timezone
from django.utils.html import format_html

from core.admin_actions import export_as_csv
from reports.queries import agent_leaderboard, source_performance
from .models import CallLog, Lead, Tag
```

```
from .templatetags.crm_filters import mask_phone


class CallLogInline(admin.TabularInline):                                       #
11, 33
    model = CallLog
    extra = 0
    fields = ["started_at", "agent", "outcome", "duration_seconds"]
    readonly_fields = fields
    ordering = ["-started_at"]
    show_change_link = True
    can_delete = False

    def has_add_permission(self, request, obj=None):
        return False


class ContactStateFilter(admin.SimpleListFilter):                               #
15
    title = "contact state"
    parameter_name = "contact"

    def lookups(self, request, model_admin):
        return [("never", "Never contacted"), ("overdue", "No touch in 48
h"), ("recent", "Touched in 48 h")]

    def queryset(self, request, queryset):
        cutoff = timezone.now() - timedelta(hours=48)
        if self.value() == "never":
            return queryset.filter(last_contacted_at__isnull=True)
        if self.value() == "overdue":
            return queryset.filter(last_contacted_at__lt=cutoff)
        if self.value() == "recent":
            return queryset.filter(last_contacted_at__gte=cutoff)
        return queryset


class ReassignForm(forms.Form):                                                 #
31
    agent = forms.ModelChoiceField(
        queryset=get_user_model().objects.filter(role="agent",
is_active=True),
    )


@admin.register(Lead)
```

```
class LeadAdmin(admin.ModelAdmin):
    list_display = ["name", "masked_phone", "status", "source",
"assigned_to",          # 1, 9-10, 16, 29
                    "call_count", "hot", "client_link", "created_at"]
    list_display_links = ["name"]
# 5
    list_editable = ["status"]
# 4
    list_filter = ["status", "source", ContactStateFilter, "assigned_to",
"tags"]       # 2, 15
    date_hierarchy = "created_at"
    search_fields = ["name", "phone", "email", "assigned_to__username"]
# 3
    ordering = ["-created_at"]
    list_per_page = 50
    readonly_fields = ["created_at", "updated_at", "contact_attempts",
"last_contacted_at"]
    autocomplete_fields = ["assigned_to", "broker"]
# 7
    filter_horizontal = ["tags"]
# 8
    inlines = [CallLogInline]
    actions = ["mark_contacted", "reassign", export_as_csv]
# 13, 31, 14
    fieldsets = [
# 6
        ("Contact", {"fields": ["name", "phone", "email", "country"]}),
        ("Pipeline", {"fields": ["status", "source", "assigned_to", "broker",
"tags", "expected_deposit"]}),
        ("Dialer", {"fields": ["contact_attempts", "last_contacted_at"],
"classes": ["collapse"]}),
        ("Notes", {"fields": ["notes", "is_active"]}),
        ("Timestamps", {"fields": ["created_at", "updated_at"], "classes":
["collapse"]}),
    ]

    # ---- queryset & columns ---------------------------------------------
--
    def get_queryset(self, request):
# 16
        return (super().get_queryset(request)
                .select_related("assigned_to", "broker", "client")
                .annotate(_call_count=Count("calls")))

    @admin.display(description="Phone")
    def masked_phone(self, obj):
```

```
        return mask_phone(obj.phone)

    @admin.display(description="Calls", ordering="_call_count")
    def call_count(self, obj):
        return obj._call_count

    @admin.display(description="Hot", boolean=True)
    def hot(self, obj):
        return obj.is_hot

    @admin.display(description="Client")
# 29
    def client_link(self, obj):
        if not hasattr(obj, "client"):
            return "—"
        url = reverse("admin:clients_client_change", args=[obj.client.pk])
        return format_html('<a href="{}">{}</a>', url,
obj.client.account_number)

    # ---- actions ----------------------------------------------------------
----
    @admin.action(description="Mark selected as contacted")
# 13
    def mark_contacted(self, request, queryset):
        pks = list(queryset.values_list("pk", flat=True))     # drop the
Count annotation
        n =
Lead.objects.filter(pk__in=pks).update(status=Lead.Status.CONTACTED,
updated_at=timezone.now())
        self.message_user(request, f"{n} lead(s) marked contacted.",
messages.SUCCESS)

    @admin.action(description="Reassign selected to an agent…")
# 31
    def reassign(self, request, queryset):
        if "apply" in request.POST:
            form = ReassignForm(request.POST)
            if form.is_valid():
                agent = form.cleaned_data["agent"]
                pks = list(queryset.values_list("pk", flat=True))
                n = Lead.objects.filter(pk__in=pks).update(assigned_to=agent,
updated_at=timezone.now())
                self.message_user(request, f"{n} lead(s) reassigned to
{agent}.", messages.SUCCESS)
                return None                    # back to the change list
        else:
```

```
            form = ReassignForm()
        return render(request, "admin/leads/reassign.html", {
            **self.admin_site.each_context(request),
            "title": "Reassign leads",
            "form": form,
            "leads": queryset,
            "action_checkbox_name": helpers.ACTION_CHECKBOX_NAME,
        })

    # ---- hooks ----------------------------------------------------------
----
    def save_model(self, request, obj, form, change):
# 18
        if not change and obj.assigned_to is None and getattr(request.user,
"role", "") == "agent":
            obj.assigned_to = request.user
        super().save_model(request, obj, form, change)

    def get_readonly_fields(self, request, obj=None):
# 19
        fields = list(super().get_readonly_fields(request, obj))
        if not request.user.is_superuser and getattr(request.user, "role",
"") == "agent":
            fields.append("expected_deposit")
        return fields

    def formfield_for_foreignkey(self, db_field, request, **kwargs):
# 20
        if db_field.name == "assigned_to":
            kwargs["queryset"] =
get_user_model().objects.filter(role="agent", is_active=True)
        return super().formfield_for_foreignkey(db_field, request, **kwargs)

    def get_search_results(self, request, queryset, search_term):
# 36
        digits = re.sub(r"\D", "", search_term)
        if len(digits) >= 4 and not re.search(r"[A-Za-z]", search_term):
            return queryset.filter(phone__contains=digits), False
        return super().get_search_results(request, queryset, search_term)

    def changelist_view(self, request, extra_context=None):
# 35
        extra_context = {
            **(extra_context or {}),
            "today_count":
Lead.objects.filter(created_at__date=timezone.localdate()).count(),
```

```
        }
        return super().changelist_view(request, extra_context=extra_context)

    # ---- custom page ------------------------------------------------------
----
    def get_urls(self):
# 32
        custom = [
            path("stats/", self.admin_site.admin_view(self.stats_view),
name="leads_lead_stats"),
        ]
        return custom + super().get_urls()     # custom first, or
<path:object_id>/ swallows it

    def stats_view(self, request):
        return TemplateResponse(request, "admin/leads/stats.html", {
            **self.admin_site.each_context(request),
            "title": "Lead stats",
            "sources": source_performance(),
            "leaderboard": agent_leaderboard(),
        })


@admin.register(CallLog)
class CallLogAdmin(admin.ModelAdmin):                                   #
26, 33
    list_display = ["started_at", "lead", "agent", "outcome",
"duration_seconds"]
    list_filter = ["outcome", "agent"]
    date_hierarchy = "started_at"
    search_fields = ["lead__name", "lead__phone", "agent__username"]
    list_select_related = ["lead", "agent"]

    def has_add_permission(self, request):
        return False

    def has_change_permission(self, request, obj=None):
        return False                  # view permission still lets staff read
it

    def has_delete_permission(self, request, obj=None):
        return False


@admin.register(Tag)
class TagAdmin(admin.ModelAdmin):                                       #
```

```
28
    list_display = ["name", "swatch", "lead_count"]
    search_fields = ["name"]

    def get_queryset(self, request):
        return super().get_queryset(request).annotate(_leads=Count("leads"))

    @admin.display(description="Colour")
    def swatch(self, obj):
        return format_html(
            '<span style="display:inline-block;width:1rem;height:1rem;border-
radius:4px;'
            'background:{}"></span> {}', obj.color, obj.color,
        )

    @admin.display(description="Leads", ordering="_leads")
    def lead_count(self, obj):
        return obj._leads
```

`format_html` escapes its arguments — never build admin HTML with f-strings (28, 29).

17.

```
# django_project/urls.py
admin.site.site_header = "Nexora CRM back office"
admin.site.site_title = "Nexora admin"
admin.site.index_title = "Operations"
```

C. Nexora CRM

21–22.

```
# brokers/admin.py
from django.contrib import admin
from django.db.models import Count
from .models import Broker


class SubBrokerInline(admin.TabularInline):
    model = Broker
    fk_name = "parent"              # the self-FK this inline follows
    fields = ["code", "name", "tier", "commission_rate", "is_active"]
    extra = 0
    show_change_link = True
    verbose_name_plural = "Sub-brokers"
```

```
@admin.register(Broker)
class BrokerAdmin(admin.ModelAdmin):
    list_display = ["code", "name", "tier", "parent", "commission_rate",
"clients_n", "is_active"]
    list_filter = ["tier", "is_active"]
    search_fields = ["code", "name"]
    autocomplete_fields = ["parent"]
    list_select_related = ["parent"]
    inlines = [SubBrokerInline]

    def get_queryset(self, request):
        return
super().get_queryset(request).annotate(_clients=Count("clients",
distinct=True))

    @admin.display(description="Clients", ordering="_clients")
    def clients_n(self, obj):
        return obj._clients
```

23–25, 30.

```
# clients/queries.py
from decimal import Decimal

from django.db.models import DecimalField, OuterRef, Subquery, Sum, Value
from django.db.models.functions import Coalesce

from .models import Deposit


def deposits_total():
    """Annotation: total deposits for each Client row (join-safe)."""
    per_client = (Deposit.objects
        .filter(account__client=OuterRef("pk"))
        .order_by()
        .values("account__client")
        .annotate(t=Sum("amount"))
        .values("t"))
    return Coalesce(Subquery(per_client), Value(Decimal("0")),
                    output_field=DecimalField(max_digits=14,
decimal_places=2))
```

```
# clients/admin.py
from django.contrib import admin, messages
from django.db.models import Sum
from django.urls import reverse
from django.utils.html import format_html

from core.admin_actions import export_as_csv
from .models import Client, Deposit, TradingAccount
from .queries import deposits_total


class TradingAccountInline(admin.StackedInline):
    model = TradingAccount
    extra = 0
    show_change_link = True


class DepositInline(admin.TabularInline):
    model = Deposit
    extra = 1
    fields = ["amount", "method", "created_at"]
    can_delete = False             # money records are never deleted from
here


@admin.register(Client)
class ClientAdmin(admin.ModelAdmin):
# 23
    list_display = ["name", "account_number", "account_type", "kyc_status",
"broker",
                    "total_deposits", "date_joined"]
    list_filter = ["kyc_status", "account_type", "broker"]
    search_fields = ["name", "email", "account_number", "accounts__number"]
    readonly_fields = ["ref", "date_joined", "lead_link"]
    autocomplete_fields = ["broker", "lead"]
    list_select_related = ["broker"]
    inlines = [TradingAccountInline]
    actions = ["approve_kyc", export_as_csv]
# 30

    def get_queryset(self, request):
        return
super().get_queryset(request).annotate(_deposits=deposits_total())

    @admin.display(description="Deposits", ordering="_deposits")
    def total_deposits(self, obj):
```

```
        return f"{obj._deposits:,.2f}"

    @admin.display(description="Source lead")
    def lead_link(self, obj):
        if not obj.lead_id:
            return "—"
        url = reverse("admin:leads_lead_change", args=[obj.lead_id])
        return format_html('<a href="{}">{}</a>', url, obj.lead.name)

    @admin.action(description="Approve KYC")
    def approve_kyc(self, request, queryset):
        n =
Client.objects.filter(pk__in=queryset.values("pk")).update(kyc_status=Client.
KYCStatus.APPROVED)
        self.message_user(request, f"{n} client(s) approved.",
messages.SUCCESS)


@admin.register(TradingAccount)
class TradingAccountAdmin(admin.ModelAdmin):
# 24
    list_display = ["number", "client", "platform", "currency", "leverage",
"deposits_sum"]
    list_filter = ["platform", "currency"]
    search_fields = ["number", "client__name"]
    list_select_related = ["client"]
    inlines = [DepositInline]

    def get_queryset(self, request):
        return
super().get_queryset(request).annotate(_total=Sum("deposits__amount",
default=0))

    @admin.display(description="Deposits", ordering="_total")
    def deposits_sum(self, obj):
        return f"{obj._total:,.2f}"


@admin.register(Deposit)
class DepositAdmin(admin.ModelAdmin):
# 25
    list_display = ["created_at", "amount", "method", "account",
"client_name"]
    list_filter = ["method"]
    date_hierarchy = "created_at"
    search_fields = ["account__number", "account__client__name"]
```

```
    list_select_related = ["account__client"]
    actions = [export_as_csv]
# 30

    @admin.display(description="Client", ordering="account__client__name")
    def client_name(self, obj):
        return obj.account.client.name
```

27.

```
# accounts/admin.py (extend Day 8's CustomUserAdmin)
from django.db.models import Count


class TeamMemberInline(admin.TabularInline):
    model = CustomUser
    fk_name = "team_lead"
    fields = ["username", "first_name", "role", "is_active"]
    readonly_fields = fields       # create users through the normal add form
(password hashing)
    extra = 0
    max_num = 0
    can_delete = False
    show_change_link = True
    verbose_name_plural = "Team members"


@admin.register(CustomUser)
class CustomUserAdmin(UserAdmin):
    ...
    list_display = ["username", "email", "role", "team_lead", "lead_count",
"is_active"]
    list_filter = UserAdmin.list_filter + ("role", "team_lead")
    inlines = [TeamMemberInline]

    def get_queryset(self, request):
        return super().get_queryset(request).annotate(_leads=Count("leads",
distinct=True))

    @admin.display(description="Leads", ordering="_leads")
    def lead_count(self, obj):
        return obj._leads
```

D. Challenge — templates and the second site

31.

```
<!-- templates/admin/leads/reassign.html -->
{% extends "admin/base_site.html" %}
{% block content %}
<form method="post">{% csrf_token %}
  <p>Reassign {{ leads|length }} lead{{ leads|length|pluralize }}:</p>
  <ul>{% for lead in leads|slice:":20" %}<li>{{ lead }}</li>{% endfor %}</ul>
  {% for lead in leads %}<input type="hidden" name="{{ action_checkbox_name
}}" value="{{ lead.pk }}">{% endfor %}
  {{ form.as_p }}
  <input type="hidden" name="action" value="reassign">
  <input type="submit" name="apply" value="Reassign">
</form>
{% endblock content %}
```

32.

```
<!-- templates/admin/leads/stats.html  → /admin/leads/lead/stats/ -->
{% extends "admin/base_site.html" %}
{% block content %}
<h2>Sources</h2>
<table>
  <tr><th>Source</th><th>Leads</th><th>Qualified</th><th>Conversion</th></tr>
  {% for s in sources %}<tr><td>{{ s.source }}</td><td>{{ s.leads_n }}</td>
<td>{{ s.qualified_n }}</td><td>{{ s.conversion|floatformat:1 }}%</td></tr>{%
endfor %}
</table>
<h2>Agents</h2>
<table>
  <tr><th>Agent</th><th>Leads</th><th>Conversion</th></tr>
  {% for a in leaderboard %}<tr><td>{{ a }}</td><td>{{ a.leads_n }}</td><td>
{{ a.conversion|floatformat:1 }}%</td></tr>{% endfor %}
</table>
{% endblock content %}
```

34.

```
# core/admin_site.py
from django.contrib.admin import AdminSite


class BackOfficeSite(AdminSite):
    site_header = "Nexora back office"
    site_title = "Back office"
    index_title = "Daily operations"
```

```
backoffice = BackOfficeSite(name="backoffice")

# at the bottom of each app's admin.py — autocomplete targets must be
registered on this site too
from core.admin_site import backoffice
backoffice.register(Lead, LeadAdmin)            # leads/admin.py
backoffice.register(CallLog, CallLogAdmin)
backoffice.register(CustomUser, CustomUserAdmin)   # accounts/admin.py
backoffice.register(Broker, BrokerAdmin)           # brokers/admin.py

# django_project/urls.py
from core.admin_site import backoffice
path("backoffice/", backoffice.urls),
```

35.

```
<!-- templates/admin/leads/lead/change_list.html  (the admin finds this path
automatically) -->
{% extends "admin/change_list.html" %}
{% block content_title %}
  {{ block.super }}
  <p><strong>{{ today_count }}</strong> new lead{{ today_count|pluralize }}
today</p>
{% endblock content_title %}
```

37–40.

```
# leads/tests.py
class LeadAdminTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.boss = get_user_model().objects.create_superuser(
            "boss", "boss@nexora.test", "pass12345", role="admin",
        )
        cls.lead = Lead.objects.create(name="Admin Test",
phone="971509990301")

    def setUp(self):
        self.client.force_login(self.boss)

    def test_changelist_with_filters(self):                              # 37
        response = self.client.get(
            reverse("admin:leads_lead_changelist"), {"status__exact": "new",
"contact": "never"},
        )
        self.assertEqual(response.status_code, 200)
```

```
        self.assertContains(response, "Admin Test")

    def test_mark_contacted_action(self):                                # 38
        response = self.client.post(reverse("admin:leads_lead_changelist"), {
            "action": "mark_contacted",
            "_selected_action": [self.lead.pk],
            "index": 0,
        })
        self.assertEqual(response.status_code, 302)
        self.lead.refresh_from_db()
        self.assertEqual(self.lead.status, Lead.Status.CONTACTED)

    def test_calllog_add_forbidden(self):                                # 39

self.assertEqual(self.client.get(reverse("admin:leads_calllog_add")).status_c
ode, 403)

    def test_search_by_formatted_phone(self):                            # 40
        response = self.client.get(reverse("admin:leads_lead_changelist"),
{"q": "+971 50 999 0301"})
        self.assertContains(response, "Admin Test")
```

Day 12 — ListView, DetailView, pagination

Goal: replace the hand-rolled `TemplateView` pages with Django's generic display views — paginated lists that keep their filters, detail pages that load in a few queries, and date-based archives for calls and deposits.

Django 5.1+ ships `{% querystring %}`, which builds pagination links that keep every other GET parameter.

A. Drills (1–10) — ListView and DetailView basics

- [ ] **1.** `LeadListView(ListView)`: `context_object_name = "leads"`, same template, `get_queryset()` returns `self.get_leads()`.
- [ ] **2.** `paginate_by = 25`; in the template use `page_obj`, `paginator`, `is_paginated`.
- [ ] **3.** Pagination partial `templates/partials/_pagination.html`: First / Prev / "Page x of y" / Next / Last.
- [ ] **4.** Use `{% querystring page=... %}` so `q`, `status`, `sort` survive page changes.
- [ ] **5.** `?per_page=` with `get_paginate_by()` (whitelist 10/25/50/100).
- [ ] **6.** `LeadDetailView(DetailView)` with `pk_url_kwarg = "lead_id"` and `context_object_name = "lead"`.
- [ ] **7.** `BrokerDetailView(DetailView)` looked up by code: `slug_field` + `slug_url_kwarg`.
- [ ] **8.** Move the detail page extras (tree, counts, session "recent") into `get_context_data` using `self.object`.
- [ ] **9.** `get_queryset()` on detail views with the Day 10 `select_related` / `prefetch_related`.
- [ ] **10.** In the shell: `Paginator` — `page()`, `get_page()` with junk input, `num_pages`, `page_range`, `get_elided_page_range()`.

B. Core (11–20) — more list pages

- [ ] **11.** `ClientListView`: search (name, account, trading account number), KYC filter, total deposits column, 25 per page.
- [ ] **12.** `ClientDetailView(DetailView)` looked up by UUID `ref`.
- [ ] **13.** `AgentDetailView(DetailView)` whose queryset only allows agents.
- [ ] **14.** `CallLogListView` at `/leads/calls/`: 50 per page, filters outcome / agent / day, `ordering` attribute.

- [ ] **15.** Paginate the call history inside lead detail with its own `?calls_page=`.
- [ ] **16.** `allow_empty = False` on a strict calls route — empty result → 404.
- [ ] **17.** Try `?page=999` (404) and `?page=last` (last page).
- [ ] **18.** `get_ordering()` on calls from `?sort=` with a whitelist.
- [ ] **19.** Delete `template_name` from `ClientListView` and rely on the default `clients/client_list.html`.
- [ ] **20.** Pass `queryset = ...` as a class attribute (instead of `model`) on a view that needs no request data.

C. Nexora CRM (21–30)

- [ ] **21.** Lead list on `ListView` keeps everything: filters, chips, sort, annotations, tags.
- [ ] **22.** One pagination partial used by leads, clients and calls.
- [ ] **23.** "Showing 26–50 of 213" under each table.
- [ ] **24.** Lead detail: Older / Newer links (`get_previous_by_created_at` / `get_next_by_created_at`).
- [ ] **25.** Lead detail sidebar: the 5 recently viewed leads from the session.
- [ ] **26.** Clients page wired into the sidebar; deposits via `clients/queries.deposits_total()`.
- [ ] **27.** Broker list page `/brokers/`: whole tree, indented by depth, one query (`brokers/tree.py`).
- [ ] **28.** Agent call queue `/team/agents/<pk>/queue/`: their follow-up-due + stale leads, never-called first, paginated.
- [ ] **29.** `/leads/api/page/?page=`: JSON with page, num_pages, count, has_next, results.
- [ ] **30.** `?partial=1` returns only the table rows (ready for "load more" on Day 27).

D. Challenge (31–40)

- [ ] **31.** Keyset pagination for calls: `/leads/api/calls/?after=<timestamp>,<id>` — no OFFSET, stable under inserts.
- [ ] **32.** `FilteredListView` base: a `filters = {param: lookup}` dict drives filtering; use it for calls.
- [ ] **33.** `paginate_orphans = 5` and see the last page absorb a short tail.
- [ ] **34.** `ArchiveIndexView` + `DayArchiveView` for calls: `/leads/calls/archive/` and `/leads/calls/2026/10/15/`.
- [ ] **35.** `MonthArchiveView` for deposits with a month total.

- [ ] **36.** Check a paginated page runs the same number of queries on page 1 and page 3.
- [ ] **37.** Test: page 1 has 25 leads and a next page.
- [ ] **38.** Test: the next-page link keeps `q`.
- [ ] **39.** Test: detail page Older / Newer leads match the model methods.
- [ ] **40.** Test: two keyset pages don't overlap.

Solutions

A + C. Leads

1–2, 5, 6, 8–9, 15, 21, 24–25, 30.

```
# leads/views.py
from django.core.paginator import Paginator
from django.views.generic import DetailView, ListView


@method_decorator(log_request, name="dispatch")
class LeadListView(PageTitleMixin, LeadQueryMixin, ListView):           # 1–
2, 5, 21, 30
    template_name = "leads/lead_list.html"
    context_object_name = "leads"          # the current page's rows
    paginate_by = 25
    page_title = "Leads"
    PER_PAGE = {10, 25, 50, 100}

    def get_queryset(self):
        return self.get_leads()            # filters, sort, annotations,
select/prefetch from Days 7–10

    def get_paginate_by(self, queryset):                                  # 5
        value = self.request.GET.get("per_page", "")
        return int(value) if value.isdigit() and int(value) in self.PER_PAGE
else self.paginate_by

    def get_template_names(self):                                         #
30
        if self.request.GET.get("partial") == "1":
            return ["leads/partials/_lead_rows.html"]
        if self.request.GET.get("compact") == "1":
            return ["leads/lead_list_compact.html"]
        return [self.template_name]
```

```
class LeadDetailView(PageTitleMixin, DetailView):                          # 6,
8-9, 15, 24-25
    template_name = "leads/lead_detail.html"
    context_object_name = "lead"
    pk_url_kwarg = "lead_id"               # URL is <int:lead_id>, not
<int:pk>
    page_title = "Lead"

    def get_queryset(self):                                                 # 9
        return (Lead.objects.filter(is_active=True)
                .select_related("assigned_to", "broker", "client")
                .prefetch_related("tags"))

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        lead = self.object

        recent = [pk for pk in self.request.session.get("recent_leads", [])
if pk != lead.pk]   # 25
        found = Lead.objects.only("id", "name").in_bulk(recent)
        context["recent_leads"] = [found[pk] for pk in recent if pk in found]
        self.request.session["recent_leads"] = [lead.pk, *recent][:5]

        calls = lead.calls.select_related("agent")                          #
15
        context["calls_page"] = Paginator(calls,
10).get_page(self.request.GET.get("calls_page"))

        context["older_lead"] =
self._neighbour(lead.get_previous_by_created_at)   # 24
        context["newer_lead"] = self._neighbour(lead.get_next_by_created_at)
        return context

    @staticmethod
    def _neighbour(method):
        try:
            return method(is_active=True)      # extra kwargs filter the
neighbour search
        except Lead.DoesNotExist:
            return None
```

The Day 3 "By status" `{% regroup %}` block on the lead list now only sees one page — delete it.

3-4, 22-23.

```
<!-- templates/partials/_pagination.html -->
{% if page_obj.paginator.num_pages > 1 %}
<nav class="pagination">
  {% if page_obj.has_previous %}
    <a href="{% querystring page=1 %}">« First</a>
    <a href="{% querystring page=page_obj.previous_page_number %}">‹ Prev</a>
  {% endif %}
  <span>Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}
</span>
  {% if page_obj.has_next %}
    <a href="{% querystring page=page_obj.next_page_number %}">Next ›</a>
    <a href="{% querystring page=page_obj.paginator.num_pages %}">Last »</a>
  {% endif %}
</nav>
{% endif %}
{% if page_obj.paginator.count %}
  <p class="muted">Showing {{ page_obj.start_index }}–{{ page_obj.end_index
}} of {{ page_obj.paginator.count }}</p>
{% endif %}
```

`{% querystring page=3 %}` copies the current `request.GET`, sets `page`, and outputs `?q=khan&status=new&page=3`.

```
<!-- leads/lead_list.html — table body + pagination -->
<tbody>
  {% include "leads/partials/_lead_rows.html" %}
</tbody>
</table>
{% include "partials/_pagination.html" %}
```

```
<!-- leads/templates/leads/partials/_lead_rows.html  (30: returned alone for
?partial=1) -->
{% load crm_filters %}
{% for lead in leads %}
<tr class="{% cycle 'row-light' 'row-dark' %}">
  <td><a href="{{ lead.get_absolute_url }}">{{ lead.name }}</a></td>
  <td>{{ lead.phone|mask_phone }}</td>
  <td>{% include "partials/_badge.html" with status=lead.status only %}</td>
  <td>{{ lead.get_source_display }}</td>
  <td>{{ lead.assigned_to|default:"—" }}</td>
  <td>{% for tag in lead.tags.all %}<span class="badge">{{ tag }}</span> {%
endfor %}</td>
  <td>{{ lead.call_count }}</td>
  <td>{{ lead.last_call_at|date:"d M H:i"|default:"—" }}</td>
</tr>
{% empty %}
<tr><td colspan="8">No leads match.</td></tr>
{% endfor %}
```

```
<!-- leads/lead_detail.html — additions (15, 24, 25) -->
<p>
  {% if older_lead %}<a href="{{ older_lead.get_absolute_url }}">‹ Older: {{
older_lead.name }}</a>{% endif %}
  {% if newer_lead %}<a href="{{ newer_lead.get_absolute_url }}">Newer: {{
newer_lead.name }} ›</a>{% endif %}
</p>

<h2>Calls ({{ calls_page.paginator.count }})</h2>
<table class="table">
  {% for call in calls_page %}
    <tr><td>{{ call.started_at|date:"d M H:i" }}</td><td>{{ call.agent }}
</td>
        <td>{{ call.get_outcome_display }}</td><td>{{ call.duration_seconds
}}s</td></tr>
  {% empty %}<tr><td colspan="4">No calls yet.</td></tr>{% endfor %}
</table>
{% if calls_page.has_previous %}<a href="{% querystring
calls_page=calls_page.previous_page_number %}">‹ Newer calls</a>{% endif %}
{% if calls_page.has_next %}<a href="{% querystring
calls_page=calls_page.next_page_number %}">Older calls ›</a>{% endif %}

<aside>
```

```
  <h3>Recently viewed</h3>
  <ul>{% for r in recent_leads %}<li><a href="{{ r.get_absolute_url }}">{{
r.name }}</a></li>{% empty %}<li>—</li>{% endfor %}</ul>
</aside>
```

10.

```
# python manage.py shell
from django.core.paginator import EmptyPage, Paginator
from leads.models import Lead

p = Paginator(Lead.objects.all(), 25)
p.count, p.num_pages, list(p.page_range)
page = p.page(2)
page.has_next(), page.start_index(), page.end_index(), len(page.object_list)
p.get_page("abc").number          # 1 — junk falls back to the first page
p.get_page(999).number            # last page
try:
    p.page(999)                   # strict version
except EmptyPage:
    print("no such page")
list(p.get_elided_page_range(5, on_each_side=1, on_ends=1))   # [1, '…', 4,
5, 6, '…', 9]
```

Brokers (7, 27)

```
# brokers/views.py
from django.views.generic import DetailView, TemplateView
from .tree import ancestors_of, load_tree, walk


class BrokerDetailView(PageTitleMixin, DetailView):                    # 7
    model = Broker                    # context name defaults to "broker"
    slug_field = "code"               # model field
    slug_url_kwarg = "code"           # URL kwarg
    template_name = "brokers/broker_detail.html"
    page_title = "Broker"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        by_id, children = load_tree()
        broker = self.object
        context.update(
            ancestors=ancestors_of(broker, by_id)[::-1],
            children=children[broker.pk],
            descendant_count=sum(1 for _ in walk(children, broker.pk)),
```

```
            client_count=broker.clients.count(),
        )
        return context


class BrokerListView(PageTitleMixin, TemplateView):                    # 27
    template_name = "brokers/broker_list.html"
    page_title = "Brokers"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        _, children = load_tree()
        context["rows"] = list(walk(children))      # [(broker, depth), ...]
        return context

# brokers/urls.py
urlpatterns = [
    path("", views.BrokerListView.as_view(), name="list"),
    path("<str:code>/", views.BrokerDetailView.as_view(), name="detail"),
]
```

```
<!-- brokers/templates/brokers/broker_list.html -->
{% extends "base.html" %}
{% block content %}
<h1>Broker network</h1>
{% for broker, depth in rows %}
  <div class="depth-{{ depth }}">
    <a href="{% url 'brokers:detail' broker.code %}">{{ broker }}</a> · Tier
{{ broker.tier }} · {{ broker.commission_rate }}%
  </div>
{% endfor %}
{% endblock content %}
```

```
.depth-1 { padding-left: 1.5rem; }
.depth-2 { padding-left: 3rem; }
.depth-3 { padding-left: 4.5rem; }
```

B. Clients, agents, calls

11–12, 19, 26.

```
# clients/views.py
from django.db.models import Q
from django.views.generic import DetailView, ListView


```

```
from core.mixins import PageTitleMixin
from .models import Client
from .queries import deposits_total


class ClientListView(PageTitleMixin, ListView):                              #
11, 19, 26
    context_object_name = "clients"
    paginate_by = 25
    page_title = "Clients"
    extra_context = {"kyc_choices": Client.KYCStatus.choices}
    # no template_name → Django uses clients/client_list.html
(<app>/<model>_list.html)

    def get_queryset(self):
        qs =
Client.objects.select_related("broker").annotate(total_deposits=deposits_tota
l())
        p = self.request.GET
        if kyc := p.get("kyc"):
            qs = qs.filter(kyc_status=kyc)
        if q := p.get("q", "").strip():
            qs = qs.filter(
                Q(name__icontains=q) | Q(account_number__startswith=q) |
Q(accounts__number=q)
            ).distinct()
        return qs.order_by("name")


class ClientDetailView(PageTitleMixin, DetailView):                          # 12
    slug_field = "ref"
    slug_url_kwarg = "ref"
    context_object_name = "client"
    template_name = "clients/client_detail.html"
    page_title = "Client"

    def get_queryset(self):
        return Client.objects.select_related("broker",
"lead").prefetch_related("accounts__deposits")

# clients/urls.py
path("", views.ClientListView.as_view(), name="list"),
path("<uuid:ref>/", views.ClientDetailView.as_view(), name="detail"),
```

```
<!-- clients/templates/clients/client_list.html -->
{% extends "base.html" %}
{% block content %}
<h1>Clients</h1>
<form method="get">
  <input name="q" value="{{ request.GET.q }}" placeholder="Name or account
number">
  <select name="kyc">
    <option value="">Any KYC</option>
    {% for value, label in kyc_choices %}
      <option value="{{ value }}"{% if request.GET.kyc == value %} selected{%
endif %}>{{ label }}</option>
    {% endfor %}
  </select>
  <button type="submit">Filter</button>
</form>
<table class="table">
  <tr><th>Name</th><th>Client no.</th><th>KYC</th><th>Broker</th>
<th>Deposits</th></tr>
  {% for c in clients %}
  <tr>
    <td><a href="{{ c.get_absolute_url }}">{{ c.name }}</a></td><td>{{
c.account_number }}</td>
    <td>{{ c.get_kyc_status_display }}</td><td>{{ c.broker|default:"Direct"
}}</td>
    <td>{{ c.total_deposits|floatformat:2 }}</td>
  </tr>
  {% empty %}<tr><td colspan="5">No clients.</td></tr>{% endfor %}
</table>
{% include "partials/_pagination.html" %}
{% endblock content %}
```

13, 28.

```
# accounts/views.py
from django.db.models import F, Q
from django.views.generic import DetailView, ListView
from leads.queries import follow_up_due, stale_new_leads


class AgentDetailView(PageTitleMixin, DetailView):                        # 13
    template_name = "accounts/agent_detail.html"
    context_object_name = "agent"
    page_title = "Agent"

    def get_queryset(self):            # non-agents → 404
```

```
        return
get_user_model().objects.filter(role="agent").select_related("team_lead")

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["leads"] = self.object.leads.order_by("status", "-
created_at")
        return context


class AgentQueueView(PageTitleMixin, ListView):                          # 28
    template_name = "accounts/agent_queue.html"
    context_object_name = "leads"
    paginate_by = 20
    page_title = "Call queue"

    def get_queryset(self):
        self.agent = get_object_or_404(get_user_model(),
pk=self.kwargs["pk"], role="agent")
        due = follow_up_due().values("pk")
        stale = stale_new_leads(days=1).values("pk")
        return (self.agent.leads
                .filter(Q(pk__in=due) | Q(pk__in=stale))
                .order_by(F("last_contacted_at").asc(nulls_first=True),
"created_at"))

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["agent"] = self.agent
        return context

# accounts/urls.py
path("agents/<int:pk>/", views.AgentDetailView.as_view(),
name="agent_detail"),
path("agents/<int:pk>/queue/", views.AgentQueueView.as_view(),
name="agent_queue"),
```

14, 16, 18, 20, 32–33.

```
# core/views_generic.py                                                 # 32
from django.core.exceptions import ValidationError
from django.views.generic import ListView


class FilteredListView(ListView):
    filters = {}                        # {"GET param": "ORM lookup"}

    def get_queryset(self):
        qs = super().get_queryset()    # applies get_ordering()
        for param, lookup in self.filters.items():
            if value := self.request.GET.get(param):
                try:
                    qs = qs.filter(**{lookup: value})
                except (ValueError, ValidationError):   # e.g. agent=abc,
day=not-a-date
                    pass
        return qs
```

```
# leads/views.py
from core.views_generic import FilteredListView


class CallLogListView(PageTitleMixin, FilteredListView):                # 14
    queryset = CallLog.objects.select_related("lead", "agent")           #
20: cloned per request
    template_name = "leads/call_list.html"
    context_object_name = "calls"
    paginate_by = 50
    paginate_orphans = 5               # 33: a final page of ⩽ 5 rows merges
into the previous one
    ordering = "-started_at"
    page_title = "Calls"
    filters = {"outcome": "outcome", "agent": "agent_id", "day":
"started_at__date"}
    SORTABLE = {"started_at", "duration_seconds", "outcome"}

    def get_ordering(self):                                              #
18
        sort = self.request.GET.get("sort", "")
        return sort if sort.lstrip("-") in self.SORTABLE else self.ordering
```

```
# leads/urls.py
path("calls/", views.CallLogListView.as_view(), name="calls"),
path("calls/strict/", views.CallLogListView.as_view(allow_empty=False),
name="calls_strict"),   # 16
```

```
<!-- leads/templates/leads/call_list.html -->
{% extends "base.html" %}
{% block content %}
<h1>Calls</h1>
<form method="get">
  <select name="outcome">
    <option value="">Any outcome</option>
    <option value="answered">Answered</option><option value="no_answer">No
answer</option>
    <option value="busy">Busy</option><option
value="voicemail">Voicemail</option>
  </select>
  <input type="date" name="day" value="{{ request.GET.day }}">
  <button type="submit">Filter</button>
</form>
<table class="table">
  <tr><th><a href="{% querystring sort='-started_at' %}">When</a></th>
<th>Lead</th><th>Agent</th>
      <th>Outcome</th><th><a href="{% querystring sort='-duration_seconds'
%}">Talk</a></th></tr>
  {% for call in calls %}
  <tr><td>{{ call.started_at|date:"d M H:i" }}</td>
      <td><a href="{{ call.lead.get_absolute_url }}">{{ call.lead.name }}</a>
</td>
      <td>{{ call.agent }}</td><td>{{ call.get_outcome_display }}</td><td>{{
call.duration_seconds }}s</td></tr>
  {% empty %}<tr><td colspan="5">No calls.</td></tr>{% endfor %}
</table>
{% include "partials/_pagination.html" %}
{% endblock content %}
```

17. `/leads/?page=999` → 404 (`ListView` raises it for out-of-range pages). `/leads/?page=last` → last page — "last" is a built-in keyword.

C. JSON endpoints (29, 31)

```
# leads/views.py
from django.utils.dateparse import parse_datetime
```

```
class LeadPageAPI(LeadQueryMixin, JsonResponseMixin, View):              # 29
    def get(self, request):
        paginator = Paginator(self.get_leads().values(*LEAD_FIELDS), 25)
        page = paginator.get_page(request.GET.get("page"))
        return self.render_to_json({
            "page": page.number,
            "num_pages": paginator.num_pages,
            "count": paginator.count,
            "has_next": page.has_next(),
            "results": list(page.object_list),
        })


class CallFeedAPI(JsonResponseMixin, View):                              # 31
    page_size = 50

    def get(self, request):
        qs = CallLog.objects.select_related("lead", "agent").order_by("-
started_at", "-id")
        if after := request.GET.get("after"):
            ts_raw, _, last_id = after.rpartition(",")
            ts = parse_datetime(ts_raw)
            if ts is None or not last_id.isdigit():
                return json_error("bad cursor")
            # rows strictly after the cursor in (started_at DESC, id DESC)
order
            qs = qs.filter(Q(started_at__lt=ts) | Q(started_at=ts,
id__lt=int(last_id)))
        rows = list(qs[: self.page_size + 1])          # fetch one extra to
know if there's more
        has_more = len(rows) > self.page_size
        rows = rows[: self.page_size]
        last = rows[-1] if rows else None
        return self.render_to_json({
            "results": [
                {"id": c.id, "lead": c.lead.name, "agent": str(c.agent),
                 "outcome": c.outcome, "started_at": c.started_at}
                for c in rows
            ],
            "next": f"{last.started_at.isoformat()},{last.id}" if has_more
else None,
        })

# leads/urls.py
path("api/page/", views.LeadPageAPI.as_view(), name="api_page"),
path("api/calls/", views.CallFeedAPI.as_view(), name="call_feed"),
```

URL-encode the cursor when you send it — the `+` in `+00:00` would otherwise arrive as a space (31).

D. Date-based views (34–35)

```
# leads/views.py                                                    # 34
from django.views.generic.dates import ArchiveIndexView, DayArchiveView


class CallArchiveIndexView(PageTitleMixin, ArchiveIndexView):
    queryset = CallLog.objects.select_related("lead", "agent")
    date_field = "started_at"
    date_list_period = "day"          # date_list = days that have calls
    paginate_by = 50
    template_name = "leads/call_archive.html"
    page_title = "Call archive"


class CallDayArchiveView(PageTitleMixin, DayArchiveView):
    queryset = CallLog.objects.select_related("lead", "agent")
    date_field = "started_at"
    month_format = "%m"               # numeric months in the URL
    allow_empty = True
    template_name = "leads/call_archive_day.html"
    page_title = "Calls by day"

# leads/urls.py
path("calls/archive/", views.CallArchiveIndexView.as_view(),
name="call_archive"),
path("calls/<int:year>/<int:month>/<int:day>/",
views.CallDayArchiveView.as_view(), name="call_day"),
```

```
<!-- leads/templates/leads/call_archive.html -->
{% extends "base.html" %}
{% block content %}
<h1>Call archive</h1>
<p>{% for d in date_list %}<a href="{% url 'leads:call_day' d.year d.month
d.day %}">{{ d|date:"d M" }}</a> {% endfor %}</p>
<ul>{% for call in latest %}<li>{{ call.started_at|date:"d M H:i" }} · {{
call.lead.name }} · {{ call.get_outcome_display }}</li>{% endfor %}</ul>
{% include "partials/_pagination.html" %}
{% endblock content %}
```

```
<!-- leads/templates/leads/call_archive_day.html -->
{% extends "base.html" %}
{% block content %}
<h1>Calls on {{ day|date:"D d M Y" }} ({{ object_list|length }})</h1>
<p>
  {% if previous_day %}<a href="{% url 'leads:call_day' previous_day.year
previous_day.month previous_day.day %}">‹ {{ previous_day|date:"d M" }}</a>{%
endif %}
  {% if next_day %}<a href="{% url 'leads:call_day' next_day.year
next_day.month next_day.day %}">{{ next_day|date:"d M" }} ›</a>{% endif %}
</p>
<ul>{% for call in object_list %}<li>{{ call.started_at|date:"H:i" }} · {{
call.lead.name }} · {{ call.agent }} · {{ call.get_outcome_display }}</li>{%
endfor %}</ul>
{% endblock content %}
```

```
# clients/views.py                                                    # 35
from django.db.models import Sum
from django.views.generic.dates import MonthArchiveView
from .models import Deposit


class DepositMonthView(PageTitleMixin, MonthArchiveView):
    queryset = Deposit.objects.select_related("account__client")
    date_field = "created_at"
    month_format = "%m"
    allow_empty = True
    template_name = "clients/deposit_month.html"
    page_title = "Deposits"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["total"] = self.object_list.aggregate(t=Sum("amount",
default=0))["t"]
        return context

# clients/urls.py
path("deposits/<int:year>/<int:month>/", views.DepositMonthView.as_view(),
name="deposit_month"),
```

```
<!-- clients/templates/clients/deposit_month.html -->
{% extends "base.html" %}
{% block content %}
<h1>Deposits · {{ month|date:"F Y" }} · {{ total|floatformat:2 }}</h1>
<p>
  {% if previous_month %}<a href="{% url 'clients:deposit_month'
previous_month.year previous_month.month %}">‹ {{ previous_month|date:"M Y"
}}</a>{% endif %}
  {% if next_month %}<a href="{% url 'clients:deposit_month' next_month.year
next_month.month %}">{{ next_month|date:"M Y" }} ›</a>{% endif %}
</p>
<table class="table">
  {% for d in object_list %}
    <tr><td>{{ d.created_at|date:"d M" }}</td><td>{{ d.account.client.name }}
</td><td>{{ d.get_method_display }}</td><td>{{ d.amount|floatformat:2 }}</td>
</tr>
  {% empty %}<tr><td colspan="4">No deposits this month.</td></tr>{% endfor
%}
</table>
{% endblock content %}
```

36.

```
from django.db import connection
from django.test import Client as HttpClient
from django.test.utils import CaptureQueriesContext, setup_test_environment

setup_test_environment()
http = HttpClient()
for page in (1, 3):
    with CaptureQueriesContext(connection) as ctx:
        http.get("/leads/", {"page": page})
    print(page, len(ctx.captured_queries))     # same number: COUNT + page
rows + tags prefetch
```

Tests (37–40)

```
# leads/tests.py
import re
from datetime import timedelta


class ListDetailPaginationTests(TestCase):
    @classmethod
    def setUpTestData(cls):
```

```
        Lead.objects.bulk_create([
            Lead(name=f"Page Lead {i}", phone=f"9715099{i:05d}") for i in
range(60)
        ])

    def test_first_page(self):                                          # 37
        response = self.client.get(reverse("leads:list"))
        self.assertEqual(len(response.context["leads"]), 25)
        self.assertTrue(response.context["page_obj"].has_next())

    def test_next_link_keeps_query(self):                               # 38
        response = self.client.get(reverse("leads:list"), {"q": "page lead"})
        self.assertRegex(response.content.decode(),
r"q=page\+lead(&amp;|&)page=2")

    def test_older_newer_links(self):                                   # 39
        leads = list(Lead.objects.order_by("created_at", "pk"))
        middle = leads[len(leads) // 2]
        response = self.client.get(middle.get_absolute_url())
        self.assertEqual(response.context["older_lead"],
middle.get_previous_by_created_at(is_active=True))
        self.assertEqual(response.context["newer_lead"],
middle.get_next_by_created_at(is_active=True))


class CallFeedTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        agent = get_user_model().objects.create_user("feed_agent",
password="x", role="agent")
        lead = Lead.objects.create(name="Feed", phone="971509990401")
        base = timezone.now() - timedelta(days=1)
        CallLog.objects.bulk_create([
            CallLog(lead=lead, agent=agent, outcome="busy", started_at=base +
timedelta(minutes=i))
            for i in range(120)
        ])

    def test_keyset_pages_do_not_overlap(self):                         # 40
        first = self.client.get(reverse("leads:call_feed")).json()
        second = self.client.get(reverse("leads:call_feed"), {"after":
first["next"]}).json()  # dict → URL-encoded
        ids1 = {r["id"] for r in first["results"]}
```

```
        ids2 = {r["id"] for r in second["results"]}
        self.assertEqual(len(ids1), 50)
        self.assertEqual(len(ids2), 50)
        self.assertFalse(ids1 & ids2)
```

The Day 7 sort test now sees one page: compare against `expected[:25]`.

```
python manage.py test
git add -A && git commit -m "day 12"
```

Day 13 — Forms and validation

Goal: every input into the CRM goes through a Django form — normalised, validated, with friendly errors — and the lead list filters run off a form instead of raw `request.GET`.

Files: `leads/forms.py`, `leads/validators.py`, `clients/forms.py`. Do A in the shell.

A. Drills (1–10) — `forms.Form`

- [ ] **1.** `LeadSearchForm` (q, status, source — all optional, blank choice first). Print it, `as_div()`, and one field.
- [ ] **2.** Bind it to data: `is_bound`, `is_valid()`, `cleaned_data` (note the stripped `q`).
- [ ] **3.** `FieldTypesDemoForm` with CharField, EmailField, IntegerField (0–10), DecimalField, DateField, BooleanField, ChoiceField, MultipleChoiceField.
- [ ] **4.** Invalid data: read `errors`, `errors.as_json()`, one field's errors.
- [ ] **5.** Field options: `label`, `help_text`, `initial`, widget `attrs` (placeholder, type="date").
- [ ] **6.** `ContactForm.clean_phone()` normalising `+971 50 ...` to `9715...`.
- [ ] **7.** `DateRangeForm.clean()`: start must not be after end.
- [ ] **8.** Attach that error to `date_to` with `add_error()`.
- [ ] **9.** Reusable validator `validate_uae_mobile` with an error `code` and `params`.
- [ ] **10.** `has_changed()` and `changed_data` against `initial`.

B. Core (11–20) — `ModelForm`

- [ ] **11.** `LeadForm(ModelForm)` with an explicit field list: name, phone, email, source, assigned_to, expected_deposit, tags, notes.
- [ ] **12.** `Meta.widgets`, `labels`, `help_texts` (tags as checkboxes, notes 3 rows).
- [ ] **13.** Why never `fields = "__all__"` on user-facing forms — list what it would expose.
- [ ] **14.** Custom message for the duplicate-phone error via `Meta.error_messages`.
- [ ] **15.** `LeadStatusForm` (status + notes) — watch `Lead.clean()` (lost needs notes) fire through the form.
- [ ] **16.** `LeadForm.clean_phone()` normalising before the uniqueness check runs.
- [ ] **17.** `save(commit=False)` → set `assigned_to` → `save()` → `save_m2m()`.
- [ ] **18.** Edit with `instance=`: the unique check skips the row itself.
- [ ] **19.** In `__init__`, limit the agent dropdown to active agents and make it optional.

- [ ] **20.** `__init__(..., user=None)`: agents don't see `expected_deposit` or `assigned_to`.

C. Nexora CRM (21–30)

- [ ] **21.** Function view `lead_create` (GET empty form / POST validate → save → redirect) at `/leads/new/`.
- [ ] **22.** `leads/lead_form.html`: CSRF token, non-field errors, hidden fields, each visible field with `as_field_group`.
- [ ] **23.** Error styling: red border and message on fields with errors.
- [ ] **24.** `lead_edit` at `/leads/<id>/edit/` reusing the same form and template.
- [ ] **25.** `LogCallForm` (outcome, talk seconds, notes; talk time only for answered calls) shown on lead detail and posted to `/leads/<id>/log-call/`, saved with Day 10 `log_call()`.
- [ ] **26.** `DepositForm`: amount > 0, ⩽ 1,000,000, crypto capped at 50,000 (SOP rule).
- [ ] **27.** `ConvertLeadForm` (platform, currency, leverage) + `/leads/<id>/convert/` calling `convert_lead_to_client` (add a `leverage` argument to it).
- [ ] **28.** `LeadFilterForm` (q, status, source, agent, tag, finder, deposit range, date range, sort) with cross-field checks; `LeadQueryMixin` reads `cleaned_data`.
- [ ] **29.** Render the filter form above the lead list (GET, values kept, Reset link).
- [ ] **30.** Quick-add 5 leads at once with `formset_factory`; reject the same phone twice in one batch.

D. Challenge (31–40)

- [ ] **31.** `inlineformset_factory(Client, TradingAccount)` page to edit a client's accounts.
- [ ] **32.** Custom field `UAEPhoneField(forms.CharField)`: `to_python` normalises, validator checks.
- [ ] **33.** Custom widget `ColorInput` (`input_type = "color"`) in a `TagForm`.
- [ ] **34.** Dynamic form: `TagPickerForm` builds one checkbox per tag in `__init__`.
- [ ] **35.** `clean_email()`: no other active lead with the same email (case-insensitive), excluding itself.
- [ ] **36.** `Meta.field_classes` so `LeadForm.phone` uses `UAEPhoneField` automatically (replaces 16).
- [ ] **37.** Test: `LeadForm` accepts `+971 50 123 9999` and saves `971501239999`.
- [ ] **38.** Test: duplicate phone shows your custom message.

- [ ] **39.** Test: `DateRangeForm` rejects from > to.
- [ ] **40.** Test: `DepositForm` crypto cap error.

Solutions

Validation order for one form: each field's `clean()` (to_python → validate → validators) → `clean_<field>()` → form `clean()` → for a ModelForm, the model's `full_clean()` (field validators, `Model.clean()`, unique and constraint checks).

Validator + custom field (9, 32)

```
# leads/validators.py
import re
from django.core.exceptions import ValidationError

UAE_MOBILE = re.compile(r"^9715\d{8}$")


def validate_uae_mobile(value):
    if not UAE_MOBILE.match(value or ""):
        raise ValidationError(
            "%(value)s is not a UAE mobile (9715XXXXXXXX).",
            code="invalid_uae_mobile",
            params={"value": value},
        )
```

```
# leads/fields.py
from django import forms
from .models import normalize_uae_phone
from .validators import validate_uae_mobile


class UAEPhoneField(forms.CharField):
    default_validators = [validate_uae_mobile]

    def __init__(self, **kwargs):
        kwargs["max_length"] = 20          # room for "+971 50 123 4567"
before normalising
        kwargs.setdefault("widget", forms.TextInput(attrs={"placeholder":
"+971 50 123 4567", "inputmode": "tel"}))
        super().__init__(**kwargs)

    def to_python(self, value):
        value = super().to_python(value)   # strips whitespace
        return normalize_uae_phone(value) if value else value
```

Final `leads/forms.py` (1–8, 10–20, 25, 28, 30, 33–36)

```
# leads/forms.py
from django import forms
from django.contrib.auth import get_user_model
from django.core.exceptions import ValidationError

from clients.models import TradingAccount
from .fields import UAEPhoneField
from .models import CallLog, Lead, Tag, normalize_uae_phone
from .queries import FINDERS
from .validators import validate_uae_mobile

DATE = forms.DateInput(attrs={"type": "date"})


# ---------- A: plain forms ----------------------------------------------
---
class LeadSearchForm(forms.Form):
# 1, 5
    q = forms.CharField(required=False, label="Search",
                        widget=forms.TextInput(attrs={"placeholder": "Name,
email or phone"}))
    status = forms.ChoiceField(required=False, choices=[("", "Any status"),
*Lead.Status.choices])
```

```
    source = forms.ChoiceField(required=False, choices=[("", "Any source"),
*Lead.Source.choices])


class FieldTypesDemoForm(forms.Form):
# 3
    name = forms.CharField(max_length=50, help_text="Full name as on ID")
    email = forms.EmailField()
    attempts = forms.IntegerField(min_value=0, max_value=10, initial=0)
    deposit = forms.DecimalField(max_digits=12, decimal_places=2,
min_value=0)
    callback_on = forms.DateField(widget=DATE)
    vip = forms.BooleanField(required=False)
    source = forms.ChoiceField(choices=Lead.Source.choices)
    channels = forms.MultipleChoiceField(
        choices=[("call", "Call"), ("sms", "SMS"), ("email", "Email")],
        widget=forms.CheckboxSelectMultiple,
    )


class ContactForm(forms.Form):
# 6
    name = forms.CharField(max_length=120)
    phone = forms.CharField(max_length=20)

    def clean_phone(self):
        phone = normalize_uae_phone(self.cleaned_data["phone"])
        validate_uae_mobile(phone)
# 9
        return phone                        # the returned value replaces
cleaned_data["phone"]


class DateRangeForm(forms.Form):
# 7-8
    date_from = forms.DateField(required=False, widget=DATE)
    date_to = forms.DateField(required=False, widget=DATE)

    def clean(self):
        cleaned = super().clean()
        start, end = cleaned.get("date_from"), cleaned.get("date_to")   #
.get: a field may have failed
        if start and end and start > end:
            self.add_error("date_to", "End date must be on or after the start
date.")
            # non-field version: raise ValidationError("...")
```

```
        return cleaned


# ---------- B: model forms ----------------------------------------------
---
class LeadForm(forms.ModelForm):
    class Meta:
        model = Lead
        fields = ["name", "phone", "email", "source", "assigned_to",
# 11, 13
                  "expected_deposit", "tags", "notes"]
        field_classes = {"phone": UAEPhoneField}
# 36
        widgets = {
# 12
            "notes": forms.Textarea(attrs={"rows": 3}),
            "expected_deposit": forms.NumberInput(attrs={"step": "500",
"min": "0"}),
            "tags": forms.CheckboxSelectMultiple,
        }
        labels = {"expected_deposit": "Expected first deposit (USD)"}
        help_texts = {"email": "Optional — used for KYC and statements."}
        error_messages = {"phone": {"unique": "A lead with this phone number
already exists."}}   # 14

    def __init__(self, *args, user=None, **kwargs):
# 19-20
        super().__init__(*args, **kwargs)
        self.fields["assigned_to"].queryset =
get_user_model().objects.filter(role="agent", is_active=True)
        self.fields["assigned_to"].required = False
        if user is not None and getattr(user, "role", "") == "agent":
            del self.fields["expected_deposit"]
            del self.fields["assigned_to"]

    def clean_name(self):
        return self.cleaned_data["name"].strip().title()

    def clean_email(self):
# 35
        email = self.cleaned_data.get("email", "").strip().lower()
        if email:
            clash = Lead.objects.filter(email__iexact=email, is_active=True)
            if self.instance.pk:
                clash = clash.exclude(pk=self.instance.pk)
            if clash.exists():
```

```
                raise ValidationError("Another active lead already uses this
email.")
        return email

    def clean_expected_deposit(self):
        value = self.cleaned_data.get("expected_deposit")
        if value is not None and value < 0:
            raise ValidationError("Deposit can't be negative.")
        return value


class LeadStatusForm(forms.ModelForm):
# 15
    class Meta:
        model = Lead
        fields = ["status", "notes"]
        widgets = {"notes": forms.Textarea(attrs={"rows": 2})}
    # Lead.clean() runs during is_valid(): "lost" without notes → error on
the notes field


class LogCallForm(forms.ModelForm):
# 25
    class Meta:
        model = CallLog
        fields = ["outcome", "duration_seconds", "notes"]
        labels = {"duration_seconds": "Talk time (seconds)"}
        widgets = {"notes": forms.Textarea(attrs={"rows": 2})}

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.fields["duration_seconds"].required = False

    def clean_duration_seconds(self):
        return self.cleaned_data.get("duration_seconds") or 0

    def clean(self):
        cleaned = super().clean()
        if cleaned.get("outcome") != CallLog.Outcome.ANSWERED and
cleaned.get("duration_seconds"):
            self.add_error("duration_seconds", "Only answered calls have talk
time.")
        return cleaned


class ConvertLeadForm(forms.Form):
```

```
# 27
    platform = forms.ChoiceField(choices=TradingAccount.Platform.choices,
initial="mt5")
    currency = forms.ChoiceField(choices=TradingAccount.Currency.choices,
initial="USD")
    leverage = forms.TypedChoiceField(
        choices=[(50, "1:50"), (100, "1:100"), (200, "1:200"), (500,
"1:500")],
        coerce=int, initial=100,
    )

    def __init__(self, *args, lead=None, **kwargs):
        self.lead = lead
        super().__init__(*args, **kwargs)

    def clean(self):
        cleaned = super().clean()
        if self.lead is not None and hasattr(self.lead, "client"):
            raise ValidationError("This lead is already a client.")
        return cleaned


# ---------- C: filter form ----------------------------------------------
---
class LeadFilterForm(forms.Form):
# 28
    SORT_CHOICES = [
        ("-created_at", "Newest"), ("created_at", "Oldest"), ("name", "Name
A–Z"),
        ("-expected_deposit", "Deposit high → low"), ("-call_count", "Most
calls"),
        ("-last_call_at", "Recently called"),
    ]
    q = forms.CharField(required=False, label="Search")
    status = forms.ChoiceField(required=False, choices=[("", "Any status"),
*Lead.Status.choices])
    source = forms.ChoiceField(required=False, choices=[("", "Any source"),
*Lead.Source.choices])
    agent = forms.ModelChoiceField(required=False, empty_label="Any agent",

queryset=get_user_model().objects.filter(role="agent"))
    tag = forms.ModelChoiceField(required=False, empty_label="Any tag",
                                 queryset=Tag.objects.all(),
to_field_name="name")   # ?tag=vip keeps working
    finder = forms.ChoiceField(required=False,
                               choices=[("", "—"), *[(k, k.replace("_", "
```

```
").title()) for k in FINDERS]])
    min_deposit = forms.DecimalField(required=False, min_value=0)
    max_deposit = forms.DecimalField(required=False, min_value=0)
    created_from = forms.DateField(required=False, widget=DATE)
    created_to = forms.DateField(required=False, widget=DATE)
    sort = forms.ChoiceField(required=False, choices=SORT_CHOICES)

    def clean(self):
        cleaned = super().clean()
        lo, hi = cleaned.get("min_deposit"), cleaned.get("max_deposit")
        if lo is not None and hi is not None and lo > hi:
            self.add_error("max_deposit", "Max must be at least the min.")
        start, end = cleaned.get("created_from"), cleaned.get("created_to")
        if start and end and start > end:
            self.add_error("created_to", "End date must be on or after the
start date.")
        return cleaned


# ---------- C/D: formset, tag forms ----------------------------------------
-----
class QuickLeadForm(forms.Form):
# 30
    name = forms.CharField(max_length=120)
    phone = UAEPhoneField()
    source = forms.ChoiceField(choices=Lead.Source.choices,
initial=Lead.Source.MANUAL)


class BaseQuickLeadFormSet(forms.BaseFormSet):
    def clean(self):
        if any(self.errors):
            return
        phones = [f.cleaned_data["phone"] for f in self.forms if
f.cleaned_data]
        if len(phones) != len(set(phones)):
            raise ValidationError("The same phone appears twice in this
batch.")   # → non_form_errors


QuickLeadFormSet = forms.formset_factory(QuickLeadForm,
formset=BaseQuickLeadFormSet, extra=5)


class ColorInput(forms.TextInput):
# 33
```

```
    input_type = "color"


class TagForm(forms.ModelForm):
    class Meta:
        model = Tag
        fields = ["name", "color"]
        widgets = {"color": ColorInput}


class TagPickerForm(forms.Form):
# 34
    def __init__(self, *args, lead=None, **kwargs):
        super().__init__(*args, **kwargs)
        current = set(lead.tags.values_list("pk", flat=True)) if lead else
set()
        for tag in Tag.objects.all():
            self.fields[f"tag_{tag.pk}"] = forms.BooleanField(
                required=False, label=tag.name, initial=tag.pk in current,
            )

    def selected_tag_ids(self):
        return [int(name.removeprefix("tag_")) for name, on in
self.cleaned_data.items() if on]

# usage: form = TagPickerForm(request.POST, lead=lead); form.is_valid();
lead.tags.set(form.selected_tag_ids())
```

16 (before 36 replaced it):

```
# inside LeadForm
def clean_phone(self):
    phone = normalize_uae_phone(self.cleaned_data["phone"])
    validate_uae_mobile(phone)
    return phone          # the ModelForm's unique check then runs on the
normalised value
```

13. `"__all__"` would expose `status`, `is_active`, `contact_attempts`, `last_contacted_at`, `country` — any field you add later appears in the form automatically. Always list fields.

A + B in the shell (1–4, 10, 17–18)

```
# python manage.py shell
from accounts.models import CustomUser
from leads.forms import FieldTypesDemoForm, LeadForm, LeadSearchForm
from leads.models import Lead, Tag

form = LeadSearchForm()                                          # 1
print(form)                          # Django 5+: <div>-based rendering
print(form.as_div())
print(form["q"])

bound = LeadSearchForm({"q": "  khan ", "status": "new"})  # 2
bound.is_bound, bound.is_valid(), bound.cleaned_data
# (True, True, {'q': 'khan', 'status': 'new', 'source': ''})

FieldTypesDemoForm({"name": "Ali", "email": "ali@x.test", "attempts": 3,
"deposit": "2500.50",   # 3
                    "callback_on": "2026-10-20", "source": "google",
"channels": ["call", "sms"]}).is_valid()

bad = LeadSearchForm({"status": "won"})                          # 4
bad.is_valid()                       # False
bad.errors                           # {'status': ['Select a valid choice. won
is not one of ...']}
bad.errors.as_json()
bad["status"].errors

f = LeadSearchForm({"q": "khan", "status": "new", "source": ""}, initial=
{"status": "new"})   # 10
f.has_changed(), f.changed_data     # (True, ['q'])

vip, _ = Tag.objects.get_or_create(name="vip")                   # 17
form = LeadForm({"name": "zara ahmed", "phone": "+971 50 777 0101", "source":
"google", "tags": [vip.pk]})
form.is_valid()
lead = form.save(commit=False)      # instance built, nothing written
lead.assigned_to = CustomUser.objects.filter(role="agent").first()
lead.save()
form.save_m2m()                      # tags need the saved pk — only after
commit=False

form = LeadForm({"name": lead.name, "phone": lead.phone, "source":
lead.source}, instance=lead)   # 18
form.is_valid()                      # True — unique check excludes this row
```

C. Views and templates

21, 24–25, 27, 30.

```
# leads/views.py
from django.shortcuts import get_object_or_404, redirect, render
from django.views.decorators.http import require_POST

from clients.services import convert_lead_to_client
from .forms import ConvertLeadForm, LeadForm, LogCallForm, QuickLeadFormSet
from .services import log_call, upsert_leads


def lead_create(request):
# 21
    if request.method == "POST":
        form = LeadForm(request.POST, user=request.user)
        if form.is_valid():
            lead = form.save()
            return redirect(lead)          # → lead.get_absolute_url()
    else:
        form = LeadForm(user=request.user)
    return render(request, "leads/lead_form.html", {"form": form,
"page_title": "New lead"})


def lead_edit(request, lead_id):
# 24
    lead = get_object_or_404(Lead, pk=lead_id, is_active=True)
    form = LeadForm(request.POST or None, instance=lead, user=request.user)
    if request.method == "POST" and form.is_valid():
        form.save()
        return redirect(lead)
    return render(request, "leads/lead_form.html", {"form": form,
"page_title": f"Edit {lead.name}"})


@require_POST
def log_call_view(request, lead_id):
# 25
    lead = get_object_or_404(Lead, pk=lead_id, is_active=True)
    form = LogCallForm(request.POST)
    if form.is_valid():
        agent = request.user if request.user.is_authenticated else
lead.assigned_to   # Day 15: request.user only
        log_call(lead, agent, **form.cleaned_data)
        return redirect(lead)
```

```
    return render(request, "leads/log_call.html", {"form": form, "lead":
lead})


def convert_lead(request, lead_id):
# 27
    lead = get_object_or_404(Lead, pk=lead_id, is_active=True)
    form = ConvertLeadForm(request.POST or None, lead=lead)
    if request.method == "POST" and form.is_valid():
        client = convert_lead_to_client(lead.pk, **form.cleaned_data)
        return redirect(client)
    return render(request, "leads/convert.html", {"form": form, "lead":
lead})


def quick_add_many(request):
# 30
    formset = QuickLeadFormSet(request.POST or None)
    if request.method == "POST" and formset.is_valid():
        rows = [f.cleaned_data for f in formset if f.cleaned_data]   #
untouched extra rows are empty
        upsert_leads(rows)
        return redirect("leads:list")
    return render(request, "leads/quick_add_many.html", {"formset": formset})
```

```
# clients/services.py — 27: accept leverage
def convert_lead_to_client(lead_id, platform="mt5", currency="USD",
leverage=100):
    ...
    TradingAccount.objects.create(client=client,
number=next_account_number(),
                                  platform=platform, currency=currency,
leverage=leverage)

# leads/views.py — LeadDetailView.get_context_data (25)
context["log_call_form"] = LogCallForm()

# leads/urls.py
path("new/", views.lead_create, name="new"),
path("quick-add-many/", views.quick_add_many, name="quick_add_many"),
path("<int:lead_id>/edit/", views.lead_edit, name="edit"),
path("<int:lead_id>/log-call/", views.log_call_view, name="log_call"),
path("<int:lead_id>/convert/", views.convert_lead, name="convert"),
```

`request.POST or None` is unbound on GET; a POST with no fields would also look unbound, which is fine for these forms (24).

22–23.

```
<!-- leads/templates/leads/lead_form.html (also used by convert.html / log_call.html with other headings) -->
{% extends "base.html" %}
{% block content %}
<h1>{{ page_title }}</h1>
<form method="post" novalidate>      {# novalidate: let Django's errors show while you practise #}
  {% csrf_token %}
  {{ form.non_field_errors }}
  {% for hidden in form.hidden_fields %}{{ hidden }}{{ hidden.errors }}{% endfor %}
  {% for field in form.visible_fields %}
    <div class="field{% if field.errors %} has-error{% endif %}">
      {{ field.as_field_group }}     {# label + help text + errors + widget (Django 5.0+) #}
    </div>
  {% endfor %}
  <button type="submit">Save</button>
  <a href="{% url 'leads:list' %}">Cancel</a>
</form>
{% endblock content %}
```

```
<!-- the long form of as_field_group, if you want full control -->
<label for="{{ field.id_for_label }}">{{ field.label }}</label>
{{ field }}
{% if field.help_text %}<small class="muted">{{ field.help_text }}</small>{% endif %}
{{ field.errors }}
```

```
/* static/css/crm.css — 23 */
.field { margin-bottom: .75rem; display: grid; gap: .25rem; }
.has-error input, .has-error select, .has-error textarea { border: 1px solid #dc2626; }
.errorlist { color: #dc2626; list-style: none; padding: 0; margin: 0; font-size: .875rem; }
```

```
<!-- leads/lead_detail.html — 25 -->
<h2>Log a call</h2>
<form method="post" action="{% url 'leads:log_call' lead.pk %}">
  {% csrf_token %}
  {{ log_call_form.as_div }}
  <button type="submit">Log call</button>
</form>
```

```
<!-- leads/templates/leads/quick_add_many.html — 30 -->
{% extends "base.html" %}
{% block content %}
<h1>Quick add</h1>
<form method="post">
  {% csrf_token %}
  {{ formset.management_form }}     {# TOTAL_FORMS etc. — required #}
  {{ formset.non_form_errors }}
  <table class="table">
    <tr><th>Name</th><th>Phone</th><th>Source</th></tr>
    {% for form in formset %}
      <tr>
        <td>{{ form.name }}{{ form.name.errors }}</td>
        <td>{{ form.phone }}{{ form.phone.errors }}</td>
        <td>{{ form.source }}</td>
      </tr>
    {% endfor %}
  </table>
  <button type="submit">Add leads</button>
</form>
{% endblock content %}
```

26.

```
# clients/forms.py
from decimal import Decimal

from django import forms

from .models import Client, Deposit, TradingAccount

CRYPTO_CAP = Decimal("50000")


class DepositForm(forms.ModelForm):
    class Meta:
        model = Deposit
        fields = ["amount", "method"]

    def clean_amount(self):
        amount = self.cleaned_data["amount"]
        if amount <= 0:
            raise forms.ValidationError("Amount must be positive.")
        if amount > Decimal("1000000"):
            raise forms.ValidationError("Amounts over 1,000,000 need manual
approval.")
```

```
        return amount

    def clean(self):
        cleaned = super().clean()
        if cleaned.get("method") == Deposit.Method.CRYPTO and
cleaned.get("amount", 0) > CRYPTO_CAP:
            self.add_error("amount", f"Crypto deposits are capped at
{CRYPTO_CAP:,.0f}.")
        return cleaned


TradingAccountFormSet = forms.inlineformset_factory(
# 31
    Client, TradingAccount,
    fields=["number", "platform", "currency", "leverage"],
    extra=1, can_delete=False,
)
```

28–29.

```
# leads/views.py — LeadQueryMixin now reads the form
from django.db.models import Count, F, Max
from .forms import LeadFilterForm


class LeadQueryMixin:
    def get_filter_form(self):
        if not hasattr(self, "_filter_form"):
            self._filter_form = LeadFilterForm(self.request.GET or None)
        return self._filter_form

    def get_leads(self):
        qs = (Lead.objects.filter(is_active=True)
              .select_related("assigned_to", "broker")
              .prefetch_related("tags")
              .annotate(call_count=Count("calls", distinct=True),
last_call_at=Max("calls__started_at")))
        form = self.get_filter_form()
        form.is_valid()
        data = getattr(form, "cleaned_data", {})     # valid fields only;
invalid ones are left out
        if q := data.get("q"):
            qs = qs.filter(search_q(q))
        if status := data.get("status"):
            qs = qs.filter(status=status)
        if source := data.get("source"):
```

```
            qs = qs.filter(source=source)
        if agent := data.get("agent"):
            qs = qs.filter(assigned_to=agent)
        if tag := data.get("tag"):
            qs = qs.filter(tags=tag)
        if finder := data.get("finder"):
            qs = qs.filter(pk__in=FINDERS[finder]().values("pk"))
        if data.get("min_deposit") is not None:
            qs = qs.filter(expected_deposit__gte=data["min_deposit"])
        if data.get("max_deposit") is not None:
            qs = qs.filter(expected_deposit__lte=data["max_deposit"])
        if data.get("created_from"):
            qs = qs.filter(created_at__date__gte=data["created_from"])
        if data.get("created_to"):
            qs = qs.filter(created_at__date__lte=data["created_to"])
        sort = data.get("sort") or "-created_at"
        field = sort.lstrip("-")
        order = F(field).desc(nulls_last=True) if sort.startswith("-") else
F(field).asc(nulls_last=True)
        return qs.order_by(order, "-pk")


# LeadListView — expose the form
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context["filter_form"] = self.get_filter_form()
    return context
```

```
<!-- leads/lead_list.html — 29 (replaces the Day 3 search box) -->
<form method="get" class="filters">
  {% for field in filter_form %}{{ field.as_field_group }}{% endfor %}
  <button type="submit">Apply</button>
  <a href="{% url 'leads:list' %}">Reset</a>
</form>
```

```
.filters { display: grid; grid-template-columns: repeat(auto-fill,
minmax(160px, 1fr)); gap: .75rem; align-items: end; }
```

D. Challenge

31.

```
# clients/views.py
from .forms import TradingAccountFormSet


def client_accounts(request, ref):
    client = get_object_or_404(Client, ref=ref)
    formset = TradingAccountFormSet(request.POST or None, instance=client)
    if request.method == "POST" and formset.is_valid():
        formset.save()                    # sets account.client for new rows
        return redirect(client)
    return render(request, "clients/client_accounts.html", {"client": client,
"formset": formset})

# clients/urls.py
path("<uuid:ref>/accounts/", views.client_accounts, name="accounts"),
```

```
<!-- clients/templates/clients/client_accounts.html -->
{% extends "base.html" %}
{% block content %}
<h1>{{ client.name }} · trading accounts</h1>
<form method="post">{% csrf_token %}
  {{ formset.management_form }}
  {% for form in formset %}<fieldset>{{ form.as_div }}</fieldset>{% endfor %}
  <button type="submit">Save accounts</button>
</form>
{% endblock content %}
```

Tests (37–40)

```
# leads/tests.py
from clients.forms import DepositForm
from .forms import DateRangeForm, LeadForm


class FormTests(TestCase):
    def test_leadform_normalises_phone(self):                        # 37
        form = LeadForm({"name": "zara ahmed", "phone": "+971 50 123 9999",
"source": "google"})
        self.assertTrue(form.is_valid(), form.errors)
        lead = form.save()
        self.assertEqual(lead.phone, "971501239999")
        self.assertEqual(lead.name, "Zara Ahmed")

    def test_duplicate_phone_message(self):                          # 38
        Lead.objects.create(name="First", phone="971501239998")
```

```
        form = LeadForm({"name": "Second", "phone": "0501239998", "source":
"google"})
        self.assertFalse(form.is_valid())
        self.assertIn("already exists", form.errors["phone"][0])

    def test_date_range(self):                                          # 39
        form = DateRangeForm({"date_from": "2026-10-10", "date_to": "2026-10-
01"})
        self.assertFalse(form.is_valid())
        self.assertIn("date_to", form.errors)

    def test_crypto_cap(self):                                          # 40
        form = DepositForm({"amount": "60000", "method": "crypto"})
        self.assertFalse(form.is_valid())
        self.assertIn("capped", form.errors["amount"][0])
```

Day 14 — Editing views and messages

Goal: rebuild Day 13's function views as generic editing views, give every action feedback through the messages framework, and add the CRM's day-to-day write actions: status changes, conversion, deposits, bulk updates, archive with undo, and an audit trail.

URL names stay the same as Day 13 (`new`, `edit`, `log_call`, `convert`), so templates keep working.

A. Drills (1–10) — the editing views

- [ ] **1.** `LeadCreateView(CreateView)` with `form_class = LeadForm`; redirect to the lead (via `get_absolute_url`).
- [ ] **2.** `LeadUpdateView(UpdateView)` with `pk_url_kwarg`, same template, active leads only.
- [ ] **3.** `LeadConfirmDeleteView(DeleteView)` with a confirm page at `/leads/<id>/delete/confirm/`.
- [ ] **4.** `UserFormKwargsMixin.get_form_kwargs()` passing `user=request.user` to the form.
- [ ] **5.** `form_valid()`: an agent creating a lead with no assignee gets it.
- [ ] **6.** `form_invalid()`: add an error message, then return the normal response.
- [ ] **7.** Messages block in `base.html` (CSS per level).
- [ ] **8.** `SuccessMessageMixin`: "Lead Zara Ahmed created."
- [ ] **9.** "Save and add another" button via `get_success_url()`.
- [ ] **10.** `LogCallView(FormView)` replacing Day 13's `log_call_view`.

B. Core (11–20)

- [ ] **11.** `LeadStatusView(UpdateView)` with `LeadStatusForm`; message shows the new status label.
- [ ] **12.** `ConvertLeadView(FormView)`: load the lead in `setup()`, call the service, turn `ValueError` into a form error.
- [ ] **13.** `ClientUpdateView` editing the client and its trading accounts (inline formset) atomically.
- [ ] **14.** `DepositCreateView` at `/clients/accounts/<number>/deposits/new/`; the account comes from the URL.
- [ ] **15.** Full Tag CRUD with four generic views and a shared mixin.

- [ ] **16.** `LeadArchiveView`: POST-only soft delete.
- [ ] **17.** `MESSAGE_TAGS` (error → `danger`) and `extra_tags`.
- [ ] **18.** Info message on lead detail when a new lead has waited 7+ days without a call.
- [ ] **19.** Delete confirm page lists what goes with the lead (call count, client record).
- [ ] **20.** `ClientDeleteView` catches `ProtectedError` (clients with deposits) and shows an error instead of a 500.

C. Nexora CRM (21–30)

- [ ] **21.** Action bar on lead detail: Edit, Change status, Convert (hidden if already a client), Archive.
- [ ] **22.** Bulk actions on the lead list: checkboxes + "mark contacted / assign to agent / archive" (`LeadBulkForm` + `LeadBulkActionView`).
- [ ] **23.** Reassign a single lead from its detail page by reusing the bulk view.
- [ ] **24.** `get_initial()`: `/leads/new/?source=referral&agent=3` pre-fills the form.
- [ ] **25.** Duplicate phone on create → redirect to the existing lead with a warning instead of an error.
- [ ] **26.** Every bulk action reports how many rows changed.
- [ ] **27.** Edit conflict guard: hidden `loaded_at`; reject the save if the lead changed since the form was opened.
- [ ] **28.** Log-call form on detail posts to `LogCallView` and comes back with a success message (Post/Redirect/Get).
- [ ] **29.** Saving a deposit also adds it to `TradingAccount.balance` with `F()`, in the same transaction.
- [ ] **30.** Archive message with an Undo link to a restore page (`LeadRestoreView`, POST to restore).

D. Challenge (31–40)

- [ ] **31.** `ActivityLog` model with a `GenericForeignKey` + `AuditMixin` that records create/update with changed fields; show the last 10 on lead detail.
- [ ] **32.** `get_form_class()` returning a slimmer `modelform_factory` form for agents.
- [ ] **33.** Two forms on one page with `prefix` (edit lead + log call) and one `post()` that tells them apart.
- [ ] **34.** `success_url` with a format string: `"/leads/tags/?created={id}"`.
- [ ] **35.** `JsonFormMixin`: the same create view answers JSON when `Accept: application/json`.

- [ ] **36.** `safe_next()` helper using `url_has_allowed_host_and_scheme`; use it for `?next=` redirects.
- [ ] **37.** Test: create view redirects to the new lead and shows the success message.
- [ ] **38.** Test: a stale `loaded_at` is rejected.
- [ ] **39.** Test: deleting a client with deposits keeps the row and shows the error message.
- [ ] **40.** Test: bulk "mark contacted" updates both selected leads.

Solutions

Generic editing flow: GET → `get_form()` → render. POST → `get_form()` → `is_valid()` → `form_valid()` (save + redirect to `get_success_url()`) or `form_invalid()` (re-render with errors). Override those hooks, always calling `super()`.

Helpers (4, 31, 36)

```
# core/utils.py — 36
from django.utils.http import url_has_allowed_host_and_scheme


def safe_next(request, default="/"):
    nxt = request.POST.get("next") or request.GET.get("next")
    if nxt and url_has_allowed_host_and_scheme(
        nxt, allowed_hosts={request.get_host()},
require_https=request.is_secure(),
    ):
        return nxt
    return default          # never redirect to another site from user input
```

```
# core/models.py — 31
from django.conf import settings
from django.contrib.contenttypes.fields import GenericForeignKey
from django.contrib.contenttypes.models import ContentType
from django.db import models


class ActivityLog(models.Model):
    actor = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                              on_delete=models.SET_NULL,
related_name="activity")
    verb = models.CharField(max_length=20)                    # created /
updated / deleted
    content_type = models.ForeignKey(ContentType, on_delete=models.CASCADE)
```

```
    object_id = models.CharField(max_length=64)                # str: works
for int and UUID pks
    target = GenericForeignKey("content_type", "object_id")      # the object
itself, any model
    object_repr = models.CharField(max_length=200)
    changes = models.JSONField(default=dict, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["content_type", "object_id"],
name="activity_target_idx")]

    def __str__(self):
        return f"{self.actor or 'system'} {self.verb} {self.object_repr}"
```

```
# core/mixins.py — 4, 31 (add)
from django.contrib.contenttypes.models import ContentType
from core.models import ActivityLog


class UserFormKwargsMixin:                                          # 4
    def get_form_kwargs(self):
        kwargs = super().get_form_kwargs()
        kwargs["user"] = self.request.user
        return kwargs


class AuditMixin:                                                   # 31 — for
Create/UpdateView
    audit_verb = "updated"

    def form_valid(self, form):
        response = super().form_valid(form)                         # self.object
is saved now
        ActivityLog.objects.create(
            actor=self.request.user if self.request.user.is_authenticated
else None,
            verb=self.audit_verb,
            content_type=ContentType.objects.get_for_model(self.object),
            object_id=str(self.object.pk),
            object_repr=str(self.object)[:200],
```

```
            changes={name: str(form.cleaned_data.get(name)) for name in
form.changed_data},
        )
        return response
```

```
python manage.py makemigrations core && python manage.py migrate
```

Forms added today (22, 27)

```
# leads/forms.py (add)
from django.utils import timezone


class LeadEditForm(LeadForm):                                    # 27
    loaded_at = forms.DateTimeField(widget=forms.HiddenInput)

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        if self.instance.pk and not self.is_bound:
            self.initial["loaded_at"] = self.instance.updated_at.isoformat()

    def clean(self):
        cleaned = super().clean()
        loaded = cleaned.get("loaded_at")
        if loaded and self.instance.pk and self.instance.updated_at > loaded:
            raise ValidationError("Someone else changed this lead after you
opened it. Reload and try again.")
        return cleaned


class LeadBulkForm(forms.Form):                                  # 22
    ACTIONS = [("contacted", "Mark contacted"), ("assign", "Assign to
agent"), ("archive", "Archive")]
    leads =
forms.ModelMultipleChoiceField(queryset=Lead.objects.filter(is_active=True),
                                           widget=forms.MultipleHiddenInput)
    action = forms.ChoiceField(choices=ACTIONS)
    agent = forms.ModelChoiceField(required=False, empty_label="— agent —",

queryset=get_user_model().objects.filter(role="agent", is_active=True))

    def clean(self):
```

```
        cleaned = super().clean()
        if cleaned.get("action") == "assign" and not cleaned.get("agent"):
            self.add_error("agent", "Pick an agent to assign to.")
        return cleaned
```

Lead views (1–12, 16, 18–19, 21–25, 28, 30–36)

```
# leads/views.py
from datetime import timedelta

from django.contrib import messages
from django.contrib.messages.views import SuccessMessageMixin
from django.forms import modelform_factory
from django.http import JsonResponse
from django.shortcuts import get_object_or_404, redirect
from django.urls import reverse, reverse_lazy
from django.utils import timezone
from django.utils.html import format_html
from django.views import View
from django.views.generic import TemplateView
from django.views.generic.edit import CreateView, DeleteView, FormView,
UpdateView

from clients.services import convert_lead_to_client
from core.mixins import AuditMixin, PageTitleMixin, UserFormKwargsMixin
from core.utils import safe_next
from .forms import ConvertLeadForm, LeadBulkForm, LeadEditForm, LeadForm,
LeadStatusForm, LogCallForm
from .models import Lead, normalize_uae_phone
from .services import log_call


class JsonFormMixin:                                                    #
35
    def wants_json(self):
        return "application/json" in self.request.headers.get("Accept", "")

    def form_invalid(self, form):
        if self.wants_json():
            return JsonResponse({"errors": form.errors.get_json_data()},
status=400)
        return super().form_invalid(form)

    def form_valid(self, form):
        response = super().form_valid(form)
        if self.wants_json():
```

```
            return JsonResponse({"id": self.object.pk, "url":
self.object.get_absolute_url()}, status=201)
        return response


class LeadCreateView(JsonFormMixin, AuditMixin, PageTitleMixin,
UserFormKwargsMixin,
                     SuccessMessageMixin, CreateView):
# 1, 4–6, 8–9, 24–25, 31, 35
    model = Lead
    form_class = LeadForm
    template_name = "leads/lead_form.html"
    page_title = "New lead"
    success_message = "Lead %(name)s created."        # %(field)s comes from
cleaned_data
    audit_verb = "created"

    def get_initial(self):                                                  #
24
        initial = super().get_initial()
        if source := self.request.GET.get("source"):
            initial["source"] = source
        if (agent := self.request.GET.get("agent", "")).isdigit():
            initial["assigned_to"] = agent
        return initial

    def form_valid(self, form):                                             #
5
        if form.instance.assigned_to is None and getattr(self.request.user,
"role", "") == "agent":
            form.instance.assigned_to = self.request.user
        return super().form_valid(form)

    def form_invalid(self, form):                                           #
6, 25
        if form.has_error("phone", code="unique"):
            existing = Lead.objects.filter(
                phone=normalize_uae_phone(form.data.get("phone", "")),
is_active=True,
            ).first()
            if existing:
                messages.warning(self.request, f"{existing.name} already has
this number — opened their record.")
                return redirect(existing)
        messages.error(self.request, "Please fix the errors below.")
        return super().form_invalid(form)
```

```
    def get_success_url(self):                                          #
9
        if "_addanother" in self.request.POST:
            return reverse("leads:new")
        return self.object.get_absolute_url()


class LeadUpdateView(AuditMixin, PageTitleMixin, UserFormKwargsMixin,
                     SuccessMessageMixin, UpdateView):
# 2, 27, 31
    model = Lead
    form_class = LeadEditForm
    pk_url_kwarg = "lead_id"
    template_name = "leads/lead_form.html"
    page_title = "Edit lead"
    success_message = "Saved %(name)s."

    def get_queryset(self):
        return Lead.objects.filter(is_active=True)

    def get_form_class(self):                                           #
32
        if getattr(self.request.user, "role", "") == "agent":
            return modelform_factory(Lead, form=LeadEditForm,
                                     fields=["name", "phone", "email",
"source", "tags", "notes"])
        return super().get_form_class()


class LeadStatusView(PageTitleMixin, SuccessMessageMixin, UpdateView):     #
11
    model = Lead
    form_class = LeadStatusForm
    pk_url_kwarg = "lead_id"
    template_name = "leads/lead_form.html"
    page_title = "Change status"

    def get_success_message(self, cleaned_data):
        return f"Status changed to {self.object.get_status_display()}."


class LeadConfirmDeleteView(PageTitleMixin, SuccessMessageMixin, DeleteView):
# 3, 19, 36
    model = Lead
    pk_url_kwarg = "lead_id"
```

```
    template_name = "leads/lead_confirm_delete.html"
    success_message = "Lead deleted."
    page_title = "Delete lead"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["call_count"] = self.object.calls.count()
        context["is_client"] = hasattr(self.object, "client")
        return context

    def get_success_url(self):
        return safe_next(self.request, default=reverse("leads:list"))


class LogCallView(SuccessMessageMixin, FormView):                               #
10, 28
    form_class = LogCallForm
    template_name = "leads/log_call.html"
    success_message = "Call logged."

    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, **kwargs)
        self.lead = get_object_or_404(Lead, pk=kwargs["lead_id"],
is_active=True)

    def form_valid(self, form):
        agent = self.request.user if self.request.user.is_authenticated else
self.lead.assigned_to
        log_call(self.lead, agent, **form.cleaned_data)
        return super().form_valid(form)

    def get_success_url(self):
        return self.lead.get_absolute_url()

    def get_context_data(self, **kwargs):
        return {**super().get_context_data(**kwargs), "lead": self.lead}


class ConvertLeadView(PageTitleMixin, FormView):                                #
12
    form_class = ConvertLeadForm
    template_name = "leads/convert.html"
    page_title = "Convert to client"

    def setup(self, request, *args, **kwargs):     # runs before dispatch —
good place to load objects
```

```
        super().setup(request, *args, **kwargs)
        self.lead = get_object_or_404(Lead, pk=kwargs["lead_id"],
is_active=True)

    def get_form_kwargs(self):
        return {**super().get_form_kwargs(), "lead": self.lead}

    def form_valid(self, form):
        try:
            client = convert_lead_to_client(self.lead.pk,
**form.cleaned_data)
        except ValueError as exc:
            form.add_error(None, str(exc))
            return self.form_invalid(form)
        messages.success(self.request, f"{self.lead.name} is now client
{client.account_number}.")
        return redirect(client)

    def get_context_data(self, **kwargs):
        return {**super().get_context_data(**kwargs), "lead": self.lead}


class LeadArchiveView(View):                                          #
16, 30
    http_method_names = ["post"]

    def post(self, request, lead_id):
        lead = get_object_or_404(Lead, pk=lead_id, is_active=True)
        lead.archive()
        undo = reverse("leads:restore", args=[lead.pk])
        messages.success(request, format_html('{} archived. <a href="
{}">Undo</a>', lead.name, undo))
        return redirect("leads:list")


class LeadRestoreView(PageTitleMixin, TemplateView):
# 30
    template_name = "leads/lead_restore.html"
    page_title = "Restore lead"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["lead"] = get_object_or_404(Lead, pk=self.kwargs["lead_id"],
is_active=False)
        return context
```

```
    def post(self, request, lead_id):              # state changes only on
POST
        lead = get_object_or_404(Lead, pk=lead_id, is_active=False)
        Lead.objects.filter(pk=lead.pk).update(is_active=True,
updated_at=timezone.now())
        messages.success(request, f"{lead.name} restored.")
        return redirect(lead)


class LeadBulkActionView(FormView):                                        #
22-23, 26
    form_class = LeadBulkForm
    http_method_names = ["post"]

    def form_valid(self, form):
        qs = form.cleaned_data["leads"]               # a queryset of the
ticked leads
        action, now = form.cleaned_data["action"], timezone.now()
        if action == "contacted":
            n = qs.update(status=Lead.Status.CONTACTED, updated_at=now)
        elif action == "assign":
            n = qs.update(assigned_to=form.cleaned_data["agent"],
updated_at=now)
        else:
            n = qs.update(is_active=False, updated_at=now)
        messages.success(self.request, f"{n} lead{'s' if n != 1 else ''}
updated.")
        return redirect(safe_next(self.request,
default=reverse("leads:list")))

    def form_invalid(self, form):
        errors = "; ".join(e for errs in form.errors.values() for e in errs)
        messages.error(self.request, f"Bulk action failed: {errors}")
        return redirect(safe_next(self.request,
default=reverse("leads:list")))


class LeadWorkspaceView(PageTitleMixin, TemplateView):                     #
33
    template_name = "leads/lead_workspace.html"
    page_title = "Workspace"

    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, **kwargs)
        self.lead = get_object_or_404(Lead, pk=kwargs["lead_id"],
is_active=True)
```

```
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context.setdefault("lead_form", LeadForm(instance=self.lead,
prefix="lead", user=self.request.user))
        context.setdefault("call_form", LogCallForm(prefix="call"))
        context["lead"] = self.lead
        return context

    def post(self, request, *args, **kwargs):
        if "save_lead" in request.POST:
            form = LeadForm(request.POST, instance=self.lead, prefix="lead",
user=request.user)
            if form.is_valid():
                form.save()
                messages.success(request, "Lead saved.")
                return redirect(request.path)
            return
self.render_to_response(self.get_context_data(lead_form=form))
        form = LogCallForm(request.POST, prefix="call")
        if form.is_valid():
            log_call(self.lead, request.user, **form.cleaned_data)
            messages.success(request, "Call logged.")
            return redirect(request.path)
        return self.render_to_response(self.get_context_data(call_form=form))
```

```
# LeadDetailView.get_context_data — 18, 31 (add)
from django.contrib.contenttypes.models import ContentType
from core.models import ActivityLog

if (lead.status == Lead.Status.NEW and lead.last_contacted_at is None
        and lead.created_at < timezone.now() - timedelta(days=7)):
    messages.info(self.request, "This lead has waited over 7 days without a
call.")   # shown in this same render
context["activity"] = ActivityLog.objects.filter(
    content_type=ContentType.objects.get_for_model(Lead),
object_id=str(lead.pk),
).select_related("actor")[:10]
context["bulk_form"] = LeadBulkForm()          # 23: agent dropdown for the
reassign box

# LeadListView.get_context_data — 22 (add)
context["bulk_form"] = LeadBulkForm()
```

```
# leads/urls.py (today's routes)
path("new/", views.LeadCreateView.as_view(), name="new"),
path("bulk/", views.LeadBulkActionView.as_view(), name="bulk"),
path("<int:lead_id>/edit/", views.LeadUpdateView.as_view(), name="edit"),
path("<int:lead_id>/status/edit/", views.LeadStatusView.as_view(),
name="status_edit"),
path("<int:lead_id>/log-call/", views.LogCallView.as_view(),
name="log_call"),
path("<int:lead_id>/convert/", views.ConvertLeadView.as_view(),
name="convert"),
path("<int:lead_id>/archive/", views.LeadArchiveView.as_view(),
name="archive"),
path("<int:lead_id>/restore/", views.LeadRestoreView.as_view(),
name="restore"),
path("<int:lead_id>/delete/confirm/", views.LeadConfirmDeleteView.as_view(),
name="delete_confirm"),
path("<int:lead_id>/workspace/", views.LeadWorkspaceView.as_view(),
name="workspace"),
```

Templates (3, 7, 9, 17, 19, 21–23, 30)

```
<!-- templates/base.html — 7: above {% block content %} -->
{% if messages %}
<ul class="messages">
  {% for message in messages %}
    <li class="message {{ message.tags }}">{{ message }}</li>
  {% endfor %}
</ul>
{% endif %}
```

```
.messages { list-style: none; padding: 0; margin: 0 0 1rem; }
.message { padding: .6rem .9rem; border-radius: 8px; margin-bottom: .5rem; }
.message.success { background: #dcfce7; }
.message.info { background: #dbeafe; }
.message.warning { background: #fef9c3; }
.message.danger { background: #fee2e2; }
```

```
# settings.py — 17
from django.contrib.messages import constants as message_constants
MESSAGE_TAGS = {message_constants.ERROR: "danger"}
# extra tags: messages.success(request, "...", extra_tags="sticky") →
class="message sticky success"
```

```
<!-- leads/lead_form.html — 9: second submit button -->
<button type="submit">Save</button>
<button type="submit" name="_addanother">Save and add another</button>
```

```
<!-- leads/templates/leads/lead_confirm_delete.html — 3, 19 -->
{% extends "base.html" %}
{% block content %}
<h1>Delete {{ object.name }}?</h1>
<p>This also deletes {{ call_count }} call log{{ call_count|pluralize }}.
   {% if is_client %}The client record stays but loses its link to this lead.
{% endif %}</p>
<p>Prefer <strong>Archive</strong> unless this lead was created by mistake.
</p>
<form method="post">{% csrf_token %}
  <input type="hidden" name="next" value="{{ request.GET.next }}">
  <button type="submit" class="danger">Delete permanently</button>
  <a href="{{ object.get_absolute_url }}">Cancel</a>
</form>
{% endblock content %}
```

```
<!-- leads/lead_detail.html — 21, 23, 31 -->
<div class="actions">
  <a href="{% url 'leads:edit' lead.pk %}">Edit</a>
  <a href="{% url 'leads:status_edit' lead.pk %}">Change status</a>
  {% if not lead.client %}<a href="{% url 'leads:convert' lead.pk %}">Convert
to client</a>{% endif %}
  <form method="post" action="{% url 'leads:archive' lead.pk %}"
class="inline">{% csrf_token %}
    <button type="submit">Archive</button>
  </form>
</div>

<form method="post" action="{% url 'leads:bulk' %}" class="inline">{%
csrf_token %}
  <input type="hidden" name="leads" value="{{ lead.pk }}">
  <input type="hidden" name="action" value="assign">
  <input type="hidden" name="next" value="{{ request.path }}">
  {{ bulk_form.agent }} <button type="submit">Reassign</button>
</form>

<h2>Activity</h2>
<ul>{% for a in activity %}<li>{{ a.created_at|date:"d M H:i" }} · {{
a.actor|default:"system" }} {{ a.verb }} {{ a.changes|join:", " }}</li>{%
empty %}<li>—</li>{% endfor %}</ul>
```

`{% if not lead.client %}` is safe: the missing reverse one-to-one raises a `DoesNotExist`, which templates treat as empty.

```
<!-- leads/lead_list.html — 22: bulk bar above the table -->
<form id="bulk-form" method="post" action="{% url 'leads:bulk' %}">{%
csrf_token %}
  <input type="hidden" name="next" value="{{ request.get_full_path }}">
  {{ bulk_form.action }} {{ bulk_form.agent }}
  <button type="submit">Apply to selected</button>
</form>

<!-- _lead_rows.html — first cell of each row; form= ties the checkbox to the
bulk form above -->
<td><input type="checkbox" name="leads" value="{{ lead.pk }}" form="bulk-
form"></td>
```

```
<!-- leads/templates/leads/lead_restore.html — 30 -->
{% extends "base.html" %}
{% block content %}
<h1>Restore {{ lead.name }}?</h1>
<form method="post">{% csrf_token %}<button type="submit">Restore</button>
</form>
{% endblock content %}
```

The Undo link survives the redirect because `format_html` returns a safe string and Django's message storage keeps it marked safe (30).

Clients (13–14, 20, 29)

```
# clients/views.py
from django.contrib import messages
from django.db import transaction
from django.db.models import F, ProtectedError
from django.shortcuts import get_object_or_404, redirect
from django.urls import reverse_lazy
from django.views.generic.edit import CreateView, DeleteView, UpdateView

from .forms import DepositForm, TradingAccountFormSet
from .models import Client, TradingAccount


class ClientUpdateView(PageTitleMixin, UpdateView):
# 13
    model = Client
    slug_field = "ref"
```

```
    slug_url_kwarg = "ref"
    fields = ["name", "email", "account_type", "kyc_status", "broker"]
    template_name = "clients/client_form.html"
    page_title = "Edit client"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context.setdefault("formset",
TradingAccountFormSet(instance=self.object))
        return context

    def post(self, request, *args, **kwargs):
        self.object = self.get_object()
        form = self.get_form()
        formset = TradingAccountFormSet(request.POST, instance=self.object)
        if form.is_valid() and formset.is_valid():
            with transaction.atomic():             # both or neither
                self.object = form.save()
                formset.save()
            messages.success(request, "Client saved.")
            return redirect(self.object)
        return self.render_to_response(self.get_context_data(form=form,
formset=formset))


class DepositCreateView(PageTitleMixin, CreateView):
# 14, 29
    form_class = DepositForm
    template_name = "clients/deposit_form.html"
    page_title = "New deposit"

    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, **kwargs)
        self.account =
get_object_or_404(TradingAccount.objects.select_related("client"),
number=kwargs["number"])

    def form_valid(self, form):
        form.instance.account = self.account
        with transaction.atomic():
            response = super().form_valid(form)
            TradingAccount.objects.filter(pk=self.account.pk).update(
                balance=F("balance") + form.instance.amount,
            )
        messages.success(self.request, f"Deposit of
{form.instance.amount:,.2f} added to {self.account.number}.")
```

```
        return response

    def get_success_url(self):
        return self.account.client.get_absolute_url()

    def get_context_data(self, **kwargs):
        return {**super().get_context_data(**kwargs), "account":
self.account}


class ClientDeleteView(PageTitleMixin, DeleteView):
# 20
    model = Client
    slug_field = "ref"
    slug_url_kwarg = "ref"
    template_name = "clients/client_confirm_delete.html"
    success_url = reverse_lazy("clients:list")
    page_title = "Delete client"

    def form_valid(self, form):
        try:
            response = super().form_valid(form)    # self.object.delete()
        except ProtectedError:
            messages.error(self.request, "This client has deposits and can't
be deleted. Deactivate them instead.")
            return redirect(self.object)
        messages.success(self.request, "Client deleted.")
        return response

# clients/urls.py
path("<uuid:ref>/edit/", views.ClientUpdateView.as_view(), name="edit"),
path("<uuid:ref>/delete/", views.ClientDeleteView.as_view(), name="delete"),
path("accounts/<str:number>/deposits/new/",
views.DepositCreateView.as_view(), name="deposit_new"),
```

```
<!-- clients/templates/clients/client_form.html — 13 -->
{% extends "base.html" %}
{% block content %}
<h1>Edit {{ object.name }}</h1>
<form method="post">{% csrf_token %}
  {{ form.as_div }}
  <h2>Trading accounts</h2>
  {{ formset.management_form }}
  {{ formset.non_form_errors }}
  {% for f in formset %}<fieldset>{{ f.as_div }}</fieldset>{% endfor %}
  <button type="submit">Save</button>
</form>
{% endblock content %}
```

Tags (15, 34)

```
# leads/views.py
from django.views.generic import ListView
from .forms import TagForm
from .models import Tag


class TagFormMixin(SuccessMessageMixin):
    model = Tag
    form_class = TagForm
    template_name = "leads/tag_form.html"
    success_url = "/leads/tags/?saved={id}"            # 34: formatted with
the saved object's fields
    success_message = "Tag %(name)s saved."


class TagListView(ListView):
    model = Tag                                         # template:
leads/tag_list.html, context: tag_list


class TagCreateView(TagFormMixin, CreateView):
    pass


class TagUpdateView(TagFormMixin, UpdateView):
    pass


class TagDeleteView(SuccessMessageMixin, DeleteView):
```

```
    model = Tag
    template_name = "leads/tag_confirm_delete.html"
    success_url = reverse_lazy("leads:tags")
    success_message = "Tag deleted."

# leads/urls.py
path("tags/", views.TagListView.as_view(), name="tags"),
path("tags/new/", views.TagCreateView.as_view(), name="tag_new"),
path("tags/<int:pk>/edit/", views.TagUpdateView.as_view(), name="tag_edit"),
path("tags/<int:pk>/delete/", views.TagDeleteView.as_view(),
name="tag_delete"),
```

```
<!-- leads/templates/leads/tag_list.html -->
{% extends "base.html" %}
{% block content %}
<h1>Tags</h1>
<a href="{% url 'leads:tag_new' %}">+ New tag</a>
<ul>
  {% for tag in tag_list %}
    <li><span class="badge" style="background:{{ tag.color }};color:#fff">{{
tag.name }}</span>
        <a href="{% url 'leads:tag_edit' tag.pk %}">Edit</a> · <a href="{%
url 'leads:tag_delete' tag.pk %}">Delete</a></li>
  {% endfor %}
</ul>
{% endblock content %}
```

Workspace template (33)

```
<!-- leads/templates/leads/lead_workspace.html -->
{% extends "base.html" %}
{% block content %}
<h1>{{ lead.name }}</h1>
<div class="cards">
  <form method="post" class="card">{% csrf_token %}
    {{ lead_form.as_div }}                  {# inputs are named lead-name,
lead-phone, ... #}
    <button type="submit" name="save_lead">Save lead</button>
  </form>
  <form method="post" class="card">{% csrf_token %}
    {{ call_form.as_div }}                  {# call-outcome, call-
duration_seconds, ... #}
    <button type="submit" name="log_call">Log call</button>
  </form>
</div>
{% endblock content %}
```

Tests (37–40)

```
# leads/tests.py
from datetime import timedelta

from clients.models import Client, Deposit, TradingAccount


class EditingViewTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.lead = Lead.objects.create(name="Edit Me", phone="971509990701")

    def test_create_redirects_with_message(self):                   # 37
        response = self.client.post(
            reverse("leads:new"),
            {"name": "new person", "phone": "0509990702", "source":
"google"},
            follow=True,
        )
        lead = Lead.objects.get(phone="971509990702")
        self.assertRedirects(response, lead.get_absolute_url())
        self.assertTrue(any("New Person created" in str(m) for m in
response.context["messages"]))

    def test_stale_edit_rejected(self):                             # 38
```

```
        stale = (self.lead.updated_at - timedelta(seconds=5)).isoformat()
        response = self.client.post(reverse("leads:edit", args=
[self.lead.pk]), {
            "name": "Edit Me", "phone": self.lead.phone, "source": "manual",
"loaded_at": stale,
        })
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Someone else changed this lead")

    def test_protected_client_delete(self):                          # 39
        rich = Client.objects.create(name="Rich", email="r@x.test",
account_number="7100001")
        account = TradingAccount.objects.create(client=rich,
number="7100001")
        Deposit.objects.create(account=account, amount=500, method="bank")
        response = self.client.post(reverse("clients:delete", args=
[rich.ref]), follow=True)
        self.assertTrue(Client.objects.filter(pk=rich.pk).exists())
        self.assertTrue(any("can't be deleted" in str(m) for m in
response.context["messages"]))

    def test_bulk_mark_contacted(self):                              # 40
        other = Lead.objects.create(name="Bulk Two", phone="971509990703")
        self.client.post(reverse("leads:bulk"), {"leads": [self.lead.pk,
other.pk], "action": "contacted"})
        self.assertEqual(
            Lead.objects.filter(pk__in=[self.lead.pk, other.pk],
status=Lead.Status.CONTACTED).count(), 2,
        )
```

Day 15 — Authentication

Goal: the whole CRM sits behind a login. Accounts are created by admins only — no sign-up page — and new staff get an invite email to set their own password. Agents can log in with username or email; repeated failures lock the account for 15 minutes.

After exercise 5, every page needs a logged-in user, so earlier view tests need a login too (solution 5 gives a `LoggedInTestCase` base).

A. Drills (1–10) — Django's auth views

- [ ] **1.** Include `django.contrib.auth.urls` at `/accounts/`; `registration/login.html`; set `LOGIN_URL`, `LOGIN_REDIRECT_URL`, `LOGOUT_REDIRECT_URL`.
- [ ] **2.** Sign-out as a POST form (Django 5+ refuses GET logout).
- [ ] **3.** `base.html`: signed-in user's name + role and Sign out; hide the sidebar when signed out.
- [ ] **4.** Protect one function view with `@login_required` and one CBV with `LoginRequiredMixin`.
- [ ] **5.** Django 5.1 `LoginRequiredMiddleware`: everything requires login; mark health, robots.txt and the API-key endpoints `login_not_required`. Fix the old tests.
- [ ] **6.** Check `?next=` brings you back to the page you asked for.
- [ ] **7.** In the shell: `authenticate()` with right and wrong passwords; `check_password()`.
- [ ] **8.** Password change: templates for the form and done pages; link from the user menu.
- [ ] **9.** Password reset: console email backend, the 4 page templates, and a custom email body/subject.
- [ ] **10.** `AUTH_PASSWORD_VALIDATORS`: minimum length 10.

B. Core (11–20) — admin-created accounts

- [ ] **11.** `invite_staff_user(username, email, role, ...)`: creates the user with an unusable password and emails a set-password link.
- [ ] **12.** Admin action "Send set-password invite" on users.
- [ ] **13.** Separate invite email templates (subject + body) from the normal reset email.
- [ ] **14.** `EmailOrUsernameBackend`: log in with either.
- [ ] **15.** Confirm a deactivated user (`is_active=False`) can't log in.
- [ ] **16.** `LoginEvent` model + `user_logged_in` / `user_login_failed` signal receivers (username, success, IP).

- [ ] **17.** Lockout: 5 failures in 15 minutes blocks that username (`CRMAuthenticationForm.clean`).
- [ ] **18.** `CRMLoginView(LoginView)`: custom form, `redirect_authenticated_user=True`.
- [ ] **19.** Sessions: 8-hour cookie by default; "Keep me signed in" stretches it to 14 days, unticked expires at browser close.
- [ ] **20.** Profile page `/team/me/`: edit name, email, extension; last 10 sign-ins; change-password link.

C. Nexora CRM (21–30)

- [ ] **21.** Use `request.user` as the acting agent: calls you log are yours.
- [ ] **22.** "My leads" at `/leads/mine/` — the lead list filtered to `request.user`.
- [ ] **23.** "My queue" at `/team/me/queue/` reusing the Day 12 agent queue.
- [ ] **24.** Dashboard greeting plus personal numbers: my open leads, my calls today.
- [ ] **25.** Role-based landing page after login: agents → my queue, managers/admins → reports.
- [ ] **26.** JSON endpoints answer `401` JSON instead of redirecting to the login page.
- [ ] **27.** Remove `csrf_exempt` from the browser-facing POST endpoints (status, delete, quick-add, create).
- [ ] **28.** Prove changing your password signs out your other sessions but keeps the current one.
- [ ] **29.** Admin user form (Django 5.1+): create users with password login disabled, then send the invite.
- [ ] **30.** Sidebar by role: agents don't see Reports or Brokers; managers see everything.

D. Challenge (31–40)

- [ ] **31.** `must_change_password` flag + middleware that sends flagged users to the password-change page until they comply.
- [ ] **32.** Custom password validator: no "nexora", no username.
- [ ] **33.** Argon2 as the default hasher; watch an old hash upgrade on next login.
- [ ] **34.** Shell: `make_password`, `set_unusable_password`, `has_usable_password`, `validate_password`.
- [ ] **35.** Build a set-password link by hand with `default_token_generator` + `urlsafe_base64_encode`; check when the token stops working.
- [ ] **36.** Admin action "Unlock" that clears recent failed sign-ins.
- [ ] **37.** Test: anonymous visit to `/leads/` redirects to login with `next`.

- [ ] **38.** Test: login with email works.
- [ ] **39.** Test: invite sends one email with a reset link; the user has no usable password.
- [ ] **40.** Test: 5 wrong passwords lock the account even for the right password.

Solutions

Settings (1, 5, 9–10, 14, 19, 26, 31–33)

```
# django_project/settings.py
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "core.middleware.CRMLoginRequiredMiddleware",        # 5, 26 — must come
after AuthenticationMiddleware
    "core.middleware.ForcePasswordChangeMiddleware",     # 31
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]

LOGIN_URL = "login"                       # 1 — URL names work here
LOGIN_REDIRECT_URL = "dashboard"
LOGOUT_REDIRECT_URL = "login"

AUTHENTICATION_BACKENDS = ["accounts.backends.EmailOrUsernameBackend"]     #
14

AUTH_PASSWORD_VALIDATORS = [                                               #
10, 32
    {"NAME":
"django.contrib.auth.password_validation.UserAttributeSimilarityValidator"},
    {"NAME":
"django.contrib.auth.password_validation.MinimumLengthValidator", "OPTIONS":
{"min_length": 10}},
    {"NAME":
"django.contrib.auth.password_validation.CommonPasswordValidator"},
    {"NAME":
"django.contrib.auth.password_validation.NumericPasswordValidator"},
    {"NAME": "accounts.validators.NoCompanyNameValidator"},
]

PASSWORD_HASHERS = [                                                       #
```

```
33
    "django.contrib.auth.hashers.Argon2PasswordHasher",     # first = used
for new hashes
    "django.contrib.auth.hashers.PBKDF2PasswordHasher",     # still verifies
(and upgrades) old ones
    "django.contrib.auth.hashers.PBKDF2SHA1PasswordHasher",
    "django.contrib.auth.hashers.ScryptPasswordHasher",
]

SESSION_COOKIE_AGE = 60 * 60 * 8                                        #
19: one working day

EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend"         # 9:
emails print in the terminal
DEFAULT_FROM_EMAIL = "Nexora CRM <no-reply@nexora.test>"
```

```
python -m pip install "django[argon2]"      # 33
```

Middleware (5, 26, 31)

```
# core/middleware.py
from django.contrib.auth.middleware import LoginRequiredMiddleware
from django.http import JsonResponse
from django.shortcuts import redirect


class CRMLoginRequiredMiddleware(LoginRequiredMiddleware):              # 5,
26
    def handle_no_permission(self, request, view_func):
        if "/api/" in request.path or "application/json" in
request.headers.get("Accept", ""):
            return JsonResponse({"error": "authentication required"},
status=401)
        return super().handle_no_permission(request, view_func)    # redirect
to LOGIN_URL?next=...


class ForcePasswordChangeMiddleware:                                    # 31
    ALLOWED = {"password_change", "password_change_done", "logout"}

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        return self.get_response(request)
```

```
    def process_view(self, request, view_func, view_args, view_kwargs):
        user = request.user
        if (user.is_authenticated and getattr(user, "must_change_password",
False)
                and request.resolver_match.url_name not in self.ALLOWED):
            return redirect("password_change")
        return None
```

5 — public endpoints and old tests.

```
from django.contrib.auth.decorators import login_not_required    # Django
5.1+

# core/urls.py
path("health/", login_not_required(views.HealthView.as_view()),
name="health"),
path("robots.txt",
login_not_required(TemplateView.as_view(template_name="robots.txt",
content_type="text/plain"))),

# leads/api_urls.py — machine clients authenticate with X-API-Key instead
path("leads/", login_not_required(views.ApiLeadListView.as_view()),
name="list"),
path("leads/<int:lead_id>/",
login_not_required(views.ApiLeadDetailView.as_view()), name="detail"),
```

```
# core/testing.py
from django.contrib.auth import get_user_model
from django.test import TestCase


class LoggedInTestCase(TestCase):
    role = "manager"

    @classmethod
    def setUpTestData(cls):
        super().setUpTestData()
        cls.staff = get_user_model().objects.create_user(f"test_{cls.role}",
password="x", role=cls.role)

    def setUp(self):
        super().setUp()
        self.client.force_login(self.staff)    # skips the password check
entirely
```

Every earlier test class that calls `self.client` (including the old `SimpleTestCase` ones — sessions now need the database) inherits from `LoggedInTestCase` instead. Classes with their own `setUpTestData` must call `super().setUpTestData()` first. The CSRF test still gets 403: CSRF runs before the login check.

Accounts models, backend, validator, signals (11, 14, 16, 31–32)

```
# accounts/models.py (add)
from django.conf import settings


class CustomUser(AbstractUser):
    ...
    must_change_password = models.BooleanField(default=False)          # 31


class LoginEvent(models.Model):                                         # 16
    user = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True,
                             on_delete=models.CASCADE,
related_name="login_events")
    username = models.CharField(max_length=150)       # what was typed —
failures may not match a user
    success = models.BooleanField()
    ip = models.GenericIPAddressField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["username", "created_at"],
name="login_user_created_idx")]
```

```
# accounts/backends.py                                                  # 14
from django.contrib.auth import get_user_model
from django.contrib.auth.backends import ModelBackend
from django.db.models import Q


class EmailOrUsernameBackend(ModelBackend):
    def authenticate(self, request, username=None, password=None, **kwargs):
        if username is None or password is None:
            return None
        User = get_user_model()
        try:
            user = User.objects.get(Q(username__iexact=username) |
Q(email__iexact=username))
        except (User.DoesNotExist, User.MultipleObjectsReturned):
            User().set_password(password)      # spend the same time as a
real check (timing attacks)
            return None
        if user.check_password(password) and
self.user_can_authenticate(user):   # inactive → False (15)
            return user
        return None
```

```
# accounts/validators.py                                                # 32
from django.core.exceptions import ValidationError


class NoCompanyNameValidator:
    def validate(self, password, user=None):
        lowered = password.lower()
        if "nexora" in lowered:
            raise ValidationError("Your password can't contain the company
name.", code="password_has_company")
        if user is not None and user.get_username() and
user.get_username().lower() in lowered:
            raise ValidationError("Your password can't contain your
username.", code="password_has_username")

    def get_help_text(self):
        return "Your password can't contain “nexora” or your username."
```

Validators run in forms and `validate_password()`, not in `set_password()` — the seeded `nexora123` passwords still exist until users change them.

```
# accounts/signals.py                                                    # 16
from django.contrib.auth.signals import user_logged_in, user_login_failed
from django.dispatch import receiver

from .models import LoginEvent


def _ip(request):
    return request.META.get("REMOTE_ADDR") if request else None


@receiver(user_logged_in)
def record_login(sender, request, user, **kwargs):
    LoginEvent.objects.create(user=user, username=user.get_username(),
success=True, ip=_ip(request))


@receiver(user_login_failed)
def record_failure(sender, credentials, request=None, **kwargs):
    LoginEvent.objects.create(username=credentials.get("username", "")[:150],
success=False, ip=_ip(request))


# accounts/apps.py — connect the receivers
class AccountsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "accounts"

    def ready(self):
        from . import signals  # noqa: F401
```

```
python manage.py makemigrations accounts && python manage.py migrate
```

Forms (17, 19–20, 29)

```
# accounts/forms.py
from datetime import timedelta

from django import forms
from django.contrib.auth.forms import AdminUserCreationForm,
AuthenticationForm, UserChangeForm
from django.utils import timezone

from .models import CustomUser, LoginEvent
```

```
LOCK_ATTEMPTS = 5
LOCK_WINDOW = timedelta(minutes=15)


class CustomUserCreationForm(AdminUserCreationForm):     # 29: Django 5.1+
adds "password-based auth: disabled"
    class Meta(AdminUserCreationForm.Meta):
        model = CustomUser
        fields = ("username", "email", "role", "team_lead")


class CRMAuthenticationForm(AuthenticationForm):                        # 17,
19
    remember_me = forms.BooleanField(required=False, label="Keep me signed in
for 14 days")

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.fields["username"].label = "Username or email"

    def clean(self):
        username = self.cleaned_data.get("username", "")
        recent_failures = LoginEvent.objects.filter(
            username__iexact=username, success=False,
created_at__gte=timezone.now() - LOCK_WINDOW,
        ).count()
        if recent_failures >= LOCK_ATTEMPTS:
            raise forms.ValidationError("Too many failed attempts. Try again
in 15 minutes.", code="locked")
        return super().clean()       # authenticate(); a failure fires
user_login_failed → LoginEvent


class ProfileForm(forms.ModelForm):                                      # 20
    class Meta:
        model = CustomUser
        fields = ["first_name", "last_name", "email", "phone_ext"]

    def clean_email(self):
        email = self.cleaned_data["email"].strip().lower()
        if email and
CustomUser.objects.filter(email__iexact=email).exclude(pk=self.instance.pk).e
xists():
            raise forms.ValidationError("That email belongs to another
user.")   # email login needs uniqueness
        return email
```

Invite service (11, 13)

```
# accounts/services.py
from django.contrib.auth import get_user_model
from django.contrib.auth.forms import PasswordResetForm
from django.db import transaction


class InviteForm(PasswordResetForm):
    def get_users(self, email):
        # the default skips users without a usable password — exactly the
people we invite
        return get_user_model()._default_manager.filter(email__iexact=email,
is_active=True)


def send_invite(user, request=None):
    form = InviteForm({"email": user.email})
    form.is_valid()
    form.save(
        request=request,
        domain_override=None if request else "localhost:8000",     # no
request → say where links point
        use_https=request.is_secure() if request else False,
        subject_template_name="registration/invite_subject.txt",
        email_template_name="registration/invite_email.txt",
    )


def invite_staff_user(username, email, role, team_lead=None, first_name="",
request=None):
    with transaction.atomic():
        user = get_user_model().objects.create_user(
            username=username, email=email, role=role, team_lead=team_lead,
first_name=first_name,
        )
        user.set_unusable_password()       # nobody knows a password until
they set one
        user.save(update_fields=["password"])
    send_invite(user, request)
    return user
```

```
# templates/registration/invite_subject.txt
Your Nexora CRM account
```

```
# templates/registration/invite_email.txt
Hi {{ user.get_short_name|default:user.get_username }},

An account has been created for you on Nexora CRM.
Username: {{ user.get_username }}

Set your password here (valid for 3 days):
{{ protocol }}://{{ domain }}{% url 'password_reset_confirm' uidb64=uid
token=token %}
```

Views (18–25, 31)

```
# accounts/views.py (add)
from django.contrib import messages
from django.contrib.auth.mixins import LoginRequiredMixin
from django.contrib.auth.views import LoginView, PasswordChangeView
from django.contrib.messages.views import SuccessMessageMixin
from django.urls import reverse, reverse_lazy
from django.views.generic.edit import UpdateView

from .forms import CRMAuthenticationForm, ProfileForm


class CRMLoginView(LoginView):                                      # 18,
19, 25
    form_class = CRMAuthenticationForm
    redirect_authenticated_user = True        # signed-in users skip the
login page
    template_name = "registration/login.html"

    def form_valid(self, form):
        response = super().form_valid(form)   # calls login(); session key
rotates here
        if form.cleaned_data.get("remember_me"):
            self.request.session.set_expiry(60 * 60 * 24 * 14)
        else:
            self.request.session.set_expiry(0)          # cookie dies when
the browser closes
        return response

    def get_default_redirect_url(self):       # used only when there's no
safe ?next=
        role = getattr(self.request.user, "role", "")
        if role == "agent":
            return reverse("accounts:my_queue")
```

```
        if role in {"manager", "admin"}:
            return reverse("reports:index")
        return super().get_default_redirect_url()


class CRMPasswordChangeView(PasswordChangeView):                        # 31
    def form_valid(self, form):
        response = super().form_valid(form)   # also update_session_auth_hash
→ this session survives

type(self.request.user).objects.filter(pk=self.request.user.pk).update(must_c
hange_password=False)
        messages.success(self.request, "Password changed.")
        return response


class ProfileView(LoginRequiredMixin, PageTitleMixin, SuccessMessageMixin,
UpdateView):   # 20
    form_class = ProfileForm
    template_name = "accounts/profile.html"
    success_url = reverse_lazy("accounts:profile")
    success_message = "Profile saved."
    page_title = "My profile"

    def get_object(self, queryset=None):
        return self.request.user               # no pk in the URL — you can
only edit yourself

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["logins"] = self.request.user.login_events.all()[:10]
        return context


class MyQueueView(AgentQueueView):                                      # 23
    page_title = "My queue"

    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, pk=request.user.pk, **kwargs)   #
managers get 404 (agents only)

# accounts/urls.py
path("me/", views.ProfileView.as_view(), name="profile"),
path("me/queue/", views.MyQueueView.as_view(), name="my_queue"),
```

```
# django_project/urls.py — 1, 18, 31 (custom views before the include so they
win)
from accounts.views import CRMLoginView, CRMPasswordChangeView

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/login/", CRMLoginView.as_view(), name="login"),
    path("accounts/password_change/", CRMPasswordChangeView.as_view(),
name="password_change"),
    path("accounts/", include("django.contrib.auth.urls")),   # logout,
password_change_done, password_reset*...
    path("team/", include("accounts.urls")),
    ...
]
```

```
# leads/views.py — 21, 22, 4
from django.contrib.auth.mixins import LoginRequiredMixin


class LogCallView(SuccessMessageMixin, FormView):
    ...
    def form_valid(self, form):
        log_call(self.lead, self.request.user, **form.cleaned_data)    # 21:
the caller is whoever is signed in
        return super().form_valid(form)


class MyLeadsView(LeadListView):                                       # 22
    page_title = "My leads"

    def get_queryset(self):
        return super().get_queryset().filter(assigned_to=self.request.user)

# leads/urls.py
path("mine/", views.MyLeadsView.as_view(), name="mine"),

# 4 — explicit protection (works with or without the middleware)
class LeadCreateView(LoginRequiredMixin, JsonFormMixin, AuditMixin, ...):   #
LoginRequiredMixin first
    ...

from django.contrib.auth.decorators import login_required
```

```
@login_required
def quick_add_many(request):
    ...
```

```
# core/views.py — DashboardView.get_context_data (24)
from leads.models import CallLog
from leads.queries import OPEN_STATUSES

user = self.request.user
context.update(
    my_open=Lead.objects.filter(assigned_to=user, is_active=True,
status__in=OPEN_STATUSES).count(),
    my_calls_today=CallLog.objects.filter(agent=user,
started_at__date=timezone.localdate()).count(),
)
```

```
<!-- core/dashboard.html — 24 -->
<h1>Welcome back, {{ user.first_name|default:user.username }}</h1>
<p>{{ my_open }} open lead{{ my_open|pluralize }} · {{ my_calls_today }}
call{{ my_calls_today|pluralize }} today</p>
```

27. Delete `@method_decorator(csrf_exempt, name="dispatch")` from `LeadStatusUpdateView`, `LeadDeleteView` (the JSON one), `QuickAddView` and `create_lead`. Browser forms already send `{% csrf_token %}`; JavaScript callers send the `X-CSRFToken` header (Day 27). Endpoints for machines keep the API key instead.

Templates (1–3, 8–9, 20, 30)

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}
{% block title %}Sign in · {{ block.super }}{% endblock title %}
{% block content %}
<h1>Sign in</h1>
<form method="post">{% csrf_token %}
  {{ form.as_div }}                                  {# username, password,
remember_me + errors #}
  <input type="hidden" name="next" value="{{ next }}">
  <button type="submit">Sign in</button>
</form>
<p><a href="{% url 'password_reset' %}">Forgot your password?</a></p>
{% endblock content %}
```

```
<!-- templates/partials/_navbar.html — 2, 3 -->
{% load static %}
<header class="topbar">
  <img src="{% static 'img/logo.png' %}" alt="Nexora" height="28">
  <span>{{ page_title|default:"Nexora CRM" }}</span>
  {% if user.is_authenticated %}
    <span class="spacer"></span>
    <a href="{% url 'accounts:profile' %}">{{
user.get_full_name|default:user.username }}</a>
    <span class="muted">{{ user.get_role_display }}</span>
    <form method="post" action="{% url 'logout' %}" class="inline">{%
csrf_token %}
      <button type="submit">Sign out</button>   {# GET logout was removed in
Django 5.0 #}
    </form>
  {% endif %}
</header>
```

```
<!-- templates/base.html — sidebar, 3 + 30 -->
<aside class="sidebar">
  <a href="{% url 'home' %}" class="brand">Nexora CRM</a>
  {% if user.is_authenticated %}
    {% with ns=request.resolver_match.namespace
name=request.resolver_match.url_name %}
      <a href="{% url 'dashboard' %}" class="{% if name == 'dashboard'
%}active{% endif %}">Dashboard</a>
      <a href="{% url 'leads:mine' %}">My leads</a>
      <a href="{% url 'leads:list' %}" class="{% if ns == 'leads' %}active{%
endif %}">All leads</a>
      <a href="{% url 'clients:list' %}" class="{% if ns == 'clients'
%}active{% endif %}">Clients</a>
      {% if user.role != "agent" %}
        <a href="{% url 'brokers:list' %}" class="{% if ns == 'brokers'
%}active{% endif %}">Brokers</a>
        <a href="{% url 'reports:index' %}" class="{% if ns == 'reports'
%}active{% endif %}">Reports</a>
      {% endif %}
    {% endwith %}
  {% endif %}
</aside>
```

Hiding a link is not security — Day 16 blocks the views themselves (30).

```
<!-- templates/registration/password_change_form.html — 8 -->
{% extends "base.html" %}
{% block content %}
<h1>Change password</h1>
<form method="post">{% csrf_token %}{{ form.as_div }}<button
type="submit">Change password</button></form>
{% endblock content %}

<!-- templates/registration/password_change_done.html -->
{% extends "base.html" %}
{% block content %}<h1>Password changed</h1><p><a href="{% url 'dashboard'
%}">Back to the dashboard</a></p>{% endblock content %}
```

```
<!-- 9: the four reset pages, all in templates/registration/ -->

<!-- password_reset_form.html -->
{% extends "base.html" %}
{% block content %}
<h1>Reset your password</h1>
<form method="post">{% csrf_token %}{{ form.as_div }}<button
type="submit">Email me a link</button></form>
{% endblock content %}

<!-- password_reset_done.html -->
{% extends "base.html" %}
{% block content %}<h1>Check your inbox</h1><p>If that email belongs to an
account, a link is on its way.</p>{% endblock content %}

<!-- password_reset_confirm.html — also the invite landing page -->
{% extends "base.html" %}
{% block content %}
{% if validlink %}
  <h1>Set a new password</h1>
  <form method="post">{% csrf_token %}{{ form.as_div }}<button
type="submit">Save password</button></form>
{% else %}
  <h1>Link expired</h1><p>Ask your manager to send a new invite.</p>
{% endif %}
{% endblock content %}

<!-- password_reset_complete.html -->
{% extends "base.html" %}
{% block content %}<h1>Password set</h1><p><a href="{% url 'login' %}">Sign
in</a></p>{% endblock content %}
```

```
# templates/registration/password_reset_subject.txt
Reset your Nexora CRM password

# templates/registration/password_reset_email.html   (rendered as the plain-text body)
Hi {{ user.get_short_name|default:user.get_username }},

Someone asked to reset the password for {{ user.get_username }} on Nexora CRM.
If that was you, open this link within 3 days:
{{ protocol }}://{{ domain }}{% url 'password_reset_confirm' uidb64=uid token=token %}

If it wasn't, ignore this email.
```

```
<!-- accounts/templates/accounts/profile.html — 20 -->
{% extends "base.html" %}
{% block content %}
<h1>My profile</h1>
<form method="post">{% csrf_token %}{{ form.as_div }}<button type="submit">Save</button></form>
<p><a href="{% url 'password_change' %}">Change password</a></p>
<h2>Recent sign-ins</h2>
<ul>{% for e in logins %}<li>{{ e.created_at|date:"d M Y H:i" }} · {{ e.ip|default:"?" }}</li>{% empty %}<li>—</li>{% endfor %}</ul>
{% endblock content %}
```

Admin actions (12, 36)

```
# accounts/admin.py — CustomUserAdmin
from .models import LoginEvent
from .services import send_invite

    actions = ["send_invites", "unlock"]

    @admin.action(description="Send set-password invite")
    def send_invites(self, request, queryset):
        users = list(queryset.exclude(email=""))
        for user in users:
            send_invite(user, request)
        self.message_user(request, f"Invites sent to {len(users)} user(s).")

    @admin.action(description="Unlock (clear failed sign-ins)")
    def unlock(self, request, queryset):
        names = list(queryset.values_list("username", flat=True)) + list(
            queryset.exclude(email="").values_list("email", flat=True))     #
people may type either
        n, _ = LoginEvent.objects.filter(username__in=names,
success=False).delete()
        self.message_user(request, f"Cleared {n} failed attempt(s).")
```

Shell (7, 15, 33–35)

```
# python manage.py shell
from django.contrib.auth import authenticate
from django.contrib.auth.hashers import check_password, make_password
from django.contrib.auth.password_validation import validate_password
from django.contrib.auth.tokens import default_token_generator
from django.urls import reverse
from django.utils.encoding import force_bytes
from django.utils.http import urlsafe_base64_encode
from accounts.models import CustomUser

authenticate(username="agent1", password="nexora123")      # 7 → <CustomUser:
Agent1>
authenticate(username="agent1", password="wrong")          # None (and a
failed LoginEvent row)
u = CustomUser.objects.get(username="agent1")
u.check_password("nexora123")                              # True

u8 = CustomUser.objects.get(username="agent8")              # 15
u8.is_active = False
u8.save(update_fields=["is_active"])
```

```
authenticate(username="agent8", password="nexora123")      # None

u.password[:20]                     # 33: 'pbkdf2_sha256$...' before,
'argon2$argon2id$...' after next login

h = make_password("S3cure-pass-2026")                        # 34
check_password("S3cure-pass-2026", h)                       # True
try:
    validate_password("nexora123", user=u)
except Exception as e:
    print(e.messages)               # too short, contains company name, ...

uid = urlsafe_base64_encode(force_bytes(u.pk))              # 35
token = default_token_generator.make_token(u)
reverse("password_reset_confirm", kwargs={"uidb64": uid, "token": token})
default_token_generator.check_token(u, token)               # True
u.set_password("Another-pass-2026"); u.save()
default_token_generator.check_token(u, token)               # False — tokens
die when the password or last_login changes
```

Tests (28, 37–40)

```
# accounts/tests.py
from django.contrib.auth import get_user_model
from django.core import mail
from django.test import Client, TestCase
from django.urls import reverse

from .services import invite_staff_user

PASSWORD = "Str0ng-pass-2026"


class AuthTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.agent = get_user_model().objects.create_user(
            "auth_agent", email="agent@nexora.test", password=PASSWORD,
role="agent",
        )

    def test_anonymous_redirected_to_login(self):                      # 37
        response = self.client.get(reverse("leads:list"))
        self.assertRedirects(response, f"{reverse('login')}?next=/leads/",
fetch_redirect_response=False)
```

```
    def test_login_with_email(self):                                    # 38
        response = self.client.post(reverse("login"), {"username":
"agent@nexora.test", "password": PASSWORD})
        self.assertEqual(response.status_code, 302)
        self.assertEqual(int(self.client.session["_auth_user_id"]),
self.agent.pk)

    def test_invite_email(self):                                        # 39
        invite_staff_user("new_agent", "new@nexora.test", role="agent")
        self.assertEqual(len(mail.outbox), 1)                    # tests use
the in-memory backend
        self.assertIn("/accounts/reset/", mail.outbox[0].body)

self.assertFalse(get_user_model().objects.get(username="new_agent").has_usabl
e_password())

    def test_lockout_after_five_failures(self):                         # 40
        for _ in range(5):
            self.client.post(reverse("login"), {"username": "auth_agent",
"password": "wrong"})
        response = self.client.post(reverse("login"), {"username":
"auth_agent", "password": PASSWORD})
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Too many failed attempts")

    def test_password_change_signs_out_other_sessions(self):            # 28
        other = Client()
        self.assertTrue(self.client.login(username="auth_agent",
password=PASSWORD))
        self.assertTrue(other.login(username="auth_agent",
password=PASSWORD))
        self.client.post(reverse("password_change"), {
            "old_password": PASSWORD,
            "new_password1": "Brand-new-pass-77",
            "new_password2": "Brand-new-pass-77",
        })
        self.assertEqual(self.client.get(reverse("dashboard")).status_code,
200)   # this session kept
        self.assertEqual(other.get(reverse("dashboard")).status_code, 302)
# the other one is out
```

```
python manage.py test
git add -A && git commit -m "day 15"
```

Day 16 — Permissions and groups

Goal: who can see and do what. Agents see their own leads; team leads see their team's; managers see everything. Pages a user can't open return 403; leads they can't see return 404, so the URL doesn't confirm the lead exists.

Two layers: **model permissions** (Django's `app.codename` system, via groups) decide which *actions* a role may take; **row scoping** (`visible_leads()`) decides which *rows* a user may touch.

A. Drills (1–10) — Django's permission system

- [ ] **1.** List the default permissions Django made for `Lead` (add / change / delete / view).
- [ ] **2.** `has_perm` , `has_perms` , `get_all_permissions` for an agent and a superuser.
- [ ] **3.** Custom permissions on `Lead.Meta` : `view_all_leads` , `reassign_lead` , `export_lead` .
- [ ] **4.** In the shell: create groups Agents / Team leads / Managers, give them permissions, add a user.
- [ ] **5.** The permission cache: grant a permission, see `has_perm` still say False, re-fetch the user.
- [ ] **6.** `@permission_required("leads.add_lead", raise_exception=True)` on `quick_add_many` .
- [ ] **7.** `PermissionRequiredMixin` on `LeadCSVView` (`leads.export_lead`).
- [ ] **8.** `{% if perms.leads.export_lead %}` around the Export link.
- [ ] **9.** `UserPassesTestMixin` on `ReportsView` : team leads, managers, admins only.
- [ ] **10.** `templates/403.html` showing the reason passed to `PermissionDenied` .

B. Core (11–20) — roles and row scoping

- [ ] **11.** `accounts/roles.py` : role → group map and group → permissions map; a `post_migrate` receiver creates/syncs the groups on every `migrate` .
- [ ] **12.** `post_save` on `CustomUser` : the user's role group follows their `role` .
- [ ] **13.** `leads/permissions.py` : `sees_everything(user)` and `visible_leads(user, qs=None)` .
- [ ] **14.** `LeadQueryMixin` starts from `visible_leads(request.user)` .

- [ ] **15.** `LeadScopeMixin.get_queryset()` for detail/edit/status/delete views → other people's leads 404.
- [ ] **16.** Views that load the lead in `setup()` (log call, convert, archive, workspace) use the scoped queryset too.
- [ ] **17.** Object rule `can_edit_lead(user, lead)`: agents can't edit qualified leads (they're with onboarding); `{% can_edit lead as editable %}` hides the buttons.
- [ ] **18.** Bulk "assign" requires `leads.reassign_lead`; bulk actions only touch visible leads.
- [ ] **19.** `visible_clients(user)` for client list, detail, edit, delete.
- [ ] **20.** `agent_leaderboard(team_lead=...)`: team leads see only their team on the reports page.

C. Nexora CRM (21–30)

- [ ] **21.** Reports page by role: team lead → team leaderboard only; managers → everything.
- [ ] **22.** `TeamView` at `/team/` (managers): each team lead with members and open-lead counts.
- [ ] **23.** Agent pages and queues scoped: agents see only themselves, team leads their team.
- [ ] **24.** Admin export action gated by `permissions=["export"]` + `has_export_permission()`.
- [ ] **25.** `LeadAdmin.get_queryset()` scoped for non-superuser staff.
- [ ] **26.** `LeadAdmin.has_change_permission(request, obj)` uses `can_edit_lead`.
- [ ] **27.** JSON endpoints (page API, list JSON, recent) scoped like the pages.
- [ ] **28.** `PermissionDenied("Qualified leads are read-only for agents…")` — message appears on the 403 page.
- [ ] **29.** Permanent delete only with `leads.delete_lead` (Managers).
- [ ] **30.** Hide bulk bar, Export and Delete when the user lacks the permission.

D. Challenge (31–40)

- [ ] **31.** `LeadObjectPermissionBackend`: `user.has_perm("leads.change_lead", lead)` answers per object.
- [ ] **32.** `ObjectPermissionRequiredMixin`: `PermissionRequiredMixin` that checks against `get_object()`.

- [ ] **33.** `@role_required("manager", "admin")` decorator for function views.
- [ ] **34.** Print the permission matrix: each group and its codenames.
- [ ] **35.** Give one agent `export_lead` directly (`user_permissions`) without changing their group.
- [ ] **36.** Sweep test: every lead URL returns 404 for a lead the user can't see.
- [ ] **37.** Test: an agent's list shows their lead and not another agent's.
- [ ] **38.** Test: an agent gets 404 on another agent's lead detail.
- [ ] **39.** Test: a team lead sees team members' leads only.
- [ ] **40.** Test: CSV export is 403 for an agent and 200 for a manager.

Solutions

A. Drills

3.

```
# leads/models.py — Lead.Meta
permissions = [
    ("view_all_leads", "Can see every lead"),
    ("reassign_lead", "Can reassign leads"),
    ("export_lead", "Can export leads"),
]
```

```
python manage.py makemigrations leads && python manage.py migrate   #
permissions are created after migrate
```

1–2, 4–5.

```
# python manage.py shell
from django.contrib.auth.models import Group, Permission
from accounts.models import CustomUser

Permission.objects.filter(content_type__app_label="leads",
content_type__model="lead") \
    .values_list("codename", flat=True)
# ['add_lead', 'change_lead', 'delete_lead', 'view_lead', 'view_all_leads',
'reassign_lead', 'export_lead']

agent = CustomUser.objects.get(username="agent1")                    # 2
```

```
agent.has_perm("leads.change_lead")                                    # False
— no groups yet
agent.has_perms(["leads.view_lead", "leads.change_lead"])
agent.get_all_permissions()                                            # set
of "app.codename" strings
CustomUser.objects.filter(is_superuser=True).first().has_perm("anything.at_al
l")   # True

agents, _ = Group.objects.get_or_create(name="Agents")                # 4
agents.permissions.add(*Permission.objects.filter(codename__in=["view_lead",
"change_lead", "add_lead"]))
agent.groups.add(agents)

agent.has_perm("leads.export_lead")                                    # 5:
False — and now cached on this object
agent.user_permissions.add(Permission.objects.get(codename="export_lead"))
agent.has_perm("leads.export_lead")                                    # still
False (stale _perm_cache)
agent = CustomUser.objects.get(pk=agent.pk)                            # re-
fetch
agent.has_perm("leads.export_lead")                                    # True
```

6–7, 9.

```
# leads/views.py
from django.contrib.auth.decorators import permission_required
from django.contrib.auth.mixins import PermissionRequiredMixin


@permission_required("leads.add_lead", raise_exception=True)   # 403 instead
of a login redirect
def quick_add_many(request):
    ...


class LeadCSVView(PermissionRequiredMixin, LeadQueryMixin, View):
# 7
    permission_required = "leads.export_lead"
    ...

# reports/views.py
# 9
from django.contrib.auth.mixins import UserPassesTestMixin
```

```
class ReportsView(UserPassesTestMixin, PageTitleMixin, TemplateView):
    def test_func(self):
        user = self.request.user
        return user.is_superuser or user.role in {"team_lead", "manager",
"admin"}
    ...
```

8, 10.

```
<!-- leads/lead_list.html — 8 -->
{% if perms.leads.export_lead %}
  <a href="{% url 'leads:export' %}?{{ request.GET.urlencode }}">Export
CSV</a>
{% endif %}
```

```
<!-- templates/403.html — 10: Django passes the PermissionDenied message as
`exception` -->
{% extends "base.html" %}
{% block title %}Not allowed · {{ block.super }}{% endblock title %}
{% block content %}
<h1>Not allowed</h1>
<p>{{ exception|default:"You don't have permission to open this page." }}</p>
<p><a href="{% url 'dashboard' %}">Back to the dashboard</a></p>
{% endblock content %}
```

B. Roles and scoping

11–12.

```
# accounts/roles.py
ROLE_GROUPS = {"agent": "Agents", "team_lead": "Team leads", "manager":
"Managers", "admin": "Managers"}

GROUP_PERMS = {
    "Agents": [
        "leads.view_lead", "leads.add_lead", "leads.change_lead",
        "leads.view_calllog", "leads.add_calllog", "leads.view_tag",
        "clients.view_client",
    ],
}
```

```
GROUP_PERMS["Team leads"] = GROUP_PERMS["Agents"] + ["leads.reassign_lead",
"leads.export_lead"]
GROUP_PERMS["Managers"] = GROUP_PERMS["Team leads"] + [
    "leads.view_all_leads", "leads.delete_lead", "leads.add_tag",
"leads.change_tag",
    "clients.change_client", "clients.add_deposit", "brokers.view_broker",
]


def sync_role_groups(**kwargs):                                          # 11 —
idempotent
    from django.contrib.auth.models import Group, Permission
    for group_name, perms in GROUP_PERMS.items():
        group, _ = Group.objects.get_or_create(name=group_name)
        wanted = []
        for dotted in perms:
            app_label, codename = dotted.split(".")
            perm =
Permission.objects.filter(content_type__app_label=app_label,
codename=codename).first()
            if perm:                           # not created yet on early
post_migrate calls → next call adds it
                wanted.append(perm)
        group.permissions.set(wanted)
```

```
# accounts/apps.py
from django.db.models.signals import post_migrate


class AccountsConfig(AppConfig):
    ...
    def ready(self):
        from . import signals  # noqa: F401
        from .roles import sync_role_groups
        # fires once per app after every migrate (tests included); the last
call sees every permission
        post_migrate.connect(sync_role_groups,
dispatch_uid="accounts.sync_role_groups")
```

```
# accounts/signals.py (add)                                          # 12
from django.contrib.auth.models import Group
from django.db.models.signals import post_save

from .models import CustomUser
from .roles import GROUP_PERMS, ROLE_GROUPS


@receiver(post_save, sender=CustomUser)
def put_user_in_role_group(sender, instance, **kwargs):
    role_groups = set(GROUP_PERMS)
    wanted = ROLE_GROUPS.get(instance.role)
    instance.groups.remove(*Group.objects.filter(name__in=role_groups -
{wanted}))   # keep non-role groups
    if wanted and (group := Group.objects.filter(name=wanted).first()):
        instance.groups.add(group)
```

```
python manage.py migrate        # creates the three groups
python manage.py shell -c "from accounts.models import CustomUser as U;
[u.save() for u in U.objects.all()]"   # sync existing users
```

13, 17, 19.

```
# leads/permissions.py
from django.db.models import Q

FULL_ACCESS_ROLES = {"manager", "admin"}


def sees_everything(user):
    return (user.is_superuser
            or getattr(user, "role", "") in FULL_ACCESS_ROLES
            or user.has_perm("leads.view_all_leads"))


def visible_leads(user, qs=None):                                    # 13
    from .models import Lead
    qs = Lead.objects.all() if qs is None else qs
    if not user.is_authenticated:
        return qs.none()
    if sees_everything(user):
        return qs
```

```
    if user.role == "team_lead":
        return qs.filter(Q(assigned_to=user) |
Q(assigned_to__team_lead=user))
    return qs.filter(assigned_to=user)


def can_edit_lead(user, lead):                                      # 17 —
call only on visible leads
    if sees_everything(user) or user.role == "team_lead":
        return True
    return lead.assigned_to_id == user.pk and lead.status !=
lead.Status.QUALIFIED


def visible_clients(user, qs=None):                                 # 19
    from clients.models import Client
    qs = Client.objects.all() if qs is None else qs
    if not user.is_authenticated:
        return qs.none()
    if sees_everything(user):
        return qs
    if user.role == "team_lead":
        return qs.filter(Q(lead__assigned_to=user) |
Q(lead__assigned_to__team_lead=user))
    return qs.filter(lead__assigned_to=user)
```

14–18, 28–29.

```
# leads/views.py
from django.core.exceptions import PermissionDenied
from .permissions import can_edit_lead, visible_leads


class LeadScopeMixin:                                               # 15
    """Object lookups only see rows the user may see → everything else is a
404."""
    def get_queryset(self):
        return visible_leads(self.request.user, super().get_queryset())


class LeadEditPermissionMixin:                                      # 17,
28
    def get_object(self, queryset=None):
```

```
        lead = super().get_object(queryset)
        if not can_edit_lead(self.request.user, lead):
            raise PermissionDenied("Qualified leads are read-only for agents
— ask your team lead.")
        return lead


# 14 — first line of LeadQueryMixin.get_leads()
qs = visible_leads(self.request.user).filter(is_active=True)  #
.select_related(...)... as before


class LeadDetailView(LeadScopeMixin, PageTitleMixin, DetailView):
    def get_queryset(self):            # call super() so LeadScopeMixin runs
        return (super().get_queryset().filter(is_active=True)
                .select_related("assigned_to", "broker",
"client").prefetch_related("tags"))


class LeadUpdateView(LeadScopeMixin, LeadEditPermissionMixin, AuditMixin,
PageTitleMixin,
                     UserFormKwargsMixin, SuccessMessageMixin, UpdateView):
    def get_queryset(self):
        return super().get_queryset().filter(is_active=True)


class LeadStatusView(LeadScopeMixin, LeadEditPermissionMixin, PageTitleMixin,
SuccessMessageMixin, UpdateView):
    ...


class LeadConfirmDeleteView(PermissionRequiredMixin, LeadScopeMixin,
PageTitleMixin,
                            SuccessMessageMixin, DeleteView):           # 29
    permission_required = "leads.delete_lead"
    ...


# 16 — views that load the lead in setup(): LogCallView, ConvertLeadView,
LeadWorkspaceView
def setup(self, request, *args, **kwargs):
    super().setup(request, *args, **kwargs)
    self.lead = get_object_or_404(visible_leads(request.user),
```

```
pk=kwargs["lead_id"], is_active=True)
    if not can_edit_lead(request.user, self.lead):        # convert +
workspace edit the lead
        raise PermissionDenied("Qualified leads are read-only for agents —
ask your team lead.")

# LeadArchiveView.post
lead = get_object_or_404(visible_leads(request.user), pk=lead_id,
is_active=True)


# 18 — LeadBulkActionView.form_valid
def form_valid(self, form):
    qs = visible_leads(self.request.user, form.cleaned_data["leads"])   #
never trust posted ids
    action, now = form.cleaned_data["action"], timezone.now()
    if action == "assign" and not
self.request.user.has_perm("leads.reassign_lead"):
        raise PermissionDenied("You can't reassign leads.")
    ...
```

```
# leads/templatetags/crm_filters.py — 17
from leads.permissions import can_edit_lead


@register.simple_tag(takes_context=True)
def can_edit(context, lead):
    return can_edit_lead(context["request"].user, lead)
```

```
<!-- leads/lead_detail.html — 17, 30 -->
{% load crm_filters %}
{% can_edit lead as editable %}
{% if editable %}
  <a href="{% url 'leads:edit' lead.pk %}">Edit</a>
  <a href="{% url 'leads:status_edit' lead.pk %}">Change status</a>
{% endif %}
{% if perms.leads.reassign_lead %}{# reassign box #}{% endif %}
{% if perms.leads.delete_lead %}<a href="{% url 'leads:delete_confirm'
lead.pk %}">Delete permanently</a>{% endif %}

<!-- leads/lead_list.html — 30 -->
{% if perms.leads.reassign_lead %}<form id="bulk-form" ...>...</form>{% endif
%}
```

```
# clients/views.py — 19 (list, detail, edit, delete)
from leads.permissions import visible_clients


class ClientListView(PageTitleMixin, ListView):
    def get_queryset(self):
        qs =
visible_clients(self.request.user).select_related("broker").annotate(total_de
posits=deposits_total())
        ...


class ClientScopeMixin:
    def get_queryset(self):
        return visible_clients(self.request.user, super().get_queryset())


class ClientDetailView(ClientScopeMixin, PageTitleMixin, DetailView):
    def get_queryset(self):
        return super().get_queryset().select_related("broker",
"lead").prefetch_related("accounts__deposits")

# ClientUpdateView(ClientScopeMixin, ...), ClientDeleteView(ClientScopeMixin,
...) the same way
```

20–21.

```
# reports/queries.py
def agent_leaderboard(team_lead=None):
    qs = get_user_model().objects.filter(role="agent")
    if team_lead is not None:
        qs = qs.filter(team_lead=team_lead)
    return (qs.annotate(...)          # unchanged from Day 9
              ...)

# reports/views.py — ReportsView.get_context_data
user = self.request.user
full = sees_everything(user)
context.update(
    leaderboard=queries.agent_leaderboard(team_lead=None if full else user),
    brokers=queries.broker_report()[:10] if full else None,
    sources=queries.source_performance() if full else None,
    breaches=visible_leads(user, queries.sop_first_call_breaches())[:20],
    full_access=full,
)
```

```
<!-- reports/index.html: wrap the broker and source tables -->
{% if full_access %} ...broker table... ...sources table... {% endif %}
```

C. Nexora CRM

22–23.

```
# accounts/views.py
from django.contrib.auth.mixins import UserPassesTestMixin
from django.db.models import Count, Prefetch, Q
from leads.permissions import sees_everything
from leads.queries import OPEN_STATUSES


class TeamView(UserPassesTestMixin, PageTitleMixin, TemplateView):        #
22
    template_name = "accounts/team.html"
    page_title = "Team"

    def test_func(self):
        return sees_everything(self.request.user)

    def get_context_data(self, **kwargs):
```

```
        context = super().get_context_data(**kwargs)
        User = get_user_model()
        members = User.objects.filter(role="agent").annotate(
            open_n=Count("leads", filter=Q(leads__status__in=OPEN_STATUSES)),
        ).order_by("username")
        context["team_leads"] = (User.objects.filter(role="team_lead")
                                 .prefetch_related(Prefetch("team_members",
queryset=members)))
        return context


class AgentScopeMixin:                                                  #
23
    def scoped_agents(self):
        qs =
get_user_model().objects.filter(role="agent").select_related("team_lead")
        user = self.request.user
        if sees_everything(user):
            return qs
        if user.role == "team_lead":
            return qs.filter(team_lead=user)
        return qs.filter(pk=user.pk)


class AgentDetailView(AgentScopeMixin, PageTitleMixin, DetailView):
    def get_queryset(self):
        return self.scoped_agents()


class AgentQueueView(AgentScopeMixin, PageTitleMixin, ListView):
    def get_queryset(self):
        self.agent = get_object_or_404(self.scoped_agents(),
pk=self.kwargs["pk"])
        ...

# accounts/urls.py
path("", views.TeamView.as_view(), name="team"),
```

```
<!-- accounts/templates/accounts/team.html -->
{% extends "base.html" %}
{% block content %}
<h1>Team</h1>
{% for tl in team_leads %}
  <h2>{{ tl }}</h2>
  <table class="table">
    <tr><th>Agent</th><th>Open leads</th></tr>
    {% for a in tl.team_members.all %}
      <tr><td><a href="{% url 'accounts:agent_detail' a.pk %}">{{ a }}</a>
</td><td>{{ a.open_n }}</td></tr>
    {% endfor %}
  </table>
{% endfor %}
{% endblock content %}
```

24–26.

```
# core/admin_actions.py — 24
@admin.action(description="Export selected to CSV", permissions=["export"])
# → has_export_permission()
def export_as_csv(modeladmin, request, queryset):
    ...
# unchanged


class ExportPermissionMixin:
    def has_export_permission(self, request):
        opts = self.opts
        return request.user.has_perm(f"
{opts.app_label}.export_{opts.model_name}")
        # clients/deposits have no export_* permission → only superusers see
the action there
```

```
# leads/admin.py — 24–26
from core.admin_actions import ExportPermissionMixin, export_as_csv
from .permissions import can_edit_lead, visible_leads


@admin.register(Lead)
class LeadAdmin(ExportPermissionMixin, admin.ModelAdmin):
    ...
```

```
    def get_queryset(self, request):                                    # 25
        qs = (super().get_queryset(request)
              .select_related("assigned_to", "broker", "client")
              .annotate(_call_count=Count("calls")))
        return qs if request.user.is_superuser else
visible_leads(request.user, qs)

    def has_change_permission(self, request, obj=None):                 # 26
        allowed = super().has_change_permission(request, obj)
        if obj is None or not allowed:
            return allowed
        return can_edit_lead(request.user, obj)

# ClientAdmin / DepositAdmin: add ExportPermissionMixin to their bases too
```

27.

```
# leads/views.py
class RecentLeadsView(JsonResponseMixin, View):
    def get(self, request):
        ids = request.session.get("recent_leads", [])
        rows = {r["id"]: r for r in
visible_leads(request.user).filter(pk__in=ids).values(*LEAD_FIELDS)}
        return self.render_to_json([rows[i] for i in ids if i in rows])
# LeadPageAPI and LeadListJSONView already go through
LeadQueryMixin.get_leads() → scoped by 14
```

D. Challenge

31–33.

```
# leads/backends.py                                                  # 31
from .permissions import can_edit_lead, visible_leads


class LeadObjectPermissionBackend:
    """Answers has_perm(perm, obj) for Lead objects; ModelBackend answers the
model-level question."""

    def authenticate(self, request, **kwargs):
        return None                         # never logs anyone in

    def has_perm(self, user_obj, perm, obj=None):
```

```
        from .models import Lead
        if obj is None or not isinstance(obj, Lead) or not
user_obj.is_active:
            return False
        if perm == "leads.view_lead":
            return visible_leads(user_obj).filter(pk=obj.pk).exists()
        if perm == "leads.change_lead":
            return user_obj.has_perm("leads.change_lead") and
can_edit_lead(user_obj, obj)
        return False

# settings.py
AUTHENTICATION_BACKENDS = [
    "accounts.backends.EmailOrUsernameBackend",
    "leads.backends.LeadObjectPermissionBackend",
]

# shell
agent.has_perm("leads.change_lead", own_lead)        # True
agent.has_perm("leads.change_lead", someone_elses)   # False
```

```
# core/mixins.py — 32
from django.contrib.auth.mixins import PermissionRequiredMixin


class ObjectPermissionRequiredMixin(PermissionRequiredMixin):
    def has_permission(self):
        return self.request.user.has_perms(self.get_permission_required(),
self.get_object())

# usage
class LeadUpdateView(ObjectPermissionRequiredMixin, LeadScopeMixin, ...):
    permission_required = "leads.change_lead"
```

```
# core/decorators.py — 33
from django.core.exceptions import PermissionDenied


def role_required(*roles):
    def decorator(view):
        @functools.wraps(view)
        def wrapper(request, *args, **kwargs):
            if request.user.is_superuser or getattr(request.user, "role",
None) in roles:
                return view(request, *args, **kwargs)
            raise PermissionDenied(f"Only {', '.join(roles)} can open this
page.")
        return wrapper
    return decorator


@role_required("manager", "admin")
def quick_add_many(request):
    ...
```

34–35.

```
for group in Group.objects.prefetch_related("permissions").order_by("name"):
# 34
    print(f"{group.name:<11}", ", ".join(sorted(p.codename for p in
group.permissions.all())))

agent = CustomUser.objects.get(username="agent2")
# 35
agent.user_permissions.add(Permission.objects.get(codename="export_lead"))
# on top of the group
```

Tests (36–40)

```
# leads/tests.py
from .permissions import visible_leads


class PermissionTests(TestCase):
    @classmethod
    def setUpTestData(cls):
```

```
        User = get_user_model()
        cls.tl = User.objects.create_user("p_tl", password="x",
role="team_lead")
        cls.a1 = User.objects.create_user("p_a1", password="x", role="agent",
team_lead=cls.tl)
        cls.a2 = User.objects.create_user("p_a2", password="x", role="agent")
        cls.manager = User.objects.create_user("p_mgr", password="x",
role="manager")
        cls.l1 = Lead.objects.create(name="Perm One", phone="9715099920001",
assigned_to=cls.a1)
        cls.l2 = Lead.objects.create(name="Perm Two", phone="9715099920002",
assigned_to=cls.a2)

    def test_agent_list_is_scoped(self):                                # 37
        self.client.force_login(self.a1)
        names = {lead.name for lead in
self.client.get(reverse("leads:list")).context["leads"]}
        self.assertIn("Perm One", names)
        self.assertNotIn("Perm Two", names)

    def test_foreign_lead_is_404_everywhere(self):                      # 36 +
38
        self.client.force_login(self.a1)
        for name in ["detail", "edit", "status_edit", "log_call", "convert",
"workspace"]:
            with self.subTest(url=name):
                response = self.client.get(reverse(f"leads:{name}", args=
[self.l2.pk]))
                self.assertEqual(response.status_code, 404)

    def test_team_lead_scope(self):                                     # 39
        visible = visible_leads(self.tl)
        self.assertIn(self.l1, visible)
        self.assertNotIn(self.l2, visible)

    def test_export_permission(self):                                   # 40
        self.client.force_login(self.a1)

self.assertEqual(self.client.get(reverse("leads:export")).status_code, 403)
        self.client.force_login(self.manager)      # Managers group (via
role) has export_lead

self.assertEqual(self.client.get(reverse("leads:export")).status_code, 200)
```

The groups exist in tests because `post_migrate` also runs when the test database is built; `create_user` triggers the role-group receiver.

```
python manage.py test
git add -A && git commit -m "day 16"
```

Day 17 — Search, filters, sorting

Goal: the filtering layer most Django projects actually use — `django-filter` — replacing Day 13's hand-written `LeadFilterForm` plumbing. Then the CRM's search experience: facet counts, removable filter chips, click-to-sort headers, saved views, a global search box with ranking, highlighting and type-ahead.

Everything stays scoped by Day 16's `visible_leads()`.

A. Drills (1–10) — django-filter

- [ ] **1.** Install `django-filter`, add `django_filters` to `INSTALLED_APPS`.
- [ ] **2.** Minimal `LeadFilter(FilterSet)` with `Meta.fields` for status and source; use it in the shell (`.qs`, `.form`).
- [ ] **3.** `CharFilter(method=...)` for `q` using Day 7's `search_q`.
- [ ] **4.** `RangeFilter` on expected deposit (`deposit_min` / `deposit_max`).
- [ ] **5.** `DateFromToRangeFilter` on `created_at` (`created_after` / `created_before`).
- [ ] **6.** `ModelChoiceFilter` for agent whose queryset is a callable that receives `request`.
- [ ] **7.** `ModelMultipleChoiceFilter` for tags by name (`?tags=vip&tags=whale`).
- [ ] **8.** `BooleanFilter(method=...)`: `has_client` (converted or not).
- [ ] **9.** `OrderingFilter` with public names (`?o=-deposit`), NULLs last.
- [ ] **10.** Bound vs unbound filtersets, `is_valid()`, form errors; what happens to an invalid field.

B. Core (11–20) — wire it into the CRM

- [ ] **11.** `LeadQueryMixin` builds the annotated, scoped queryset and returns `LeadFilter(...).qs` (filter form from Day 13 retired).
- [ ] **12.** Same page with `django_filters.views.FilterView` — compare with your `ListView` version.
- [ ] **13.** `period` filter: today / last 7 days / last 30 days / this month.
- [ ] **14.** Tag mode: `?tag_mode=any` (OR) vs default all (AND).
- [ ] **15.** Status facets: count per status for the *current* filters, ignoring the status filter itself.
- [ ] **16.** Active filter chips, each with a link that removes just that filter.

- [ ] **17.** `{% sort_link "deposit" "Deposit" %}` header tag: toggles direction, shows ▲ / ▼.
- [ ] **18.** Multi-column sort: `?o=status,-created`.
- [ ] **19.** `SavedView` model (user, name, querystring) + "Save this view" form; list your views above the table.
- [ ] **20.** Remember the last filters in the session; `?reset=1` clears them.

C. Nexora CRM (21–30) — search

- [ ] **21.** Global search `/search/?q=`: leads, clients, trading accounts, brokers — top 5 each with totals, all permission-scoped.
- [ ] **22.** Ranking for leads with `Case` / `When`: exact phone 100, exact name 80, name starts 50, name contains 10, email 5.
- [ ] **23.** `|highlight:q` filter wrapping matches in `<mark>` without breaking HTML escaping.
- [ ] **24.** Type-ahead JSON `/search/suggest/?q=` (≥ 2 chars, 10 results: type, label, url).
- [ ] **25.** Phone search that understands `050 999 1003` and `+971…` as well as the last 4 digits.
- [ ] **26.** `ClientFilter`: search, KYC, account type, broker, joined range, minimum total deposits, ordering.
- [ ] **27.** `CallLogFilter`: outcomes (multi), agent, date range, minimum talk time.
- [ ] **28.** Zero results → "Try removing: …" with the chip links.
- [ ] **29.** CSV export uses the same `LeadFilter` (export exactly what's on screen).
- [ ] **30.** Global search box in the top bar on every page.

D. Challenge (31–40)

- [ ] **31.** SQLite FTS5 full-text index on lead notes: migration with virtual table + triggers; `search_notes()` with `MATCH`.
- [ ] **32.** "Did you mean…?" with `difflib.get_close_matches` when a name search finds nothing.
- [ ] **33.** `min_calls` filter on the `call_count` annotation.
- [ ] **34.** Strict mode: invalid filters → empty result + errors (instead of silently ignoring them).

- [ ] **35.** Label chips with display values (agent's name, not id).
- [ ] **36.** `explain()` on `icontains` vs exact phone: see which uses an index on SQLite.
- [ ] **37.** Test: `LeadFilter` status/agent combinations.
- [ ] **38.** Test: facets follow the other filters.
- [ ] **39.** Test: ranking order exact → starts with → contains.
- [ ] **40.** Test: `highlight` escapes HTML in the text.

Solutions

A `FilterSet` is a form (`.form`) plus a queryset recipe (`.qs`). Each filter turns one GET param into one `.filter()`; params that fail validation are skipped.

Setup and the filters (1–10, 13–14, 26–27, 33)

```
python -m pip install django-filter
```

```
# settings.py
INSTALLED_APPS = [..., "django_filters", ...]
```

```
# leads/filters.py
from datetime import timedelta

import django_filters as df
from django import forms
from django.contrib.auth import get_user_model
from django.db.models import Count, F
from django.utils import timezone

from .models import CallLog, Lead, Tag
from .queries import FINDERS, search_q


class MiniLeadFilter(df.FilterSet):                                    # 2 —
generated filters
    class Meta:
        model = Lead
        fields = ["status", "source"]
```

```
def agents_for(request):                                                # 6 —
called with the request
    qs = get_user_model().objects.filter(role="agent",
is_active=True).order_by("username")
    user = getattr(request, "user", None)
    if user is None or not user.is_authenticated:
        return qs
    from .permissions import sees_everything
    if sees_everything(user):
        return qs
    if user.role == "team_lead":
        return qs.filter(team_lead=user)
    return qs.filter(pk=user.pk)


class NullsLastOrderingFilter(df.OrderingFilter):                       # 9
    def get_ordering_value(self, param):
        descending = param.startswith("-")
        name = param.lstrip("-")
        field = self.param_map.get(name, name)        # public name → model
field / annotation
        return F(field).desc(nulls_last=True) if descending else
F(field).asc(nulls_last=True)


PERIODS = [("today", "Today"), ("7d", "Last 7 days"), ("30d", "Last 30
days"), ("month", "This month")]


class LeadFilter(df.FilterSet):
    q = df.CharFilter(method="filter_q", label="Search")
# 3
    status = df.ChoiceFilter(choices=Lead.Status.choices, empty_label="Any
status")
    source = df.ChoiceFilter(choices=Lead.Source.choices, empty_label="Any
source")
    agent = df.ModelChoiceFilter(field_name="assigned_to",
queryset=agents_for, empty_label="Any agent")
    tags = df.ModelMultipleChoiceFilter(
# 7, 14
        queryset=Tag.objects.all(), to_field_name="name",
method="filter_tags",
        widget=forms.CheckboxSelectMultiple,
```

```
    )
    deposit = df.RangeFilter(field_name="expected_deposit", label="Deposit")
# 4
    created = df.DateFromToRangeFilter(field_name="created_at",
label="Created")         # 5
    period = df.ChoiceFilter(choices=PERIODS, method="filter_period",
empty_label="Any time")   # 13
    has_client = df.BooleanFilter(method="filter_has_client",
label="Converted")        # 8
    finder = df.ChoiceFilter(
# keeps the Day 7 chips working
        choices=[(k, k.replace("_", " ").title()) for k in FINDERS],
method="filter_finder", empty_label="—",
    )
    min_calls = df.NumberFilter(field_name="call_count", lookup_expr="gte",
label="Calls ≥")   # 33
    o = NullsLastOrderingFilter(fields=(
# 9, 18
        ("created_at", "created"), ("name", "name"), ("expected_deposit",
"deposit"),
        ("status", "status"), ("call_count", "calls"), ("last_call_at",
"last_call"),
    ))

    class Meta:
        model = Lead
        fields = []                    # everything is declared above

    def filter_q(self, queryset, name, value):
        return queryset.filter(search_q(value))

    def filter_tags(self, queryset, name, value):
        if not value:
            return queryset
        if self.data.get("tag_mode") == "any":                    # 14: OR
            return queryset.filter(tags__in=value).distinct()
        for tag in value:                                          # default:
AND (one join per tag)
            queryset = queryset.filter(tags=tag)
        return queryset

    def filter_period(self, queryset, name, value):
        now = timezone.localtime()
```

```
        midnight = now.replace(hour=0, minute=0, second=0, microsecond=0)
        start = {"today": midnight, "7d": now - timedelta(days=7),
                 "30d": now - timedelta(days=30), "month":
midnight.replace(day=1)}[value]
        return queryset.filter(created_at__gte=start)

    def filter_has_client(self, queryset, name, value):
        return queryset.filter(client__isnull=not value)

    def filter_finder(self, queryset, name, value):
        return queryset.filter(pk__in=FINDERS[value]().values("pk"))


class CallLogFilter(df.FilterSet):                                      # 27
    outcome = df.MultipleChoiceFilter(choices=CallLog.Outcome.choices,
widget=forms.CheckboxSelectMultiple)
    agent = df.ModelChoiceFilter(queryset=agents_for, empty_label="Any
agent")
    started = df.DateFromToRangeFilter(field_name="started_at", label="Date")
    min_talk = df.NumberFilter(field_name="duration_seconds",
lookup_expr="gte", label="Talk ≥ (s)")
    o = df.OrderingFilter(fields=(("started_at", "when"),
("duration_seconds", "talk")))

    class Meta:
        model = CallLog
        fields = []


# ---- helpers for the list page (15, 16, 35) ------------------------------
---
def status_facets(request, base_qs):
    """Count per status for the current filters, ignoring the status filter
itself."""
    data = request.GET.copy()
    for key in ("status", "page", "o"):
        data.pop(key, None)
    filtered = LeadFilter(data, queryset=base_qs, request=request).qs
    rows = (Lead.objects.filter(pk__in=filtered.values("pk"))   # subquery
keeps HAVING/joins honest
            .order_by().values("status").annotate(n=Count("id")))
    return {row["status"]: row["n"] for row in rows}
```

```
SKIP_PARAMS = {"page", "per_page", "o", "partial", "tag_mode"}


def display_value(filterset, key, value):                                  # 35
    field = filterset.form.fields.get(key)
    for choice_value, label in getattr(field, "choices", None) or []:
        if str(choice_value) == value:
            return str(label)          # agent's name instead of their id
    return value


def filter_chips(request, filterset):                                      # 16
    labels = {name: (field.label or name) for name, field in
filterset.form.fields.items()}
    chips = []
    for key in request.GET:
        values = [v for v in request.GET.getlist(key) if v]
        if key in SKIP_PARAMS or not values:
            continue
        params = request.GET.copy()
        params.pop(key)
        params.pop("page", None)
        label = labels.get(key, key.replace("_", " ")).capitalize()
        shown = ", ".join(display_value(filterset, key, v) for v in values)
        chips.append({"label": f"{label}: {shown}", "remove_url": f"?
{params.urlencode()}"})
    return chips
```

2, 10 in the shell.

```
from leads.filters import LeadFilter, MiniLeadFilter
from leads.models import Lead

f = MiniLeadFilter({"status": "new"}, queryset=Lead.objects.all())
f.is_bound, f.is_valid(), f.qs.count()
print(f.form)

bad = LeadFilter({"status": "won", "source": "google"},
queryset=Lead.objects.all())
bad.is_valid(), bad.errors      # (False, {'status': [...]})
bad.qs.count()                  # source still applied; the invalid status is
skipped
LeadFilter(None, queryset=Lead.objects.all()).qs.count()     # unbound → no
filtering
```

B. Wiring (11–12, 15–20, 34)

```
# leads/views.py
from django.db.models import Count, Max
from .filters import LeadFilter, filter_chips, status_facets


class LeadQueryMixin:                                                # 11
(replaces the Day 13 version)
    def base_leads(self):
        return visible_leads(self.request.user).filter(is_active=True)

    def get_filterset(self):
        if not hasattr(self, "_filterset"):
            qs = (self.base_leads()
                  .select_related("assigned_to", "broker")
                  .prefetch_related("tags")
                  .annotate(call_count=Count("calls", distinct=True),
last_call_at=Max("calls__started_at"))
                  .order_by("-created_at", "-pk"))           # default when
there's no ?o=
            self._filterset = LeadFilter(self.request.GET or None,
queryset=qs, request=self.request)
        return self._filterset

    def get_leads(self):
        fs = self.get_filterset()
```

```
        # 34 strict mode: uncomment to show nothing until the filters are
fixed
        # if fs.is_bound and not fs.is_valid():
        #     return fs.queryset.none()
        return fs.qs


@method_decorator(log_request, name="dispatch")
class LeadListView(PageTitleMixin, LeadQueryMixin, ListView):
    session_key = "lead_filters"
    ...

    def get(self, request, *args, **kwargs):                              # 20
        if "reset" in request.GET:
            request.session.pop(self.session_key, None)
            return redirect(request.path)
        params = request.GET.copy()
        params.pop("page", None)
        if params:
            request.session[self.session_key] = params.urlencode()
        elif not request.GET and (saved :=
request.session.get(self.session_key)):
            return redirect(f"{request.path}?{saved}")
        return super().get(request, *args, **kwargs)

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        fs = self.get_filterset()
        counts = status_facets(self.request, fs.queryset)               # 15
        context.update(
            filterset=fs,
            facets=[(value, label, counts.get(value, 0)) for value, label in
Lead.Status.choices],
            chips=filter_chips(self.request, fs),                         #
16
            saved_views=self.request.user.saved_views.all(),              #
19
            bulk_form=LeadBulkForm(),
        )
        return context
```

```
class MyLeadsView(LeadListView):
    session_key = "my_lead_filters"
    ...
```

```
# 12 — the same page with django-filter's own view (less code, fewer hooks)
from django_filters.views import FilterView


class SimpleLeadFilterView(FilterView):
    filterset_class = LeadFilter
    template_name = "leads/lead_filter_simple.html"   # context: filter,
object_list, page_obj
    paginate_by = 25
    strict = True                                     # invalid filters →
empty list

    def get_queryset(self):
        return visible_leads(self.request.user).filter(is_active=True)
```

17.

```
# leads/templatetags/crm_filters.py
from django.utils.html import format_html


@register.simple_tag(takes_context=True)
def sort_link(context, param, label):
    request = context["request"]
    current = request.GET.get("o", "")
    if current == param:                # ascending now → next click
descends
        new, arrow = f"-{param}", " ▲"
    elif current == f"-{param}":
        new, arrow = param, " ▼"
    else:
        new, arrow = f"-{param}", ""
    query = request.GET.copy()
    query["o"] = new
    query.pop("page", None)
    return format_html('<a href="?{}">{}{}</a>', query.urlencode(), label,
arrow)
```

19.

```
# leads/models.py
class SavedView(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name="saved_views")
    name = models.CharField(max_length=60)
    querystring = models.CharField(max_length=1000, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["name"]
        constraints = [models.UniqueConstraint(fields=["user", "name"],
name="unique_saved_view_per_user")]

    def __str__(self):
        return self.name

    def get_absolute_url(self):
        return f"{reverse('leads:list')}?{self.querystring}"


# leads/forms.py
class SavedViewForm(forms.ModelForm):
    class Meta:
        model = SavedView
        fields = ["name", "querystring"]
        widgets = {"querystring": forms.HiddenInput}

    def __init__(self, *args, user=None, **kwargs):
        self.user = user
        super().__init__(*args, **kwargs)

    def clean_name(self):
        # user isn't a form field, so the (user, name) constraint isn't
checked by the form — do it here
        name = self.cleaned_data["name"].strip()
        if SavedView.objects.filter(user=self.user,
name__iexact=name).exists():
            raise forms.ValidationError("You already have a view with this
name.")
        return name
```

```
# leads/views.py
class SavedViewCreateView(UserFormKwargsMixin, SuccessMessageMixin,
CreateView):
    form_class = SavedViewForm
    http_method_names = ["post"]
    success_message = "View “%(name)s” saved."

    def form_valid(self, form):
        form.instance.user = self.request.user
        return super().form_valid(form)          # redirects to
SavedView.get_absolute_url()

    def form_invalid(self, form):
        messages.error(self.request, " ".join(form.errors.get("name",
["Couldn't save that view."])))
        return redirect("leads:list")

# leads/urls.py
path("views/save/", views.SavedViewCreateView.as_view(), name="save_view"),
```

Lead list template (15–19, 28).

```
<!-- leads/lead_list.html -->
{% load crm_filters %}
<form method="get" class="filters">
  {% for field in filterset.form %}{{ field.as_field_group }}{% endfor %}
  <label><input type="checkbox" name="tag_mode" value="any"{% if
request.GET.tag_mode == "any" %} checked{% endif %}> Any selected tag</label>
  <button type="submit">Apply</button>
  <a href="?reset=1">Reset</a>
</form>

<p class="chips">                                                {# 15:
facets #}
  {% for value, label, n in facets %}
    <a href="{% querystring status=value page=None %}">{{ label }} ({{ n }})
</a>
  {% endfor %}
</p>

{% if chips %}                                                   {# 16 #}
<p class="chips">{% for chip in chips %}<a href="{{ chip.remove_url }}">{{
```

```
chip.label }} x</a>{% endfor %}</p>
{% endif %}

<p>                                                                    {# 19 #}
  {% for v in saved_views %}<a href="{{ v.get_absolute_url }}">{{ v.name }}
</a> · {% endfor %}
  <form method="post" action="{% url 'leads:save_view' %}" class="inline">{%
csrf_token %}
    <input type="hidden" name="querystring" value="{{ request.GET.urlencode
}}">
    <input name="name" placeholder="Name this view" required>
    <button type="submit">Save view</button>
  </form>
</p>

<table class="table">
  <thead><tr>                                                        {# 17 #}
    <th></th>
    <th>{% sort_link "name" "Name" %}</th><th>Phone</th><th>{% sort_link
"status" "Status" %}</th>
    <th>Source</th><th>Agent</th><th>Tags</th>
    <th>{% sort_link "calls" "Calls" %}</th><th>{% sort_link "last_call"
"Last call" %}</th>
  </tr></thead>
  <tbody>{% include "leads/partials/_lead_rows.html" %}</tbody>
</table>

{% if not page_obj.paginator.count and chips %}                      {# 28 #}
  <p>No leads match. Try removing:
    {% for chip in chips %}<a href="{{ chip.remove_url }}">{{ chip.label }}
</a>{% if not forloop.last %}, {% endif %}{% endfor %}
  </p>
{% endif %}
{% include "partials/_pagination.html" %}
```

`{% querystring status=value page=None %}` sets `status` and removes `page` in one go (15).

C. Search (21–25, 29–30, 32)

25.

```
# leads/queries.py — search_q understands local and international formats
from .models import Lead, normalize_uae_phone


def search_q(term):
    term = term.strip()
    if not term:
        return Q()
    q = Q(name__icontains=term) | Q(email__icontains=term)
    digits = re.sub(r"\D", "", term)
    if len(digits) >= 4:                    # "1003", "050 999 1003", "+971
50 999 1003"
        q |= Q(phone__contains=digits) |
Q(phone__contains=normalize_uae_phone(digits))
    return q
```

21–22, 32.

```
# core/search.py
import re
from difflib import get_close_matches

from django.db.models import Case, IntegerField, Q, Value, When

from brokers.models import Broker
from clients.models import TradingAccount
from leads.models import normalize_uae_phone
from leads.permissions import sees_everything, visible_clients, visible_leads
from leads.queries import search_q


def lead_rank(q):                                                  # 22
    digits = re.sub(r"\D", "", q)
    whens = []
    if len(digits) >= 4:
        whens.append(When(phone=normalize_uae_phone(digits),
then=Value(100)))
    whens += [
        When(name__iexact=q, then=Value(80)),
        When(name__istartswith=q, then=Value(50)),
        When(name__icontains=q, then=Value(10)),
        When(email__icontains=q, then=Value(5)),
    ]
```

```
    return Case(*whens, default=Value(1), output_field=IntegerField())   #
first matching When wins


def search_everything(user, q, limit=5):                                  # 21
    q = q.strip()
    digits = re.sub(r"\D", "", q)
    leads = (visible_leads(user).filter(is_active=True).filter(search_q(q))
             .annotate(rank=lead_rank(q)).order_by("-rank", "-created_at"))
    clients = (visible_clients(user)
               .filter(Q(name__icontains=q) | Q(email__icontains=q)
                       | Q(account_number__startswith=q) |
Q(accounts__number__startswith=q))
               .distinct().order_by("name"))
    accounts =
(TradingAccount.objects.filter(client__in=visible_clients(user),
number__startswith=digits)
                .select_related("client") if len(digits) >= 3 else
TradingAccount.objects.none())
    brokers = (Broker.objects.filter(Q(code__icontains=q) |
Q(name__icontains=q)).order_by("code")
               if sees_everything(user) or user.role == "team_lead" else
Broker.objects.none())
    return {
        name: {"items": list(qs[:limit]), "total": qs.count()}
        for name, qs in [("leads", leads), ("clients", clients), ("accounts",
accounts), ("brokers", brokers)]
    }


def did_you_mean(user, q, n=3):                                           # 32
    names = list(visible_leads(user).order_by("name")     # order_by: Meta
ordering would break distinct()
                 .values_list("name", flat=True).distinct()[:5000])
    return get_close_matches(q.title(), names, n=n, cutoff=0.7)
```

21, 24, 30 — views and templates.

```
# core/views.py
from django.urls import reverse
from .search import did_you_mean, search_everything
```

```
class SearchView(PageTitleMixin, TemplateView):
    template_name = "core/search.html"
    page_title = "Search"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        q = self.request.GET.get("q", "").strip()
        context["q"] = q
        if len(q) >= 2:
            results = search_everything(self.request.user, q)
            context["results"] = results
            if not results["leads"]["total"]:
                context["did_you_mean"] = did_you_mean(self.request.user, q)
        return context


class SuggestView(JsonResponseMixin, View):                                  # 24
    def get(self, request):
        q = request.GET.get("q", "").strip()
        if len(q) < 2:
            return self.render_to_json([])
        r = search_everything(request.user, q, limit=4)
        items = (
            [{"type": "lead", "label": l.name, "url": l.get_absolute_url()}
for l in r["leads"]["items"]]
            + [{"type": "client", "label": f"{c.name} ({c.account_number})",
"url": c.get_absolute_url()}
               for c in r["clients"]["items"]]
            + [{"type": "account", "label": a.number, "url":
a.client.get_absolute_url()} for a in r["accounts"]["items"]]
            + [{"type": "broker", "label": str(b), "url":
reverse("brokers:detail", args=[b.code])}
               for b in r["brokers"]["items"]]
        )
        return self.render_to_json(items[:10])

# core/urls.py
path("search/", views.SearchView.as_view(), name="search"),
path("search/suggest/", views.SuggestView.as_view(), name="search_suggest"),
```

```
<!-- core/templates/core/search.html -->
{% extends "base.html" %}
{% load crm_filters %}
{% block content %}
<h1>Search</h1>
<form method="get"><input name="q" value="{{ q }}" autofocus> <button
type="submit">Search</button></form>

{% if did_you_mean %}
  <p>Did you mean {% for name in did_you_mean %}<a href="?q={{ name|urlencode
}}">{{ name }}</a>{% if not forloop.last %}, {% endif %}{% endfor %}?</p>
{% endif %}

{% if results %}
  <h2>Leads ({{ results.leads.total }})</h2>
  <ul>
    {% for lead in results.leads.items %}
      <li><a href="{{ lead.get_absolute_url }}">{{ lead.name|highlight:q }}
</a> · {{ lead.phone|mask_phone }}</li>
    {% empty %}<li>—</li>{% endfor %}
  </ul>
  {% if results.leads.total > 5 %}<a href="{% url 'leads:list' %}?q={{
q|urlencode }}">All {{ results.leads.total }} leads →</a>{% endif %}

  <h2>Clients ({{ results.clients.total }})</h2>
  <ul>{% for c in results.clients.items %}<li><a href="{{ c.get_absolute_url
}}">{{ c.name|highlight:q }}</a> · {{ c.account_number }}</li>{% empty %}<li>
—</li>{% endfor %}</ul>

  <h2>Trading accounts ({{ results.accounts.total }})</h2>
  <ul>{% for a in results.accounts.items %}<li><a href="{{
a.client.get_absolute_url }}">{{ a.number }}</a> · {{ a.client.name }}</li>{%
empty %}<li>—</li>{% endfor %}</ul>

  <h2>Brokers ({{ results.brokers.total }})</h2>
  <ul>{% for b in results.brokers.items %}<li><a href="{% url
'brokers:detail' b.code %}">{{ b|highlight:q }}</a></li>{% empty %}<li>—</li>
{% endfor %}</ul>
{% endif %}
{% endblock content %}
```

```
<!-- templates/partials/_navbar.html — 30, inside {% if user.is_authenticated
%} -->
<form action="{% url 'search' %}" method="get" class="topbar-search">
  <input name="q" placeholder="Search leads, clients, accounts…" value="{{
request.GET.q }}">
</form>
```

23.

```
# leads/templatetags/crm_filters.py
import re
from django.utils.html import conditional_escape
from django.utils.safestring import mark_safe


@register.filter(needs_autoescape=True)
def highlight(text, query, autoescape=True):
    text = str(text)
    esc = conditional_escape if autoescape else (lambda s: s)
    if not query:
        return esc(text)
    out, last = [], 0
    for m in re.finditer(re.escape(str(query)), text, flags=re.IGNORECASE):
        out.append(esc(text[last:m.start()]))          # escape the pieces...
        out.append(f"<mark>{esc(m.group())}</mark>")   # ...then add our own
tag
        last = m.end()
    out.append(esc(text[last:]))
    return mark_safe("".join(out))                     # safe: every user-
supplied piece was escaped
```

26–27, 29.

```
# clients/filters.py                                                    # 26
import django_filters as df
from django.db.models import Q

from brokers.models import Broker
from .models import Client


class ClientFilter(df.FilterSet):
    q = df.CharFilter(method="filter_q", label="Search")
```

```
    kyc_status = df.ChoiceFilter(choices=Client.KYCStatus.choices,
empty_label="Any KYC")
    account_type = df.ChoiceFilter(choices=Client.AccountType.choices,
empty_label="Any type")
    broker = df.ModelChoiceFilter(queryset=Broker.objects.order_by("code"),
empty_label="Any broker")
    joined = df.DateFromToRangeFilter(field_name="date_joined",
label="Joined")
    min_deposits = df.NumberFilter(field_name="total_deposits",
lookup_expr="gte", label="Deposits ≥")
    o = df.OrderingFilter(fields=(("name", "name"), ("date_joined",
"joined"), ("total_deposits", "deposits")))

    class Meta:
        model = Client
        fields = []

    def filter_q(self, queryset, name, value):
        return queryset.filter(
            Q(name__icontains=value) | Q(account_number__startswith=value) |
Q(accounts__number=value)
        ).distinct()


# clients/views.py
class ClientListView(PageTitleMixin, ListView):
    context_object_name = "clients"
    paginate_by = 25
    page_title = "Clients"

    def get_queryset(self):
        base = (visible_clients(self.request.user).select_related("broker")
                .annotate(total_deposits=deposits_total()).order_by("name"))
        self.filterset = ClientFilter(self.request.GET or None,
queryset=base, request=self.request)
        return self.filterset.qs

    def get_context_data(self, **kwargs):
        return {**super().get_context_data(**kwargs), "filterset":
self.filterset}
```

```
# leads/views.py — 27 (replaces FilteredListView usage)
from .filters import CallLogFilter


class CallLogListView(PageTitleMixin, ListView):
    template_name = "leads/call_list.html"
    context_object_name = "calls"
    paginate_by = 50
    paginate_orphans = 5
    page_title = "Calls"

    def get_queryset(self):
        base =
(CallLog.objects.filter(lead__in=visible_leads(self.request.user))
                .select_related("lead", "agent").order_by("-started_at"))
        self.filterset = CallLogFilter(self.request.GET or None,
queryset=base, request=self.request)
        return self.filterset.qs

    def get_context_data(self, **kwargs):
        return {**super().get_context_data(**kwargs), "filterset":
self.filterset}
```

29. Nothing to add: `LeadCSVView` already inherits `LeadQueryMixin`, so `get_leads()` returns the filterset's queryset. The Export link passes `?{{ request.GET.urlencode }}`, so the file matches the screen.

D. Challenge

31.

```
# leads/migrations/00NN_lead_notes_fts.py
from django.db import migrations

FORWARD = [
    "CREATE VIRTUAL TABLE lead_notes_fts USING fts5(notes,
content='crm_leads', content_rowid='id')",
    "INSERT INTO lead_notes_fts(rowid, notes) SELECT id, notes FROM
crm_leads",
    """CREATE TRIGGER lead_notes_ai AFTER INSERT ON crm_leads BEGIN
         INSERT INTO lead_notes_fts(rowid, notes) VALUES (new.id, new.notes);
       END""",
```

```
    """CREATE TRIGGER lead_notes_ad AFTER DELETE ON crm_leads BEGIN
         INSERT INTO lead_notes_fts(lead_notes_fts, rowid, notes) VALUES
('delete', old.id, old.notes);
       END""",
    """CREATE TRIGGER lead_notes_au AFTER UPDATE OF notes ON crm_leads BEGIN
         INSERT INTO lead_notes_fts(lead_notes_fts, rowid, notes) VALUES
('delete', old.id, old.notes);
         INSERT INTO lead_notes_fts(rowid, notes) VALUES (new.id, new.notes);
       END""",
]
BACKWARD = [
    "DROP TRIGGER IF EXISTS lead_notes_au",
    "DROP TRIGGER IF EXISTS lead_notes_ad",
    "DROP TRIGGER IF EXISTS lead_notes_ai",
    "DROP TABLE IF EXISTS lead_notes_fts",
]


def run(statements):
    def apply(apps, schema_editor):
        if schema_editor.connection.vendor != "sqlite":     # PostgreSQL gets
real full-text search on Day 22
            return
        for sql in statements:
            schema_editor.execute(sql)
    return apply


class Migration(migrations.Migration):
    dependencies = [("leads", "00NN_previous")]          # filled in by
makemigrations --empty
    operations = [migrations.RunPython(run(FORWARD), run(BACKWARD))]
```

```
# leads/queries.py
from django.db import connection


def search_notes(term, user):
    phrase = '"' + term.replace('"', '""') + '"'           # quote it: FTS5
has its own query syntax
    with connection.cursor() as cursor:
        cursor.execute(
            "SELECT rowid FROM lead_notes_fts WHERE lead_notes_fts MATCH %s
ORDER BY rank LIMIT 50", [phrase],
        )
        ids = [row[0] for row in cursor.fetchall()]
    found = visible_leads(user).in_bulk(ids)
    return [found[i] for i in ids if i in found]           # keep FTS
relevance order
```

The triggers also catch `update()` and `bulk_create()`, which skip Python code. Caveat: SQLite rebuilds the table for most later `ALTER`s on `Lead`, which drops these triggers — re-run the forwards SQL after such migrations.

36.

```
print(Lead.objects.filter(name__icontains="khan").explain())     # SCAN
crm_leads — LIKE '%khan%' can't use an index
print(Lead.objects.filter(phone="971501234567").explain())        # SEARCH
crm_leads USING INDEX ... (phone=?)
```

On SQLite every case-insensitive `LIKE` scans; exact matches use the unique index. PostgreSQL trigram indexes fix this on Day 22.

Tests (37–40)

```
# leads/tests.py
from django.test import RequestFactory, SimpleTestCase

from core.search import search_everything
from core.testing import LoggedInTestCase
from .filters import LeadFilter, status_facets
from .templatetags.crm_filters import highlight
```

```
class FilterSearchTests(LoggedInTestCase):          # logged in as a manager
    @classmethod
    def setUpTestData(cls):
        super().setUpTestData()
        User = get_user_model()
        cls.a1 = User.objects.create_user("f_agent1", password="x",
role="agent")
        cls.a2 = User.objects.create_user("f_agent2", password="x",
role="agent")
        Lead.objects.create(name="F One", phone="971509991001",
assigned_to=cls.a1)
        Lead.objects.create(name="F Two", phone="971509991002",
status="lost", notes="n/a", assigned_to=cls.a1)
        Lead.objects.create(name="F Three", phone="971509991003",
assigned_to=cls.a2)

    def base(self):
        return Lead.objects.filter(name__startswith="F ")

    def test_filter_combinations(self):                                 # 37
        def count(**params):
            return LeadFilter(params, queryset=self.base(),
request=None).qs.count()
        self.assertEqual(count(status="new"), 2)
        self.assertEqual(count(agent=self.a1.pk), 2)
        self.assertEqual(count(status="new", agent=self.a1.pk), 1)

    def test_facets_follow_other_filters(self):                         # 38
        request = RequestFactory().get("/leads/", {"agent": self.a1.pk,
"status": "new"})
        request.user = self.staff
        self.assertEqual(status_facets(request, self.base()), {"new": 1,
"lost": 1})

    def test_ranking(self):                                             # 39
        for name, phone in [("Ray Zebulon", "971509991011"), ("Zebulon Ray",
"971509991012"), ("Zebulon", "971509991013")]:
            Lead.objects.create(name=name, phone=phone)
        names = [lead.name for lead in search_everything(self.staff,
"zebulon")["leads"]["items"]]
        self.assertEqual(names, ["Zebulon", "Zebulon Ray", "Ray Zebulon"])
```

```
class HighlightTests(SimpleTestCase):
    def test_escapes_and_marks(self):                                    # 40
        self.assertEqual(highlight("<b>Ali</b> Khan", "ali"), "&lt;b&gt;
<mark>Ali</mark>&lt;/b&gt; Khan")
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py test
git add -A && git commit -m "day 17"
```

Day 18 — Testing

Goal: turn the tests written along the way into a real suite — organised by layer, built with factories instead of hand-made rows, fast, measured with coverage, and run on every push.

A. Drills (1–10) — layout, running, factories

- [ ] **1.** Split `leads/tests.py` into a package: `test_models.py`, `test_queries.py`, `test_forms.py`, `test_views.py`, `test_permissions.py`, `test_admin.py`.
- [ ] **2.** Run one module, one class, one test, `-k` by name, `--parallel`, `--keepdb`, `--failfast`, `-v 2`.
- [ ] **3.** `@tag("slow")` a test; run with `--exclude-tag slow`.
- [ ] **4.** Install factory_boy; `UserFactory` (sequence username, Faker name, `factory.django.Password`), plus `AgentFactory`, `TeamLeadFactory`, `ManagerFactory`.
- [ ] **5.** `LeadFactory`: sequence phone, Faker name, `SubFactory(AgentFactory)`.
- [ ] **6.** `CallLogFactory`, `BrokerFactory`, `ClientFactory`, `TradingAccountFactory`, `DepositFactory`.
- [ ] **7.** Traits on `LeadFactory`: `qualified`, `lost`, `unassigned`.
- [ ] **8.** `@factory.post_generation` tags: `LeadFactory(tags=[vip])`.
- [ ] **9.** `build()` vs `create()` vs `create_batch()`; how many queries each runs.
- [ ] **10.** Fixtures: `dumpdata` brokers to JSON, load them with `fixtures = [...]`; compare with factories.

B. Core (11–20) — testing tools

- [ ] **11.** `@override_settings(CRM_API_KEY="test-key")` on the API-key tests.
- [ ] **12.** Freeze time with `mock.patch("django.utils.timezone.now")` to test stale-lead logic without `update()` tricks.
- [ ] **13.** Signals: a login creates a `LoginEvent`; a disconnected receiver doesn't.
- [ ] **14.** Email: invite lands in `mail.outbox` with subject, recipient, link.
- [ ] **15.** Messages: read them with `get_messages(response.wsgi_request)`.
- [ ] **16.** Permission matrix: roles × URLs → expected status codes, one test with `subTest`.

- [ ] **17.** `assertNumQueries` on the lead list built from factory data.
- [ ] **18.** JSON endpoints: status, content type, `response.json()` shape.
- [ ] **19.** Admin action test built from factories.
- [ ] **20.** `assertInHTML` for an exact rendered snippet (the status badge).

C. Nexora CRM (21–30) — the suite

- [ ] **21.** Model tests: phone/name normalisation, `clean()`, `is_hot`, `mark_contacted`, `archive`.
- [ ] **22.** Query tests: `search_q`, finders, SOP breaches — with frozen time.
- [ ] **23.** Service tests: `log_call` (counters + new→contacted), `convert_lead_to_client`, `upsert_leads`.
- [ ] **24.** Table-driven form tests: a list of phone inputs → valid/invalid with `subTest`.
- [ ] **25.** View flow tests as an agent: create → edit → status → archive → restore.
- [ ] **26.** Scoping tests for agent / team lead / manager with factories.
- [ ] **27.** Report tests: leaderboard numbers from a known factory dataset.
- [ ] **28.** Coverage: run, report, HTML, `.coveragerc` (omit migrations, branch coverage).
- [ ] **29.** GitHub Actions workflow: install, `makemigrations --check`, run tests on every push.
- [ ] **30.** `settings_test.py`: fast password hasher, in-memory email; run with `--settings`.

D. Challenge (31–40)

- [ ] **31.** `TestCase` vs `TransactionTestCase`: watch an `on_commit` callback fire for real.
- [ ] **32.** `RequestFactory` for a view that needs session + messages (attach them by hand).
- [ ] **33.** Hypothesis: property test that `normalize_uae_phone` is idempotent — and fix the bug it finds.
- [ ] **34.** CSV export test: header row and one data row.
- [ ] **35.** Custom assertion mixin `assertLeadVisible(user, lead)` / `assertLeadHidden`.
- [ ] **36.** Migration round trip: migrate `leads` to zero and back in a `TransactionTestCase`.
- [ ] **37.** Same tests in pytest-django: `pytest.ini`, `conftest.py` fixtures, `parametrize`.
- [ ] **38.** Red → green: break `visible_leads()` on purpose, watch the suite catch it, revert.
- [ ] **39.** Anonymous calls to `/leads/api/page/` get JSON 401.

- [ ] **40.** End-to-end: create lead → log call → convert → deposit → total shows in the broker report.

Solutions

A. Layout and running (1–3)

```
leads/
└── tests/                    # delete leads/tests.py — a module and a
package can't share the name
    ├── __init__.py
    ├── test_models.py
    ├── test_queries.py
    ├── test_forms.py
    ├── test_views.py
    ├── test_permissions.py
    └── test_admin.py
```

```
python manage.py test leads.tests.test_models
# 2: one module
python manage.py test leads.tests.test_models.LeadModelTests
# one class
python manage.py test
leads.tests.test_models.LeadModelTests.test_save_normalises   # one test
python manage.py test -k phone                # every test whose name
contains "phone"
python manage.py test --parallel              # one process per CPU core
python manage.py test --keepdb                # reuse the test database
between runs
python manage.py test --failfast -v 2         # stop at first failure, list
each test
python manage.py test --exclude-tag slow      # 3
```

```
from django.test import tag


@tag("slow")                                                    # 3
class MigrationRoundTripTests(TransactionTestCase):
    ...
```

Factories (4–8)

```
python -m pip install factory_boy
```

```
# core/factories.py
from decimal import Decimal

import factory
from factory.django import DjangoModelFactory, Password


class UserFactory(DjangoModelFactory):                                  # 4
    class Meta:
        model = "accounts.CustomUser"          # "app.Model" strings: no
model import at module load
        django_get_or_create = ("username",)

    username = factory.Sequence(lambda n: f"user{n}")
    email = factory.LazyAttribute(lambda o: f"{o.username}@nexora.test")
    first_name = factory.Faker("first_name")
    last_name = factory.Faker("last_name")
    role = "agent"
    password = Password("Test-pass-2026")   # stored hashed; log in with the
plain text


class AgentFactory(UserFactory):
    username = factory.Sequence(lambda n: f"agent_f{n}")
    role = "agent"


class TeamLeadFactory(UserFactory):
    username = factory.Sequence(lambda n: f"tl_f{n}")
    role = "team_lead"


class ManagerFactory(UserFactory):
    username = factory.Sequence(lambda n: f"mgr_f{n}")
    role = "manager"


class BrokerFactory(DjangoModelFactory):                                # 6
    class Meta:
```

```
        model = "brokers.Broker"
        django_get_or_create = ("code",)

    code = factory.Sequence(lambda n: f"BRF{8000 + n:04d}")
    name = factory.Faker("company")
    tier = 1


class TagFactory(DjangoModelFactory):
    class Meta:
        model = "leads.Tag"
        django_get_or_create = ("name",)

    name = factory.Sequence(lambda n: f"tag-{n}")


class LeadFactory(DjangoModelFactory):                                  # 5, 7–
8
    class Meta:
        model = "leads.Lead"
        skip_postgeneration_save = True        # the tags hook doesn't need a
second save()

    name = factory.Faker("name")
    phone = factory.Sequence(lambda n: f"97158{n:07d}")   # 058 range: never
clashes with the seeds
    source = "google"
    status = "new"
    assigned_to = factory.SubFactory(AgentFactory)
    expected_deposit = Decimal("5000.00")

    class Params:                                                        # 7
        qualified = factory.Trait(status="qualified")
        lost = factory.Trait(status="lost", notes="Not interested")
        unassigned = factory.Trait(assigned_to=None)

    @factory.post_generation                                             # 8
    def tags(self, create, extracted, **kwargs):
        if create and extracted:
            self.tags.add(*extracted)


class CallLogFactory(DjangoModelFactory):
```

```
    class Meta:
        model = "leads.CallLog"

    lead = factory.SubFactory(LeadFactory)
    agent = factory.LazyAttribute(lambda o: o.lead.assigned_to)
    outcome = "answered"
    duration_seconds = 120


class ClientFactory(DjangoModelFactory):
    class Meta:
        model = "clients.Client"

    name = factory.Faker("name")
    email = factory.Sequence(lambda n: f"client{n}@nexora.test")
    account_number = factory.Sequence(lambda n: f"{3_000_000 + n:07d}")


class TradingAccountFactory(DjangoModelFactory):
    class Meta:
        model = "clients.TradingAccount"

    client = factory.SubFactory(ClientFactory)
    number = factory.Sequence(lambda n: f"{4_000_000 + n:07d}")


class DepositFactory(DjangoModelFactory):
    class Meta:
        model = "clients.Deposit"

    account = factory.SubFactory(TradingAccountFactory)
    amount = Decimal("1000.00")
    method = "bank"


# usage
LeadFactory()                                  # saved lead + saved agent
LeadFactory(qualified=True, assigned_to=agent)  # trait + override
LeadFactory(tags=[TagFactory(name="vip")])
LeadFactory.create_batch(30, source="referral")
```

9–10

```
# leads/tests/test_models.py
class FactoryStrategyTests(TestCase):
    def test_build_hits_no_database(self):                          # 9
        with self.assertNumQueries(0):
            lead = LeadFactory.build()     # SubFactory builds the agent too
— nothing saved
        self.assertIsNone(lead.pk)

    def test_create_batch(self):
        LeadFactory.create_batch(10)

self.assertEqual(Lead.objects.filter(phone__startswith="97158").count(), 10)
```

```
mkdir -p brokers/fixtures
python manage.py dumpdata brokers.broker --indent 2 -o
brokers/fixtures/brokers.json    # 10
```

```
# brokers/tests.py — 10
class BrokerFixtureTests(TestCase):
    fixtures = ["brokers.json"]            # loaded before every test in the
class

    def test_tree_loaded(self):
        self.assertTrue(Broker.objects.filter(parent__isnull=True).exists())
```

Fixtures freeze pks and break when the schema changes; factories describe only what each test needs. Prefer factories; keep fixtures for fixed reference data.

B. Tools (11–20)

```
# leads/tests/test_views.py
import csv
import io
from datetime import datetime, timedelta
from unittest import mock

from django.contrib.auth.signals import user_logged_in
from django.contrib.messages import get_messages
from django.core import mail
from django.db import connection
```

```
from django.test import TestCase, override_settings
from django.test.utils import CaptureQueriesContext
from django.urls import reverse
from django.utils import timezone

from accounts.models import LoginEvent
from accounts.services import invite_staff_user
from accounts.signals import record_login
from core.factories import (AgentFactory, LeadFactory, ManagerFactory,
TagFactory,
                            TeamLeadFactory, UserFactory)
from leads.models import Lead
from leads.queries import stale_new_leads

PASSWORD = "Test-pass-2026"
T0 = timezone.make_aware(datetime(2026, 10, 1, 9, 0))


@override_settings(CRM_API_KEY="test-key")                                  # 11
class ApiKeyTests(TestCase):
    def test_wrong_key(self):
        self.assertEqual(self.client.get("/api/v1/leads/", headers={"X-API-
Key": "nope"}).status_code, 401)

    def test_right_key(self):
        self.assertEqual(self.client.get("/api/v1/leads/", headers={"X-API-
Key": "test-key"}).status_code, 200)


class FrozenTimeTests(TestCase):                                             # 12
    def test_stale_after_seven_days(self):
        with mock.patch("django.utils.timezone.now", return_value=T0):
            lead = LeadFactory()                            # created_at = T0
(auto_now_add calls timezone.now)
        with mock.patch("django.utils.timezone.now", return_value=T0 +
timedelta(days=8)):
            self.assertIn(lead, stale_new_leads(days=7))
        with mock.patch("django.utils.timezone.now", return_value=T0 +
timedelta(days=6)):
            self.assertNotIn(lead, stale_new_leads(days=7))


class SignalEmailMessageTests(TestCase):
```

```
    def test_login_records_event(self):                                    # 13
        user = AgentFactory()
        self.client.login(username=user.username, password=PASSWORD)
        self.assertTrue(LoginEvent.objects.filter(user=user,
success=True).exists())

    def test_disconnected_receiver(self):
        user = AgentFactory()
        user_logged_in.disconnect(record_login)
        try:
            self.client.login(username=user.username, password=PASSWORD)
        finally:
            user_logged_in.connect(record_login)      # always reconnect
        self.assertFalse(LoginEvent.objects.filter(user=user).exists())

    def test_invite_email(self):                                           # 14
        invite_staff_user("mail_agent", "mail@nexora.test", role="agent")
        message = mail.outbox[0]
        self.assertEqual(message.subject, "Your Nexora CRM account")
        self.assertEqual(message.to, ["mail@nexora.test"])
        self.assertIn("/accounts/reset/", message.body)

    def test_archive_message(self):                                        # 15
        agent = AgentFactory()
        lead = LeadFactory(assigned_to=agent)
        self.client.force_login(agent)
        response = self.client.post(reverse("leads:archive", args=[lead.pk]))
        texts = [str(m) for m in get_messages(response.wsgi_request)]
        self.assertTrue(any("archived" in t for t in texts))


class PermissionMatrixTests(TestCase):                                     # 16
    CASES = [
        ("agent", "leads:list", 200),
        ("agent", "reports:index", 403),
        ("team_lead", "reports:index", 200),
        ("manager", "reports:index", 200),
        ("agent", "accounts:team", 403),
        ("team_lead", "accounts:team", 403),
        ("manager", "accounts:team", 200),
        ("agent", "leads:export", 403),
        ("team_lead", "leads:export", 200),
    ]
```

```
    def test_matrix(self):
        users = {role: UserFactory(role=role) for role in ("agent",
"team_lead", "manager")}
        for role, url_name, expected in self.CASES:
            with self.subTest(role=role, url=url_name):
                self.client.force_login(users[role])

self.assertEqual(self.client.get(reverse(url_name)).status_code, expected)


class ListQueryAndJsonTests(TestCase):
    def setUp(self):
        self.client.force_login(ManagerFactory())

    def _queries(self):
        with CaptureQueriesContext(connection) as ctx:
            self.client.get(reverse("leads:list"))
        return len(ctx.captured_queries)

    def test_query_count_is_constant(self):                                 # 17
        LeadFactory.create_batch(5)
        baseline = self._queries()
        LeadFactory.create_batch(30, tags=[TagFactory()])
        with self.assertNumQueries(baseline):
            self.client.get(reverse("leads:list"))

    def test_page_api_shape(self):                                          # 18
        LeadFactory.create_batch(3)
        response = self.client.get(reverse("leads:api_page"))
        self.assertEqual(response.status_code, 200)
        self.assertEqual(response["Content-Type"], "application/json")
        self.assertEqual(set(response.json()), {"page", "num_pages", "count",
"has_next", "results"})

    def test_badge_html(self):                                              # 20
        lead = LeadFactory()
        response = self.client.get(lead.get_absolute_url())
        self.assertInHTML('<span class="badge badge-new">New</span>',
response.content.decode())


# leads/tests/test_admin.py                                                 # 19
```

```
class AdminActionTests(TestCase):
    def test_mark_contacted(self):
        boss = UserFactory(role="admin", is_staff=True, is_superuser=True)
        self.client.force_login(boss)
        leads = LeadFactory.create_batch(3)
        self.client.post(reverse("admin:leads_lead_changelist"), {
            "action": "mark_contacted", "_selected_action": [l.pk for l in
leads], "index": 0,
        })
        self.assertEqual(Lead.objects.filter(pk__in=[l.pk for l in leads],
status="contacted").count(), 3)
```

`mock.patch("django.utils.timezone.now")` works because the code calls `timezone.now()` through the module. Code that did `from django.utils.timezone import now` keeps the real function (12).

C. The suite (21–27)

```
# leads/tests/test_models.py — 21
from django.core.exceptions import ValidationError


class LeadModelTests(TestCase):
    def test_save_normalises(self):
        lead = LeadFactory(name="zara ahmed", phone="+971 50 777 0001")
        self.assertEqual((lead.name, lead.phone), ("Zara Ahmed",
"971507770001"))

    def test_lost_needs_notes(self):
        with self.assertRaises(ValidationError):
            LeadFactory.build(status="lost", notes="").full_clean(exclude=
["assigned_to"])

    def test_is_hot(self):
        self.assertTrue(LeadFactory.build(status="new",
source="facebook").is_hot)
        self.assertFalse(LeadFactory.build(status="new",
source="referral").is_hot)

    def test_mark_contacted(self):
        lead = LeadFactory()
        lead.mark_contacted()
```

```
        lead.refresh_from_db()
        self.assertEqual((lead.status, lead.contact_attempts), ("contacted",
1))
        self.assertIsNotNone(lead.last_contacted_at)

    def test_archive(self):
        lead = LeadFactory()
        lead.archive()
        lead.refresh_from_db()
        self.assertFalse(lead.is_active)
```

```
# leads/tests/test_queries.py — 22
from core.factories import CallLogFactory
from leads.queries import follow_up_due, search_leads
from reports.queries import sop_first_call_breaches


class QueryTests(TestCase):
    def test_search_formats(self):
        lead = LeadFactory(phone="971507770002")
        for term in ["0507770002", "+971 50 777 0002", "0002"]:
            with self.subTest(term=term):
                self.assertIn(lead, search_leads(term))

    def test_follow_up_due(self):
        lead = LeadFactory(status="contacted", contact_attempts=1,
last_contacted_at=T0)
        with mock.patch("django.utils.timezone.now", return_value=T0 +
timedelta(hours=50)):
            self.assertIn(lead, follow_up_due(hours=48))

    def test_sop_breach(self):
        with mock.patch("django.utils.timezone.now", return_value=T0):
            late, ok = LeadFactory(), LeadFactory()
        CallLogFactory(lead=ok, started_at=T0 + timedelta(hours=2))
        CallLogFactory(lead=late, started_at=T0 + timedelta(hours=30))
        with mock.patch("django.utils.timezone.now", return_value=T0 +
timedelta(days=3)):
            breaches = sop_first_call_breaches(hours=24)
            self.assertIn(late, breaches)
            self.assertNotIn(ok, breaches)
```

```
# leads/tests/test_services.py — 23
from clients.services import convert_lead_to_client
from core.factories import BrokerFactory
from leads.services import log_call


class ServiceTests(TestCase):
    def test_log_call_moves_new_to_contacted(self):
        lead = LeadFactory()
        log_call(lead, lead.assigned_to, "answered", 120)
        lead.refresh_from_db()
        self.assertEqual((lead.status, lead.contact_attempts), ("contacted",
1))

    def test_log_call_keeps_later_statuses(self):
        lead = LeadFactory(qualified=True)
        log_call(lead, lead.assigned_to, "answered", 60)
        lead.refresh_from_db()
        self.assertEqual(lead.status, "qualified")

    def test_convert_once(self):
        lead = LeadFactory(broker=BrokerFactory())
        client = convert_lead_to_client(lead.pk)
        self.assertEqual(client.broker, lead.broker)
        self.assertEqual(client.accounts.count(), 1)
        with self.assertRaises(ValueError):
            convert_lead_to_client(lead.pk)          # second time: already a
client
```

```
# leads/tests/test_forms.py — 24
from leads.forms import LeadForm

PHONE_CASES = [
    ("+971 50 777 8899", True), ("0507778899", True), ("00971507778899",
True), ("971507778899", True),
    ("12345", False), ("+44 7700 900123", False), ("", False),
]


class PhoneFieldTests(TestCase):
    def test_phone_inputs(self):
        for raw, ok in PHONE_CASES:
            with self.subTest(raw=raw):
                form = LeadForm({"name": "Case", "phone": raw, "source":
"manual"})
                self.assertEqual(form.is_valid(), ok, form.errors)
```

```
# leads/tests/test_views.py — 25
class AgentFlowTests(TestCase):
    def test_create_edit_status_archive_restore(self):
        agent = AgentFactory()
        self.client.force_login(agent)

        response = self.client.post(reverse("leads:new"), {"name": "flow
test", "phone": "0509994001", "source": "google"})
        lead = Lead.objects.get(phone="971509994001")
        self.assertRedirects(response, lead.get_absolute_url())
        self.assertEqual(lead.assigned_to, agent)               # agents own
what they create

        self.client.post(reverse("leads:edit", args=[lead.pk]), {
            "name": "Flow Test", "phone": lead.phone, "source": "referral",
            "loaded_at": lead.updated_at.isoformat(),
        })
        lead.refresh_from_db()
        self.assertEqual(lead.source, "referral")

        self.client.post(reverse("leads:status_edit", args=[lead.pk]),
{"status": "contacted", "notes": ""})
        self.client.post(reverse("leads:archive", args=[lead.pk]))
        lead.refresh_from_db()
```

```
        self.assertEqual((lead.status, lead.is_active), ("contacted", False))

        self.client.post(reverse("leads:restore", args=[lead.pk]))
        lead.refresh_from_db()
        self.assertTrue(lead.is_active)
```

```
# core/testing.py — 35 (used by 26)
from leads.permissions import visible_leads


class LeadVisibilityAssertions:
    def assertLeadVisible(self, user, lead):
        self.assertTrue(visible_leads(user).filter(pk=lead.pk).exists(), f"
{user} should see {lead}")

    def assertLeadHidden(self, user, lead):
        self.assertFalse(visible_leads(user).filter(pk=lead.pk).exists(), f"
{user} should not see {lead}")


# leads/tests/test_permissions.py — 26
class ScopeTests(LeadVisibilityAssertions, TestCase):
    def test_scopes(self):
        tl = TeamLeadFactory()
        a1, a2 = AgentFactory(team_lead=tl), AgentFactory()
        l1, l2 = LeadFactory(assigned_to=a1), LeadFactory(assigned_to=a2)
        manager = ManagerFactory()
        self.assertLeadVisible(a1, l1)
        self.assertLeadHidden(a1, l2)
        self.assertLeadVisible(tl, l1)
        self.assertLeadHidden(tl, l2)
        self.assertLeadVisible(manager, l2)
```

```
# reports/tests.py — 27
from core.factories import AgentFactory, CallLogFactory, LeadFactory
from .queries import agent_leaderboard


class LeaderboardFactoryTests(TestCase):
    def test_numbers(self):
        agent = AgentFactory()
        LeadFactory.create_batch(3, assigned_to=agent)
        LeadFactory(assigned_to=agent, qualified=True)
        called = LeadFactory(assigned_to=agent)
        CallLogFactory.create_batch(4, lead=called, agent=agent)
        row = agent_leaderboard().get(pk=agent.pk)
        self.assertEqual((row.leads_n, row.qualified_n, row.calls_n), (5, 1,
4))
        self.assertAlmostEqual(row.conversion, 20.0)
```

C. Coverage, CI, test settings (28–30)

```
python -m pip install coverage                                          # 28
coverage run manage.py test
coverage report                 # per-file % with missing lines
coverage html                   # open htmlcov/index.html, red = never ran
```

```
# .coveragerc
[run]
source = .
branch = True
omit =
    */migrations/*
    */tests/*
    manage.py
    django_project/asgi.py
    django_project/wsgi.py
    .venv/*

[report]
show_missing = True
skip_covered = True
```

```
# .github/workflows/tests.yml                                          # 29
name: tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements.txt
      - run: python manage.py makemigrations --check --dry-run
      - run: python manage.py test --parallel --
settings=django_project.settings_test
```

```
# django_project/settings_test.py                                       # 30
from .settings import *  # noqa: F401,F403

PASSWORD_HASHERS = ["django.contrib.auth.hashers.MD5PasswordHasher"]   #
fast; never in production
EMAIL_BACKEND = "django.core.mail.backends.locmem.EmailBackend"
```

```
python -m pip freeze > requirements.txt      # CI installs exactly what you
run locally
python manage.py test --settings=django_project.settings_test
```

D. Challenge

31.

```
# clients/tests.py
from django.test import TransactionTestCase


class OnCommitTests(TransactionTestCase):
    def test_callback_really_runs(self):
        lead = LeadFactory()
        with mock.patch("builtins.print") as fake_print:
            convert_lead_to_client(lead.pk)          # real COMMIT here → on_commit fires
        printed = [str(c.args[0]) for c in fake_print.call_args_list if c.args]
        self.assertTrue(any("Welcome email" in p for p in printed))
```

`TestCase` wraps every test in a transaction that's rolled back, so `on_commit` never fires unless you use `captureOnCommitCallbacks`. `TransactionTestCase` commits for real and empties the tables afterwards — slower, and Django runs these after all `TestCase` classes.

32.

```
# core/testing.py
from django.contrib.messages.storage.fallback import FallbackStorage
from django.contrib.sessions.middleware import SessionMiddleware
from django.test import RequestFactory


def build_request(path, user, method="get", data=None):
    request = getattr(RequestFactory(), method)(path, data or {})
    SessionMiddleware(lambda r: None).process_request(request)   # RequestFactory skips middleware
    request.session.save()
    request._messages = FallbackStorage(request)
    request.user = user
    return request

# usage
request = build_request("/leads/", ManagerFactory())
response = LeadListView.as_view()(request)
```

33.

```
python -m pip install hypothesis
```

```
# leads/tests/test_phone_properties.py
from django.test import SimpleTestCase
from hypothesis import given, strategies as st

from leads.models import normalize_uae_phone


class PhoneProperties(SimpleTestCase):
    @given(st.text(alphabet="0123456789 +-", max_size=20))
    def test_normalize_is_idempotent(self, raw):
        once = normalize_uae_phone(raw)
        self.assertEqual(normalize_uae_phone(once), once)

    @given(st.from_regex(r"\A9715\d{8}\Z", fullmatch=True))
    def test_clean_numbers_unchanged(self, phone):
        self.assertEqual(normalize_uae_phone(phone), phone)
```

The first property fails with an input like `raw='0000'`: one `"00"` strip leaves `"00"`, and the second call strips again. The fix also handles Arabic-Indic digits, which `str.isdigit()` accepts and agents with Arabic keyboards will type:

```
# leads/models.py
ARABIC_DIGITS = str.maketrans("٠١٢٣٤٥٦٧٨٩", "0123456789")


def normalize_uae_phone(raw):
    digits = "".join(ch for ch in (raw or "").translate(ARABIC_DIGITS) if ch
in "0123456789")
    while digits.startswith("00"):          # was: a single if
        digits = digits[2:]
    if digits.startswith("05"):
        digits = "971" + digits[1:]
    return digits
```

34.

```
class CsvExportTests(TestCase):
    def test_header_and_row(self):
        self.client.force_login(ManagerFactory())
        LeadFactory(name="Csv Person")
        response = self.client.get(reverse("leads:export"), {"q": "csv
person"})
        rows = list(csv.reader(io.StringIO(response.content.decode())))
        self.assertEqual(rows[0], ["id", "name", "phone", "status",
"source"])
        self.assertEqual(rows[1][1], "Csv Person")
        self.assertEqual(len(rows), 2)
```

36.

```
from django.core.management import call_command
from django.test import TransactionTestCase, tag


@tag("slow")
class MigrationRoundTripTests(TransactionTestCase):
    def test_leads_down_and_up(self):
        call_command("migrate", "leads", "zero", verbosity=0)   # also
unapplies migrations that depend on leads
        call_command("migrate", verbosity=0)
        self.assertTrue(Lead.objects.filter(phone="971501234567").exists())
# the Day 5 seed ran again
```

37.

```
python -m pip install pytest pytest-django
```

```
# pytest.ini
[pytest]
DJANGO_SETTINGS_MODULE = django_project.settings_test
python_files = tests.py test_*.py
```

```
# conftest.py (project root)
import pytest
from core.factories import AgentFactory


@pytest.fixture
```

```
def agent(db):                    # `db` = this test may use the database
    return AgentFactory()


@pytest.fixture
def agent_client(client, agent):  # `client` comes from pytest-django
    client.force_login(agent)
    return client


# leads/tests/test_pytest_style.py
import pytest
from django.urls import reverse
from core.factories import LeadFactory
from leads.models import normalize_uae_phone


@pytest.mark.parametrize("raw,expected", [
    ("+971 50 123 4567", "971501234567"),
    ("0501234567", "971501234567"),
    ("00971501234567", "971501234567"),
])
def test_normalize(raw, expected):
    assert normalize_uae_phone(raw) == expected


def test_agent_sees_own_lead(agent_client, agent):
    lead = LeadFactory(assigned_to=agent)
    response = agent_client.get(reverse("leads:list"))
    assert lead in response.context["leads"]
```

```
pytest -q                         # runs both pytest functions and Django
TestCase classes
```

38. In `visible_leads()`, change the last line to `return qs` and run the suite: `test_agent_list_is_scoped`, the 404 sweep and `ScopeTests` fail. Revert and they pass. If nothing failed, the rule had no test.

39–40.

```
# leads/tests/test_views.py
from decimal import Decimal
from clients.models import Client
from core.factories import BrokerFactory
from reports.queries import broker_report


class ApiAuthTests(TestCase):
    def test_anonymous_api_gets_json_401(self):                          # 39
        response = self.client.get(reverse("leads:api_page"))
        self.assertEqual(response.status_code, 401)
        self.assertEqual(response.json()["error"], "authentication required")


class LifecycleTests(TestCase):                                           # 40
    def test_lead_to_deposit_appears_in_broker_report(self):
        broker = BrokerFactory()
        agent = AgentFactory()
        self.client.force_login(agent)
        self.client.post(reverse("leads:new"), {"name": "life cycle",
"phone": "0509993001", "source": "referral"})
        lead = Lead.objects.get(phone="971509993001")
        Lead.objects.filter(pk=lead.pk).update(broker=broker)

        self.client.post(reverse("leads:log_call", args=[lead.pk]),
{"outcome": "answered", "duration_seconds": 300})
        lead.refresh_from_db()
        self.assertEqual(lead.status, "contacted")

        self.client.force_login(ManagerFactory())
        self.client.post(reverse("leads:convert", args=[lead.pk]),
{"platform": "mt5", "currency": "USD", "leverage": 100})
        crm_client = Client.objects.get(lead=lead)
        account = crm_client.accounts.get()
        self.client.post(reverse("clients:deposit_new", args=
[account.number]), {"amount": "2500", "method": "bank"})

        self.assertEqual(broker_report().get(pk=broker.pk).deposits_total,
Decimal("2500.00"))
```

```
coverage run manage.py test --settings=django_project.settings_test &&
coverage report
git add -A && git commit -m "day 18"
```

Day 19 — Files: uploads, CSV import/export

Goal: files in and out of the CRM. KYC documents are uploaded, checked and stored *outside* the public media folder, and only reachable through a permission-checked view. Leads arrive from CSV/Excel with a preview step; data leaves as streaming CSV or formatted Excel.

Install once: `python -m pip install Pillow openpyxl`.

A. Drills (1–10) — upload basics

- [ ] **1.** `MEDIA_ROOT` / `MEDIA_URL`; serve media in development with `static()`; ignore `media/` in git.
- [ ] **2.** Throwaway upload page: `forms.FileField`, `enctype="multipart/form-data"`, print `name`, `size`, `content_type`.
- [ ] **3.** Save an upload by hand with `.chunks()` and a safe filename (`get_valid_filename`).
- [ ] **4.** `FileField(upload_to="kyc/%Y/%m/")` — date folders.
- [ ] **5.** `upload_to` callable: `kyc/<client ref>/<doc type>/<uuid>.<ext>`.
- [ ] **6.** `FileExtensionValidator` + a deconstructible `MaxSizeValidator(5)` (MB).
- [ ] **7.** Sniff the real type from the first bytes (`%PDF`, JPEG, PNG) — never trust the browser's `content_type`.
- [ ] **8.** `ImageField` avatar on `CustomUser` (needs Pillow).
- [ ] **9.** Delete the stored file when its row is deleted (`post_delete` + `on_commit`).
- [ ] **10.** Storage API in the shell: `save`, `exists`, `size`, `open`, `delete`, `ContentFile`.

B. Core (11–20) — KYC documents

- [ ] **11.** `PrivateStorage` (outside `MEDIA_ROOT`) registered in `STORAGES`; `KYCDocument` model using it.
- [ ] **12.** `KYCUploadView` under a client: fills uploader, original name, size, sniffed type, next version number.
- [ ] **13.** `KYCDownloadView`: permission-scoped, `FileResponse`, attachment by default.
- [ ] **14.** `?inline=1` shows PDFs/images in the browser; add `nosniff` and `no-store` headers.

- [ ] **15.** Review: approve/reject (POST, managers); the client's `kyc_status` updates when every required document is approved.
- [ ] **16.** Checklist on the client page: each required document and its latest status, missing ones flagged.
- [ ] **17.** ID expiry: documents expiring in 30 days; expired ones count as missing.
- [ ] **18.** Re-uploads become new versions; the history stays visible.
- [ ] **19.** Multi-file upload (bank statements) with the `MultipleFileField` pattern from Django's docs.
- [ ] **20.** Avatar resized to 256 px JPEG with Pillow before saving.

C. Nexora CRM (21–30) — import and export

- [ ] **21.** `LeadImportForm`: file (.csv/.xlsx), default source, optional agent to assign new leads to.
- [ ] **22.** `read_rows()`: CSV with BOM handling, header aliases (`Mobile` → `phone`), missing-column error.
- [ ] **23.** `validate_rows()`: each row through a form; collect errors with row numbers.
- [ ] **24.** `ImportBatch` model + preview page: totals, valid rows, first 50 errors, Confirm button.
- [ ] **25.** Confirm: re-read the stored file, upsert valid rows, record created/updated counts.
- [ ] **26.** Import history page (your batches, newest first).
- [ ] **27.** `.xlsx` reading with openpyxl through the same pipeline (watch out for numeric phone cells).
- [ ] **28.** Streaming CSV export with `StreamingHttpResponse` + `iterator()`.
- [ ] **29.** Excel-friendly CSV: UTF-8 BOM so Arabic names open correctly; local times instead of UTC.
- [ ] **30.** `.xlsx` export: bold header, frozen first row, column widths, number format on deposits.

D. Challenge (31–40)

- [ ] **31.** Chunked import: 500 rows per transaction with a progress counter on the batch.
- [ ] **32.** Download rejected rows as CSV with an `error` column.
- [ ] **33.** Reject duplicate phones *within* the file before touching the database.

- [ ] **34.** Zip all of a client's KYC files on the fly.
- [ ] **35.** Point the private storage at S3 with django-storages (settings only).
- [ ] **36.** Virus-scan hook: reject files containing the EICAR test marker.
- [ ] **37.** Test: upload a PDF, download it back; a fake "PDF" is rejected.
- [ ] **38.** Test: an agent can't download another client's document (404).
- [ ] **39.** Test: preview counts valid, invalid and duplicate rows.
- [ ] **40.** Test: streaming export returns header + one row per lead.

Solutions

Uploads arrive in `request.FILES` (only when the form has `enctype="multipart/form-data"`). Bind them as the second form argument: `Form(request.POST, request.FILES)`.

Settings and storage (1, 11, 35)

```
# django_project/settings.py
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"                    # public: avatars
PRIVATE_MEDIA_ROOT = BASE_DIR / "private_media"    # never served directly:
KYC, imports
FILE_UPLOAD_MAX_MEMORY_SIZE = 5 * 1024 * 1024      # bigger uploads spool to
a temp file

STORAGES = {                                        # defining STORAGES
replaces the defaults — list all three
    "default": {"BACKEND": "django.core.files.storage.FileSystemStorage"},
    "staticfiles": {"BACKEND":
"django.contrib.staticfiles.storage.StaticFilesStorage"},
    "private": {"BACKEND": "core.storage.PrivateStorage"},
}

# 35 — production: python -m pip install "django-storages[s3]"
# STORAGES["private"] = {
#     "BACKEND": "storages.backends.s3.S3Storage",
#     "OPTIONS": {"bucket_name": os.environ["KYC_BUCKET"], "region_name":
"me-central-1",
#                 "default_acl": "private", "querystring_auth": True,
"querystring_expire": 300},
# }
```

```
# django_project/urls.py — 1 (development only; production serves media from
the web server / bucket)
from django.conf import settings
from django.conf.urls.static import static

urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

```
# .gitignore
media/
private_media/
```

```
# core/storage.py — 11
import os
from django.conf import settings
from django.core.files.storage import FileSystemStorage, storages


class PrivateStorage(FileSystemStorage):
    """Outside MEDIA_ROOT: reachable only through permission-checked
views."""

    @property
    def base_location(self):          # read the setting every time →
override_settings works in tests
        return settings.PRIVATE_MEDIA_ROOT

    @property
    def location(self):
        return os.path.abspath(self.base_location)

    def url(self, name):
        return "#private"             # there is deliberately no public URL


def private_storage():
    return storages["private"]      # a callable keeps the storage out of
migration files
```

A. Drills (2–10)

2–3.

```
# core/views.py
import os
from pathlib import Path

from django import forms
from django.utils.text import get_valid_filename


class UploadTestForm(forms.Form):
    file = forms.FileField()


def upload_test(request):                                              # 2
    form = UploadTestForm(request.POST or None, request.FILES or None)
    if request.method == "POST" and form.is_valid():
        f = form.cleaned_data["file"]
        path = save_upload(f, "scratch")
        return HttpResponse(f"{f.name} · {f.size} bytes · {f.content_type} →
{path}")
    return render(request, "core/upload_test.html", {"form": form})


def save_upload(f, folder):                                            # 3
    safe = get_valid_filename(os.path.basename(f.name))   # strips ../ tricks
and odd characters
    path = Path(settings.MEDIA_ROOT) / folder / safe
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "wb") as out:
        for chunk in f.chunks():           # never f.read() a big file into
memory
            out.write(chunk)
    return path
```

```
<!-- core/templates/core/upload_test.html -->
<form method="post" enctype="multipart/form-data">{% csrf_token %}
  {{ form.as_div }}<button type="submit">Upload</button>
</form>
```

4–7, 36.

```
# core/files.py
import uuid
from pathlib import Path

from django.core.exceptions import ValidationError
from django.utils.deconstruct import deconstructible

SIGNATURES = {
    b"%PDF": "application/pdf",
    b"\xff\xd8\xff": "image/jpeg",
    b"\x89PNG\r\n\x1a\n": "image/png",
}
EICAR_MARKER = b"EICAR-STANDARD-ANTIVIRUS-TEST-FILE"


def sniff_content_type(f):                                          # 7
    f.seek(0)
    head = f.read(8)
    f.seek(0)
    for magic, ctype in SIGNATURES.items():
        if head.startswith(magic):
            return ctype
    return None


@deconstructible            # migrations must be able to write the validator
into a file
class MaxSizeValidator:                                             # 6
    def __init__(self, mb):
        self.mb = mb

    def __call__(self, f):
        if f.size > self.mb * 1024 * 1024:
            raise ValidationError(f"Maximum file size is {self.mb} MB.",
code="too_big")

    def __eq__(self, other):
        return isinstance(other, MaxSizeValidator) and self.mb == other.mb


@deconstructible
class RealFileTypeValidator:                                        # 7
    def __call__(self, f):
```

```
        if sniff_content_type(f) is None:
            raise ValidationError("The file content isn't a PDF, JPEG or
PNG.", code="bad_type")

    def __eq__(self, other):
        return isinstance(other, RealFileTypeValidator)


@deconstructible
class VirusScanValidator:                                          # 36 —
swap the body for ClamAV / a cloud scanner
    def __call__(self, f):
        f.seek(0)
        data = f.read(1024 * 1024)
        f.seek(0)
        if EICAR_MARKER in data:
            raise ValidationError("The file failed the virus scan.",
code="infected")

    def __eq__(self, other):
        return isinstance(other, VirusScanValidator)


def kyc_upload_to(instance, filename):                             # 5
    ext = Path(filename).suffix.lower()
    return f"kyc/{instance.client.ref}/{instance.doc_type}/{uuid.uuid4().hex}
{ext}"


def avatar_upload_to(instance, filename):
    return f"avatars/{instance.pk}/{uuid.uuid4().hex}.jpg"

# 4 — the string form: models.FileField(upload_to="kyc/%Y/%m/") →
kyc/2026/10/<original name>
```

8.

```
# accounts/models.py — CustomUser
from core.files import avatar_upload_to

avatar = models.ImageField(upload_to=avatar_upload_to, blank=True)     #
checks it really is an image
```

10.

```
# python manage.py shell
from django.core.files.base import ContentFile
from django.core.files.storage import default_storage, storages

name = default_storage.save("tmp/hello.txt", ContentFile(b"hi"))   # may
rename to avoid clashes
default_storage.exists(name), default_storage.size(name),
default_storage.url(name)
with default_storage.open(name) as f:
    f.read()
default_storage.delete(name)
storages["private"].location                                        #
.../private_media
```

B. KYC documents (9, 11–20, 34)

```
# clients/models.py
from django.conf import settings
from django.core.validators import FileExtensionValidator
from django.utils import timezone

from core.files import MaxSizeValidator, RealFileTypeValidator,
VirusScanValidator, kyc_upload_to
from core.storage import private_storage


class KYCDocument(models.Model):
    class DocType(models.TextChoices):
        PASSPORT = "passport", "Passport"
        EMIRATES_ID = "emirates_id", "Emirates ID"
        ADDRESS = "address", "Proof of address"
        BANK = "bank", "Bank statement"

    class Status(models.TextChoices):
        PENDING = "pending"
        APPROVED = "approved"
        REJECTED = "rejected"

    REQUIRED = [DocType.PASSPORT, DocType.EMIRATES_ID, DocType.ADDRESS]
    ID_TYPES = {DocType.PASSPORT, DocType.EMIRATES_ID}
```

```
    client = models.ForeignKey(Client, on_delete=models.CASCADE,
related_name="documents")
    doc_type = models.CharField(max_length=20, choices=DocType.choices)
    file = models.FileField(
        upload_to=kyc_upload_to, storage=private_storage,
        validators=[FileExtensionValidator(["pdf", "jpg", "jpeg", "png"]),
                    MaxSizeValidator(5), RealFileTypeValidator(),
VirusScanValidator()],
    )
    original_name = models.CharField(max_length=255, editable=False)
    size = models.PositiveIntegerField(default=0, editable=False)
    content_type = models.CharField(max_length=50, blank=True,
editable=False)
    version = models.PositiveSmallIntegerField(default=1, editable=False)
    status = models.CharField(max_length=10, choices=Status.choices,
default=Status.PENDING)
    expiry_date = models.DateField(null=True, blank=True)
    uploaded_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
on_delete=models.SET_NULL, related_name="+")
    uploaded_at = models.DateTimeField(auto_now_add=True)
    reviewed_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                                    on_delete=models.SET_NULL,
related_name="+")
    reviewed_at = models.DateTimeField(null=True, blank=True)
    review_note = models.CharField(max_length=255, blank=True)

    class Meta:
        ordering = ["doc_type", "-version"]
        constraints = [
            models.UniqueConstraint(fields=["client", "doc_type", "version"],
name="unique_kyc_version"),
        ]

    def __str__(self):
        return f"{self.get_doc_type_display()} v{self.version} ·
{self.client.name}"

    @property
    def is_expired(self):
        return bool(self.expiry_date and self.expiry_date <
timezone.localdate())
```

```
# clients/kyc.py — 15–18
from datetime import timedelta
from django.utils import timezone
from .models import Client, KYCDocument


def latest_documents(client):
    latest = {}
    for doc in client.documents.order_by("doc_type", "-version"):
        latest.setdefault(doc.doc_type, doc)       # first one per type = highest version
    return latest


def refresh_client_kyc(client):                                          # 15, 17
    latest = latest_documents(client)
    usable = {t: d for t, d in latest.items() if not d.is_expired}
    statuses = [usable[t].status if t in usable else None for t in KYCDocument.REQUIRED]
    if all(s == KYCDocument.Status.APPROVED for s in statuses):
        new = Client.KYCStatus.APPROVED
    elif any(s == KYCDocument.Status.REJECTED for s in statuses):
        new = Client.KYCStatus.REJECTED
    else:
        new = Client.KYCStatus.PENDING
    if client.kyc_status != new:
        Client.objects.filter(pk=client.pk).update(kyc_status=new)
    return new


def checklist(client):                                                   # 16
    latest = latest_documents(client)
    return [{"label": label, "doc": latest.get(value),
             "missing": value not in latest or latest[value].is_expired}
            for value, label in KYCDocument.DocType.choices if value in KYCDocument.REQUIRED]


def expiring_documents(days=30):                                         # 17
    today = timezone.localdate()
```

```
    return (KYCDocument.objects.filter(status=KYCDocument.Status.APPROVED,
                                       expiry_date__range=(today, today +
timedelta(days=days)))
            .select_related("client").order_by("expiry_date"))
```

```
# clients/forms.py — 12, 19
from .models import KYCDocument


class KYCUploadForm(forms.ModelForm):
    class Meta:
        model = KYCDocument
        fields = ["doc_type", "file", "expiry_date"]
        widgets = {"expiry_date": forms.DateInput(attrs={"type": "date"})}

    def clean(self):
        cleaned = super().clean()
        if cleaned.get("doc_type") in KYCDocument.ID_TYPES and not
cleaned.get("expiry_date"):
            self.add_error("expiry_date", "ID documents need an expiry
date.")
        return cleaned


class MultipleFileInput(forms.ClearableFileInput):                    # 19 —
Django docs pattern
    allow_multiple_selected = True


class MultipleFileField(forms.FileField):
    def __init__(self, *args, **kwargs):
        kwargs.setdefault("widget", MultipleFileInput())
        super().__init__(*args, **kwargs)

    def clean(self, data, initial=None):
        single = super().clean
        if isinstance(data, (list, tuple)):
            return [single(d, initial) for d in data]
        return [single(data, initial)]
```

```
class BankStatementsForm(forms.Form):
    files = MultipleFileField()
```

```
# clients/views.py — 12-15, 19, 34
import zipfile
from io import BytesIO
from pathlib import Path

from django.contrib.auth.mixins import PermissionRequiredMixin
from django.core.exceptions import ValidationError
from django.db.models import Max
from django.http import FileResponse, HttpResponse, HttpResponseBadRequest
from django.utils import timezone

from core.files import sniff_content_type
from .forms import BankStatementsForm, KYCUploadForm
from .kyc import refresh_client_kyc
from .models import KYCDocument


def next_version(client, doc_type):
    last = KYCDocument.objects.filter(client=client,
doc_type=doc_type).aggregate(m=Max("version"))["m"]
    return (last or 0) + 1


def fill_upload_meta(doc, upload, user):
    doc.uploaded_by = user
    doc.original_name = upload.name[:255]
    doc.size = upload.size
    doc.content_type = sniff_content_type(upload) or ""
    doc.version = next_version(doc.client, doc.doc_type)              # 18:
never overwrite — add a version


class ClientFromURLMixin:
    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, **kwargs)
        self.crm_client = get_object_or_404(visible_clients(request.user),
ref=kwargs["ref"])
```

```
    def get_context_data(self, **kwargs):
        return {**super().get_context_data(**kwargs), "crm_client":
self.crm_client}


class KYCUploadView(ClientFromURLMixin, PageTitleMixin, CreateView):
# 12
    form_class = KYCUploadForm
    template_name = "clients/kyc_upload.html"
    page_title = "Upload KYC document"

    def form_valid(self, form):
        form.instance.client = self.crm_client             # upload_to reads
client.ref at save time
        fill_upload_meta(form.instance, form.cleaned_data["file"],
self.request.user)
        response = super().form_valid(form)
        refresh_client_kyc(self.crm_client)
        messages.success(self.request, f"{form.instance} uploaded.")
        return response

    def get_success_url(self):
        return self.crm_client.get_absolute_url()


class BankStatementsUploadView(ClientFromURLMixin, PageTitleMixin, FormView):
# 19
    form_class = BankStatementsForm
    template_name = "clients/kyc_upload.html"
    page_title = "Upload bank statements"

    def form_valid(self, form):
        saved, failed = 0, []
        for upload in form.cleaned_data["files"]:
            doc = KYCDocument(client=self.crm_client,
doc_type=KYCDocument.DocType.BANK, file=upload)
            fill_upload_meta(doc, upload, self.request.user)
            try:
                doc.full_clean()           # run the model validators file by
file
            except ValidationError as exc:
                failed.append(f"{upload.name}: {'; '.join(exc.messages)}")
                continue
```

```
            doc.save()
            saved += 1
        messages.success(self.request, f"{saved} statement(s) uploaded.")
        for problem in failed:
            messages.error(self.request, problem)
        return redirect(self.crm_client)


class KYCDownloadView(View):                                                #
13-14
    INLINE_TYPES = {"application/pdf", "image/jpeg", "image/png"}

    def get(self, request, pk):
        doc =
get_object_or_404(KYCDocument.objects.filter(client__in=visible_clients(reque
st.user)), pk=pk)
        inline = request.GET.get("inline") == "1" and doc.content_type in
self.INLINE_TYPES
        response = FileResponse(
            doc.file.open("rb"),
            as_attachment=not inline,
            filename=doc.original_name,
            content_type=doc.content_type or "application/octet-stream",
        )
        response["X-Content-Type-Options"] = "nosniff"     # the browser must
not guess another type
        response["Cache-Control"] = "private, no-store"    # ID documents
never sit in a shared cache
        return response


class KYCReviewView(PermissionRequiredMixin, View):                         #
15
    permission_required = "clients.change_client"
    http_method_names = ["post"]

    def post(self, request, pk):
        doc = get_object_or_404(

KYCDocument.objects.filter(client__in=visible_clients(request.user)).select_r
elated("client"), pk=pk,
        )
        decision = request.POST.get("decision")
```

```
        if decision not in {KYCDocument.Status.APPROVED,
KYCDocument.Status.REJECTED}:
            return HttpResponseBadRequest("decision must be approved or
rejected")
        doc.status = decision
        doc.reviewed_by = request.user
        doc.reviewed_at = timezone.now()
        doc.review_note = request.POST.get("note", "")[:255]
        doc.save(update_fields=["status", "reviewed_by", "reviewed_at",
"review_note"])
        new_status = refresh_client_kyc(doc.client)
        messages.success(request, f"{doc} {decision}. Client KYC:
{new_status}.")
        return redirect(doc.client)


class KYCZipView(View):                                                 #
34
    def get(self, request, ref):
        crm_client = get_object_or_404(visible_clients(request.user),
ref=ref)
        buffer = BytesIO()
        with zipfile.ZipFile(buffer, "w", zipfile.ZIP_DEFLATED) as zf:
            for doc in crm_client.documents.all():
                with doc.file.open("rb") as fh:
                    zf.writestr(f"{doc.doc_type}-v{doc.version}
{Path(doc.file.name).suffix}", fh.read())
        response = HttpResponse(buffer.getvalue(),
content_type="application/zip")
        response["Content-Disposition"] = f'attachment; filename="kyc-
{crm_client.account_number}.zip"'
        return response

# clients/urls.py
path("<uuid:ref>/kyc/upload/", views.KYCUploadView.as_view(),
name="kyc_upload"),
path("<uuid:ref>/kyc/statements/", views.BankStatementsUploadView.as_view(),
name="kyc_statements"),
path("<uuid:ref>/kyc/zip/", views.KYCZipView.as_view(), name="kyc_zip"),
path("kyc/<int:pk>/download/", views.KYCDownloadView.as_view(),
name="kyc_download"),
path("kyc/<int:pk>/review/", views.KYCReviewView.as_view(),
name="kyc_review"),
```

```
# clients/signals.py — 9 (import it in ClientsConfig.ready())
from django.db import transaction
from django.db.models.signals import post_delete
from django.dispatch import receiver
from .models import KYCDocument


@receiver(post_delete, sender=KYCDocument)
def delete_kyc_file(sender, instance, **kwargs):
    if instance.file:
        # after COMMIT: a rolled-back delete must not lose the file
        transaction.on_commit(lambda: instance.file.delete(save=False))
```

```
<!-- clients/templates/clients/kyc_upload.html -->
{% extends "base.html" %}
{% block content %}
<h1>{{ page_title }} · {{ crm_client.name }}</h1>
<form method="post" enctype="multipart/form-data">{% csrf_token %}
  {{ form.as_div }}<button type="submit">Upload</button>
</form>
{% endblock content %}
```

```
<!-- clients/client_detail.html — 16, 18 (context: checklist =
checklist(client), documents = client.documents.all) -->
<h2>KYC · {{ client.get_kyc_status_display }}</h2>
<ul>
  {% for item in checklist %}
    <li>{{ item.label }}:
      {% if item.missing %}<strong>missing{% if item.doc and
item.doc.is_expired %} (expired){% endif %}</strong>
      {% else %}{{ item.doc.get_status_display }} · v{{ item.doc.version }}{%
endif %}
    </li>
  {% endfor %}
</ul>
<a href="{% url 'clients:kyc_upload' client.ref %}">Upload document</a> ·
<a href="{% url 'clients:kyc_zip' client.ref %}">Download all (.zip)</a>

<table class="table">
  {% for doc in documents %}
  <tr>
    <td>{{ doc.get_doc_type_display }} v{{ doc.version }}</td>
```

```
    <td><a href="{% url 'clients:kyc_download' doc.pk %}?inline=1">{{
doc.original_name }}</a> ({{ doc.size|filesizeformat }})</td>
    <td>{{ doc.get_status_display }}{% if doc.expiry_date %} · expires {{
doc.expiry_date }}{% endif %}</td>
    <td>{% if perms.clients.change_client and doc.status == "pending" %}
      <form method="post" action="{% url 'clients:kyc_review' doc.pk %}"
class="inline">{% csrf_token %}
        <button name="decision" value="approved">Approve</button>
        <button name="decision" value="rejected">Reject</button>
      </form>{% endif %}</td>
  </tr>
  {% endfor %}
</table>
```

20.

```
# accounts/images.py
from io import BytesIO
from django.core.files.base import ContentFile
from PIL import Image, ImageOps


def make_avatar(upload, size=256):
    img = Image.open(upload)
    img = ImageOps.exif_transpose(img).convert("RGB")   # phone photos: fix
rotation, drop alpha
    img.thumbnail((size, size))                          # keeps the aspect
ratio
    buffer = BytesIO()
    img.save(buffer, "JPEG", quality=85)
    return ContentFile(buffer.getvalue(), name="avatar.jpg")

# accounts/forms.py — ProfileForm.Meta.fields += ["avatar"]
# accounts/views.py — ProfileView
def form_valid(self, form):
    if "avatar" in form.changed_data and form.cleaned_data.get("avatar"):
        form.instance.avatar = make_avatar(form.cleaned_data["avatar"])
    return super().form_valid(form)
```

The profile template's form needs `enctype="multipart/form-data"` too.

C. Import and export (21–33)

```
# leads/models.py
from core.storage import private_storage


class ImportBatch(models.Model):
    class Status(models.TextChoices):
        PREVIEW = "preview"
        RUNNING = "running"
        DONE = "done"
        FAILED = "failed"

    created_by = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name="imports")
    file = models.FileField(upload_to="imports/%Y/%m/",
storage=private_storage)
    source = models.CharField(max_length=20, choices=Lead.Source.choices,
default=Lead.Source.MANUAL)
    assign_to = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                                  on_delete=models.SET_NULL,
related_name="+")
    status = models.CharField(max_length=10, choices=Status.choices,
default=Status.PREVIEW)
    total_rows = models.PositiveIntegerField(default=0)
    valid_rows = models.PositiveIntegerField(default=0)
    processed_n = models.PositiveIntegerField(default=0)
    created_n = models.PositiveIntegerField(default=0)
    updated_n = models.PositiveIntegerField(default=0)
    errors = models.JSONField(default=list, blank=True)   # [{"row": 5,
"data": {...}, "errors": {...}}]
    created_at = models.DateTimeField(auto_now_add=True)
    finished_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ["-created_at"]
```

```
# leads/forms.py — 21, 23
from django.core.validators import FileExtensionValidator


class LeadImportForm(forms.Form):
    file = forms.FileField(validators=[FileExtensionValidator(["csv",
"xlsx"])],
                           help_text="Columns: name, phone (email optional)")
    source = forms.ChoiceField(choices=Lead.Source.choices,
initial=Lead.Source.MANUAL)
    assign_to = forms.ModelChoiceField(required=False,

queryset=get_user_model().objects.filter(role="agent", is_active=True))


class ImportRowForm(QuickLeadForm):
    email = forms.EmailField(required=False)
```

```
# leads/importers.py — 22–25, 27, 31, 33
import csv
import io

from django.db import transaction
from django.utils import timezone
from openpyxl import load_workbook

from .forms import ImportRowForm
from .models import ImportBatch, Lead
from .services import upsert_leads

REQUIRED_HEADERS = {"name", "phone"}
HEADER_ALIASES = {"full name": "name", "mobile": "phone", "phone number":
"phone", "e-mail": "email"}
ENCODINGS = ("utf-8-sig", "cp1256")              # UTF-8 with or without BOM,
then Arabic Windows


class ImportFileError(Exception):
    pass


def _header(value):
```

```
    key = str(value or "").strip().lower()
    return HEADER_ALIASES.get(key, key)


def _check(headers):
    missing = REQUIRED_HEADERS - set(headers)
    if missing:
        raise ImportFileError(f"Missing column(s): {',
'.join(sorted(missing))}")


def _read_csv(fh):                                                  # 22
    fh.seek(0)
    raw = fh.read()
    for encoding in ENCODINGS:
        try:
            text = raw.decode(encoding)
            break
        except UnicodeDecodeError:
            continue
    else:
        raise ImportFileError("Couldn't read the file — save it as CSV UTF-
8.")
    reader = csv.reader(io.StringIO(text))
    headers = [_header(h) for h in next(reader, [])]
    _check(headers)
    for values in reader:
        if any(v.strip() for v in values):
            yield dict(zip(headers, (v.strip() for v in values)))


def _cell(value):
    if isinstance(value, float) and value.is_integer():
        value = int(value)              # 971501234567.0 would gain an extra
digit after normalising
    return "" if value is None else str(value).strip()


def _read_xlsx(fh):                                                 # 27
    workbook = load_workbook(fh, read_only=True, data_only=True)
    rows = workbook.active.iter_rows(values_only=True)
    headers = [_header(h) for h in next(rows, [])]
    _check(headers)
```

```
    for values in rows:
        cells = [_cell(v) for v in values]
        if any(cells):
            yield dict(zip(headers, cells))


def read_rows(fh):
    name = (getattr(fh, "name", "") or "").lower()
    if name.endswith(".xlsx"):
        return list(_read_xlsx(fh))
    if name.endswith(".csv"):
        return list(_read_csv(fh))
    raise ImportFileError("Upload a .csv or .xlsx file.")


def validate_rows(rows, default_source):                                # 23,
33
    valid, errors, seen = [], [], set()
    for number, row in enumerate(rows, start=2):                        # row 1
is the header
        form = ImportRowForm({
            "name": row.get("name", ""), "phone": row.get("phone", ""),
            "source": row.get("source") or default_source, "email":
row.get("email", ""),
        })
        if not form.is_valid():
            errors.append({"row": number, "data": row,
                           "errors": {f: [str(m) for m in msgs] for f, msgs
in form.errors.items()}})
            continue
        phone = form.cleaned_data["phone"]
        if phone in seen:                                               # 33:
duplicate inside the file
            errors.append({"row": number, "data": row, "errors": {"phone":
["Duplicate phone in this file."]}})
            continue
        seen.add(phone)
        valid.append(form.cleaned_data)
    return valid, errors


def run_import(batch, rows, chunk_size=500):                            # 25,
31
```

```
    phones = [r["phone"] for r in rows]
    existing = set(Lead.objects.filter(phone__in=phones).values_list("phone",
flat=True))

ImportBatch.objects.filter(pk=batch.pk).update(status=ImportBatch.Status.RUNN
ING)
    try:
        for start in range(0, len(rows), chunk_size):
            chunk = rows[start:start + chunk_size]
            with transaction.atomic():           # one transaction per chunk
→ progress is visible to others
                upsert_leads(chunk, assign_to=batch.assign_to)

ImportBatch.objects.filter(pk=batch.pk).update(processed_n=start +
len(chunk))
    except Exception:

ImportBatch.objects.filter(pk=batch.pk).update(status=ImportBatch.Status.FAIL
ED,

finished_at=timezone.now())
        raise
    batch.created_n = sum(1 for p in phones if p not in existing)
    batch.updated_n = len(rows) - batch.created_n
    batch.processed_n = len(rows)
    batch.status = ImportBatch.Status.DONE
    batch.finished_at = timezone.now()
    batch.save(update_fields=["created_n", "updated_n", "processed_n",
"status", "finished_at"])
    return batch
```

```
# leads/services.py — upsert_leads gains assign_to (applies to new rows only;
existing leads keep their agent)
def upsert_leads(rows, assign_to=None):
    objs = [Lead(name=r["name"].strip().title(),
phone=normalize_uae_phone(r["phone"]),
                 source=r.get("source", Lead.Source.MANUAL),
email=r.get("email", ""), assigned_to=assign_to)
            for r in rows]
    return Lead.objects.bulk_create(
        objs, update_conflicts=True, unique_fields=["phone"],
        update_fields=["name", "source", "email", "updated_at"],
batch_size=500,
    )
```

```
# leads/views.py — 24-26, 32
from .forms import LeadImportForm
from .importers import ImportFileError, read_rows, run_import, validate_rows
from .models import ImportBatch


class LeadImportView(PermissionRequiredMixin, PageTitleMixin, FormView):    #
24
    permission_required = "leads.add_lead"
    form_class = LeadImportForm
    template_name = "leads/import_upload.html"
    page_title = "Import leads"

    def form_valid(self, form):
        upload = form.cleaned_data["file"]
        try:
            valid, errors = validate_rows(read_rows(upload),
form.cleaned_data["source"])
        except ImportFileError as exc:
            form.add_error("file", str(exc))
            return self.form_invalid(form)
        batch = ImportBatch.objects.create(
            created_by=self.request.user, file=upload,           # saving
rewinds the upload first
            source=form.cleaned_data["source"],
assign_to=form.cleaned_data["assign_to"],
            total_rows=len(valid) + len(errors), valid_rows=len(valid),
errors=errors,
```

```
        )
        return redirect("leads:import_detail", pk=batch.pk)


class ImportBatchDetailView(PageTitleMixin, DetailView):                    #
24: preview + result page
    template_name = "leads/import_detail.html"
    context_object_name = "batch"
    page_title = "Import"

    def get_queryset(self):
        return ImportBatch.objects.filter(created_by=self.request.user)


class ImportConfirmView(PermissionRequiredMixin, View):                     #
25
    permission_required = "leads.add_lead"
    http_method_names = ["post"]

    def post(self, request, pk):
        batch = get_object_or_404(ImportBatch, pk=pk,
created_by=request.user,
                                  status=ImportBatch.Status.PREVIEW)
        with batch.file.open("rb") as fh:
            valid, _ = validate_rows(read_rows(fh), batch.source)    # re-
validate: the DB may have changed
        run_import(batch, valid)
        messages.success(request, f"Imported: {batch.created_n} new,
{batch.updated_n} updated.")
        return redirect("leads:import_detail", pk=batch.pk)


class ImportHistoryView(PageTitleMixin, ListView):                          #
26
    template_name = "leads/import_history.html"
    context_object_name = "batches"
    paginate_by = 20
    page_title = "Imports"

    def get_queryset(self):
        return ImportBatch.objects.filter(created_by=self.request.user)
```

```
class ImportErrorsCSVView(View):                                        #
32
    def get(self, request, pk):
        batch = get_object_or_404(ImportBatch, pk=pk,
created_by=request.user)
        response = HttpResponse(content_type="text/csv; charset=utf-8")
        response["Content-Disposition"] = f'attachment; filename="import-
{batch.pk}-errors.csv"'
        response.write("\ufeff")
        writer = csv.writer(response)
        columns = sorted({key for e in batch.errors for key in e["data"]})
        writer.writerow(["row", *columns, "error"])
        for e in batch.errors:
            problems = "; ".join(f"{field}: {' '.join(msgs)}" for field, msgs
in e["errors"].items())
            writer.writerow([e["row"], *[e["data"].get(c, "") for c in
columns], problems])
        return response

# leads/urls.py
path("import/", views.LeadImportView.as_view(), name="import"),
path("import/history/", views.ImportHistoryView.as_view(),
name="import_history"),
path("import/<int:pk>/", views.ImportBatchDetailView.as_view(),
name="import_detail"),
path("import/<int:pk>/confirm/", views.ImportConfirmView.as_view(),
name="import_confirm"),
path("import/<int:pk>/errors.csv", views.ImportErrorsCSVView.as_view(),
name="import_errors"),
```

```
<!-- leads/templates/leads/import_detail.html -->
{% extends "base.html" %}
{% block content %}
<h1>Import #{{ batch.pk }} · {{ batch.get_status_display }}</h1>
<p>{{ batch.total_rows }} rows · {{ batch.valid_rows }} valid · {{
batch.errors|length }} with errors</p>
{% if batch.status == "preview" %}
  <form method="post" action="{% url 'leads:import_confirm' batch.pk %}">{%
csrf_token %}
    <button type="submit">Import {{ batch.valid_rows }} lead{{
batch.valid_rows|pluralize }}</button>
  </form>
{% else %}
```

```
  <p>Processed {{ batch.processed_n }} · {{ batch.created_n }} new · {{
batch.updated_n }} updated</p>
{% endif %}
{% if batch.errors %}
  <a href="{% url 'leads:import_errors' batch.pk %}">Download rejected
rows</a>
  <table class="table">
    <tr><th>Row</th><th>Data</th><th>Problem</th></tr>
    {% for e in batch.errors|slice:":50" %}
      <tr><td>{{ e.row }}</td><td>{{ e.data.name }} · {{ e.data.phone }}</td>
          <td>{% for field, msgs in e.errors.items %}{{ field }}: {{
msgs|join:" " }} {% endfor %}</td></tr>
    {% endfor %}
  </table>
{% endif %}
{% endblock content %}
```

The upload template (`import_upload.html`) is the usual form with `enctype="multipart/form-data"`.

28–30.

```
# leads/exports.py
import csv
from datetime import datetime
from io import BytesIO

from django.utils import timezone
from openpyxl import Workbook
from openpyxl.styles import Font
from openpyxl.utils import get_column_letter

EXPORT_FIELDS = ("id", "name", "phone", "email", "status", "source",
                 "assigned_to__username", "expected_deposit", "created_at")
EXPORT_HEADER = ("ID", "Name", "Phone", "Email", "Status", "Source", "Agent",
"Expected deposit", "Created")


def localize(value):                                                  # 29
    if isinstance(value, datetime):
        return timezone.localtime(value).replace(tzinfo=None)    # Dubai
time; Excel can't store time zones
    return value
```

```
class Echo:
    def write(self, value):
        return value              # csv.writer "writes" by handing back the
formatted line


def stream_csv(rows):                                                  # 28–29
    writer = csv.writer(Echo())
    yield "\ufeff"                # BOM: Excel then reads UTF-8, so Arabic
names stay readable
    yield writer.writerow(EXPORT_HEADER)
    for row in rows:
        yield writer.writerow([localize(v) for v in row])


def build_xlsx(rows):                                                  # 30
    wb = Workbook()
    ws = wb.active
    ws.title = "Leads"
    ws.append(EXPORT_HEADER)
    for cell in ws[1]:
        cell.font = Font(bold=True)
    for row in rows:
        ws.append([localize(v) for v in row])
    ws.freeze_panes = "A2"
    for i, width in enumerate([8, 24, 16, 28, 12, 12, 14, 16, 20], start=1):
        ws.column_dimensions[get_column_letter(i)].width = width
    for cell in ws["H"][1:]:
        cell.number_format = "#,##0.00"
    buffer = BytesIO()
    wb.save(buffer)
    return buffer.getvalue()
```

```
# leads/views.py
from django.http import StreamingHttpResponse
from .exports import EXPORT_FIELDS, build_xlsx, stream_csv


class LeadStreamExportView(PermissionRequiredMixin, LeadQueryMixin, View):  #
28
    permission_required = "leads.export_lead"
```

```
    def get(self, request):
        rows =
self.get_leads().values_list(*EXPORT_FIELDS).iterator(chunk_size=2000)   #
never all in memory
        response = StreamingHttpResponse(stream_csv(rows),
content_type="text/csv; charset=utf-8")
        response["Content-Disposition"] = 'attachment; filename="leads.csv"'
        return response


class LeadXlsxExportView(PermissionRequiredMixin, LeadQueryMixin, View):    #
30
    permission_required = "leads.export_lead"

    def get(self, request):
        rows =
self.get_leads().values_list(*EXPORT_FIELDS).iterator(chunk_size=2000)
        response = HttpResponse(
            build_xlsx(rows),
            content_type="application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet",
        )
        response["Content-Disposition"] = 'attachment; filename="leads.xlsx"'
        return response

# leads/urls.py
path("export/stream/", views.LeadStreamExportView.as_view(),
name="export_stream"),
path("export/xlsx/", views.LeadXlsxExportView.as_view(), name="export_xlsx"),
```

Tests (37–40)

```
# clients/tests.py
import csv
import io
import shutil
import tempfile

from django.core.files.uploadedfile import SimpleUploadedFile
from django.test import TestCase, override_settings
from django.urls import reverse
```

```
from core.factories import AgentFactory, ClientFactory, LeadFactory,
ManagerFactory
from leads.importers import read_rows, validate_rows
from .models import KYCDocument

TEMP_MEDIA = tempfile.mkdtemp()
PDF_BYTES = b"%PDF-1.4\n1 0 obj<<>>endobj\ntrailer<<>>\n%%EOF"


def pdf(name="bill.pdf", content=PDF_BYTES):
    return SimpleUploadedFile(name, content, content_type="application/pdf")


@override_settings(MEDIA_ROOT=TEMP_MEDIA, PRIVATE_MEDIA_ROOT=TEMP_MEDIA)   #
never write into real folders
class FileTests(TestCase):
    @classmethod
    def tearDownClass(cls):
        shutil.rmtree(TEMP_MEDIA, ignore_errors=True)
        super().tearDownClass()

    def test_upload_download_and_fake_rejected(self):                  # 37
        self.client.force_login(ManagerFactory())
        crm_client = ClientFactory()
        url = reverse("clients:kyc_upload", args=[crm_client.ref])
        self.client.post(url, {"doc_type": "address", "file": pdf()})
        doc = crm_client.documents.get()
        self.assertEqual((doc.original_name, doc.content_type, doc.version),
("bill.pdf", "application/pdf", 1))
        response = self.client.get(reverse("clients:kyc_download", args=
[doc.pk]))
        self.assertEqual(b"".join(response.streaming_content), PDF_BYTES)

        response = self.client.post(url, {"doc_type": "address", "file":
pdf("evil.pdf", b"MZ\x90\x00 not a pdf")})
        self.assertContains(response, "t a PDF, JPEG or PNG")
        self.assertEqual(crm_client.documents.count(), 1)

    def test_other_agents_cannot_download(self):                       # 38
        owner, stranger = AgentFactory(), AgentFactory()
        crm_client = ClientFactory(lead=LeadFactory(assigned_to=owner))
        doc = KYCDocument.objects.create(client=crm_client,
```

```
doc_type="address", file=pdf(), original_name="bill.pdf")
        url = reverse("clients:kyc_download", args=[doc.pk])
        self.client.force_login(stranger)
        self.assertEqual(self.client.get(url).status_code, 404)
        self.client.force_login(owner)
        self.assertEqual(self.client.get(url).status_code, 200)


class ImportExportTests(TestCase):
    def test_preview_counts(self):                                      # 39
        content = "Name,Mobile,Email\nAli Khan,0501112233,ali@x.test\nBad
Phone,12345,\nDup,0501112233,\n"
        upload = SimpleUploadedFile("leads.csv", content.encode("utf-8-sig"),
content_type="text/csv")
        valid, errors = validate_rows(read_rows(upload), "manual")
        self.assertEqual(len(valid), 1)
        self.assertEqual([e["row"] for e in errors], [3, 4])          # bad
phone, then the duplicate

    def test_stream_export(self):                                       # 40
        self.client.force_login(ManagerFactory())
        LeadFactory.create_batch(3, source="walk_in")
        response = self.client.get(reverse("leads:export_stream"), {"source":
"walk_in"})
        content = b"".join(response.streaming_content).decode("utf-8-sig")
        rows = list(csv.reader(io.StringIO(content)))
        self.assertEqual(len(rows), 4)                                   #
header + 3
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py test
git add -A && git commit -m "day 19"
```

Day 20 — Managers, QuerySets, signals

Goal: move query logic onto the models (`Lead.objects.visible_to(user).active().open()`), and let signals record what happened — status history, an audit trail that also catches admin and shell edits, account balances and large-deposit alerts.

Rule of thumb: **QuerySet methods** filter many rows, **model methods** act on one row, **services** run multi-step business actions, **signals** react to saves you don't control.

A. Drills (1–10) — QuerySets and managers

- [ ] **1.** `LeadQuerySet` with `active()`, `open()`, `by_status()`; `objects = LeadQuerySet.as_manager()`.
- [ ] **2.** Chain them with normal filters: `Lead.objects.active().open().filter(source="google")`.
- [ ] **3.** Second manager `active_objects` whose `get_queryset()` hides archived leads.
- [ ] **4.** Why the *first* manager is the default (admin, `dumpdata`, related lookups) and what `Meta.default_manager_name` / `base_manager_name` change.
- [ ] **5.** `LeadManager = Manager.from_queryset(LeadQuerySet)` plus a manager-only method `create_from_phone()`.
- [ ] **6.** `with_call_stats()` annotation method.
- [ ] **7.** Related managers inherit the methods: `agent.leads.open()`.
- [ ] **8.** Move the Day 7 finders onto the QuerySet: `stale()`, `follow_up_due()`, `burned()`, `high_value()`, `created_today()`.
- [ ] **9.** Move `visible_leads()` onto it: `Lead.objects.visible_to(user)`; keep the old function as a one-line wrapper.
- [ ] **10.** Property for one row: `lead.is_stale`.

B. Core (11–20) — signals

- [ ] **11.** `post_save` on `Lead`: log `created` / `updated` to `ActivityLog` (replaces Day 14's `AuditMixin`).
- [ ] **12.** Change tracking: remember loaded values in `from_db()`; `changed_fields()` diff.
- [ ] **13.** `LeadStatusChange` history model filled by the receiver.

- [ ] **14.** Who did it: a `contextvars` "current user" set by middleware, read by receivers.
- [ ] **15.** `m2m_changed` on `Lead.tags`: log tags added/removed.
- [ ] **16.** `post_delete`: log deletions with the object's text.
- [ ] **17.** Denormalised `Lead.last_call_outcome` kept up to date by a `CallLog` `post_save`.
- [ ] **18.** Custom signal `lead_converted`, sent by the conversion service after commit.
- [ ] **19.** Side effects (emails, alerts) wrapped in `transaction.on_commit` inside receivers.
- [ ] **20.** `audit_disabled()` context manager that switches the audit receivers off (seeds, imports).

C. Nexora CRM (21–30)

- [ ] **21.** One generic audit receiver for `Client`, `TradingAccount`, `Deposit`, registered in `ready()` with `dispatch_uid`.
- [ ] **22.** Field-level diffs in the audit row (`{"status": ["new", "contacted"]}`).
- [ ] **23.** Lead timeline: activity, status changes and calls merged by time on the detail page.
- [ ] **24.** `ClientQuerySet`: `with_total_deposits()`, `kyc_pending()`, `visible_to(user)`.
- [ ] **25.** `CallLogQuerySet`: `today()`, `answered()`, `for_agent(user)`.
- [ ] **26.** `TradingAccount.balance` maintained by `Deposit` save/delete signals (remove Day 14's `F()` update in the view).
- [ ] **27.** `Alert` model; a deposit ≥ 50,000 creates a manager alert.
- [ ] **28.** Unread alerts on the dashboard with a "mark read" button.
- [ ] **29.** Connect every receiver in `AppConfig.ready()`; prove nothing registers twice.
- [ ] **30.** Refactor two report queries to use the new QuerySet methods.

D. Challenge (31–40)

- [ ] **31.** `use_in_migrations = True` — and why methods that use `self.model.Status` break in data migrations.
- [ ] **32.** Soft delete as a reusable abstract model: `deleted_at`, `delete()` archives, `hard_delete()`, `objects` / `all_objects`; apply to `ImportBatch`.
- [ ] **33.** `Prefetch("leads", queryset=Lead.objects.open())` on agents.
- [ ] **34.** `send_robust()`: one broken receiver doesn't stop the others.
- [ ] **35.** Recursion trap: a `post_save` receiver that calls `save()` — and the `update()` fix.

- [ ] **36.** `update()` / `bulk_create()` skip signals: write status history explicitly in the bulk action.
- [ ] **37.** Test: QuerySet chains and related-manager methods.
- [ ] **38.** Test: a status change through the UI creates a history row with `changed_by`.
- [ ] **39.** Test: deposits add to and subtract from the account balance.
- [ ] **40.** Test: `audit_disabled()` prevents audit rows.

Solutions

A. QuerySets and managers (1–10)

```
# leads/models.py (above Lead)
from datetime import timedelta

from django.db.models import Count, Max, Q
from django.utils import timezone

from .permissions import sees_everything


class LeadQuerySet(models.QuerySet):
    # ---- 1: building blocks ------------------------------------------------
----
    def active(self):
        return self.filter(is_active=True)

    def open(self):
        S = self.model.Status                  # self.model: works for
subclasses too
        return self.filter(status__in=[S.NEW, S.CONTACTED])

    def by_status(self, status):
        return self.filter(status=status)

    # ---- 6 -----------------------------------------------------------------
------
    def with_call_stats(self):
        return self.annotate(call_count=Count("calls", distinct=True),
last_call_at=Max("calls__started_at"))
```

```
    # ---- 8: the Day 7 finders ------------------------------------------
----
    def stale(self, days=7):
        cutoff = timezone.now() - timedelta(days=days)
        return self.filter(status=self.model.Status.NEW,
created_at__lt=cutoff,

last_contacted_at__isnull=True).order_by("created_at")

    def follow_up_due(self, hours=48, max_attempts=3):
        cutoff = timezone.now() - timedelta(hours=hours)
        return self.filter(status=self.model.Status.CONTACTED,
last_contacted_at__lt=cutoff,

contact_attempts__lt=max_attempts).order_by("last_contacted_at")

    def burned(self, max_attempts=3):
        return self.open().filter(contact_attempts__gte=max_attempts)

    def high_value(self, threshold=20_000):
        return (self.filter(expected_deposit__gte=threshold)
                .exclude(status=self.model.Status.LOST).order_by("-
expected_deposit"))

    def created_today(self):
        return self.filter(created_at__date=timezone.localdate())

    # ---- 9: row scoping ------------------------------------------------
------
    def visible_to(self, user):
        if not user.is_authenticated:
            return self.none()
        if sees_everything(user):
            return self
        if user.role == "team_lead":
            return self.filter(Q(assigned_to=user) |
Q(assigned_to__team_lead=user))
        return self.filter(assigned_to=user)


class LeadManager(models.Manager.from_queryset(LeadQuerySet)):          # 5
    def create_from_phone(self, phone, **extra):
        lead, _ = self.get_or_create(
```

```
            phone=normalize_uae_phone(phone),
            defaults={"name": extra.pop("name", "Unknown"), **extra},
        )
        return lead


class ActiveLeadManager(LeadManager):                                   # 3
    def get_queryset(self):
        return super().get_queryset().filter(is_active=True)


from core.models import TrackChangesMixin      # defined in section B


class Lead(TrackChangesMixin, models.Model):
    ...
    objects = LeadManager()            # 4: listed first → the default
manager (admin, dumpdata, related)
    active_objects = ActiveLeadManager()

    class Meta:
        ...
        default_manager_name = "objects"   # explicit, so reordering managers
can't hide archived rows

    @property
    def is_stale(self):                                                 # 10
        return (self.status == self.Status.NEW and self.last_contacted_at is
None
                and self.created_at < timezone.now() - timedelta(days=7))
```

1 without the custom manager is just `objects = LeadQuerySet.as_manager()`; `from_queryset` (5) is for when you also need manager-only methods.

```
# leads/permissions.py — 9: old call sites keep working
def visible_leads(user, qs=None):
    from .models import Lead
    return (Lead.objects.all() if qs is None else qs).visible_to(user)


# leads/queries.py — 8: FINDERS now point at the QuerySet
FINDERS = {
    "stale": lambda: Lead.objects.stale(),
    "follow_up": lambda: Lead.objects.follow_up_due(),
    "burned": lambda: Lead.objects.burned(),
    "high_value": lambda: Lead.objects.high_value(),
    "today": lambda: Lead.objects.created_today(),
}
```

```
# python manage.py shell — 2, 7
Lead.objects.active().open().filter(source="google").count()
Lead.objects.visible_to(agent).active().stale()
agent.leads.open()            # 7: reverse managers subclass the default
manager → same methods
Lead.active_objects.count() < Lead.objects.count()
```

B. Signals (11–20)

12.

```
# core/models.py
from django.db.models import DEFERRED


class TrackChangesMixin(models.Model):
    tracked_fields = ()

    class Meta:
        abstract = True

    @classmethod
    def from_db(cls, db, field_names, values):          # called for every
row loaded from the DB
        instance = super().from_db(db, field_names, values)
        instance._loaded_values = dict(zip(field_names, values))   # keyed by
attname (assigned_to_id)
```

```
        return instance

    def changed_fields(self, only=None):
        loaded = getattr(self, "_loaded_values", None)
        if loaded is None:
            return {}
        changes = {}
        for name in self.tracked_fields:
            if only is not None and name not in only:
                continue
            attname = self._meta.get_field(name).attname
            old = loaded.get(attname, DEFERRED)
            if old is DEFERRED:
                continue                                # field wasn't loaded
(only()/defer())
            new = getattr(self, attname)
            if old != new:
                changes[name] = (old, new)
        return changes

    def reset_tracking(self):
        self._loaded_values = {self._meta.get_field(n).attname: getattr(self,
self._meta.get_field(n).attname)
                               for n in self.tracked_fields}


# leads/models.py
class Lead(TrackChangesMixin, models.Model):
    tracked_fields = ("status", "assigned_to", "expected_deposit",
"is_active", "source")
    ...
```

13, 17.

```
# leads/models.py
class LeadStatusChange(models.Model):                                    # 13
    lead = models.ForeignKey(Lead, on_delete=models.CASCADE,
related_name="status_changes")
    from_status = models.CharField(max_length=20, blank=True)
    to_status = models.CharField(max_length=20)
    changed_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                                   on_delete=models.SET_NULL,
related_name="+")
    changed_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-changed_at", "-id"]


# Lead (17)
last_call_outcome = models.CharField(max_length=20, blank=True)
```

14, 20.

```
# core/context.py
from contextlib import contextmanager
from contextvars import ContextVar

_current_user = ContextVar("current_user", default=None)
_audit_enabled = ContextVar("audit_enabled", default=True)


def current_user():
    user = _current_user.get()
    return user if user is not None and getattr(user, "is_authenticated",
False) else None


@contextmanager
def audit_disabled():                                                    # 20
    token = _audit_enabled.set(False)
    try:
        yield
    finally:
        _audit_enabled.reset(token)
```

```
def audit_is_enabled():
    return _audit_enabled.get()


# core/middleware.py — 14 (add right after AuthenticationMiddleware in
MIDDLEWARE)
from core.context import _current_user


class CurrentUserMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        token = _current_user.set(getattr(request, "user", None))
        try:
            return self.get_response(request)
        finally:
            _current_user.reset(token)        # contextvars: safe with
threads and async
```

A contextvar (not a global) keeps each request's user separate, even with threads or async. Shell scripts and management commands have no current user, so `changed_by` stays empty there.

```
# core/audit.py
from django.contrib.contenttypes.models import ContentType


def log_activity(obj, verb, changes=None, actor=None):
    from core.models import ActivityLog
    return ActivityLog.objects.create(
        actor=actor, verb=verb,
        content_type=ContentType.objects.get_for_model(obj),
        object_id=str(obj.pk), object_repr=str(obj)[:200],
        changes=changes or {},
    )


def as_text(changes):
    return {name: ["" if old is None else str(old), "" if new is None else
str(new)]
            for name, (old, new) in changes.items()}
```

11, 13, 15–17.

```
# leads/signals.py
from django.db.models.signals import m2m_changed, post_delete, post_save
from django.dispatch import receiver

from core.audit import as_text, log_activity
from core.context import audit_is_enabled, current_user
from .models import CallLog, Lead, LeadStatusChange, Tag


@receiver(post_save, sender=Lead, dispatch_uid="lead_saved")
def record_lead_changes(sender, instance, created, raw=False,
update_fields=None, **kwargs):
    if raw or not audit_is_enabled():          # raw: loaddata — don't react
to fixtures
        return
    user = current_user()
    if created:                                                    #
11, 13
        log_activity(instance, "created", actor=user)
        LeadStatusChange.objects.create(lead=instance,
to_status=instance.status, changed_by=user)
    else:
```

```
        changes = instance.changed_fields(only=update_fields)            #
update_fields: only what was saved
        if changes:
            log_activity(instance, "updated", as_text(changes), actor=user)
            if "status" in changes:
                old, new = changes["status"]
                LeadStatusChange.objects.create(lead=instance,
from_status=old or "", to_status=new, changed_by=user)
    instance.reset_tracking()                  # next save diffs against
what's in the DB now


@receiver(m2m_changed, sender=Lead.tags.through,
dispatch_uid="lead_tags_changed")
def record_tag_changes(sender, instance, action, reverse, pk_set, **kwargs):
# 15
    if reverse or action not in {"post_add", "post_remove"} or not pk_set or
not audit_is_enabled():
        return                                 # reverse=True means
tag.leads.add(...) — skipped here
    names = sorted(Tag.objects.filter(pk__in=pk_set).values_list("name",
flat=True))
    log_activity(instance, "tagged" if action == "post_add" else "untagged",
{"tags": names}, current_user())


@receiver(post_delete, sender=Lead, dispatch_uid="lead_deleted")
def record_lead_delete(sender, instance, **kwargs):                      # 16
    if audit_is_enabled():
        log_activity(instance, "deleted", actor=current_user())


@receiver(post_save, sender=CallLog, dispatch_uid="call_saved")
def update_last_outcome(sender, instance, created, raw=False, **kwargs):  #
17
    if created and not raw:
        # update(): no save(), no signals, no recursion

Lead.objects.filter(pk=instance.lead_id).update(last_call_outcome=instance.ou
tcome)
```

```
# leads/apps.py — 29
class LeadsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "leads"

    def ready(self):
        from . import signals  # noqa: F401  — importing registers the
@receiver functions
```

Remove `AuditMixin` from the lead views: the signal now logs every save, including admin and shell edits, and two rows per edit would be noise (11).

18–19.

```
# clients/signals.py
from django.db import transaction
from django.dispatch import Signal, receiver

from core.audit import log_activity

lead_converted = Signal()          # sent with: lead, client, actor


@receiver(lead_converted, dispatch_uid="log_conversion")
def log_conversion(sender, lead, client, actor=None, **kwargs):
    log_activity(lead, "converted", {"client": client.account_number}, actor)


@receiver(lead_converted, dispatch_uid="notify_broker")
def notify_broker(sender, lead, client, **kwargs):
    if client.broker_id:
        # 19: anything that leaves the database waits for COMMIT
        transaction.on_commit(lambda: print(f"Notify {client.broker}:
{client.name} converted"))   # Day 23: email


# clients/services.py — convert_lead_to_client(), at the end
from core.context import current_user
from .signals import lead_converted
```

```
actor = current_user()
transaction.on_commit(lambda: lead_converted.send_robust(sender=Client,
lead=lead, client=client, actor=actor))
```

C. Nexora CRM (21–30)

21–22, 29.

```
# core/signals.py — one receiver pair for several models
from django.db.models.signals import post_delete, post_save

from core.audit import as_text, log_activity
from core.context import audit_is_enabled, current_user


def audit_save(sender, instance, created, raw=False, update_fields=None,
**kwargs):
    if raw or not audit_is_enabled():
        return
    if created:
        log_activity(instance, "created", actor=current_user())
    else:
        tracking = hasattr(instance, "changed_fields")
        changes = instance.changed_fields(only=update_fields) if tracking
else {}
        if tracking and not changes:
            return                                          # saved, but nothing
we track moved
        log_activity(instance, "updated", as_text(changes),
actor=current_user())   # 22: field diffs
    if hasattr(instance, "reset_tracking"):
        instance.reset_tracking()


def audit_delete(sender, instance, **kwargs):
    if audit_is_enabled():
        log_activity(instance, "deleted", actor=current_user())


def connect_audit():
    from clients.models import Client, Deposit, TradingAccount
    for model in (Client, TradingAccount, Deposit):
```

```
        label = model._meta.label_lower
        # dispatch_uid (29): connecting twice — e.g. ready() running again —
never adds a second receiver
        post_save.connect(audit_save, sender=model,
dispatch_uid=f"audit_save:{label}")
        post_delete.connect(audit_delete, sender=model,
dispatch_uid=f"audit_delete:{label}")


# core/apps.py
class CoreConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "core"

    def ready(self):
        from .signals import connect_audit
        connect_audit()


# clients/models.py
class Client(TrackChangesMixin, models.Model):
    tracked_fields = ("kyc_status", "account_type", "broker", "is_active",
"email")
    ...
```

`refresh_client_kyc()` updates `kyc_status` with `queryset.update()`, so it never shows in this trail; switch it to `client.save(update_fields=["kyc_status"])` if you want KYC decisions audited (36 explains why).

23.

```
# leads/timeline.py
from django.contrib.contenttypes.models import ContentType
from core.models import ActivityLog
from .models import Lead


def lead_timeline(lead, limit=30):
    ct = ContentType.objects.get_for_model(Lead)
    events = [
        *[{"at": a.created_at, "kind": "activity", "text": f"{a.actor or
'system'} {a.verb}", "detail": a.changes}
          for a in ActivityLog.objects.filter(content_type=ct,
```

```
object_id=str(lead.pk)).select_related("actor")[:limit]],
        *[{"at": s.changed_at, "kind": "status", "text": f"{s.from_status or
'—'} → {s.to_status}", "detail": s.changed_by}
          for s in lead.status_changes.select_related("changed_by")[:limit]],
        *[{"at": c.started_at, "kind": "call", "text": f"
{c.get_outcome_display()} call ({c.duration_seconds}s)", "detail": c.agent}
          for c in lead.calls.select_related("agent")[:limit]],
    ]
    return sorted(events, key=lambda e: e["at"], reverse=True)[:limit]

# LeadDetailView.get_context_data
context["timeline"] = lead_timeline(lead)
```

```
<!-- leads/lead_detail.html -->
<h2>Timeline</h2>
<ul class="timeline">
  {% for e in timeline %}
    <li class="timeline-{{ e.kind }}">{{ e.at|date:"d M H:i" }} · {{ e.text
}}{% if e.detail %} · {{ e.detail }}{% endif %}</li>
  {% empty %}<li>—</li>{% endfor %}
</ul>
```

24–25.

```
# clients/models.py — 24
from decimal import Decimal
from django.db.models import DecimalField, OuterRef, Q, Subquery, Sum, Value
from django.db.models.functions import Coalesce


class ClientQuerySet(models.QuerySet):
    def with_total_deposits(self):
        per_client =
(Deposit.objects.filter(account__client=OuterRef("pk")).order_by()

.values("account__client").annotate(t=Sum("amount")).values("t"))
        return self.annotate(total_deposits=Coalesce(
            Subquery(per_client), Value(Decimal("0")),
            output_field=DecimalField(max_digits=14, decimal_places=2),
        ))

    def kyc_pending(self):
        return self.filter(kyc_status=self.model.KYCStatus.PENDING)
```

```
    def visible_to(self, user):
        from leads.permissions import sees_everything
        if not user.is_authenticated:
            return self.none()
        if sees_everything(user):
            return self
        if user.role == "team_lead":
            return self.filter(Q(lead__assigned_to=user) |
Q(lead__assigned_to__team_lead=user))
        return self.filter(lead__assigned_to=user)


class Client(TrackChangesMixin, models.Model):
    ...
    objects = ClientQuerySet.as_manager()

# usage:
Client.objects.visible_to(request.user).with_total_deposits().kyc_pending()
# visible_clients() and deposits_total() become one-line wrappers or go away
```

```
# leads/models.py — 25
class CallLogQuerySet(models.QuerySet):
    def today(self):
        return self.filter(started_at__date=timezone.localdate())

    def answered(self):
        return self.filter(outcome=self.model.Outcome.ANSWERED)

    def for_agent(self, user):
        return self.filter(agent=user)

    def visible_to(self, user):
        return self.filter(lead__in=Lead.objects.visible_to(user))


class CallLog(models.Model):
    ...
    objects = CallLogQuerySet.as_manager()

# DashboardView: my_calls_today =
CallLog.objects.for_agent(user).today().count()
```

26–28.

```
# clients/signals.py (add) — 26, 27
from decimal import Decimal
from django.db.models import F
from django.db.models.signals import post_delete, post_save

from core.models import Alert
from .models import Deposit, TradingAccount

LARGE_DEPOSIT = Decimal("50000")


@receiver(post_save, sender=Deposit, dispatch_uid="deposit_balance_add")
def add_to_balance(sender, instance, created, raw=False, **kwargs):
    if created and not raw:

TradingAccount.objects.filter(pk=instance.account_id).update(balance=F("balan
ce") + instance.amount)


@receiver(post_delete, sender=Deposit, dispatch_uid="deposit_balance_remove")
def remove_from_balance(sender, instance, **kwargs):

TradingAccount.objects.filter(pk=instance.account_id).update(balance=F("balan
ce") - instance.amount)


@receiver(post_save, sender=Deposit, dispatch_uid="large_deposit_alert")
def large_deposit_alert(sender, instance, created, raw=False, **kwargs):
    if not created or raw or instance.amount < LARGE_DEPOSIT:
        return
    account = instance.account
    crm_client = account.client
    transaction.on_commit(lambda: Alert.objects.create(
        level=Alert.Level.WARNING,
        title=f"Large deposit: {instance.amount:,.2f} {account.currency}",
        body=f"{crm_client.name} · account {account.number} ·
{instance.get_method_display()}",
        url=crm_client.get_absolute_url(),
        for_role="manager",
    ))
```

Delete the `F("balance")` update from `DepositCreateView.form_valid` (Day 14 #29), or every deposit counts twice. Editing an existing deposit's amount isn't covered — deposits should be immutable anyway (reverse and re-enter). One statement brings old balances in line:

```
# python manage.py shell — rebuild every balance from its deposits
from decimal import Decimal
from django.db.models import OuterRef, Subquery, Sum, Value
from django.db.models.functions import Coalesce
from clients.models import Deposit, TradingAccount

totals =
Deposit.objects.filter(account=OuterRef("pk")).order_by().values("account").a
nnotate(t=Sum("amount")).values("t")
TradingAccount.objects.update(balance=Coalesce(Subquery(totals),
Value(Decimal("0"))))
```

```
# core/models.py — 27
from django.db.models import Q


class AlertQuerySet(models.QuerySet):
    def for_user(self, user):
        roles = {user.role}
        if user.is_superuser or user.role == "admin":
            roles.add("manager")
        return self.filter(Q(for_role="") | Q(for_role__in=roles))

    def unread_for(self, user):
        return self.for_user(user).exclude(read_by=user)


class Alert(models.Model):
    class Level(models.TextChoices):
        INFO = "info"
        WARNING = "warning"
        CRITICAL = "critical"

    level = models.CharField(max_length=10, choices=Level.choices,
default=Level.INFO)
    title = models.CharField(max_length=120)
    body = models.TextField(blank=True)
```

```
    url = models.CharField(max_length=300, blank=True)
    for_role = models.CharField(max_length=20, blank=True)      # "" =
everyone
    read_by = models.ManyToManyField(settings.AUTH_USER_MODEL, blank=True,
related_name="read_alerts")
    created_at = models.DateTimeField(auto_now_add=True)

    objects = AlertQuerySet.as_manager()

    class Meta:
        ordering = ["-created_at"]
```

```
# core/views.py — 28
class AlertReadView(View):
    http_method_names = ["post"]

    def post(self, request, pk):
        alert = get_object_or_404(Alert.objects.for_user(request.user),
pk=pk)
        alert.read_by.add(request.user)
        return redirect(safe_next(request, default=reverse("dashboard")))

# DashboardView.get_context_data
context["alerts"] = Alert.objects.unread_for(self.request.user)[:5]

# core/urls.py
path("alerts/<int:pk>/read/", views.AlertReadView.as_view(),
name="alert_read"),
```

```
<!-- core/dashboard.html — 28 -->
{% for alert in alerts %}
  <div class="message {{ alert.level }}">
    <strong>{{ alert.title }}</strong> · {{ alert.body }}
    {% if alert.url %}<a href="{{ alert.url }}">Open</a>{% endif %}
    <form method="post" action="{% url 'alert_read' alert.pk %}"
class="inline">{% csrf_token %}
      <input type="hidden" name="next" value="{{ request.path }}"><button
type="submit">Mark read</button>
    </form>
  </div>
{% endfor %}
```

30.

```
# reports/queries.py
def source_performance():
    return (Lead.objects.active().values("source")
            .annotate(leads_n=Count("id"), qualified_n=Count("id",
filter=Q(status="qualified")),
                      avg_deposit=Avg("expected_deposit"))
            .annotate(conversion=pct("qualified_n", "leads_n"))
            .order_by("-conversion"))


def sop_first_call_breaches(hours=24):
    window = timedelta(hours=hours)
    called_in_time = CallLog.objects.filter(lead=OuterRef("pk"),
started_at__lte=OuterRef("deadline"))
    return (Lead.objects.active()
            .filter(created_at__lte=timezone.now() - window)
            .annotate(deadline=ExpressionWrapper(F("created_at") + window,
output_field=DateTimeField()))
            .filter(~Exists(called_in_time))
            .select_related("assigned_to").order_by("created_at"))
```

D. Challenge (31–36)

31. `use_in_migrations = True` on `LeadManager` gives data migrations the custom manager. But migrations use *historical* models built from migration files: they have fields, not your nested `Status` class or methods. `self.model.Status` raises `AttributeError` there — use plain strings in methods that migrations call.

32.

```
# core/models.py
class SoftDeleteQuerySet(models.QuerySet):
    def delete(self):                          # bulk delete → archive
        return self.update(deleted_at=timezone.now())

    def hard_delete(self):
        return super().delete()

    def dead(self):
        return self.filter(deleted_at__isnull=False)
```

```
class AliveManager(models.Manager.from_queryset(SoftDeleteQuerySet)):
    def get_queryset(self):
        return super().get_queryset().filter(deleted_at__isnull=True)


class SoftDeleteModel(models.Model):
    deleted_at = models.DateTimeField(null=True, blank=True, editable=False)

    objects = AliveManager()                    # default: hides deleted rows
    all_objects = SoftDeleteQuerySet.as_manager()

    class Meta:
        abstract = True

    def delete(self, using=None, keep_parents=False):
        self.deleted_at = timezone.now()
        self.save(update_fields=["deleted_at"])

    def hard_delete(self):
        return super().delete()

    def restore(self):
        self.deleted_at = None
        self.save(update_fields=["deleted_at"])


# leads/models.py
class ImportBatch(SoftDeleteModel):            # deleting an import hides it,
the audit trail stays
    ...
```

In the admin, override `ImportBatchAdmin.get_queryset()` to return `ImportBatch.all_objects.all()` so deleted batches stay reachable.

33–35.

```
from django.contrib.auth import get_user_model
from django.db.models import Prefetch

agents = get_user_model().objects.filter(role="agent").prefetch_related(
# 33
```

```
    Prefetch("leads", queryset=Lead.objects.open().active(),
to_attr="open_leads"),
)
for a in agents:
    print(a.username, len(a.open_leads))


results = lead_converted.send_robust(sender=Client, lead=lead,
client=crm_client)   # 34
for receiver, outcome in results:
    if isinstance(outcome, Exception):         # send() would have raised and
skipped the rest
        print(f"{receiver.__name__} failed: {outcome!r}")


# 35 — the trap: save() inside post_save fires post_save again →
RecursionError
@receiver(post_save, sender=CallLog)
def bad(sender, instance, **kwargs):
    instance.notes = instance.notes.strip()
    instance.save()                            # ← loops forever

# the fix: write with update() (no signals) — or do the cleanup in
clean()/save() before saving
CallLog.objects.filter(pk=instance.pk).update(notes=instance.notes.strip())
```

36.

```
# leads/views.py — LeadBulkActionView.form_valid, "contacted" branch
from .models import LeadStatusChange

changing = list(qs.exclude(status=Lead.Status.CONTACTED).values_list("pk",
"status"))   # read BEFORE update()
n = qs.update(status=Lead.Status.CONTACTED, updated_at=now)       # update():
no save(), no signals
LeadStatusChange.objects.bulk_create([
    LeadStatusChange(lead_id=pk, from_status=old,
to_status=Lead.Status.CONTACTED, changed_by=self.request.user)
    for pk, old in changing
])
```

`update()`, `bulk_create()` and `bulk_update()` never call `save()`, so `pre_save` / `post_save` don't fire and no history or audit rows appear on their own. Record them explicitly where you use those calls. (`queryset.delete()` is different: it still sends `pre_delete` / `post_delete` for every row.)

Tests (37–40)

```
# leads/tests/test_signals.py
from decimal import Decimal

from django.contrib.contenttypes.models import ContentType
from django.test import TestCase
from django.urls import reverse

from core.context import audit_disabled
from core.factories import AgentFactory, DepositFactory, LeadFactory,
TradingAccountFactory
from core.models import ActivityLog
from leads.models import Lead


class ManagerAndSignalTests(TestCase):
    def test_queryset_chains(self):                                    # 37
        agent = AgentFactory()
        LeadFactory(assigned_to=agent)
        LeadFactory(assigned_to=agent, qualified=True)
        LeadFactory(assigned_to=agent, is_active=False)

self.assertEqual(Lead.objects.visible_to(agent).active().open().count(), 1)
        self.assertEqual(agent.leads.open().count(), 2)          # the
archived one is still "open"

self.assertEqual(Lead.active_objects.filter(assigned_to=agent).count(), 2)

    def test_status_history_records_user(self):                        # 38
(+ 29: exactly one audit row)
        agent = AgentFactory()
        lead = LeadFactory(assigned_to=agent)
        self.client.force_login(agent)
        self.client.post(reverse("leads:status_edit", args=[lead.pk]),
{"status": "contacted", "notes": ""})
        latest = lead.status_changes.first()
```

```
        self.assertEqual((latest.from_status, latest.to_status,
latest.changed_by), ("new", "contacted", agent))
        self.assertEqual(lead.status_changes.count(), 2)          # created +
this change
        ct = ContentType.objects.get_for_model(Lead)
        self.assertEqual(ActivityLog.objects.filter(content_type=ct,
object_id=str(lead.pk), verb="updated").count(), 1)

    def test_balance_follows_deposits(self):                              # 39
        account = TradingAccountFactory()
        first = DepositFactory(account=account, amount=Decimal("1500.00"))
        DepositFactory(account=account, amount=Decimal("500.00"))
        account.refresh_from_db()
        self.assertEqual(account.balance, Decimal("2000.00"))
        first.delete()
        account.refresh_from_db()
        self.assertEqual(account.balance, Decimal("500.00"))

    def test_audit_disabled(self):                                        # 40
        with audit_disabled():
            lead = LeadFactory()
        ct = ContentType.objects.get_for_model(Lead)
        self.assertFalse(ActivityLog.objects.filter(content_type=ct,
object_id=str(lead.pk)).exists())
        self.assertFalse(lead.status_changes.exists())
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py test
git add -A && git commit -m "day 20"
```

Day 21 — Middleware, context processors, template tags

Goal: the three hooks that touch every request or every page. **Middleware** wraps each request (ids, timing, security headers, audit of who opened which client file). **Context processors** add data to every template (counters, navigation). **Template tags and filters** keep templates short (badges, money, `{% ifrole %}`).

A. Drills (1–10) — middleware

- [ ] **1.** Function-style middleware adding `X-Response-Time`.
- [ ] **2.** Class-style middleware: `__init__(get_response)` + `__call__(request)`.
- [ ] **3.** `process_view` records the view's name on the request; `process_exception` returns JSON 500 for API URLs (not for 404/403).
- [ ] **4.** `process_template_response` adds a `build` value to every `TemplateResponse`; see why function views using `render()` don't get it.
- [ ] **5.** Ordering experiment: two tracer middlewares print → A → B → view ← B ← A.
- [ ] **6.** `RequestIDMiddleware`: reuse a valid incoming `X-Request-ID` or create one; echo it back; expose it to logging.
- [ ] **7.** Finalise Day 20's `CurrentUserMiddleware` position in `MIDDLEWARE` and say why it must follow `AuthenticationMiddleware`.
- [ ] **8.** `LastSeenMiddleware`: update `user.last_seen` at most every 5 minutes.
- [ ] **9.** `MaintenanceModeMiddleware`: `MAINTENANCE_MODE = True` → 503 page for everyone except staff (login and health stay open).
- [ ] **10.** A middleware that works under both WSGI and ASGI (`sync_and_async_middleware`).

B. Core (11–20) — context processors and tags

- [ ] **11.** `crm_globals`: company name, environment label, build — available in every template.
- [ ] **12.** `crm_counters`: my open leads, follow-ups due, unread alerts — lazy, so pages that don't show them don't run the queries.

- [ ] **13.** `navigation`: the sidebar defined in Python (label, URL, permission/role, active state).
- [ ] **14.** Saved views (Day 17) in the sidebar via a processor.
- [ ] **15.** Measure: a page that never shows the counters runs zero counter queries.
- [ ] **16.** `{% status_badge lead.status lead.get_status_display %}` simple tag.
- [ ] **17.** `{% kpi_card "New leads" today yesterday %}` inclusion tag with a trend %.
- [ ] **18.** Filters `|aed` (AED 12,500.00) and `|percent`.
- [ ] **19.** Filter `|ago`: 5m, 2h, 3d.
- [ ] **20.** `{% active_if "leads:list" "leads:mine" %}` returning `active` for the current page.

C. Nexora CRM (21–30)

- [ ] **21.** `LeadAccessAuditMiddleware`: every successful open of a lead or client page writes an `AccessLog` row (who, what, IP, request id) — a compliance trail for personal data.
- [ ] **22.** Export rate limit: 10 exports per user per hour → 429.
- [ ] **23.** Security headers: Content-Security-Policy, Referrer-Policy, Permissions-Policy.
- [ ] **24.** Per-user time zone: `CustomUser.timezone`, activated per request.
- [ ] **25.** Business-hours banner (Mon–Fri 09:00–18:00 Dubai) from a context processor.
- [ ] **26.** Filters `|mask_email` and `|mask_account`.
- [ ] **27.** `{% perm_link "leads.export_lead" "leads:export" "Export" %}` — renders only with the permission.
- [ ] **28.** Block tag `{% ifrole "manager" "admin" %}…{% else %}…{% endifrole %}` with its own parser and `Node`.
- [ ] **29.** Put the CRM tag library in `builtins` so templates don't need `{% load %}`.
- [ ] **30.** Friendly 500 page that shows the request id (and doesn't depend on the database).

D. Challenge (31–40)

- [ ] **31.** One async view using the async ORM (`acount()`).
- [ ] **32.** Unit-test a middleware with `RequestFactory` and a fake `get_response`.
- [ ] **33.** `QueryCountWarningMiddleware` (DEBUG only): `X-Query-Count` header, warning above 50.

- [ ] **34.** `SlowRequestLogMiddleware`: log requests over 500 ms with view name and request id.
- [ ] **35.** Test filters and tags with `Template(...).render(Context(...))`.
- [ ] **36.** Test a context processor directly.
- [ ] **37.** Test: every response carries `X-Request-ID`; a valid incoming one is reused.
- [ ] **38.** Test: maintenance mode gives 503 to an agent and 200 to staff.
- [ ] **39.** Test: `{% ifrole %}` for manager vs agent.
- [ ] **40.** Test: opening a lead creates exactly one `AccessLog` row.

Final `MIDDLEWARE` order is in the solutions.

Solutions

A middleware is a callable that takes `get_response` once at startup and returns a callable that takes `request` and returns a response. Code before `get_response(request)` runs on the way in, code after it on the way out. Optional hooks: `process_view` (just before the view), `process_exception` (when the view raises), `process_template_response` (before a `TemplateResponse` is rendered).

Middleware (1–10, 21–24, 33–34)

```
# core/middleware.py (added to the Day 15/20 classes)
import logging
import re
import time
import uuid
import zoneinfo
from contextvars import ContextVar

from asgiref.sync import iscoroutinefunction, markcoroutinefunction
from django.conf import settings
from django.core.cache import cache
from django.core.exceptions import PermissionDenied
from django.db import connection
from django.http import Http404, HttpResponse, JsonResponse
from django.shortcuts import render
from django.test.utils import CaptureQueriesContext
from django.utils import timezone
from django.utils.decorators import sync_and_async_middleware
```

```
logger = logging.getLogger("crm.requests")
_request_id = ContextVar("request_id", default="-")


def url_key(request):
    match = getattr(request, "resolver_match", None)
    if match is None:
        return ""
    return f"{match.namespace}:{match.url_name}" if match.namespace else
(match.url_name or "")


# ---- 1: function style ------------------------------------------------
---
def timing_middleware(get_response):
    def middleware(request):
        start = time.perf_counter()
        response = get_response(request)
        response["X-Response-Time"] = f"{(time.perf_counter() - start) *
1000:.1f}ms"
        return response
    return middleware


# ---- 2, 6: class style + request id -----------------------------------
---
class RequestIDMiddleware:
    header = "X-Request-ID"
    valid = re.compile(r"^[A-Za-z0-9-]{8,64}$")

    def __init__(self, get_response):          # once, at startup
        self.get_response = get_response

    def __call__(self, request):               # once per request
        incoming = request.headers.get(self.header, "")
        request.id = incoming if self.valid.match(incoming) else
uuid.uuid4().hex   # reuse a proxy's id if sane
        token = _request_id.set(request.id)
        try:
            response = self.get_response(request)
        finally:
            _request_id.reset(token)
```

```
        response[self.header] = request.id
        return response


class RequestIDLogFilter(logging.Filter):      # 6: adds %(request_id)s to
every log line (Day 22 logging config)
    def filter(self, record):
        record.request_id = _request_id.get()
        return True


# ---- 3: process_view / process_exception ----------------------------------
-
class ViewNameMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        return self.get_response(request)

    def process_view(self, request, view_func, view_args, view_kwargs):
        target = getattr(view_func, "view_class", view_func)        # CBVs:
as_view() sets view_class
        request.view_name = f"{target.__module__}.{target.__name__}"
        return None                             # None = carry on to the view

    def process_exception(self, request, exception):
        if isinstance(exception, (Http404, PermissionDenied)):
            return None                         # let Django build the normal
404/403
        if "/api/" in request.path:
            logger.exception("API error on %s", request.path)
            return JsonResponse({"error": "server error", "request_id":
getattr(request, "id", None)}, status=500)
        return None


# ---- 4: process_template_response -----------------------------------------
-
class BuildInfoMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response
```

```
    def __call__(self, request):
        return self.get_response(request)

    def process_template_response(self, request, response):
        # only TemplateResponse (what CBVs return); render() builds a
finished HttpResponse → hook never runs
        response.context_data = {**(response.context_data or {}), "build":
getattr(settings, "CRM_BUILD", "local")}
        return response


# ---- 5: ordering experiment ----------------------------------------------
---
def make_tracer(name):
    def factory(get_response):
        def middleware(request):
            print(f"→ {name}")
            response = get_response(request)
            print(f"← {name}")
            return response
        return middleware
    return factory


tracer_a = make_tracer("A")
tracer_b = make_tracer("B")
# MIDDLEWARE = ["core.middleware.tracer_a", "core.middleware.tracer_b", ...]
→ → A → B (view) ← B ← A


# ---- 8 -------------------------------------------------------------------
-----
class LastSeenMiddleware:
    INTERVAL = 300

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)
        user = getattr(request, "user", None)
        if user is not None and user.is_authenticated:
            now = time.time()
```

```
            if now - request.session.get("last_seen_ts", 0) > self.INTERVAL:
# one UPDATE per 5 min, not per request

type(user).objects.filter(pk=user.pk).update(last_seen=timezone.now())
                request.session["last_seen_ts"] = now
        return response


# ---- 9 -----------------------------------------------------------------
-----
class MaintenanceModeMiddleware:
    OPEN_PATHS = ("/accounts/login/", "/admin/", "/health/", "/static/")

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if getattr(settings, "MAINTENANCE_MODE", False):
            user = getattr(request, "user", None)
            is_staff = bool(user and user.is_authenticated and user.is_staff)
            if not is_staff and not request.path.startswith(self.OPEN_PATHS):
                return render(request, "503.html", status=503)      # short-
circuit: the view never runs
        return self.get_response(request)


# ---- 10 ----------------------------------------------------------------
-----
@sync_and_async_middleware
def async_timing_middleware(get_response):
    if iscoroutinefunction(get_response):
        async def middleware(request):
            start = time.perf_counter()
            response = await get_response(request)
            response["X-Response-Time"] = f"{(time.perf_counter() - start) *
1000:.1f}ms"
            return response
        markcoroutinefunction(middleware)
    else:
        def middleware(request):
            start = time.perf_counter()
            response = get_response(request)
            response["X-Response-Time"] = f"{(time.perf_counter() - start) *
```

```
1000:.1f}ms"
            return response
    return middleware


# ---- 21 ----------------------------------------------------------------
-----
class LeadAccessAuditMiddleware:
    AUDITED = {"leads:detail": "lead_id", "clients:detail": "ref",
"clients:kyc_download": "pk"}

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)
        key = url_key(request)
        user = getattr(request, "user", None)
        if key in self.AUDITED and response.status_code == 200 and user and
user.is_authenticated:
            from core.models import AccessLog
            AccessLog.objects.create(
                user=user, view_name=key,

object_key=str(request.resolver_match.kwargs.get(self.AUDITED[key], "")),
                ip=request.META.get("REMOTE_ADDR"),
request_id=getattr(request, "id", ""),
            )
        return response


# ---- 22 ----------------------------------------------------------------
-----
class ExportRateLimitMiddleware:
    LIMITED = {"leads:export", "leads:export_stream", "leads:export_xlsx"}
    LIMIT, WINDOW = 10, 3600

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        return self.get_response(request)
```

```
    def process_view(self, request, view_func, view_args, view_kwargs):
        if url_key(request) not in self.LIMITED or not
request.user.is_authenticated:
            return None
        key = f"export-rate:{request.user.pk}"
        cache.add(key, 0, timeout=self.WINDOW)       # creates the counter
only if it's missing
        if cache.incr(key) > self.LIMIT:
            return HttpResponse("Export limit reached (10 per hour).",
status=429)
        return None


# ---- 23 ----------------------------------------------------------------
-----
class SecurityHeadersMiddleware:
    CSP = ("default-src 'self'; script-src 'self' https://cdn.jsdelivr.net; "
           "style-src 'self' 'unsafe-inline'; img-src 'self' data:; frame-
ancestors 'none'")

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)
        response.setdefault("Content-Security-Policy", self.CSP)   #
setdefault: a view can still override
        response.setdefault("Referrer-Policy", "same-origin")
        response.setdefault("Permissions-Policy", "camera=(), microphone=(),
geolocation=()")
        return response


# ---- 24 ----------------------------------------------------------------
-----
class UserTimezoneMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        user = getattr(request, "user", None)
        tzname = getattr(user, "timezone", "") if user is not None and
user.is_authenticated else ""
```

```
        if tzname:
            timezone.activate(zoneinfo.ZoneInfo(tzname))
        else:
            timezone.deactivate()               # back to TIME_ZONE
(Asia/Dubai)
        return self.get_response(request)


# ---- 33 ----------------------------------------------------------------
-----
class QueryCountWarningMiddleware:
    THRESHOLD = 50

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if not settings.DEBUG:
            return self.get_response(request)
        with CaptureQueriesContext(connection) as ctx:
            response = self.get_response(request)
        count = len(ctx.captured_queries)
        response["X-Query-Count"] = str(count)
        if count > self.THRESHOLD:
            logger.warning("%s ran %s queries", request.path, count)
        return response


# ---- 34 ----------------------------------------------------------------
-----
class SlowRequestLogMiddleware:
    THRESHOLD_MS = 500

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.perf_counter()
        response = self.get_response(request)
        ms = (time.perf_counter() - start) * 1000
        if ms > self.THRESHOLD_MS:
```

```
            logger.warning("slow %s %s %.0fms view=%s", request.method,
request.path, ms,
                           getattr(request, "view_name", "?"))
        return response
```

Models for 8, 21, 24.

```
# accounts/models.py — CustomUser
TIMEZONES = [("Asia/Dubai", "Dubai"), ("Asia/Kolkata", "India"),
("Europe/London", "London")]
last_seen = models.DateTimeField(null=True, blank=True)
timezone = models.CharField(max_length=40, choices=TIMEZONES,
default="Asia/Dubai")   # add to ProfileForm


# core/models.py — 21
class AccessLog(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name="access_logs")
    view_name = models.CharField(max_length=60)
    object_key = models.CharField(max_length=64)
    ip = models.GenericIPAddressField(null=True, blank=True)
    request_id = models.CharField(max_length=64, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["view_name", "object_key"],
name="access_target_idx")]
```

```
<!-- templates/503.html — 9 -->
{% extends "base.html" %}
{% block content %}<h1>Back soon</h1><p>Nexora CRM is being updated. Please
try again in a few minutes.</p>{% endblock content %}
```

Final `MIDDLEWARE` (7 and the order rules).

```
# django_project/settings.py
MAINTENANCE_MODE = False
CRM_BUILD = "local"

MIDDLEWARE = [
    "core.middleware.RequestIDMiddleware",              # first: every
```

```
response and log line gets an id
    "core.middleware.SlowRequestLogMiddleware",         # outermost timer →
measures everything below
    "django.middleware.security.SecurityMiddleware",
    "core.middleware.SecurityHeadersMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",   #
request.user exists from here down
    "core.middleware.CurrentUserMiddleware",            # 7: needs
request.user
    "core.middleware.UserTimezoneMiddleware",
    "core.middleware.MaintenanceModeMiddleware",        # needs request.user
(staff bypass)
    "core.middleware.CRMLoginRequiredMiddleware",
    "core.middleware.ForcePasswordChangeMiddleware",
    "core.middleware.LastSeenMiddleware",               # needs session +
user
    "core.middleware.ExportRateLimitMiddleware",
    "core.middleware.LeadAccessAuditMiddleware",
    "core.middleware.ViewNameMiddleware",
    "core.middleware.QueryCountWarningMiddleware",      # DEBUG only,
innermost → counts just the view
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

On the way in, middleware runs top to bottom; on the way out, bottom to top. Anything that reads `request.user` must sit below `AuthenticationMiddleware` (7). `process_view` hooks all run after every `__call__` has passed the request down, so the maintenance check (in `__call__`) wins over the login check (in `process_view`).

Context processors (11–15, 25)

A context processor is a function `(request) → dict`; Django merges its dict into the context of every template rendered with a request. It runs on every such render, so keep it cheap or lazy.

```
# core/context_processors.py
from django.conf import settings
from django.urls import reverse
from django.utils import timezone
from django.utils.functional import SimpleLazyObject


def crm_globals(request):                                                  # 11
    return {
        "COMPANY_NAME": settings.CRM_COMPANY_NAME,
        "ENVIRONMENT": getattr(settings, "CRM_ENVIRONMENT", "dev"),
        "BUILD": getattr(settings, "CRM_BUILD", "local"),
    }


def crm_counters(request):                                                 # 12,
15
    user = getattr(request, "user", None)
    if user is None or not user.is_authenticated:
        return {}

    def compute():
        from core.models import Alert
        from leads.models import Lead
        mine = Lead.objects.active().filter(assigned_to=user)
        return {
            "my_open": mine.open().count(),
            "follow_ups": mine.follow_up_due().count(),
            "unread_alerts": Alert.objects.unread_for(user).count(),
        }

    # lazy: the 3 COUNT queries run only if a template actually reads
counters.something
    return {"counters": SimpleLazyObject(compute)}


NAV = [                                                                    # 13
    {"label": "Dashboard", "url": "dashboard", "match": ["dashboard"]},
    {"label": "My leads", "url": "leads:mine", "match": ["leads:mine"]},
    {"label": "All leads", "url": "leads:list", "match": ["leads:list",
"leads:detail", "leads:edit"]},
    {"label": "Calls", "url": "leads:calls", "match": ["leads:calls"]},
    {"label": "Clients", "url": "clients:list", "match": ["clients:list",
```

```
"clients:detail"]},
    {"label": "Brokers", "url": "brokers:list", "match": ["brokers:list",
"brokers:detail"], "perm": "brokers.view_broker"},
    {"label": "Reports", "url": "reports:index", "match": ["reports:index"],
"roles": {"team_lead", "manager", "admin"}},
    {"label": "Team", "url": "accounts:team", "match": ["accounts:team"],
"perm": "leads.view_all_leads"},
]


def navigation(request):
    from core.middleware import url_key
    user = getattr(request, "user", None)
    if user is None or not user.is_authenticated:
        return {"nav_items": []}
    current = url_key(request)
    items = []
    for item in NAV:
        if "perm" in item and not user.has_perm(item["perm"]):
            continue
        if "roles" in item and not (user.is_superuser or user.role in
item["roles"]):
            continue
        items.append({"label": item["label"], "href": reverse(item["url"]),
"active": current in item["match"]})
    return {"nav_items": items}


def saved_views(request):                                          # 14
    user = getattr(request, "user", None)
    if user is None or not user.is_authenticated:
        return {}
    return {"sidebar_saved_views": SimpleLazyObject(lambda:
list(user.saved_views.all()[:8]))}


def business_hours(request):                                       # 25:
UAE weekend is Sat-Sun
    now = timezone.localtime()
    return {"outside_business_hours": not (now.weekday() < 5 and 9 <=
now.hour < 18)}
```

```
# django_project/settings.py — 11–14, 25, 29
TEMPLATES = [{
    "BACKEND": "django.template.backends.django.DjangoTemplates",
    "DIRS": [BASE_DIR / "templates"],
    "APP_DIRS": True,
    "OPTIONS": {
        "context_processors": [
            "django.template.context_processors.request",
            "django.contrib.auth.context_processors.auth",
            "django.contrib.messages.context_processors.messages",
            "core.context_processors.crm_globals",
            "core.context_processors.crm_counters",
            "core.context_processors.navigation",
            "core.context_processors.saved_views",
            "core.context_processors.business_hours",
        ],
        "builtins": [                                    # 29: usable in
every template without {% load %}
            "core.templatetags.crm_ui",
            "leads.templatetags.crm_filters",
        ],
    },
}]
CRM_ENVIRONMENT = "dev"
```

```
<!-- templates/base.html — sidebar now comes from Python (13, 14), plus
counters (12) and the banner (25) -->
<aside class="sidebar">
  <a href="{% url 'home' %}" class="brand">{{ COMPANY_NAME }} CRM <small>{{
ENVIRONMENT }}</small></a>
  {% for item in nav_items %}
    <a href="{{ item.href }}" class="{% if item.active %}active{% endif %}">
{{ item.label }}
      {% if item.label == "My leads" %}<span class="pill">{{ counters.my_open
}}</span>{% endif %}
    </a>
  {% endfor %}
  {% if sidebar_saved_views %}
    <p class="muted">Saved views</p>
    {% for v in sidebar_saved_views %}<a href="{{ v.get_absolute_url }}">{{
v.name }}</a>{% endfor %}
  {% endif %}
</aside>
...
{% if user.is_authenticated and outside_business_hours %}
  <div class="message info">Outside business hours — log callbacks for
tomorrow morning.</div>
{% endif %}
```

15.

```
# python manage.py shell — a template that never reads `counters` runs none
of its queries
from django.db import connection
from django.template.loader import render_to_string
from django.test import RequestFactory
from django.test.utils import CaptureQueriesContext
from accounts.models import CustomUser

request = RequestFactory().get("/")
request.user = CustomUser.objects.filter(role="agent").first()
with CaptureQueriesContext(connection) as ctx:
    render_to_string("robots.txt", request=request)
print(len(ctx.captured_queries))     # 0 — SimpleLazyObject never evaluated
```

Template tags and filters (16–20, 26–28)

```
# core/templatetags/__init__.py   ← empty
# core/templatetags/crm_ui.py
from decimal import Decimal, InvalidOperation

from django import template
from django.urls import reverse
from django.utils import timezone
from django.utils.html import format_html

register = template.Library()


@register.simple_tag                                                  # 16
def status_badge(status, label=None):
    return format_html('<span class="badge badge-{}">{}</span>', status,
                       label or str(status).replace("_", " ").title())


@register.inclusion_tag("partials/_kpi_card.html")                   # 17
def kpi_card(label, value, previous=None):
    trend = None
    if previous:                                        # skip None and 0 (no
division by zero)
        trend = round((value - previous) * 100 / previous, 1)
    return {"label": label, "value": f"{value:,}", "trend": trend}


@register.filter                                                      # 18
def aed(value, decimals=2):
    try:
        amount = Decimal(value)
    except (TypeError, ValueError, InvalidOperation):
        return ""
    return f"AED {amount:,.{int(decimals)}f}"


@register.filter
def percent(value, decimals=1):
    try:
        return f"{float(value):.{int(decimals)}f}%"
    except (TypeError, ValueError):
```

```
        return ""


@register.filter                                                    # 19
def ago(value):
    if not value:
        return ""
    seconds = int((timezone.now() - value).total_seconds())
    for unit, size in (("d", 86400), ("h", 3600), ("m", 60)):
        if seconds >= size:
            return f"{seconds // size}{unit}"
    return "now"


@register.simple_tag(takes_context=True)                            # 20
def active_if(context, *url_names):
    from core.middleware import url_key
    return "active" if url_key(context["request"]) in url_names else ""


@register.filter                                                    # 26
def mask_email(value):
    if not value or "@" not in value:
        return value
    local, domain = value.split("@", 1)
    return f"{local[:1]}{'*' * max(len(local) - 1, 1)}@{domain}"


@register.filter
def mask_account(value):
    value = str(value or "")
    return f"•••{value[-3:]}" if len(value) > 3 else value


@register.simple_tag(takes_context=True)                            # 27
def perm_link(context, perm, url_name, label, *args):
    if not context["request"].user.has_perm(perm):
        return ""
    return format_html('<a href="{}">{}</a>', reverse(url_name, args=args),
label)


# ---- 28: a block tag with its own parser ---------------------------------
```

```
class IfRoleNode(template.Node):
    def __init__(self, roles, nodelist_true, nodelist_false):
        self.roles, self.nodelist_true, self.nodelist_false = roles,
nodelist_true, nodelist_false

    def render(self, context):
        user = context.get("user") or getattr(context.get("request"), "user",
None)
        wanted = {role.resolve(context) for role in self.roles}
        allowed = bool(user) and user.is_authenticated and (user.is_superuser
or getattr(user, "role", None) in wanted)
        return (self.nodelist_true if allowed else
self.nodelist_false).render(context)


@register.tag
def ifrole(parser, token):
    bits = token.split_contents()[1:]                          # drop the tag
name
    if not bits:
        raise template.TemplateSyntaxError("{% ifrole %} needs at least one
role")
    roles = [parser.compile_filter(bit) for bit in bits]      # "manager" or
a variable
    nodelist_true = parser.parse(("else", "endifrole"))        # everything
up to {% else %} / {% endifrole %}
    token = parser.next_token()
    if token.contents == "else":
        nodelist_false = parser.parse(("endifrole",))
        parser.delete_first_token()
    else:
        nodelist_false = template.NodeList()
    return IfRoleNode(roles, nodelist_true, nodelist_false)
```

```
<!-- templates/partials/_kpi_card.html — 17 -->
<div class="card">
  <p class="muted">{{ label }}</p>
  <h3>{{ value }}</h3>
  {% if trend is not None %}
    <p class="{% if trend >= 0 %}up{% else %}down{% endif %}">{% if trend >=
0 %}▲{% else %}▼{% endif %} {{ trend|floatformat:1 }}% vs yesterday</p>
  {% endif %}
</div>

<!-- usage (no {% load %} needed thanks to builtins) -->
{% kpi_card "New leads" new_today new_yesterday %}
{% status_badge lead.status lead.get_status_display %}
<td>{{ deposit.amount|aed }}</td><td>{{ row.conversion|percent }}</td><td>{{
lead.last_call_at|ago }}</td>
<td>{{ client.email|mask_email }} · {{ client.account_number|mask_account }}
</td>
<a class="{% active_if 'leads:list' 'leads:mine' %}" href="{% url
'leads:list' %}">Leads</a>
{% perm_link "leads.export_lead" "leads:export" "Export" %}
{% ifrole "manager" "admin" %}<a href="{% url 'reports:index' %}">Full
reports</a>{% else %}<span>Team view</span>{% endifrole %}
```

```
# core/views.py — DashboardView: values for the KPI card
from datetime import timedelta
yesterday = timezone.localdate() - timedelta(days=1)
context["new_today"] = Lead.objects.active().created_today().count()
context["new_yesterday"] =
Lead.objects.active().filter(created_at__date=yesterday).count()
```

500 page and an async view (30–31)

```
# core/views.py
from django.http import HttpResponseServerError, JsonResponse
from django.template import loader


def server_error(request):                                          # 30
    # no context processors here: if the DB is what broke, the error page
must still render
    html = loader.get_template("500.html").render({"request_id":
```

```
getattr(request, "id", None)})
    return HttpResponseServerError(html)


async def lead_count_async(request):                                    # 31
    user = await request.auser()            # async-safe way to load the user
(Django 5.0+)
    count = await Lead.objects.visible_to(user).active().acount()
    return JsonResponse({"active_leads": count})

# django_project/urls.py
handler500 = "core.views.server_error"

# core/urls.py
path("api/lead-count/", views.lead_count_async, name="lead_count_async"),
```

```
<!-- templates/500.html — standalone on purpose: no base.html, no DB-backed
context -->
<!DOCTYPE html>
<html lang="en"><head><meta charset="utf-8"><title>Something went
wrong</title></head>
<body style="font-family: system-ui, sans-serif; padding: 2rem">
  <h1>Something went wrong</h1>
  <p>The team has been notified. If you contact support, quote this
reference:</p>
  <p><code>{{ request_id|default:"n/a" }}</code></p>
</body></html>
```

Tests (32, 35–40)

```
# core/tests.py
from datetime import timedelta
from decimal import Decimal
from unittest import mock

from django.http import HttpResponse
from django.template import Context, Template
from django.test import RequestFactory, SimpleTestCase, TestCase,
override_settings
from django.urls import reverse
from django.utils import timezone
```

```
from core.context_processors import crm_counters
from core.factories import AgentFactory, LeadFactory, ManagerFactory,
UserFactory
from core.middleware import RequestIDMiddleware
from core.models import AccessLog


class TemplateAndMiddlewareUnitTests(SimpleTestCase):
    def test_request_id_middleware(self):                               # 32 +
37
        mw = RequestIDMiddleware(lambda request: HttpResponse("ok"))   # fake
get_response
        self.assertRegex(mw(RequestFactory().get("/"))["X-Request-ID"],
r"^[0-9a-f]{32}$")
        reused = mw(RequestFactory().get("/", headers={"X-Request-ID": "abc-
12345"}))
        self.assertEqual(reused["X-Request-ID"], "abc-12345")

    def test_filters(self):                                             # 35
        render = lambda src, **ctx: Template(src).render(Context(ctx))
        self.assertEqual(render("{{ v|aed }}", v=Decimal("12500")), "AED
12,500.00")
        self.assertEqual(render("{{ v|percent }}", v=33.333), "33.3%")
        self.assertEqual(render("{{ e|mask_email }}", e="aisha@nexora.test"),
"a****@nexora.test")
        with mock.patch("django.utils.timezone.now",
return_value=timezone.now()):
            self.assertEqual(render("{{ t|ago }}", t=timezone.now() -
timedelta(hours=2)), "2h")

    def test_ifrole(self):                                              # 39
        tpl = Template('{% ifrole "manager" "admin" %}yes{% else %}no{%
endifrole %}')
        self.assertEqual(tpl.render(Context({"user":
ManagerFactory.build()})), "yes")
        self.assertEqual(tpl.render(Context({"user": AgentFactory.build()})),
"no")


class RequestLevelTests(TestCase):
    def test_counters_processor(self):                                  # 36
        agent = AgentFactory()
        LeadFactory.create_batch(2, assigned_to=agent)
```

```
        request = RequestFactory().get("/")
        request.user = agent
        self.assertEqual(crm_counters(request)["counters"]["my_open"], 2)

    def test_every_response_has_request_id(self):                       # 37
        self.assertIn("X-Request-ID", self.client.get(reverse("health")))

    @override_settings(MAINTENANCE_MODE=True)
    def test_maintenance_mode(self):                                    # 38
        self.client.force_login(AgentFactory())
        self.assertEqual(self.client.get(reverse("dashboard")).status_code,
503)
        self.client.force_login(UserFactory(role="manager", is_staff=True))
        self.assertEqual(self.client.get(reverse("dashboard")).status_code,
200)

    def test_lead_open_is_audited(self):                                # 40
        agent = AgentFactory()
        lead = LeadFactory(assigned_to=agent)
        self.client.force_login(agent)
        self.client.get(lead.get_absolute_url())
        self.assertEqual(
            AccessLog.objects.filter(user=agent, view_name="leads:detail",
object_key=str(lead.pk)).count(), 1,
        )
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py test
git add -A && git commit -m "day 21"
```

Day 22 — Settings, env vars, PostgreSQL, logging

Goal: one codebase, three environments. Settings split into `base` / `dev` / `prod` / `test`, every secret read from the environment, SQLite swapped for PostgreSQL (with real full-text and fuzzy search), and structured logs that carry the request id.

From today the project is **PostgreSQL-only** (extensions, GIN indexes). Run Postgres locally (Docker is easiest) and in CI.

A. Drills (1–10) — settings and environment

- [ ] **1.** Split `settings.py` into a `settings/` package: `base.py`, `dev.py`, `prod.py`, `test.py`. Fix `BASE_DIR` (one level deeper now) and the defaults in `manage.py`, `wsgi.py`, `asgi.py`.
- [ ] **2.** `environs`: read a `.env` file; `env.str`, `env.bool`, `env.int`, `env.list`.
- [ ] **3.** `SECRET_KEY` from the environment (generate a new one); `DEBUG` defaults to False.
- [ ] **4.** `ALLOWED_HOSTS` and `CSRF_TRUSTED_ORIGINS` as comma-separated env lists.
- [ ] **5.** `DATABASES` from one `DATABASE_URL`.
- [ ] **6.** Email settings from env (host, port, user, password, TLS, from address).
- [ ] **7.** CRM settings from env: `CRM_API_KEY`, `MAINTENANCE_MODE`, `CRM_ENVIRONMENT`, `CRM_BUILD`, `CRM_BASE_URL`.
- [ ] **8.** `diffsettings`: what dev changes vs Django's defaults.
- [ ] **9.** `check --deploy` against the prod settings.
- [ ] **10.** Move Day 18's `settings_test.py` into `settings/test.py` with safe env defaults so tests run without a `.env`.

B. Core (11–20) — PostgreSQL

- [ ] **11.** Start Postgres 16 (Docker) or create the role and database with SQL; give the role `CREATEDB` for the test database.
- [ ] **12.** Install psycopg 3; point `DATABASE_URL` at Postgres; `migrate`.
- [ ] **13.** Move existing data: `dumpdata` with natural keys → `migrate` on Postgres → `loaddata` (or re-run the seed).

- [ ] **14.** Connection settings: `CONN_MAX_AGE`, `CONN_HEALTH_CHECKS`, a statement timeout; Django 5.1's built-in pool as the alternative.
- [ ] **15.** Confirm times are stored in UTC and shown in Dubai time (`SHOW timezone`, a raw row vs the template).
- [ ] **16.** Full-text search with `SearchVector` / `SearchQuery(search_type="websearch")` / `SearchRank` over name, email, notes.
- [ ] **17.** Fuzzy name search with `pg_trgm`: `TrigramExtension`, `TrigramSimilarity`, and a GIN trigram index on `UPPER(name)` so `icontains` stops scanning.
- [ ] **18.** JSONField queries on the audit log (`has_key`, path lookups) + a GIN index.
- [ ] **19.** `ArrayField`: `Lead.languages`; `contains` and `overlap` lookups to match leads to agents.
- [ ] **20.** `explain(analyze=True, buffers=True)` on the lead list query.

C. Nexora CRM (21–30) — logging and production settings

- [ ] **21.** `LOGGING`: console handler with the request id in every line; `crm` and `django.request` loggers.
- [ ] **22.** Rotating log file in dev; JSON log lines in prod.
- [ ] **23.** Log business events (lead converted, import finished, export run) with `extra` fields.
- [ ] **24.** `LOG_LEVEL` from env; SQL logging switchable for debugging.
- [ ] **25.** `ADMINS` + `mail_admins` for 500s in prod.
- [ ] **26.** Health check that really checks the database and returns JSON (503 when down).
- [ ] **27.** Secrets hygiene: `.env` ignored, `.env.example` committed; what to do if `.env` was ever committed.
- [ ] **28.** `prod.py` security: HTTPS redirect, HSTS, secure cookies, proxy SSL header.
- [ ] **29.** Feature flags from env (`FEATURES=imports,exports`) used in templates and views.
- [ ] **30.** Read replica for reports: a second database alias used with `.using("replica")`, mirrored in tests.

D. Challenge (31–40)

- [ ] **31.** `docker-compose.yml` for Postgres with a health check; CI workflow with a Postgres service.

- [ ] **32.** Migration that creates the `pg_trgm` and `unaccent` extensions before any index needs them.
- [ ] **33.** `pg_stat_statements`: enable it and list the 10 most expensive queries.
- [ ] **34.** `db_default=Now()` (Django 5.0) on a new model's timestamp.
- [ ] **35.** `GeneratedField`: `phone_last4` stored and indexed for "last 4 digits" search.
- [ ] **36.** Custom deploy system check (`crm.E001`) failing on a development API key.
- [ ] **37.** Test: the deploy check flags `CRM_API_KEY="dev-key-123"`.
- [ ] **38.** Test: health check returns `db: ok`.
- [ ] **39.** Test: converting a lead writes a `lead converted` log line (`assertLogs`).
- [ ] **40.** Test (PostgreSQL only): trigram search finds a misspelt name.

Solutions

A. Settings and environment (1–10)

```
django_project/
├── settings/
|   ├── __init__.py      # empty
|   ├── base.py          # everything shared — the old settings.py, now
reading env
|   ├── dev.py
|   ├── prod.py
|   └── test.py
├── urls.py
├── wsgi.py              # → settings.prod
└── asgi.py              # → settings.prod
```

```
python -m pip install "environs[django]" "psycopg[binary]"
git mv django_project/settings.py django_project/settings/base.py
python -c "import secrets; print(secrets.token_urlsafe(50))"     # 3: new
SECRET_KEY
```

```
# django_project/settings/base.py (top of file + the env-driven parts)
from pathlib import Path
from environs import Env

BASE_DIR = Path(__file__).resolve().parent.parent.parent       # 1: settings/
is one level deeper
```

```
env = Env()
env.read_env(str(BASE_DIR / ".env"))                              # 2: silently
skipped if the file is absent

SECRET_KEY = env.str("SECRET_KEY")                                # 3: no default
— fail fast if missing
DEBUG = env.bool("DEBUG", default=False)                           # safe default
ALLOWED_HOSTS = env.list("ALLOWED_HOSTS", default=["localhost", "127.0.0.1"])
# 4
CSRF_TRUSTED_ORIGINS = env.list("CSRF_TRUSTED_ORIGINS", default=[])

DATABASES = {"default": env.dj_db_url(                             # 5
    "DATABASE_URL", default=f"sqlite:///{(BASE_DIR /
'db.sqlite3').as_posix()}",
)}
DATABASES["default"]["CONN_MAX_AGE"] = env.int("CONN_MAX_AGE", default=60)
# 14
DATABASES["default"]["CONN_HEALTH_CHECKS"] = True

EMAIL_HOST = env.str("EMAIL_HOST", default="localhost")        # 6
EMAIL_PORT = env.int("EMAIL_PORT", default=587)
EMAIL_HOST_USER = env.str("EMAIL_HOST_USER", default="")
EMAIL_HOST_PASSWORD = env.str("EMAIL_HOST_PASSWORD", default="")
EMAIL_USE_TLS = env.bool("EMAIL_USE_TLS", default=True)
DEFAULT_FROM_EMAIL = env.str("DEFAULT_FROM_EMAIL", default="Nexora CRM <no-
reply@nexora.test>")
SERVER_EMAIL = env.str("SERVER_EMAIL", default=DEFAULT_FROM_EMAIL)

CRM_COMPANY_NAME = "Nexora"                                        # 7
CRM_API_KEY = env.str("CRM_API_KEY")
MAINTENANCE_MODE = env.bool("MAINTENANCE_MODE", default=False)
CRM_ENVIRONMENT = env.str("CRM_ENVIRONMENT", default="dev")
CRM_BUILD = env.str("CRM_BUILD", default="local")
CRM_BASE_URL = env.str("CRM_BASE_URL", default="http://localhost:8000")   #
absolute links in emails (Day 23)
FEATURES = set(env.list("FEATURES", default=[]))                   # 29
LOG_LEVEL = env.str("LOG_LEVEL", default="INFO")                   # 24

INSTALLED_APPS += ["django.contrib.postgres"]                      # 16-19 (or
add it to the list directly)

# ...everything else from the old settings.py stays here unchanged
```

```
# django_project/settings/dev.py
from .base import *  # noqa: F401,F403

DEBUG = env.bool("DEBUG", default=True)
EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend"


# django_project/settings/test.py                                    # 10
import os

os.environ.setdefault("SECRET_KEY", "test-only-not-secret")       # before
base reads them
os.environ.setdefault("CRM_API_KEY", "test-key")

from .base import *  # noqa: E402,F401,F403

PASSWORD_HASHERS = ["django.contrib.auth.hashers.MD5PasswordHasher"]
EMAIL_BACKEND = "django.core.mail.backends.locmem.EmailBackend"
```

```
# manage.py — 1
os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"django_project.settings.dev")

# wsgi.py and asgi.py
os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"django_project.settings.prod")
```

```
# .env  (never committed)
DEBUG=True
SECRET_KEY=<paste the generated key>
DATABASE_URL=postgres://nexora:nexora-dev@localhost:5432/nexora_crm
CRM_API_KEY=<long random string>
ALLOWED_HOSTS=localhost,127.0.0.1
LOG_LEVEL=DEBUG
FEATURES=imports,exports

# .env.example  (committed: same keys, no real values)
DEBUG=True
SECRET_KEY=change-me
DATABASE_URL=postgres://USER:PASSWORD@localhost:5432/nexora_crm
CRM_API_KEY=change-me
ALLOWED_HOSTS=localhost,127.0.0.1
```

```
python manage.py diffsettings                                    # 8: only
settings that differ from Django defaults
python manage.py check --deploy --settings=django_project.settings.prod   # 9
(prod needs its env vars set)
python manage.py test --settings=django_project.settings.test   # 10
```

B. PostgreSQL (11–20)

11–13.

```
# 11: Postgres 16 in Docker
docker run --name nexora-pg -d -p 5432:5432 \
  -e POSTGRES_USER=nexora -e POSTGRES_PASSWORD=nexora-dev -e
POSTGRES_DB=nexora_crm \
  -v nexora-pgdata:/var/lib/postgresql/data postgres:16
```

```
-- 11: or on an existing server (psql as a superuser)
CREATE ROLE nexora WITH LOGIN PASSWORD 'nexora-dev' CREATEDB;   -- CREATEDB:
Django builds test_nexora_crm
CREATE DATABASE nexora_crm OWNER nexora ENCODING 'UTF8';
ALTER ROLE nexora SET timezone TO 'UTC';
```

```
# 13: move the SQLite data across
#   a) while DATABASE_URL still points at SQLite:
python manage.py dumpdata --natural-foreign --natural-primary \
  -e contenttypes -e auth.permission -e admin.logentry -e sessions --indent 2
-o data.json
#   b) set DATABASE_URL to Postgres in .env, then:
python manage.py migrate
python manage.py loaddata data.json        # loaddata also resets the
Postgres id sequences
#   or skip a–b's data and re-seed: python manage.py shell -c "from
leads.seed import run_all; run_all()"
```

Natural keys load content types and permissions by name instead of by id, which differ between databases. The receivers that skip `raw=True` saves stay quiet during `loaddata`; add `if kwargs.get("raw"): return` to Day 16's `put_user_in_role_group` too.

14.

```
# settings/base.py
DATABASES["default"]["OPTIONS"] = {"options": "-c statement_timeout=5000"}
# ms: kill runaway queries

# Django 5.1+ alternative — a real connection pool (pip install
"psycopg[binary,pool]")
# DATABASES["default"]["CONN_MAX_AGE"] = 0                    # required
with pool
# DATABASES["default"]["OPTIONS"] = {"pool": {"min_size": 2, "max_size": 10}}
```

15.

```
# python manage.py shell
from django.db import connection
with connection.cursor() as c:
    c.execute("SHOW timezone")
    print(c.fetchone())                    # ('UTC',) — Django sets the
connection to UTC when USE_TZ=True
    c.execute("SELECT created_at FROM crm_leads ORDER BY id DESC LIMIT 1")
    print(c.fetchone())                    # aware UTC datetime
# templates show the same moment converted to TIME_ZONE (Asia/Dubai, UTC+4)
```

16–20.

```
# leads/search_pg.py
from django.contrib.postgres.search import SearchQuery, SearchRank,
SearchVector, TrigramSimilarity
from .models import Lead

LEAD_VECTOR = (SearchVector("name", weight="A", config="simple")        #
"simple": no stemming, fine for names
               + SearchVector("email", weight="B", config="simple")
               + SearchVector("notes", weight="C", config="simple"))


def full_text_leads(text, qs=None):                                     # 16
    qs = Lead.objects.all() if qs is None else qs
    query = SearchQuery(text, search_type="websearch", config="simple")   #
"khan -lost", "\"gold account\""
    return (qs.annotate(rank=SearchRank(LEAD_VECTOR, query))
              .filter(rank__gt=0.01).order_by("-rank"))


def fuzzy_leads(text, threshold=0.3, qs=None):                          # 17
    qs = Lead.objects.all() if qs is None else qs
    return (qs.annotate(similarity=TrigramSimilarity("name", text))
              .filter(similarity__gt=threshold).order_by("-similarity"))
```

```
# leads/migrations/00NN_pg_extensions.py — 17 (and 32)
from django.contrib.postgres.operations import TrigramExtension,
UnaccentExtension
from django.db import migrations


class Migration(migrations.Migration):
    dependencies = [("leads", "00NN_previous")]
    operations = [TrigramExtension(), UnaccentExtension()]   # trusted
extensions: the DB owner may create them
```

```
# leads/models.py — 17–19: indexes and the array field
from django.contrib.postgres.fields import ArrayField
from django.contrib.postgres.indexes import GinIndex, OpClass
from django.db.models.functions import Upper


class Lead(TrackChangesMixin, models.Model):
```

```
    ...
    languages = ArrayField(models.CharField(max_length=20), default=list,
blank=True)   # 19

    class Meta:
        indexes = [
            ...,
            # 17: Django's icontains on Postgres compiles to UPPER(name) LIKE
UPPER('%..%') → index that expression
            GinIndex(OpClass(Upper("name"), name="gin_trgm_ops"),
name="lead_name_upper_trgm"),
            GinIndex(fields=["name"], opclasses=["gin_trgm_ops"],
name="lead_name_trgm"),    # TrigramSimilarity
        ]


# core/models.py — 18
class ActivityLog(models.Model):
    ...
    class Meta:
        indexes = [
            models.Index(fields=["content_type", "object_id"],
name="activity_target_idx"),
            GinIndex(fields=["changes"], name="activity_changes_gin"),
# jsonb containment / keys
        ]
```

```
# python manage.py shell — 16–20
from leads.search_pg import full_text_leads, fuzzy_leads
from core.models import ActivityLog
from leads.models import Lead

full_text_leads("khan callback")[:5]
fuzzy_leads("aysha")[:5]                                          #
finds Aisha
Lead.objects.filter(name__trigram_similar="rahmen")               #
lookup form

ActivityLog.objects.filter(changes__has_key="status")             # 18
ActivityLog.objects.filter(changes__status__1="lost")             # new
value = index 1 of [old, new]
ActivityLog.objects.filter(changes__contains={"tags": ["vip"]})   # uses
the GIN index

Lead.objects.filter(languages__contains=["arabic"])               # 19:
has arabic
Lead.objects.filter(languages__overlap=["arabic", "hindi"])       # has
either

qs = Lead.objects.filter(status="new", is_active=True).order_by("-
created_at")[:25]
print(qs.explain(analyze=True, buffers=True))                     # 20:
real timings + buffer hits
print(Lead.objects.filter(name__icontains="khan").explain())      #
Bitmap Index Scan on lead_name_upper_trgm
```

C. Logging and production (21–30)

Logging is configured during `django.setup()`, *before* apps load. Anything `LOGGING` imports (filters, formatters) must not import models or `django.contrib.auth` — so the request-id filter moves out of `core/middleware.py` into a tiny `core/logging.py`, and the contextvar moves to `core/context.py`.

```
# core/context.py (add)
_request_id = ContextVar("request_id", default="-")

# core/middleware.py — RequestIDMiddleware now imports it from there
from core.context import _request_id
```

```
# core/logging.py — 21-22
import json
import logging

from core.context import _request_id


class RequestIDLogFilter(logging.Filter):
    def filter(self, record):
        record.request_id = _request_id.get()
        return True


class JsonFormatter(logging.Formatter):
    STANDARD = set(vars(logging.makeLogRecord({}))) | {"message", "asctime"}

    def format(self, record):
        payload = {
            "ts": self.formatTime(record, "%Y-%m-%dT%H:%M:%S%z"),
            "level": record.levelname,
            "logger": record.name,
            "msg": record.getMessage(),
            "request_id": getattr(record, "request_id", "-"),
        }
        payload.update({k: v for k, v in vars(record).items() if k not in
self.STANDARD and k not in payload})
        if record.exc_info:
            payload["exc"] = self.formatException(record.exc_info)
        return json.dumps(payload, default=str)      # one JSON object per
line → easy to ship and query
```

```
# settings/base.py — 21, 24, 25
LOG_DIR = BASE_DIR / "logs"
LOG_DIR.mkdir(exist_ok=True)

LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "filters": {
        "request_id": {"()": "core.logging.RequestIDLogFilter"},
        "require_debug_false": {"()": "django.utils.log.RequireDebugFalse"},
    },
```

```
    "formatters": {
        "console": {"format": "{asctime} {levelname:<7} {name} [{request_id}]
{message}", "style": "{"},
        "json": {"()": "core.logging.JsonFormatter"},
    },
    "handlers": {
        "console": {"class": "logging.StreamHandler", "filters":
["request_id"], "formatter": "console"},
        "file": {
            "class": "logging.handlers.RotatingFileHandler", "filename":
LOG_DIR / "crm.log",
            "maxBytes": 5 * 1024 * 1024, "backupCount": 5, "encoding": "utf-
8",
            "filters": ["request_id"], "formatter": "console",
        },
        "mail_admins": {                                          # 25:
emails the traceback of every 500
            "class": "django.utils.log.AdminEmailHandler", "level": "ERROR",
"filters": ["require_debug_false"],
        },
    },
    "loggers": {
        "crm": {"handlers": ["console", "file"], "level": LOG_LEVEL,
"propagate": False},
        "django.request": {"handlers": ["console", "mail_admins"], "level":
"WARNING", "propagate": False},
        "django.db.backends": {                                   # 24: set
SQL_LOG_LEVEL=DEBUG to print every query
            "handlers": ["console"], "level": env.str("SQL_LOG_LEVEL",
default="INFO"), "propagate": False,
        },
    },
    "root": {"handlers": ["console"], "level": "WARNING"},
}
```

`django.db.backends` only logs SQL when `DEBUG=True` (24). Rename Day 21's loggers to the `crm.` namespace (`crm.requests`, `crm.email`, …) so they inherit these handlers.

```
# settings/prod.py — 22, 25, 28
from .base import *  # noqa: F401,F403

DEBUG = False
ADMINS = [("Nexora Ops", env.str("ADMIN_EMAIL"))]

SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")   # behind a
load balancer / nginx
SECURE_SSL_REDIRECT = env.bool("SECURE_SSL_REDIRECT", default=True)
SECURE_HSTS_SECONDS = env.int("SECURE_HSTS_SECONDS", default=60 * 60 * 24 *
30)   # start small, raise later
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
X_FRAME_OPTIONS = "DENY"

LOGGING["handlers"]["console"]["formatter"] = "json"            # 22:
structured logs in production
LOGGING["loggers"]["crm"]["handlers"] = ["console"]              #
containers: stdout, not files
```

23.

```
# clients/services.py — convert_lead_to_client()
import logging
logger = logging.getLogger("crm.leads")

logger.info("lead converted", extra={"lead_id": lead.pk, "client_id":
client.pk,
                                     "broker_id": client.broker_id,
"actor_id": getattr(actor, "pk", None)})

# leads/importers.py — end of run_import()
logging.getLogger("crm.imports").info("import finished", extra={
    "batch_id": batch.pk, "created": batch.created_n, "updated":
batch.updated_n, "errors": len(batch.errors)})

# leads/views.py — the export views
logging.getLogger("crm.exports").info("export", extra={"user_id":
request.user.pk, "format": "xlsx",
                                                       "query":
request.GET.urlencode()})
```

Use `extra=` fields rather than f-strings: in JSON logs they arrive as searchable keys (`lead_id=4231`).

26.

```
# core/views.py
import logging
from django.db import connection


class HealthView(View):
    def get(self, request):
        try:
            with connection.cursor() as cursor:
                cursor.execute("SELECT 1")
        except Exception as exc:                        # any DB failure =
unhealthy
            logging.getLogger("crm.health").error("database check failed:
%s", exc)
            return JsonResponse({"status": "error", "db": "down"},
status=503)
        return JsonResponse({"status": "ok", "db": "ok", "vendor":
connection.vendor, "build": settings.CRM_BUILD})
```

27.

```
# .env is in .gitignore; .env.example is committed. If .env was ever
committed:
git rm --cached .env
git commit -m "stop tracking .env"
# then ROTATE every secret it held (SECRET_KEY, DB password, API keys) — git
history keeps the old file
```

29.

```
# core/context_processors.py — crm_globals adds: "FEATURES":
settings.FEATURES

# core/decorators.py
from django.http import Http404


def feature_required(name):
    def decorator(view):
        @functools.wraps(view)
        def wrapper(request, *args, **kwargs):
            if name not in settings.FEATURES:
                raise Http404("Feature disabled")
            return view(request, *args, **kwargs)
        return wrapper
    return decorator

# leads/urls.py
path("import/", feature_required("imports")(views.LeadImportView.as_view()),
name="import"),
```

```
{% if "imports" in FEATURES %}<a href="{% url 'leads:import' %}">Import
leads</a>{% endif %}
```

30.

```
# settings/base.py
DATABASES["replica"] = env.dj_db_url("REPLICA_DATABASE_URL",
default=env.str("DATABASE_URL", default="") or
                                    f"sqlite:///{(BASE_DIR /
'db.sqlite3').as_posix()}")
DATABASES["replica"]["TEST"] = {"MIRROR": "default"}     # tests: replica =
default, so fixtures are visible
DATABASE_ROUTERS = ["core.routers.PrimaryOnlyMigrations"]


# core/routers.py
class PrimaryOnlyMigrations:
    def allow_migrate(self, db, app_label, model_name=None, **hints):
        return db == "default"           # never run migrations against the
read-only replica


# reports/queries.py — explicit, so read-after-write pages never hit a
lagging replica
def source_performance(using="replica"):
    return Lead.objects.using(using).active().values("source")...   # rest
unchanged
```

D. Challenge (31–36)

31.

```
# docker-compose.yml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: nexora
      POSTGRES_PASSWORD: nexora-dev
      POSTGRES_DB: nexora_crm
    command: postgres -c shared_preload_libraries=pg_stat_statements    # 33
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U nexora -d nexora_crm"]
      interval: 5s
      retries: 10
volumes:
  pgdata:
```

```
# .github/workflows/tests.yml — add a Postgres service
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: {POSTGRES_USER: nexora, POSTGRES_PASSWORD: nexora, POSTGRES_DB:
nexora_crm}
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U nexora" --health-interval 5s --health-
retries 10
    env:
      DATABASE_URL: postgres://nexora:nexora@localhost:5432/nexora_crm
    steps:
      # ...checkout, setup-python, pip install as before
      - run: python manage.py test --parallel --
settings=django_project.settings.test
```

33.

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT calls,
       round(mean_exec_time::numeric, 1)  AS mean_ms,
       round(total_exec_time::numeric)    AS total_ms,
       left(query, 120)                   AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

34–35.

```
# core/models.py — 34: the database fills the timestamp, even for rows
inserted by raw SQL
from django.db.models.functions import Now


class AccessLog(models.Model):
    ...
    created_at = models.DateTimeField(db_default=Now())


# leads/models.py — 35
from django.db.models.functions import Right

phone_last4 = models.GeneratedField(
    expression=Right("phone", 4),
    output_field=models.CharField(max_length=4),
    db_persist=True,                    # stored column → indexable
)
# Meta.indexes += [models.Index(fields=["phone_last4"],
name="lead_phone_last4_idx")]
# search_q(): if the term is exactly 4 digits → Q(phone_last4=digits)
(index lookup instead of LIKE)
```

36.

```
# core/checks.py
from django.conf import settings
from django.core.checks import Error, Tags, Warning, register

DEV_KEYS = {"dev-key-123", "test-key", "change-me"}


@register(Tags.security, deploy=True)      # deploy=True: only with check --
deploy
def crm_deploy_checks(app_configs, **kwargs):
    problems = []
    if settings.CRM_API_KEY in DEV_KEYS or len(settings.CRM_API_KEY) < 32:
        problems.append(Error("CRM_API_KEY looks like a development value.",
id="crm.E001",
                              hint="Set a long random CRM_API_KEY in the
production environment."))
    if settings.MAINTENANCE_MODE:
        problems.append(Warning("MAINTENANCE_MODE is on.", id="crm.W001"))
    return problems

# core/apps.py — CoreConfig.ready(): from . import checks  # noqa: F401
```

Tests (37–40)

```
# core/tests.py
from unittest import skipUnless

from django.db import connection
from django.test import TestCase, override_settings
from django.urls import reverse

from clients.services import convert_lead_to_client
from core.checks import crm_deploy_checks
from core.factories import LeadFactory
from leads.search_pg import fuzzy_leads


class SettingsAndOpsTests(TestCase):
    @override_settings(CRM_API_KEY="dev-key-123")
    def test_deploy_check_flags_dev_key(self):                        # 37
        self.assertIn("crm.E001", [p.id for p in crm_deploy_checks(None)])
```

```
    def test_health(self):                                          # 38
        response = self.client.get(reverse("health"))
        self.assertEqual(response.json()["db"], "ok")

    def test_conversion_is_logged(self):                            # 39
        lead = LeadFactory()
        with self.assertLogs("crm.leads", level="INFO") as logs:
            convert_lead_to_client(lead.pk)
        self.assertTrue(any("lead converted" in line for line in
logs.output))


@skipUnless(connection.vendor == "postgresql", "needs PostgreSQL + pg_trgm")
class TrigramTests(TestCase):
    def test_misspelt_name(self):                                   # 40
        lead = LeadFactory(name="Aisha Rahman")
        self.assertIn(lead, fuzzy_leads("aisha rahmen"))
```

```
docker compose up -d db
python manage.py migrate
python manage.py test --settings=django_project.settings.test
git add -A && git commit -m "day 22"
```

Day 23 — Email and management commands

Goal: the CRM talks to people and runs on a schedule. Agents get an email when a lead is assigned to them and a morning digest; managers get the daily KPI report; every email is logged and failures can be retried. Routine jobs become management commands that cron (or Task Scheduler) runs.

A. Drills (1–10) — sending email

- [ ] **1.** `send_mail()` with the console backend; read the output.
- [ ] **2.** `EmailMessage` with cc, bcc, `reply_to` and a custom header.
- [ ] **3.** `EmailMultiAlternatives`: plain text + HTML version.
- [ ] **4.** Subject and bodies from templates with `render_to_string` (subject forced to one line).
- [ ] **5.** Attach a CSV built in memory.
- [ ] **6.** Several messages over one connection (`get_connection()` + `send_messages`).
- [ ] **7.** `mail_admins()` / `mail_managers()` with `ADMINS` / `MANAGERS`.
- [ ] **8.** Real SMTP from env (`EMAIL_HOST` …) — a provider like SES, SendGrid or Microsoft 365.
- [ ] **9.** `DEFAULT_FROM_EMAIL` vs `SERVER_EMAIL` vs a per-message `from_email`.
- [ ] **10.** Assert on `mail.outbox` in a test.

B. Core (11–20) — management commands

- [ ] **11.** Smallest command: `core/management/commands/hello.py`; run it.
- [ ] **12.** Arguments: positional, `--days` (int), `--dry-run` (flag).
- [ ] **13.** Output with `self.stdout.write` + `self.style.SUCCESS/WARNING`; respect `-v 2`.
- [ ] **14.** Bad input → `CommandError` (clean message, exit code 1).
- [ ] **15.** `seed_crm --leads 500 --reset` wrapping the Day 6/8 seed (DEBUG only).
- [ ] **16.** `mark_burned_lost --attempts 3 --dry-run`.
- [ ] **17.** `daily_report [--date YYYY-MM-DD] [--email]`: yesterday's KPIs as a table; optionally emailed to managers.
- [ ] **18.** `call_command()` from code and tests, capturing `stdout`.

- [ ] **19.** `create_agent USERNAME EMAIL --role --team-lead` using Day 15's invite flow (no self sign-up).
- [ ] **20.** Scheduling: a cron line and a Windows Task Scheduler line; a PostgreSQL advisory lock so two runs can't overlap.

C. Nexora CRM (21–30) — CRM emails

- [ ] **21.** Assignment email: when a lead gets (or changes) an agent, email that agent after commit — not when you assign yourself.
- [ ] **22.** Templates `emails/lead_assigned_subject.txt`, `.txt`, `.html`.
- [ ] **23.** `send_templated_email(prefix, to, context)` helper used everywhere.
- [ ] **24.** `send_agent_digests`: each agent's follow-ups due and stale leads, one SMTP connection for all.
- [ ] **25.** KYC approved → email the client.
- [ ] **26.** Large deposit (Day 20 alert) → also email managers.
- [ ] **27.** `CustomUser.email_digest` opt-out on the profile page; digests respect it.
- [ ] **28.** `EmailLog` model: every send recorded with status and error.
- [ ] **29.** `resend_failed_emails --max-attempts 3`.
- [ ] **30.** `CRM_BASE_URL` for absolute links (emails have no request); use it in the invite email too.

D. Challenge (31–40)

- [ ] **31.** Custom email backend that logs every message it sends.
- [ ] **32.** DEBUG-only email preview page `/dev/emails/<name>/` for designers.
- [ ] **33.** Progress output for a long command (`ending=""`, flush).
- [ ] **34.** `requires_system_checks` and `output_transaction` — when you'd set them.
- [ ] **35.** `export_daily_leads`: save a CSV to private storage and email managers a link.
- [ ] **36.** Signed, expiring download link with `django.core.signing` (24 h); expired → 410.
- [ ] **37.** Test: reassigning a lead sends exactly one email to the new agent.
- [ ] **38.** Test: `daily_report --date` prints the right numbers (frozen time).
- [ ] **39.** Test: agents with `email_digest=False` get no digest.
- [ ] **40.** Test: an invalid `--date` raises `CommandError`.

Solutions

A. Sending email (1–10)

```
# python manage.py shell  (dev settings: console backend → emails print in
the terminal)
import csv
import io
from django.core.mail import (EmailMessage, EmailMultiAlternatives,
get_connection,
                              mail_admins, send_mail)

send_mail("Test subject", "Plain body", None, ["agent1@nexora.test"])
# 1: None → DEFAULT_FROM_EMAIL

EmailMessage(
# 2
    "Callback list", "See the list below.",
    to=["tl1@nexora.test"], cc=["manager@nexora.test"], bcc=
["audit@nexora.test"],
    reply_to=["ops@nexora.test"], headers={"X-CRM-Category": "callbacks"},
).send()

msg = EmailMultiAlternatives("Welcome", "Text version for old clients", to=
["a@nexora.test"])   # 3
msg.attach_alternative("<h1>Welcome</h1><p>HTML version</p>", "text/html")
msg.send()

buffer = io.StringIO()
# 5
csv.writer(buffer).writerows([["id", "name"], [1, "Aisha Khan"]])
msg = EmailMessage("Today's leads", "Attached.", to=["manager@nexora.test"])
msg.attach("leads.csv", buffer.getvalue(), "text/csv")
msg.send()

with get_connection() as connection:
# 6: one SMTP session
    batch = [EmailMessage(f"Hi {name}", "…", to=[f"{name}@nexora.test"],
connection=connection)
             for name in ("agent1", "agent2", "agent3")]
```

```
    connection.send_messages(batch)

mail_admins("Disk almost full", "/var 92% used")                              #
7: to ADMINS, from SERVER_EMAIL
```

```
# settings/base.py — 7, 9
ADMINS = [("Nexora Ops", env.str("ADMIN_EMAIL", default="ops@nexora.test"))]
MANAGERS = ADMINS
EMAIL_SUBJECT_PREFIX = "[Nexora CRM] "       # prefixed to
mail_admins/mail_managers subjects
# DEFAULT_FROM_EMAIL: normal outgoing mail · SERVER_EMAIL: error mails to
ADMINS · from_email=: one message
```

```
# .env on a server — 8 (any SMTP provider; prod.py uses Django's default SMTP
backend)
EMAIL_HOST=email-smtp.me-central-1.amazonaws.com      # Amazon SES (UAE
region); Microsoft 365: smtp.office365.com
EMAIL_PORT=587
EMAIL_HOST_USER=<smtp user>
EMAIL_HOST_PASSWORD=<smtp password>
EMAIL_USE_TLS=True
DEFAULT_FROM_EMAIL=Nexora CRM <crm@nexora.ae>
```

```
# 10 — tests use the in-memory backend automatically
from django.core import mail

send_mail("Hi", "Body", None, ["x@nexora.test"])
assert len(mail.outbox) == 1 and mail.outbox[0].subject == "Hi"
```

C. CRM emails (21–23, 25–28, 30)

28.

```
# core/models.py
class EmailLog(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending"
        SENT = "sent"
        FAILED = "failed"

    template = models.CharField(max_length=60)
    to = models.TextField()
```

```
    subject = models.CharField(max_length=200)
    body_text = models.TextField()
    body_html = models.TextField(blank=True)
    status = models.CharField(max_length=10, choices=Status.choices,
default=Status.PENDING)
    attempts = models.PositiveSmallIntegerField(default=0)
    error = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    sent_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["status", "created_at"],
name="email_status_idx")]
```

23. The rendered message is stored, so a retry (29) never needs the original objects.

```
# core/emails.py
import logging

from django.core.mail import EmailMultiAlternatives
from django.template import TemplateDoesNotExist
from django.template.loader import render_to_string
from django.utils import timezone

from core.models import EmailLog

logger = logging.getLogger("crm.email")


def render_email(prefix, context):
    subject = " ".join(render_to_string(f"emails/{prefix}_subject.txt",
context).split())   # one line, always
    text = render_to_string(f"emails/{prefix}.txt", context)
    try:
        html = render_to_string(f"emails/{prefix}.html", context)
    except TemplateDoesNotExist:
        html = ""
    return subject, text, html


def deliver(log, connection=None):
    msg = EmailMultiAlternatives(log.subject, log.body_text,
```

```
to=log.to.split(", "), connection=connection)
    if log.body_html:
        msg.attach_alternative(log.body_html, "text/html")
    log.attempts += 1
    try:
        msg.send()
    except Exception as exc:                    # SMTP down, bad credentials,
rejected address…
        log.status, log.error = EmailLog.Status.FAILED, str(exc)[:1000]
        logger.warning("email %s to %s failed: %s", log.template, log.to,
exc)
    else:
        log.status, log.error, log.sent_at = EmailLog.Status.SENT, "",
timezone.now()
    log.save(update_fields=["attempts", "status", "error", "sent_at"])
    return log.status == EmailLog.Status.SENT


def send_templated_email(prefix, to, context, connection=None):
    to = [address for address in to if address]
    if not to:
        return False
    subject, text, html = render_email(prefix, context)
    log = EmailLog.objects.create(template=prefix, to=", ".join(to),
subject=subject[:200],
                                  body_text=text, body_html=html)
    return deliver(log, connection)
```

22. Plain-text templates need `{% autoescape off %}` — otherwise `render_to_string` turns `O'Brien & Co` into HTML entities.

```
# templates/emails/lead_assigned_subject.txt
New lead for you: {{ lead.name }}
```

```
# templates/emails/lead_assigned.txt
{% autoescape off %}Hi {{ agent.first_name|default:agent.username }},

{{ lead.name }} has been assigned to you.
Phone: {{ lead.phone|mask_phone }} · Source: {{ lead.get_source_display }}
SOP: first call within 24 hours.

Open the lead: {{ url }}
{% endautoescape %}
```

```
<!-- templates/emails/lead_assigned.html -->
<p>Hi {{ agent.first_name|default:agent.username }},</p>
<p><strong>{{ lead.name }}</strong> has been assigned to you.</p>
<p>Phone {{ lead.phone|mask_phone }} · Source {{ lead.get_source_display }}
<br>SOP: first call within 24 hours.</p>
<p><a href="{{ url }}">Open the lead</a></p>
```

The phone is masked: emails get forwarded and sit in inboxes; the full number stays behind the login.

21. The Day 20 receiver already knows what changed, so it announces assignments with its own signal; a second receiver emails after commit.

```
# leads/signals.py
from django.conf import settings
from django.contrib.auth import get_user_model
from django.db import transaction
from django.dispatch import Signal

from core.emails import send_templated_email

lead_assigned = Signal()          # sent with: lead, agent_id, actor


# inside record_lead_changes(), before reset_tracking():
if created and instance.assigned_to_id:
    lead_assigned.send(sender=Lead, lead=instance,
agent_id=instance.assigned_to_id, actor=user)
elif not created and "assigned_to" in changes and changes["assigned_to"][1]:
    lead_assigned.send(sender=Lead, lead=instance,
agent_id=changes["assigned_to"][1], actor=user)
```

```
@receiver(lead_assigned, dispatch_uid="email_lead_assigned")
def email_lead_assigned(sender, lead, agent_id, actor=None, **kwargs):
    agent = get_user_model().objects.filter(pk=agent_id,
is_active=True).exclude(email="").first()
    if agent is None or (actor is not None and actor.pk == agent.pk):
        return                                  # nobody to tell, or you
assigned it to yourself
    context = {"agent": agent, "lead": lead, "url": settings.CRM_BASE_URL +
lead.get_absolute_url()}
    transaction.on_commit(lambda: send_templated_email("lead_assigned",
[agent.email], context))
```

Bulk reassignment uses `update()` (no signals): send the emails from the bulk view if you want them there. Seeds and imports use `bulk_create` / `bulk_update`, so they don't flood inboxes.

25–27.

```
# clients/views.py — KYCReviewView.post (25)
old_status = doc.client.kyc_status
new_status = refresh_client_kyc(doc.client)
if new_status == Client.KYCStatus.APPROVED and old_status != new_status:
    crm_client = doc.client
    transaction.on_commit(lambda: send_templated_email("kyc_approved",
[crm_client.email], {"client": crm_client}))


# clients/signals.py — large_deposit_alert (26): after creating the Alert
def notify():
    Alert.objects.create(...)                   # as on Day 20
    managers = list(get_user_model().objects.filter(role__in=["manager",
"admin"], is_active=True)
                    .exclude(email="").values_list("email", flat=True))
    send_templated_email("large_deposit", managers,
                         {"deposit": instance, "client": crm_client, "url":
settings.CRM_BASE_URL + crm_client.get_absolute_url()})
transaction.on_commit(notify)
```

```
# accounts/models.py — 27
email_digest = models.BooleanField(default=True, help_text="Morning email
with follow-ups and stale leads")
# accounts/forms.py — ProfileForm.Meta.fields += ["email_digest"]
```

```
# templates/emails/kyc_approved_subject.txt
Your Nexora account is verified

# templates/emails/kyc_approved.txt
{% autoescape off %}Dear {{ client.name }},

Your identity documents have been approved. Your trading account is now fully
active.

Nexora Client Services
{% endautoescape %}
```

30.

```
# accounts/services.py — send_invite(): links without a request use
CRM_BASE_URL
from urllib.parse import urlparse
from django.conf import settings

base = urlparse(settings.CRM_BASE_URL)
form.save(
    request=request,
    domain_override=None if request else base.netloc,
    use_https=request.is_secure() if request else base.scheme == "https",
    subject_template_name="registration/invite_subject.txt",
    email_template_name="registration/invite_email.txt",
)
```

B. Management commands (11–20)

A command is a `Command(BaseCommand)` class in `<app>/management/commands/<name>.py`; both folders need an `__init__.py`. Django passes parsed options to `handle()` as keyword arguments.

```
core/management/__init__.py
core/management/commands/__init__.py
core/management/commands/hello.py
leads/management/__init__.py
leads/management/commands/__init__.py        # seed_crm, mark_burned_lost,
send_agent_digests, ...
reports/management/__init__.py
reports/management/commands/__init__.py      # daily_report,
export_daily_leads
```

```
# core/management/commands/hello.py — 11
from django.core.management.base import BaseCommand


class Command(BaseCommand):
    help = "Smallest possible command"

    def handle(self, *args, **options):
        self.stdout.write("Hello from Nexora CRM")
```

```
python manage.py hello
python manage.py help mark_burned_lost      # argparse help from
add_arguments()
```

12–14, 16.

```
# leads/management/commands/mark_burned_lost.py
from django.core.management.base import BaseCommand, CommandError
from django.db.models import F, TextField, Value
from django.db.models.functions import Concat
from django.utils import timezone

from leads.models import Lead, LeadStatusChange


class Command(BaseCommand):
    help = "Mark open leads with too many call attempts as lost"

    def add_arguments(self, parser):                                    # 12
        parser.add_argument("--attempts", type=int, default=3)
        parser.add_argument("--dry-run", action="store_true")
```

```
    def handle(self, *args, attempts, dry_run, verbosity, **options):
        if attempts < 1:
            raise CommandError("--attempts must be at least 1")        # 14:
red message, exit code 1
        qs = Lead.objects.active().burned(max_attempts=attempts)
        rows = list(qs.values_list("pk", "name", "status",
"contact_attempts"))
        if verbosity >= 2:                                             # 13:
python manage.py ... -v 2
            for pk, name, status, n in rows[:20]:
                self.stdout.write(f"  #{pk} {name} ({status}, {n} attempts)")
        if dry_run:
            self.stdout.write(self.style.WARNING(f"Dry run: {len(rows)}
lead(s) would be marked lost."))
            return
        changed = qs.update(
            status=Lead.Status.LOST, updated_at=timezone.now(),
            notes=Concat(F("notes"), Value(f" | auto-closed after {attempts}
attempts"), output_field=TextField()),
        )
        LeadStatusChange.objects.bulk_create([            # update() skips
signals → write history ourselves
            LeadStatusChange(lead_id=pk, from_status=status,
to_status=Lead.Status.LOST)
            for pk, _, status, _ in rows
        ])
        self.stdout.write(self.style.SUCCESS(f"Marked {changed} lead(s)
lost."))
```

15.

```
# leads/management/commands/seed_crm.py
from django.conf import settings
from django.core.management.base import BaseCommand, CommandError
from django.db import transaction

from clients.models import Client, Deposit, TradingAccount
from core.context import audit_disabled
from leads import seed
from leads.models import CallLog, Lead


class Command(BaseCommand):
```

```
    help = "Seed demo data: users, brokers, leads, calls, clients
(development only)"

    def add_arguments(self, parser):
        parser.add_argument("--leads", type=int, default=200)
        parser.add_argument("--reset", action="store_true", help="delete
existing CRM data first")

    def handle(self, *args, leads, reset, **options):
        if not settings.DEBUG:
            raise CommandError("seed_crm only runs with DEBUG=True")    #
never on production data
        with transaction.atomic(), audit_disabled():
            if reset:
                Deposit.objects.all().delete()          # PROTECT on accounts
→ deposits go first
                TradingAccount.objects.all().delete()
                Client.objects.all().delete()
                CallLog.objects.all().delete()
                Lead.objects.all().delete()
                self.stdout.write(self.style.WARNING("Existing CRM data
deleted."))
            created = seed.run(leads)
            agents, brokers, tags = seed.seed_people()
            seed.assign_leads(agents)
            seed.attach_brokers_and_tags(brokers, tags)
            seed.seed_calls()
            seed.convert_qualified()
        self.stdout.write(self.style.SUCCESS(f"Seeded {created} leads."))
```

17, 20.

```
# core/management/locks.py — 20
from contextlib import contextmanager

from django.core.management.base import CommandError
from django.db import connection


@contextmanager
def advisory_lock(name):
    """PostgreSQL session lock: a second run of the same job exits instead of
doubling the work."""
```

```
    with connection.cursor() as cursor:
        cursor.execute("SELECT pg_try_advisory_lock(hashtext(%s))", [name])
        acquired = cursor.fetchone()[0]
    if not acquired:
        raise CommandError(f"{name} is already running elsewhere")
    try:
        yield
    finally:
        with connection.cursor() as cursor:
            cursor.execute("SELECT pg_advisory_unlock(hashtext(%s))", [name])
```

```
# reports/queries.py — 17
from django.db.models import Sum
from clients.models import Deposit
from leads.models import CallLog, Lead, LeadStatusChange


def daily_stats(day):
    calls = CallLog.objects.filter(started_at__date=day)
    deposits = Deposit.objects.filter(created_at__date=day)
    return {
        "New leads": Lead.objects.filter(created_at__date=day).count(),
        "Calls": calls.count(),
        "Answered calls": calls.answered().count(),
        "Conversions": LeadStatusChange.objects.filter(changed_at__date=day,
to_status="qualified").count(),
        "Deposits": deposits.count(),
        "Deposit total": deposits.aggregate(s=Sum("amount", default=0))["s"],
    }
```

```
# reports/management/commands/daily_report.py — 17
from datetime import date, timedelta

from django.contrib.auth import get_user_model
from django.core.management.base import BaseCommand, CommandError
from django.utils import timezone

from core.emails import send_templated_email
from core.management.locks import advisory_lock
from reports.queries import daily_stats
```

```
class Command(BaseCommand):
    help = "Yesterday's KPIs (or --date). Prints a table; --email sends it to
managers."

    def add_arguments(self, parser):
        parser.add_argument("--date", help="YYYY-MM-DD (default: yesterday,
Dubai time)")
        parser.add_argument("--email", action="store_true")

    def parse_day(self, raw):
        if not raw:
            return timezone.localdate() - timedelta(days=1)
        try:
            return date.fromisoformat(raw)
        except ValueError:
            raise CommandError(f"--date must be YYYY-MM-DD, got {raw!r}")

    def handle(self, *args, **options):
        day = self.parse_day(options["date"])          # validate before
taking the lock
        with advisory_lock("daily_report"):
            stats = daily_stats(day)
            self.stdout.write(f"Nexora CRM — {day:%a %d %b %Y}")
            for label, value in stats.items():
                self.stdout.write(f"  {label:<20}{value:>14}")
            if options["email"]:
                managers = list(get_user_model().objects.filter(role__in=
["manager", "admin"], is_active=True)
                                .exclude(email="").values_list("email",
flat=True))
                send_templated_email("daily_report", managers, {"day": day,
"stats": stats})
                self.stdout.write(self.style.SUCCESS(f"Emailed
{len(managers)} manager(s)."))
```

```
# templates/emails/daily_report_subject.txt
Daily report · {{ day|date:"d M Y" }}

# templates/emails/daily_report.txt
{% autoescape off %}Nexora CRM — {{ day|date:"l d M Y" }}
{% for label, value in stats.items %}
{{ label }}: {{ value }}{% endfor %}
{% endautoescape %}
```

```
# 20 — Linux cron (server clock in UTC: 04:00 UTC = 08:00 Dubai)
0 4 * * * cd /srv/nexora-crm && .venv/bin/python manage.py daily_report --
email --settings=django_project.settings.prod >> logs/cron.log 2>&1
30 3 * * 1-5 cd /srv/nexora-crm && .venv/bin/python manage.py
send_agent_digests --settings=django_project.settings.prod >> logs/cron.log
2>&1
```

```
REM 20 — Windows Task Scheduler
schtasks /Create /SC DAILY /ST 08:00 /TN "Nexora daily report" /TR
"C:\nexora-crm\.venv\Scripts\python.exe C:\nexora-crm\manage.py daily_report
--email"
```

18.

```
from io import StringIO
from django.core.management import call_command

out = StringIO()
call_command("daily_report", date="2026-10-01", stdout=out)   # options by
their dest name
print(out.getvalue())
call_command("mark_burned_lost", "--dry-run", "--attempts", "4")  # CLI-style
strings work too
```

19.

```
# accounts/management/commands/create_agent.py
from django.contrib.auth import get_user_model
from django.core.management.base import BaseCommand, CommandError

from accounts.models import CustomUser
from accounts.services import invite_staff_user
```

```
class Command(BaseCommand):
    help = "Create a staff user and email them a set-password invite (there
is no sign-up page)"

    def add_arguments(self, parser):
        parser.add_argument("username")
        parser.add_argument("email")
        parser.add_argument("--role", choices=CustomUser.Role.values,
default="agent")
        parser.add_argument("--team-lead", dest="team_lead")
        parser.add_argument("--first-name", dest="first_name", default="")

    def handle(self, *args, username, email, role, team_lead, first_name,
**options):
        User = get_user_model()
        if User.objects.filter(username__iexact=username).exists():
            raise CommandError(f"User {username!r} already exists")
        lead = None
        if team_lead:
            lead = User.objects.filter(username=team_lead,
role="team_lead").first()
            if lead is None:
                raise CommandError(f"No team lead called {team_lead!r}")
        user = invite_staff_user(username, email, role=role, team_lead=lead,
first_name=first_name)
        self.stdout.write(self.style.SUCCESS(f"Created {user.username}
({role}); invite sent to {email}."))
```

```
python manage.py create_agent sara.k sara.k@nexora.ae --team-lead tl1 --
first-name Sara
```

C. Digests and retries (24, 29)

```
# leads/management/commands/send_agent_digests.py — 24
from django.conf import settings
from django.contrib.auth import get_user_model
from django.core.mail import get_connection
from django.core.management.base import BaseCommand

from core.emails import send_templated_email
from leads.models import Lead
```

```
class Command(BaseCommand):
    help = "Email each agent their follow-ups due and never-called leads"

    def handle(self, *args, **options):
        agents = (get_user_model().objects.filter(role="agent",
is_active=True, email_digest=True)   # 27
                  .exclude(email=""))
        sent = 0
        with get_connection() as connection:        # one SMTP login for the
whole run
            for agent in agents:
                mine = Lead.objects.active().filter(assigned_to=agent)
                follow_ups = list(mine.follow_up_due()[:20])
                stale = list(mine.stale(days=3)[:20])
                if not (follow_ups or stale):
                    continue                         # no empty emails
                context = {"agent": agent, "follow_ups": follow_ups, "stale":
stale, "base_url": settings.CRM_BASE_URL}
                sent += send_templated_email("agent_digest", [agent.email],
context, connection=connection)
        self.stdout.write(self.style.SUCCESS(f"Sent {sent} digest(s)."))
```

```
# templates/emails/agent_digest_subject.txt
Your call list for {% now "l" %}

# templates/emails/agent_digest.txt
{% autoescape off %}Good morning {{ agent.first_name|default:agent.username
}},
{% if follow_ups %}
Follow-ups due ({{ follow_ups|length }}):
{% for lead in follow_ups %}  - {{ lead.name }} · last call {{
lead.last_contacted_at|date:"d M H:i" }} · {{ base_url }}{{
lead.get_absolute_url }}
{% endfor %}{% endif %}{% if stale %}
Never called ({{ stale|length }}):
{% for lead in stale %}  - {{ lead.name }} · waiting since {{
lead.created_at|date:"d M" }} · {{ base_url }}{{ lead.get_absolute_url }}
{% endfor %}{% endif %}
{% endautoescape %}
```

```
# core/management/commands/resend_failed_emails.py — 29
from django.core.mail import get_connection
from django.core.management.base import BaseCommand

from core.emails import deliver
from core.models import EmailLog


class Command(BaseCommand):
    help = "Retry emails that failed, up to --max-attempts each"

    def add_arguments(self, parser):
        parser.add_argument("--max-attempts", type=int, default=3)

    def handle(self, *args, max_attempts, **options):
        failed = EmailLog.objects.filter(status=EmailLog.Status.FAILED,
attempts__lt=max_attempts).order_by("created_at")
        ok = total = 0
        with get_connection() as connection:
            for log in failed:
                total += 1
                ok += deliver(log, connection)
        self.stdout.write(self.style.SUCCESS(f"Resent {ok}/{total}."))
```

D. Challenge (31–36)

```
# core/mail_backends.py — 31
import logging
from django.core.mail.backends.smtp import EmailBackend

logger = logging.getLogger("crm.email")


class LoggingSMTPBackend(EmailBackend):
    def send_messages(self, email_messages):
        sent = super().send_messages(email_messages)
        for message in email_messages or []:
            logger.info("smtp sent", extra={"to": message.to, "subject":
message.subject})
        return sent

# settings/prod.py: EMAIL_BACKEND = "core.mail_backends.LoggingSMTPBackend"
```

```
# core/views.py — 32 (route only when DEBUG)
from django.utils.html import escape
from core.emails import render_email


class EmailPreviewView(UserPassesTestMixin, View):
    def test_func(self):
        return settings.DEBUG and self.request.user.is_staff

    def sample(self, name):
        lead =
Lead.objects.select_related("assigned_to").exclude(assigned_to=None).first()
        samples = {
            "lead_assigned": {"agent": lead.assigned_to, "lead": lead, "url":
"#"},
            "agent_digest": {"agent": lead.assigned_to, "follow_ups": [lead],
"stale": [], "base_url": ""},
        }
        return samples[name]

    def get(self, request, name):
        subject, text, html = render_email(name, self.sample(name))
        return HttpResponse(f"<h2>{escape(subject)}</h2>" + (html or f"<pre>
{escape(text)}</pre>"))
```

```
# django_project/urls.py
if settings.DEBUG:
    urlpatterns += [path("dev/emails/<slug:name>/",
core_views.EmailPreviewView.as_view())]
```

```
# 33 — inside any long handle()
total = qs.count()
for i, lead in enumerate(qs.iterator(chunk_size=500), start=1):
    ...                                              # work per lead
    if i % 100 == 0 or i == total:
        self.stdout.write(f"\r{i}/{total}", ending="")
        self.stdout.flush()
self.stdout.write("")                                # newline at the end
```

34. `requires_system_checks = []` skips Django's startup checks (useful for a command that must run while something is misconfigured, e.g. a DB fixer). `output_transaction = True` wraps printed SQL in `BEGIN; … COMMIT;` for commands that print SQL to pipe into `psql`.

35–36.

```
# reports/management/commands/export_daily_leads.py — 35
from datetime import timedelta

from django.conf import settings
from django.contrib.auth import get_user_model
from django.core import signing
from django.core.files.base import ContentFile
from django.core.files.storage import storages
from django.core.management.base import BaseCommand
from django.urls import reverse
from django.utils import timezone

from core.emails import send_templated_email
from leads.exports import EXPORT_FIELDS, stream_csv
from leads.models import Lead


class Command(BaseCommand):
    help = "Save yesterday's new leads as CSV in private storage and email
managers a 24-hour link"
```

```
    def handle(self, *args, **options):
        day = timezone.localdate() - timedelta(days=1)
        rows =
Lead.objects.filter(created_at__date=day).values_list(*EXPORT_FIELDS)
        content = "".join(stream_csv(rows)).encode("utf-8")
        name = storages["private"].save(f"exports/leads-{day}.csv",
ContentFile(content))
        token = signing.dumps(name, salt="crm.exports")     # url-safe;
carries a timestamp
        url = settings.CRM_BASE_URL + reverse("reports:signed_download",
args=[token])
        managers = list(get_user_model().objects.filter(role__in=["manager",
"admin"], is_active=True)
                        .exclude(email="").values_list("email", flat=True))
        send_templated_email("export_ready", managers, {"day": day, "url":
url})
        self.stdout.write(self.style.SUCCESS(f"Saved {name}; link sent to
{len(managers)} manager(s)."))
```

```
# reports/views.py — 36
from pathlib import Path

from django.core import signing
from django.core.files.storage import storages
from django.http import FileResponse, Http404, HttpResponse


class SignedDownloadView(View):
    def get(self, request, token):
        try:
            name = signing.loads(token, salt="crm.exports", max_age=60 * 60 *
24)
        except signing.SignatureExpired:            # subclass of
BadSignature → catch it first
            return HttpResponse("This link has expired. Ask for a new
export.", status=410)
        except signing.BadSignature:
            raise Http404
        storage = storages["private"]
        if not storage.exists(name):
            raise Http404
        return FileResponse(storage.open(name, "rb"), as_attachment=True,
filename=Path(name).name)
```

```
# reports/urls.py — still behind the login: the signature limits the file,
the login limits the person
path("downloads/<str:token>/", views.SignedDownloadView.as_view(),
name="signed_download"),
```

Tests (37–40)

```
# core/tests.py
from datetime import datetime, timedelta
from io import StringIO
from unittest import mock

from django.core import mail
from django.core.management import call_command
from django.core.management.base import CommandError
from django.test import TestCase
from django.utils import timezone

from core.factories import AgentFactory, CallLogFactory, LeadFactory
from leads.models import Lead


class EmailAndCommandTests(TestCase):
    def test_reassign_sends_one_email(self):                                # 37
        old = AgentFactory(email="old@nexora.test")
        new = AgentFactory(email="new@nexora.test")
        lead = Lead.objects.get(pk=LeadFactory(assigned_to=old).pk)     #
fresh, tracked instance
        mail.outbox.clear()
        with self.captureOnCommitCallbacks(execute=True):
            lead.assigned_to = new
            lead.save()
        self.assertEqual([m.to for m in mail.outbox], [["new@nexora.test"]])

    def test_daily_report_numbers(self):                                    # 38
        t0 = timezone.make_aware(datetime(2026, 10, 1, 11, 0))
        with mock.patch("django.utils.timezone.now", return_value=t0):
            lead = LeadFactory()
        CallLogFactory.create_batch(2, lead=lead, agent=lead.assigned_to,
started_at=t0)
        out = StringIO()
```

```
        call_command("daily_report", date="2026-10-01", stdout=out)
        self.assertRegex(out.getvalue(), r"New leads\s+1\b")
        self.assertRegex(out.getvalue(), r"Calls\s+2\b")

    def test_digest_respects_opt_out(self):                                 # 39
        last_touch = timezone.now() - timedelta(days=3)
        on = AgentFactory(email="on@nexora.test")
        off = AgentFactory(email="off@nexora.test", email_digest=False)
        for agent in (on, off):
            LeadFactory(assigned_to=agent, status="contacted",
contact_attempts=1, last_contacted_at=last_touch)
        mail.outbox.clear()
        call_command("send_agent_digests", stdout=StringIO())
        self.assertEqual([m.to for m in mail.outbox], [["on@nexora.test"]])

    def test_bad_date(self):                                                # 40
        with self.assertRaises(CommandError):
            call_command("daily_report", date="15/10/2026")
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py test --settings=django_project.settings.test
git add -A && git commit -m "day 23"
```

Day 24 — Caching and profiling

Goal: measure first, then cache. Find where time goes (debug toolbar, profilers, `EXPLAIN ANALYZE`, a load test), then cache the expensive parts — dashboards, reports, the broker tree — with keys that go stale the moment the data changes.

The golden rule: a cache must never show one user another user's data. Every key for a scoped page includes who it's for.

A. Drills (1–10) — the cache framework

- [ ] **1.** `CACHES`: local memory in dev, Redis when `REDIS_URL` is set (Django's built-in Redis backend).
- [ ] **2.** Shell: `set`, `get` (with default), `add`, `get_or_set`, `incr`, `delete`, `touch`, `get_many` / `set_many`.
- [ ] **3.** `KEY_PREFIX` and versions: `cache.set(k, v, version=2)`.
- [ ] **4.** `cache_page(300)` on the calls-by-hour JSON.
- [ ] **5.** Why `cache_page` on a logged-in page is keyed per session (`Vary: Cookie`), and what goes wrong on pages that don't read the session.
- [ ] **6.** Template fragment cache `{% cache 300 "sidebar" request.user.pk %}` — and why it saves nothing when the view already ran the queries.
- [ ] **7.** Low-level cache for the agent leaderboard (cache a list, not a QuerySet).
- [ ] **8.** Generation counters: `generation("leads")` in every key; `bump("leads")` invalidates them all at once.
- [ ] **9.** `cached_property` on a model vs `functools.lru_cache` on a method (the leak).
- [ ] **10.** `SESSION_ENGINE = "django.contrib.sessions.backends.cached_db"`.

B. Core (11–20) — profiling

- [ ] **11.** Install django-debug-toolbar in `dev.py` only.
- [ ] **12.** Use its SQL panel on lead detail: find the duplicated queries and remove them.
- [ ] **13.** `max_queries(testcase, n)` context manager + `@query_budget(n)` test decorator.
- [ ] **14.** `cProfile` a view through `RequestFactory`; sort by cumulative time.
- [ ] **15.** Print every SQL statement for one request (`django.db.backends` at DEBUG).
- [ ] **16.** `EXPLAIN ANALYZE` the leaderboard on PostgreSQL; add an index if it helps.

- [ ] **17.** `?profile=1` (DEBUG only) renders a pyinstrument flame report instead of the page.
- [ ] **18.** Time template rendering on its own with `render_to_string`.
- [ ] **19.** Locust load test: agents logging in and browsing leads, queue and dashboard.
- [ ] **20.** Before/after benchmark script: average ms for 30 requests to a page.

C. Nexora CRM (21–30)

- [ ] **21.** Dashboard numbers cached 60 s per user (`dashboard_stats(user)`).
- [ ] **22.** Reports cached 5 min per scope (everyone vs a team lead's team); `?fresh=1` recomputes; show "computed at".
- [ ] **23.** Broker tree cached 1 hour; dropped on any broker save or delete.
- [ ] **24.** Sidebar counters cached 30 s per user.
- [ ] **25.** `cached(key, ttl, compute)` helper with stampede protection (one process computes, others wait briefly).
- [ ] **26.** Per-request memoisation with `cached_property` on a view.
- [ ] **27.** ETag on the calls-by-hour JSON with `@condition`; a repeat request gets 304.
- [ ] **28.** Cache headers: `private` on CRM pages, `never_cache` where data is sensitive.
- [ ] **29.** Redis in Docker; the Day 21 export rate limit now works across processes.
- [ ] **30.** Signals bump the `leads` generation on every save/delete of leads, calls, deposits, clients.

D. Challenge (31–40)

- [ ] **31.** `warm_cache` command: precompute dashboards and reports for managers after deploy.
- [ ] **32.** The QuerySet-in-cache pitfall: what gets pickled, and why `values()` lists are safer.
- [ ] **33.** Safe keys for long inputs (hash a querystring with `hashlib`).
- [ ] **34.** Tests and caches: local-memory cache shared across tests — clear it in `setUp`.
- [ ] **35.** Hit/miss counters around `cached()`.
- [ ] **36.** PostgreSQL materialized view for daily lead stats + unmanaged model with a `CompositePrimaryKey` (Django 5.2) + `refresh_stats` command.
- [ ] **37.** Test: the second dashboard request runs fewer queries.
- [ ] **38.** Test: saving a lead changes the cached dashboard total immediately.

- [ ] **39.** Test: `If-None-Match` gets 304.
- [ ] **40.** Test: `max_queries` fails when the budget is exceeded.

Solutions

A. The cache framework (1–10)

```
python -m pip install redis              # Django's built-in Redis backend
needs the client library
docker run --name nexora-redis -d -p 6379:6379 redis:7     # 29
```

```
# settings/base.py — 1, 3, 10
REDIS_URL = env.str("REDIS_URL", default="")          # e.g.
redis://localhost:6379/1
if REDIS_URL:
    CACHES = {"default": {
        "BACKEND": "django.core.cache.backends.redis.RedisCache",
        "LOCATION": REDIS_URL, "KEY_PREFIX": "crm", "TIMEOUT": 300,
    }}
else:
    CACHES = {"default": {
        "BACKEND": "django.core.cache.backends.locmem.LocMemCache",   # one
per process — dev/tests only
        "LOCATION": "crm", "TIMEOUT": 300,
    }}

SESSION_ENGINE = "django.contrib.sessions.backends.cached_db"        # 10:
read from cache, written to DB too
```

```
# python manage.py shell — 2, 3
from django.core.cache import cache

cache.set("greeting", "marhaba", timeout=60)
cache.get("greeting"), cache.get("missing", "default")
cache.add("greeting", "hello")          # False — add only writes if the key
is absent
cache.get_or_set("answer", lambda: 6 * 7, 60)   # computes once, then serves
the cached 42
cache.set("hits", 0)
cache.incr("hits"); cache.incr("hits", 5)        # atomic on Redis
cache.touch("greeting", 600)            # extend the timeout
cache.set_many({"a": 1, "b": 2}); cache.get_many(["a", "b"])
cache.delete("greeting")

cache.set("report", "v1 data", version=1)
cache.set("report", "v2 data", version=2)       # 3: same key, separate
entries
cache.get("report", version=2)                  # 'v2 data'
```

4–5.

```
# reports/urls.py — 4
from django.views.decorators.cache import cache_page

path("api/calls-by-hour/", cache_page(300)(views.CallsByHourAPI.as_view()),
name="calls_by_hour"),
```

5. `cache_page` keys on the URL *plus* every header the response varies on. Our pages read `request.user`, which reads the session, so Django adds `Vary: Cookie` — each logged-in session gets its own entry. A page that never touches the session has no `Vary: Cookie`, and one cached copy goes to everyone. Never put `cache_page` on a view whose output depends on the user unless you're sure it varies on the cookie.

6.

```
{% load cache %}
{% cache 300 "sidebar" request.user.pk %}      {# key includes the user —
golden rule #}
  ... sidebar markup reading counters / saved views ...
{% endcache %}
```

Fragment caching only skips *rendering*. Our `counters` are lazy, so caching that fragment does skip their queries. The dashboard's `.count()` calls run in the view before the template starts, so fragment-caching the dashboard saves nothing — that's what 21 fixes.

7–8, 25, 35.

```
# core/cache.py
import hashlib
import time

from django.core.cache import cache

_MISS = object()


def generation(namespace):                                          # 8
    key = f"gen:{namespace}"
    value = cache.get(key)
    if value is None:
        cache.add(key, 1, timeout=None)       # never expires
        value = cache.get(key, 1)
    return value


def bump(namespace):
    try:
        cache.incr(f"gen:{namespace}")        # every key that embeds the old
number is now unreachable
    except ValueError:                        # counter missing (evicted /
fresh cache)
        cache.set(f"gen:{namespace}", 2, timeout=None)


def short_key(prefix, raw):                                         # 33
    return f"{prefix}:{hashlib.md5(raw.encode()).hexdigest()}"


def _count(kind):                                                   # 35
    try:
        cache.incr(f"stats:{kind}")
    except ValueError:
        cache.set(f"stats:{kind}", 1, timeout=None)
```

```
def cached(key, ttl, compute, lock_ttl=30):                              # 25
    value = cache.get(key, _MISS)
    if value is not _MISS:
        _count("hit")
        return value
    _count("miss")
    if cache.add(f"lock:{key}", 1, lock_ttl):     # we won: compute once
        try:
            value = compute()
            cache.set(key, value, ttl)
            return value
        finally:
            cache.delete(f"lock:{key}")
    for _ in range(30):                            # someone else is
computing — wait up to ~3 s
        time.sleep(0.1)
        value = cache.get(key, _MISS)
        if value is not _MISS:
            return value
    return compute()                               # still nothing: compute
rather than fail


def hit_rate():
    hits, misses = cache.get("stats:hit", 0), cache.get("stats:miss", 0)
    return round(hits * 100 / (hits + misses), 1) if hits + misses else None
```

```
# reports/queries.py — 7
from core.cache import cached, generation


def cached_leaderboard(team_lead=None):
    scope = team_lead.pk if team_lead else "all"
    key = f"leaderboard:{scope}:g{generation('leads')}"
    # list(): a QuerySet is lazy — cache the evaluated rows, never the query
object
    return cached(key, 300, lambda: list(agent_leaderboard(team_lead)))
```

9.

```
# brokers/models.py
from functools import cached_property


class Broker(models.Model):
    @cached_property
    def descendant_ids(self):                # computed once per instance;
`del broker.descendant_ids` to reset
        return [b.pk for b in self.descendants()]

    # @functools.lru_cache on a method: the cache is global, keeps every
Broker alive (it holds `self`)
    # and never notices database changes — don't.
```

B. Profiling (11–20)

```
python -m pip install django-debug-toolbar pyinstrument locust
```

```
# settings/dev.py — 11
INSTALLED_APPS += ["debug_toolbar"]
MIDDLEWARE.insert(2, "debug_toolbar.middleware.DebugToolbarMiddleware")   #
early: after RequestID + SlowRequestLog
INTERNAL_IPS = ["127.0.0.1"]


# django_project/urls.py
if settings.DEBUG and "debug_toolbar" in settings.INSTALLED_APPS:
    urlpatterns += [path("__debug__/", include("debug_toolbar.urls"))]
```

If the Day 21 Content-Security-Policy blocks the toolbar in development, skip `SecurityHeadersMiddleware` in `dev.py`.

12. On lead detail the SQL panel shows duplicates such as the call count twice (`lead.calls.count` in the heading *and* the paginator's own count) and repeated permission lookups. Fixes: show `calls_page.paginator.count` instead of `lead.calls.count`, and read `perms.*` once into a `{% with %}`. The panel's "duplicated" badge should reach 0.

13.

```
# core/testing.py
import functools
from contextlib import contextmanager

from django.db import connection
from django.test.utils import CaptureQueriesContext


@contextmanager
def max_queries(testcase, limit):
    with CaptureQueriesContext(connection) as ctx:
        yield ctx
    if len(ctx.captured_queries) > limit:
        sql = "\n".join(q["sql"][:160] for q in ctx.captured_queries)
        testcase.fail(f"{len(ctx.captured_queries)} queries, budget
{limit}:\n{sql}")


def query_budget(limit):
    def decorator(test_method):
        @functools.wraps(test_method)
        def wrapper(self, *args, **kwargs):
            with max_queries(self, limit):
                return test_method(self, *args, **kwargs)
        return wrapper
    return decorator

# usage
@query_budget(12)
def test_lead_list_is_cheap(self):
    self.client.get(reverse("leads:list"))
```

14–15, 18.

```
# python manage.py shell
import cProfile
import pstats
import time

from django.template.loader import render_to_string
from django.test import RequestFactory

from accounts.models import CustomUser
```

```
from reports.views import ReportsView

request = RequestFactory().get("/reports/")
request.user = CustomUser.objects.get(username="manager")

profiler = cProfile.Profile()                                          # 14
profiler.enable()
response = ReportsView.as_view()(request)
response.render()                    # TemplateResponse renders lazily — include it in the profile
profiler.disable()
pstats.Stats(profiler).sort_stats("cumulative").print_stats(15)

context = response.context_data                                        # 18: template time alone
start = time.perf_counter()
render_to_string("reports/index.html", context, request=request)
print(f"template: {(time.perf_counter() - start) * 1000:.1f} ms")
```

```
# 15 — every SQL statement in the console (DEBUG must be True)
SQL_LOG_LEVEL=DEBUG python manage.py runserver
```

16.

```
from reports.queries import agent_leaderboard
print(agent_leaderboard().explain(analyze=True, buffers=True))
# look for Seq Scan on leads_calllog with a large "rows removed by filter":
# the Day 10 index call_agent_started_idx (agent_id, started_at) should turn it into an Index Scan.
```

17.

```
# core/middleware.py — add to MIDDLEWARE in dev.py only
class PyInstrumentMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if not (settings.DEBUG and "profile" in request.GET):
            return self.get_response(request)
        from pyinstrument import Profiler
        profiler = Profiler()
        profiler.start()
        response = self.get_response(request)
        if hasattr(response, "render") and not getattr(response,
"is_rendered", True):
            response.render()
        profiler.stop()
        return HttpResponse(profiler.output_html())    # /reports/?profile=1
```

19.

```
# locustfile.py (project root) — run: locust -H http://127.0.0.1:8000  → open
http://localhost:8089
from locust import HttpUser, between, task


class Agent(HttpUser):
    wait_time = between(1, 3)

    def on_start(self):
        self.client.get("/accounts/login/")
        token = self.client.cookies.get("csrftoken")
        self.client.post("/accounts/login/",
                         {"username": "agent1", "password": "nexora123",
"csrfmiddlewaretoken": token},
                         headers={"Referer": f"{self.host}/accounts/login/"})

    @task(5)
    def leads(self):
        self.client.get("/leads/")

    @task(2)
    def queue(self):
        self.client.get("/team/me/queue/")
```

```
    @task(1)
    def dashboard(self):
        self.client.get("/dashboard/")
```

Run it against `runserver` only to compare before/after; real numbers need gunicorn (Day 28).

20.

```
# bench.py — python manage.py shell < bench.py
import statistics
import time

from django.test import Client
from django.test.utils import setup_test_environment

from accounts.models import CustomUser

setup_test_environment()
http = Client()
http.force_login(CustomUser.objects.get(username="manager"))
for url in ["/dashboard/", "/reports/", "/leads/"]:
    times = []
    for _ in range(30):
        start = time.perf_counter()
        http.get(url)
        times.append((time.perf_counter() - start) * 1000)
    print(f"{url:<14} median {statistics.median(times):6.1f} ms   p90
{sorted(times)[26]:6.1f} ms")
```

C. Nexora CRM (21–30)

30 first — everything below embeds `generation("leads")` in its keys, so one bump invalidates all of it.

```
# core/signals.py (add)
from core.cache import bump


def bump_leads_generation(sender, **kwargs):
    bump("leads")


def connect_cache_invalidation():
    from clients.models import Client, Deposit, TradingAccount
    from leads.models import CallLog, Lead, LeadStatusChange
    for model in (Lead, CallLog, LeadStatusChange, Client, TradingAccount,
Deposit):
        label = model._meta.label_lower
        post_save.connect(bump_leads_generation, sender=model,
dispatch_uid=f"bump_save:{label}")
        post_delete.connect(bump_leads_generation, sender=model,
dispatch_uid=f"bump_delete:{label}")

# core/apps.py — CoreConfig.ready(): connect_audit();
connect_cache_invalidation()
# views and commands that use update()/bulk_create(): call bump("leads")
afterwards
```

The trade-off: on a busy floor every logged call bumps the counter, so dashboards are recomputed often. That's correct (never stale) and still caps the cost at one computation per change instead of one per page view. If that's too often, drop the generation from the dashboard key and rely on its 60-second TTL.

21.

```
# core/dashboard.py
from datetime import timedelta

from django.db.models import Count, Q
from django.utils import timezone

from core.cache import cached, generation
from leads.models import CallLog, Lead


def dashboard_stats(user):
```

```
    key = f"dash:{user.pk}:g{generation('leads')}"          # per user
(golden rule) + generation

    def compute():
        active = Lead.objects.visible_to(user).active()
        yesterday = timezone.localdate() - timedelta(days=1)
        return {
            "stats": active.aggregate(total=Count("id"),
                                      **{v: Count("id", filter=Q(status=v))
for v in Lead.Status.values}),
            "my_open": active.filter(assigned_to=user).open().count(),
            "my_calls_today":
CallLog.objects.for_agent(user).today().count(),
            "new_today": active.created_today().count(),
            "new_yesterday":
active.filter(created_at__date=yesterday).count(),
            "computed_at": timezone.now(),
        }

    return cached(key, 60, compute)


# core/views.py — DashboardView.get_context_data
data = dashboard_stats(self.request.user)
context.update(
    total=data["stats"]["total"],
    counts={label: data["stats"][value] for value, label in
Lead.Status.choices},
    my_open=data["my_open"], my_calls_today=data["my_calls_today"],
    new_today=data["new_today"], new_yesterday=data["new_yesterday"],
)
# lead cards, activity and alerts stay live — they're cheap and should be
fresh
```

22, 26.

```
# reports/cached.py
from django.core.cache import cache
from django.utils import timezone

from core.cache import cached, generation
from leads.permissions import sees_everything, visible_leads
from . import queries
```

```
def report_data(user, fresh=False):
    full = sees_everything(user)
    scope = "all" if full else f"team:{user.pk}"           # team leads never
see each other's numbers
    key = f"reports:{scope}:g{generation('leads')}"
    if fresh:
        cache.delete(key)

    def compute():
        return {
            "leaderboard": list(queries.agent_leaderboard(team_lead=None if
full else user)),
            "brokers": list(queries.broker_report()[:10]) if full else None,
            "sources": list(queries.source_performance()) if full else None,
            "breaches": list(visible_leads(user,
queries.sop_first_call_breaches())[:20]),
            "computed_at": timezone.now(),
        }

    return cached(key, 300, compute)


# reports/views.py
from functools import cached_property
from .cached import report_data


class ReportsView(UserPassesTestMixin, PageTitleMixin, TemplateView):
    ...
    @cached_property
    def full_access(self):                    # 26: computed once per
request, reused by test_func + context
        return sees_everything(self.request.user)

    def test_func(self):
        return self.full_access or self.request.user.role == "team_lead"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
```

```
        context.update(report_data(self.request.user,
fresh=self.request.GET.get("fresh") == "1"),
                       full_access=self.full_access)
        return context
```

```
<!-- reports/index.html -->
<p class="muted">Computed {{ computed_at|timesince }} ago · <a href="?
fresh=1">Refresh now</a></p>
```

23.

```
# brokers/tree.py
from django.core.cache import cache

TREE_KEY = "broker-tree"


def load_tree():
    brokers = cache.get(TREE_KEY)
    if brokers is None:
        brokers = list(Broker.objects.order_by("code"))
        cache.set(TREE_KEY, brokers, 60 * 60)
    by_id = {b.pk: b for b in brokers}
    children = defaultdict(list)
    for b in brokers:
        children[b.parent_id].append(b)
    return by_id, children


# brokers/signals.py (import in BrokersConfig.ready())
from django.core.cache import cache
from django.db.models.signals import post_delete, post_save
from django.dispatch import receiver

from .models import Broker
from .tree import TREE_KEY


@receiver([post_save, post_delete], sender=Broker,
dispatch_uid="broker_tree_cache")
def drop_broker_tree(sender, **kwargs):
    cache.delete(TREE_KEY)
```

24.

```
# core/context_processors.py — crm_counters: the lazy object now reads
through the cache
from core.cache import cached, generation

return {"counters": SimpleLazyObject(
    lambda: cached(f"counters:{user.pk}:g{generation('leads')}", 30, compute)
)}
```

27.

```
# reports/urls.py — replaces 4's cache_page: the browser keeps its copy, we
only answer "not modified"
from django.db.models import Count, Max
from django.views.decorators.http import condition
from leads.models import CallLog


def calls_etag(request, *args, **kwargs):
    agg = CallLog.objects.aggregate(n=Count("id"), last=Max("started_at"))
    return f"calls-{agg['n']}-{agg['last'].timestamp() if agg['last'] else
0}"   # changes when calls change


path("api/calls-by-hour/", condition(etag_func=calls_etag)
(views.CallsByHourAPI.as_view()), name="calls_by_hour"),
```

The ETag query is two aggregates on an indexed column — far cheaper than the hourly breakdown it saves.

28.

```
# core/middleware.py — SecurityHeadersMiddleware.__call__, before returning
from django.utils.cache import patch_cache_control

if getattr(request, "user", None) is not None and
request.user.is_authenticated:
    patch_cache_control(response, private=True)     # browsers may keep it;
shared proxies/CDNs must not


# clients/views.py — KYC downloads are never stored anywhere
from django.utils.decorators import method_decorator
from django.views.decorators.cache import never_cache


@method_decorator(never_cache, name="dispatch")
class KYCDownloadView(View):
    ...
```

29. With `REDIS_URL=redis://localhost:6379/1` in `.env`, the Day 21 export limiter's `cache.add` + `cache.incr` run against Redis: atomic, and shared by every gunicorn worker and server. With local memory each process kept its own count, so 4 workers allowed 40 exports an hour.

D. Challenge (31–36)

```
# core/management/commands/warm_cache.py — 31
from django.contrib.auth import get_user_model
from django.core.management.base import BaseCommand

from core.dashboard import dashboard_stats
from reports.cached import report_data


class Command(BaseCommand):
    help = "Precompute dashboards and reports for managers and team leads
(run after deploy)"

    def handle(self, *args, **options):
        users = get_user_model().objects.filter(is_active=True, role__in=
["manager", "admin", "team_lead"])
        for user in users:
            dashboard_stats(user)
            report_data(user, fresh=True)
        self.stdout.write(self.style.SUCCESS(f"Warmed caches for
{users.count()} user(s)."))
```

32. `cache.set("k", Lead.objects.filter(...))` pickles the QuerySet, which evaluates it and stores model instances plus the query state; on Django upgrades or model changes the pickle can fail to load. Cache plain data: `list(qs.values("id", "name", "status"))`, or `list(qs)` for short-lived caches of instances. Never cache something with an un-evaluated `prefetch_related`.

33.

```
# a filtered lead count keyed by the full querystring (could be long and
contain spaces)
from core.cache import cached, generation, short_key

key = short_key("leadcount", f"{request.user.pk}:
{request.GET.urlencode()}:g{generation('leads')}")
count = cached(key, 120, lambda: view.get_leads().count())
```

34.

```
# core/testing.py — local-memory cache survives between tests in the same
process
from django.core.cache import cache


class CacheSafeTestCase(TestCase):
    def setUp(self):
        super().setUp()
        cache.clear()       # also resets rate-limit counters and generation
numbers from earlier tests

# LoggedInTestCase.setUp(): add cache.clear() too
```

35. `cached()` already increments `stats:hit` / `stats:miss`; `hit_rate()` returns the percentage. Show it on the admin index or log it from `warm_cache`.

36.

```
# reports/models.py
class DailyLeadStat(models.Model):
    pk = models.CompositePrimaryKey("day", "status")      # Django 5.2: the
view has no id column
    day = models.DateField()
    status = models.CharField(max_length=20)
    leads = models.IntegerField()

    class Meta:
        managed = False                 # Django never creates, alters or
drops it
        db_table = "daily_lead_stats"
        ordering = ["-day", "status"]
```

```
# reports/migrations/00NN_daily_lead_stats.py
from django.db import migrations

CREATE_VIEW = """
CREATE MATERIALIZED VIEW daily_lead_stats AS
SELECT (created_at AT TIME ZONE 'Asia/Dubai')::date AS day,
       status,
       count(*)::int                              AS leads
FROM crm_leads
WHERE is_active
```

```
GROUP BY 1, 2
"""


class Migration(migrations.Migration):
    dependencies = [("reports", "00NN_previous"), ("leads", "00NN_latest")]
    operations = [
        migrations.RunSQL(
            [CREATE_VIEW,
             "CREATE UNIQUE INDEX daily_lead_stats_pk ON daily_lead_stats
(day, status)"],   # needed for CONCURRENTLY
            reverse_sql=["DROP MATERIALIZED VIEW IF EXISTS
daily_lead_stats"],
        ),
    ]
```

```
# reports/management/commands/refresh_stats.py
from django.core.management.base import BaseCommand
from django.db import connection


class Command(BaseCommand):
    help = "Refresh the daily_lead_stats materialized view"

    def handle(self, *args, **options):
        with connection.cursor() as cursor:
            cursor.execute("REFRESH MATERIALIZED VIEW CONCURRENTLY
daily_lead_stats")   # readers aren't blocked
        self.stdout.write(self.style.SUCCESS("daily_lead_stats refreshed."))

# usage — a 90-day chart becomes an index read instead of a GROUP BY over
every lead
from django.db.models import Sum
DailyLeadStat.objects.filter(day__gte="2026-07-
01").values("day").annotate(total=Sum("leads")).order_by("day")
```

The view depends on `crm_leads.created_at`, `status` and `is_active`: a later migration that alters those columns must drop the view first and recreate it afterwards. Schedule `refresh_stats` every 15 minutes with cron.

Tests (37–40)

```
# core/tests.py
from django.core.cache import cache
from django.db import connection
from django.test import TestCase
from django.test.utils import CaptureQueriesContext
from django.urls import reverse

from core.dashboard import dashboard_stats
from core.factories import LeadFactory, ManagerFactory
from core.testing import max_queries
from leads.models import Lead


class CachingTests(TestCase):
    def setUp(self):
        cache.clear()

    def test_second_dashboard_request_is_cheaper(self):                # 37
        self.client.force_login(ManagerFactory())
        LeadFactory.create_batch(3)
        with CaptureQueriesContext(connection) as first:
            self.client.get(reverse("dashboard"))
        with CaptureQueriesContext(connection) as second:
            self.client.get(reverse("dashboard"))
        self.assertLess(len(second.captured_queries),
len(first.captured_queries))

    def test_lead_save_invalidates_dashboard(self):                    # 38
        manager = ManagerFactory()
        before = dashboard_stats(manager)["stats"]["total"]
        LeadFactory()                                       # post_save →
bump("leads")
        self.assertEqual(dashboard_stats(manager)["stats"]["total"], before +
1)

    def test_etag_gives_304(self):                                     # 39
        self.client.force_login(ManagerFactory())
        url = reverse("reports:calls_by_hour")
        first = self.client.get(url)
        second = self.client.get(url, headers={"If-None-Match":
first["ETag"]})
```

```
        self.assertEqual(second.status_code, 304)

    def test_max_queries_fails_over_budget(self):                        # 40
        with self.assertRaises(AssertionError):
            with max_queries(self, 1):
                list(Lead.objects.all())
                list(Lead.objects.all())
```

```
docker compose up -d db
python manage.py migrate
python manage.py test --settings=django_project.settings.test
git add -A && git commit -m "day 24"
```

Day 25 — Django REST Framework 1

Goal: a proper JSON API for the CRM at `/api/v2/`, built with Django REST Framework (DRF). Day 2's hand-made `/api/v1/` stays for old clients. Today: serializers (Python objects ↔ JSON with validation), views, permissions, throttling, a consistent error format, and the lead endpoints. Day 26 turns them into ViewSets.

New app: `python manage.py startapp api` — all API code lives there (`api/serializers.py`, `api/views.py`, `api/urls.py`, namespace `api`).

A. Drills (1–10) — serializers

- [ ] **1.** Install DRF; `REST_FRAMEWORK` settings (session auth, `IsAuthenticated`); let DRF handle auth for `/api/v2/` (exempt it from the login middleware); open the browsable API.
- [ ] **2.** Plain `Serializer` for a lead summary; serialize one lead and `many=True` in the shell; render to JSON bytes.
- [ ] **3.** Deserialize: `is_valid()`, `errors`, `validated_data`.
- [ ] **4.** `LeadSerializer(ModelSerializer)` with an explicit field list and `read_only_fields`.
- [ ] **5.** `SerializerMethodField` (`masked_phone`) and `source=` (`status_label` from `get_status_display`).
- [ ] **6.** Relations: nested read-only agent, write-only `assigned_to_id`, tags by name (`SlugRelatedField`).
- [ ] **7.** `validate_name()` (field) and `validate()` (object).
- [ ] **8.** `UAEPhoneSerializerField`: normalise in `to_internal_value` so the explicit `UniqueValidator` checks the normalised number.
- [ ] **9.** `HyperlinkedIdentityField` for each lead's `url`.
- [ ] **10.** Read-only computed fields: `is_hot`, `call_count` (annotation, default 0).

B. Core (11–20) — views and policies

- [ ] **11.** `@api_view` function: active lead count for the current user.
- [ ] **12.** `APIView` with `get` (list) and `post` (create, 201).

- [ ] **13.** `APIView` detail with `get` / `patch` / `delete` and a 404 for unknown or invisible leads.
- [ ] **14.** Replace 12–13 with `ListCreateAPIView` + `RetrieveUpdateDestroyAPIView`, queryset scoped with `visible_to()`.
- [ ] **15.** `perform_create()`: an agent's new lead is assigned to them.
- [ ] **16.** Object permission `CanEditLead` (Day 16 rule) for writes.
- [ ] **17.** `DjangoModelPermissions`: DELETE needs `leads.delete_lead` (Managers group).
- [ ] **18.** Throttling: 1,000 requests/hour per user; a tighter scoped rate for exports.
- [ ] **19.** Custom exception handler: every error looks like `{"error", "detail", "request_id"}`.
- [ ] **20.** `201` with a `Location` header on create.

C. Nexora CRM (21–30)

- [ ] **21.** `/api/v2/leads/<id>/calls/`: list and log calls (via Day 10 `log_call`).
- [ ] **22.** `/api/v2/leads/<id>/status/`: allowed transitions only (new → contacted/lost, contacted → qualified/lost, lost → contacted); lost needs notes.
- [ ] **23.** Client detail by `ref` with nested trading accounts, deposits and total.
- [ ] **24.** Broker tree as nested JSON (recursive).
- [ ] **25.** Report endpoints: leaderboard and daily new leads (scoped like the page).
- [ ] **26.** Search endpoint wrapping Day 17's `search_everything`.
- [ ] **27.** Bulk create: a JSON list of leads, per-item errors, duplicates inside the batch rejected.
- [ ] **28.** Version in the URL (`v2`) vs a header — and why `/api/v1/` stays untouched.
- [ ] **29.** OpenAPI schema + Swagger UI with drf-spectacular.
- [ ] **30.** Audit API reads of a lead the same way as page views (Day 21).

D. Challenge (31–40)

- [ ] **31.** Role-aware serializer: agents don't see or send `expected_deposit` / `assigned_to_id`.
- [ ] **32.** Writable nested create: client + first trading account in one atomic POST.
- [ ] **33.** Query budget on the list endpoint (`select_related` / `prefetch_related` in `get_queryset`).

- [ ] **34.** CSV renderer: `?format=csv` on the lead list.
- [ ] **35.** JSON-only renderers in production (no browsable API).
- [ ] **36.** Validate query parameters with a serializer (`?days=` 1–365).
- [ ] **37.** Test: an agent's list contains only their leads.
- [ ] **38.** Test: create normalises the phone and assigns the agent; a duplicate returns 400 with the message.
- [ ] **39.** Test: new → qualified returns 400.
- [ ] **40.** Test: throttling returns 429 after the limit.

Solutions

The DRF request cycle: authentication → permissions → throttles → your handler → serializer → `Response` → renderer (JSON, browsable HTML, CSV…). Serializers are DRF's forms: they validate input and shape output.

Setup (1)

```
python -m pip install djangorestframework drf-spectacular
python manage.py startapp api
```

```
# settings/base.py
INSTALLED_APPS += ["rest_framework", "drf_spectacular", "api.apps.ApiConfig"]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES":
["rest_framework.authentication.SessionAuthentication"],
    "DEFAULT_PERMISSION_CLASSES":
["rest_framework.permissions.IsAuthenticated"],
    "DEFAULT_THROTTLE_CLASSES":
["rest_framework.throttling.UserRateThrottle"],         # 18
    "DEFAULT_THROTTLE_RATES": {"user": "1000/hour", "exports": "10/hour"},
    "EXCEPTION_HANDLER": "api.exceptions.exception_handler",
# 19
    "DEFAULT_SCHEMA_CLASS": "drf_spectacular.openapi.AutoSchema",
# 29
}

# django_project/urls.py
path("api/v2/", include("api.urls")),
```

```
# core/middleware.py — DRF authenticates /api/v2/ itself (session today,
tokens and API keys on Day 26)
class CRMLoginRequiredMiddleware(LoginRequiredMiddleware):
    EXEMPT_PREFIXES = ("/api/v2/",)

    def process_view(self, request, view_func, view_args, view_kwargs):
        if request.path.startswith(self.EXEMPT_PREFIXES):
            return None
        return super().process_view(request, view_func, view_args,
view_kwargs)
    # handle_no_permission() from Day 15 unchanged
```

Log in on the site, then open `/api/v2/leads/` in the browser: the browsable API renders the same JSON as HTML with forms.

A. Serializers (2–10, 27, 31)

```
# api/fields.py — 8
from rest_framework import serializers

from leads.models import normalize_uae_phone
from leads.validators import validate_uae_mobile


class UAEPhoneSerializerField(serializers.CharField):
    """Normalise first, so every validator (uniqueness included) sees
9715XXXXXXXX."""

    def __init__(self, **kwargs):
        kwargs["validators"] = [validate_uae_mobile,
*kwargs.get("validators", [])]
        kwargs.setdefault("max_length", 20)
        super().__init__(**kwargs)

    def to_internal_value(self, data):
        return normalize_uae_phone(super().to_internal_value(data))
```

Order matters: DRF runs `to_internal_value` → field validators → `validate_<field>()`. Normalising in `validate_phone()` would be too late — `UniqueValidator` would already have checked `+971 50 …` and found no match (the Day 13 #16 vs #36 lesson again).

```
# api/serializers.py
from django.contrib.auth import get_user_model
from rest_framework import serializers
from rest_framework.validators import UniqueValidator

from leads.models import Lead, Tag
from leads.templatetags.crm_filters import mask_phone
from .fields import UAEPhoneSerializerField


# ---- 2-3: plain serializers ----------------------------------------------
-
class LeadSummarySerializer(serializers.Serializer):
    id = serializers.IntegerField(read_only=True)
    name = serializers.CharField(max_length=120)
    status = serializers.CharField(read_only=True)
    created_at = serializers.DateTimeField(read_only=True)


class LeadInputSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=120)
    phone = UAEPhoneSerializerField()
    source = serializers.ChoiceField(choices=Lead.Source.choices,
default=Lead.Source.MANUAL)


# ---- 6 -------------------------------------------------------------------
----
class AgentMiniSerializer(serializers.ModelSerializer):
    class Meta:
        model = get_user_model()
        fields = ["id", "username", "first_name", "last_name"]


# ---- 27: list-level validation for bulk create ---------------------------
-
class LeadListSerializer(serializers.ListSerializer):
    def validate(self, attrs):
        phones = [item["phone"] for item in attrs]
        dupes = {p for p in phones if phones.count(p) > 1}
        if dupes:
            raise serializers.ValidationError(f"Duplicate phones in this
batch: {', '.join(sorted(dupes))}")
```

```
        return attrs


# ---- 4–10, 31 ----------------------------------------------------------
----
class LeadSerializer(serializers.ModelSerializer):
    url = serializers.HyperlinkedIdentityField(view_name="api:lead-detail")
# 9
    phone = UAEPhoneSerializerField(validators=[UniqueValidator(
# 8
        queryset=Lead.objects.all(), message="A lead with this phone number
already exists.")])
    masked_phone = serializers.SerializerMethodField()
# 5
    status_label = serializers.CharField(source="get_status_display",
read_only=True)
    assigned_to = AgentMiniSerializer(read_only=True)
# 6: nested on read
    assigned_to_id = serializers.PrimaryKeyRelatedField(
#    id on write
        source="assigned_to", write_only=True, required=False,
allow_null=True,
        queryset=get_user_model().objects.filter(role="agent",
is_active=True),
    )
    tags = serializers.SlugRelatedField(slug_field="name", many=True,
required=False, queryset=Tag.objects.all())
    is_hot = serializers.BooleanField(read_only=True)
# 10
    call_count = serializers.IntegerField(read_only=True, default=0)
# annotation, if present

    class Meta:
# 4
        model = Lead
        fields = ["id", "url", "name", "phone", "masked_phone", "email",
"status", "status_label",
                  "source", "expected_deposit", "assigned_to",
"assigned_to_id", "tags",
                  "is_hot", "call_count", "created_at", "updated_at"]
        read_only_fields = ["status", "created_at", "updated_at"]     #
status moves via /status/ only (22)
        list_serializer_class = LeadListSerializer                     # used
```

```
when many=True

    def __init__(self, *args, **kwargs):
# 31
        super().__init__(*args, **kwargs)
        request = self.context.get("request")
        if request is not None and getattr(request.user, "role", "") ==
"agent":
            self.fields.pop("expected_deposit")
            self.fields.pop("assigned_to_id")

    def get_masked_phone(self, obj):
        return mask_phone(obj.phone)

    def validate_name(self, value):
# 7: one field
        return value.strip().title()

    def validate(self, attrs):
# 7: whole object
        if attrs.get("expected_deposit") is not None and
attrs["expected_deposit"] < 0:
            raise serializers.ValidationError({"expected_deposit": "Deposit
can't be negative."})
        return attrs
```

```
# python manage.py shell — 2-3
from rest_framework.renderers import JSONRenderer
from api.serializers import LeadInputSerializer, LeadSummarySerializer
from leads.models import Lead

s = LeadSummarySerializer(Lead.objects.first())
s.data                                                  # {'id': 1, 'name':
'Aisha Khan', ...}
LeadSummarySerializer(Lead.objects.all()[:3], many=True).data
JSONRenderer().render(s.data)                           #
b'{"id":1,"name":"Aisha Khan",...}'

inp = LeadInputSerializer(data={"name": "Test", "phone": "12345"})
inp.is_valid(), inp.errors                              # (False, {'phone':
['12345 is not a UAE mobile ...']})
inp = LeadInputSerializer(data={"name": "Test", "phone": "+971 50 111 2233"})
inp.is_valid(), inp.validated_data                      # phone
'971501112233', source 'manual'
```

B. Views and policies (11–20)

```
# api/permissions.py — 16
from rest_framework.permissions import SAFE_METHODS, BasePermission
from leads.permissions import can_edit_lead


class CanEditLead(BasePermission):
    message = "Qualified leads are read-only for agents."

    def has_object_permission(self, request, view, obj):
        return request.method in SAFE_METHODS or can_edit_lead(request.user,
obj)


class IsTeamLeadOrManager(BasePermission):
    def has_permission(self, request, view):
        user = request.user
        return user.is_authenticated and (user.is_superuser or user.role in
{"team_lead", "manager", "admin"})
```

```
class CanSeeBrokers(BasePermission):
    def has_permission(self, request, view):
        return request.user.has_perm("brokers.view_broker")
```

```
# api/exceptions.py — 19
from rest_framework.views import exception_handler as drf_exception_handler
from core.context import _request_id


def exception_handler(exc, context):
    response = drf_exception_handler(exc, context)    # None = not an API
error → Django's 500 handling
    if response is not None:
        response.data = {
            "error": getattr(exc, "default_code", "error"),     # invalid,
not_found, permission_denied, throttled…
            "detail": response.data,
            "request_id": _request_id.get(),
        }
    return response
```

```
# api/views.py — 11-18, 20
from django.shortcuts import get_object_or_404
from rest_framework import generics, status
from rest_framework.decorators import api_view, throttle_classes
from rest_framework.permissions import DjangoModelPermissions,
IsAuthenticated
from rest_framework.response import Response
from rest_framework.throttling import UserRateThrottle
from rest_framework.views import APIView

from leads.models import Lead
from .permissions import CanEditLead
from .serializers import LeadSerializer


def scoped_leads(user):
    return (Lead.objects.visible_to(user).active()
            .select_related("assigned_to", "broker").prefetch_related("tags")
# 33: no N+1
            .with_call_stats().order_by("-created_at", "-pk"))
```

```
class LeadCountThrottle(UserRateThrottle):
    rate = "60/min"                  # its own rate (40's test patches it)


@api_view(["GET"])                                                  # 11
@throttle_classes([LeadCountThrottle])
def lead_count(request):
    return Response({"active":
Lead.objects.visible_to(request.user).active().count()})


# ---- 12–13: APIView versions (to see what the generics do for you) --------
---------
class LeadListAPIView(APIView):
    def get(self, request):
        leads = scoped_leads(request.user)[:50]
        return Response(LeadSerializer(leads, many=True, context={"request":
request}).data)

    def post(self, request):
        serializer = LeadSerializer(data=request.data, context={"request":
request})
        serializer.is_valid(raise_exception=True)      # 400 with field
errors, no try/except needed
        lead = serializer.save()
        return Response(LeadSerializer(lead, context={"request":
request}).data, status=status.HTTP_201_CREATED)


class LeadDetailAPIView(APIView):
    def get_object(self, request, pk):
        return get_object_or_404(scoped_leads(request.user), pk=pk)       #
invisible = 404, like the pages

    def get(self, request, pk):
        return Response(LeadSerializer(self.get_object(request, pk), context=
{"request": request}).data)

    def patch(self, request, pk):
        serializer = LeadSerializer(self.get_object(request, pk),
data=request.data, partial=True,
```

```
                                    context={"request": request})
        serializer.is_valid(raise_exception=True)
        serializer.save()
        return Response(serializer.data)

    def delete(self, request, pk):
        self.get_object(request, pk).delete()
        return Response(status=status.HTTP_204_NO_CONTENT)


# ---- 14-17: the generic versions actually routed ------------------------
-------
class LeadQuerysetMixin:
    serializer_class = LeadSerializer

    def get_queryset(self):
        return scoped_leads(self.request.user)


class LeadListCreateAPI(LeadQuerysetMixin, generics.ListCreateAPIView):     #
14
    permission_classes = [IsAuthenticated, DjangoModelPermissions]

    def perform_create(self, serializer):                                   #
15
        extra = {}
        if self.request.user.role == "agent":
            extra["assigned_to"] = self.request.user
        serializer.save(**extra)
    # 20: CreateModelMixin adds a Location header automatically because the
data has a "url" field


class LeadDetailAPI(LeadQuerysetMixin,
generics.RetrieveUpdateDestroyAPIView):   # 14, 16-17
    permission_classes = [IsAuthenticated, DjangoModelPermissions,
CanEditLead]
    # DjangoModelPermissions: POST→add, PUT/PATCH→change, DELETE→delete_lead
(only the Managers group has it)
```

```
# api/urls.py (Day 25 version — Day 26 replaces most of it with a router)
from django.urls import path
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView
from . import views

app_name = "api"
urlpatterns = [
    path("leads/count/", views.lead_count, name="lead-count"),
    path("leads/", views.LeadListCreateAPI.as_view(), name="lead-list"),
    path("leads/bulk/", views.LeadBulkCreateAPI.as_view(), name="lead-bulk"),
    path("leads/<int:pk>/", views.LeadDetailAPI.as_view(), name="lead-
detail"),
    path("leads/<int:pk>/calls/", views.LeadCallsAPI.as_view(), name="lead-
calls"),
    path("leads/<int:pk>/status/", views.LeadStatusAPI.as_view(), name="lead-
set-status"),
    path("clients/<uuid:ref>/", views.ClientDetailAPI.as_view(),
name="client-detail"),
    path("brokers/tree/", views.broker_tree, name="broker-tree"),
    path("reports/leaderboard/", views.leaderboard, name="report-
leaderboard"),
    path("reports/daily-leads/", views.daily_leads, name="report-daily-
leads"),
    path("search/", views.search, name="search"),
    path("schema/", SpectacularAPIView.as_view(), name="schema"),
# 29
    path("docs/", SpectacularSwaggerView.as_view(url_name="api:schema"),
name="docs"),
]
```

18. `UserRateThrottle` counts per user in the default cache (Redis from Day 24, so all workers share the count). Exports get `throttle_scope = "exports"` with `ScopedRateThrottle` in their `throttle_classes` (Day 26 #29).

C. Nexora CRM endpoints (21–30)

```
# api/serializers.py (add) — 21–23
from clients.models import Client, Deposit, TradingAccount
from leads.models import CallLog


class CallLogSerializer(serializers.ModelSerializer):                # 21
```

```
    agent = serializers.StringRelatedField(read_only=True)

    class Meta:
        model = CallLog
        fields = ["id", "started_at", "agent", "outcome", "duration_seconds",
"notes"]
        read_only_fields = ["started_at"]

    def validate(self, attrs):
        if attrs.get("outcome") != CallLog.Outcome.ANSWERED and
attrs.get("duration_seconds"):
            raise serializers.ValidationError({"duration_seconds": "Only
answered calls have talk time."})
        return attrs


class StatusChangeSerializer(serializers.Serializer):                   # 22
    ALLOWED = {
        Lead.Status.NEW: {Lead.Status.CONTACTED, Lead.Status.LOST},
        Lead.Status.CONTACTED: {Lead.Status.QUALIFIED, Lead.Status.LOST},
        Lead.Status.LOST: {Lead.Status.CONTACTED},          # re-open
        Lead.Status.QUALIFIED: set(),                       # leaves only
through conversion
    }
    status = serializers.ChoiceField(choices=Lead.Status.choices)
    notes = serializers.CharField(required=False, allow_blank=True)

    def validate(self, attrs):
        lead = self.context["lead"]
        new = attrs["status"]
        if new not in self.ALLOWED.get(lead.status, set()):
            raise serializers.ValidationError({"status": f"Can't move a
{lead.status} lead to {new}."})
        if new == Lead.Status.LOST and not (attrs.get("notes") or
lead.notes).strip():
            raise serializers.ValidationError({"notes": "Give a reason when
marking a lead lost."})
        return attrs


class DepositSerializer(serializers.ModelSerializer):
    class Meta:
        model = Deposit
```

```
        fields = ["id", "amount", "method", "created_at"]


class TradingAccountSerializer(serializers.ModelSerializer):
    deposits = DepositSerializer(many=True, read_only=True)

    class Meta:
        model = TradingAccount
        fields = ["number", "platform", "currency", "leverage", "balance",
"opened_at", "deposits"]


class ClientSerializer(serializers.ModelSerializer):                    # 23
    broker = serializers.StringRelatedField()
    accounts = TradingAccountSerializer(many=True, read_only=True)
    total_deposits = serializers.DecimalField(max_digits=14,
decimal_places=2, read_only=True, default=0)

    class Meta:
        model = Client
        fields = ["ref", "name", "email", "account_number", "account_type",
"kyc_status",
                  "broker", "total_deposits", "accounts"]


class SearchLeadSerializer(serializers.ModelSerializer):                # 26
    masked_phone = serializers.SerializerMethodField()

    class Meta:
        model = Lead
        fields = ["id", "name", "masked_phone", "status"]

    def get_masked_phone(self, obj):
        return mask_phone(obj.phone)


class DaysParams(serializers.Serializer):                               # 36
    days = serializers.IntegerField(min_value=1, max_value=365, default=30)
```

```
# api/views.py (add) — 21–27
from datetime import timedelta

from django.db import transaction
```

```
from django.db.models import Count
from django.db.models.functions import TruncDate
from django.utils import timezone
from rest_framework.decorators import permission_classes
from rest_framework.exceptions import PermissionDenied, ValidationError

from brokers.tree import load_tree
from clients.models import Client
from core.search import search_everything
from leads.permissions import can_edit_lead
from leads.services import log_call
from reports.cached import report_data
from .permissions import CanSeeBrokers, IsTeamLeadOrManager
from .serializers import (CallLogSerializer, ClientSerializer, DaysParams,
SearchLeadSerializer,
                          StatusChangeSerializer)


class LeadCallsAPI(generics.ListCreateAPIView):                           # 21
    serializer_class = CallLogSerializer

    def get_lead(self):
        if not hasattr(self, "_lead"):
            self._lead =
get_object_or_404(Lead.objects.visible_to(self.request.user),
pk=self.kwargs["pk"])
        return self._lead

    def get_queryset(self):
        return self.get_lead().calls.select_related("agent")

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        call = log_call(self.get_lead(), request.user,
**serializer.validated_data)   # the service, not .save()
        return Response(self.get_serializer(call).data,
status=status.HTTP_201_CREATED)


class LeadStatusAPI(APIView):                                             # 22
    def post(self, request, pk):
        lead = get_object_or_404(scoped_leads(request.user), pk=pk)
```

```
        if not can_edit_lead(request.user, lead):
            raise PermissionDenied("Qualified leads are read-only for
agents.")
        serializer = StatusChangeSerializer(data=request.data, context=
{"lead": lead})
        serializer.is_valid(raise_exception=True)
        lead.status = serializer.validated_data["status"]
        if serializer.validated_data.get("notes"):
            lead.notes = serializer.validated_data["notes"]
        lead.save(update_fields=["status", "notes", "updated_at"])     # Day
20 signals record the history
        return Response(LeadSerializer(lead, context={"request":
request}).data)


class ClientDetailAPI(generics.RetrieveAPIView):                          # 23
    serializer_class = ClientSerializer
    lookup_field = "ref"

    def get_queryset(self):
        return
(Client.objects.visible_to(self.request.user).with_total_deposits()

.select_related("broker").prefetch_related("accounts__deposits"))


def broker_node(broker, children):
    return {"code": broker.code, "name": broker.name, "tier": broker.tier,
            "children": [broker_node(child, children) for child in
children[broker.pk]]}


@api_view(["GET"])                                                        # 24
@permission_classes([IsAuthenticated, CanSeeBrokers])
def broker_tree(request):
    _, children = load_tree()                        # cached (Day 24)
    return Response([broker_node(top, children) for top in children[None]])


@api_view(["GET"])                                                        # 25
@permission_classes([IsAuthenticated, IsTeamLeadOrManager])
def leaderboard(request):
    rows = report_data(request.user)["leaderboard"]     # same scope + cache
```

```
as the page
    return Response([{"agent_id": a.pk, "agent": str(a), "leads": a.leads_n,
"qualified": a.qualified_n,
                     "calls": a.calls_n, "conversion": round(a.conversion,
1)} for a in rows])


@api_view(["GET"])                                                  # 25,
36
@permission_classes([IsAuthenticated, IsTeamLeadOrManager])
def daily_leads(request):
    params = DaysParams(data=request.query_params)
    params.is_valid(raise_exception=True)            # ?days=abc or 999 → 400
in the standard error shape
    since = timezone.now() - timedelta(days=params.validated_data["days"])
    rows =
(Lead.objects.visible_to(request.user).filter(created_at__gte=since)

.annotate(day=TruncDate("created_at")).values("day").annotate(n=Count("id")).
order_by("day"))
    return Response(list(rows))


@api_view(["GET"])                                                  # 26
def search(request):
    q = request.query_params.get("q", "").strip()
    if len(q) < 2:
        raise ValidationError({"q": "Use at least 2 characters."})
    r = search_everything(request.user, q, limit=10)
    return Response({
        "leads": {"total": r["leads"]["total"], "items":
SearchLeadSerializer(r["leads"]["items"], many=True).data},
        "clients": {"total": r["clients"]["total"],
                    "items": [{"ref": c.ref, "name": c.name,
"account_number": c.account_number}
                              for c in r["clients"]["items"]]},
        "accounts": {"total": r["accounts"]["total"],
                     "items": [{"number": a.number, "client_ref":
a.client.ref} for a in r["accounts"]["items"]]},
        "brokers": {"total": r["brokers"]["total"],
                    "items": [{"code": b.code, "name": b.name} for b in
r["brokers"]["items"]]},
    })
```

```
class LeadBulkCreateAPI(APIView):                                    # 27
    permission_classes = [IsAuthenticated, DjangoModelPermissions]
    queryset = Lead.objects.none()           # DjangoModelPermissions reads
the model from here

    def post(self, request):
        if not isinstance(request.data, list) or not 1 <= len(request.data)
<= 500:
            raise ValidationError("Send a JSON list of 1–500 leads.")
        serializer = LeadSerializer(data=request.data, many=True, context=
{"request": request})
        serializer.is_valid(raise_exception=True)        # errors come back
as a list, one entry per input item
        extra = {"assigned_to": request.user} if request.user.role == "agent"
else {}
        with transaction.atomic():                       # all or nothing
            leads = serializer.save(**extra)
        return Response({"created": len(leads)},
status=status.HTTP_201_CREATED)
```

28. The version lives in the URL (`/api/v2/`): it's visible in logs and easy to route at a proxy. `/api/v1/` keeps its old shape because integrations built on it would break if it changed; it's retired only after every client has moved. Day 26 #27 shows header-based versions inside v2.

29.

```
python manage.py spectacular --file schema.yml --validate      # writes and
checks the OpenAPI document
# browse: /api/v2/docs/ (Swagger UI)
```

Swagger UI loads its JS and CSS from `cdn.jsdelivr.net`. Day 21's CSP allows that for scripts but not styles: add `https://cdn.jsdelivr.net` to `style-src`, or `pip install drf-spectacular[sidecar]` to serve the files yourself.

30.

```
# core/middleware.py — LeadAccessAuditMiddleware
AUDITED = {"leads:detail": "lead_id", "clients:detail": "ref",
"clients:kyc_download": "pk",
           "api:lead-detail": "pk", "api:client-detail": "ref"}          #
API reads are audited too
```

D. Challenge (32–35)

31 is the `__init__` in `LeadSerializer` (section A). **33** is `scoped_leads()`'s `select_related` / `prefetch_related` — check it with `max_queries` from Day 24.

```
# api/serializers.py — 32
class TradingAccountWriteSerializer(serializers.ModelSerializer):
    class Meta:
        model = TradingAccount
        fields = ["number", "platform", "currency", "leverage"]


class ClientCreateSerializer(serializers.ModelSerializer):
    first_account = TradingAccountWriteSerializer(write_only=True)

    class Meta:
        model = Client
        fields = ["ref", "name", "email", "account_number", "account_type",
"broker", "first_account"]
        read_only_fields = ["ref"]

    def create(self, validated_data):
        account = validated_data.pop("first_account")
        with transaction.atomic():                    # client and account
together, or neither
            client = Client.objects.create(**validated_data)
            TradingAccount.objects.create(client=client, **account)
        return client


# api/views.py
class ClientCreateAPI(generics.CreateAPIView):
    serializer_class = ClientCreateSerializer
    permission_classes = [IsAuthenticated, DjangoModelPermissions]    # needs
clients.add_client
```

```
    queryset = Client.objects.none()

# api/urls.py: path("clients/", views.ClientCreateAPI.as_view(),
name="client-create"),
```

```
# api/renderers.py — 34
import csv
import io
from rest_framework.renderers import BaseRenderer


class LeadCSVRenderer(BaseRenderer):
    media_type = "text/csv"
    format = "csv"                          # picked by ?format=csv
    charset = "utf-8"
    COLUMNS = ["id", "name", "masked_phone", "status", "source",
"call_count", "created_at"]

    def render(self, data, accepted_media_type=None, renderer_context=None):
        rows = data.get("results", data) if isinstance(data, dict) else data
        if not isinstance(rows, list):
            rows = []                       # an error body — return just the
header
        buffer = io.StringIO()
        writer = csv.writer(buffer)
        writer.writerow(self.COLUMNS)
        for row in rows:
            writer.writerow([row.get(column, "") for column in self.COLUMNS])
        return "\ufeff" + buffer.getvalue()


# api/views.py — LeadListCreateAPI
from rest_framework.settings import api_settings
from .renderers import LeadCSVRenderer

renderer_classes = [*api_settings.DEFAULT_RENDERER_CLASSES, LeadCSVRenderer]
```

```
# settings/prod.py — 35: no browsable API in production
REST_FRAMEWORK = {**REST_FRAMEWORK, "DEFAULT_RENDERER_CLASSES":
["rest_framework.renderers.JSONRenderer"]}
```

Tests (37–40)

```
# api/tests.py
from unittest import mock

from django.core.cache import cache
from django.urls import reverse
from rest_framework.test import APITestCase

from core.factories import AgentFactory, LeadFactory
from leads.models import Lead
from .views import LeadCountThrottle


class LeadAPITests(APITestCase):          # self.client is an APIClient
    def setUp(self):
        cache.clear()                     # throttle counters live in the
cache

    def test_agent_list_is_scoped(self):                                  # 37
        a1, a2 = AgentFactory(), AgentFactory()
        mine = LeadFactory(assigned_to=a1)
        LeadFactory(assigned_to=a2)
        self.client.force_authenticate(a1)
        ids = [row["id"] for row in self.client.get(reverse("api:lead-
list")).data]
        self.assertEqual(ids, [mine.pk])

    def test_create_and_duplicate(self):                                  # 38
        agent = AgentFactory()
        self.client.force_authenticate(agent)
        url = reverse("api:lead-list")
        r1 = self.client.post(url, {"name": "api lead", "phone": "+971 50 888
0001", "source": "google"}, format="json")
        self.assertEqual(r1.status_code, 201)
        self.assertEqual(r1.data["masked_phone"], "9715****0001")
        self.assertIn("Location", r1)
        self.assertEqual(Lead.objects.get(phone="971508880001").assigned_to,
agent)
        r2 = self.client.post(url, {"name": "dup", "phone": "0508880001",
"source": "google"}, format="json")
        self.assertEqual(r2.status_code, 400)
        self.assertIn("already exists", str(r2.data["detail"]["phone"]))
```

```
    def test_invalid_transition(self):                                  # 39
        agent = AgentFactory()
        lead = LeadFactory(assigned_to=agent)
        self.client.force_authenticate(agent)
        response = self.client.post(reverse("api:lead-set-status", args=
[lead.pk]), {"status": "qualified"}, format="json")
        self.assertEqual(response.status_code, 400)
        self.assertEqual(response.data["error"], "invalid")

    def test_throttle(self):                                            # 40
        self.client.force_authenticate(AgentFactory())
        with mock.patch.object(LeadCountThrottle, "rate", "2/min"):
            codes = [self.client.get(reverse("api:lead-count")).status_code
for _ in range(3)]
        self.assertEqual(codes, [200, 200, 429])
```

`force_authenticate` skips authentication (and CSRF) entirely — right for testing view logic; Day 27 #38 tests the real session + CSRF path.

```
python manage.py test api --settings=django_project.settings.test
git add -A && git commit -m "day 25"
```

Day 26 — Django REST Framework 2

Goal: production-shaped API. Day 25's views collapse into ViewSets behind a router; clients authenticate with tokens, JWTs or API keys instead of a browser session; lists filter and paginate; and two real integrations arrive — the dialer pushing calls in, and payment/broker webhooks signed with HMAC.

URL names stay the same (`api:lead-list`, `api:lead-detail`, `api:lead-set-status`, `api:lead-calls`), so Day 25's tests keep working — except list responses become paginated (`data["results"]`).

A. Drills (1–10) — ViewSets and routers

- [ ] **1.** `LeadViewSet(ModelViewSet)` registered on a `DefaultRouter` with basename `lead`.
- [ ] **2.** `ReadOnlyModelViewSet` for clients (`lookup_field="ref"`) and calls.
- [ ] **3.** `@action`s: `status` and `calls` (detail), `mine` and `next` (list).
- [ ] **4.** `get_serializer_class()` per action (a lighter row serializer for `list`).
- [ ] **5.** `get_queryset()` per action: annotate call stats only where shown.
- [ ] **6.** Generated URL names; `reverse("api:lead-set-status", args=[pk])`.
- [ ] **7.** `url_path` / `url_name` on actions.
- [ ] **8.** A `GenericViewSet` built from mixins (list + retrieve + create, no delete) for tags.
- [ ] **9.** `get_permissions()` per action.
- [ ] **10.** `CRMAPIViewMixin`: after DRF authenticates, publish the user to Day 20's current-user contextvar (token users get audited too) and log each call.

B. Core (11–20) — auth, filtering, pagination

- [ ] **11.** `TokenAuthentication`: `rest_framework.authtoken`, a throttled `obtain_auth_token` endpoint, `Authorization: Token …`.
- [ ] **12.** `drf_create_token` and token rotation for a user.
- [ ] **13.** JWT with djangorestframework-simplejwt: obtain / refresh / lifetimes.
- [ ] **14.** `ApiKey` model (hashed keys) + `ApiKeyAuthentication` reading `X-API-Key`; `create_api_key` command shows the key once.
- [ ] **15.** `StandardPagination` (page number, `page_size` param, totals) as the default.

- [ ] **16.** `CursorPagination` for the calls feed.
- [ ] **17.** `LimitOffsetPagination` on clients.
- [ ] **18.** `DjangoFilterBackend` with Day 17's `LeadFilter` on leads; `SearchFilter` + `OrderingFilter` on clients.
- [ ] **19.** `filterset_fields` shortcut on calls (`outcome__in`, date range).
- [ ] **20.** CORS for a separate BI dashboard origin (django-cors-headers, `/api/v2/` only).

C. Nexora CRM (21–30) — integrations

- [ ] **21.** Dialer endpoint `POST /api/v2/dialer/calls/` (API key): match the lead by phone, log the call with the dialer's timestamp.
- [ ] **22.** Outgoing webhook: when a lead converts, POST JSON to the broker's `webhook_url`, signed with HMAC-SHA256.
- [ ] **23.** Incoming webhook `POST /api/v2/webhooks/deposits/`: verify the HMAC signature, create the deposit once per event id.
- [ ] **24.** `IdempotencyRecord`: a repeated `Idempotency-Key` returns the first response instead of logging twice.
- [ ] **25.** Per-API-key throttle.
- [ ] **26.** Schema polish: `@extend_schema` tags, summaries, request/response types.
- [ ] **27.** Header versioning inside v2 (`Accept: application/json; version=3`) switching the serializer.
- [ ] **28.** `?fields=id,name,status` sparse fieldsets on GET.
- [ ] **29.** Streaming CSV export as a ViewSet action, scoped throttle `exports`.
- [ ] **30.** Re-check scoping: agents can't list other agents' clients or calls.

D. Challenge (31–40)

- [ ] **31.** Throttle keyed by API key, ignored for normal users.
- [ ] **32.** `Link` / `X-Total-Count` header pagination (GitHub style).
- [ ] **33.** Optimistic locking: `ETag` on retrieve, `If-Match` on update → 412 when stale.
- [ ] **34.** Bulk status change `PATCH /api/v2/leads/bulk-status/` with per-item results.
- [ ] **35.** Validate the OpenAPI schema in CI (`spectacular --validate --fail-on-warn`).
- [ ] **36.** Update Day 25's tests for pagination; `api_client_for(user)` helper.
- [ ] **37.** Test: token auth works with no session.

- [ ] **38.** Test: two cursor pages never overlap.
- [ ] **39.** Test: the same `Idempotency-Key` twice logs one call.
- [ ] **40.** Test: a bad webhook signature is rejected; a good one creates the deposit.

Solutions

A ViewSet is one class holding `list`, `retrieve`, `create`, `update`, `partial_update`, `destroy` (+ your `@action`s); a router turns it into URLs named `<basename>-list`, `<basename>-detail`, `<basename>-<action>`.

A. ViewSets and routers (1–10)

```
# api/serializers.py (add) — 4, 28
class DynamicFieldsMixin:                                                    #
28: ?fields=id,name,status
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        request = self.context.get("request")
        wanted = request.query_params.get("fields") if request is not None
and request.method == "GET" else None
        if wanted:
            for name in set(self.fields) - set(wanted.split(",")):
                self.fields.pop(name)


class LeadRowSerializer(DynamicFieldsMixin, serializers.ModelSerializer):   #
4: light, for lists
    url = serializers.HyperlinkedIdentityField(view_name="api:lead-detail")
    masked_phone = serializers.SerializerMethodField()
    assigned_to = serializers.StringRelatedField()
    call_count = serializers.IntegerField(read_only=True, default=0)
    last_call_at = serializers.DateTimeField(read_only=True, default=None)

    class Meta:
        model = Lead
        fields = ["id", "url", "name", "masked_phone", "status", "source",
"assigned_to",
                  "call_count", "last_call_at", "created_at"]

    def get_masked_phone(self, obj):
        return mask_phone(obj.phone)
```

```
class TagSerializer(serializers.ModelSerializer):
    class Meta:
        model = Tag
        fields = ["id", "name", "color"]


class CallFeedSerializer(serializers.ModelSerializer):
    lead = serializers.PrimaryKeyRelatedField(read_only=True)
    lead_name = serializers.CharField(source="lead.name", read_only=True)
    agent = serializers.StringRelatedField()

    class Meta:
        model = CallLog
        fields = ["id", "started_at", "lead", "lead_name", "agent",
"outcome", "duration_seconds"]

# LeadSerializer becomes `class LeadSerializer(DynamicFieldsMixin,
serializers.ModelSerializer)` too
```

```
# api/viewsets.py — 1-5, 7-10
import logging

from django_filters.rest_framework import DjangoFilterBackend
from rest_framework import filters, mixins, status, viewsets
from rest_framework.decorators import action
from rest_framework.pagination import CursorPagination, LimitOffsetPagination
from rest_framework.permissions import BasePermission,
DjangoModelPermissions, IsAuthenticated
from rest_framework.response import Response
from rest_framework.settings import api_settings

from clients.models import Client
from core.context import _current_user
from leads.filters import LeadFilter
from leads.models import CallLog, Lead, Tag
from leads.services import log_call, next_lead_to_dial
from .permissions import CanEditLead
from .renderers import LeadCSVRenderer
from .serializers import (CallFeedSerializer, CallLogSerializer,
ClientCreateSerializer, ClientSerializer,
                          LeadRowSerializer, LeadSerializer,
```

```
StatusChangeSerializer, TagSerializer)

logger = logging.getLogger("crm.api")


class CRMAPIViewMixin:                                                  # 10
    def initial(self, request, *args, **kwargs):
        super().initial(request, *args, **kwargs)        # authentication,
permissions, throttles
        # middleware only knew the session user; token/JWT/API-key users
exist from here on
        self._user_token = _current_user.set(request.user)

    def finalize_response(self, request, response, *args, **kwargs):
        token = getattr(self, "_user_token", None)
        if token is not None:
            _current_user.reset(token)
            self._user_token = None
        logger.info("api call", extra={"user_id": getattr(request.user, "pk",
None), "method": request.method,
                                       "path": request.path, "status":
response.status_code})
        return super().finalize_response(request, response, *args, **kwargs)


class IsAgent(BasePermission):
    def has_permission(self, request, view):
        return getattr(request.user, "role", "") == "agent"


class LeadViewSet(CRMAPIViewMixin, viewsets.ModelViewSet):              # 1
    permission_classes = [IsAuthenticated, DjangoModelPermissions,
CanEditLead]
    filter_backends = [DjangoFilterBackend]                               #
18: Day 17's LeadFilter
    filterset_class = LeadFilter
    lookup_value_regex = r"\d+"                # so /leads/bulk-status/ etc.
never look like a pk
    renderer_classes = [*api_settings.DEFAULT_RENDERER_CLASSES,
LeadCSVRenderer]

    def get_queryset(self):                                                 # 5
        qs = (Lead.objects.visible_to(self.request.user).active()
```

```
              .select_related("assigned_to",
"broker").prefetch_related("tags"))
        if self.action in {"list", "retrieve", "mine"}:
            qs = qs.with_call_stats()             # only where the numbers
are shown
        return qs.order_by("-created_at", "-pk")

    def get_serializer_class(self):                                        # 4
        return {
            "list": LeadRowSerializer, "mine": LeadRowSerializer,
            "set_status": StatusChangeSerializer, "calls": CallLogSerializer,
        }.get(self.action, LeadSerializer)

    def get_permissions(self):                                             # 9
        if self.action == "next_lead":
            return [IsAuthenticated(), IsAgent()]
        return super().get_permissions()

    def perform_create(self, serializer):
        serializer.save(**({"assigned_to": self.request.user} if
self.request.user.role == "agent" else {}))

    @action(detail=True, methods=["post"], url_path="status", url_name="set-
status")    # 3, 7
    def set_status(self, request, pk=None):
        lead = self.get_object()                 # scoped queryset +
CanEditLead in one call
        serializer = StatusChangeSerializer(data=request.data, context=
{"lead": lead})
        serializer.is_valid(raise_exception=True)
        lead.status = serializer.validated_data["status"]
        if serializer.validated_data.get("notes"):
            lead.notes = serializer.validated_data["notes"]
        lead.save(update_fields=["status", "notes", "updated_at"])
        return Response(LeadSerializer(lead,
context=self.get_serializer_context()).data)

    @action(detail=True, methods=["get", "post"])
    def calls(self, request, pk=None):
        lead = self.get_object()
        if request.method == "GET":
            page = self.paginate_queryset(lead.calls.select_related("agent"))
            return self.get_paginated_response(CallLogSerializer(page,
```

```
many=True).data)
        serializer = CallLogSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        call = log_call(lead, request.user, **serializer.validated_data)
        return Response(CallLogSerializer(call).data,
status=status.HTTP_201_CREATED)

    @action(detail=False)
    def mine(self, request):
        qs =
self.filter_queryset(self.get_queryset().filter(assigned_to=request.user))
        page = self.paginate_queryset(qs)
        return self.get_paginated_response(self.get_serializer(page,
many=True).data)

    @action(detail=False, url_path="next", url_name="next")
    def next_lead(self, request):
        lead = next_lead_to_dial(request.user)            # Day 10: locks the
row it hands out
        if lead is None:
            return Response(status=status.HTTP_204_NO_CONTENT)
        return Response(LeadSerializer(lead,
context=self.get_serializer_context()).data)


class ClientViewSet(CRMAPIViewMixin, mixins.CreateModelMixin,
viewsets.ReadOnlyModelViewSet):   # 2, 17-18
    lookup_field = "ref"
    lookup_value_regex = r"[0-9a-f-]{36}"
    pagination_class = LimitOffsetPagination                              #
17: ?limit=50&offset=100
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ["name", "email", "account_number", "accounts__number"]
# ?search=
    ordering_fields = ["name", "date_joined", "total_deposits"]          # ?
ordering=-total_deposits
    ordering = ["name"]
    permission_classes = [IsAuthenticated, DjangoModelPermissions]

    def get_queryset(self):                                               #
30: scoped like the pages
        return
(Client.objects.visible_to(self.request.user).with_total_deposits()
```

```
.select_related("broker").prefetch_related("accounts__deposits"))

    def get_serializer_class(self):
        return ClientCreateSerializer if self.action == "create" else
ClientSerializer   # Day 25 #32 moves here


class CallPagination(CursorPagination):                                  #
16
    page_size = 50
    ordering = ("-started_at", "-id")            # id last: a unique tie-
breaker keeps pages exact


class CallViewSet(CRMAPIViewMixin, viewsets.ReadOnlyModelViewSet):
    serializer_class = CallFeedSerializer
    pagination_class = CallPagination
    filter_backends = [DjangoFilterBackend]
    filterset_fields = {                                                 #
19
        "outcome": ["exact", "in"],              # ?
outcome__in=busy,no_answer
        "agent": ["exact"],
        "started_at": ["date__gte", "date__lte"],   # ?
started_at__date__gte=2026-10-01
    }

    def get_queryset(self):
        return
CallLog.objects.visible_to(self.request.user).select_related("lead", "agent")


class TagViewSet(CRMAPIViewMixin, mixins.ListModelMixin,
mixins.RetrieveModelMixin,
                 mixins.CreateModelMixin, viewsets.GenericViewSet):     # 8:
no update, no delete
    queryset = Tag.objects.all()
    serializer_class = TagSerializer
    permission_classes = [IsAuthenticated, DjangoModelPermissions]      #
create needs leads.add_tag (Managers)
```

```
# api/urls.py — 1, 6 (explicit paths first: the router's detail patterns
would swallow "leads/bulk/")
from django.urls import include, path
from rest_framework.routers import DefaultRouter
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView
from rest_framework_simplejwt.views import TokenObtainPairView,
TokenRefreshView

from . import integrations, views, viewsets
from .auth_views import ThrottledObtainAuthToken

router = DefaultRouter()
router.register("leads", viewsets.LeadViewSet, basename="lead")
router.register("clients", viewsets.ClientViewSet, basename="client")
router.register("calls", viewsets.CallViewSet, basename="call")
router.register("tags", viewsets.TagViewSet, basename="tag")

app_name = "api"
urlpatterns = [
    path("auth/token/", ThrottledObtainAuthToken.as_view(), name="token"),
    path("auth/jwt/", TokenObtainPairView.as_view(), name="jwt"),
    path("auth/jwt/refresh/", TokenRefreshView.as_view(), name="jwt-
refresh"),
    path("leads/count/", views.lead_count, name="lead-count"),
    path("leads/bulk/", views.LeadBulkCreateAPI.as_view(), name="lead-bulk"),
    path("dialer/calls/", integrations.DialerCallAPI.as_view(), name="dialer-
calls"),
    path("webhooks/deposits/", integrations.DepositWebhookAPI.as_view(),
name="deposit-webhook"),
    path("brokers/tree/", views.broker_tree, name="broker-tree"),
    path("reports/leaderboard/", views.leaderboard, name="report-
leaderboard"),
    path("reports/daily-leads/", views.daily_leads, name="report-daily-
leads"),
    path("search/", views.search, name="search"),
    path("schema/", SpectacularAPIView.as_view(), name="schema"),
    path("docs/", SpectacularSwaggerView.as_view(url_name="api:schema"),
name="docs"),
    path("", include(router.urls)),      # leads/, leads/{pk}/,
```

```
leads/{pk}/status/, leads/mine/, clients/{ref}/ ...
]
# Day 25's LeadListCreateAPI, LeadDetailAPI, LeadCallsAPI, LeadStatusAPI,
ClientDetailAPI, ClientCreateAPI are retired.
```

```
# python manage.py shell — 6
from django.urls import reverse
reverse("api:lead-list")                    # '/api/v2/leads/'
reverse("api:lead-detail", args=[5])        # '/api/v2/leads/5/'
reverse("api:lead-set-status", args=[5])    # '/api/v2/leads/5/status/'
reverse("api:lead-mine"), reverse("api:lead-next"), reverse("api:client-
detail", args=["<uuid>"])
```

B. Authentication, pagination, filtering (11–20)

```
python -m pip install djangorestframework-simplejwt django-cors-headers
```

```
# settings/base.py
from datetime import timedelta

INSTALLED_APPS += ["rest_framework.authtoken", "corsheaders"]

REST_FRAMEWORK.update({
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.TokenAuthentication",   # first →
anonymous calls get 401 + WWW-Authenticate
        "rest_framework_simplejwt.authentication.JWTAuthentication",
        "api.authentication.ApiKeyAuthentication",
        "rest_framework.authentication.SessionAuthentication",  # the browser
UI (CSRF enforced)
    ],
    "DEFAULT_PAGINATION_CLASS": "api.pagination.StandardPagination",   # 15
    "PAGE_SIZE": 25,
    "DEFAULT_FILTER_BACKENDS":
["django_filters.rest_framework.DjangoFilterBackend"],
    "DEFAULT_THROTTLE_RATES": {"user": "1000/hour", "exports": "10/hour",
"login": "10/min", "apikey": "600/min"},
})

SIMPLE_JWT = {                                                          # 13
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=15),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=1),
```

```
    "ROTATE_REFRESH_TOKENS": True,
    "AUTH_HEADER_TYPES": ("Bearer",),
}

MIDDLEWARE.insert(MIDDLEWARE.index("django.middleware.common.CommonMiddleware
"),   # 20: before CommonMiddleware
                  "corsheaders.middleware.CorsMiddleware")
CORS_ALLOWED_ORIGINS = env.list("CORS_ALLOWED_ORIGINS", default=[])   # e.g.
https://bi.nexora.ae
CORS_URLS_REGEX = r"^/api/v2/.*$"                                       #
nothing else is cross-origin
```

```
python manage.py migrate                        # authtoken table
```

11–13.

```
# api/auth_views.py — 11
from rest_framework.authtoken.views import ObtainAuthToken
from rest_framework.throttling import ScopedRateThrottle


class ThrottledObtainAuthToken(ObtainAuthToken):
    # this endpoint skips Day 15's lockout form, so it gets its own brake
    throttle_classes = [ScopedRateThrottle]
    throttle_scope = "login"
```

```
# 11: get a token, then use it (no cookies involved)
curl -X POST -d "username=agent1&password=nexora123"
http://127.0.0.1:8000/api/v2/auth/token/
curl -H "Authorization: Token <key>" http://127.0.0.1:8000/api/v2/leads/mine/

# 12
python manage.py drf_create_token agent1        # create (or show) agent1's
token
python manage.py drf_create_token -r agent1     # rotate: old token stops
working

# 13: JWT — short-lived access token + refresh token
curl -X POST -H "Content-Type: application/json" -d "
{\"username\":\"agent1\",\"password\":\"nexora123\"}" \
     http://127.0.0.1:8000/api/v2/auth/jwt/                 # → {"refresh":
"...", "access": "..."}
curl -H "Authorization: Bearer <access>" http://127.0.0.1:8000/api/v2/leads/
curl -X POST -H "Content-Type: application/json" -d "{\"refresh\":\"
<refresh>\"}" http://127.0.0.1:8000/api/v2/auth/jwt/refresh/
```

DRF tokens never expire until rotated; JWTs expire on their own (15 minutes here) but can't be revoked early without extra work. Tokens suit integrations; JWTs suit mobile/SPA logins.

14.

```
# api/models.py
from django.conf import settings
from django.db import models


class ApiKey(models.Model):
    name = models.CharField(max_length=60)
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name="api_keys")
    prefix = models.CharField(max_length=8, db_index=True)      # shown in
the admin to tell keys apart
    key_hash = models.CharField(max_length=64, unique=True)     # the raw key
is never stored
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    last_used_at = models.DateTimeField(null=True, blank=True)
```

```
    def __str__(self):
        return f"{self.name} ({self.prefix}…)"


# api/keys.py
import hashlib
import secrets
from .models import ApiKey


def hash_key(raw):
    return hashlib.sha256(raw.encode()).hexdigest()


def create_api_key(name, user):
    raw = "nx_" + secrets.token_urlsafe(32)
    key = ApiKey.objects.create(name=name, user=user, prefix=raw[:8],
key_hash=hash_key(raw))
    return raw, key


# api/authentication.py
from django.utils import timezone
from rest_framework.authentication import BaseAuthentication
from rest_framework.exceptions import AuthenticationFailed
from .keys import hash_key
from .models import ApiKey


class ApiKeyAuthentication(BaseAuthentication):
    def authenticate(self, request):
        raw = request.headers.get("X-API-Key")
        if not raw:
            return None                    # not ours — let the next
authenticator try
        try:
            api_key =
ApiKey.objects.select_related("user").get(key_hash=hash_key(raw),
is_active=True)
        except ApiKey.DoesNotExist:
            raise AuthenticationFailed("Invalid API key.")

ApiKey.objects.filter(pk=api_key.pk).update(last_used_at=timezone.now())
```

```
        return api_key.user, api_key        # → request.user, request.auth

    def authenticate_header(self, request):
        return "X-API-Key"
```

```
# api/management/commands/create_api_key.py
from django.contrib.auth import get_user_model
from django.core.management.base import BaseCommand
from api.keys import create_api_key


class Command(BaseCommand):
    help = "Create an API key for an integration (printed once — store it in
the integration's secrets)"

    def add_arguments(self, parser):
        parser.add_argument("name")
        parser.add_argument("--user", default="integration-bot")

    def handle(self, *args, name, user, **options):
        bot, created = get_user_model().objects.get_or_create(
            username=user, defaults={"role": "manager", "first_name":
"Integration"})   # sees all leads
        if created:
            bot.set_unusable_password()      # can never log in through the
website
            bot.save(update_fields=["password"])
        raw, key = create_api_key(name, bot)
        self.stdout.write(self.style.SUCCESS(f"{key}: {raw}"))
        self.stdout.write(self.style.WARNING("Copy it now — it can't be shown
again."))
```

15.

```
# api/pagination.py
from rest_framework.pagination import PageNumberPagination
from rest_framework.response import Response


class StandardPagination(PageNumberPagination):
    page_size = 25
    page_size_query_param = "page_size"      # ?page_size=100
    max_page_size = 200

    def get_paginated_response(self, data):
        return Response({
            "count": self.page.paginator.count,
            "num_pages": self.page.paginator.num_pages,
            "page": self.page.number,
            "next": self.get_next_link(),
            "previous": self.get_previous_link(),
            "results": data,
        })
```

16–19 are in `api/viewsets.py` above: `CallPagination` (cursor), `LimitOffsetPagination` on clients, `LeadFilter` on leads (`?status=new&agent=3&o=-deposit`), `SearchFilter` / `OrderingFilter` on clients, `filterset_fields` on calls. Cursor pages can't jump to "page 7", but they stay exact while new calls arrive — the Day 12 keyset idea, built in.

C. Integrations (21–30)

```
# leads/services.py — log_call accepts the dialer's own timestamp (21)
def log_call(lead, agent, outcome, duration_seconds=0, notes="",
started_at=None):
    started_at = started_at or timezone.now()
    call = CallLog.objects.create(lead=lead, agent=agent, outcome=outcome,
duration_seconds=duration_seconds,
                                  notes=notes, started_at=started_at)
    ...   # counter/status update unchanged, using started_at for
last_contacted_at


# clients/models.py — 23: one deposit per provider event
external_ref = models.CharField(max_length=64, unique=True, null=True,
blank=True)   # on Deposit

# brokers/models.py — 22
webhook_url = models.URLField(blank=True)
webhook_secret = models.CharField(max_length=64, blank=True)
```

```
# api/models.py (add) — 24
class IdempotencyRecord(models.Model):
    api_key = models.ForeignKey(ApiKey, on_delete=models.CASCADE)
    key = models.CharField(max_length=100)
    status_code = models.PositiveSmallIntegerField(default=0)      # 0 =
still being processed
    response_body = models.JSONField(default=dict)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        constraints = [models.UniqueConstraint(fields=["api_key", "key"],
name="unique_idempotency_key")]
```

```
# api/throttles.py — 25, 31
from rest_framework.throttling import SimpleRateThrottle
from .models import ApiKey


class ApiKeyRateThrottle(SimpleRateThrottle):
    scope = "apikey"                                    # rate from
DEFAULT_THROTTLE_RATES["apikey"]

    def get_cache_key(self, request, view):
        if not isinstance(request.auth, ApiKey):
            return None                                 # not an API-key call
→ this throttle doesn't apply
        return self.cache_format % {"scope": self.scope, "ident":
request.auth.pk}
```

```
# api/integrations.py — 21, 23, 24
import hashlib
import hmac
import json
from decimal import Decimal

from django.conf import settings
from django.contrib.auth import get_user_model
from django.db import transaction
from rest_framework import serializers, status
from rest_framework.permissions import AllowAny, IsAuthenticated
from rest_framework.response import Response
from rest_framework.views import APIView

from clients.models import Deposit, TradingAccount
from leads.models import CallLog, Lead
from leads.services import log_call
from .authentication import ApiKeyAuthentication
from .fields import UAEPhoneSerializerField
from .models import IdempotencyRecord
from .throttles import ApiKeyRateThrottle


class DialerCallSerializer(serializers.Serializer):
    phone = UAEPhoneSerializerField()
    agent = serializers.SlugRelatedField(slug_field="username",
```

```
source="agent_obj",

queryset=get_user_model().objects.filter(role="agent", is_active=True))
    outcome = serializers.ChoiceField(choices=CallLog.Outcome.choices)
    duration_seconds = serializers.IntegerField(min_value=0, default=0)
    started_at = serializers.DateTimeField()
    external_id = serializers.CharField(max_length=64)


class DialerCallAPI(APIView):                                          #
21, 24
    authentication_classes = [ApiKeyAuthentication]
    permission_classes = [IsAuthenticated]
    throttle_classes = [ApiKeyRateThrottle]

    def post(self, request):
        key = request.headers.get("Idempotency-Key")
        if not key:
            raise serializers.ValidationError({"Idempotency-Key": "Header
required."})
        with transaction.atomic():
            # a concurrent duplicate blocks on the unique index here until
the first one commits
            record, created =
IdempotencyRecord.objects.select_for_update().get_or_create(
                api_key=request.auth, key=key)
            if not created and record.status_code:
                return Response(record.response_body,
status=record.status_code)   # replay, don't redo
            data = DialerCallSerializer(data=request.data)
            data.is_valid(raise_exception=True)
            v = data.validated_data
            lead = Lead.objects.filter(phone=v["phone"]).first()
            if lead is None:
                body, code = {"detail": "No lead with this phone."},
status.HTTP_404_NOT_FOUND
            else:
                call = log_call(lead, v["agent_obj"], v["outcome"],
v["duration_seconds"],
                                notes=f"dialer:{v['external_id']}",
started_at=v["started_at"])
                body, code = {"call_id": call.pk, "lead_id": lead.pk},
status.HTTP_201_CREATED
```

```
            record.status_code, record.response_body = code, body
            record.save(update_fields=["status_code", "response_body"])
        return Response(body, status=code)


class DepositWebhookAPI(APIView):                                    # 23
    authentication_classes = []           # trust comes from the signature,
not a user
    permission_classes = [AllowAny]

    def post(self, request):
        raw = request.body                # read the raw bytes BEFORE
request.data parses them
        expected = "sha256=" +
hmac.new(settings.PAYMENT_WEBHOOK_SECRET.encode(), raw,
hashlib.sha256).hexdigest()
        if not hmac.compare_digest(request.headers.get("X-Pay-Signature",
""), expected):   # constant-time
            return Response({"detail": "Bad signature."},
status=status.HTTP_401_UNAUTHORIZED)
        event = json.loads(raw)
        account =
TradingAccount.objects.filter(number=event["account_number"]).first()
        if account is None:
            return Response({"detail": "Unknown account."},
status=status.HTTP_404_NOT_FOUND)
        deposit, created = Deposit.objects.get_or_create(          # provider
retries → still one deposit
            external_ref=event["event_id"],
            defaults={"account": account, "amount":
Decimal(str(event["amount"])), "method": event["method"]},   # created_at is
auto_now_add: the receive time
        )
        return Response({"deposit_id": deposit.pk},
                        status=status.HTTP_201_CREATED if created else
status.HTTP_200_OK)

# settings/base.py
PAYMENT_WEBHOOK_SECRET = env.str("PAYMENT_WEBHOOK_SECRET", default="")
```

```
# brokers/webhooks.py — 22
import hashlib
import hmac
import json
import logging
import urllib.request

from django.core.serializers.json import DjangoJSONEncoder
from django.utils import timezone

logger = logging.getLogger("crm.webhooks")


def send_webhook(broker, event, payload):
    body = json.dumps({"event": event, "data": payload, "sent_at":
timezone.now()}, cls=DjangoJSONEncoder).encode()
    signature = hmac.new(broker.webhook_secret.encode(), body,
hashlib.sha256).hexdigest()
    request = urllib.request.Request(broker.webhook_url, data=body,
method="POST", headers={
        "Content-Type": "application/json", "X-Nexora-Signature": f"sha256=
{signature}"})
    try:
        with urllib.request.urlopen(request, timeout=5) as response:
            return response.status
    except Exception as exc:                # a broker's server must never
break our conversion
        logger.warning("webhook to %s failed: %s", broker.code, exc)
        return None


# clients/signals.py
@receiver(lead_converted, dispatch_uid="broker_webhook")
def broker_webhook(sender, lead, client, **kwargs):
    broker = client.broker
    if broker and broker.webhook_url and broker.webhook_secret:
        payload = {"client_ref": client.ref, "account_number":
client.account_number, "lead_id": lead.pk}
        transaction.on_commit(lambda: send_webhook(broker, "lead.converted",
payload))
```

The HTTP call runs inside the request that converted the lead (up to 5 s). Day 28's notes cover moving slow side effects to a worker queue.

26–27.

```
# api/viewsets.py — 26
from drf_spectacular.utils import OpenApiParameter, extend_schema,
extend_schema_view


@extend_schema_view(
    list=extend_schema(tags=["Leads"], summary="Leads visible to you",
                       parameters=[OpenApiParameter("fields", str,
description="Comma-separated fields")]),
    retrieve=extend_schema(tags=["Leads"]),
)
class LeadViewSet(CRMAPIViewMixin, viewsets.ModelViewSet):
    ...
    @extend_schema(tags=["Leads"], summary="Change status",
request=StatusChangeSerializer, responses=LeadSerializer)
    @action(detail=True, methods=["post"], url_path="status", url_name="set-
status")
    def set_status(self, request, pk=None):
        ...
```

```
# settings/base.py — 27: the version travels in the Accept header; the URL
stays /api/v2/
REST_FRAMEWORK.update({
    "DEFAULT_VERSIONING_CLASS":
"rest_framework.versioning.AcceptHeaderVersioning",
    "DEFAULT_VERSION": "2",
    "ALLOWED_VERSIONS": ("2", "3"),
})


# api/serializers.py
class LeadV3Serializer(LeadSerializer):
    first_name = serializers.SerializerMethodField()
    last_name = serializers.SerializerMethodField()

    class Meta(LeadSerializer.Meta):
        fields = [f for f in LeadSerializer.Meta.fields if f != "name"] +
["first_name", "last_name"]
```

```
    def get_first_name(self, obj):
        return obj.name.split(" ", 1)[0]

    def get_last_name(self, obj):
        return obj.name.split(" ", 1)[1] if " " in obj.name else ""


# LeadViewSet.get_serializer_class — first line
if self.request.version == "3" and self.action == "retrieve":
    return LeadV3Serializer
```

```
curl -H "Authorization: Token <key>" -H "Accept: application/json; version=3"
http://127.0.0.1:8000/api/v2/leads/5/
```

29–30.

```
# api/viewsets.py — LeadViewSet (29)
from django.http import StreamingHttpResponse
from rest_framework.throttling import ScopedRateThrottle
from leads.exports import EXPORT_FIELDS, stream_csv


class CanExportLeads(BasePermission):
    def has_permission(self, request, view):
        return request.user.has_perm("leads.export_lead")


@action(detail=False, url_path="export", permission_classes=[IsAuthenticated,
CanExportLeads],
        throttle_classes=[ScopedRateThrottle], throttle_scope="exports")   #
10 per hour
def export(self, request):
    rows =
self.filter_queryset(self.get_queryset()).values_list(*EXPORT_FIELDS).iterato
r(chunk_size=2000)
    response = StreamingHttpResponse(stream_csv(rows),
content_type="text/csv; charset=utf-8")
    response["Content-Disposition"] = 'attachment; filename="leads.csv"'
    return response
```

30. Every ViewSet's `get_queryset()` starts from `visible_to(request.user)` (leads, clients, calls). The last test below checks an agent sees no other agent's calls through `/api/v2/calls/`.

D. Challenge (32–36)

```
# api/pagination.py — 32
class LinkHeaderPagination(PageNumberPagination):
    page_size = 25

    def get_paginated_response(self, data):
        links = []
        if next_url := self.get_next_link():
            links.append(f'<{next_url}>; rel="next"')
        if prev_url := self.get_previous_link():
            links.append(f'<{prev_url}>; rel="prev"')
        headers = {"X-Total-Count": str(self.page.paginator.count)}
        if links:
            headers["Link"] = ", ".join(links)
        return Response(data, headers=headers)          # body is just the
list
```

```
# api/viewsets.py — LeadViewSet: 33
def lead_etag(lead):
    return f'"{int(lead.updated_at.timestamp() * 1_000_000)}"'


def retrieve(self, request, *args, **kwargs):
    instance = self.get_object()
    response = Response(self.get_serializer(instance).data)
    response["ETag"] = lead_etag(instance)
    return response


def update(self, request, *args, **kwargs):
    if_match = request.headers.get("If-Match")
    if if_match and if_match != lead_etag(self.get_object()):
        return Response({"error": "precondition_failed", "detail": "The lead
changed since you loaded it."},
                        status=status.HTTP_412_PRECONDITION_FAILED)
    return super().update(request, *args, **kwargs)      # partial_update
calls update() too
```

```
# 34
@action(detail=False, methods=["patch"], url_path="bulk-status")
def bulk_status(self, request):
    items = request.data if isinstance(request.data, list) else []
    leads = self.get_queryset().in_bulk([i.get("id") for i in items if
isinstance(i, dict)])
    results = []
    for item in items[:200]:
        lead = leads.get(item.get("id"))
        if lead is None or not can_edit_lead(request.user, lead):
            results.append({"id": item.get("id"), "ok": False, "error": "not
found"})
            continue
        s = StatusChangeSerializer(data=item, context={"lead": lead})
        if not s.is_valid():
            results.append({"id": lead.pk, "ok": False, "error": s.errors})
            continue
        lead.status = s.validated_data["status"]
        lead.save(update_fields=["status", "updated_at"])   # save(), so
history + audit signals fire
        results.append({"id": lead.pk, "ok": True})
    return Response({"results": results})
```

```
# .github/workflows/tests.yml — 35: a broken schema breaks the build
      - run: python manage.py spectacular --file schema.yml --validate --
fail-on-warn --settings=django_project.settings.test
```

```
# core/testing.py — 36
from rest_framework.test import APIClient


def api_client_for(user):
    client = APIClient()
    client.force_authenticate(user)
    return client

# Day 25 test 37: ids = [row["id"] for row in response.data["results"]]
(lists are paginated now)
```

Tests (37–40)

```
# api/tests.py (add)
import hashlib
import hmac
import json

from django.test import override_settings
from django.utils import timezone
from rest_framework.authtoken.models import Token
from rest_framework.test import APIClient

from core.factories import CallLogFactory, ManagerFactory,
TradingAccountFactory, UserFactory
from core.testing import api_client_for
from .keys import create_api_key


class IntegrationAPITests(APITestCase):
    def setUp(self):
        cache.clear()

    def test_token_auth_without_session(self):                          # 37
        agent = AgentFactory()
        LeadFactory(assigned_to=agent)
        client = APIClient()
        client.credentials(HTTP_AUTHORIZATION=f"Token
{Token.objects.create(user=agent).key}")
        response = client.get(reverse("api:lead-list"))
        self.assertEqual(response.status_code, 200)
        self.assertEqual(response.data["count"], 1)

    def test_cursor_pages_do_not_overlap(self):                         # 38
        client = api_client_for(ManagerFactory())
        lead = LeadFactory()
        CallLogFactory.create_batch(120, lead=lead, agent=lead.assigned_to)
        first = client.get(reverse("api:call-list")).data
        second = client.get(first["next"]).data                          #
the next link carries the cursor
        ids1 = {row["id"] for row in first["results"]}
        ids2 = {row["id"] for row in second["results"]}
        self.assertEqual((len(ids1), len(ids2)), (50, 50))
        self.assertFalse(ids1 & ids2)
```

```
    def test_dialer_idempotency(self):                                  # 39
        raw, _ = create_api_key("dialer", UserFactory(role="manager"))
        agent = AgentFactory()
        lead = LeadFactory(phone="971508881234", assigned_to=agent)
        body = {"phone": "0508881234", "agent": agent.username, "outcome":
"answered", "duration_seconds": 60,
                "started_at": timezone.now().isoformat(), "external_id":
"ext-1"}
        for _ in range(2):
            response = self.client.post(reverse("api:dialer-calls"), body,
format="json",
                                        HTTP_X_API_KEY=raw,
HTTP_IDEMPOTENCY_KEY="abc-1")
            self.assertEqual(response.status_code, 201)
        self.assertEqual(lead.calls.count(), 1)

    @override_settings(PAYMENT_WEBHOOK_SECRET="whsec-test")
    def test_deposit_webhook_signature(self):                           # 40
        account = TradingAccountFactory()
        body = json.dumps({"event_id": "evt_1", "account_number":
account.number, "amount": "2500.00",
                           "method": "card", "created_at":
timezone.now().isoformat()}).encode()
        url = reverse("api:deposit-webhook")
        bad = self.client.post(url, body, content_type="application/json",
HTTP_X_PAY_SIGNATURE="sha256=bad")
        self.assertEqual(bad.status_code, 401)
        good_sig = "sha256=" + hmac.new(b"whsec-test", body,
hashlib.sha256).hexdigest()
        good = self.client.post(url, body, content_type="application/json",
HTTP_X_PAY_SIGNATURE=good_sig)
        self.assertEqual(good.status_code, 201)
        self.assertEqual(account.deposits.count(), 1)

    def test_agent_sees_only_own_calls(self):                           # 30
        a1, a2 = AgentFactory(), AgentFactory()
        CallLogFactory(lead=LeadFactory(assigned_to=a2), agent=a2)
        self.assertEqual(api_client_for(a1).get(reverse("api:call-
list")).data["results"], [])
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py test api --settings=django_project.settings.test
git add -A && git commit -m "day 26"
```

Day 27 — JSON + JavaScript front end

Goal: the CRM feels instant. Plain JavaScript modules (no build step, no framework) talk to the Day 25–26 API: inline status changes, a call-logging dialog, a power dialer, a drag-and-drop pipeline, type-ahead search and live Chart.js dashboards. Every page still works without JavaScript.

Rules for the whole day: data goes into the page with `textContent` (never `innerHTML` with API data), every POST carries the CSRF token, no inline `onclick` (Day 21's CSP blocks it), URLs come from Django, not hard-coded strings.

A. Drills (1–10) — fetch and the DOM

- [ ] **1.** `static/js/` as ES modules; a `js_config` context processor + `json_script` gives JS the API URLs and the user's role.
- [ ] **2.** `api.js`: one `api(url, {method, body})` wrapper — JSON in/out, timeout, typed `ApiError`.
- [ ] **3.** CSRF: read the `csrftoken` cookie and send `X-CSRFToken` on unsafe methods.
- [ ] **4.** Lead inbox page: render rows from `/api/v2/leads/` with an `el()` helper and `textContent`.
- [ ] **5.** Pager from the API's `next` / `previous` / `num_pages`.
- [ ] **6.** Type-ahead search: debounced input, `AbortController` cancels stale requests, grouped suggestions.
- [ ] **7.** Show errors: DRF field errors as text, 403/404 messages, network failures.
- [ ] **8.** Toast notifications in an `aria-live` region.
- [ ] **9.** "Load more" on the classic lead list using Day 12's `?partial=1` HTML.
- [ ] **10.** Retry GETs with backoff on 502/503/504 and network errors.

B. Core (11–20) — charts

- [ ] **11.** Chart.js 4 from jsDelivr (pinned version; allowed by the CSP).
- [ ] **12.** `/api/v2/charts/<name>/?days=7|30|90` returning `{labels, datasets}` with missing days filled with 0.
- [ ] **13.** Daily new leads: line chart.
- [ ] **14.** Calls by hour: bars for calls + line for answer rate on a second axis.

- [ ] **15.** Status mix: doughnut.
- [ ] **16.** Source conversion %: horizontal bars.
- [ ] **17.** Range selector re-fetches and calls `chart.update()` (no re-creating charts).
- [ ] **18.** Auto-refresh every 60 s, paused while the tab is hidden.
- [ ] **19.** Loading and empty states.
- [ ] **20.** Export a chart as PNG and its data as CSV, client-side.

C. Nexora CRM (21–30)

- [ ] **21.** Live dashboard: KPI tiles from `/api/v2/dashboard/` + the four charts.
- [ ] **22.** Inline status change in the inbox: optimistic, rolled back on error.
- [ ] **23.** "Log call" `<dialog>` on lead detail → POST → prepend to the timeline.
- [ ] **24.** Power dialer: next lead, `tel:` link, one-tap outcomes, auto-advance.
- [ ] **25.** Pipeline board (kanban) with drag and drop between status columns; invalid moves snap back.
- [ ] **26.** Keyboard shortcuts: `/` search, `n` new lead, `?` help.
- [ ] **27.** Alerts badge: poll the unread count every 30 s; mark read with JSON.
- [ ] **28.** The same "mark read" with HTMX — compare the two approaches.
- [ ] **29.** Progressive enhancement: the lead status form posts normally without JS; with JS it's intercepted.
- [ ] **30.** Accessibility: dialog focus, `aria-live`, a keyboard alternative to drag and drop.

D. Challenge (31–40)

- [ ] **31.** Server-sent events for live alerts (async view; needs ASGI).
- [ ] **32.** Optimistic locking in the UI: `If-Match` from the lead's `ETag`; 412 → "reload" message.
- [ ] **33.** Inbox filters mirrored in the address bar (`history.replaceState`), so reload and share keep them.
- [ ] **34.** `debounce` helper.
- [ ] **35.** Event delegation everywhere (one listener per container).
- [ ] **36.** `Intl` formatting: AED amounts and Dubai-time dates.
- [ ] **37.** Test: `?days=7` returns 7 labels with zeros filled.
- [ ] **38.** Test: a session POST without the CSRF token gets 403.

- [ ] **39.** Test: an agent changing another agent's lead gets 404.
- [ ] **40.** Test: unread-alert count goes down after mark-read.

Solutions

```
static/js/
├── config.js      # reads the json_script config, builds URLs
├── api.js         # fetch wrapper: JSON, CSRF, timeout, retries, ApiError
├── ui.js          # el(), toast(), errorText(), debounce(), Intl formatters
├── app.js         # every page: search, shortcuts, alerts badge
├── inbox.js       # lead inbox (4, 5, 22, 33)
├── charts.js      # Chart.js helpers (11–20)
├── dashboard.js   # 21
├── lead-detail.js # 23, 29, 32
├── dialer.js      # 24
└── board.js       # 25, 30
```

Django side: config and the new endpoints (1, 12, 21, 27)

```
# core/context_processors.py — 1 (add to TEMPLATES context_processors)
from django.urls import reverse

ZERO_UUID = "00000000-0000-0000-0000-000000000000"


def js_config(request):
    if not request.user.is_authenticated:
        return {}
    urls = {   # placeholders 0 / ZERO_UUID / x-name are swapped in by
config.js
        "leads": reverse("api:lead-list"),
        "leadStatus": reverse("api:lead-set-status", args=[0]),
        "leadCalls": reverse("api:lead-calls", args=[0]),
        "leadApi": reverse("api:lead-detail", args=[0]),
        "nextLead": reverse("api:lead-next"),
        "search": reverse("api:search"),
        "dashboard": reverse("api:dashboard"),
        "chart": reverse("api:chart", args=["x-name"]),
        "alertsUnread": reverse("api:alerts-unread"),
        "alertRead": reverse("api:alert-read", args=[0]),
        "leadPage": reverse("leads:detail", args=[0]),
```

```
        "clientPage": reverse("clients:detail", args=[ZERO_UUID]),
        "newLead": reverse("leads:create"),
    }
    return {"js_config": {"urls": urls, "userId": request.user.pk, "role":
request.user.role}}
```

```
<!-- templates/base.html, end of <body> -->
<div id="toasts" class="toasts" aria-live="polite"></div>
{% if js_config %}{{ js_config|json_script:"crm-config" }}{% endif %}
<script type="module" src="{% static 'js/app.js' %}"></script>
{% block scripts %}{% endblock %}
```

`json_script` writes `<script type="application/json">` with `<`, `>` and `&` escaped: it's data, not code, so the CSP doesn't block it and a lead called `</script>` can't break out.

```
# api/charts.py — 12
from datetime import timedelta

from django.db.models import Count, Q
from django.db.models.functions import ExtractHour, TruncDate
from django.utils import timezone

from leads.models import CallLog, Lead


def daily_leads_chart(user, days):
    start = timezone.localdate() - timedelta(days=days - 1)
    rows = (Lead.objects.visible_to(user).filter(created_at__date__gte=start)

.annotate(day=TruncDate("created_at")).values("day").annotate(n=Count("id")))
    counts = {r["day"]: r["n"] for r in rows}
    every_day = [start + timedelta(days=i) for i in range(days)]     # fill
gaps → no jumps in the line
    return {"labels": [d.isoformat() for d in every_day],
            "datasets": [{"label": "New leads", "data": [counts.get(d, 0) for
d in every_day]}]}


def calls_by_hour_chart(user, days):
    since = timezone.now() - timedelta(days=days)
    rows = (CallLog.objects.visible_to(user).filter(started_at__gte=since)
```

```
            .annotate(hour=ExtractHour("started_at")).values("hour")      #
hour in Dubai time
            .annotate(calls=Count("id"), answered=Count("id",
filter=Q(outcome=CallLog.Outcome.ANSWERED))))
    by_hour = {r["hour"]: r for r in rows}
    hours = range(8, 22)                                                   #
the call-centre day
    calls = [by_hour.get(h, {}).get("calls", 0) for h in hours]
    rate = [round(by_hour[h]["answered"] * 100 / by_hour[h]["calls"], 1) if h
in by_hour else 0 for h in hours]
    return {"labels": [f"{h:02d}:00" for h in hours],
            "datasets": [{"label": "Calls", "data": calls}, {"label": "Answer
rate %", "data": rate}]}


def status_chart(user, days):
    counts =
dict(Lead.objects.visible_to(user).active().values_list("status").annotate(n=
Count("id")))
    return {"labels": [label for _, label in Lead.Status.choices],
            "datasets": [{"label": "Leads", "data": [counts.get(value, 0) for
value, _ in Lead.Status.choices]}]}


def sources_chart(user, days):
    since = timezone.now() - timedelta(days=days)
    rows =
(Lead.objects.visible_to(user).filter(created_at__gte=since).values("source")
            .annotate(total=Count("id"), won=Count("id",
filter=Q(status=Lead.Status.QUALIFIED)))
            .order_by("-total"))
    labels = dict(Lead.Source.choices)
    return {"labels": [labels.get(r["source"], r["source"]) for r in rows],
            "datasets": [{"label": "Conversion %", "data": [round(r["won"] *
100 / r["total"], 1) for r in rows]}]}


CHARTS = {"daily-leads": daily_leads_chart, "calls-by-hour":
calls_by_hour_chart,
          "status": status_chart, "sources": sources_chart}
```

```
# api/views.py (add) — 12, 21, 27
from rest_framework.exceptions import NotFound

from core.dashboard import dashboard_stats
from core.models import Alert
from .charts import CHARTS


class ChartParams(serializers.Serializer):
    days = serializers.ChoiceField(choices=[7, 30, 90], default=30)     # "7"
from the querystring → 7


@api_view(["GET"])
def chart(request, name):
    if name not in CHARTS:
        raise NotFound(f"No chart called {name!r}.")
    params = ChartParams(data=request.query_params)
    params.is_valid(raise_exception=True)
    return Response(CHARTS[name](request.user,
params.validated_data["days"]))


@api_view(["GET"])
def dashboard(request):
    return Response(dashboard_stats(request.user))            # Day 24:
cached per user


@api_view(["GET"])
def unread_alerts(request):
    qs = Alert.objects.unread_for(request.user)               # Day 21's
queryset methods
    latest = qs.order_by("-created_at")[:5]
    return Response({"count": qs.count(),
                     "latest": [{"id": a.pk, "title": a.title, "url": a.url,
"created_at": a.created_at} for a in latest]})


@api_view(["POST"])
def mark_alert_read(request, pk):
    alert = get_object_or_404(Alert.objects.for_user(request.user), pk=pk)
    alert.read_by.add(request.user)
```

```
    return Response(status=status.HTTP_204_NO_CONTENT)


# api/urls.py (before the router include)
path("charts/<slug:name>/", views.chart, name="chart"),
path("dashboard/", views.dashboard, name="dashboard"),
path("alerts/unread/", views.unread_alerts, name="alerts-unread"),
path("alerts/<int:pk>/read/", views.mark_alert_read, name="alert-read"),
```

A. The JS core (1–3, 7–8, 10, 34, 36)

```
// static/js/config.js — 1
const node = document.getElementById("crm-config");
export const config = node ? JSON.parse(node.textContent) : { urls: {} };
const ZERO_UUID = "00000000-0000-0000-0000-000000000000";

export function url(name, id) {
  const template = config.urls[name];
  if (id === undefined) return template;
  return template.replace(ZERO_UUID, id).replace("x-name", id).replace("/0/",
`/${id}/`);
}
```

```
// static/js/api.js — 2, 3, 10
const SAFE = new Set(["GET", "HEAD", "OPTIONS"]);
const RETRY_STATUSES = new Set([502, 503, 504]);

export class ApiError extends Error {
  constructor(status, body) {
    super(`HTTP ${status}`);
    this.status = status;
    this.body = body;                          // DRF shape: {error, detail,
request_id}
  }
}

function csrfToken() {
// 3
  const match = document.cookie.match(/(?:^|;\s*)csrftoken=([^;]+)/);
  return match ? decodeURIComponent(match[1]) : "";
}
```

```
async function request(url, { method, body, signal, timeout, headers }) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), timeout);
  signal?.addEventListener("abort", () => controller.abort());
  try {
    const response = await fetch(url, {
      method,
      signal: controller.signal,
      credentials: "same-origin",
      headers: {
        Accept: "application/json",
        ...(body !== undefined ? { "Content-Type": "application/json" } :
{}),
        ...(SAFE.has(method) ? {} : { "X-CSRFToken": csrfToken() }),
        ...headers,
      },
      body: body === undefined ? undefined : JSON.stringify(body),
    });
    const data = response.status === 204 ? null : await
response.json().catch(() => null);
    if (!response.ok) throw new ApiError(response.status, data);
    return { data, response };
  } finally {
    clearTimeout(timer);
  }
}

export async function apiWithResponse(url, options = {}) {
  const { method = "GET", body, signal, timeout = 10000, headers = {} } =
options;
  const retries = options.retries ?? (SAFE.has(method) ? 2 : 0);           //
never retry a POST
  for (let attempt = 0; ; attempt++) {
    try {
      return await request(url, { method, body, signal, timeout, headers });
    } catch (error) {
      const retryable = error instanceof TypeError ||
// network failure
        (error instanceof ApiError && RETRY_STATUSES.has(error.status));
      if (attempt >= retries || !retryable || signal?.aborted) throw error;
      await new Promise((resolve) => setTimeout(resolve, 500 * 2 **
attempt)); // 0.5 s, 1 s
    }
```

```
  }
}

export async function api(url, options) {
  return (await apiWithResponse(url, options)).data;
}
```

The `csrftoken` cookie must be readable by JS: keep `CSRF_COOKIE_HTTPONLY = False` (the default). Pages without any `{% csrf_token %}` need `@ensure_csrf_cookie` on their view so the cookie exists.

```
// static/js/ui.js — 4, 7, 8, 34, 36
import { ApiError } from "./api.js";

export function el(tag, attrs = {}, ...children) {
  const node = document.createElement(tag);
  for (const [key, value] of Object.entries(attrs)) {
    if (value === undefined || value === null || value === false) continue;
    if (key === "class") node.className = value;
    else if (key === "dataset") Object.assign(node.dataset, value);
    else node.setAttribute(key, value === true ? "" : value);
  }
  node.append(...children.flat().filter((c) => c !== null && c !==
undefined));   // strings become text nodes
  return node;
}

export function toast(message, level = "info") {
// 8
  const item = el("div", { class: `toast toast-${level}`, role: level ===
"error" ? "alert" : "status" }, message);
  document.getElementById("toasts").append(item);
  setTimeout(() => item.remove(), 5000);
}

export function errorText(error) {
// 7
  if (error?.name === "AbortError") return "The request timed out.";
  if (!(error instanceof ApiError)) return "Network problem — check your
connection.";
  if (error.status === 403) return "You don't have permission to do that.";
  if (error.status === 404) return "Not found (or not yours).";
  const detail = error.body?.detail;
```

```
  if (typeof detail === "string") return detail;
  if (detail && typeof detail === "object") {
    const first = Object.values(detail).flat()[0];          // {"status":
["Can't move..."]} → the message
    if (typeof first === "string") return first;
  }
  return `Request failed (${error.status}).`;
}

export function debounce(fn, wait = 250) {
// 34
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), wait);
  };
}

const aed = new Intl.NumberFormat("en-AE", { style: "currency", currency:
"AED", maximumFractionDigits: 0 });   // 36
const dubai = new Intl.DateTimeFormat("en-GB", { timeZone: "Asia/Dubai", day:
"2-digit", month: "short",
                                                 hour: "2-digit", minute: "2-
digit" });
export const fmtAED = (value) => aed.format(Number(value));
export const fmtDate = (iso) => (iso ? dubai.format(new Date(iso)) : "—");
```

A. Inbox, search, load more (4–6, 9, 22, 33, 35)

```
<!-- templates/leads/inbox.html (route: leads/inbox/, TemplateView) -->
{% extends "base.html" %}
{% block content %}
<h1>Lead inbox</h1>
<noscript><p><a href="{% url 'leads:list' %}">JavaScript is off — use the
classic list.</a></p></noscript>
<table id="inbox" class="table">
  <thead><tr><th>Name</th><th>Phone</th><th>Status</th><th>Agent</th>
<th>Added</th></tr></thead>
  <tbody></tbody>
</table>
<nav id="pager" class="pager" aria-label="Pages"></nav>
{{ status_options|json_script:"status-options" }}   {# view context:
list(Lead.Status.choices) #}
{% endblock %}
{% block scripts %}<script type="module" src="{% static 'js/inbox.js' %}">
</script>{% endblock %}
```

```
// static/js/inbox.js — 4, 5, 22, 33, 35
import { api } from "./api.js";
import { url } from "./config.js";
import { el, errorText, fmtDate, toast } from "./ui.js";

const tbody = document.querySelector("#inbox tbody");
const pager = document.getElementById("pager");
const STATUS_OPTIONS = JSON.parse(document.getElementById("status-
options").textContent);
const message = (text) => el("tr", {}, el("td", { colspan: 5, class: "muted"
}, text));

function statusSelect(lead) {
  const select = el("select", { "aria-label": `Status of ${lead.name}`,
dataset: { id: lead.id, current: lead.status } },
    STATUS_OPTIONS.map(([value, label]) => el("option", { value, selected:
value === lead.status }, label)));
  return select;
}

function row(lead) {                                                    // 4:
textContent only
  return el("tr", { dataset: { id: lead.id } },
```

```
    el("td", {}, el("a", { href: url("leadPage", lead.id) }, lead.name)),
    el("td", {}, lead.masked_phone),
    el("td", {}, statusSelect(lead)),
    el("td", {}, lead.assigned_to ?? "—"),
    el("td", {}, fmtDate(lead.created_at)));
}

function renderPager(data) {                                          // 5
  const button = (label, target) =>
    el("button", { type: "button", class: "btn", disabled: !target, dataset:
{ href: target ?? "" } }, label);
  pager.replaceChildren(button("← Previous", data.previous), el("span", {}, `
Page ${data.page} of ${data.num_pages} `),
                        button("Next →", data.next));
}

async function load(apiUrl) {
  tbody.replaceChildren(message("Loading…"));
  try {
    const data = await api(apiUrl);
    tbody.replaceChildren(...(data.results.length ? data.results.map(row) :
[message("No leads match.")]));
    renderPager(data);
    const params = new URL(apiUrl, location.origin).searchParams;     // 33:
filters + page in the address bar
    history.replaceState(null, "", params.size ? `?${params}` :
location.pathname);
  } catch (error) {
    tbody.replaceChildren(message(errorText(error)));
  }
}

pager.addEventListener("click", (event) => {                          //
35: one listener for both buttons
  const button = event.target.closest("button[data-href]");
  if (button?.dataset.href) load(button.dataset.href);
});

tbody.addEventListener("change", async (event) => {                   // 22
  const select = event.target.closest("select[data-id]");
  if (!select) return;
  const previous = select.dataset.current;
  const body = { status: select.value };
```

```
  if (select.value === "lost") {
    const reason = window.prompt("Why is this lead lost?");
    if (!reason) { select.value = previous; return; }
    body.notes = reason;
  }
  select.disabled = true;
  try {
    const lead = await api(url("leadStatus", select.dataset.id), { method:
"POST", body });
    select.dataset.current = lead.status;
    toast(`${lead.name}: ${lead.status_label}`);
  } catch (error) {
    select.value = previous;                                  // roll back the
optimistic change
    toast(errorText(error), "error");
  } finally {
    select.disabled = false;
  }
});

load(url("leads") + location.search);                     // 33: reload
restores the same page/filters
```

```
// static/js/app.js — 6 (plus 26 and 27 in part C)
import { api } from "./api.js";
import { url } from "./config.js";
import { debounce, el } from "./ui.js";

const input = document.getElementById("global-search");        // the Day 17
header search box
const box = document.getElementById("search-suggestions");     // <div
id="search-suggestions" hidden>
let inflight;

function group(title, items, toLink) {
  if (!items.length) return null;
  return el("section", {}, el("h3", {}, title), el("ul", {}, items.map((i) =>
el("li", {}, toLink(i)))));
}

input?.addEventListener("input", debounce(async () => {
  const q = input.value.trim();
  inflight?.abort();                                           // drop the
```

```
previous, slower request
  if (q.length < 2) { box.hidden = true; return; }
  inflight = new AbortController();
  try {
    const data = await api(`${url("search")}?q=${encodeURIComponent(q)}`, {
signal: inflight.signal });
    const sections = [
      group("Leads", data.leads.items, (l) => el("a", { href: url("leadPage",
l.id) }, `${l.name} · ${l.masked_phone}`)),
      group("Clients", data.clients.items, (c) => el("a", { href:
url("clientPage", c.ref) }, `${c.name} · ${c.account_number}`)),
    ].filter(Boolean);
    box.replaceChildren(...(sections.length ? sections : [el("p", { class:
"muted" }, "No matches")]));
    box.hidden = false;
  } catch (error) {
    if (error.name !== "AbortError") box.hidden = true;
  }
}, 250));
```

```
// static/js/load-more.js — 9 (classic list: <button data-load-more data-
next="?page=2&partial=1" data-target="#lead-rows">)
document.addEventListener("click", async (event) => {
  const button = event.target.closest("[data-load-more]");
  if (!button) return;
  button.disabled = true;
  const response = await fetch(button.dataset.next, { credentials: "same-
origin" });
  // our own template, auto-escaped by Django — safe to insert as HTML (never
do this with API JSON)

document.querySelector(button.dataset.target).insertAdjacentHTML("beforeend",
await response.text());
  const next = response.headers.get("X-Next-Page");          // the view sets
it when page.has_next()
  if (next) { button.dataset.next = next; button.disabled = false; } else {
button.remove(); }
});
```

B. Charts (11–20)

```
<!-- in templates that show charts — 11: pinned version from the CSP-allowed
CDN -->
<script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"
crossorigin="anonymous"></script>
<!-- add the integrity="sha384-…" value jsDelivr shows for this file -->
```

```
// static/js/charts.js — 13–20
import { api } from "./api.js";
import { url } from "./config.js";
import { el, errorText, toast } from "./ui.js";

const charts = new Map();                       // canvas → Chart instance
const percent = (v) => `${v}%`;

const CONFIGS = {
  "daily-leads": (d) => ({
// 13
    type: "line",
    data: { labels: d.labels, datasets: [{ ...d.datasets[0], tension: 0.3,
fill: true }] },
    options: { scales: { y: { beginAtZero: true, ticks: { precision: 0 } } } }
},
  }),
  "calls-by-hour": (d) => ({
// 14
    type: "bar",
    data: { labels: d.labels, datasets: [
      { ...d.datasets[0], type: "bar", yAxisID: "y" },
      { ...d.datasets[1], type: "line", yAxisID: "rate", tension: 0.3 },
    ] },
    options: { scales: {
      y: { beginAtZero: true, ticks: { precision: 0 } },
      rate: { position: "right", min: 0, max: 100, ticks: { callback: percent
}, grid: { drawOnChartArea: false } },
    } },
  }),
  status: (d) => ({ type: "doughnut", data: d, options: { plugins: { legend:
{ position: "right" } } } }),   // 15
  sources: (d) => ({
// 16
```

```
    type: "bar", data: d,
    options: { indexAxis: "y", scales: { x: { min: 0, max: 100, ticks: {
callback: percent } } } },
  }),
};

export async function renderChart(canvas, days) {
  const name = canvas.dataset.chart;
  const box = canvas.parentElement;
  box.classList.add("is-loading");
// 19: CSS shows a spinner
  try {
    const data = await api(`${url("chart", name)}?days=${days}`);
    box.classList.toggle("is-empty", data.datasets.every((ds) =>
ds.data.every((v) => !v)));   // 19
    const config = CONFIGS[name](data);
    const chart = charts.get(canvas);
    if (chart) {
// 17: update, don't rebuild
      chart.data = config.data;
      chart.update();
    } else {
      charts.set(canvas, new Chart(canvas, config));
    }
  } catch (error) {
    toast(errorText(error), "error");
  } finally {
    box.classList.remove("is-loading");
  }
}

function download(href, filename) {
  const link = el("a", { href, download: filename });
  document.body.append(link);
  link.click();
  link.remove();
}

function exportChart(canvas, kind) {
// 20
  const chart = charts.get(canvas);
  if (!chart) return;
  const name = canvas.dataset.chart;
```

```
  if (kind === "png") return download(chart.toBase64Image("image/png", 1),
`${name}.png`);
  const header = ["label", ...chart.data.datasets.map((ds) => ds.label)];
  const lines = chart.data.labels.map((label, i) => [label,
...chart.data.datasets.map((ds) => ds.data[i])]);
  const csv = [header, ...lines].map((r) => r.map((v) =>
`"${String(v).replaceAll('"', '""')}"`).join(",")).join("\n");
  const href = URL.createObjectURL(new Blob(["\ufeff" + csv], { type:
"text/csv" }));
  download(href, `${name}.csv`);
  setTimeout(() => URL.revokeObjectURL(href), 1000);
}

export function initCharts(root = document) {
  const canvases = [...root.querySelectorAll("canvas[data-chart]")];
  const range = root.querySelector("#chart-range");
// <select> 7 / 30 / 90
  const refresh = () => canvases.forEach((c) => renderChart(c,
Number(range?.value ?? 30)));
  range?.addEventListener("change", refresh);
// 17
  root.addEventListener("click", (event) => {
// 20: <button data-export="png" data-canvas="id">
    const button = event.target.closest("[data-export]");
    if (button) exportChart(document.getElementById(button.dataset.canvas),
button.dataset.export);
  });
  setInterval(() => { if (!document.hidden) refresh(); }, 60_000);
// 18
  document.addEventListener("visibilitychange", () => { if (!document.hidden)
refresh(); });
  refresh();
  return refresh;
}
```

```
/* static/css/crm.css — 19 */
.chart-box { position: relative; min-height: 260px; }
.chart-box.is-loading canvas { opacity: .35; }
.chart-box.is-empty canvas { display: none; }
.chart-box.is-empty::after { content: "No data for this period"; color: var(-
-muted); }
```

C. Nexora CRM pages (21, 23–30)

21. The dashboard keeps its server-rendered numbers (so it works without JS); the script refreshes them.

```
<!-- templates/dashboard.html (excerpt) -->
<div class="kpis">
  <div class="kpi"><span>Active leads</span><strong data-kpi="stats.total">{{
total }}</strong></div>
  <div class="kpi"><span>My open</span><strong data-kpi="my_open">{{ my_open
}}</strong></div>
  <div class="kpi"><span>My calls today</span><strong data-
kpi="my_calls_today">{{ my_calls_today }}</strong></div>
  <div class="kpi"><span>New today</span><strong data-kpi="new_today">{{
new_today }}</strong></div>
</div>

<label>Range
  <select id="chart-range"><option value="7">7 days</option><option
value="30" selected>30 days</option><option value="90">90 days</option>
</select>
</label>
<div class="grid-2">
  {% for id, name, title in charts %}   {# view: [("c-daily","daily-
leads","New leads per day"), ...] #}
  <figure>
    <figcaption>{{ title }}
      <button type="button" data-export="png" data-canvas="{{ id
}}">PNG</button>
      <button type="button" data-export="csv" data-canvas="{{ id
}}">CSV</button>
    </figcaption>
    <div class="chart-box"><canvas id="{{ id }}" data-chart="{{ name }}"
role="img" aria-label="{{ title }}"></canvas></div>
  </figure>
  {% endfor %}
</div>
{% block scripts %}
<script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"
crossorigin="anonymous"></script>
<script type="module" src="{% static 'js/dashboard.js' %}"></script>
{% endblock %}
```

```
// static/js/dashboard.js
import { api } from "./api.js";
import { initCharts } from "./charts.js";
import { url } from "./config.js";

const pick = (obj, path) => path.split(".").reduce((value, key) => value?.
[key], obj);

async function loadKpis() {
  try {
    const data = await api(url("dashboard"));
    document.querySelectorAll("[data-kpi]").forEach((node) => {
      node.textContent = pick(data, node.dataset.kpi) ?? "—";
    });
  } catch {
    /* keep the last numbers; the next tick tries again */
  }
}

loadKpis();
setInterval(() => { if (!document.hidden) loadKpis(); }, 60_000);
initCharts();
```

23, 29, 32.

```
<!-- templates/leads/detail.html (additions) -->
<span id="status-badge" class="badge badge-{{ lead.status }}">{{
lead.get_status_display }}</span>

{# 29: a normal form that posts to your Day 14 status view; JS upgrades it #}
<form id="status-form" method="post" action="{% url 'leads:status' lead.pk
%}">
  {% csrf_token %}
  <select name="status">{% for v, l in status_choices %}<option value="{{ v
}}"{% if v == lead.status %} selected{% endif %}>{{ l }}</option>{% endfor %}
</select>
  <input name="notes" placeholder="Reason (required for lost)">
  <button class="btn">Update</button>
</form>

<button type="button" class="btn btn-primary" data-open="log-call">Log
call</button>
<dialog id="log-call" aria-labelledby="log-call-title">
```

```
  <form id="log-call-form" method="dialog">
    <h2 id="log-call-title">Log a call</h2>
    <label>Outcome <select name="outcome">{% for v, l in outcome_choices %}
<option value="{{ v }}">{{ l }}</option>{% endfor %}</select></label>
    <label>Talk time (seconds) <input name="duration_seconds" type="number"
min="0" value="0"></label>
    <label>Notes <textarea name="notes" rows="3"></textarea></label>
    <p class="form-error" role="alert" hidden></p>
    <menu><button value="cancel" formnovalidate>Cancel</button><button
value="save" class="btn-primary">Save</button></menu>
  </form>
</dialog>

<ol id="timeline" data-lead-id="{{ lead.pk }}">...</ol>

{# 32 #}
<textarea id="notes-editor" rows="3">{{ lead.notes }}</textarea>
<button type="button" id="save-notes" class="btn">Save notes</button>
```

```
// static/js/lead-detail.js
import { ApiError, api, apiWithResponse } from "./api.js";
import { url } from "./config.js";
import { el, errorText, fmtDate, toast } from "./ui.js";

const timeline = document.getElementById("timeline");
const leadId = timeline.dataset.leadId;

// ---- 23: log-call dialog ------------------------------------------------
-----
const dialog = document.getElementById("log-call");
const form = document.getElementById("log-call-form");
const formError = form.querySelector(".form-error");

document.querySelector("[data-open='log-call']").addEventListener("click", ()
=> {
  formError.hidden = true;
  dialog.showModal();                       // 30: focus moves into the
dialog; Esc closes; focus returns after
  form.elements.outcome.focus();
});

form.addEventListener("submit", async (event) => {
  if (event.submitter?.value !== "save") return;          // Cancel closes
```

```
the dialog normally
  event.preventDefault();
  const body = Object.fromEntries(new FormData(form));
  body.duration_seconds = Number(body.duration_seconds || 0);
  try {
    const call = await api(url("leadCalls", leadId), { method: "POST", body
});
    timeline.prepend(el("li", { class: "timeline-call" },
      el("strong", {}, call.outcome), ` · ${fmtDate(call.started_at)} ·
${call.agent}`,
      call.notes ? el("p", {}, call.notes) : null));
    form.reset();
    dialog.close();
    toast("Call logged");
  } catch (error) {
    formError.textContent = errorText(error);               // e.g. "Only
answered calls have talk time."
    formError.hidden = false;
  }
});

// ---- 29: progressive enhancement of the status form ----------------------
----
const statusForm = document.getElementById("status-form");
const badge = document.getElementById("status-badge");

statusForm?.addEventListener("submit", async (event) => {
  event.preventDefault();
  const { status, notes } = Object.fromEntries(new FormData(statusForm));
  try {
    const lead = await api(url("leadStatus", leadId), { method: "POST", body:
{ status, notes } });
    badge.textContent = lead.status_label;
    badge.className = `badge badge-${lead.status}`;
    toast(`Status: ${lead.status_label}`);
  } catch (error) {
    if (error instanceof ApiError) toast(errorText(error), "error");
    else statusForm.submit();       // network/JS trouble → fall back to the
classic POST (no submit event)
  }
});

// ---- 32: If-Match with the lead's ETag (add "notes" to
```

```
LeadSerializer.Meta.fields) ----
let etag = null;
const loadEtag = async () => { etag = (await apiWithResponse(url("leadApi",
leadId))).response.headers.get("ETag"); };
loadEtag();

document.getElementById("save-notes")?.addEventListener("click", async () =>
{
  try {
    await api(url("leadApi", leadId), {
      method: "PATCH",
      body: { notes: document.getElementById("notes-editor").value },
      headers: etag ? { "If-Match": etag } : {},
    });
    await loadEtag();
    toast("Notes saved");
  } catch (error) {
    toast(error.status === 412 ? "Someone else changed this lead — reload to
see their version." : errorText(error), "error");
  }
});
```

24.

```
<!-- templates/leads/dialer.html (route team/me/dialer/, agents only) -->
<section id="dialer" aria-live="polite">
  <p data-state="loading" class="muted">Finding your next lead…</p>
  <div data-state="lead" hidden>
    <h2 data-field="name"></h2>
    <p><a data-field="phone" class="btn btn-primary btn-lg" href="#">Call</a>
</p>
    <p data-field="meta" class="muted"></p>
    <div class="outcomes">
      {% for v, l in outcome_choices %}<button type="button" class="btn"
data-outcome="{{ v }}">{{ l }}</button>{% endfor %}
    </div>
  </div>
  <p data-state="empty" hidden>Nothing to dial right now.</p>
</section>
```

```
// static/js/dialer.js
import { api } from "./api.js";
import { url } from "./config.js";
import { errorText, fmtDate, toast } from "./ui.js";

const root = document.getElementById("dialer");
const field = (name) => root.querySelector(`[data-field="${name}"]`);
let current = null;

function show(state) {
  root.querySelectorAll("[data-state]").forEach((node) => { node.hidden =
node.dataset.state !== state; });
}

async function next() {
  show("loading");
  try {
    current = await api(url("nextLead"));            // 204 → null
    if (!current) return show("empty");
    field("name").textContent = current.name;
    field("phone").href = `tel:+${current.phone}`;   // agents may see their
own leads' numbers
    field("phone").textContent = `Call ${current.masked_phone}`;
    field("meta").textContent = `${current.status_label} · added
${fmtDate(current.created_at)}`;
    show("lead");
    field("phone").focus();
  } catch (error) {
    toast(errorText(error), "error");
    show("empty");
  }
}

root.addEventListener("click", async (event) => {
  const button = event.target.closest("[data-outcome]");
  if (!button || !current) return;
  root.querySelectorAll("[data-outcome]").forEach((b) => { b.disabled = true;
});
  try {
    await api(url("leadCalls", current.id), { method: "POST", body: {
outcome: button.dataset.outcome } });
    toast(`${current.name}: ${button.textContent}`);
    await next();                                    // auto-advance
```

```
  } catch (error) {
    toast(errorText(error), "error");
  } finally {
    root.querySelectorAll("[data-outcome]").forEach((b) => { b.disabled =
false; });
  }
});

next();
```

25, 30.

```
# leads/views.py
class LeadBoardView(PageTitleMixin, TemplateView):
    template_name = "leads/board.html"
    page_title = "Pipeline"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        leads =
Lead.objects.visible_to(self.request.user).active().select_related("assigned_
to").order_by("-updated_at")
        counts = dict(leads.values_list("status").annotate(n=Count("id")))
        context["columns"] = [{"status": value, "label": label, "count":
counts.get(value, 0),
                               "leads": list(leads.filter(status=value)
[:50])}
                              for value, label in Lead.Status.choices]
        return context
```

```
<!-- templates/leads/board.html -->
<div id="board" class="board">
{% for col in columns %}
  <section class="column" data-status="{{ col.status }}" aria-
labelledby="col-{{ col.status }}">
    <h2 id="col-{{ col.status }}">{{ col.label }} <span class="count">{{
col.count }}</span></h2>
    <ol class="cards">
    {% for lead in col.leads %}
      <li class="card" draggable="true" data-id="{{ lead.pk }}">
        <a href="{{ lead.get_absolute_url }}">{{ lead.name }}</a>
        <small>{{ lead.assigned_to|default:"Unassigned" }}</small>
        {# 30: keyboard / screen-reader alternative to dragging #}
```

```
        <label class="visually-hidden" for="move-{{ lead.pk }}">Move {{
lead.name }} to</label>
        <select id="move-{{ lead.pk }}" data-move>
          {% for c in columns %}<option value="{{ c.status }}"{% if c.status
== col.status %} selected{% endif %}>{{ c.label }}</option>{% endfor %}
        </select>
      </li>
    {% endfor %}
    </ol>
  </section>
{% endfor %}
</div>
```

```
// static/js/board.js
import { api } from "./api.js";
import { url } from "./config.js";
import { errorText, toast } from "./ui.js";

const board = document.getElementById("board");
let dragged = null;

function bumpCount(column, delta) {
  const node = column.querySelector(".count");
  node.textContent = Number(node.textContent) + delta;
}

async function move(card, column) {
  const from = card.closest(".column");
  if (from === column) return;
  const status = column.dataset.status;
  const body = { status };
  if (status === "lost") {
    const reason = window.prompt("Why is this lead lost?");
    if (!reason) return;
    body.notes = reason;
  }
  column.querySelector(".cards").prepend(card);           // optimistic
  card.querySelector("[data-move]").value = status;
  bumpCount(from, -1);
  bumpCount(column, +1);
  try {
    await api(url("leadStatus", card.dataset.id), { method: "POST", body });
  } catch (error) {
```

```
    from.querySelector(".cards").prepend(card);           // invalid move
(e.g. new → qualified): snap back
    card.querySelector("[data-move]").value = from.dataset.status;
    bumpCount(column, -1);
    bumpCount(from, +1);
    toast(errorText(error), "error");
  }
}

board.addEventListener("dragstart", (event) => {
  dragged = event.target.closest(".card");
  event.dataTransfer.effectAllowed = "move";
  event.dataTransfer.setData("text/plain", dragged.dataset.id);
});
board.addEventListener("dragover", (event) => {
  if (event.target.closest(".column")) event.preventDefault();   // allow
dropping here
});
board.addEventListener("drop", (event) => {
  const column = event.target.closest(".column");
  if (!column || !dragged) return;
  event.preventDefault();
  move(dragged, column);
  dragged = null;
});
board.addEventListener("change", (event) => {                    // 30: the
select does the same move
  const select = event.target.closest("[data-move]");
  if (select) move(select.closest(".card"),
board.querySelector(`.column[data-status="${select.value}"]`));
});
```

26–27.

```
// static/js/app.js (add)
import { config } from "./config.js";
import { errorText, toast } from "./ui.js";

// 26 — shortcuts (ignored while typing)
document.addEventListener("keydown", (event) => {
  if (event.target.closest("input, textarea, select, [contenteditable]"))
return;
  if (event.ctrlKey || event.metaKey || event.altKey) return;
```

```
  if (event.key === "/") { event.preventDefault(); input?.focus(); }
  else if (event.key === "n") location.href = config.urls.newLead;
  else if (event.key === "?") document.getElementById("shortcuts-
help")?.showModal();
  else if (event.key === "Escape" && box) box.hidden = true;
});

// 27 — alerts badge: <a href="/alerts/">Alerts <span id="alerts-badge"
class="pill" hidden></span></a>
const badge = document.getElementById("alerts-badge");

async function pollAlerts() {
  if (!badge || document.hidden) return;
  try {
    const { count } = await api(url("alertsUnread"), { retries: 0 });
    badge.textContent = count;
    badge.hidden = count === 0;
    badge.setAttribute("aria-label", `${count} unread alerts`);
  } catch {
    /* quiet: the next poll tries again */
  }
}
pollAlerts();
setInterval(pollAlerts, 30_000);

document.addEventListener("click", async (event) => {       // <button data-
alert-read="{{ alert.pk }}">
  const button = event.target.closest("[data-alert-read]");
  if (!button) return;
  try {
    await api(url("alertRead", button.dataset.alertRead), { method: "POST"
});
    button.closest("li")?.remove();
    pollAlerts();
  } catch (error) {
    toast(errorText(error), "error");
  }
});
```

28. HTMX swaps HTML returned by the server; no JS file to write.

```
<!-- base.html -->
<script src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.4/dist/htmx.min.js"
defer></script>
<body hx-headers='{"X-CSRFToken": "{{ csrf_token }}"}'>

<!-- alerts list -->
<li id="alert-{{ alert.pk }}">
  {{ alert.title }}
  <button hx-post="{% url 'alert_read' alert.pk %}" hx-target="#alert-{{
alert.pk }}" hx-swap="outerHTML">Mark read</button>
</li>
```

```
# core/views.py
class AlertReadView(View):
    def post(self, request, pk):
        alert = get_object_or_404(Alert.objects.for_user(request.user),
pk=pk)
        alert.read_by.add(request.user)
        if request.headers.get("HX-Request"):
            return HttpResponse("")                  # HTMX replaces the
<li> with nothing
        return redirect(request.POST.get("next") or "dashboard")   # no-JS
fallback: a normal form POST
```

The trade-off: HTMX keeps all rendering in Django templates (one source of truth, very little JS) but needs an HTML endpoint per interaction. The JSON approach reuses the API that mobile apps and integrations also use, at the cost of building DOM in JS. Nexora uses JSON for rich widgets (charts, board, dialer) and HTMX-style fragments where a template already exists.

30. Covered in place: `<dialog>` + `showModal()` (focus trap, Esc, focus restored), `aria-live` on toasts and the dialer, `role="img"` + `aria-label` on canvases, a `<select>` alternative to dragging, and status changes announced with text, not colour alone.

D. Challenge (31)

```
# core/views.py — 31: server-sent events (async view)
import asyncio
import json

from django.http import StreamingHttpResponse
```

```
async def alert_stream(request):
    user = await request.auser()
    if not user.is_authenticated:
        return HttpResponse(status=401)

    async def events():
        last_id = 0
        while True:
            async for alert in
Alert.objects.unread_for(user).filter(pk__gt=last_id).order_by("pk")[:10]:
                last_id = alert.pk
                payload = json.dumps({"id": alert.pk, "title": alert.title,
"url": alert.url})
                yield f"event: alert\ndata: {payload}\n\n"
            yield ": keep-alive\n\n"
            await asyncio.sleep(10)

    response = StreamingHttpResponse(events(), content_type="text/event-
stream")
    response["Cache-Control"] = "no-cache"
    response["X-Accel-Buffering"] = "no"          # tell nginx not to buffer
the stream
    return response
```

```
// static/js/app.js — 31 (instead of polling, when the stream is available)
const stream = new EventSource("/alerts/stream/");
stream.addEventListener("alert", (event) => {
  const alert = JSON.parse(event.data);
  toast(alert.title, "warning");
  pollAlerts();
});
```

SSE needs an ASGI server (uvicorn or daphne, Day 28): under gunicorn's WSGI workers each open stream holds a whole worker forever. Keep the 30-second polling as the fallback.

Tests (37–40)

```
# api/tests.py (add)
from decimal import Decimal

from core.factories import DepositFactory


class FrontendAPITests(APITestCase):
    def setUp(self):
        cache.clear()

    def test_chart_fills_missing_days(self):                                # 37
        LeadFactory.create_batch(2)                                           #
both created today
        client = api_client_for(ManagerFactory())
        data = client.get(reverse("api:chart", args=["daily-leads"]),
{"days": 7}).data
        self.assertEqual(len(data["labels"]), 7)
        self.assertEqual(len(data["datasets"][0]["data"]), 7)
        self.assertGreaterEqual(data["datasets"][0]["data"][-1], 2)      #
today is the last label
        self.assertEqual(data["datasets"][0]["data"][0], 0)              # a
filled gap

    def test_session_post_needs_csrf(self):                                 # 38
        client = APIClient(enforce_csrf_checks=True)
        agent = AgentFactory()
        lead = LeadFactory(assigned_to=agent)
        client.force_login(agent)                                             #
a real session, like the browser
        response = client.post(reverse("api:lead-set-status", args=
[lead.pk]), {"status": "contacted"}, format="json")
        self.assertEqual(response.status_code, 403)

    def test_agent_cannot_touch_other_lead(self):                           # 39
        lead = LeadFactory(assigned_to=AgentFactory())
        client = api_client_for(AgentFactory())
        response = client.post(reverse("api:lead-set-status", args=
[lead.pk]), {"status": "contacted"}, format="json")
        self.assertEqual(response.status_code, 404)

    def test_unread_alerts_and_mark_read(self):                             # 40
```

```
        manager = ManagerFactory()
        with self.captureOnCommitCallbacks(execute=True):
            DepositFactory(amount=Decimal("60000"))                        #
Day 20: large-deposit alert
        client = api_client_for(manager)
        data = client.get(reverse("api:alerts-unread")).data
        self.assertEqual(data["count"], 1)
        client.post(reverse("api:alert-read", args=[data["latest"][0]
["id"]]))
        self.assertEqual(client.get(reverse("api:alerts-
unread")).data["count"], 0)
```

```
python manage.py test --settings=django_project.settings.test
# then click through: /leads/inbox/, a lead's detail (Log call),
/team/me/dialer/, /leads/board/, /dashboard/
git add -A && git commit -m "day 27"
```

Day 28 — Security and deployment

Goal: the CRM holds client PII and money movements, so it ships hardened. Morning: security review and fixes (passwords, sessions, 2FA for managers, audit-log immutability, PII in logs, session revocation). Afternoon: the production stack — Docker image, gunicorn with uvicorn workers, nginx with TLS, WhiteNoise, Celery for slow side effects, backups, and a CI pipeline that deploys on merge.

A. Drills (1–10) — security review

- [ ] **1.** `check --deploy` with prod settings: zero errors; decide on each warning.
- [ ] **2.** Argon2 password hashing; 12-character minimum; a custom validator rejecting the company name.
- [ ] **3.** Session policy: 9-hour shift, expire at browser close, `SameSite=Lax`, optional idle timeout.
- [ ] **4.** Login lockout in the authentication backend, so the form, token and JWT endpoints all respect it.
- [ ] **5.** XSS audit: grep for `|safe`, `mark_safe`, `autoescape off`, `innerHTML`; justify each hit.
- [ ] **6.** SQL injection audit: grep raw SQL; every value passed as a parameter.
- [ ] **7.** CSRF audit: every `csrf_exempt` / `authentication_classes = []` has a reason (webhooks).
- [ ] **8.** Clickjacking: `X_FRAME_OPTIONS = "DENY"` + CSP `frame-ancestors 'none'`.
- [ ] **9.** Admin hardening: URL from env, IP allowlist middleware.
- [ ] **10.** Scanners: `bandit`, `pip-audit`, `gitleaks` — locally and in CI.

B. Core (11–20) — the production stack

- [ ] **11.** Pinned, hashed requirements with pip-tools (`requirements.in` → `requirements.txt`; dev separate).
- [ ] **12.** `gunicorn.conf.py` with uvicorn workers (ASGI, so Day 27's SSE works), timeouts, worker recycling.
- [ ] **13.** Static files: WhiteNoise + `CompressedManifestStaticFilesStorage`; `collectstatic`.

- [ ] **14.** Production `prod.py` fixes: console-only logging, health check exempt from the HTTPS redirect, `ALLOWED_HOSTS`.
- [ ] **15.** Multi-stage `Dockerfile`, non-root user, static files collected at build time.
- [ ] **16.** `docker-compose.prod.yml`: web, worker, db, redis, nginx, named volumes, health checks.
- [ ] **17.** nginx: TLS, HTTP→HTTPS, proxy headers, upload limit, media, no buffering for SSE.
- [ ] **18.** TLS certificates with certbot (webroot) and auto-renewal.
- [ ] **19.** Deploy-safe migrations: run before switching code; expand → backfill → contract.
- [ ] **20.** Nightly `pg_dump` with 14-day retention, off-site copy, and a restore drill.

C. Nexora CRM (21–30)

- [ ] **21.** Logging filter that redacts phone numbers and emails from log messages.
- [ ] **22.** Append-only audit log: a PostgreSQL trigger blocks UPDATE/DELETE on `ActivityLog` (purges only with an explicit session flag).
- [ ] **23.** `purge_old_data` command: retention windows for access logs, email logs, idempotency records, login events.
- [ ] **24.** `security_audit` command: elevated users, dormant accounts, unused API keys, live sessions.
- [ ] **25.** Deactivating a user ends their sessions (cache too), deletes tokens and disables API keys.
- [ ] **26.** "Log out everywhere" button on the profile page.
- [ ] **27.** HSTS and security headers verified on an HTTPS request.
- [ ] **28.** Custom 404/500 pages; the 500 page shows the request id and needs no database.
- [ ] **29.** Dependabot for pip, Docker and GitHub Actions.
- [ ] **30.** Celery + Redis: emails and broker webhooks become retried background tasks.

D. Challenge (31–40)

- [ ] **31.** TOTP two-factor for managers and admins (pyotp, QR enrolment, replay protection, enforcing middleware).
- [ ] **32.** KYC downloads served by nginx via `X-Accel-Redirect` after Django checks permission.

- [ ] **33.** CSP violation reports endpoint; trial a stricter policy with `Content-Security-Policy-Report-Only`.
- [ ] **34.** Sentry error tracking without PII.
- [ ] **35.** GitHub Actions: test → build and push image to GHCR → migrate and restart over SSH.
- [ ] **36.** Image-level checks: `check --deploy` and `migrate --check` inside the built image; `trivy` vulnerability scan.
- [ ] **37.** Test: deploy checks pass (no errors) with production-like settings.
- [ ] **38.** Test: deactivating a user kills their session and token.
- [ ] **39.** Test (PostgreSQL): updating an `ActivityLog` row raises.
- [ ] **40.** Test: the redaction filter masks a phone number.

Solutions

A. Security review (1–10)

1.

```
python manage.py check --deploy --settings=django_project.settings.prod
```

Typical leftovers and the decision for each: `security.W021` (HSTS preload) — silence it deliberately (preloading is a one-way commitment for the whole domain); anything about `SECRET_KEY`, `DEBUG`, secure cookies or `ALLOWED_HOSTS` — fix, never silence. Day 22's `crm.E001` must not appear.

2.

```
python -m pip install "django[argon2]"        # pulls in argon2-cffi
```

```
# settings/base.py
PASSWORD_HASHERS = [
    "django.contrib.auth.hashers.Argon2PasswordHasher",     # new and changed
passwords
    "django.contrib.auth.hashers.PBKDF2PasswordHasher",     # old hashes
still verify, then upgrade on next login
]

AUTH_PASSWORD_VALIDATORS = [
    {"NAME":
"django.contrib.auth.password_validation.UserAttributeSimilarityValidator"},
    {"NAME":
"django.contrib.auth.password_validation.MinimumLengthValidator", "OPTIONS":
{"min_length": 12}},
    {"NAME":
"django.contrib.auth.password_validation.CommonPasswordValidator"},
    {"NAME":
"django.contrib.auth.password_validation.NumericPasswordValidator"},
    {"NAME": "accounts.validators.NoCompanyNameValidator"},
]
```

```
# accounts/validators.py
from django.core.exceptions import ValidationError


class NoCompanyNameValidator:
    BANNED = ("nexora", "password", "dubai")

    def validate(self, password, user=None):
        lowered = password.lower()
        if any(word in lowered for word in self.BANNED):
            raise ValidationError("Don't use the company name, 'password' or
'dubai' in your password.",
                                  code="password_has_company_word")

    def get_help_text(self):
        return "Your password can't contain the company name."
```

The Day 8 seed passwords (`nexora123`) now fail validation for *new* passwords only; existing hashes still log in. Set real passwords via Day 15's invite flow before go-live.

3.

```
# settings/base.py
SESSION_COOKIE_AGE = 60 * 60 * 9            # one shift
SESSION_EXPIRE_AT_BROWSER_CLOSE = True
SESSION_COOKIE_SAMESITE = "Lax"
CSRF_COOKIE_SAMESITE = "Lax"
# idle timeout instead of a fixed shift: SESSION_COOKIE_AGE = 30 * 60 +
SESSION_SAVE_EVERY_REQUEST = True
# (every request rewrites the session — fine with Day 24's cached_db
sessions)
```

Django already rotates the session key on login (`cycle_key`), which blocks session fixation.

4.

```
# accounts/backends.py
from .lockout import is_locked_out          # the Day 15 helper your login
form used


class EmailOrUsernameBackend(ModelBackend):
    ...
    def user_can_authenticate(self, user):
        # every authenticate() call goes through here: login form, admin, DRF
token, JWT
        return super().user_can_authenticate(user) and not
is_locked_out(user.username)
```

`authenticate()` also sends `user_login_failed` on every path, so Day 15's `record_failure` receiver counts API failures too.

5–7.

```
# 5: XSS — each hit needs a reason
grep -rnE "\|safe|mark_safe|autoescape off|innerHTML|insertAdjacentHTML" --
include=*.py --include=*.html --include=*.js --include=*.txt .
# 6: SQL — values must be parameters, never f-strings
grep -rnE "\.raw\(|\.extra\(|cursor\.execute|RawSQL\(" --include=*.py .
# 7: CSRF exemptions
grep -rnE "csrf_exempt|authentication_classes = \[\]" --include=*.py .
```

Expected justified hits: `autoescape off` only in plain-text email templates; `mark_safe` only inside template tags that build HTML with `format_html`; `insertAdjacentHTML` only for Django-rendered partials (Day 27 #9); raw SQL in Day 22/24/29 with `%s` / `%(name)s` parameters (table names come from `Model._meta.db_table`, never from input); `authentication_classes = []` only on HMAC-verified webhooks.

```
# 6: the rule
cursor.execute(f"SELECT * FROM crm_leads WHERE phone = '{phone}'")          #
NO — injectable
cursor.execute("SELECT * FROM crm_leads WHERE phone = %s", [phone])         #
yes — the driver quotes it
```

8.

```
# settings/base.py
X_FRAME_OPTIONS = "DENY"
# core/middleware.py — SecurityHeadersMiddleware's CSP gains: frame-ancestors
'none'
```

9.

```
# settings/base.py
ADMIN_URL = env.str("ADMIN_URL", default="admin/")              # e.g. "ops-
7f3a/" in production
ADMIN_ALLOWED_IPS = env.list("ADMIN_ALLOWED_IPS", default=[])   # office /
VPN egress IPs

# django_project/urls.py
path(settings.ADMIN_URL, admin.site.urls),


# core/middleware.py (after SecurityMiddleware)
class AdminAllowlistMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        allowed = settings.ADMIN_ALLOWED_IPS
        if allowed and request.path.startswith("/" + settings.ADMIN_URL):
            ip = request.META.get("HTTP_X_REAL_IP") or
request.META.get("REMOTE_ADDR")   # nginx sets X-Real-IP
```

```
            if ip not in allowed:
                raise Http404                   # don't even reveal the admin
exists
        return self.get_response(request)
```

Trust `X-Real-IP` only because nginx overwrites it; if Django is ever reachable without nginx, a client could forge it.

10.

```
python -m pip install bandit pip-audit
bandit -r . -x ./tests,./.venv -ll          # medium+ severity Python
security smells
pip-audit -r requirements.txt               # known CVEs in pinned
dependencies
docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:latest detect --
source=/repo   # secrets in git history
```

```
# .github/workflows/tests.yml — extra steps
      - run: pip install bandit pip-audit
      - run: bandit -r . -x ./.venv -ll
      - run: pip-audit -r requirements.txt
```

B. The production stack (11–20)

11.

```
# requirements.in — what the app needs (ranges)
Django>=5.2,<5.3
environs[django]
psycopg[binary,pool]
djangorestframework
djangorestframework-simplejwt
drf-spectacular
django-filter
django-crispy-forms
crispy-bootstrap5
django-cors-headers
redis
Pillow
openpyxl
argon2-cffi
gunicorn
```

```
uvicorn-worker
whitenoise
celery
pyotp
qrcode
sentry-sdk

# requirements-dev.in
-c requirements.txt
factory-boy
coverage
django-debug-toolbar
pyinstrument
locust
ruff
bandit
pip-audit
```

```
python -m pip install pip-tools
pip-compile --generate-hashes requirements.in -o requirements.txt          #
exact versions + hashes
pip-compile requirements-dev.in -o requirements-dev.txt
pip-sync requirements.txt requirements-dev.txt                              #
venv = exactly these
```

12.

```
# gunicorn.conf.py
import multiprocessing
import os

bind = "0.0.0.0:8000"
workers = int(os.environ.get("WEB_CONCURRENCY", multiprocessing.cpu_count() *
2 + 1))
worker_class = "uvicorn_worker.UvicornWorker"   # ASGI: async views (SSE)
work; sync views run in a thread pool
timeout = 60
graceful_timeout = 30                           # finish in-flight requests
on SIGTERM
keepalive = 5
max_requests = 1000                             # recycle workers: no slow
memory creep
max_requests_jitter = 100                       # ...and not all at once
accesslog = "-"
errorlog = "-"
forwarded_allow_ips = "*"                       # only nginx can reach this
port
```

```
gunicorn django_project.asgi:application -c gunicorn.conf.py
```

13-14.

```
# settings/base.py — 13
MIDDLEWARE.insert(MIDDLEWARE.index("django.middleware.security.SecurityMiddle
ware") + 1,
                  "whitenoise.middleware.WhiteNoiseMiddleware")
STATIC_ROOT = BASE_DIR / "staticfiles"
STORAGES["staticfiles"] = {"BACKEND":
"whitenoise.storage.CompressedManifestStaticFilesStorage"}
# hashed file names (crm.4f2a9c.css) → cached for a year, busted on every
change


# settings/prod.py — 14 (additions)
ALLOWED_HOSTS = env.list("ALLOWED_HOSTS")             #
crm.nexora.ae,localhost,127.0.0.1 (container health check)
SECURE_REDIRECT_EXEMPT = [r"^health/$"]             # the health check speaks
plain HTTP inside the network
SILENCED_SYSTEM_CHECKS = ["security.W021"]          # HSTS preload: decided
later, on purpose
LOGGING["handlers"].pop("file", None)               # containers log to
stdout; no file to open as non-root
LOGGING["loggers"]["crm"]["handlers"] = ["console"]
USE_X_ACCEL_REDIRECT = True                          # 32
```

15.

```
# Dockerfile
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app

FROM base AS build
RUN apt-get update && apt-get install -y --no-install-recommends build-
essential \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip wheel --wheel-dir /wheels -r requirements.txt     # hashes checked
here

FROM base AS runtime
RUN useradd --create-home --uid 1000 crm
COPY --from=build /wheels /wheels
RUN pip install --no-index --find-links=/wheels /wheels/* && rm -rf /wheels
```

```
COPY --chown=crm:crm . .
RUN DJANGO_SETTINGS_MODULE=django_project.settings.prod \
    SECRET_KEY=build-only CRM_API_KEY=build-only
ADMIN_EMAIL=build@example.com \
    ALLOWED_HOSTS=localhost DATABASE_URL=sqlite:////tmp/build.db \
    python manage.py collectstatic --noinput
USER crm
EXPOSE 8000
CMD ["gunicorn", "django_project.asgi:application", "-c", "gunicorn.conf.py"]
```

```
# .dockerignore
.git
.venv
.env*
*.sqlite3
media/
private_media/
logs/
staticfiles/
__pycache__/
```

16.

```
# docker-compose.prod.yml
x-app: &app
  image: ghcr.io/nexora/nexora-crm:${TAG:-latest}
  env_file: .env.prod
  environment:
    DJANGO_SETTINGS_MODULE: django_project.settings.prod
  volumes:
    - media:/app/media
    - private_media:/app/private_media
  depends_on:
    db: {condition: service_healthy}
    redis: {condition: service_started}
  restart: unless-stopped

services:
  web:
    <<: *app
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/health/', timeout=3)"]
```

```
      interval: 30s
      timeout: 5s
      retries: 3
  worker:
    <<: *app
    command: celery -A django_project worker -l info --concurrency 4
  db:
    image: postgres:16
    env_file: .env.db
    command: postgres -c shared_preload_libraries=pg_stat_statements
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U nexora -d nexora_crm"]
      interval: 5s
      retries: 10
    restart: unless-stopped
  redis:
    image: redis:7
    command: redis-server --appendonly yes
    volumes: ["redisdata:/data"]
    restart: unless-stopped
  nginx:
    image: nginx:1.27
    ports: ["80:80", "443:443"]
    volumes:
      - ./deploy/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - media:/srv/media:ro
      - private_media:/srv/private_media:ro
      - /etc/letsencrypt:/etc/letsencrypt:ro
      - /srv/certbot:/var/www/certbot:ro
    depends_on: [web]
    restart: unless-stopped

volumes:
  pgdata:
  redisdata:
  media:
  private_media:
```

The database and Redis publish no ports: only containers on the compose network reach them.

17.

```
# deploy/nginx.conf
upstream crm { server web:8000; }

server {
    listen 80;
    server_name crm.nexora.ae;
    location /.well-known/acme-challenge/ { root /var/www/certbot; }
    location / { return 301 https://$host$request_uri; }
}

server {
    listen 443 ssl;
    http2 on;
    server_name crm.nexora.ae;
    ssl_certificate     /etc/letsencrypt/live/crm.nexora.ae/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/crm.nexora.ae/privkey.pem;

    client_max_body_size 12m;            # 10 MB KYC files + form overhead
(Day 19)
    gzip on;
    gzip_types text/css application/javascript application/json
image/svg+xml;

    location /media/ { alias /srv/media/; expires 7d; }                #
avatars only — public
    location /protected/ { internal; alias /srv/private_media/; }      # 32:
only via X-Accel-Redirect

    location /alerts/stream/ {                                         # Day
27 SSE
        proxy_pass http://crm;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_buffering off;
        proxy_read_timeout 1h;
    }

    location / {
        proxy_pass http://crm;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;       # Django's
SECURE_PROXY_SSL_HEADER reads this
```

```
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 75s;
    }
}
```

18.

```
# first certificate: start nginx with only the port-80 server block (no certs
exist yet), then:
docker run --rm -v /etc/letsencrypt:/etc/letsencrypt -v
/srv/certbot:/var/www/certbot \
  certbot/certbot certonly --webroot -w /var/www/certbot -d crm.nexora.ae --
email ops@nexora.ae
# add the 443 block back and reload nginx. Renewal (host crontab, daily):
0 3 * * * docker run --rm -v /etc/letsencrypt:/etc/letsencrypt -v
/srv/certbot:/var/www/certbot certbot/certbot renew --quiet && docker compose
-f /srv/nexora-crm/docker-compose.prod.yml exec nginx nginx -s reload
```

19. Order on every deploy: pull image → `migrate` with the *new* image while the *old* code still serves → restart web/worker. So every migration must work with the old code running:

```
expand    add the column nullable / with a default; add new tables
(deploy 1)
backfill  data migration or management command in batches
(deploy 1 or a command)
contract  make it NOT NULL, drop the old column, add the constraint
(deploy 2, after code stops using the old shape)
```

Never rename a column in one step; never add a NOT NULL column without a default to a big table in the same deploy as the code that writes it. `python manage.py migrate --check` (non-zero exit when migrations are pending) guards the health check script.

20.

```
#!/usr/bin/env bash
# deploy/backup.sh — host crontab: 30 1 * * * /srv/nexora-
crm/deploy/backup.sh
set -euo pipefail
cd /srv/nexora-crm
STAMP=$(date -u +%Y%m%dT%H%M%SZ)
mkdir -p /backups
docker compose -f docker-compose.prod.yml exec -T db pg_dump -U nexora -d
nexora_crm -Fc > "/backups/nexora_crm-$STAMP.dump"
docker run --rm -v nexora-crm_private_media:/data:ro -v /backups:/backups
alpine \
  tar czf "/backups/private_media-$STAMP.tgz" -C /data .
find /backups -name 'nexora_crm-*.dump' -mtime +14 -delete
find /backups -name 'private_media-*.tgz' -mtime +14 -delete
# off-site: copy to another provider/region, e.g. aws s3 cp
"/backups/nexora_crm-$STAMP.dump" s3://nexora-backups/db/
```

```
# restore drill (monthly, into a scratch database — a backup you never
restored is a hope, not a backup)
docker compose -f docker-compose.prod.yml exec -T db createdb -U nexora
nexora_restore
docker compose -f docker-compose.prod.yml exec -T db pg_restore -U nexora -d
nexora_restore --no-owner < /backups/<file>.dump
docker compose -f docker-compose.prod.yml exec -T db psql -U nexora -d
nexora_restore -c "SELECT count(*) FROM crm_leads;"
```

C. Nexora CRM security (21–30)

21.

```
# core/logging.py (add)
import re

PHONE_RE = re.compile(r"(?<!\d)(?:\+?971|0)?5\d{8}(?!\d)")        # UAE
mobiles: 9715…, 05…, +9715…
EMAIL_RE = re.compile(r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+")


def redact(text):
    text = PHONE_RE.sub(lambda m: "***" + m.group()[-4:], text)
    return EMAIL_RE.sub(lambda m: m.group()[0] + "***@" +
m.group().split("@", 1)[1], text)
```

```
class RedactPIIFilter(logging.Filter):
    def filter(self, record):
        record.msg = redact(record.getMessage())      # format first, then
redact the final text
        record.args = None
        return True

# settings/base.py — LOGGING
# "filters": {..., "redact_pii": {"()": "core.logging.RedactPIIFilter"}}
# console and file handlers: "filters": ["request_id", "redact_pii"]
```

22.

```
# core/migrations/00NN_activitylog_append_only.py
from django.db import migrations

FUNCTION = """
CREATE OR REPLACE FUNCTION crm_append_only() RETURNS trigger AS $$
BEGIN
    IF TG_OP = 'DELETE' AND current_setting('crm.allow_purge', true) = 'on'
THEN
        RETURN OLD;                                  -- deliberate, flagged
purge
    END IF;
    RAISE EXCEPTION '% on % is not allowed: the audit log is append-only',
TG_OP, TG_TABLE_NAME;
END;
$$ LANGUAGE plpgsql;
"""


def create_trigger(apps, schema_editor):
    table = apps.get_model("core", "ActivityLog")._meta.db_table
    schema_editor.execute(FUNCTION, params=None)   # params=None: the % signs
are PL/pgSQL's, not placeholders
    schema_editor.execute(f'CREATE TRIGGER activitylog_append_only BEFORE
UPDATE OR DELETE ON "{table}" '
                          "FOR EACH ROW EXECUTE FUNCTION crm_append_only()",
params=None)
```

```
def drop_trigger(apps, schema_editor):
    table = apps.get_model("core", "ActivityLog")._meta.db_table
    schema_editor.execute(f'DROP TRIGGER IF EXISTS activitylog_append_only ON
"{table}"', params=None)
    schema_editor.execute("DROP FUNCTION IF EXISTS crm_append_only()",
params=None)


class Migration(migrations.Migration):
    dependencies = [("core", "00NN_previous")]
    operations = [migrations.RunPython(create_trigger, drop_trigger)]
```

Even a superuser in the Django shell can't edit history now. If `Lead` has a `GenericRelation` to `ActivityLog`, deleting a lead will fail — intended: leads are soft-deleted (`is_active`).

23.

```
# core/management/commands/purge_old_data.py
from datetime import timedelta

from django.core.management.base import BaseCommand
from django.db import connection, transaction
from django.utils import timezone

from accounts.models import LoginEvent
from api.models import IdempotencyRecord
from core.models import AccessLog, ActivityLog, EmailLog

RETENTION = [                    # (model, days) — agree these numbers with
compliance before go-live
    (AccessLog, 180),
    (EmailLog, 90),
    (IdempotencyRecord, 30),
    (LoginEvent, 365),
]


class Command(BaseCommand):
    help = "Delete rows past their retention window"

    def add_arguments(self, parser):
        parser.add_argument("--dry-run", action="store_true")
```

```
        parser.add_argument("--activity-days", type=int, help="also purge the
audit log older than N days")

    def handle(self, *args, dry_run, activity_days, **options):
        now = timezone.now()
        plan = list(RETENTION) + ([(ActivityLog, activity_days)] if
activity_days else [])
        with transaction.atomic():
            if activity_days:
                with connection.cursor() as cursor:
                    cursor.execute("SET LOCAL crm.allow_purge = 'on'")     #
lifts the 22 trigger for this transaction only
            for model, days in plan:
                qs = model.objects.filter(created_at__lt=now -
timedelta(days=days))
                count = qs.count()
                if not dry_run:
                    qs.delete()
                self.stdout.write(f"{model.__name__:<20} older than {days:>3}
days: {count}")
            if dry_run:
                transaction.set_rollback(True)
```

24.

```
# accounts/management/commands/security_audit.py
from datetime import timedelta

from django.contrib.auth import get_user_model
from django.contrib.sessions.models import Session
from django.core.management.base import BaseCommand
from django.db.models import Q
from django.utils import timezone

from api.models import ApiKey


class Command(BaseCommand):
    help = "Print who holds power, who is dormant, and which keys are unused"

    def section(self, title):
        self.stdout.write(self.style.MIGRATE_HEADING(f"\n{title}"))
```

```
    def handle(self, *args, **options):
        User = get_user_model()
        now = timezone.now()

        self.section("Superusers / staff / managers")
        for u in User.objects.filter(Q(is_superuser=True) | Q(is_staff=True)
| Q(role__in=["manager", "admin"]), is_active=True):
            self.stdout.write(f"  {u.username:<20} role={u.role:<10}
superuser={u.is_superuser} 2fa={hasattr(u, 'totp')}")

        self.section("Active accounts with no login for 30+ days")
        for u in
User.objects.filter(is_active=True).filter(Q(last_login__lt=now -
timedelta(days=30)) | Q(last_login=None)):
            self.stdout.write(f"  {u.username:<20} last_login=
{u.last_login}")

        self.section("API keys unused for 90+ days")
        for k in
ApiKey.objects.filter(is_active=True).filter(Q(last_used_at__lt=now -
timedelta(days=90)) | Q(last_used_at=None)):
            self.stdout.write(f"  {k} owner={k.user.username} created=
{k.created_at:%Y-%m-%d}")

        self.section("Live sessions per user")
        counts = {}
        for s in Session.objects.filter(expire_date__gt=now).iterator():
            uid = s.get_decoded().get("_auth_user_id")
            if uid:
                counts[uid] = counts.get(uid, 0) + 1
        for u in User.objects.filter(pk__in=counts):
            self.stdout.write(f"  {u.username:<20} {counts[str(u.pk)]}
session(s)")
```

25–26.

```
# accounts/sessions.py
from importlib import import_module

from django.conf import settings
from django.contrib.sessions.models import Session
from django.utils import timezone
from rest_framework.authtoken.models import Token
```

```
from api.models import ApiKey


def end_user_sessions(user, keep_session_key=None, revoke_api_keys=True):
    store_class = import_module(settings.SESSION_ENGINE).SessionStore
    for session in
Session.objects.filter(expire_date__gt=timezone.now()).iterator():
        if session.session_key == keep_session_key:
            continue
        if session.get_decoded().get("_auth_user_id") == str(user.pk):
            # through the store, not session.delete(): cached_db keeps a copy
in Redis too
            store_class(session_key=session.session_key).delete()
    Token.objects.filter(user=user).delete()
    if revoke_api_keys:
        ApiKey.objects.filter(user=user).update(is_active=False)


# accounts/signals.py — 25
@receiver(pre_save, sender=CustomUser, dispatch_uid="detect_deactivation")
def detect_deactivation(sender, instance, raw=False, **kwargs):
    if raw or not instance.pk:
        return
    was_active =
sender.objects.filter(pk=instance.pk).values_list("is_active",
flat=True).first()
    instance._deactivated = bool(was_active) and not instance.is_active


@receiver(post_save, sender=CustomUser,
dispatch_uid="end_sessions_on_deactivate")
def end_sessions_on_deactivate(sender, instance, **kwargs):
    if getattr(instance, "_deactivated", False):
        transaction.on_commit(lambda: end_user_sessions(instance))


# accounts/views.py — 26
class LogoutEverywhereView(View):
    def post(self, request):
        end_user_sessions(request.user,
```

```
keep_session_key=request.session.session_key, revoke_api_keys=False)
        messages.success(request, "All your other sessions and API tokens
have been signed out.")
        return redirect("accounts:profile")
```

27.

```
# core/tests.py
@override_settings(SECURE_HSTS_SECONDS=3600, SECURE_SSL_REDIRECT=False)
def test_hsts_on_https(self):
    response = self.client.get(reverse("login"), secure=True)      # HSTS is
only sent over HTTPS
    self.assertIn("max-age=3600", response["Strict-Transport-Security"])
    self.assertEqual(response["X-Frame-Options"], "DENY")
    self.assertIn("frame-ancestors 'none'", response["Content-Security-
Policy"])
```

28.

```
# core/views.py
from django.http import HttpResponseServerError
from django.template import loader
from core.context import _request_id


def server_error(request):
    # no request context: context processors might hit the database that just
failed
    html = loader.get_template("500.html").render({"request_id":
_request_id.get()})
    return HttpResponseServerError(html)

# django_project/urls.py
handler500 = "core.views.server_error"
```

```
<!-- templates/500.html — standalone, inline CSS only, no {% url %}, no
database -->
<!doctype html><html lang="en"><head><meta charset="utf-8"><title>Something
went wrong</title></head>
<body style="font-family:system-ui;max-width:36rem;margin:4rem auto">
  <h1>Something went wrong</h1>
  <p>The team has been notified. If you contact support, quote this
reference:</p>
  <p><code>{{ request_id }}</code></p>
  <p><a href="/">Back to the CRM</a></p>
</body></html>
```

`templates/404.html` can extend `base.html` (a 404 means Django and the database are fine). Neither page shows tracebacks — that only happens with `DEBUG=True`.

29.

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: pip
    directory: "/"
    schedule: {interval: weekly}
    open-pull-requests-limit: 5
  - package-ecosystem: docker
    directory: "/"
    schedule: {interval: weekly}
  - package-ecosystem: github-actions
    directory: "/"
    schedule: {interval: monthly}
```

30.

```
# django_project/celery.py
import os
from celery import Celery

os.environ.setdefault("DJANGO_SETTINGS_MODULE",
"django_project.settings.prod")
app = Celery("nexora_crm")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()          # finds tasks.py in every installed app

# django_project/__init__.py
from .celery import app as celery_app
__all__ = ("celery_app",)


# settings/base.py
CELERY_BROKER_URL = env.str("CELERY_BROKER_URL",
default="redis://localhost:6379/2")
CELERY_TASK_ALWAYS_EAGER = env.bool("CELERY_TASK_ALWAYS_EAGER",
default=False)   # True in test.py and in dev without a worker
CELERY_TASK_ACKS_LATE = True          # a crashed worker's task is
redelivered
CELERY_TASK_TIME_LIMIT = 60
```

```
# core/tasks.py
from celery import shared_task
from core.emails import deliver
from core.models import EmailLog


@shared_task(autoretry_for=(RuntimeError,), retry_backoff=True,
max_retries=5)
def deliver_email(log_id):
    if not deliver(EmailLog.objects.get(pk=log_id)):
        raise RuntimeError("send failed")      # → retried after 1 s, 2 s, 4
s, …


# core/emails.py — web requests queue; commands that batch over one SMTP
connection keep send_templated_email
def queue_templated_email(prefix, to, context):
    to = [a for a in to if a]
```

```
    if not to:
        return None
    subject, text, html = render_email(prefix, context)
    log = EmailLog.objects.create(template=prefix, to=", ".join(to),
subject=subject[:200], body_text=text, body_html=html)
    transaction.on_commit(lambda: deliver_email.delay(log.pk))    # only
after the data really exists
    return log


# brokers/tasks.py
@shared_task(autoretry_for=(OSError,), retry_backoff=True, max_retries=6)
def broker_webhook_task(broker_id, event, payload):
    broker = Broker.objects.get(pk=broker_id)
    if send_webhook(broker, event, payload) is None:
        raise OSError(f"webhook to {broker.code} failed")

# Day 26 receiver: transaction.on_commit(lambda:
broker_webhook_task.delay(broker.pk, "lead.converted", payload))
# Day 23 lead-assignment / KYC / large-deposit emails: send_templated_email →
queue_templated_email
```

D. Challenge (31–36)

31.

```
# accounts/models.py
class TOTPDevice(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name="totp")
    secret = models.CharField(max_length=32)
    confirmed = models.BooleanField(default=False)
    last_used_step = models.BigIntegerField(default=0)       # replay
protection
    created_at = models.DateTimeField(auto_now_add=True)


# accounts/otp.py
import hmac
import pyotp
import qrcode
import qrcode.image.svg
from django.utils import timezone
```

```
def verify_code(device, code):
    totp = pyotp.TOTP(device.secret)
    now_step = int(timezone.now().timestamp() // 30)
    for step in (now_step - 1, now_step, now_step + 1):            # ±30 s
clock drift
        if step > device.last_used_step and hmac.compare_digest(totp.at(step
* 30), code.strip()):
            device.last_used_step = step                            # the
same code can't be used twice
            device.save(update_fields=["last_used_step"])
            return True
    return False


def qr_svg(device, user):
    uri = pyotp.TOTP(device.secret).provisioning_uri(name=user.email or
user.username, issuer_name="Nexora CRM")
    return qrcode.make(uri,
image_factory=qrcode.image.svg.SvgPathImage).to_string(encoding="unicode")
```

```
# accounts/views.py
from django.utils.http import url_has_allowed_host_and_scheme


def safe_next(request):
    next_url = request.GET.get("next", "")
    ok = url_has_allowed_host_and_scheme(next_url, allowed_hosts=
{request.get_host()}, require_https=request.is_secure())
    return next_url if ok else reverse("dashboard")


class OTPSetupView(View):
    template_name = "accounts/otp_setup.html"

    def device(self):
        device, _ = TOTPDevice.objects.get_or_create(user=self.request.user,
defaults={"secret": pyotp.random_base32()})
        return device

    def get(self, request):
        device = self.device()
```

```
        if device.confirmed:
            return redirect("accounts:otp_verify")
        return render(request, self.template_name, {"qr": qr_svg(device,
request.user), "secret": device.secret})

    def post(self, request):
        device = self.device()
        if verify_code(device, request.POST.get("code", "")):
            device.confirmed = True
            device.save(update_fields=["confirmed"])
            request.session["otp_verified"] = True
            return redirect(safe_next(request))
        return render(request, self.template_name,
                      {"qr": qr_svg(device, request.user), "secret":
device.secret, "error": "Wrong code — try the next one."})


class OTPVerifyView(View):
    def get(self, request):
        return render(request, "accounts/otp_verify.html")

    def post(self, request):
        device = getattr(request.user, "totp", None)
        if device and device.confirmed and verify_code(device,
request.POST.get("code", "")):
            request.session["otp_verified"] = True
            return redirect(safe_next(request))
        return render(request, "accounts/otp_verify.html", {"error": "Wrong
code."})

# accounts/signals.py: on user_logged_in →
request.session.pop("otp_verified", None)   (every login needs a fresh code)
```

```
# core/middleware.py — after AuthenticationMiddleware and
CRMLoginRequiredMiddleware
from urllib.parse import quote


class RequireOTPMiddleware:
    ROLES = {"manager", "admin"}
    EXEMPT = {"accounts:otp_setup", "accounts:otp_verify", "logout",
"health"}
```

```
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        return self.get_response(request)

    def process_view(self, request, view_func, view_args, view_kwargs):
        user = request.user                     # the session user;
token/JWT/API-key calls are anonymous here
        if not user.is_authenticated or not (user.is_superuser or user.role
in self.ROLES):
            return None
        if request.session.get("otp_verified") or
request.resolver_match.view_name in self.EXEMPT:
            return None
        if request.path.startswith("/api/"):    # a password-only browser
session can't use the API either
            return JsonResponse({"detail": "Two-factor verification
required."}, status=403)
        enrolled = getattr(getattr(user, "totp", None), "confirmed", False)
        target = reverse("accounts:otp_verify" if enrolled else
"accounts:otp_setup")
        return redirect(f"{target}?next={quote(request.get_full_path())}")
```

The QR is rendered with `{{ qr|safe }}`: SVG the `qrcode` library generated from our own URI, never user-typed HTML — a justified hit in the item 5 audit. The secret is stored in plain text, so anyone with a database dump can generate codes: treat backups as secrets.

32.

```
# clients/views.py — KYCDownloadView.get, after the permission check and the
audit log
from django.utils.http import content_disposition_header

if settings.USE_X_ACCEL_REDIRECT:
    response = HttpResponse(content_type=doc.content_type)
    response["X-Accel-Redirect"] = f"/protected/{doc.file.name}"       #
nginx streams the file; the worker is freed
    response["Content-Disposition"] = content_disposition_header(True,
Path(doc.file.name).name)
    response["Cache-Control"] = "no-store"
    return response
return FileResponse(doc.file.open("rb"), as_attachment=True,
filename=Path(doc.file.name).name)   # dev
```

33.

```
# core/views.py
import json
from django.contrib.auth.decorators import login_not_required
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_POST


@csrf_exempt                  # browsers post reports without a CSRF token
@login_not_required
@require_POST
def csp_report(request):
    try:
        report = json.loads(request.body or b"{}").get("csp-report", {})
    except ValueError:
        report = {}
    logging.getLogger("crm.csp").warning("csp violation", extra={
        "blocked": report.get("blocked-uri"), "directive":
report.get("violated-directive"), "page": report.get("document-uri")})
    return HttpResponse(status=204)

# SecurityHeadersMiddleware: append "; report-uri /csp-report/" to the CSP.
# To trial a stricter rule without breaking pages, send it as Content-
Security-Policy-Report-Only for a week.
```

34.

```
# settings/prod.py
import sentry_sdk

if dsn := env.str("SENTRY_DSN", default=""):
    sentry_sdk.init(
        dsn=dsn,
        environment=CRM_ENVIRONMENT,
        release=CRM_BUILD,
        send_default_pii=False,          # no user IPs, cookies or headers
        max_request_body_size="never",   # never ship form bodies (phones,
KYC notes)
        traces_sample_rate=0.05,
    )
```

35.

```
# .github/workflows/deploy.yml
name: deploy
on:
  push:
    branches: [main]

jobs:
  tests:
    uses: ./.github/workflows/tests.yml          # add `workflow_call:` to
tests.yml's `on:`

  build:
    needs: tests
    runs-on: ubuntu-latest
    permissions: {contents: read, packages: write}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: ghcr.io/nexora/nexora-crm:${{ github.sha }}     # image names
must be lowercase
```

```
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: production                     # required reviewers can
gate this job
    steps:
      - uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.DEPLOY_HOST }}
          username: deploy
          key: ${{ secrets.DEPLOY_SSH_KEY }}
          script: |
            set -e
            cd /srv/nexora-crm
            export TAG=${{ github.sha }}
            docker compose -f docker-compose.prod.yml pull web worker
            docker compose -f docker-compose.prod.yml run --rm web python
manage.py migrate --noinput
            docker compose -f docker-compose.prod.yml up -d web worker
            echo "TAG=$TAG" > .deployed                # Day 30's rollback
reads this
```

36.

```
# deploy.yml — build job, after the push
      - run: |
          docker run --rm -e
DJANGO_SETTINGS_MODULE=django_project.settings.prod \
            -e SECRET_KEY="$(openssl rand -base64 48)" -e
CRM_API_KEY="$(openssl rand -hex 24)" \
            -e ADMIN_EMAIL=ci@example.com -e ALLOWED_HOSTS=crm.nexora.ae -e
DATABASE_URL=sqlite:////tmp/ci.db \
            ghcr.io/nexora/nexora-crm:${{ github.sha }} \
            sh -c "python manage.py check --deploy --fail-level ERROR &&
python manage.py migrate --noinput && python manage.py migrate --check"
      - uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ghcr.io/nexora/nexora-crm:${{ github.sha }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
```

The image carries production dependencies only (no factory-boy or coverage), so unit tests run in the CI virtualenv; the image gets the deploy check, a migrations smoke run and the vulnerability scan. (SQLite is enough to prove the migrations load; Postgres-only migrations skip themselves by vendor or need the CI Postgres service.)

Tests (37–40)

```
# core/tests.py
import logging
import secrets
from io import StringIO

from django.contrib.sessions.models import Session
from django.core.management import call_command
from django.db import DatabaseError, connection, transaction
from django.test import TestCase, override_settings
from rest_framework.authtoken.models import Token

from core.factories import AgentFactory, LeadFactory
from core.logging import RedactPIIFilter
from core.models import ActivityLog


class DeployHardeningTests(TestCase):
    @override_settings(
        DEBUG=False, SECRET_KEY=secrets.token_urlsafe(50),
CRM_API_KEY=secrets.token_urlsafe(40),
        ALLOWED_HOSTS=["crm.nexora.ae"], SECURE_SSL_REDIRECT=True,
SECURE_HSTS_SECONDS=2_592_000,
        SECURE_HSTS_INCLUDE_SUBDOMAINS=True, SESSION_COOKIE_SECURE=True,
CSRF_COOKIE_SECURE=True,
        MAINTENANCE_MODE=False,
    )
    def test_deploy_checks_have_no_errors(self):                          #
37
        call_command("check", deploy=True, fail_level="ERROR",
stdout=StringIO(), stderr=StringIO())

    def test_deactivation_kills_session_and_token(self):                  #
38
        user = AgentFactory()
        self.client.force_login(user)
```

```
        Token.objects.create(user=user)
        with self.captureOnCommitCallbacks(execute=True):
            user.is_active = False
            user.save()
        self.assertFalse(Token.objects.filter(user=user).exists())
        self.assertEqual(Session.objects.count(), 0)

    def test_activity_log_is_append_only(self):                             #
39
        if connection.vendor != "postgresql":
            self.skipTest("trigger is PostgreSQL-only")
        LeadFactory()                                       # its post_save
writes an ActivityLog row
        log = ActivityLog.objects.latest("pk")
        with self.assertRaises(DatabaseError), transaction.atomic():
            ActivityLog.objects.filter(pk=log.pk).update(verb="tampered")

    def test_redacts_phone(self):                                           #
40
        record = logging.LogRecord("crm", logging.INFO, __file__, 1, "call to
%s failed", ("971501112233",), None)
        RedactPIIFilter().filter(record)
        self.assertEqual(record.getMessage(), "call to ***2233 failed")
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py test --settings=django_project.settings.test
docker build -t nexora-crm:local .
git add -A && git commit -m "day 28"
```

Day 29 — Capstone: SOP, dialer, brokers

Goal: the three things that make this *Nexora's* CRM rather than a generic one, built on everything from Days 1–28. An SOP engine that measures response times in working hours and records breaches; a dialer queue that decides who to call next and refuses do-not-call numbers; and broker commissions paid down a referral tree, with downline reporting in SQL.

New app: `python manage.py startapp sop`. New settings: `CRM_WORKING_DAYS = [0, 1, 2, 3, 4, 5]` (Mon–Sat) and `CRM_WORKING_HOURS = (9, 21)` — change them to the floor's real schedule.

A. SOP engine (1–10)

- [ ] **1.** `SOPRule` model: code, name, kind (first call / follow-up / max attempts), hours, attempts, active. Seed three rules in a data migration.
- [ ] **2.** `add_working_hours(start, hours)`: a deadline that skips nights and days off.
- [ ] **3.** `SOPBreach` model with one *open* breach per rule + lead (partial unique constraint).
- [ ] **4.** Detection per rule kind: which open leads are past their deadline right now.
- [ ] **5.** `scan_sop` command: create new breaches, resolve fixed ones, idempotent, advisory-locked; cron every 15 minutes.
- [ ] **6.** Notify: one alert + email per agent per scan (not one per lead); team leads get a copy.
- [ ] **7.** SLA countdown on lead detail ("2h 13m left" / "overdue 15m") with a little JS.
- [ ] **8.** Team-lead breach board: open breaches per agent, oldest first.
- [ ] **9.** Weekly compliance %: leads first-called within SLA ÷ leads received, per agent.
- [ ] **10.** Admin: rules editable, breaches resolvable in bulk.

B. Dialer (11–20)

- [ ] **11.** Lead fields for the dialer: `dnc`, `follow_up_at`, `claimed_by`, `claimed_until`.
- [ ] **12.** `dial_queue(agent)`: priority order — open breach, callback due, never called, high value, the rest; cooldown after a recent attempt.
- [ ] **13.** `next_lead_to_dial` v2: claim one lead for 5 minutes with `select_for_update(skip_locked=True)` so two tabs never get the same lead.

- [ ] **14.** Do-not-call: `DNCNumber` list, import command, a signal that flags matching leads, API refusal to log calls on DNC leads.
- [ ] **15.** Calling-hours guard on the queue (none outside working hours).
- [ ] **16.** Callback outcome: `follow_up_at` required, stored on the lead, honoured by the queue.
- [ ] **17.** `recording_url` on calls from the dialer webhook; listening requires a permission and is audited.
- [ ] **18.** Agent presence (available / on call / break / offline) via API.
- [ ] **19.** Team wall: presence, calls today, talk time, contact rate per agent (JSON polled every 15 s).
- [ ] **20.** Dialer KPIs: contact rate, average talk time, short answered calls (< 10 s), calls per active hour.

C. Brokers (21–30)

- [ ] **21.** Differential commission: each broker in the chain earns its rate minus the rate already paid below it.
- [ ] **22.** `CommissionStatement` + `CommissionLine`; draft → approved (approved is frozen).
- [ ] **23.** `generate_commissions --month 2026-10`: idempotent for drafts, locked, never touches approved.
- [ ] **24.** Downline report with a recursive CTE: leads, qualified, deposits for each broker including everyone below it.
- [ ] **25.** Broker referral link `/r/<code>/` → signed cookie → public lead form credits the broker.
- [ ] **26.** Public lead form: no login, honeypot, per-IP rate limit, round-robin assignment.
- [ ] **27.** Round-robin: next agent = the active agent with the fewest open leads.
- [ ] **28.** Statement export to Excel (Day 19 openpyxl).
- [ ] **29.** Broker API: an API key tied to a broker returns that broker's downline stats only.
- [ ] **30.** Feature flags `sop`, `dialer`, `commissions` (Day 22) around the new pages.

D. Challenge (31–40)

- [ ] **31.** Indexes for the scan and the queue; check with `EXPLAIN`.

- [ ] **32.** `seed_capstone`: realistic data for a demo (breaches, callbacks, a broker tree with deposits).
- [ ] **33.** Admin action: approve statements (managers only).
- [ ] **34.** Breach resolution reason when resolved by hand.
- [ ] **35.** Celery beat (or cron) schedule for `scan_sop` and `generate_commissions`.
- [ ] **36.** End-to-end test: web form → assignment → breach → call resolves it → qualify → convert → deposit webhook → commission.
- [ ] **37.** Test: working-hours deadlines across an evening and a day off.
- [ ] **38.** Test: queue order and DNC exclusion.
- [ ] **39.** Test: commission waterfall on a two-level tree.
- [ ] **40.** Test: `scan_sop` twice creates one breach; a call resolves it.

Solutions

A. SOP engine (1–10)

```
# settings/base.py
INSTALLED_APPS += ["sop.apps.SopConfig"]
CRM_WORKING_DAYS = [0, 1, 2, 3, 4, 5]        # Monday=0 … Saturday=5
CRM_WORKING_HOURS = (9, 21)                  # 09:00–21:00 Dubai time
```

1, 3.

```
# sop/models.py
from django.conf import settings
from django.db import models
from django.utils import timezone


class SOPRule(models.Model):
    class Kind(models.TextChoices):
        FIRST_CALL = "first_call", "First call within N working hours of
arrival"
        FOLLOW_UP = "follow_up", "Next attempt within N working hours of the
last one"
        MAX_ATTEMPTS = "max_attempts", "Escalate after N attempts without a
result"

    code = models.SlugField(unique=True)
```

```
    name = models.CharField(max_length=100)
    kind = models.CharField(max_length=20, choices=Kind.choices)
    hours = models.PositiveSmallIntegerField(default=24)
    attempts = models.PositiveSmallIntegerField(default=3)
    is_active = models.BooleanField(default=True)

    def __str__(self):
        return self.name


class SOPBreach(models.Model):
    rule = models.ForeignKey(SOPRule, on_delete=models.CASCADE,
related_name="breaches")
    lead = models.ForeignKey("leads.Lead", on_delete=models.CASCADE,
related_name="sop_breaches")
    agent = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                              on_delete=models.SET_NULL,
related_name="sop_breaches")
    due_at = models.DateTimeField()
    detected_at = models.DateTimeField(default=timezone.now)
    resolved_at = models.DateTimeField(null=True, blank=True)
    resolution = models.CharField(max_length=200, blank=True)            # 34

    class Meta:
        ordering = ["due_at"]
        constraints = [
            models.UniqueConstraint(fields=["rule", "lead"],
condition=models.Q(resolved_at__isnull=True),
                                    name="one_open_breach_per_rule_lead"),
# history allowed, one open at a time
        ]
        indexes = [models.Index(fields=["agent", "resolved_at"],
name="breach_agent_open_idx")]
```

```
# sop/migrations/0002_seed_rules.py
from django.db import migrations

RULES = [
    ("first-call", "First call within 24 working hours", "first_call", 24,
3),
    ("follow-up", "Follow-up within 48 working hours", "follow_up", 48, 3),
    ("max-attempts", "Escalate after 3 attempts", "max_attempts", 0, 3),
]


def seed(apps, schema_editor):
    SOPRule = apps.get_model("sop", "SOPRule")
    for code, name, kind, hours, attempts in RULES:
        SOPRule.objects.get_or_create(code=code, defaults={"name": name,
"kind": kind, "hours": hours, "attempts": attempts})


class Migration(migrations.Migration):
    dependencies = [("sop", "0001_initial")]
    operations = [migrations.RunPython(seed, migrations.RunPython.noop)]
```

2.

```
# sop/hours.py
from datetime import timedelta

from django.conf import settings
from django.utils import timezone


def _bounds(moment):
    open_h, close_h = settings.CRM_WORKING_HOURS
    day = moment.replace(minute=0, second=0, microsecond=0)
    return day.replace(hour=open_h), day.replace(hour=close_h)


def is_working_time(moment=None):
    local = timezone.localtime(moment or timezone.now())
    start, end = _bounds(local)
    return local.weekday() in settings.CRM_WORKING_DAYS and start <= local <
end
```

```
def next_opening(local):
    """The first working moment at or after `local` (an aware Dubai
datetime)."""
    candidate = local
    while True:
        start, end = _bounds(candidate)
        if candidate.weekday() in settings.CRM_WORKING_DAYS and candidate <
end:
            return max(candidate, start)
        candidate = (candidate + timedelta(days=1)).replace(hour=0, minute=0,
second=0, microsecond=0)


def add_working_hours(start, hours):
    remaining = timedelta(hours=hours)
    current = next_opening(timezone.localtime(start))
    while True:
        _, end = _bounds(current)
        available = end - current
        if remaining <= available:
            return current + remaining
        remaining -= available
        current = next_opening(end)          # closing time → next working
morning
```

Dubai has no daylight saving, so moving whole days with `timedelta` is safe here; in a DST zone you'd build each day with `zoneinfo` instead.

4–5.

```
# sop/engine.py
from datetime import timedelta

from django.db.models import Q
from django.utils import timezone

from leads.models import Lead
from .hours import add_working_hours
from .models import SOPBreach, SOPRule

OPEN = [Lead.Status.NEW, Lead.Status.CONTACTED]
```

```
def candidates(rule, now):
    """(lead, due_at) pairs that could be late. Calendar time is a cheap
lower bound: working hours are never faster."""
    leads = Lead.objects.active().filter(dnc=False).only(
        "id", "assigned_to", "created_at", "last_contacted_at",
"follow_up_at", "contact_attempts", "status")
    if rule.kind == SOPRule.Kind.FIRST_CALL:
        qs = leads.filter(status=Lead.Status.NEW, contact_attempts=0,
                          created_at__lte=now - timedelta(hours=rule.hours))
        return [(lead, add_working_hours(lead.created_at, rule.hours)) for
lead in qs.iterator()]
    if rule.kind == SOPRule.Kind.FOLLOW_UP:
        qs = (leads.filter(status=Lead.Status.CONTACTED,
last_contacted_at__lte=now - timedelta(hours=rule.hours))
              .exclude(follow_up_at__gt=now))                  # a booked
callback isn't late yet
        return [(lead, add_working_hours(lead.last_contacted_at, rule.hours))
for lead in qs.iterator()]
    if rule.kind == SOPRule.Kind.MAX_ATTEMPTS:
        qs = leads.filter(status__in=OPEN,
contact_attempts__gte=rule.attempts)
        return [(lead, now) for lead in qs.iterator()]
    return []


def scan(now=None):
    now = now or timezone.now()
    created, resolved = [], 0
    for rule in SOPRule.objects.filter(is_active=True):
        late = {lead.pk: (lead, due) for lead, due in candidates(rule, now)
if due <= now}
        open_breaches = SOPBreach.objects.filter(rule=rule,
resolved_at__isnull=True)
        resolved += (open_breaches.exclude(lead_id__in=list(late))
# condition no longer true
                     .update(resolved_at=now, resolution="auto: condition
cleared"))
        already = set(open_breaches.values_list("lead_id", flat=True))
        new = [SOPBreach(rule=rule, lead=lead, agent_id=lead.assigned_to_id,
due_at=due, detected_at=now)
               for pk, (lead, due) in late.items() if pk not in already]
```

```
        SOPBreach.objects.bulk_create(new, ignore_conflicts=True)
# a racing scan can't duplicate
        created += new
    return created, resolved
```

```
# sop/management/commands/scan_sop.py
from django.core.management.base import BaseCommand

from core.management.locks import advisory_lock
from sop.engine import scan
from sop.notify import notify_new_breaches


class Command(BaseCommand):
    help = "Detect and resolve SOP breaches (cron: every 15 minutes)"

    def handle(self, *args, **options):
        with advisory_lock("scan_sop"):
            created, resolved = scan()
            notify_new_breaches(created)
        self.stdout.write(self.style.SUCCESS(f"{len(created)} new breach(es),
{resolved} resolved."))

# crontab: */15 * * * * cd /srv/nexora-crm && docker compose -f docker-
compose.prod.yml exec -T web python manage.py scan_sop
```

6.

```
# sop/notify.py
from collections import defaultdict

from django.conf import settings
from django.contrib.auth import get_user_model

from core.emails import queue_templated_email
from core.models import Alert


def notify_new_breaches(breaches):
    if not breaches:
        return
    by_agent = defaultdict(list)
    for breach in breaches:
```

```
        by_agent[breach.agent_id].append(breach)
    agents = get_user_model().objects.select_related("team_lead").in_bulk([a
for a in by_agent if a])
    for agent_id, items in by_agent.items():
        agent = agents.get(agent_id)
        if agent is None:
            continue                                    # unassigned leads:
the team-lead alert covers them
        to = [agent.email, getattr(agent.team_lead, "email", "")]
        queue_templated_email("sop_breaches", to, {"agent": agent,
"breaches": items, "base_url": settings.CRM_BASE_URL})
    Alert.objects.create(level=Alert.Level.WARNING, for_role="team_lead",
                         title=f"{len(breaches)} new SOP breach(es)",
url="/sop/breaches/")
```

```
# templates/emails/sop_breaches_subject.txt
SOP: {{ breaches|length }} lead{{ breaches|length|pluralize }} need{{
breaches|length|pluralize:"s," }} attention

# templates/emails/sop_breaches.txt
{% autoescape off %}Hi {{ agent.first_name|default:agent.username }},

These leads are past their SOP deadline:
{% for b in breaches %}  - {{ b.lead.name }} · {{ b.rule.name }} · due {{
b.due_at|date:"D d M H:i" }} · {{ base_url }}{{ b.lead.get_absolute_url }}
{% endfor %}{% endautoescape %}
```

7.

```
# sop/engine.py (add)
def first_call_deadline(lead):
    rule = SOPRule.objects.filter(kind=SOPRule.Kind.FIRST_CALL,
is_active=True).first()
    if rule is None or lead.contact_attempts:
        return None
    return add_working_hours(lead.created_at, rule.hours)

# LeadDetailView.get_context_data
context["sop_due_at"] = first_call_deadline(lead)
```

```
<!-- leads/lead_detail.html -->
{% if sop_due_at %}
  <p>First-call deadline: <time data-countdown datetime="{{
sop_due_at|date:'c' }}">{{ sop_due_at|date:"D d M H:i" }}</time></p>
{% endif %}
```

```
// static/js/app.js (add)
function renderCountdown(node) {
  const diff = new Date(node.dateTime) - Date.now();
  const minutes = Math.round(Math.abs(diff) / 60000);
  const text = `${Math.floor(minutes / 60)}h ${minutes % 60}m`;
  node.textContent = diff >= 0 ? `${text} left` : `overdue by ${text}`;
  node.classList.toggle("is-overdue", diff < 0);
}
const countdowns = document.querySelectorAll("time[data-countdown]");
countdowns.forEach(renderCountdown);
if (countdowns.length) setInterval(() => countdowns.forEach(renderCountdown),
30_000);
```

The countdown is wall-clock time to a deadline computed in working hours, so overnight it keeps ticking down toward 10:00 the next morning — which is what an agent needs to see.

8.

```
# sop/views.py
from django.contrib.auth.mixins import UserPassesTestMixin
from django.db.models import Count, Min
from django.views.generic import TemplateView

from leads.models import Lead
from .models import SOPBreach


class BreachBoardView(UserPassesTestMixin, TemplateView):
    template_name = "sop/breaches.html"

    def test_func(self):
        return self.request.user.role in {"team_lead", "manager", "admin"}

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        open_qs = (SOPBreach.objects.filter(resolved_at__isnull=True,
```

```
lead__in=Lead.objects.visible_to(self.request.user))
                    .select_related("rule", "lead", "agent"))
        context["per_agent"] = (open_qs.values("agent__username")
                                .annotate(n=Count("id"),
oldest=Min("due_at")).order_by("-n"))
        context["breaches"] = open_qs.order_by("due_at")[:100]
        return context
```

9.

```
# sop/reports.py
from collections import defaultdict
from datetime import timedelta

from django.db.models import Min

from leads.models import Lead
from .hours import add_working_hours
from .models import SOPRule


def first_call_compliance(week_start, user):
    rule = SOPRule.objects.get(kind=SOPRule.Kind.FIRST_CALL, is_active=True)
    leads = (Lead.objects.visible_to(user)
             .filter(created_at__date__gte=week_start,
created_at__date__lt=week_start + timedelta(days=7),
                     assigned_to__isnull=False)

.annotate(first_call_at=Min("calls__started_at")).select_related("assigned_to
"))
    stats = defaultdict(lambda: {"received": 0, "on_time": 0})
    for lead in leads:
        row = stats[lead.assigned_to]
        row["received"] += 1
        if lead.first_call_at and lead.first_call_at <=
add_working_hours(lead.created_at, rule.hours):
            row["on_time"] += 1
    return sorted(({"agent": agent, **row, "pct": round(row["on_time"] * 100
/ row["received"], 1)}
                   for agent, row in stats.items()), key=lambda r: r["pct"])
```

10.

```
# sop/admin.py
from django.contrib import admin
from django.utils import timezone
from .models import SOPBreach, SOPRule


@admin.register(SOPRule)
class SOPRuleAdmin(admin.ModelAdmin):
    list_display = ["name", "kind", "hours", "attempts", "is_active"]
    list_editable = ["hours", "attempts", "is_active"]


@admin.register(SOPBreach)
class SOPBreachAdmin(admin.ModelAdmin):
    list_display = ["lead", "rule", "agent", "due_at", "resolved_at",
"resolution"]
    list_filter = ["rule", ("resolved_at", admin.EmptyFieldListFilter)]
    list_select_related = ["lead", "rule", "agent"]
    actions = ["resolve_manually"]

    @admin.action(description="Resolve selected breaches (manual)")
    def resolve_manually(self, request, queryset):                          #
34
        n = queryset.filter(resolved_at__isnull=True).update(
            resolved_at=timezone.now(), resolution=f"manual:
{request.user.username}")
        self.message_user(request, f"Resolved {n} breach(es).")
```

A manually resolved breach whose condition still holds will be re-detected on the next scan — by design: fix the lead, not the report.

B. Dialer (11–20)

11, 14, 16–18.

```
# leads/models.py (Lead additions)
class Source(models.TextChoices):
    ...
    WEBSITE = "website", "Website"                                          #
26

dnc = models.BooleanField(default=False, db_index=True)                     #
```

```
14
follow_up_at = models.DateTimeField(null=True, blank=True)                 #
16
claimed_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                                on_delete=models.SET_NULL, related_name="+")
claimed_until = models.DateTimeField(null=True, blank=True)                #
13


class DNCNumber(models.Model):                                              #
14
    phone = models.CharField(max_length=15, unique=True)
    reason = models.CharField(max_length=100, blank=True)
    added_at = models.DateTimeField(auto_now_add=True)

    def save(self, *args, **kwargs):
        self.phone = normalize_uae_phone(self.phone)
        super().save(*args, **kwargs)


# CallLog additions
class Outcome(models.TextChoices):
    ...
    CALLBACK = "callback", "Callback booked"                                #
16

recording_url = models.URLField(blank=True)                                 #
17

class Meta:
    permissions = [("listen_recording", "Can listen to call recordings")]  #
17 → add to team_lead/manager in roles.py


# accounts/models.py — CustomUser (18)
class Presence(models.TextChoices):
    AVAILABLE = "available", "Available"
    ON_CALL = "on_call", "On a call"
    BREAK = "break", "On break"
    OFFLINE = "offline", "Offline"
```

```
presence = models.CharField(max_length=10, choices=Presence.choices,
default=Presence.OFFLINE)
presence_changed_at = models.DateTimeField(null=True, blank=True)
```

12–13, 15.

```
# leads/dialer.py
from datetime import timedelta

from django.db import transaction
from django.db.models import Case, Exists, F, IntegerField, OuterRef, Value,
When
from django.utils import timezone

from sop.hours import is_working_time
from sop.models import SOPBreach
from .models import Lead

COOLDOWN = timedelta(hours=2)
HIGH_VALUE = 10_000
CLAIM = timedelta(minutes=5)


def dial_queue(agent, now=None):
    now = now or timezone.now()
    open_breach = SOPBreach.objects.filter(lead=OuterRef("pk"),
resolved_at__isnull=True)
    return (Lead.objects.active()
            .filter(assigned_to=agent, dnc=False, status__in=
[Lead.Status.NEW, Lead.Status.CONTACTED])
            .exclude(follow_up_at__gt=now)
# callback booked for later
            .exclude(claimed_until__gt=now)
# on screen in another tab
            .exclude(last_contacted_at__gt=now - COOLDOWN,
follow_up_at__isnull=True)   # tried recently, no callback due
            .annotate(has_breach=Exists(open_breach))
            .annotate(priority=Case(
                When(has_breach=True, then=Value(0)),
                When(follow_up_at__lte=now, then=Value(1)),
                When(contact_attempts=0, then=Value(2)),
                When(expected_deposit__gte=HIGH_VALUE, then=Value(3)),
```

```
                default=Value(4), output_field=IntegerField()))
            .order_by("priority", F("follow_up_at").asc(nulls_last=True),
                      F("last_contacted_at").asc(nulls_first=True),
"created_at"))


def next_lead_to_dial(agent):
    if not is_working_time():                                          #
15
        return None
    now = timezone.now()
    with transaction.atomic():
        lead = dial_queue(agent, now).select_for_update(skip_locked=True, of=
("self",)).first()
        if lead is not None:
            Lead.objects.filter(pk=lead.pk).update(claimed_by=agent,
claimed_until=now + CLAIM)
        return lead
# Day 26's LeadViewSet.next_lead now imports this version (leads.services re-
exports it).
```

14, 16–17 — the call path.

```
# leads/services.py
class DoNotCallError(Exception):
    pass


@transaction.atomic
def log_call(lead, agent, outcome, duration_seconds=0, notes="",
started_at=None,
             follow_up_at=None, recording_url=""):
    if lead.dnc:
        raise DoNotCallError("This number is on the do-not-call list.")
    started_at = started_at or timezone.now()
    call = CallLog.objects.create(lead=lead, agent=agent, outcome=outcome,
duration_seconds=duration_seconds,
                                  notes=notes, started_at=started_at,
recording_url=recording_url)
    Lead.objects.filter(pk=lead.pk).update(
        contact_attempts=F("contact_attempts") + 1,
        last_contacted_at=started_at,
        status=Case(When(status=Lead.Status.NEW,
```

```
then=Value(Lead.Status.CONTACTED)), default=F("status")),
        follow_up_at=follow_up_at,                # a new call replaces (or
clears) any booked callback
        claimed_by=None, claimed_until=None,      # release the dialer claim
        updated_at=timezone.now(),
    )
    return call


# api/exceptions.py — map the domain error to a 400
from rest_framework.exceptions import ValidationError
from leads.services import DoNotCallError


def exception_handler(exc, context):
    if isinstance(exc, DoNotCallError):
        exc = ValidationError({"lead": [str(exc)]})
    response = drf_exception_handler(exc, context)
    ...   # Day 25 body unchanged
```

```
# api/serializers.py — CallLogSerializer (16)
follow_up_at = serializers.DateTimeField(required=False, allow_null=True,
write_only=True)

class Meta:
    fields = ["id", "started_at", "agent", "outcome", "duration_seconds",
"notes", "follow_up_at"]

def validate(self, attrs):
    ...
    if attrs.get("outcome") == CallLog.Outcome.CALLBACK:
        when = attrs.get("follow_up_at")
        if not when or when <= timezone.now():
            raise serializers.ValidationError({"follow_up_at": "Pick a future
time for the callback."})
    return attrs
# api/integrations.py — DialerCallSerializer gains recording_url =
serializers.URLField(required=False, allow_blank=True)
```

```
# leads/signals.py — 14: keep the flag in sync from both sides
@receiver(pre_save, sender=Lead, dispatch_uid="lead_dnc_flag")
def flag_dnc(sender, instance, raw=False, **kwargs):
    if not raw and not instance.dnc:
        instance.dnc =
DNCNumber.objects.filter(phone=normalize_uae_phone(instance.phone)).exists()


@receiver(post_save, sender=DNCNumber, dispatch_uid="dnc_flags_leads")
def dnc_added(sender, instance, created, **kwargs):
    Lead.objects.filter(phone=instance.phone).update(dnc=True)
```

```
# leads/management/commands/import_dnc.py
import csv

from django.core.management.base import BaseCommand
from leads.models import DNCNumber, Lead, normalize_uae_phone


class Command(BaseCommand):
    help = "Import a do-not-call list (CSV with a 'phone' column)"

    def add_arguments(self, parser):
        parser.add_argument("path")

    def handle(self, *args, path, **options):
        with open(path, newline="", encoding="utf-8-sig") as f:
            phones = {normalize_uae_phone(row["phone"]) for row in
csv.DictReader(f) if row.get("phone")}
        DNCNumber.objects.bulk_create([DNCNumber(phone=p, reason="import")
for p in phones], ignore_conflicts=True)
        flagged = Lead.objects.filter(phone__in=phones,
dnc=False).update(dnc=True)   # bulk_create skipped the signal
        self.stdout.write(self.style.SUCCESS(f"{len(phones)} numbers loaded;
{flagged} lead(s) flagged."))
```

```
# leads/views.py — 17
from core.audit import log_activity


class CallRecordingView(PermissionRequiredMixin, View):
    permission_required = "leads.listen_recording"

    def get(self, request, pk):
        call =
get_object_or_404(CallLog.objects.visible_to(request.user).exclude(recording_
url=""), pk=pk)
        log_activity(call, "listened to recording", actor=request.user)    #
append-only audit (Day 28)
        return redirect(call.recording_url)
```

18–20.

```
# api/views.py (add)
from datetime import timedelta

from django.contrib.auth import get_user_model
from django.db.models import Avg, Count, Q, Sum
from django.db.models.functions import TruncHour

from leads.models import CallLog
from sop.models import SOPBreach


class PresenceSerializer(serializers.Serializer):
    presence =
serializers.ChoiceField(choices=get_user_model().Presence.choices)


@api_view(["GET", "POST"])                                              #
18
def my_presence(request):
    user = request.user
    if request.method == "POST":
        data = PresenceSerializer(data=request.data)
        data.is_valid(raise_exception=True)
        user.presence = data.validated_data["presence"]
        user.presence_changed_at = timezone.now()
        user.save(update_fields=["presence", "presence_changed_at"])
```

```
    return Response({"presence": user.presence, "since":
user.presence_changed_at})


def team_agents(user):
    agents = get_user_model().objects.filter(role="agent", is_active=True)
    return agents if user.role in {"manager", "admin"} or user.is_superuser
else agents.filter(team_lead=user)


@api_view(["GET"])                                                    #
19
@permission_classes([IsAuthenticated, IsTeamLeadOrManager])
def team_wall(request):
    agents = list(team_agents(request.user).order_by("username"))
    today = timezone.localdate()
    # aggregate from the CallLog side: two Counts across different joins on
the user would multiply
    calls = {r["agent"]: r for r in CallLog.objects.filter(agent__in=agents,
started_at__date=today).values("agent")
             .annotate(calls=Count("id"), answered=Count("id",
filter=Q(outcome="answered")),
                       talk=Sum("duration_seconds", default=0))}
    breaches = dict(SOPBreach.objects.filter(agent__in=agents,
resolved_at__isnull=True)
                    .values_list("agent").annotate(n=Count("id")))
    rows = []
    for a in agents:
        c = calls.get(a.pk, {"calls": 0, "answered": 0, "talk": 0})
        rows.append({"agent": str(a), "presence": a.presence, "since":
a.presence_changed_at,
                     "calls": c["calls"], "talk_minutes": round(c["talk"] /
60),
                     "contact_rate": round(c["answered"] * 100 / c["calls"],
1) if c["calls"] else 0,
                     "open_breaches": breaches.get(a.pk, 0)})
    return Response(rows)


def dialer_kpis(agents, since):                                       # 20
    rows = (CallLog.objects.filter(agent__in=agents,
started_at__gte=since).values("agent__username")
            .annotate(calls=Count("id"),
```

```
                      answered=Count("id", filter=Q(outcome="answered")),
                      short=Count("id", filter=Q(outcome="answered",
duration_seconds__lt=10)),
                      avg_talk=Avg("duration_seconds",
filter=Q(outcome="answered")),
                      active_hours=Count(TruncHour("started_at"),
distinct=True))
            .order_by("agent__username"))
    return [{**r, "contact_rate": round(r["answered"] * 100 / r["calls"], 1),
             "calls_per_hour": round(r["calls"] / r["active_hours"], 1),
             "avg_talk": round(r["avg_talk"] or 0)} for r in rows]

# api/urls.py
path("me/presence/", views.my_presence, name="my-presence"),
path("team/wall/", views.team_wall, name="team-wall"),
```

The wall page is a template with an empty table plus 15 lines of JS reusing Day 27's `api()` and `el()`: fetch `url("teamWall")` every 15 seconds, `replaceChildren` the rows, colour the presence cell. Short answered calls (< 10 s) are the number to watch: they're usually hang-ups dressed up as contacts.

C. Brokers (21–30)

21. Differential ("waterfall") commission: walk from the client's broker up to the root; each broker earns its own rate minus the highest rate already paid below it.

```
deposit 10,000 AED by a client of BRF1002
BRF1002  rate 1.50%  → earns 1.50 − 0.00 = 1.50%  → 150.00
BRF1001  rate 2.00%  → earns 2.00 − 1.50 = 0.50%  →  50.00   (BRF1001 is
BRF1002's parent)
```

```
# brokers/commissions.py
from collections import defaultdict
from datetime import timedelta
from decimal import Decimal

from django.db import transaction

from clients.models import Deposit
from .models import Broker, CommissionLine, CommissionStatement

CENT = Decimal("0.01")
```

```
def chain(broker, by_id):
    while broker is not None:
        yield broker
        broker = by_id.get(broker.parent_id)


def commission_lines_for(deposit, by_id):
    paid = Decimal("0")
    lines = []
    for broker in chain(by_id.get(deposit.account.client.broker_id), by_id):
        diff = broker.commission_rate - paid
        if diff > 0:
            lines.append((broker, diff, (deposit.amount * diff /
100).quantize(CENT)))
            paid = broker.commission_rate
    return lines


@transaction.atomic
def generate(period_start):                                          #
23
    period_end = (period_start + timedelta(days=32)).replace(day=1)
    by_id = {b.pk: b for b in Broker.objects.all()}
    frozen = set(CommissionStatement.objects.filter(period=period_start,
status=CommissionStatement.Status.APPROVED)
                 .values_list("broker_id", flat=True))
    CommissionStatement.objects.filter(period=period_start,
status=CommissionStatement.Status.DRAFT).delete()   # rebuild drafts
    per_broker = defaultdict(list)
    deposits = (Deposit.objects.filter(created_at__date__gte=period_start,
created_at__date__lt=period_end,
                                       account__client__broker__isnull=False)
                .select_related("account__client"))
    for deposit in deposits.iterator(chunk_size=1000):
        source = by_id[deposit.account.client.broker_id]
        for broker, rate, amount in commission_lines_for(deposit, by_id):
            if broker.pk not in frozen:                              # approved
statements are never touched
                per_broker[broker.pk].append((deposit, source, rate, amount))
    for broker_id, items in per_broker.items():
        statement = CommissionStatement.objects.create(
```

```
            broker_id=broker_id, period=period_start, total=sum(i[3] for i in
items))
        CommissionLine.objects.bulk_create([
            CommissionLine(statement=statement, deposit=d, source_broker=src,
rate=r, amount=a) for d, src, r, a in items])
    return len(per_broker)
```

22.

```
# brokers/models.py (add)
class CommissionStatement(models.Model):
    class Status(models.TextChoices):
        DRAFT = "draft", "Draft"
        APPROVED = "approved", "Approved"

    broker = models.ForeignKey(Broker, on_delete=models.PROTECT,
related_name="statements")
    period = models.DateField(help_text="First day of the month")
    status = models.CharField(max_length=10, choices=Status.choices,
default=Status.DRAFT)
    total = models.DecimalField(max_digits=14, decimal_places=2, default=0)
    generated_at = models.DateTimeField(auto_now=True)
    approved_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True, on_delete=models.PROTECT, related_name="+")
    approved_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ["-period", "broker__code"]
        constraints = [models.UniqueConstraint(fields=["broker", "period"],
name="one_statement_per_broker_month")]


class CommissionLine(models.Model):
    statement = models.ForeignKey(CommissionStatement,
on_delete=models.CASCADE, related_name="lines")
    deposit = models.ForeignKey("clients.Deposit", on_delete=models.PROTECT,
related_name="commission_lines")
    source_broker = models.ForeignKey(Broker, on_delete=models.PROTECT,
related_name="+")   # whose client deposited
    rate = models.DecimalField(max_digits=5, decimal_places=2)
    amount = models.DecimalField(max_digits=14, decimal_places=2)
```

```
    class Meta:
        constraints = [models.UniqueConstraint(fields=["statement",
"deposit"], name="one_line_per_deposit")]
```

23.

```
# brokers/management/commands/generate_commissions.py
from datetime import date, timedelta

from django.core.management.base import BaseCommand, CommandError
from django.utils import timezone

from brokers.commissions import generate
from core.management.locks import advisory_lock


class Command(BaseCommand):
    help = "Build (or rebuild) draft commission statements for a month"

    def add_arguments(self, parser):
        parser.add_argument("--month", help="YYYY-MM (default: last month)")

    def handle(self, *args, month, **options):
        if month:
            try:
                period = date.fromisoformat(f"{month}-01")
            except ValueError:
                raise CommandError("--month must be YYYY-MM")
        else:
            first_this_month = timezone.localdate().replace(day=1)
            period = (first_this_month - timedelta(days=1)).replace(day=1)
# first day of last month
        with advisory_lock("generate_commissions"):
            n = generate(period)
        self.stdout.write(self.style.SUCCESS(f"{n} draft statement(s) for
{period:%B %Y}."))
```

24. Recursive CTE: `tree` pairs every broker (as `root_id`) with itself and every broker below it; the two aggregates are pre-grouped per broker, so joining them can't multiply rows.

```
# brokers/reports.py
from django.db import connection

from clients.models import Client, Deposit, TradingAccount
from leads.models import Lead
from .models import Broker


def downline_report(since):
    t = {name: model._meta.db_table for name, model in
         {"broker": Broker, "lead": Lead, "client": Client, "account":
TradingAccount, "deposit": Deposit}.items()}
    sql = f"""
    WITH RECURSIVE tree AS (
        SELECT id, id AS root_id FROM "{t['broker']}"
        UNION ALL
        SELECT child.id, tree.root_id
        FROM "{t['broker']}" child JOIN tree ON child.parent_id = tree.id
    ),
    lead_totals AS (
        SELECT broker_id, count(*) AS leads, count(*) FILTER (WHERE status =
'qualified') AS qualified
        FROM "{t['lead']}" WHERE created_at >= %(since)s AND broker_id IS NOT
NULL
        GROUP BY broker_id
    ),
    deposit_totals AS (
        SELECT c.broker_id, sum(d.amount) AS deposits
        FROM "{t['deposit']}" d
        JOIN "{t['account']}" a ON a.id = d.account_id
        JOIN "{t['client']}" c ON c.id = a.client_id
        WHERE d.created_at >= %(since)s AND c.broker_id IS NOT NULL
        GROUP BY c.broker_id
    )
    SELECT b.code, b.name, b.tier,
           coalesce(sum(l.leads), 0)     AS leads,
           coalesce(sum(l.qualified), 0) AS qualified,
           coalesce(sum(dt.deposits), 0) AS deposits
    FROM "{t['broker']}" b
    JOIN tree ON tree.root_id = b.id
    LEFT JOIN lead_totals l     ON l.broker_id = tree.id
    LEFT JOIN deposit_totals dt ON dt.broker_id = tree.id
    GROUP BY b.id, b.code, b.name, b.tier
```

```
    ORDER BY deposits DESC
    """          # table names come from _meta (code), values from params —
Day 28 #6 rule
    with connection.cursor() as cursor:
        cursor.execute(sql, {"since": since})
        columns = [c.name for c in cursor.description]
        return [dict(zip(columns, row)) for row in cursor.fetchall()]
```

25–27.

```
# leads/assignment.py — 27
from django.contrib.auth import get_user_model
from django.db.models import Count, Q


def next_round_robin_agent():
    open_q = Q(leads__is_active=True, leads__status__in=["new", "contacted"])
    return (get_user_model().objects.filter(role="agent", is_active=True)
            .annotate(open_n=Count("leads", filter=open_q))
            .order_by("open_n", "pk").first())
```

```
# leads/public.py — 25, 26
from django import forms
from django.contrib.auth.decorators import login_not_required
from django.core.cache import cache
from django.http import HttpResponse
from django.shortcuts import get_object_or_404, redirect
from django.utils.decorators import method_decorator
from django.views.generic import FormView, TemplateView

from brokers.models import Broker
from .assignment import next_round_robin_agent
from .fields import UAEPhoneField
from .models import Lead

REF_COOKIE = "nx_ref"


@login_not_required
def referral(request, code):                                          #
25: /r/BRF1002/
    broker = get_object_or_404(Broker, code=code, is_active=True)
    response = redirect("public_lead_form")
```

```
    response.set_signed_cookie(REF_COOKIE, broker.code, salt="crm.referral",
max_age=60 * 60 * 24 * 30,
                               httponly=True, samesite="Lax",
secure=request.is_secure())
    return response


class PublicLeadForm(forms.ModelForm):
    phone = UAEPhoneField()
    website = forms.CharField(required=False, label="Leave this empty")   #
honeypot, hidden with CSS

    class Meta:
        model = Lead
        fields = ["name", "phone", "email"]

    def validate_unique(self):
        pass        # never tell a stranger whether a phone number is already
in our CRM


@method_decorator(login_not_required, name="dispatch")
class PublicLeadView(FormView):
    template_name = "public/lead_form.html"
    form_class = PublicLeadForm

    def post(self, request, *args, **kwargs):
        ip = request.META.get("HTTP_X_REAL_IP") or
request.META.get("REMOTE_ADDR")
        key = f"publicform:{ip}"
        cache.add(key, 0, 60 * 60)
        if cache.incr(key) > 5:                                          # 5
submissions per IP per hour
            return HttpResponse("Too many submissions — please try again
later.", status=429)
        return super().post(request, *args, **kwargs)

    def form_valid(self, form):
        if form.cleaned_data.get("website"):
            return redirect("public_thanks")                    # a bot
filled the trap: pretend it worked
        if Lead.objects.filter(phone=form.cleaned_data["phone"]).exists():
            return redirect("public_thanks")                    # existing
```

```
lead stays untouched; same answer
        code = self.request.get_signed_cookie(REF_COOKIE, default=None,
salt="crm.referral")
        lead = form.save(commit=False)
        lead.broker = Broker.objects.filter(code=code,
is_active=True).first() if code else None
        lead.source = Lead.Source.REFERRAL if lead.broker else
Lead.Source.WEBSITE
        lead.assigned_to = next_round_robin_agent()
        lead.save()                                              # signals:
history, audit, assignment email
        return redirect("public_thanks")


@method_decorator(login_not_required, name="dispatch")
class PublicThanksView(TemplateView):
    template_name = "public/thanks.html"


# django_project/urls.py
path("r/<str:code>/", public.referral, name="referral"),
path("join/", public.PublicLeadView.as_view(), name="public_lead_form"),
path("join/thanks/", public.PublicThanksView.as_view(),
name="public_thanks"),
```

```
<!-- templates/public/lead_form.html — standalone page, no CRM navigation -->
<form method="post" novalidate>
  {% csrf_token %}
  {{ form.name.as_field_group }}
  {{ form.phone.as_field_group }}
  {{ form.email.as_field_group }}
  <div class="hp" aria-hidden="true">{{ form.website }}</div>   {# .hp {
position:absolute; left:-9999px } #}
  <button type="submit">Request a call back</button>
</form>
```

28.

```
# brokers/exports.py
from io import BytesIO
from openpyxl import Workbook
from openpyxl.styles import Font
from django.utils import timezone
```

```
def statement_xlsx(statement):
    wb = Workbook()
    ws = wb.active
    ws.title = "Commission"
    ws.append(["Broker", statement.broker.code, statement.broker.name])
    ws.append(["Period", statement.period.strftime("%B %Y"),
statement.get_status_display()])
    ws.append([])
    ws.append(["Deposit", "Date", "Account", "From broker", "Deposit (AED)",
"Rate %", "Commission (AED)"])
    for cell in ws[4]:
        cell.font = Font(bold=True)
    for line in statement.lines.select_related("deposit__account",
"source_broker").order_by("deposit__created_at"):
        ws.append([line.deposit_id,
timezone.localtime(line.deposit.created_at).date(),
line.deposit.account.number,
                   line.source_broker.code, line.deposit.amount, line.rate,
line.amount])
    ws.append([])
    ws.append([None, None, None, None, None, "Total", statement.total])
    buffer = BytesIO()
    wb.save(buffer)
    return buffer.getvalue()
```

29.

```
# accounts/models.py — CustomUser: Role gains BROKER = "broker", "Broker";
plus
broker = models.ForeignKey("brokers.Broker", null=True, blank=True,
on_delete=models.SET_NULL, related_name="users")


# api/views.py
from brokers.reports import downline_report
from brokers.tree import load_tree, walk
from .authentication import ApiKeyAuthentication


@api_view(["GET"])
@authentication_classes([ApiKeyAuthentication])
```

```
def broker_stats(request):
    user = request.user
    if getattr(user, "role", "") != "broker" or user.broker_id is None:
        raise PermissionDenied("Broker API keys only.")
    _, children = load_tree()
    allowed = {user.broker.code, *(b.code for b, _ in walk(children,
user.broker_id))}   # itself + downline
    params = DaysParams(data=request.query_params)
    params.is_valid(raise_exception=True)
    since = timezone.now() - timedelta(days=params.validated_data["days"])
    return Response([row for row in downline_report(since) if row["code"] in
allowed])

# api/urls.py: path("brokers/me/stats/", views.broker_stats, name="broker-
stats")
# key: python manage.py create_api_key "BRF1002 portal" --user brf1002-bot
(then set role=broker, broker=BRF1002)
```

30.

```
# urls — each module behind its flag (Day 22 #29)
path("sop/breaches/", feature_required("sop")
(sop_views.BreachBoardView.as_view()), name="sop_breaches"),
path("team/me/dialer/", feature_required("dialer")
(lead_views.DialerView.as_view()), name="dialer"),
path("brokers/statements/", feature_required("commissions")
(broker_views.StatementListView.as_view()), name="statements"),
# .env: FEATURES=imports,exports,sop,dialer,commissions
```

D. Challenge (31–35)

```
# leads/models.py — 31: indexes shaped like the scan and the queue
class Meta:
    indexes = [
        ...,
        models.Index(fields=["assigned_to", "status", "follow_up_at"],
name="lead_queue_idx",
                     condition=models.Q(is_active=True, dnc=False)),
        models.Index(fields=["created_at"], name="lead_first_call_scan_idx",
                     condition=models.Q(status="new", contact_attempts=0,
is_active=True)),
    ]
# check: print(dial_queue(agent).explain(analyze=True)) → "Index Scan using
lead_queue_idx"
```

32. `seed_capstone` builds on Day 23's `seed_crm`: three broker trees with rates 2.00 / 1.50 / 1.00, leads backdated across the last two weeks (set `created_at` with `update()` after `bulk_create`, since `auto_now_add` ignores values you pass), a few callbacks due now, deposits spread over last month, then `scan()` and `generate(last_month)` so every page has something to show.

```
# brokers/admin.py — 33
@admin.register(CommissionStatement)
class CommissionStatementAdmin(admin.ModelAdmin):
    list_display = ["broker", "period", "status", "total", "approved_by",
"approved_at"]
    list_filter = ["status", "period"]
    actions = ["approve"]

    def get_readonly_fields(self, request, obj=None):
        if obj and obj.status == CommissionStatement.Status.APPROVED:
            return [f.name for f in obj._meta.fields]           # frozen
        return ["total", "approved_by", "approved_at"]

    @admin.action(description="Approve selected statements")
    def approve(self, request, queryset):
        if request.user.role not in {"manager", "admin"} and not
request.user.is_superuser:
            self.message_user(request, "Only managers can approve
statements.", level=messages.ERROR)
            return
```

```
        n = queryset.filter(status=CommissionStatement.Status.DRAFT).update(
            status=CommissionStatement.Status.APPROVED,
approved_by=request.user, approved_at=timezone.now())
        self.message_user(request, f"Approved {n} statement(s).")
```

```
# settings/base.py — 35 (Celery beat; add a `beat` service: command `celery -
A django_project beat -l info`)
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    "scan-sop": {"task": "sop.tasks.scan_sop_task", "schedule":
crontab(minute="*/15")},
    "commissions": {"task": "brokers.tasks.generate_commissions_task",
"schedule": crontab(minute=0, hour=2, day_of_month=1)},
}

# sop/tasks.py
@shared_task
def scan_sop_task():
    call_command("scan_sop")          # the command keeps the advisory lock
and notifications in one place
```

Run beat in exactly one container: two beats mean every job twice (the advisory lock would catch it, but don't rely on that).

Tests (36–40)

```
# sop/tests.py
from contextlib import contextmanager
from datetime import datetime, timedelta
from decimal import Decimal
from io import StringIO
from unittest import mock
from zoneinfo import ZoneInfo

import hashlib
import hmac
import json

from django.core.cache import cache
from django.core.management import call_command
from django.test import override_settings
```

```
from django.urls import reverse
from django.utils import timezone
from rest_framework.test import APITestCase

from brokers.commissions import commission_lines_for
from brokers.models import CommissionStatement
from clients.services import convert_lead_to_client
from core.factories import (AgentFactory, BrokerFactory, DepositFactory,
LeadFactory, TradingAccountFactory)
from core.testing import api_client_for
from leads.dialer import dial_queue
from leads.models import DNCNumber, Lead
from leads.services import log_call
from sop.engine import scan
from sop.hours import add_working_hours
from sop.models import SOPBreach, SOPRule

DUBAI = ZoneInfo("Asia/Dubai")
MON_10 = datetime(2026, 10, 5, 10, 0, tzinfo=DUBAI)
WED_11 = datetime(2026, 10, 7, 11, 0, tzinfo=DUBAI)       # first-call
deadline for MON_10 is Wed 10:00


@contextmanager
def at(moment):
    with mock.patch("django.utils.timezone.now", return_value=moment):
        yield


class CapstoneTests(APITestCase):
    def setUp(self):
        cache.clear()

    @override_settings(PAYMENT_WEBHOOK_SECRET="whsec-test")
    def test_end_to_end(self):
# 36
        master = BrokerFactory(code="BRF1001",
commission_rate=Decimal("2.00"), parent=None)
        ib = BrokerFactory(code="BRF1002", commission_rate=Decimal("1.50"),
parent=master)
        agent = AgentFactory(email="agent@nexora.test")

        with at(MON_10):
```

```
# web form via a referral link
            self.client.get(reverse("referral", args=["BRF1002"]))
            self.client.post(reverse("public_lead_form"),
                             {"name": "web lead", "phone": "0509990001",
"email": "web@x.test", "website": ""})
        lead = Lead.objects.get(phone="971509990001")
        self.assertEqual((lead.broker, lead.assigned_to, lead.source), (ib,
agent, Lead.Source.REFERRAL))

        with at(WED_11):
# SLA missed → breach
            call_command("scan_sop", stdout=StringIO())
        self.assertEqual(SOPBreach.objects.filter(lead=lead,
resolved_at__isnull=True).count(), 1)

        api = api_client_for(agent)
        with at(WED_11 + timedelta(minutes=5)):
# the call fixes it
            self.assertEqual(api.post(reverse("api:lead-calls", args=
[lead.pk]),
                                      {"outcome": "answered",
"duration_seconds": 120}, format="json").status_code, 201)
            call_command("scan_sop", stdout=StringIO())
            self.assertEqual(api.post(reverse("api:lead-set-status", args=
[lead.pk]),
                                      {"status": "qualified"},
format="json").status_code, 200)
        self.assertFalse(SOPBreach.objects.filter(lead=lead,
resolved_at__isnull=True).exists())

        with at(WED_11 + timedelta(hours=1)):
# convert + signed deposit webhook
            account = convert_lead_to_client(lead.pk).accounts.get()
            body = json.dumps({"event_id": "evt_e2e", "account_number":
account.number, "amount": "10000.00",
                               "method": "card", "created_at":
timezone.now().isoformat()}).encode()
            signature = "sha256=" + hmac.new(b"whsec-test", body,
hashlib.sha256).hexdigest()
            self.assertEqual(self.client.post(reverse("api:deposit-webhook"),
body, content_type="application/json",

HTTP_X_PAY_SIGNATURE=signature).status_code, 201)
```

```
        call_command("generate_commissions", month="2026-10",
stdout=StringIO())
        totals = dict(CommissionStatement.objects.values_list("broker__code",
"total"))
        self.assertEqual(totals, {"BRF1002": Decimal("150.00"), "BRF1001":
Decimal("50.00")})

    def test_working_hours(self):
# 37
        fri_20 = datetime(2026, 10, 16, 20, 0, tzinfo=DUBAI)
        sat_20 = datetime(2026, 10, 17, 20, 0, tzinfo=DUBAI)
        sun_08 = datetime(2026, 10, 18, 8, 0, tzinfo=DUBAI)
        self.assertEqual(add_working_hours(fri_20, 3), datetime(2026, 10, 17,
11, 0, tzinfo=DUBAI))   # 1 h Fri + 2 h Sat
        self.assertEqual(add_working_hours(sat_20, 3), datetime(2026, 10, 19,
11, 0, tzinfo=DUBAI))   # Sunday skipped
        self.assertEqual(add_working_hours(sun_08, 1), datetime(2026, 10, 19,
10, 0, tzinfo=DUBAI))

    def test_queue_order_and_dnc(self):
# 38
        agent = AgentFactory()
        now = timezone.now()
        contacted = {"status": "contacted", "contact_attempts": 1}
        plain = LeadFactory(assigned_to=agent, last_contacted_at=now -
timedelta(days=1), **contacted)
        fresh = LeadFactory(assigned_to=agent)
        callback = LeadFactory(assigned_to=agent, last_contacted_at=now -
timedelta(minutes=30),
                               follow_up_at=now - timedelta(minutes=1),
**contacted)
        breached = LeadFactory(assigned_to=agent, last_contacted_at=now -
timedelta(days=5), **contacted)
        SOPBreach.objects.create(rule=SOPRule.objects.get(code="follow-up"),
lead=breached, agent=agent, due_at=now)
        LeadFactory(assigned_to=agent, last_contacted_at=now -
timedelta(days=1),
                    follow_up_at=now + timedelta(hours=3), **contacted)
# callback later → not yet
        DNCNumber.objects.create(phone="0508887777")
        LeadFactory(assigned_to=agent, phone="971508887777")
# flagged dnc by pre_save
```

```
        ids = list(dial_queue(agent).values_list("pk", flat=True))
        self.assertEqual(ids, [breached.pk, callback.pk, fresh.pk, plain.pk])

    def test_commission_waterfall(self):
# 39
        master = BrokerFactory(code="BRF1001",
commission_rate=Decimal("2.00"), parent=None)
        ib = BrokerFactory(code="BRF1002", commission_rate=Decimal("1.50"),
parent=master)
        deposit =
DepositFactory(account=TradingAccountFactory(client__broker=ib),
amount=Decimal("10000.00"))
        lines = {b.code: amount for b, _, amount in
commission_lines_for(deposit, {master.pk: master, ib.pk: ib})}
        self.assertEqual(lines, {"BRF1002": Decimal("150.00"), "BRF1001":
Decimal("50.00")})

    def test_scan_is_idempotent_and_resolves(self):
# 40
        agent = AgentFactory()
        with at(MON_10):
            lead = LeadFactory(assigned_to=agent)
        with at(WED_11):
            scan()
            scan()
            self.assertEqual(SOPBreach.objects.filter(lead=lead,
resolved_at__isnull=True).count(), 1)
            log_call(lead, agent, "answered", 60)
            scan()
        self.assertFalse(SOPBreach.objects.filter(lead=lead,
resolved_at__isnull=True).exists())
        self.assertEqual(SOPBreach.objects.filter(lead=lead).count(), 1)
# history kept, now resolved
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py test sop --settings=django_project.settings.test
git add -A && git commit -m "day 29: capstone"
```

Day 30 — Final deploy and handover

Goal: ship it. Close the loose ends from earlier days, pass a launch checklist, put the CRM on a real server, prove it's alive, and leave it in a state someone else could run: monitoring, rollback, runbook, an analytics layer for BI tools, and a README that explains the whole thing.

A. Launch checklist (1–10)

- [ ] **1.** Full test suite with coverage; fail the build under 80%.
- [ ] **2.** `ruff` lint + format; pre-commit hooks so it runs before every commit.
- [ ] **3.** Migrations: `makemigrations --check`, `migrate --plan` on a copy of production, optional `squashmigrations`.
- [ ] **4.** Security gates green: `bandit`, `pip-audit`, `check --deploy` (Day 28).
- [ ] **5.** Headless Locust run against staging: 50 agents, p95 under 500 ms.
- [ ] **6.** Environment inventory: every env var, where it's used, who owns the secret.
- [ ] **7.** Loose ends: remove Day 14's double balance update and `AuditMixin`; finish `ProfileForm` (`timezone`, `avatar`, `email_digest`); `raw` guard in `put_user_in_role_group`.
- [ ] **8.** Production data plan: empty DB → migrate (roles, SOP rules) → import real leads (Day 19) → import DNC list.
- [ ] **9.** First accounts: superuser, managers and agents via `create_agent`, API keys for the dialer, payment webhook secret.
- [ ] **10.** `README.md` and `RUNBOOK.md` skeletons in the repo.

B. Go-live (11–20)

- [ ] **11.** Server hardening: Ubuntu 24.04, deploy user, SSH keys only, `ufw` (22/80/443), unattended upgrades, fail2ban.
- [ ] **12.** Docker Engine + compose plugin; secrets files with `600` permissions.
- [ ] **13.** DNS + first TLS certificate (Day 28 #18).
- [ ] **14.** First boot order: db/redis → migrate → superuser → web/worker/beat → nginx.
- [ ] **15.** Host crontab for everything scheduled.
- [ ] **16.** `smoke_test` command (inside the container) + an external curl smoke script.

- [ ] **17.** Monitoring: external uptime check, Sentry alerts, disk-space check.
- [ ] **18.** Docker log rotation.
- [ ] **19.** Rollback procedure: previous image tag, no down-migrations, restore as last resort.
- [ ] **20.** Near-zero-downtime deploys: two web replicas, nginx retries, rolling replace.

C. Operations and analytics (21–30)

- [ ] **21.** UAT checklist per role (agent, team lead, manager, broker) signed off before launch.
- [ ] **22.** PostgreSQL health queries: unused indexes, dead tuples / autovacuum, biggest tables, slowest queries.
- [ ] **23.** Scaling knobs and when to turn each one.
- [ ] **24.** Incident runbook: site down, disk full, 500 spike, dialer webhook failing, emails failing.
- [ ] **25.** Error alerts to a Slack/Teams channel via a logging handler.
- [ ] **26.** `analytics` schema: SQL views `dim_agent`, `dim_broker`, `fct_leads`, `fct_calls`, `fct_deposits` (no raw PII).
- [ ] **27.** Read-only `bi_reader` role that can only `SELECT` from `analytics`.
- [ ] **28.** Nightly CSV export of the analytics views with `COPY … TO STDOUT`.
- [ ] **29.** Data-quality checks command: uniqueness, nulls, orphans, freshness.
- [ ] **30.** Maintenance-mode drill.

D. Wrap-up (31–40)

- [ ] **31.** Sunset `/api/v1/`: `Deprecation`, `Sunset` and `Link` headers + usage logging.
- [ ] **32.** Architecture overview in the README.
- [ ] **33.** Portfolio-ready README: features, stack, run it locally in one command, tests.
- [ ] **34.** Demo mode: seeded data, a read-only demo user, no outgoing emails/webhooks.
- [ ] **35.** Retrospective: what you'd change, what to learn next.
- [ ] **36.** `v1.0.0`: changelog, git tag, GitHub release.
- [ ] **37.** Test: role × page permission matrix.
- [ ] **38.** Test (PostgreSQL): analytics views exist and expose no raw phone numbers.
- [ ] **39.** Test: `/api/v1/` responses carry the deprecation headers.
- [ ] **40.** Test: `smoke_test` passes.

Solutions

A. Launch checklist (1–10)

1.

```
# .coveragerc
[run]
source = .
branch = True
omit =
    */migrations/*
    */tests/*
    */tests.py
    manage.py
    django_project/asgi.py
    django_project/wsgi.py
    locustfile.py
    .venv/*

[report]
fail_under = 80
show_missing = True
skip_covered = True
```

```
coverage run manage.py test --settings=django_project.settings.test
coverage report          # exits 2 below 80% → CI fails
coverage html            # htmlcov/index.html: the red lines are your next
tests
```

2.

```
# pyproject.toml
[tool.ruff]
line-length = 120
target-version = "py310"
extend-exclude = ["*/migrations/*"]

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "DJ", "S"]     # pycodestyle, pyflakes,
isort, bugbear, pyupgrade, django, bandit
ignore = ["S101"]

[tool.ruff.lint.per-file-ignores]
"*/tests*.py" = ["S"]
"django_project/settings/*.py" = ["F403", "F405"]     # star imports from
base are intentional
```

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.6.9
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: check-merge-conflict
      - id: detect-private-key
      - id: check-added-large-files
```

```
python -m pip install pre-commit ruff
pre-commit install && pre-commit autoupdate      # pin the newest hook
versions
pre-commit run --all-files                       # first run reformats
everything — commit that separately
```

3.

```
python manage.py makemigrations --check --dry-run      # non-zero exit if a
model change has no migration
# on a restored copy of production (Day 28 restore drill):
python manage.py migrate --plan                        # exactly what will
run, in order
# optional, only once every environment has applied them all:
python manage.py squashmigrations leads 0001 0030
```

4. Day 28 #10 and #36 in CI; locally: `bandit -r . -x ./.venv -ll && pip-audit -r requirements.txt && python manage.py check --deploy --settings=django_project.settings.prod`.

5.

```
locust -f locustfile.py --headless -u 50 -r 5 -t 5m -H
https://staging.crm.nexora.ae --csv=load --only-summary
# load_stats.csv: the "95%" column per endpoint must be < 500 ms, "Failure
Count" 0.
# The slowest endpoint goes back through Day 24: toolbar → EXPLAIN → index or
cache.
```

6. Keep this table in `RUNBOOK.md`; anything missing from it is a secret nobody knows how to rotate.

Variable	Used by	Secret?	Owner / where it lives
`SECRET_KEY`	sessions, signing, password-reset links	yes	ops vault → `.env.prod`
`DATABASE_URL`, `.env.db`	Django, Postgres container	yes	ops vault
`REDIS_URL`, `CELERY_BROKER_URL`	cache, sessions, Celery	no	compose
`CRM_API_KEY`	`/api/v1/` (until sunset)	yes	ops vault
`PAYMENT_WEBHOOK_SECRET`	deposit webhook HMAC	yes	payment provider dashboard + vault

Variable	Used by	Secret?	Owner / where it lives
`EMAIL_HOST_USER` / `PASSWORD`	SMTP	yes	email provider
`SENTRY_DSN`	error tracking	low	Sentry project
`ADMIN_URL`, `ADMIN_ALLOWED_IPS`	admin hardening	semi	ops
`ALLOWED_HOSTS`, `CRM_BASE_URL`, `FEATURES`, `MAINTENANCE_MODE`, `LOG_LEVEL`	behaviour	no	`.env.prod`

7. The loose ends collected over the month:

```
# clients/views.py — DepositCreateView.form_valid: DELETE this line (Day 20's
signal now does it)
# TradingAccount.objects.filter(pk=account.pk).update(balance=F("balance") +
form.instance.amount)

# leads/views.py — Day 14's AuditMixin is replaced by Day 20's post_save
audit; remove it from the bases
class LeadCreateView(LoginRequiredMixin, PermissionRequiredMixin,
CreateView): ...
class LeadUpdateView(LoginRequiredMixin, PermissionRequiredMixin,
UpdateView): ...
# then delete class AuditMixin

# accounts/forms.py — ProfileForm
class ProfileForm(forms.ModelForm):
    class Meta:
        model = CustomUser
        fields = ["first_name", "last_name", "email", "phone_ext",
"timezone", "avatar", "email_digest"]
        widgets = {"timezone": forms.Select(choices=[(z, z) for z in
("Asia/Dubai", "Asia/Kolkata", "Europe/London", "UTC")])}

# accounts/views.py — ProfileView must pass files: ProfileForm(request.POST,
request.FILES, instance=request.user)
# accounts/profile.html: <form method="post" enctype="multipart/form-data">
(avatar needs Pillow — already in requirements.in)

# accounts/signals.py — put_user_in_role_group(sender, instance, raw=False,
**kwargs): if raw: return
```

```
# once, after removing the double update — rebuild balances from deposits
(Day 20 snippet)
totals =
Deposit.objects.filter(account=OuterRef("pk")).order_by().values("account").a
nnotate(t=Sum("amount")).values("t")
TradingAccount.objects.update(balance=Coalesce(Subquery(totals),
Value(Decimal("0"))))
```

8–9.

```
alias dc="docker compose -f docker-compose.prod.yml"

dc run --rm web python manage.py migrate        # roles/groups (Day 16
post_migrate) + SOP rules (Day 29) come with it
dc run --rm web python manage.py createsuperuser
dc run --rm web python manage.py create_agent sara.k sara.k@nexora.ae --role
manager --first-name Sara
dc run --rm web python manage.py create_agent omar.h omar.h@nexora.ae --team-
lead sara.k --first-name Omar
dc run --rm web python manage.py create_api_key "Dialer" --user dialer-bot
openssl rand -hex 32                            # PAYMENT_WEBHOOK_SECRET:
into .env.prod and the provider's dashboard

dc cp ./dnc.csv web:/tmp/dnc.csv && dc exec web python manage.py import_dnc
/tmp/dnc.csv
# real leads: Day 19 import page with "dry run" first; fix the error rows;
import; spot-check 20 leads by hand
```

10.

```
README.md                          RUNBOOK.md
  What it is (3 lines)               Environments and URLs
  Features                           Environment variables (item 6)
  Architecture (32)                  Deploy / rollback (19)
  Run locally (33)                   Scheduled jobs (15)
  Tests and quality gates            Backups and restore drill
  Project layout                     Monitoring and alerts (17, 25)
  License / contact                  Incidents (24)
                                     Contacts and escalation
```

B. Go-live (11–20)

11–12.

```
# as root on a fresh Ubuntu 24.04 VPS (pick a UAE region if the provider has
one)
adduser --disabled-password --gecos "" deploy && usermod -aG sudo deploy
mkdir -p /home/deploy/.ssh && cp ~/.ssh/authorized_keys /home/deploy/.ssh/ &&
chown -R deploy:deploy /home/deploy/.ssh
printf "PasswordAuthentication no\nPermitRootLogin no\n" >
/etc/ssh/sshd_config.d/hardening.conf && systemctl reload ssh
ufw default deny incoming && ufw allow OpenSSH && ufw allow 80/tcp && ufw
allow 443/tcp && ufw enable
apt update && apt install -y unattended-upgrades fail2ban && dpkg-reconfigure
-plow unattended-upgrades

# 12: Docker Engine + compose plugin
curl -fsSL https://get.docker.com | sh
usermod -aG docker deploy
mkdir -p /srv/nexora-crm && chown deploy:deploy /srv/nexora-crm
# as deploy:
cd /srv/nexora-crm && git clone <repo> . && cp .env.example .env.prod &&
chmod 600 .env.prod .env.db
```

Docker writes its own iptables rules and bypasses `ufw` for published ports. That's why only nginx publishes ports in `docker-compose.prod.yml`; never add `ports:` to `db` or `redis`.

13. DNS `A` record `crm.nexora.ae → <server IP>` (low TTL for launch week), then the certificate exactly as in Day 28 #18.

14.

```
cd /srv/nexora-crm
export TAG=<git sha of the release>
dc pull
dc up -d db redis
dc run --rm web python manage.py migrate
dc run --rm web python manage.py createsuperuser
dc up -d web worker beat nginx
dc ps                                    # every service "running", web
"healthy"
echo "TAG=$TAG" > .deployed
```

15.

```
#!/usr/bin/env bash
# deploy/manage.sh — run a management command in the live web container
cd /srv/nexora-crm && exec docker compose -f docker-compose.prod.yml exec -T
web python manage.py "$@"
```

```
# crontab -e  (as deploy; server clock is UTC, Dubai = UTC+4)
30 1 * * *    /srv/nexora-crm/deploy/backup.sh                              >>
/var/log/nexora/backup.log 2>&1
0 2 * * *     /srv/nexora-crm/deploy/manage.sh export_analytics /exports  >>
/var/log/nexora/cron.log 2>&1
15 2 * * *    /srv/nexora-crm/deploy/manage.sh data_quality                >>
/var/log/nexora/cron.log 2>&1
0 3 * * *     <certbot renew + nginx reload, Day 28 #18>
0 4 * * *     /srv/nexora-crm/deploy/manage.sh daily_report --email       >>
/var/log/nexora/cron.log 2>&1   # 08:00 Dubai
30 4 * * 1-6  /srv/nexora-crm/deploy/manage.sh send_agent_digests       >>
/var/log/nexora/cron.log 2>&1   # Mon–Sat
*/15 * * * *  /srv/nexora-crm/deploy/manage.sh refresh_stats            >>
/var/log/nexora/cron.log 2>&1
0 5 * * *     /srv/nexora-crm/deploy/manage.sh resend_failed_emails     >>
/var/log/nexora/cron.log 2>&1
0 22 * * 5    /srv/nexora-crm/deploy/manage.sh purge_old_data           >>
/var/log/nexora/cron.log 2>&1   # Sat 02:00 Dubai
0 * * * *     /srv/nexora-crm/deploy/check_disk.sh
# scan_sop and generate_commissions run on Celery beat (Day 29 #35) —
schedule each job in ONE place only
```

16.

```
# core/management/commands/smoke_test.py
from django.conf import settings
from django.contrib.auth import get_user_model
from django.core.management.base import BaseCommand, CommandError
from django.test import Client
from django.urls import reverse


class Command(BaseCommand):
    help = "Request key pages and APIs in-process; exit 1 if any answer is
wrong"

    def add_arguments(self, parser):
```

```
        parser.add_argument("--username", help="a manager account for the
logged-in checks")

    def run(self, client, checks):
        failures = 0
        for label, url, expected in checks:
            status = client.get(url, secure=True).status_code       # secure:
no HTTPS redirect in prod
            ok = status == expected
            failures += not ok
            self.stdout.write(f"{'OK  ' if ok else 'FAIL'} {status} {label:
<20} {url}")
        return failures

    def handle(self, *args, username, **options):
        client = Client(HTTP_HOST=settings.ALLOWED_HOSTS[0] if
settings.ALLOWED_HOSTS else "localhost")
        failures = self.run(client, [                                 #
anonymous first
            ("health", reverse("health"), 200),
            ("login page", reverse("login"), 200),
            ("api without auth", reverse("api:lead-list"), 401),
        ])
        if username:

client.force_login(get_user_model().objects.get(username=username))
            session = client.session
            session["otp_verified"] = True                            # Day
28 #31: a managed smoke session
            session.save()
            failures += self.run(client, [
                ("dashboard", reverse("dashboard"), 200),
                ("lead list", reverse("leads:list"), 200),
                ("api leads", reverse("api:lead-list"), 200),
                ("api dashboard", reverse("api:dashboard"), 200),
                ("api chart", reverse("api:chart", args=["daily-leads"]),
200),
            ])
        if failures:
            raise CommandError(f"{failures} smoke check(s) failed")
        self.stdout.write(self.style.SUCCESS("All smoke checks passed."))
```

```
#!/usr/bin/env bash
# deploy/smoke.sh — from outside: DNS, TLS, nginx and Django together
set -euo pipefail
BASE=${1:-https://crm.nexora.ae}
check() {
  code=$(curl -s -o /dev/null -w "%{http_code}" "$BASE$1")
  if [[ "$code" == "$2" ]]; then echo "OK   $code $1"; else echo "FAIL $code
$1 (expected $2)"; exit 1; fi
}
check /health/ 200
check /accounts/login/ 200
check /api/v2/leads/ 401
check /admin/ 404                        # admin lives at ADMIN_URL behind
the allowlist
curl -sI "$BASE/accounts/login/" | grep -qi "strict-transport-security" &&
echo "OK   HSTS header"
```

Run both after every deploy: `deploy/manage.sh smoke_test --username smoke-manager && deploy/smoke.sh`.

17.

```
#!/usr/bin/env bash
# deploy/check_disk.sh — hourly; email admins at 80%
USED=$(df --output=pcent / | tail -1 | tr -dc '0-9')
if [ "$USED" -ge 80 ]; then
  /srv/nexora-crm/deploy/manage.sh shell -c \
    "from django.core.mail import mail_admins; mail_admins('Disk ${USED}%
full', 'Prune old backups/images or grow the volume.')"
fi
```

Plus: an external uptime monitor polling `https://crm.nexora.ae/health/` every minute (it runs from outside, so it catches DNS, TLS and nginx failures the server can't see), and Sentry alert rules for new issues and error spikes.

18. Put this in `/etc/docker/daemon.json`, then `systemctl restart docker` (applies to containers created afterwards):

```
{ "log-driver": "json-file", "log-opts": { "max-size": "20m", "max-file": "5"
} }
```

19.

```
cat .deployed                                        # TAG=<current sha>
export TAG=<previous sha>                            # from git log / the GHCR
package page
dc pull web worker beat && dc up -d web worker beat
deploy/smoke.sh && echo "TAG=$TAG" > .deployed
```

Rolling back code never rolls back the database. That's safe only because of Day 28 #19: every migration is additive (expand first, contract a release later), so the previous release still runs on the new schema. Restoring a backup is the last resort, because it loses every lead, call and deposit since the dump.

20.

```
# deploy/nginx.conf — resolve "web" at request time so new replicas are
picked up
resolver 127.0.0.11 valid=10s;                 # Docker's internal DNS
location / {
    set $crm http://web:8000;
    proxy_pass $crm;
    proxy_next_upstream error timeout http_502 http_503;   # retry the other
replica
    proxy_next_upstream_tries 2;
    # ...headers as before
}
```

```
# two replicas, replaced one at a time (no fixed container_name and no ports
on web)
dc up -d --scale web=2 --no-recreate
# the docker-rollout CLI plugin automates "start new, wait healthy, stop
old":
docker rollout -f docker-compose.prod.yml web
```

Requests in flight finish thanks to gunicorn's `graceful_timeout`; a request that hits a container that's stopping is retried on the other one by `proxy_next_upstream`.

C. Operations and analytics (21–30)

21. UAT: each role runs its script on staging with real-looking data; launch waits for four signatures.

Role	Must work	Must NOT work
Agent	inbox, dialer (next lead, outcomes, callback), log call, status changes, own dashboard, digest email	other agents' leads (404), expected deposit field, reports, exports, KYC downloads
Team lead	team's leads, breach board, team wall, leaderboard, reassign within team	other teams' leads, commission approval, admin
Manager	everything in the CRM, exports, KYC review, commission approval, 2FA on login	admin without an allowlisted IP
Broker (API)	`/api/v2/brokers/me/stats/` with own key	any other broker's numbers, any lead or client endpoint

22.

```
-- indexes never used since the last stats reset (keep the ones backing
UNIQUE constraints)
SELECT relname AS table_name, indexrelname AS index_name, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes WHERE idx_scan = 0 ORDER BY
pg_relation_size(indexrelid) DESC;

-- dead rows waiting for autovacuum
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 10;

-- biggest tables (data + indexes + TOAST)
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) AS total
FROM pg_statio_user_tables ORDER BY pg_total_relation_size(relid) DESC LIMIT
10;

-- slowest queries: Day 22 #33 (pg_stat_statements), reviewed weekly
```

23. Turn the knobs in this order, measuring after each (Day 24):

Symptom	Knob
CPU pegged on web, DB idle	`WEB_CONCURRENCY` up (gunicorn workers), then a bigger server or a second web host

Symptom	Knob
Many short DB connections, "too many clients"	Django 5.1 connection pool (Day 22 #14) or PgBouncer
Slow pages, DB busy	Day 24 profiling → index / cache / materialized view before hardware
Reports slow down agents' pages	read replica for reports (Day 22 #30)
Emails / webhooks backing up	Celery `--concurrency` up, or a second worker
Redis memory growing	`maxmemory` + `allkeys-lru` for the cache DB (sessions and Celery in separate DBs)

24.

```
SITE DOWN
  1. deploy/smoke.sh → which layer? (DNS / TLS / nginx / web / DB)
  2. dc ps ; dc logs --tail=200 web nginx
  3. web unhealthy right after a deploy → roll back (19) first, investigate
second

DISK FULL
  1. df -h ; docker system df
  2. prune: find /backups -mtime +14 -delete ; docker image prune -a --filter
"until=168h"
  3. Postgres won't start → free space BEFORE restarting it; never delete
files inside pgdata

500 SPIKE
  1. Sentry: newest issue + request ids
  2. dc logs web | grep <request_id>
  3. caused by the last deploy → roll back

DIALER WEBHOOK FAILING
  1. "crm.api" logs: 4xx on /api/v2/dialer/calls/ → auth (key revoked?) or
validation (payload changed?)
  2. idempotency keys make it safe for the dialer to re-send once fixed
```

```
EMAILS FAILING
  1. EmailLog rows with status=failed: read the error column
  2. fix SMTP credentials in .env.prod → dc up -d web worker
  3. deploy/manage.sh resend_failed_emails
```

25.

```
# core/logging.py (add)
import json
import urllib.request


class ChatWebhookHandler(logging.Handler):
    """Posts ERROR records to a Slack-style incoming webhook (payload
{"text": ...})."""

    def emit(self, record):
        from django.conf import settings           # lazy: logging is
configured before apps load
        url = getattr(settings, "CHAT_WEBHOOK_URL", "")
        if not url:
            return
        text = (f"{record.levelname} {record.name} [{getattr(record,
'request_id', '-')}] "
                f"{record.getMessage()[:500]}")
        try:
            request = urllib.request.Request(url, data=json.dumps({"text":
text}).encode(), method="POST",
                                             headers={"Content-Type":
"application/json"})
            urllib.request.urlopen(request, timeout=3).close()
        except Exception:
            self.handleError(record)               # alerting must never
crash the request

# settings/prod.py
CHAT_WEBHOOK_URL = env.str("CHAT_WEBHOOK_URL", default="")
LOGGING["handlers"]["chat"] = {"class": "core.logging.ChatWebhookHandler",
"level": "ERROR",
                               "filters": ["request_id", "redact_pii"]}
LOGGING["loggers"]["django.request"]["handlers"].append("chat")
LOGGING["loggers"]["crm"]["handlers"].append("chat")
```

Microsoft Teams' newer workflow webhooks expect an Adaptive Card body instead of `{"text": ...}`; only the payload line changes.

26. A stable, PII-light contract for BI tools. Dashboards query these views, never the app tables, so app refactors don't break reports.

```
# reports/migrations/00NN_analytics_schema.py
from django.db import migrations

VIEWS = """
CREATE SCHEMA IF NOT EXISTS analytics;

CREATE OR REPLACE VIEW analytics.dim_agent AS
SELECT u.id AS agent_id, u.username, u.first_name, u.last_name, u.role,
u.team_lead_id, u.is_active, u.date_joined
FROM "{user}" u WHERE u.role IN ('agent', 'team_lead', 'manager');

CREATE OR REPLACE VIEW analytics.dim_broker AS
SELECT id AS broker_id, code, name, tier, parent_id, commission_rate,
is_active FROM "{broker}";

CREATE OR REPLACE VIEW analytics.fct_leads AS
SELECT l.id AS lead_id, l.created_at, (l.created_at AT TIME ZONE
'Asia/Dubai')::date AS created_date,
       l.status, l.source, l.assigned_to_id AS agent_id, l.broker_id,
l.expected_deposit,
       l.contact_attempts, l.last_contacted_at, l.is_active, l.dnc,
       right(l.phone, 4) AS phone_last4                                  --
never the full number
FROM "{lead}" l;

CREATE OR REPLACE VIEW analytics.fct_calls AS
SELECT c.id AS call_id, c.lead_id, c.agent_id, c.started_at,
       (c.started_at AT TIME ZONE 'Asia/Dubai')::date AS call_date,
c.outcome, c.duration_seconds
FROM "{call}" c;

CREATE OR REPLACE VIEW analytics.fct_deposits AS
SELECT d.id AS deposit_id, d.created_at, (d.created_at AT TIME ZONE
'Asia/Dubai')::date AS deposit_date,
       d.amount, d.method, a.currency, a.client_id, c.broker_id, c.lead_id
FROM "{deposit}" d
JOIN "{account}" a ON a.id = d.account_id
```

```
JOIN "{client}" c ON c.id = a.client_id;
"""


def create(apps, schema_editor):
    tables = {key: apps.get_model(app, model)._meta.db_table for key, (app,
model) in {
        "user": ("accounts", "CustomUser"), "broker": ("brokers", "Broker"),
"lead": ("leads", "Lead"),
        "call": ("leads", "CallLog"), "deposit": ("clients", "Deposit"),
"account": ("clients", "TradingAccount"),
        "client": ("clients", "Client")}.items()}
    schema_editor.execute(VIEWS.format(**tables), params=None)


def drop(apps, schema_editor):
    schema_editor.execute("DROP SCHEMA IF EXISTS analytics CASCADE",
params=None)


class Migration(migrations.Migration):
    dependencies = [("reports", "00NN_previous"), ("leads", "00NN_latest"),
("clients", "00NN_latest"),
                    ("brokers", "00NN_latest"), ("accounts", "00NN_latest")]
    operations = [migrations.RunPython(create, drop)]
```

Like Day 24's materialized view, these views pin the columns they read: a migration that alters one of them must drop the view first and recreate it afterwards (`CREATE OR REPLACE VIEW` can add columns at the end, not change existing ones).

27. Run once by hand as the database owner (the password must not live in a migration):

```
CREATE ROLE bi_reader LOGIN PASSWORD '<from the vault>';
GRANT CONNECT ON DATABASE nexora_crm TO bi_reader;
GRANT USAGE ON SCHEMA analytics TO bi_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA analytics TO bi_reader;
-- views count as tables here
ALTER DEFAULT PRIVILEGES IN SCHEMA analytics GRANT SELECT ON TABLES TO
bi_reader;    -- future views too
-- no grants on the app tables: bi_reader reads the views (they run with the
owner's rights) but not crm_leads itself
```

The database has no public port, so a BI tool connects through an SSH tunnel or, better, to the read replica (Day 22 #30) with the same role.

28.

```
# reports/management/commands/export_analytics.py
from pathlib import Path

from django.core.management.base import BaseCommand
from django.db import connection
from django.utils import timezone

VIEWS = ["dim_agent", "dim_broker", "fct_leads", "fct_calls", "fct_deposits"]
# fixed list: no injection


class Command(BaseCommand):
    help = "Export the analytics views to dated CSV files (PostgreSQL COPY,
streamed)"

    def add_arguments(self, parser):
        parser.add_argument("out_dir")

    def handle(self, *args, out_dir, **options):
        out = Path(out_dir)
        out.mkdir(parents=True, exist_ok=True)
        stamp = timezone.localdate().isoformat()
        with connection.cursor() as cursor:
            for view in VIEWS:
                path = out / f"{view}-{stamp}.csv"
                sql = f"COPY (SELECT * FROM analytics.{view}) TO STDOUT WITH
(FORMAT csv, HEADER true)"
                with path.open("wb") as f, cursor.cursor.copy(sql) as copy:
# psycopg 3 COPY, never all in memory
                    for chunk in copy:
                        f.write(chunk)
                self.stdout.write(f"{view:<14} → {path}")

# compose: mount ./exports:/exports on web; ship the files to the warehouse
bucket from the host
```

29.

```
# reports/management/commands/data_quality.py
from datetime import timedelta
from decimal import Decimal

from django.core.mail import mail_admins
from django.core.management.base import BaseCommand, CommandError
from django.db.models import Count, DecimalField, F, OuterRef, Subquery, Sum,
Value
from django.db.models.functions import Coalesce
from django.utils import timezone

from clients.models import Deposit, TradingAccount
from leads.models import CallLog, Lead
from sop.hours import is_working_time


def checks():
    now = timezone.now()
    yield "duplicate lead phones",
Lead.objects.values("phone").annotate(n=Count("id")).filter(n__gt=1).count()
    yield "active leads unassigned > 24h", Lead.objects.active().filter(
        assigned_to__isnull=True, created_at__lt=now -
timedelta(hours=24)).count()
    yield "calls logged by inactive agents today",
CallLog.objects.today().filter(agent__is_active=False).count()
    yield "negative balances",
TradingAccount.objects.filter(balance__lt=0).count()
    totals =
(Deposit.objects.filter(account=OuterRef("pk")).order_by().values("account")
             .annotate(t=Sum("amount")).values("t"))
    yield "balance != sum(deposits)", (TradingAccount.objects
        .annotate(dep=Coalesce(Subquery(totals), Value(Decimal("0")),
                               output_field=DecimalField(max_digits=14,
decimal_places=2)))
        .exclude(balance=F("dep")).count())
    if is_working_time(now) and is_working_time(now - timedelta(hours=6)):
        yield "no new leads for 6 working hours", int(not
Lead.objects.filter(created_at__gte=now - timedelta(hours=6)).exists())


class Command(BaseCommand):
    help = "Data-quality checks (dbt-test style): every check must return 0"
```

```
    def handle(self, *args, **options):
        failed = []
        for name, bad in checks():
            self.stdout.write(f"{'OK  ' if not bad else 'FAIL'} {bad:>6}
{name}")
            if bad:
                failed.append(f"{name}: {bad}")
        if failed:
            mail_admins("Data quality checks failed", "\n".join(failed))
            raise CommandError(f"{len(failed)} check(s) failed")
```

30.

```
sed -i 's/^MAINTENANCE_MODE=.*/MAINTENANCE_MODE=True/' .env.prod && dc up -d
web
curl -s -o /dev/null -w "%{http_code}\n" https://crm.nexora.ae/leads/      #
503 + the maintenance page
curl -s -o /dev/null -w "%{http_code}\n" https://crm.nexora.ae/health/     #
200: monitors don't page anyone
sed -i 's/^MAINTENANCE_MODE=.*/MAINTENANCE_MODE=False/' .env.prod && dc up -d
web
deploy/smoke.sh
```

D. Wrap-up (31–36)

31.

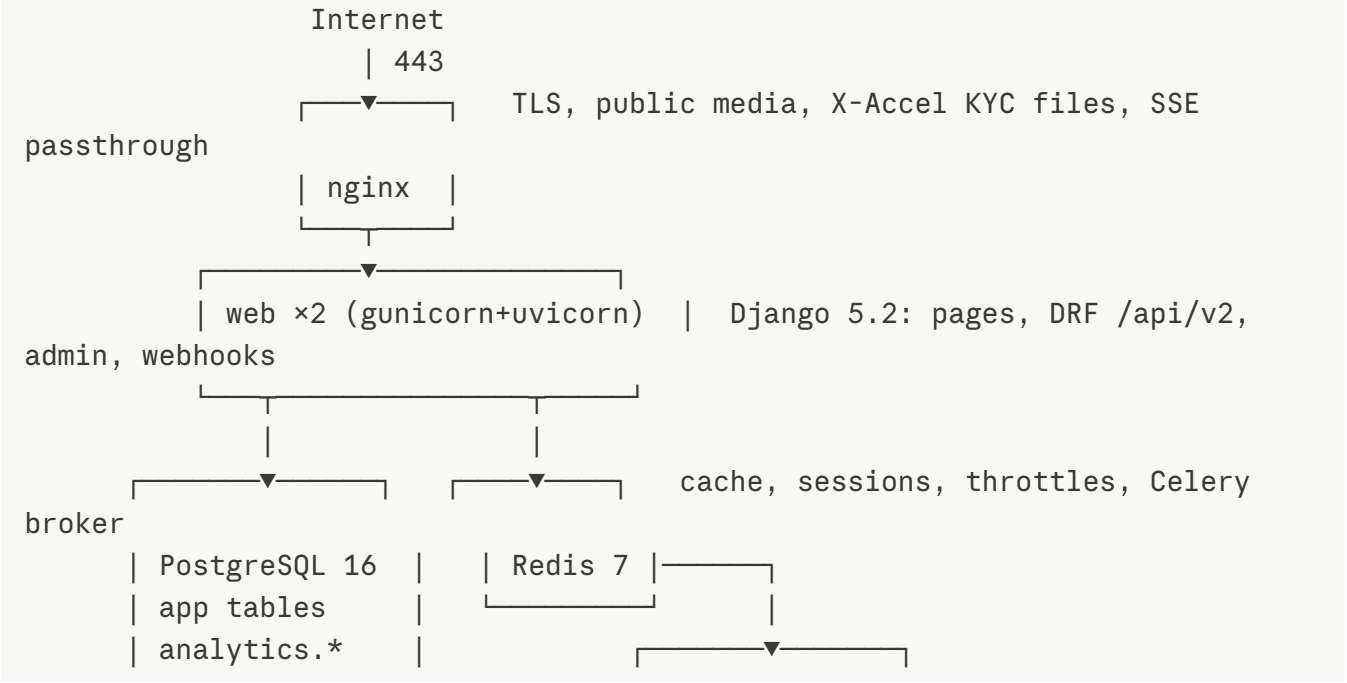
```
# core/middleware.py — high in MIDDLEWARE (right after SecurityMiddleware) so
it wraps every v1 response
class ApiV1DeprecationMiddleware:
    DEPRECATED_AT = "@1792195200"                    # RFC 9745 format: 17
Oct 2026 00:00 UTC
    SUNSET = "Thu, 31 Dec 2026 20:00:00 GMT"         # midnight Dubai, 1 Jan
2027

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)
        if request.path.startswith("/api/v1/"):
            response["Deprecation"] = self.DEPRECATED_AT
            response["Sunset"] = self.SUNSET
            response["Link"] = '</api/v2/docs/>; rel="successor-version"'
            logging.getLogger("crm.api").info("api v1 call", extra={
                "path": request.path, "client": request.headers.get("User-
Agent", "")[:100]})
        return response
```

The log line tells you *who* still calls v1. When it has been silent for a month, delete the v1 URLs.

32.



```
      | audit trigger  |◄─────────────| worker + beat |  emails, broker
webhooks, SOP scan, commissions
      └──────┬───────┘              └─────────────────┘
             |
             | bi_reader (read-only)
       BI tools / nightly CSV export

Apps: leads · clients · brokers · accounts · core · reports · sop · api
In:   dialer (API key + idempotency) · payment provider (HMAC webhook) ·
public form (honeypot + rate limit)
Out:  SMTP · broker webhooks (HMAC) · Sentry · chat alerts
```

33.

```
# Nexora CRM

A call-centre CRM for a Dubai brokerage: leads → calls → clients → deposits →
broker commissions,
with SOP response-time tracking, a priority dialer queue, and a REST API.

## Features
- Role-scoped leads (agent / team lead / manager), audit trail, status
history
- Dialer queue with SOP breaches first, callbacks, do-not-call list, claim
locking
- SOP engine measuring SLAs in working hours; breach board; compliance report
- Broker trees, differential commissions, downline reporting (recursive SQL)
- DRF API (tokens, JWT, API keys); dialer and payment webhooks with
idempotency and HMAC
- Live dashboard (Chart.js), pipeline board, CSV/XLSX import and export
- Security: 2FA for managers, append-only audit log, PII-redacted logs, CSP

## Stack
Django 5.2 · PostgreSQL 16 · Redis · Celery · DRF · gunicorn/uvicorn · nginx
· Docker · GitHub Actions

## Run it locally
    cp .env.example .env
    docker compose up -d db redis
    python -m venv .venv && source .venv/bin/activate && pip-sync
requirements.txt requirements-dev.txt
    python manage.py migrate && python manage.py seed_capstone
    python manage.py runserver
```

```
## Quality gates
    coverage run manage.py test --settings=django_project.settings.test &&
coverage report
    ruff check . && ruff format --check .
```

34.

```
# settings/base.py
DEMO_MODE = env.bool("DEMO_MODE", default=False)
if DEMO_MODE:
    EMAIL_BACKEND = "django.core.mail.backends.dummy.EmailBackend"   #
nothing leaves the box

# brokers/webhooks.py — send_webhook(): first line
if settings.DEMO_MODE:
    return 200


# core/middleware.py
class DemoReadOnlyMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        user = getattr(request, "user", None)
        if (settings.DEMO_MODE and request.method not in ("GET", "HEAD",
"OPTIONS")
                and getattr(user, "username", "") == "demo" and request.path
!= reverse("logout")):
            return HttpResponseForbidden("The demo account is read-only.")
        return self.get_response(request)
```

35. Write half a page in `RETROSPECTIVE.md` while it's fresh: the decision you'd make differently (often: custom user model on day one, a services layer from the start, PostgreSQL from the start), the bug that cost the most time, and three next topics. Natural next steps from this codebase: Django Channels or SSE at scale (live team wall), OpenTelemetry tracing, dbt models on top of the `analytics` schema, a managed container platform or Kubernetes, and multi-tenant design if a second brokerage ever uses it.

36.

```
# CHANGELOG.md: "## 1.0.0 — <launch date>" + the feature list from the README
git add -A && git commit -m "release 1.0.0"
git tag -a v1.0.0 -m "Nexora CRM 1.0.0"
git push origin main --tags
gh release create v1.0.0 --title "Nexora CRM 1.0.0" --notes-file CHANGELOG.md
```

Tests (37–40)

```
# core/tests/test_launch.py
from io import StringIO
from unittest import skipUnless

from django.core.cache import cache
from django.core.management import call_command
from django.db import connection
from django.test import TestCase, override_settings
from django.urls import reverse

from core.factories import AgentFactory, LeadFactory, ManagerFactory,
TeamLeadFactory


def login(client, user):
    client.force_login(user)
    session = client.session
    session["otp_verified"] = True          # managers pass Day 28's 2FA gate
    session.save()


@override_settings(FEATURES={"sop", "dialer", "commissions", "imports",
"exports"})
class LaunchTests(TestCase):
    MATRIX = {                                  # url name → expected status
per role
        "dashboard":     {"agent": 200, "team_lead": 200, "manager": 200},
        "leads:list":    {"agent": 200, "team_lead": 200, "manager": 200},
        "sop_breaches":  {"agent": 403, "team_lead": 200, "manager": 200},
        "api:team-wall": {"agent": 403, "team_lead": 200, "manager": 200},
    }

    def setUp(self):
        cache.clear()
```

```
    def test_role_matrix(self):
# 37
        users = {"agent": AgentFactory(), "team_lead": TeamLeadFactory(),
"manager": ManagerFactory()}
        for role, user in users.items():
            login(self.client, user)
            for name, expected in self.MATRIX.items():
                with self.subTest(role=role, page=name):

self.assertEqual(self.client.get(reverse(name)).status_code, expected[role])
            self.client.logout()

    @skipUnless(connection.vendor == "postgresql", "analytics views are
PostgreSQL-only")
    def test_analytics_views(self):
# 38
        lead = LeadFactory(phone="971501234999")
        with connection.cursor() as cursor:
            cursor.execute("SELECT * FROM analytics.fct_leads WHERE lead_id =
%s", [lead.pk])
            columns = [c.name for c in cursor.description]
            row = dict(zip(columns, cursor.fetchone()))
            for view in ("dim_agent", "dim_broker", "fct_calls",
"fct_deposits"):
                cursor.execute(f"SELECT * FROM analytics.{view} LIMIT 1")
# fixed names, not input
        self.assertNotIn("phone", columns)
        self.assertEqual(row["phone_last4"], "4999")

    def test_v1_deprecation_headers(self):
# 39
        response = self.client.get("/api/v1/leads/")    # whatever the
status, the headers must be there
        self.assertEqual(response["Deprecation"], "@1792195200")
        self.assertIn("2026", response["Sunset"])
        self.assertIn("successor-version", response["Link"])

    def test_smoke_command(self):
# 40
        manager = ManagerFactory()
```

```
        out = StringIO()
        call_command("smoke_test", username=manager.username, stdout=out)
# raises CommandError on any FAIL
        self.assertIn("All smoke checks passed", out.getvalue())
```

```
coverage run manage.py test --settings=django_project.settings.test &&
coverage report
ruff check . && ruff format --check .
git add -A && git commit -m "day 30: launch" && git tag -a v1.0.0 -m "Nexora
CRM 1.0.0"
```

What you built in 30 days

A production CRM, end to end: 8 Django apps on PostgreSQL; role-scoped data with an append-only audit trail; forms, class-based views, imports and exports; a REST API with four ways to authenticate and two signed integrations; a JavaScript front end with live charts, a dialer and a pipeline board; SOP measurement in working hours; broker commissions down a referral tree; caching, profiling, background jobs, 2FA, Docker, CI/CD, monitoring and a BI-ready analytics layer, with a test suite guarding all of it. Every piece is code you wrote and can explain.